第一章

计算机系统概论

Computer Abstractions

合肥工业大学

计算机与信息学院

阙夏

提　纲

1. 概　　述
2. 计算机系统的硬件结构
3. 中央处理器（CPU）
4. 控 制 单 元（CU）

本部分课程内容概貌

- 基本部件的结构和组织方式
- 基本部件和单元的设计思想
- 基本运算的的操作原理
- 计算机组成的一般原理，不依托具体的机型

课程学习目的

- 深入理解现代计算机的体系结构、基本问题,工程中的权衡
- 怎样设计计算机系统
- 理解计算机系统的工作方式是什么以及各种操作为什么这样执行

大纲要求

❖计算机组成原理考查目标

- 理解单处理器计算机系统中各部件的内部工作原理、组成结构以及相互连接方式，具有完整的计算机系统的整机概念。
- 理解计算机系统层次化结构概念，熟悉硬件与软件之间的界面，掌握指令集体系结构的基本知识和基本实现方法。
- 能够运用计算机组成的基本原理和基本方法，对有关计算机硬件系统中的理论和实际问题进行计算、分析，并能对一些基本部件进行简单设计。

学习要求

- 平时成绩

  （包括实验、作业、测验、出勤、专题讨论）

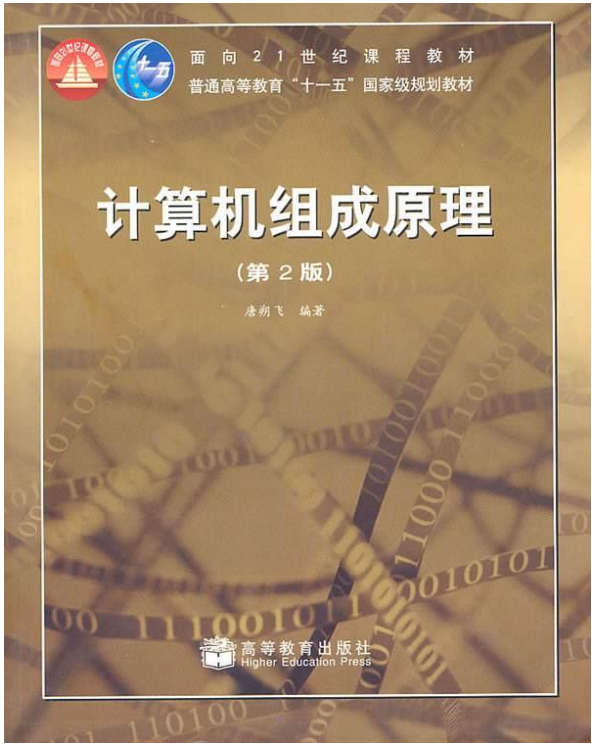
- 考试成绩

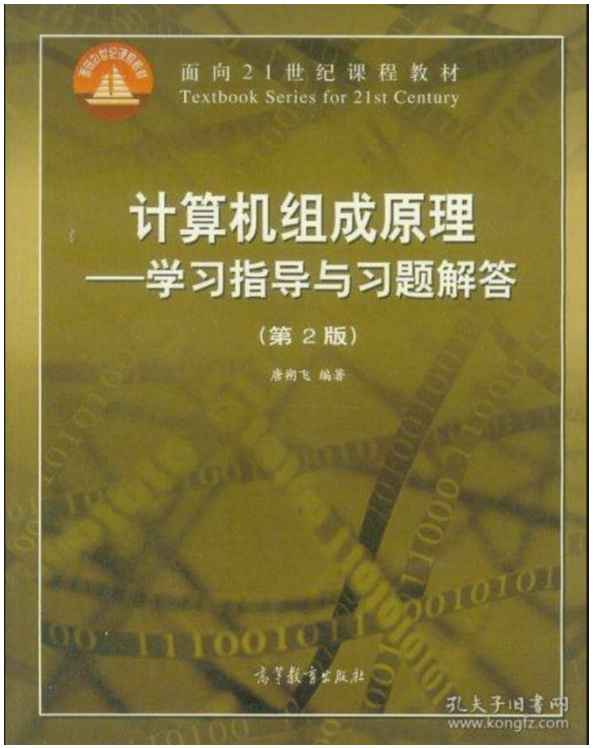
- 每一章的学习目标将在每章学习前说明，要求同学们在学习过程中能够针对每个知识点认真总结。

学习方法

- 博学
- 审问
- 慎思
- 明辨
- 笃行

教材和辅助练习





- 《计算机组成原理》（第2版）唐朔飞 高等教育出版社
- 《计算机组成原理--学习指导与习题解答》唐朔飞 高等教育出版社

参考书目

- 《Computer Organization and Design hardware and software interface》, Patterson and Hennessy,4nd Edition, Morgan Kaufmann Pub.
  - 中文版:《计算机组成和设计：硬件/软件接口（第4版或第5版）》（美）帕特森、亨尼西 著
- 《 Computer Systems A Programmer's Perspective Randal》 E. Bryant Devid O'Hallaron, Prentice-Hall Internation Inc.
  - 中文译本：《深入理解计算机系统》
- 《计算机组成与系统结构》袁春风编著 清华大学出版社
- 《计算机系统基础》袁春风编著 机械工业出版社

约翰•亨尼西 (John Hennessy)和**大卫•帕特森**(David Patterson)

主要成就

- 美国科学家约翰·亨尼西和大卫·帕特森获得2017年度图灵奖，以表彰二人。
- 开创了一种系统的、可量化的方法用以设计和评价计算机体系架构，并对精简指令集（RISC）微处理器行业产生深远影响。
- 《计算机体系结构： 量化研究方法》《Computer Architecture：A Quantitative Approach》

2017年图灵奖获得者

第一章 计算机系统概论

1.1 计算机系统概述

1.2 计算机的基本组成

1.3 计算机性能评价

大纲要求

- 计算机发展历程（第二章）
- 计算机系统的层次结构

  1.计算机硬件的基本组成

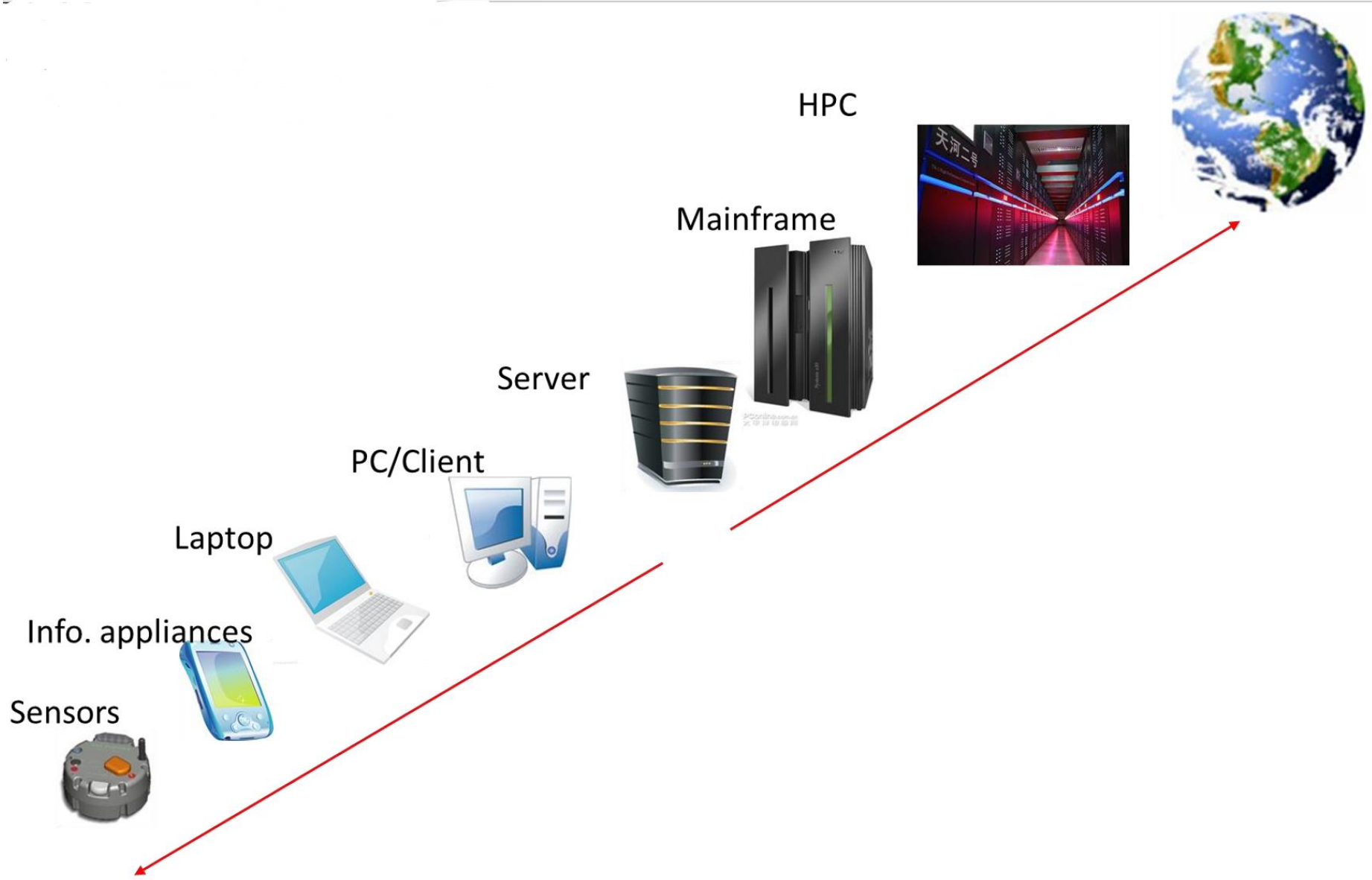
  2.计算机软件的分类

  3.计算机的工作过程

- 计算机性能指标

  吞吐量；响应时间；CPU时钟周期；主频；CPI；CPU执行时间；MIPS；MFLOPS

现代计算机的多态性


HPC
Mainframe
Server
PC/Client
Laptop
Info. appliances
Sensors

1.1.1 计算机软硬件的概念

一. 计算机系统

计算机系统

- 硬件：计算机中电子线路和物理装置
如主机、外设等
- 软件：计算机中全部程序和文件

1.1 计算机系统简介

软件

系统软件 **用于管理整个计算机系统的软件**

- 操作系统
- 语言处理程序
- 服务程序
- 数据库管理系统（DBMS）
- 网络软件

应用软件 **根据任务需要编制的各种程序**

软　件

- **System software(系统软件) - 简化编程过程，使硬件资源被有效利用**
  - 操作系统（Operating System）：硬件资源管理，用户接口
  - 语言处理系统：翻译程序+ Linker, Debug, Loader, etc …
    - 翻译程序(Translator)有三类：

      汇编程序(Assembler)：汇编语言源程序→机器语言目标程序

      编译程序(Complier)：高级语言源程序→汇编/机器语言目标程序

      解释程序(Interpreter)：将高级语言语句逐条翻译成机器指令并立即执行,不生成目标文件。
  - 其他实用程序: 如：数据库管理系统、磁盘碎片整理程序、备份程序等
- **Application software(应用软件) - 解决具体应用问题/完成具体应用任务**
  - 各类媒体处理程序：Word/ Image/ Graphics/…
  - 管理信息系统 (MIS)
  - 数据库系统
  - Game，…

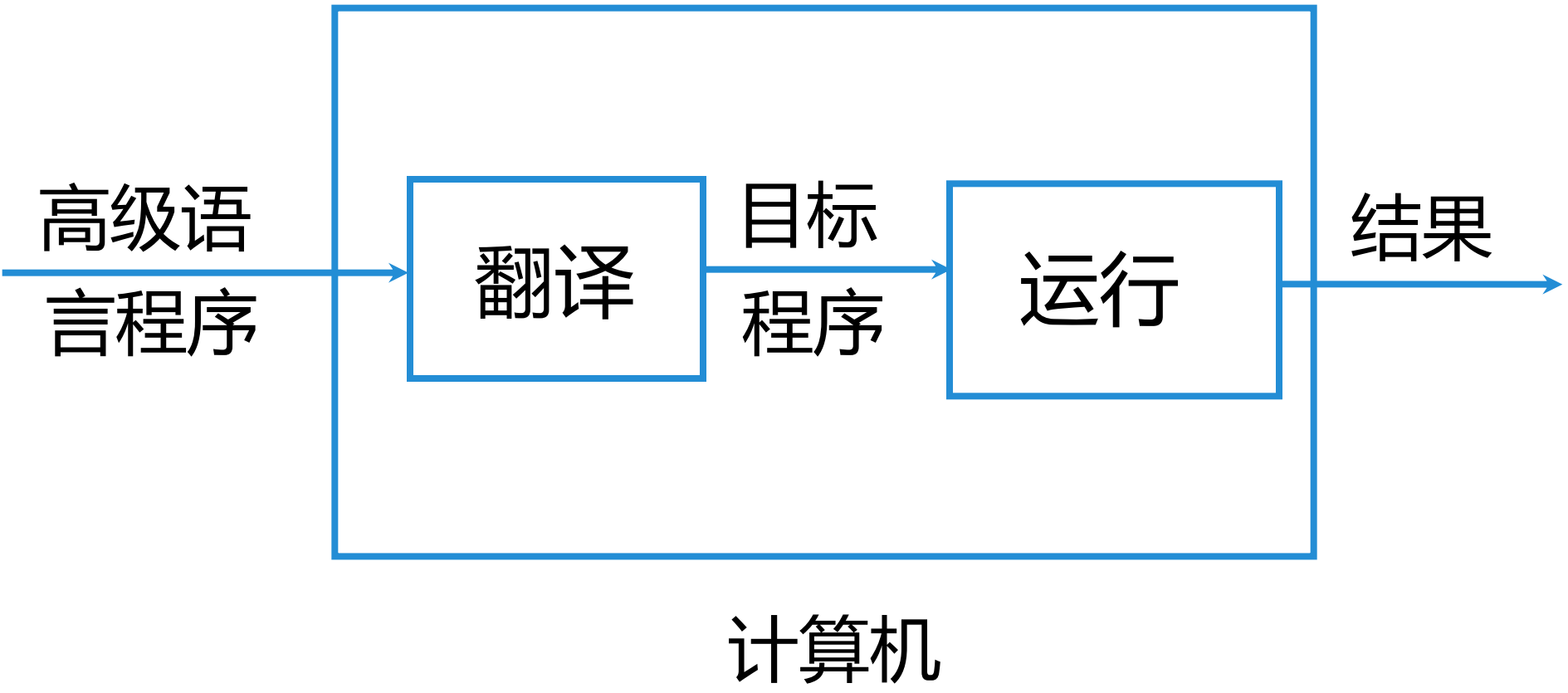
二. 计算机语言

1）机器语言：二进制代码表示的计算机语言，可以**直接执行**。

2）汇编语言：用助记符（如ADD、SUB、...）编写的语言，需要通过**汇编程序**翻译成目标程序后才执行。

3）高级语言：如PASCAL，C，JAVA等，便于编写、阅读，接近自然语言，提高了软件的产量，便于移植，独立于硬件环境。

计算机的解题过程



翻译方式有两种：区别 重点☆

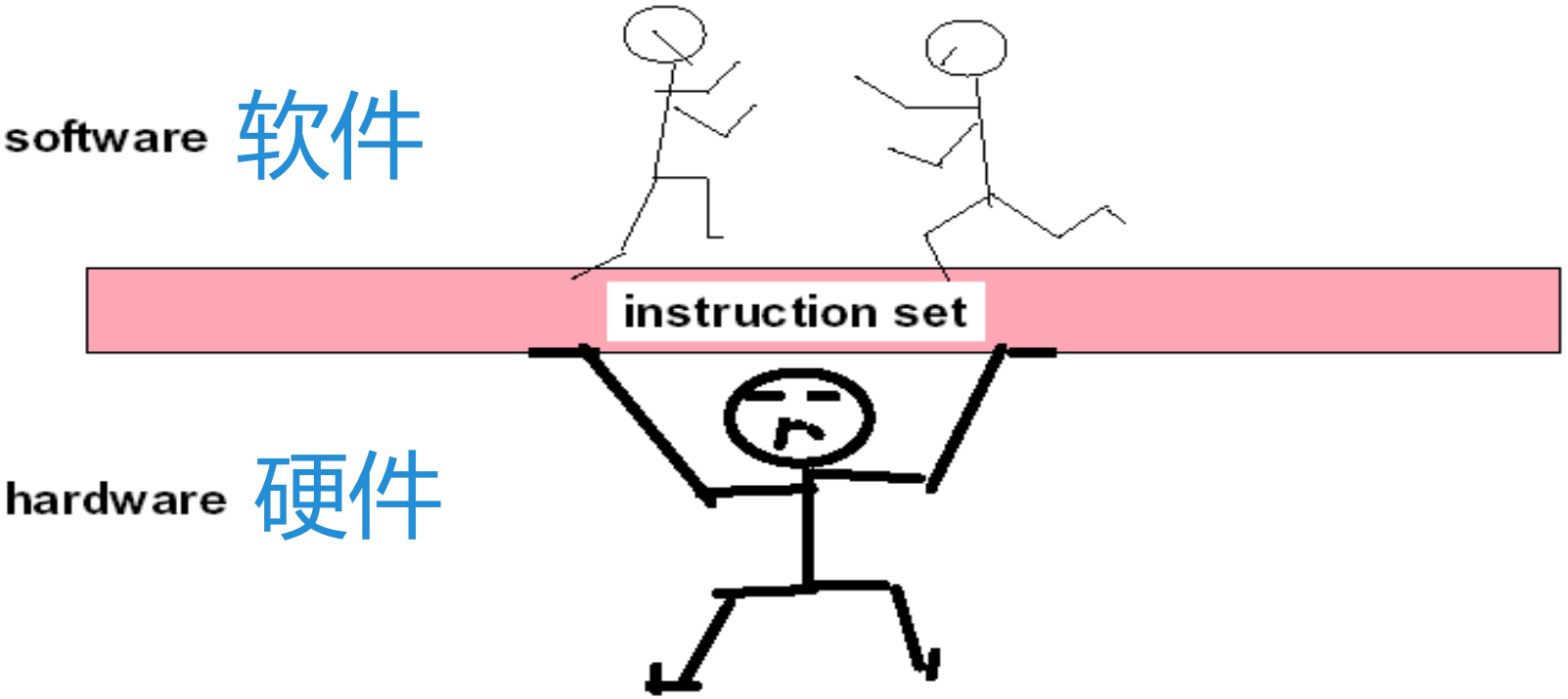
- 编译：使用编译程序把高级语言的源程序翻译成机器代码的目标程序，并以文件形式保留，然后执行。
  - C，PASCAL
  - 大多数编译程序直接产生机器语言的目标代码，形成可执行的目标文件，少数先产生汇编语言一级的符号代码文件，然后再调用汇编程序进行翻译加工处理，最后产生可执行的机器语言目标文件。

- 解释：使用解释程序，将源程序的一条语句翻译成对应机器语言的一条语句，并立即执行这条语句，接着再翻译源程序的下一条语句，并立即执行，并不形成目标程序，如此重复，直至完成源程序的全部翻译任务
  - JAVA
  - BASIC的翻译有解释和编译两种方式。

三. 硬件与软件的关系

- **相互关联：**
  - 硬件系统在最内层，是计算机系统的基础、核心
  - 系统软件为用户提供一个基本操作界面
  - 应用软件在最外层，为用户提供解决具体问题应用系统界面
- **相互转化 、互相渗透、互相融合**
  - 软硬件没有一条明确的分界线。
    - 硬件软化，可以增强系统的功能和适应性
    - 软件硬化，可以显著降低软件在时间上的开销
- **固件（Firmware）：具有软件特性的硬件**
    - 硬件的快速性
    - 软件的灵活性

Hardware/Software Interface（界面）



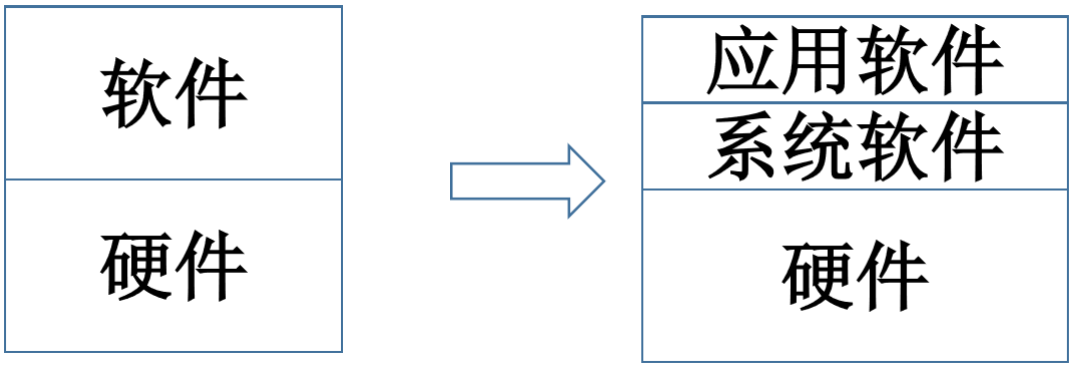
软件和硬件的界面： **ISA（Instruction Set Architecture）**

可自己写　　指令集体系结构

机器语言由指令代码构成，能被硬件直接执行。

1.1.2 计算机系统的层次结构

简单的分层



1.1.2 计算机系统的层次结构

二、计算机系统的层次结构

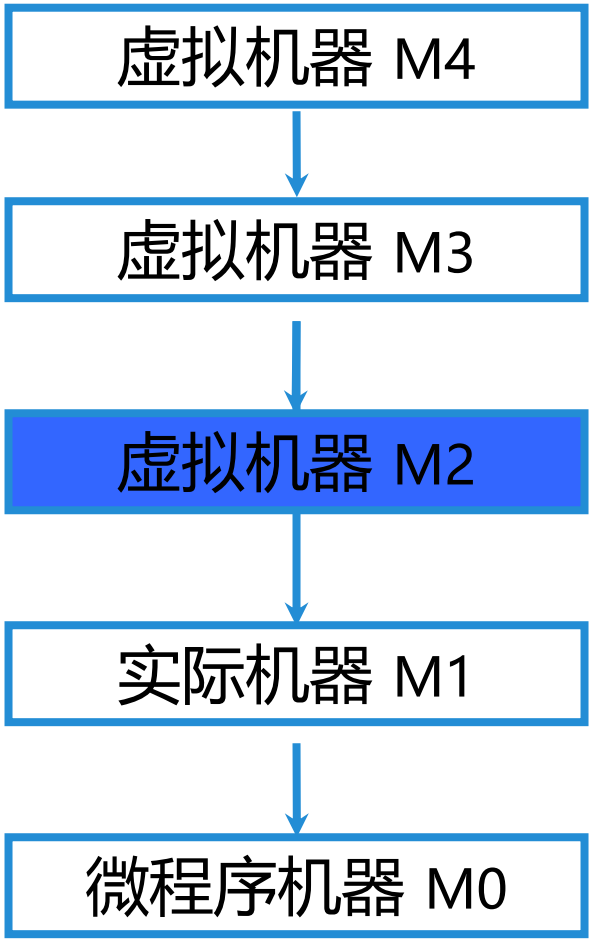
高级语言

汇编语言

操作系统

机器语言

微指令系统



二、计算机系统的层次结构

- 实际机器：指由硬件或固件实现的机器。
- 虚拟机器：指以软件为主实现的机器。
  - 虚拟机器只对该级的观察者存在，某一级观察者只需通过该级的语言来了解和使用计算机，不必关心下级是如何工作和实现的。
    - 高级语言级的用户可以不了解机器的具体组成，不熟悉指令系统，直接用所指定的语言描述所要解决的问题。

二、计算机系统的层次结构

软件

虚拟机器 M4

用编译程序翻译成汇编语言程序

虚拟机器 M3

用汇编程序翻译成机器语言程序

虚拟机器 M2

用机器语言解释操作系统

硬件

实际机器 M1

用微指令解释机器指令

微程序机器 M0

由硬件直接执行微指令

High Level Language Program (e.g., C)

Compiler

Assembly Language Program (e.g., MIPS)

Assembler

Machine Language Program (MIPS)

Machine Interpretation

Hardware Architecture Description (e.g., block diagrams)

Architecture Implementation

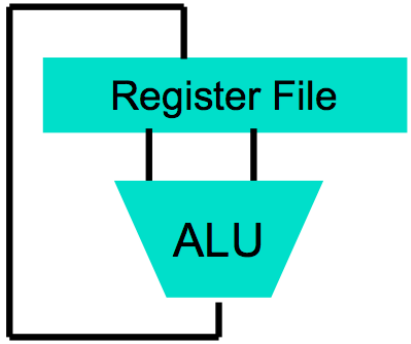
Logic Circuit Description (Circuit Schematic Diagrams)

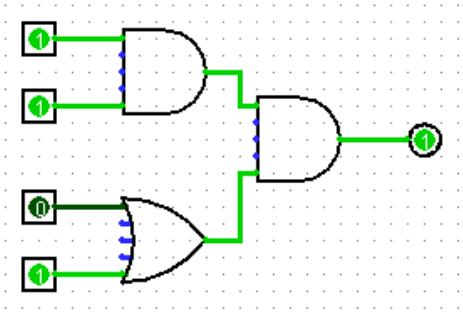
```
temp = v[k];
v[k] = v[k+1];
v[k+1] = temp;
```

```
lw   $t0, 0($2)
lw   $t1, 4($2)
sw   $t1, 0($2)
sw   $t0, 4($2)
```

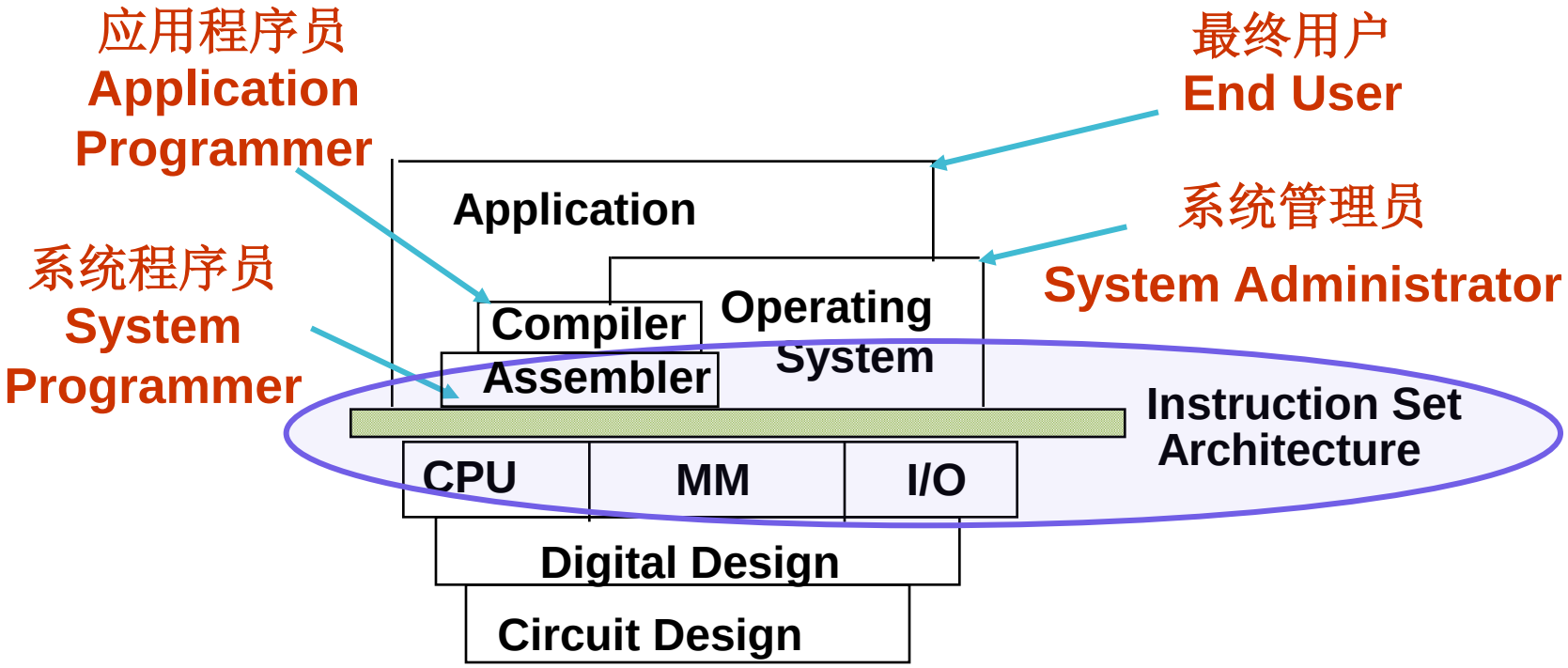
Anything can be represented as a *number*, i.e., data or instructions

```
0000 1001 1100 0110 1010 1111 0101 1000
1010 1111 0101 1000 0000 1001 1100 0110
1100 0110 1010 1111 0101 1000 0000 1001
0101 1000 0000 1001 1100 0110 1010 1111
```





What is Computer Architecture ?



- 上图给出的是计算机系统的层次结构
  指令系统（即ISA）是软/硬件的交界面
- 不同用户工作在不同层次，所看到的计算机不一样
- 中间阴影部分就是本课程主要内容，处于最核心的部分！

1.1.2 计算机系统的层次结构

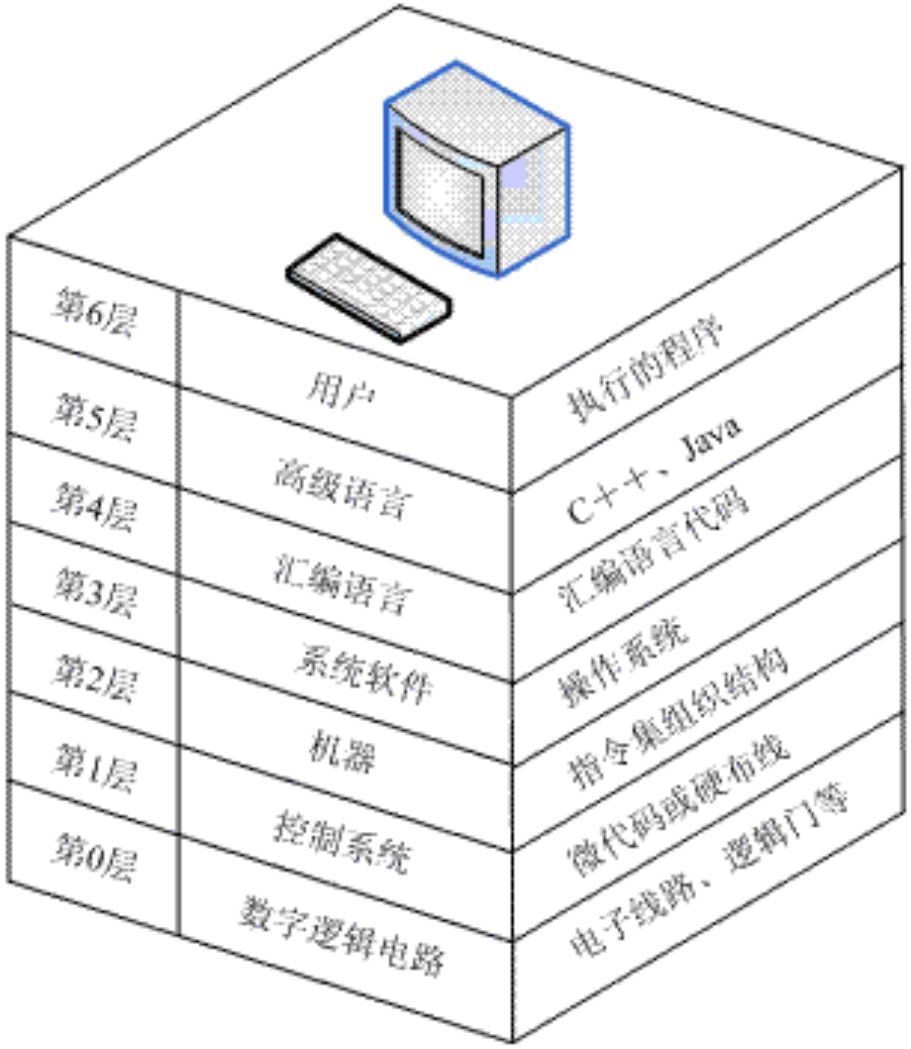


图1-17 计算机系统的分层组织结构

1.1.3 计算机组成和计算机体系结构

有无乘法指令

计算机体系结构
computer architecture

程序员所见到的计算机系统的属性
概念性的结构与功能特性

(指令系统、数据类型、寻址技术、I/O机理)

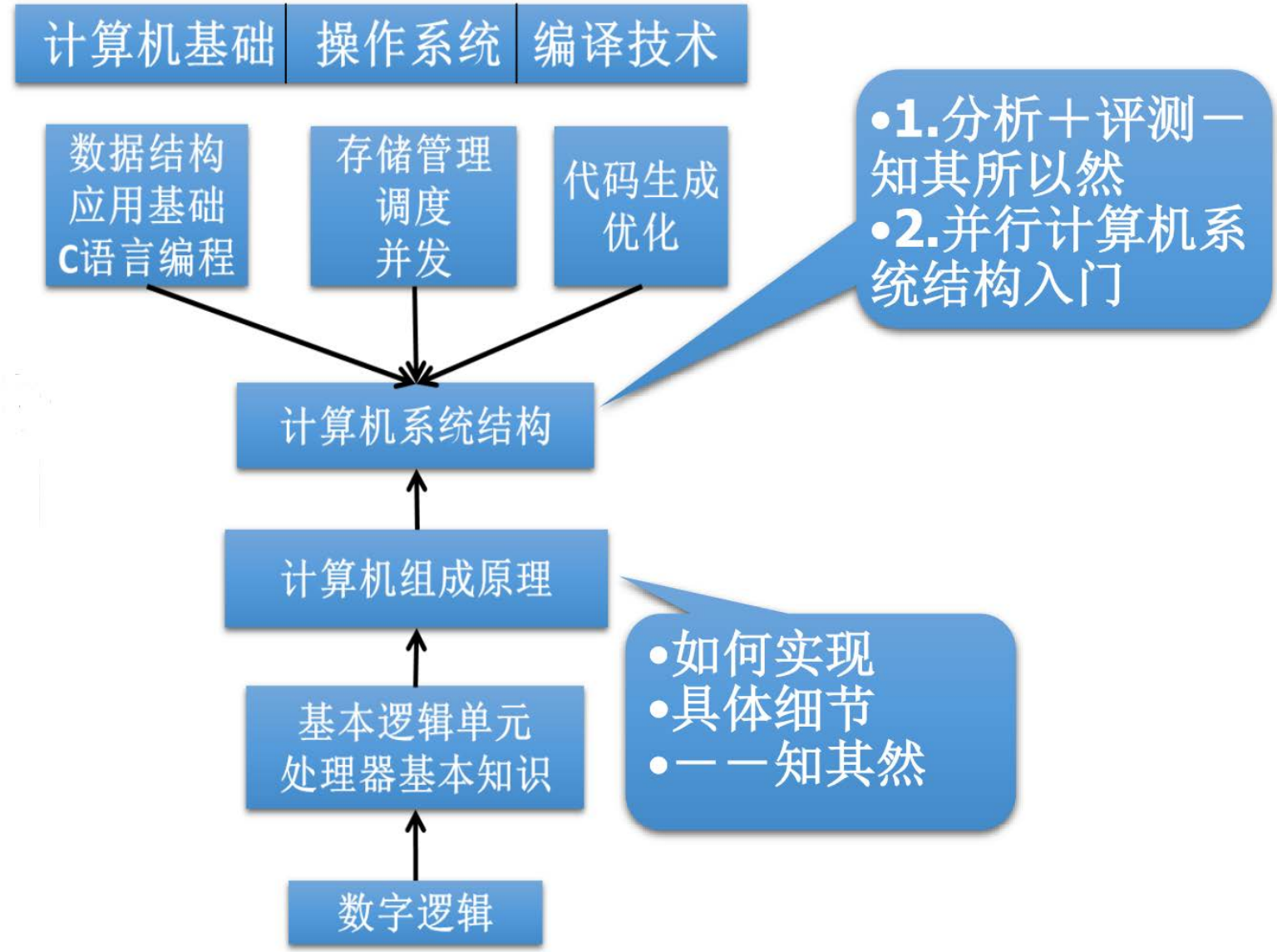
计算机组成
Computer organization

实现计算机体系结构所体现的属性

(具体指令的实现)

如何实现乘法指令

1.1.3 计算机系统的层次结构



1.2 计算机的基本组成

冯·诺依曼简介

为什么冯·诺依曼胜出？
图灵机淘汰

- 被西方人誉为“计算机之父”。
- 1945年3月他在共同讨论的基础上起草了“存储程序通用电子计算机方案”--EDVAC（Electronic Discrete Variable Automatic Computer）。这对后来计算机的设计有决定性的影响，特别是确定计算机的结构，采用存储程序以及二进制编码等，至今仍为电子计算机设计者所遵循。
- 是20世纪最重要的数学家之一，因其在现代计算机、数值分析和博弈论等领域的重大贡献成为美国科学院院士。

1.2 计算机的基本组成

一、冯·诺依曼计算机的工作原理

存储程序+程序控制

存储程序：将程序存放在计算机的存储器中；

程序控制： 按指令地址访问存储器并取出指令，经译码依次产生指令执行所需的控制信号，实现对计算的控制，完成指令的功能。

CHP4

（存储系统构建与快速访问 ）

CHP9.10

（指令系统、控制器设计等 ）

1.2 计算机的基本组成

二、冯·诺依曼计算机的特点

单指令单数据流

1. 计算机应由运算器、控制器、存储器、输入设备和输出设备五个基本部件组成。

2. 指令和数据以同等地位存放在存储器中，按地址寻访。

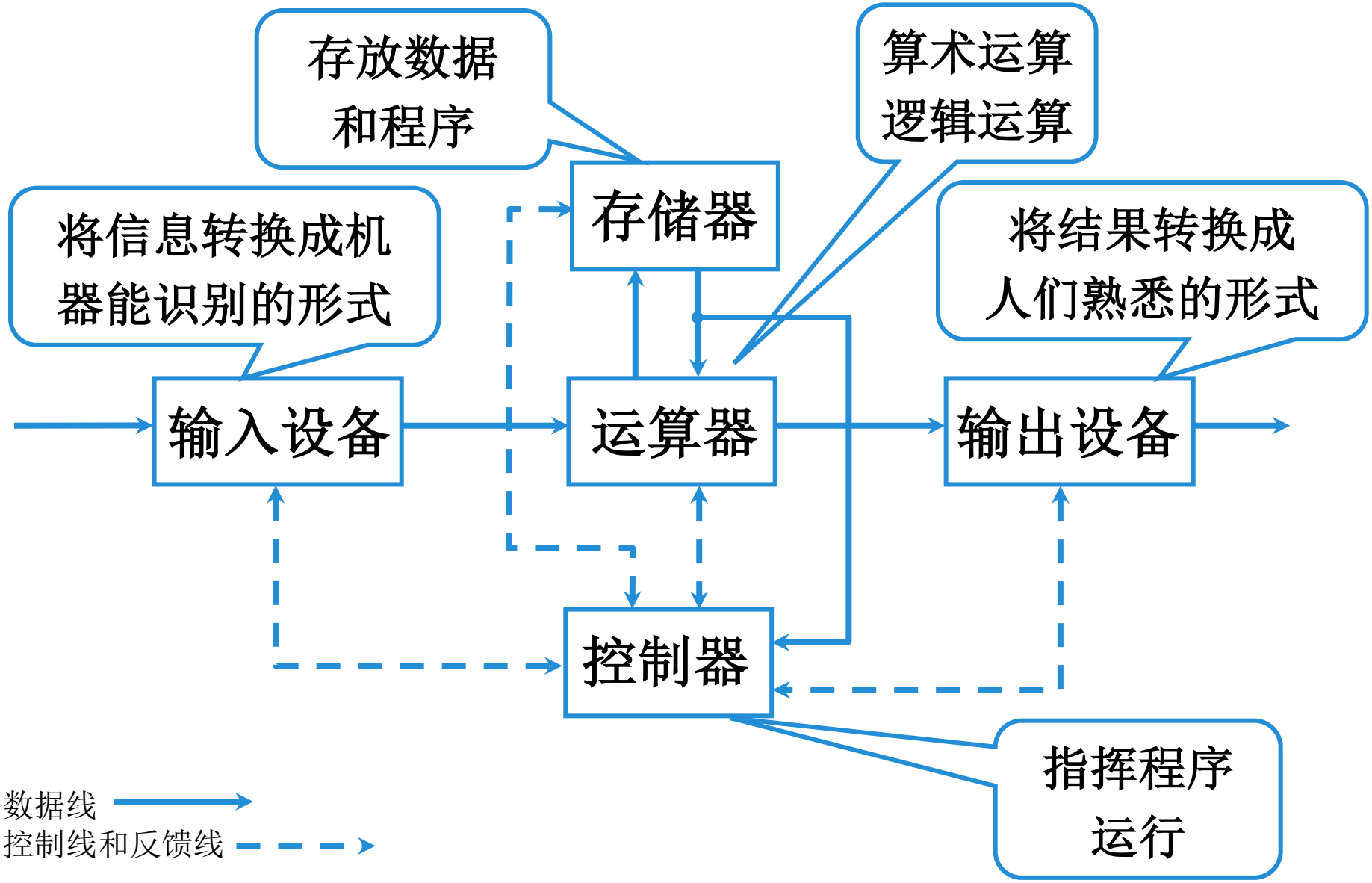
3. 指令和数据用**二进制**表示。

4. 指令由操作码和地址码组成。
   操作码指出操作类型，地址码指出操作数的地址。由一串指令组成程序。

5. 存储程序：指令在存储器中按顺序存放。通常，指令按顺序执行，但可按运算结果或外界条件改变执行顺序。计算机能在不需操作人员干预下，自动完成逐条取出指令和执行指令的任务。

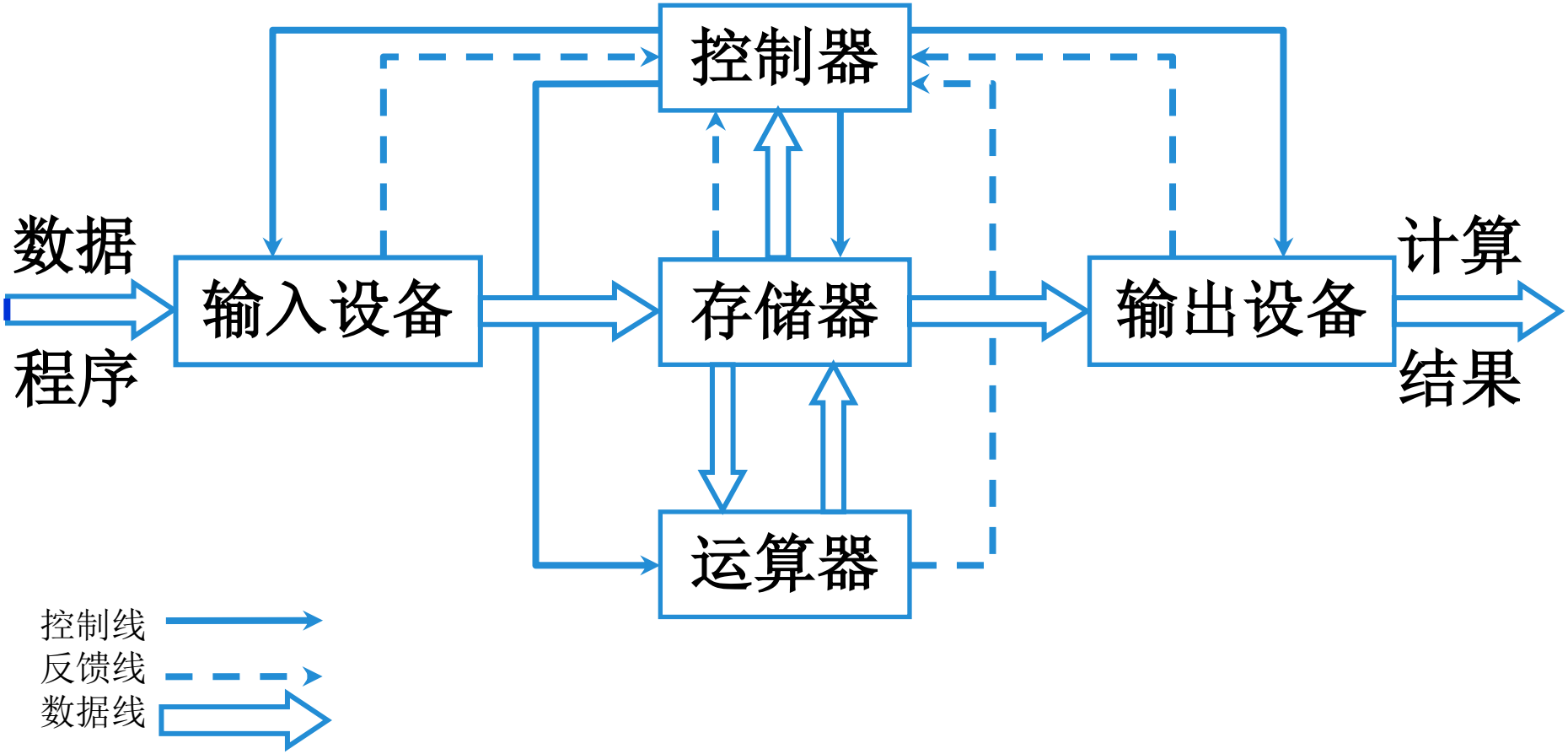
6. 以运算器为中心

冯·诺依曼计算机硬件框图



三、计算机硬件框图

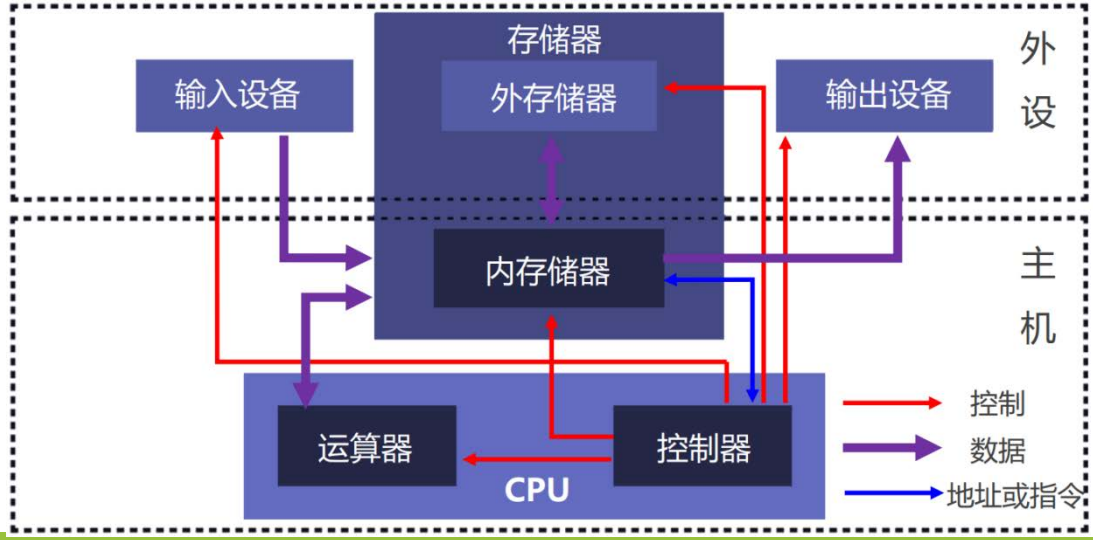
1. 以存储器为中心的计算机硬件框图(现代)



2. 现代计算机硬件框图

运算器 ALU
控制器 CU
} CPU
} 主机

存储器 { 主存 / 辅存
输入设备
输出设备
} I/O
} 硬件

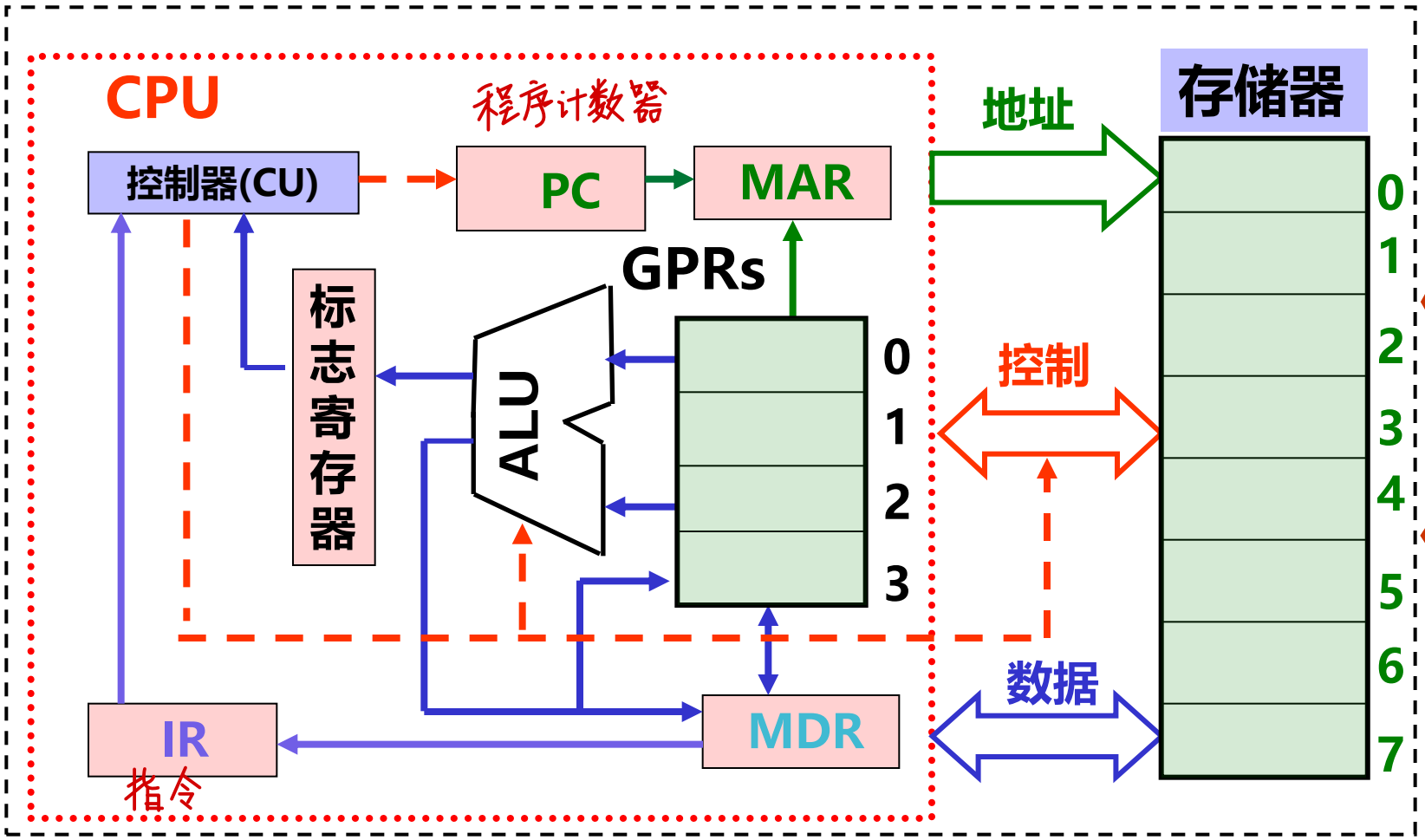


主机：CPU（运算器 + 控制器）、内存

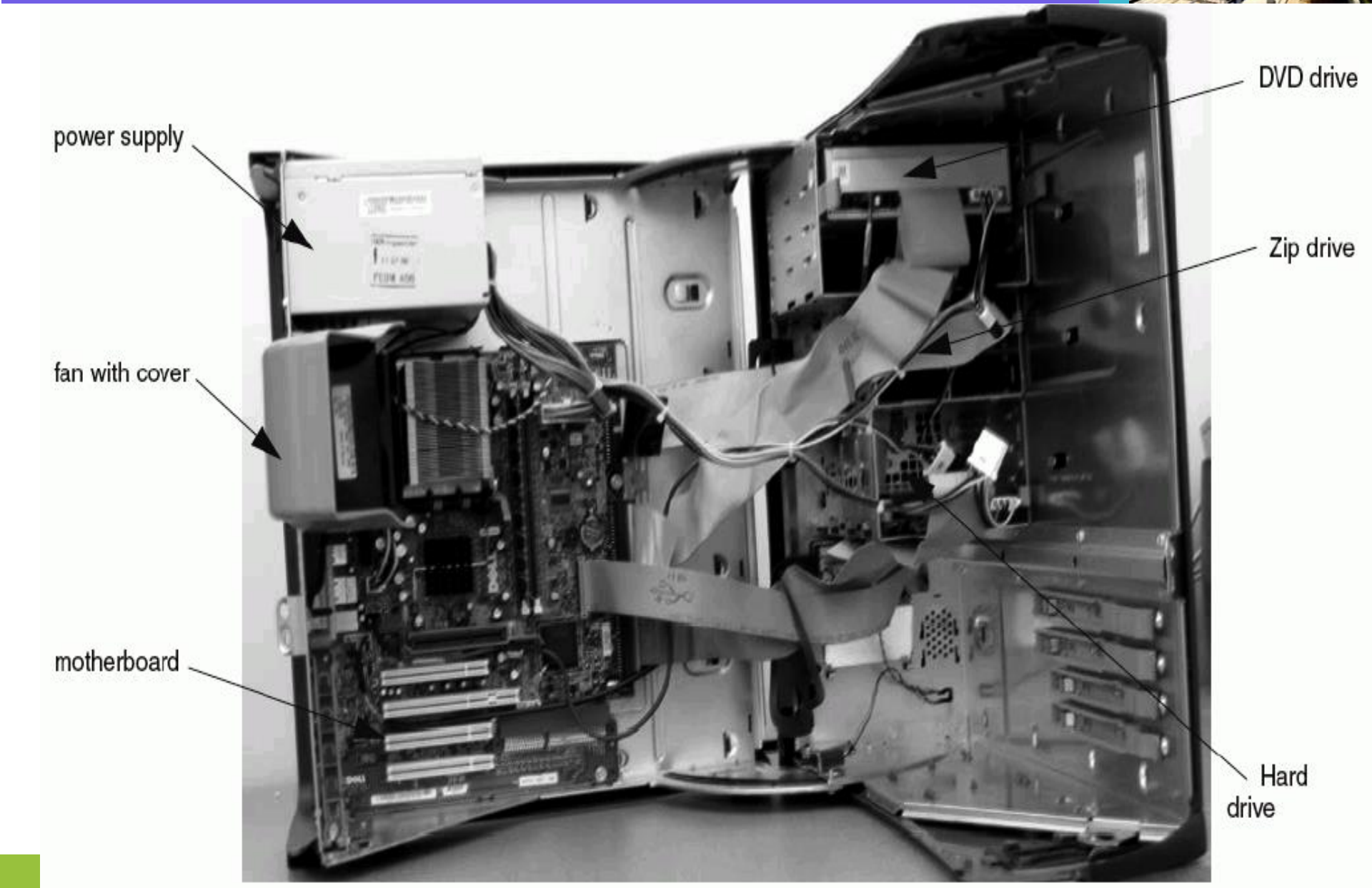
外设：输入设备、输出设备、外存储器

总线：地址线、数据线、控制线

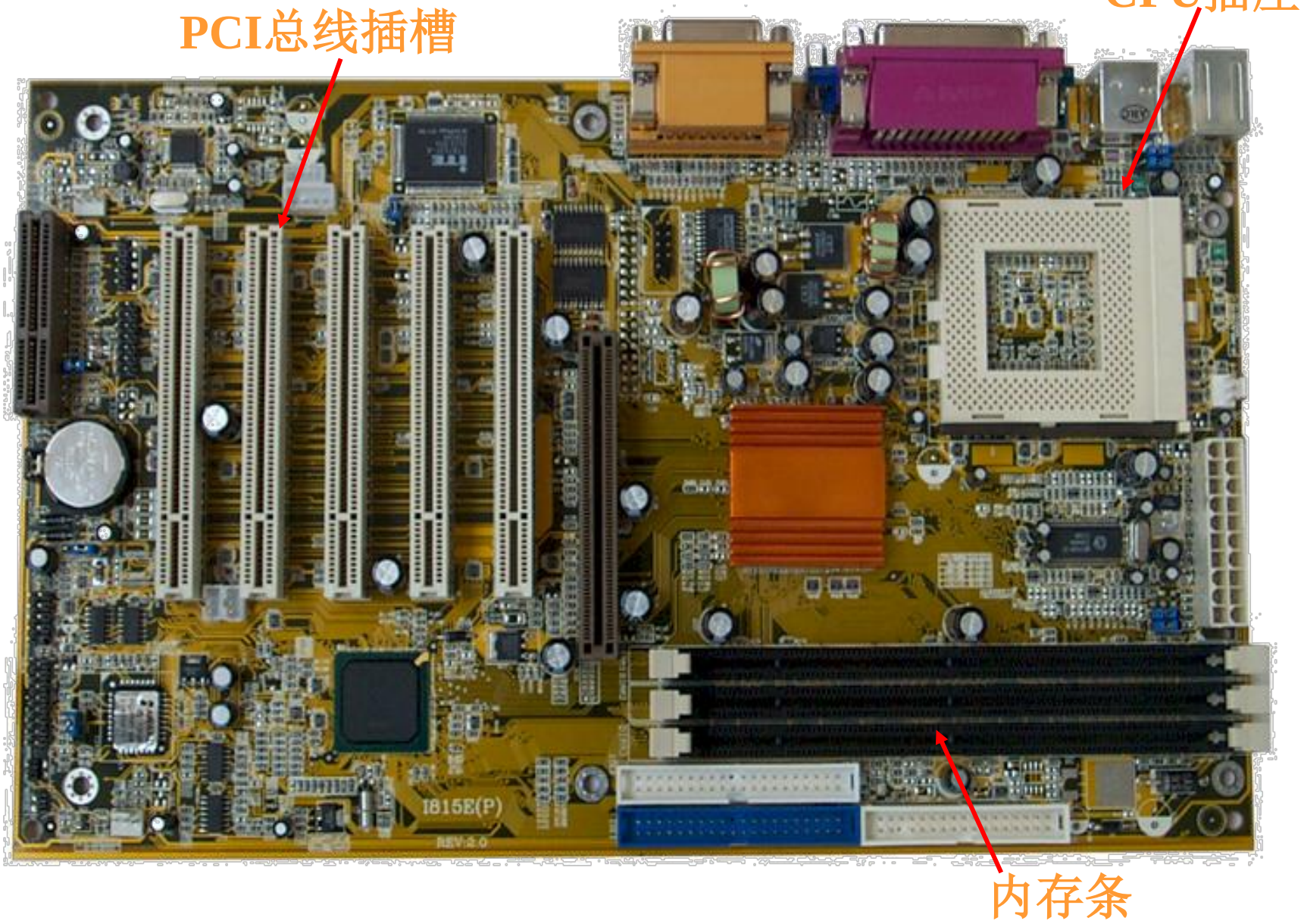
现代计算机结构模型


CPU
程序计数器
控制器(CU)
PC
MAR
地址
存储器
GPRs
标志寄存器
ALU
0
1
2
3
控制
输入设备
输出设备
0
1
2
3
4
5
6
7
IR
指令
MDR
数据

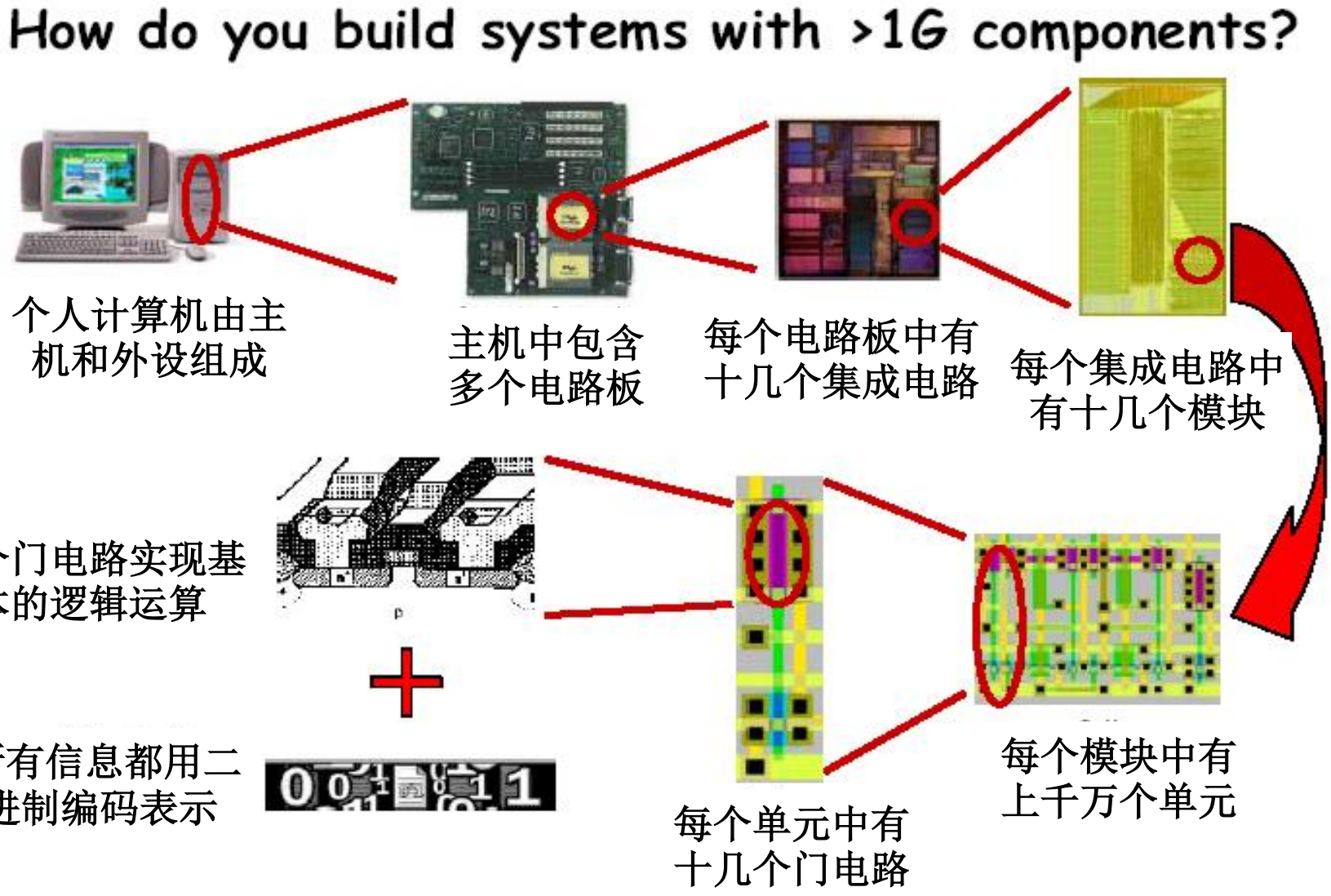
计算机硬件：打开计算机来看看



PC主板



解剖一台计算机


How do you build systems with >1G components?
个人计算机由主机和外设组成
主机中包含多个电路板
每个电路板中有十几个集成电路
每个集成电路中有十几个模块
每个门电路实现基本的逻辑运算
所有信息都用二进制编码表示
每个单元中有十几个门电路
每个模块中有上千万个单元

1.2.3 计算机的工作步骤

- 问题：如何用计算机来解决现实中的问题?
- 问题：是不是所有的问题都能通过计算机解决?

- 面对可以解决的问题，如何用计算机处理?

1.2.3 计算机的工作步骤

1.上机前的准备

- 建立数学模型 $e = E_m Sin\omega t$
- 确定计算方法

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \frac{x^9}{9!} - \cdots$$

$$\sqrt{x} = \frac{1}{2}\left(y_n + \frac{x}{y_n}\right)\left(n = 0, 1, 2, \cdots\right)$$

- 编制解题程序

程序 — 运算的 全部步骤

指令 — 每 一个步骤

编程举例

计算 $ax^2 + bx + c = (ax + b)x + c$

取x 至运算器中

乘以x 在运算器中

乘以a 在运算器中

存ax^2 在存储器中

取b 至运算器中

乘以x 在运算器中

加ax^2 在运算器中

加c 在运算器中

取x 至运算器中

乘以a 在运算器中

加b 在运算器中

乘以x 在运算器中

加c 在运算器中

指令格式举例

操作码	地址码

取数　α　　[α] → ACC

000001　0000001000

存数　β　　[ACC] → β

加　γ　　[ACC]+[γ] → ACC

乘　δ　　[ACC]×[δ] → ACC

打印　δ　　[δ] → 打印机

停机

计算 $ax^2 + bx + c$ 程序清单

指令和数据存于主存单元的地址	指令		注释
	操作码	地址码	
0	000001	0000001000	取数x至ACC
1	000100	0000001001	乘a得ax存于ACC中
2	000011	0000001010	加b得ax+b,存于ACC中
3	000100	0000001000	乘x得（ax+b)x,存于ACC中
4	000011	0000001011	加c得$ax^2 + bx + c$,存于ACC
5	000010	0000001100	将$ax^2 + bx + c$存于主存单元
6	000101	0000001100	打印
7	000110		停机
8	x		原始数据x
9	a		原始数据a
10	b		原始数据b
11	c		原始数据c
12			存放结果

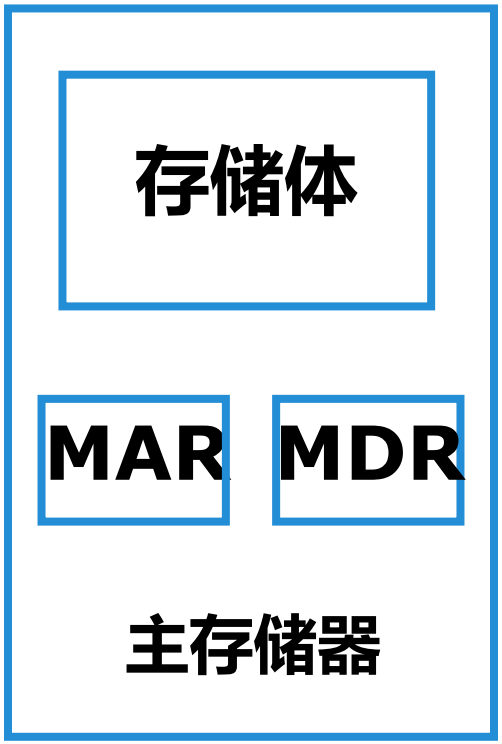
2. 计算机的解题过程

(1)存储器的基本组成

理解

存储体 – 存储单元 – 存储元件（0/1）

[教学楼 – 教室 – 座位 （无人/有人]



存储单元 存放一串二进制代码

存储字 存储单元中二进制代码的组合

存储字长 存储单元中二进制代码的位数

每个存储单元赋予一个地址号

按地址寻访

(2)存储器的基本组成

MAR 存储器地址寄存器

反映存储单元的个数

MDR 存储器数据寄存器

反映存储字长

设 MAR = 5 位

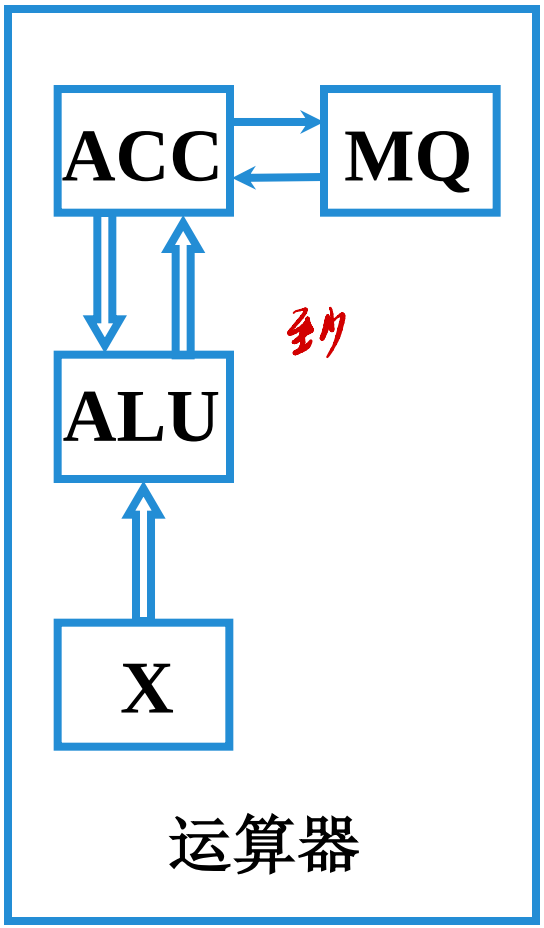
MDR = 16 位

存储单元个数 $32=2^5$

存储字长 16

总存储量=32*16位

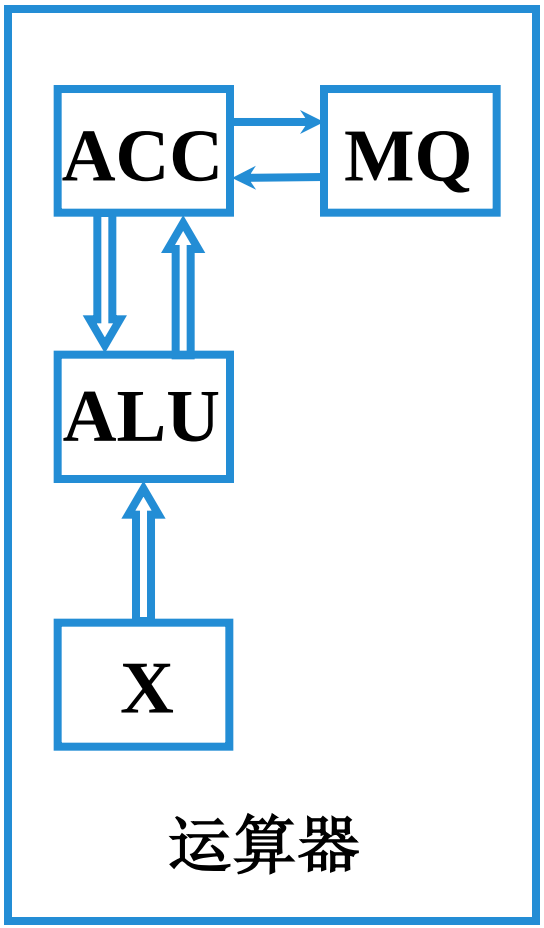
(2) 运算器的基本组成及操作过程



算术运算	逻辑运算	基本结构
加、减、乘、除法等	与、或、非、移位等	ALU (Arithmetic Logical Unit) 、寄存器、连接通路

◆ **注重功能与结构的关系？** -- 指令、数据类型、性能要求等等

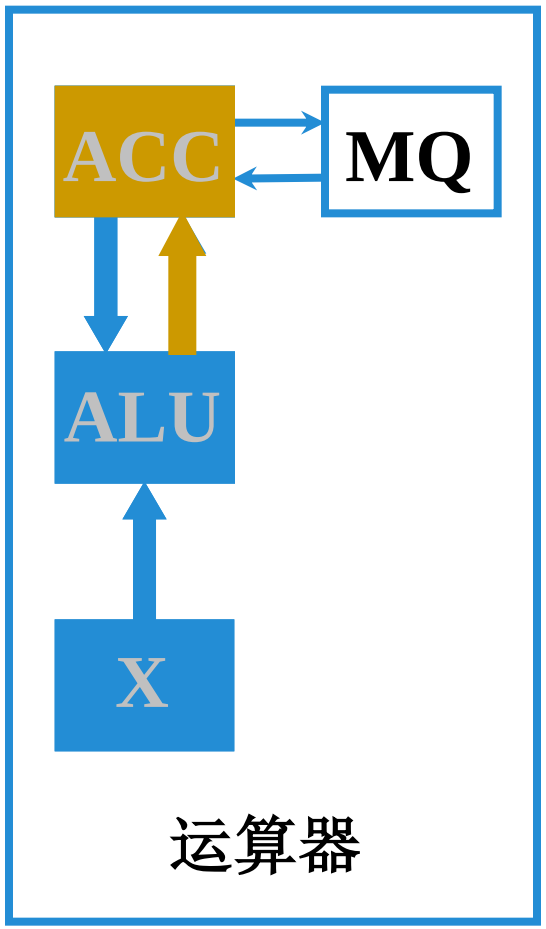
(2) 运算器的基本组成及操作过程



	ACC	MQ	X
加法	被加数 和		加数
减法	被减数 差		减数
乘法	乘积高位	乘数 乘积低位	被乘数
除法	被除数 余数	商	除数

(2)运算器的基本组成及操作过程

① 加法操作过程



指令

加	M

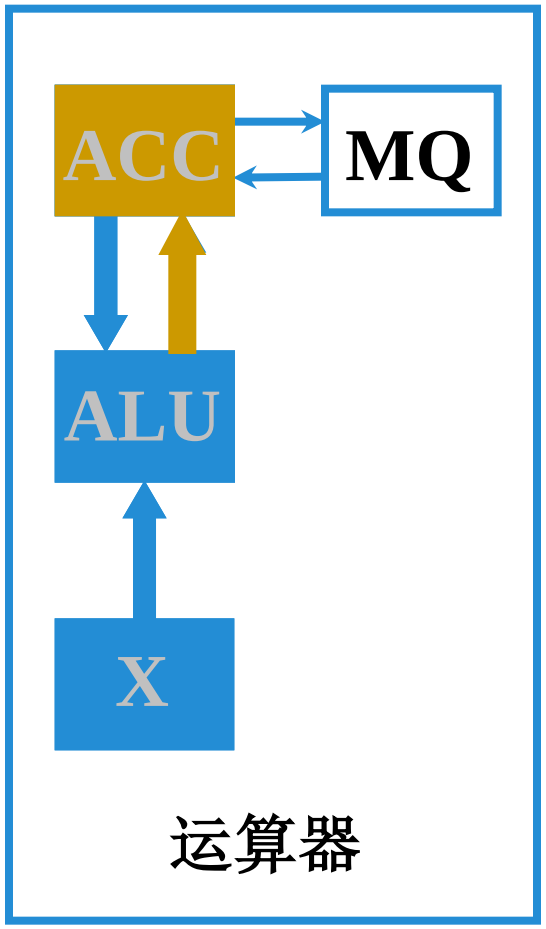
初态　ACC　　被加数

[M] ⟶ X

[ACC]+[X] ⟶ ACC

(2)运算器的基本组成及操作过程

② 减法操作过程



指令 | 减 | M

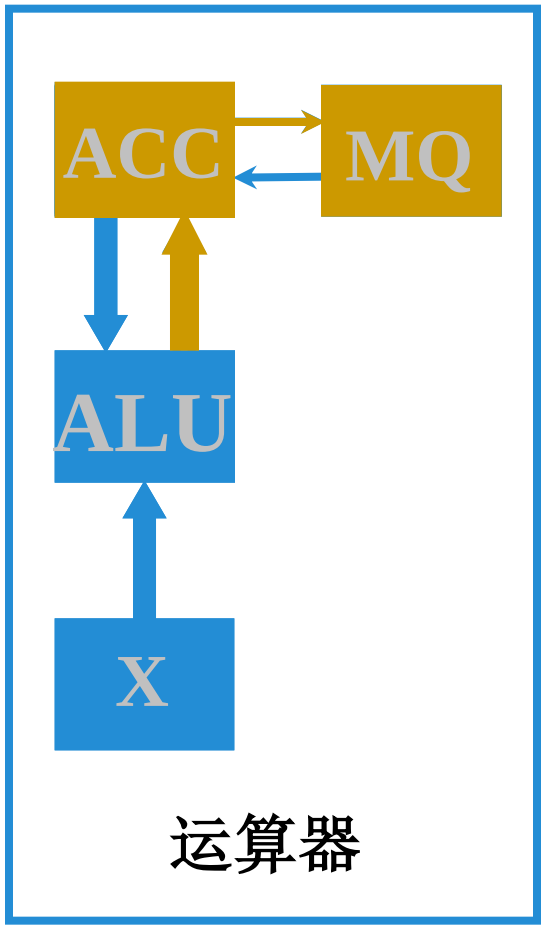
初态 ACC 被减数

[M] ⟶ X

[ACC]-[X] ⟶ ACC

(2)运算器的基本组成及操作过程

③ 乘法操作过程



指令 | 乘 | M

初态 ACC 被乘数

[M] ⟶ MQ

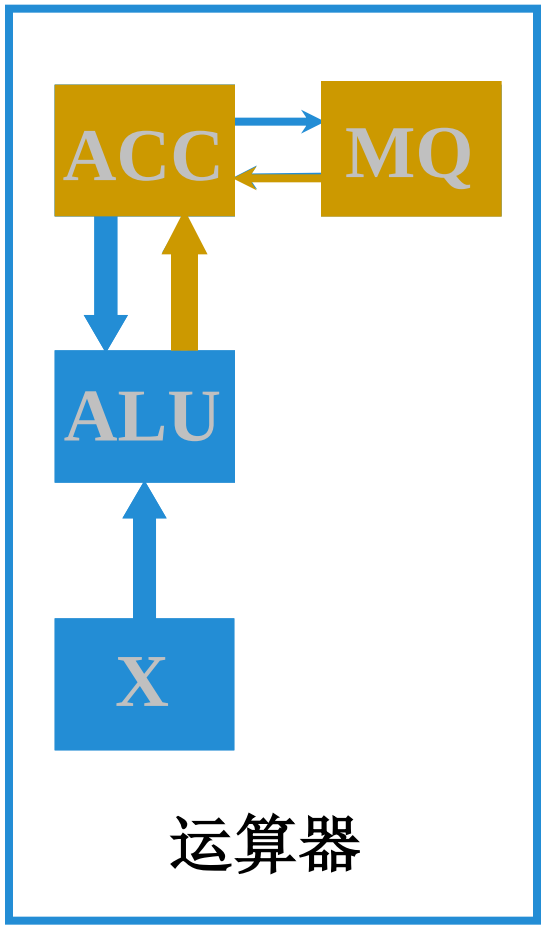
[ACC] ⟶ X

0 ⟶ ACC

[X]×[MQ] ⟶ ACC//MQ

(2)运算器的基本组成及操作过程

④ 除法操作过程



指令 | 除 | M

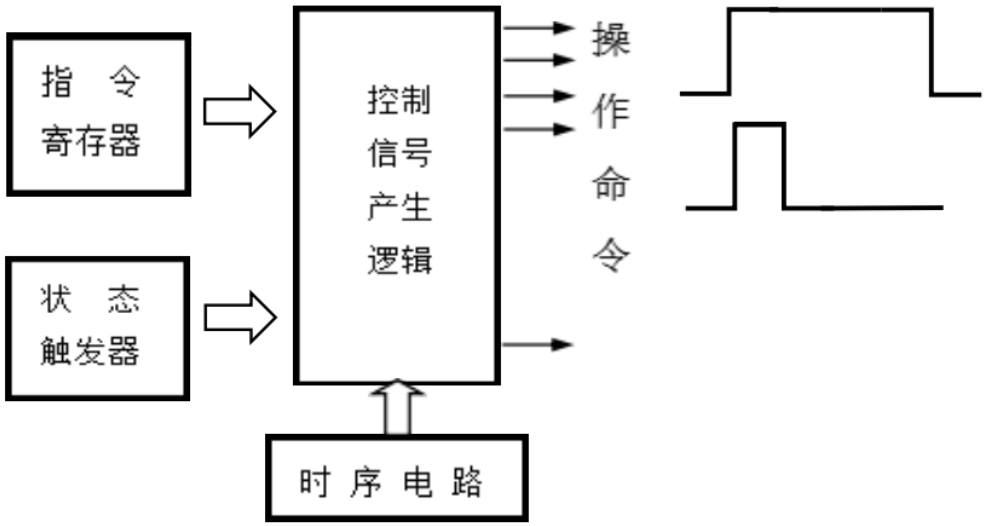
初态　ACC　被除数

[M] ⟶ X

[ACC] ÷ [X] ⟶ MQ

余数在ACC中

(3)控制器的基本组成



基本功能

产生指令执行过程所需要的所有控制信号，控制相关功能部件执行相应操作；

控制信号的形式

电平信号、
脉冲信号；

产生控制信号的依据

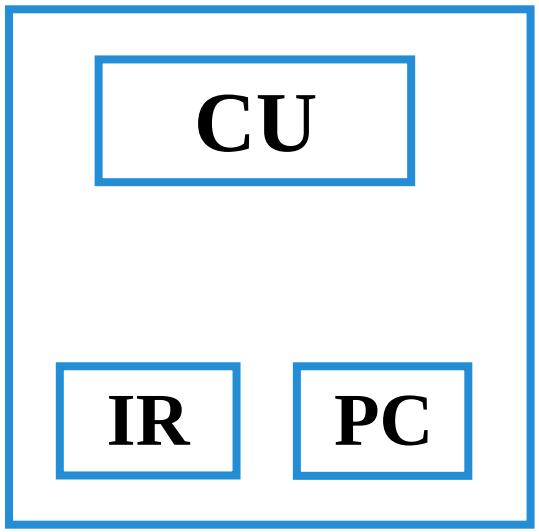
指令、状态、
时序；

控制信号的产生方式

微程序、软件
硬布线。硬件

(3)控制器的基本组成

具体而言：



完成一条指令 { 取指令 PC；分析指令 IR } 取指 访存

执行指令 CU 执行 访存

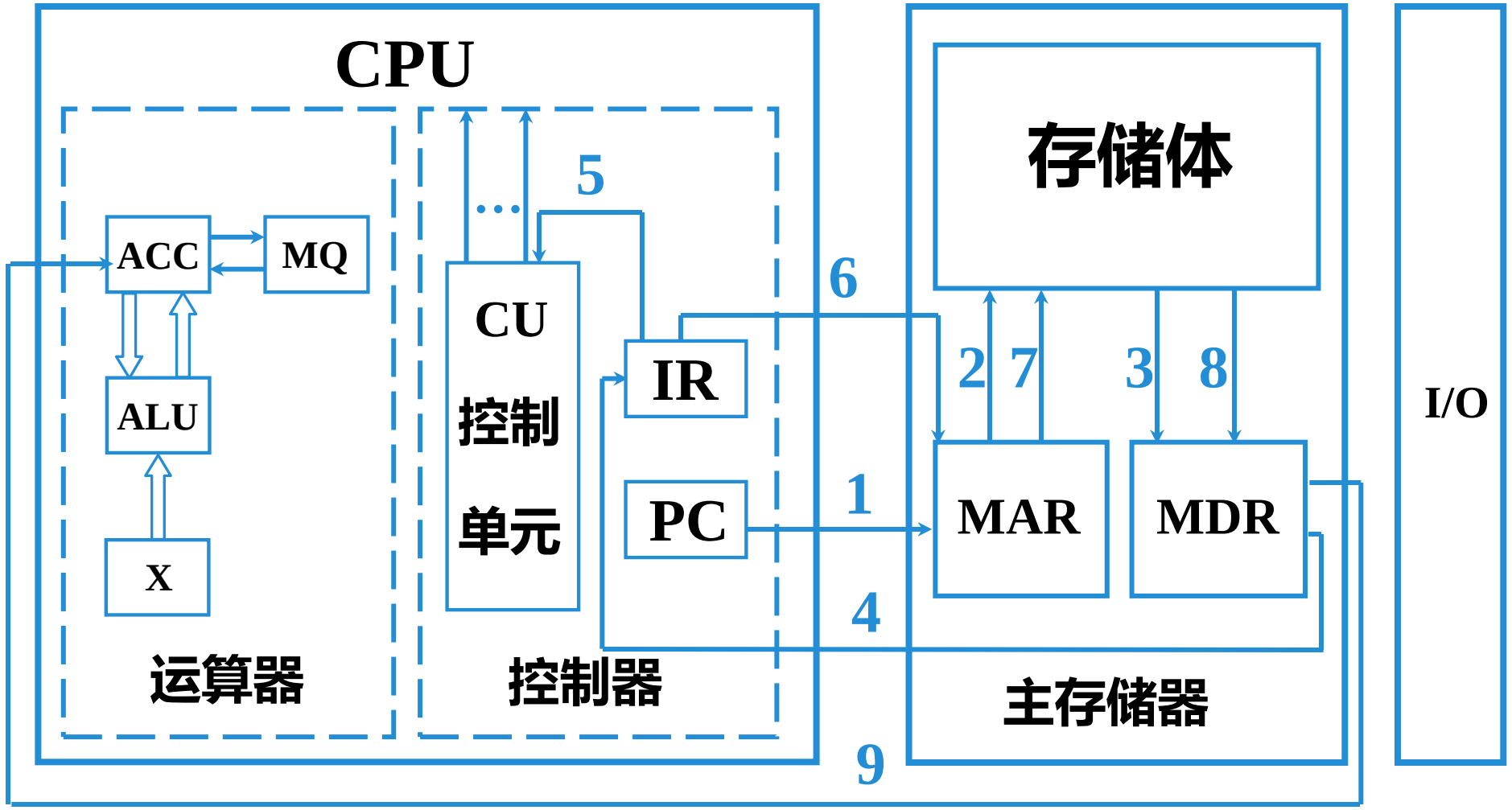
PC 存放当前欲执行指令的地址

具有计数功能（PC）+1⟶PC

IR 存放当前欲执行的指令

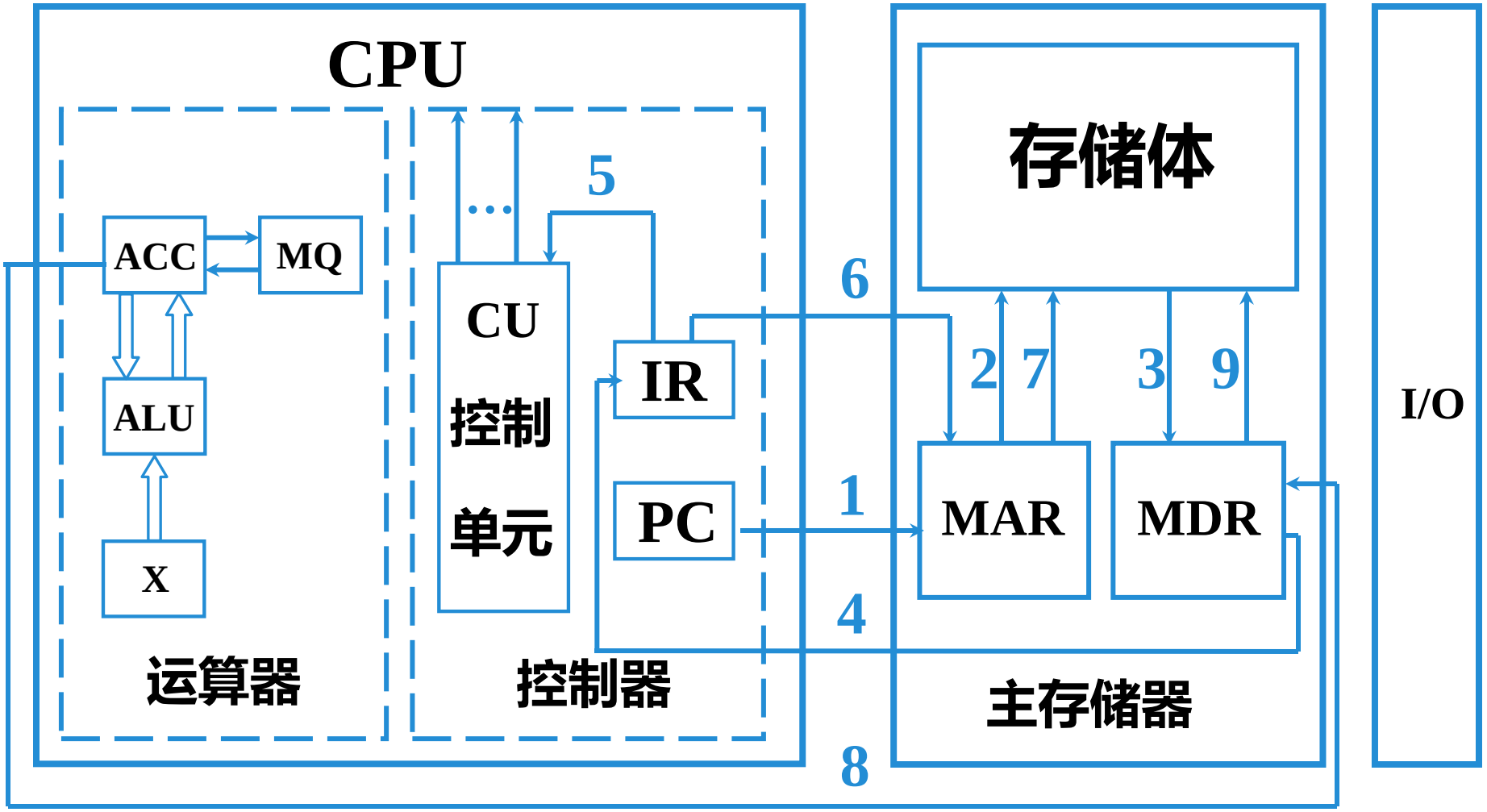
(4)主机完成一条指令的过程

以取数指令为例



(4)主机完成一条指令的过程

以存数指令为例



(5) $ax^2 + bx + c$ 程序的运行过程

- 将程序通过输入设备送至计算机
- 程序首地址 ⟶ PC
- 启动程序运行
- 取指令 PC → MAR → M → MDR → IR ，(PC)+1 → PC
- 分析指令 OP(IR) → CU
- 执行指令 Ad(IR) → MAR → M → MDR → ACC

  ⋮
- 打印结果
- 停机

1.3 计算机系统性能评价

- **如何挑选一台计算机?**
  - 计算机用途
  - 你的预算
    - 速度快，容量大

- **如何评价一台机器的性能?**

1.3 计算机系统性能评价

- **机器字长**

  CPU一次能处理的数据位数，

  通常与CPU中的**寄存器位数**有关。

- **存储容量** 存放二进制的总位数

  - 主存容量
    - = 存储单元个数×存储字长

      MAR，MDR

      10　　8　　1K×8位

      16　　32　　64K×32 位
    - = 字节数　2^{13}**b**it=1 K**B**yte　8b=2^3b=1B
  - 辅存容量　字节数　1TB=2^{40}B

- **运算速度**

K、M、G、T、P的定义

单 位	通常意义	实际表示
K(Kilo)	10^3	$2^{10}=1\ 024$
M(Mega)	10^6	$2^{20}=1\ 048\ 576$
G(Giga)	10^9	$2^{30}=1\ 073\ 741\ 824$
T(Tera)	10^{12}	$2^{40}=1\ 099\ 511\ 627\ 776$
P(Peta)	10^{15}	$2^{50}=1\ 125\ 899\ 906\ 842\ 624$

1.3 计算机系统性能评价

❖运算速度

- 主频（clock rate,时钟频率)
- 时钟周期（clock cycle，tick ） CPU产生的同步时钟定时信号，也就是CPU主脉冲信号，宽度称为时钟周期
- 核数，每个核支持的线程数
- 吉普森（Gibson）法 $T_{\mathrm{M}} = \sum_{i=1}^{n} f_i t_i$
- CPI 执行一条指令所需的时钟周器
- MIPS 百万条指令每秒
- FLOPS 浮点运算次数每秒

CPU的主频，外频和总线频率

- **CPU的主频**，即CPU内核工作的主时钟频率，表示在CPU内数字脉冲信号震荡的速度。比如我们平时看到的4.0GHZ、3.0GHZ等指的就是CPU主频，即每秒可以产生40亿、30亿个脉冲信号。
- **主频=外频×倍频系数**
- 外频通常为系统总线的工作频率，是CPU的基准频率，单位是MHz。外频是CPU与主板之间同步运行的速度，所以也可以说**CPU的外频决定着整块主板的运行速度**。外频往往比CPU的主频和内存频率低很多，一般常见的默认外频值只有100MHz。
- **倍频系数是指CPU的核心工作频率与外频之间存在着一个比值关系**，这个比值就是倍频系数，简称倍频。

CPU的主频，外频和总线频率

- 例如，如果有个主频标称为3.5GHz的处理器，那么电脑CPU倍频公式是这样的：

  100MHz的外频乘以35的倍频等3500MHz，即3.5GHz。

这就是外频乘倍频的模式，目的就是为了让CPU在与电脑其他部分相匹配的同时，仍能以高主频运行。

Computer Organization

Thank You !

计算机组成原理

第二章　计算机的发展及应用

2.1 计算机的发展史

2.2 计算机的应用

2.3 计算机的展望

合肥工业大学
计算机与信息学院
阙夏

2.1 计算机的发展史

一、计算机的产生和发展

代	时间	硬件技术	速度（次/秒）
一	1946~1957	电子管	40 000
二	1958~1964	晶体管	200 000
三	1965~1971	中小规模集成电路	1 000 000
四	1972~现在	大、超大规模集成电路	10 000 000

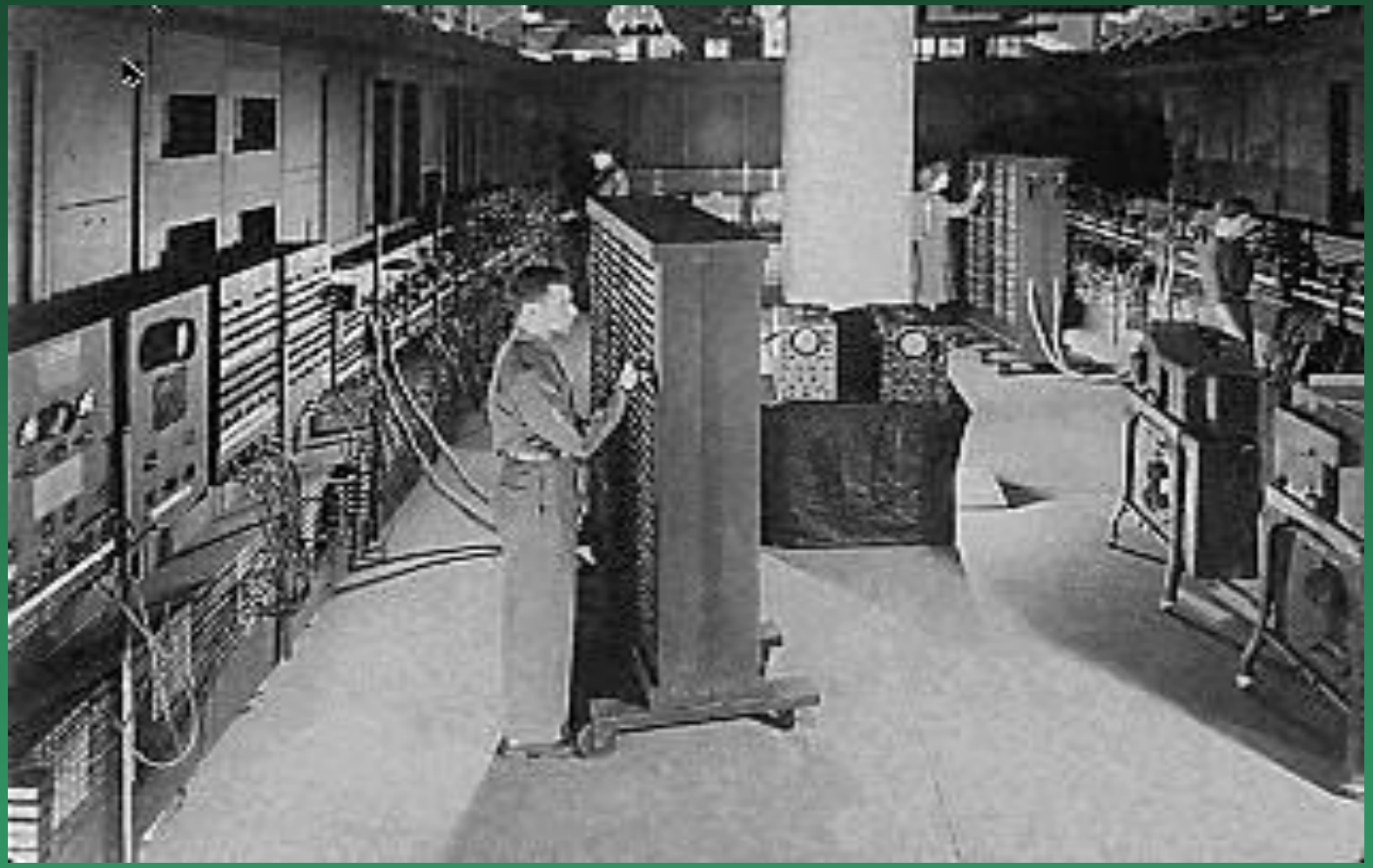
硬件技术对计算机更新换代有重要的影响

2.1 计算机的发展史

一、计算机的产生和发展

- 第一代（1946 - 1957）电子管计算机(Vacuum Tube)
  - 1946年 第一台电子计算机 **ENIAC**诞生
    - ✓ 美国宾夕法尼亚大学莫尔学院机电系
    - ✓ 速度：5 000次加法 / 秒
    - ✓ 十进制表示信息和运算：存储器由20个累加器组成，每个累加器存10位十进制数，每一位由10个真空管表示。
    - ✓ 平方、立方、sin、cos等
    - ✓ 基本器件：真空电子管
    - ✓ 用手工搬动开关和拔插电缆来编程， 采用穿孔卡输入输出数据， 1955年退役

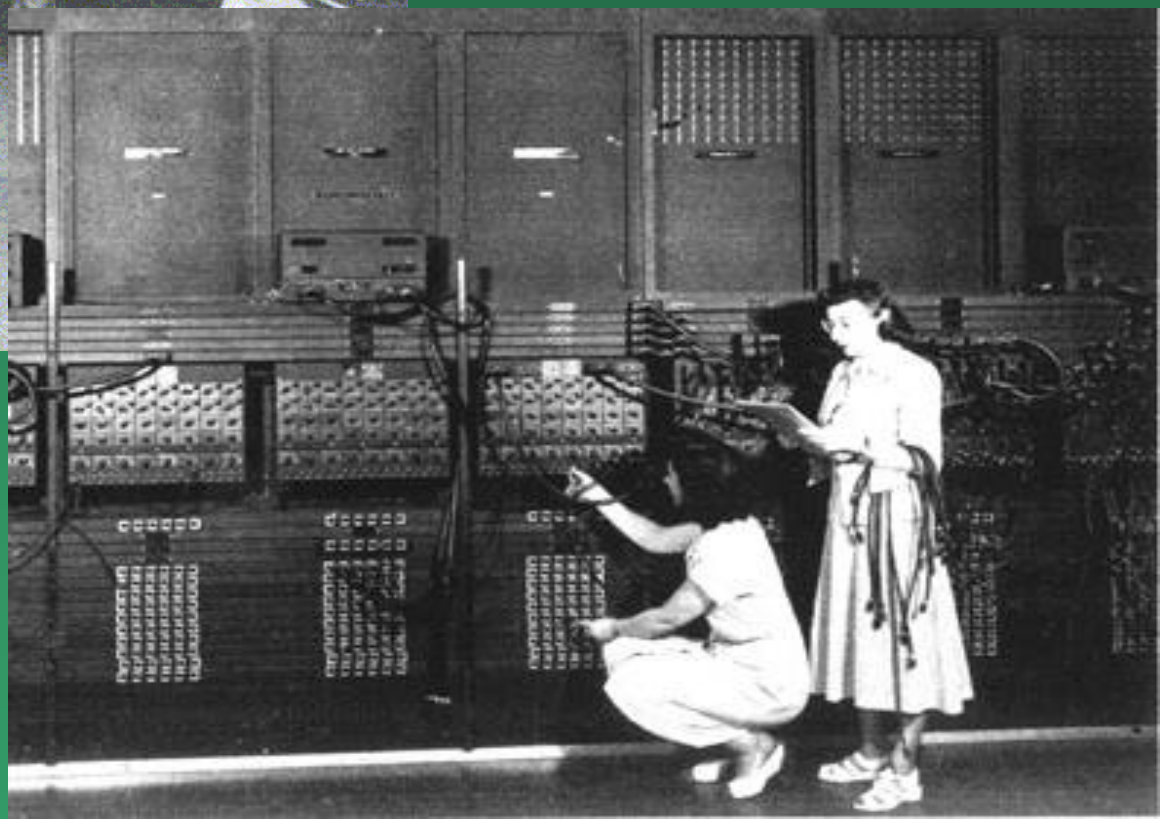
Electronic Numerical Integrator And Computer
电子数字积分计算机



世界上第一台电子计算机 ENIAC(1946)

Electronic Numerical Integrator And Computer

占地面积170平方米
重30吨
有17468个真空管
耗电160千瓦



该机正式运行到
1955年10月2日，
这十年间共运行
80 223个小时

阿塔那索夫与ABC计算机

- 1940年12月，阿塔那索夫参加在费城举行的美国科学促进协会AAAS(American Association for Advancement Of Science)年会时遇到了莫奇利，两人都对计算机感兴趣，谈得很投机;
- 次年6月莫奇利去拜访了阿塔那索夫，参观了ABC机(当时已接近完成)，看了设计图纸和资料。
- 因此诉讼中涉及莫奇利和埃克特的ENIAC专利是否有效的问题，是否剽窃了阿塔那索夫的思想的问题。经过马拉松式的听证和审判过程，1973年法院判决莫奇利和埃克特的ENIAC不是他们自己发明的，而是由阿塔那索夫的研究中导出的，因此ENIAC专利无效。
- 这样，阿塔那索夫赢了这场官司，被人称为"电子计算机之父"，声名大振。事实上，对这个判决，学术界和舆论界分歧很大，支持和反对的两种呼声都很高。我们这里也不拟作出评判。应该说，IEEE-CS对这三人都授予了计算机先驱奖是比较客观和公正的，三人各有其功劳。有评论说，ABC机是一颗"近失弹"(a near-miss)，即没有正中目标，但非常接近目标，这是比较实事求是的。

冯·诺依曼的故事

- 1944年，冯·诺依曼参加原子弹的研制工作，涉及到极为困难的计算。
- 1944年夏的一天，诺依曼巧遇美国弹道实验室的军方负责人戈尔斯坦，他正参与ENIAC的研制工作。
- 冯·诺依曼被戈尔斯坦介绍加入ENIAC研制组，1945年，他们在共同讨论的基础上，冯·诺依曼以“关于EDVAC的报告草案”为题，起草了长达101页的总结报告，发表了全新的“**存储程序通用电子计算机方案**”。
- 一向专搞理论研究的普林斯顿高等研究院批准让冯·诺依曼建造计算机，其依据就是这份报告。

Electronic Discrete Variable Automatic Computer

电子离散变量自动计算机

现代计算机的原型

1946年，普林斯顿高等研究院（the Institute for Advance Study at Princeton，IAS ）开始设计“存储程序”计算机，被称为IAS计算机（1951年才完成，它并不是第一台存储程序计算机，1949年由英国剑桥大学完成的EDSAC是第一台）。

- 在那个报告中提出的计算机结构被称为冯·诺依曼结构。
- 冯·诺依曼结构最重要的思想是“存储程序(Stored-program)”
- 工作方式：

  任何要计算机完成的工作都要先被编写成程序，然后将程序和原始数据送入主存并启动执行。一旦程序被启动，计算机应能在不需操作人员干预下，自动完成逐条取出指令和执行指令的任务。
- 冯·诺依曼结构计算机也称为冯·诺依曼机器（Von Neumann Machine）。
- 几乎现代所有的通用计算机大都采用冯·诺依曼结构，因此，IAS计算机是现代计算机的原型机。

冯•诺依曼结构的主要思想是什么呢?

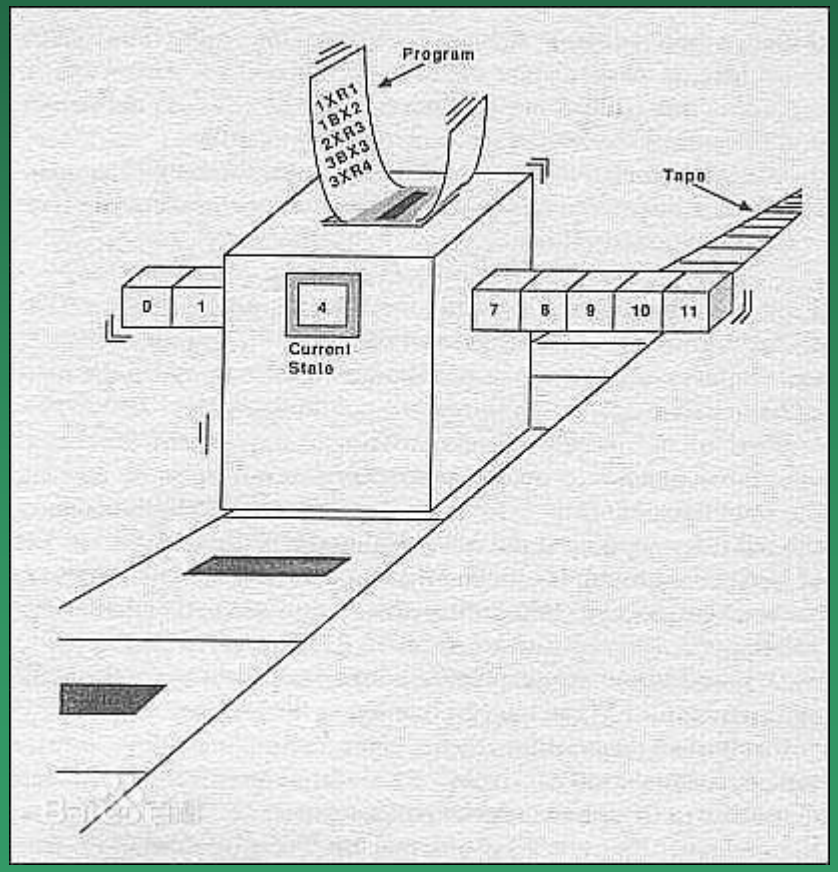
图灵和图灵机

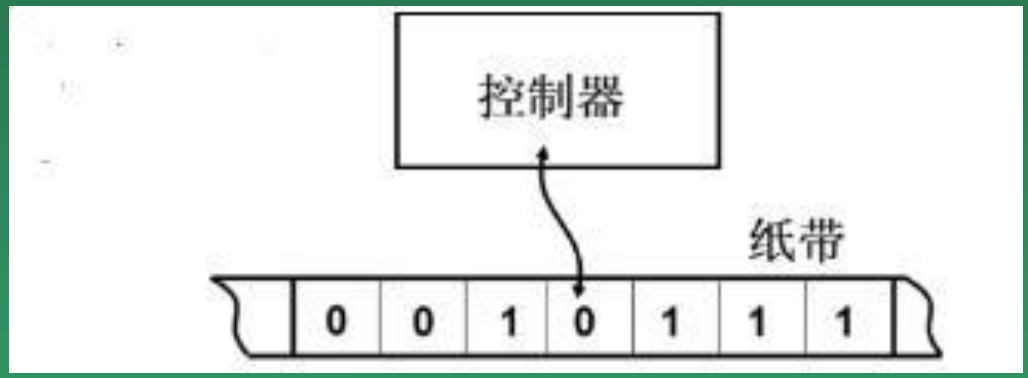
- 图灵机的提出：

1936年5月，图灵向伦敦权威的数学杂志投了一篇论文，题为《论数字计算在决断难题中的应用》。该文于1937年在《伦敦数学会文集》第42期上发表后，立即引起广泛的注意。在论文的附录里他描述了一种可以辅助数学研究的机器，后来被人称为“图灵机”，这个设想最有变革意义的地方在于，它第一次在纯数学的符号逻辑，和实体世界之间建立了联系，后来我们所熟知的电脑。

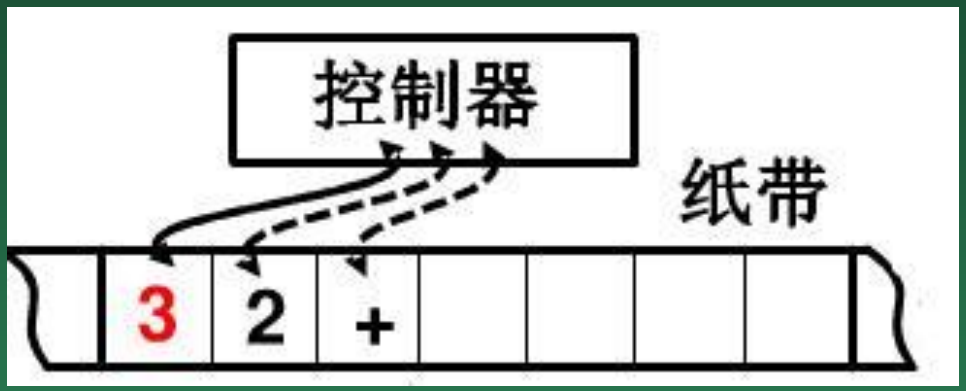
图灵机

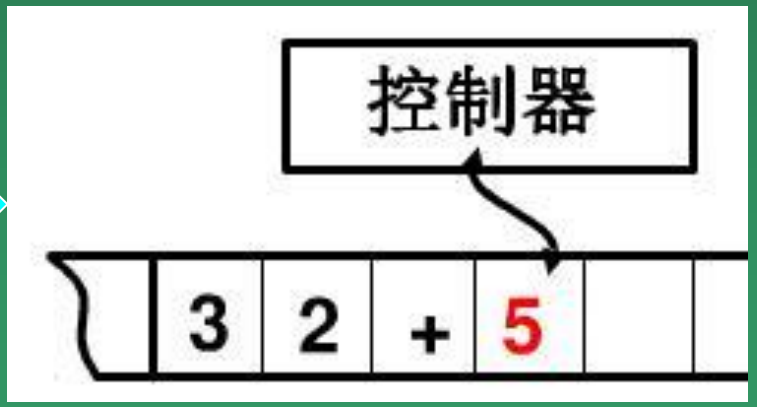
- 图灵机，又称图灵计算机，即将人们使用纸笔进行数学运算的过程进行抽象，由一个虚拟的机器替代人类进行数学运算。





图灵机进行运算的一个例子





图灵机和冯诺依曼机的区别

图灵机的指令系统单一不够完善。

- 1.图灵机没能将指令存储起来重复使用；
- 2.图灵机没能形成实现程序结构设计。

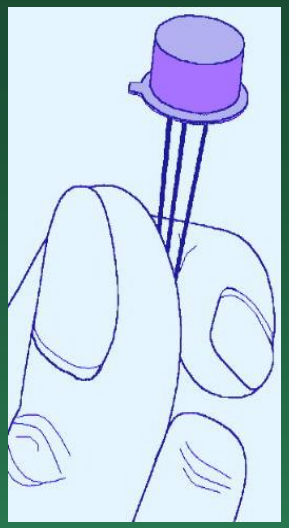
图灵机欠缺的这两点恰被冯诺依曼提出的**程序数据存储**的思想所解决。

第一代（1946 - 1957）电子管计算机

- 第一代计算机特点
  - 元器件：电子管可以存储信息,存储容量：几千存储单元;
  - 使用机器语言;
  - 输入输出速度慢;
  - 计算速度：几万次/秒;
  - 应用领域为科学计算，为军事国防尖端科技服务

第二代，1958 - 1964，晶体管计算机

- 元器件：逻辑元件采用晶体管；
- 内存由磁芯构成，外存为磁鼓与磁带；
- 存储容量：10万存储单元；
- 汇编语言代替机器语言，使用操作系统；
- 计算速度：几十万次/秒；
- 特点：变址，浮点运算，多路存储器，I/O处理机，中央交换结构(非总线结构)。
- 应用：在军事国防尖端科技上的应用范围进一步扩大，并应用在气象、数据处理、以及其它科学研究领域。
- 代表机型：IBM 7094 (scientific)、1401 (business)和 DEC PDP-1



晶体管：
Transistor

DEC PDP-1

第三代，1965 - 1971，小、中规模集成电路（SSI、MSI）计算机

- 元器件：逻辑元件与主存储器均由集成电路（IC）实现 。（功耗、体积和价格进一步下降）
- 特点：微程序控制，Cache，虚拟存储器，流水线等。
- 使用了操作系统
- 计算速度达到百万次/秒
- 代表机种：IBM 360和DEC PDP-8（大/巨型机与小型机同时发展 ）
  - 巨型机(Supercomputer)：Cray-1
  - 大型机(Mainframe)：IBM360系列
  - 小型机(Minicomputer)：DEC PDP-8

IBM System/360系列计算机

- IBM公司于1964年研制成功
- 引入了“兼容机”（“系列机”）概念
  - 兼容机的特征：
    - 相同的或相似的指令集
    - 相同或相似的操作系统
    - 更高的速度
    - 更多的I/O端口数
    - 更大的内存容量
    - 更高的价格

低端机指令集是高端机的一个子集，称为“向后兼容”。功能相同，而性能不同。

IBM System / 360

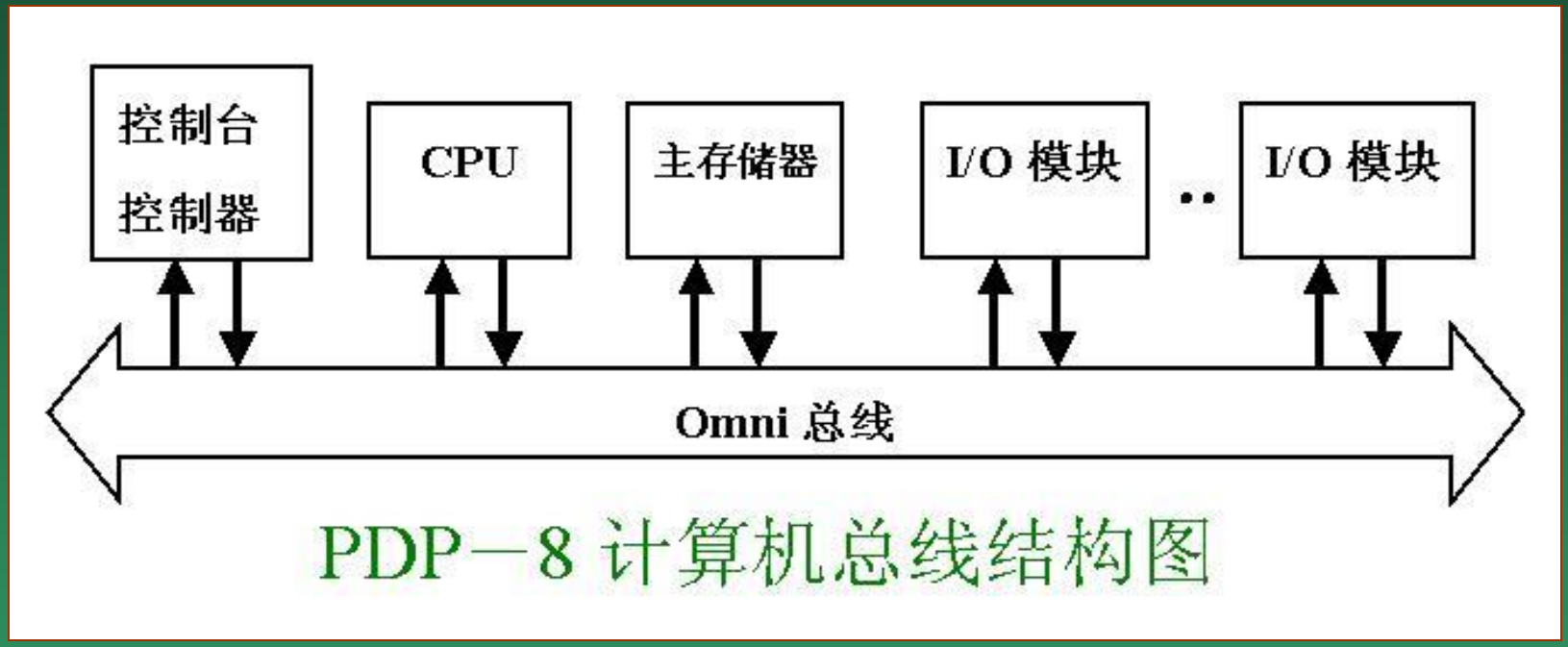
2.1

DEC公司的PDP-8机

- 同在1964年出现。与IBM 360相比，价格更低、更小巧，因而被称为小型机（Minicomputer）
- PDP-8“创造了小型机的概念，并使之成为数十亿美元的工业”，使DEC成为了最大的小型机制造商。
- 主要特点：首次采用总线结构。

  Omnibus总线包含了96个独立的信号通道，用以传送控制、地址和数据信号。这种结构具有高度的灵活性，允许将模块插入总线以形成各种配置。

PDP–8/E计算机系统框图



PDP—8 计算机总线结构图

问题：“总线结构”有什么好处?

具有高度的灵活性，允许将模块插入总线以形成各种配置
节省器件，体积小，价格便宜

第四代，1972～至今，大、超大规模集成电路（LSI/VLSI/ULSI）计算机

- 微处理器和半导体存储器技术发展迅猛，微型计算机出现。使计算机进入了几乎所有行业。

  半导体存储器

  - 70年Fairchild公司生产出第一个相对大容量半导体存储器
  - 74年位价格低于磁芯的半导体存储器出现，并快速下跌
  - 从70年起，存储密度呈4倍提高（几乎是每3年）

  微处理器

  - 微处理器芯片密度不断增加，使CPU中所有元件放在一块芯片上成为可能。71年开发出第一个微处理器芯片4004。
- 特点：共享存储器，分布式存储器及大规模并行处理系统
- 出现了包含CPU的单片IC(微处理器)，并出现了以微处理器为核心的电子计算机即微型计算机，使计算机

我国计算机发展历程

- 从1956年开始研制
- 1958年，研制成功电子管数字计算机
  - **“八一”型通用电子计算机（103机）**
- 1971年，研制成功电子管数字计算机
- 1973年，四机部决定由清华大学、安徽无线电厂、四机部电子技术推广应用研究所（六所）组成联合设计组，参考Intel 8008,研制DJS 050微型计算机，并于1977年通过鉴定。
- 1978年，研制出每秒500万次大型计算机
- 1984年国防科大研制出每秒1亿次“银河“电子计算机，及银河II型、银河III型计算机。
- 2008.11，中科院计算所国家智能计算机研究开发中心、曙光信息产业（北京）有限公司、上海超级计算中心联合研制的集群超级计算机曙光5000A在TOP500强排行榜中位居第10。
- 2009.11安装在我国国家超级计算天津中心的“天河一号”居第5位

我国计算机发展历程

- 2010.11天河1号二期工程“天河1”位居top500第1名
- **天河二号超级计算机**
- “天河二号”是由国防科学技术大学研制的超级计算机系统，以峰值计算速度每秒5.49亿亿次、持续计算速度每秒3.39亿亿次双精度浮点运算的优异性能位居榜首，成为2013年全球最快超级计算机。2014年11月17日公布的全球超级计算机500强榜单中，中国“天河二号”以比第二名美国“泰坦”快近一倍的速度连续第四次获得冠军。 2015年5月，“天河二号”上成功进行了3万亿粒子数中微子和暗物质的宇宙学N体数值模拟，揭示了宇宙大爆炸1600万年之后至今约137亿年的漫长演化进程。同时这是迄今为止世界上粒子数最多的N体数值模拟；11月16日，全球超级计算机500强榜单在美国公布，“天河二号”超级计算机以每秒33.86千万亿次连续第六度称雄。 2016年6月20日，新一期全球超级计算机500强榜单公布，使用中国自主芯片制造的“神威·太湖之光”取代“天河二号”登上榜首。2018年11月12日，新一期全球超级计算机500强榜单在美国达拉斯发布，中国超算“天河二号”位列第四名。

我国计算机发展历程

- **神威·太湖之光**超级计算机

由国家并行计算机工程技术研究中心研制、安装在国家超级计算无锡中心的超级计算机。**神威·太湖之光**超级计算机安装了40960个中国自主研发的“申威26010”众核处理器，该众核处理器采用64位自主申威指令系统，峰值性能为12.5亿亿次/秒，持续性能为9.3亿亿次/秒。2016年6月20日，在法兰克福世界超算大会上，国际TOP500组织发布的榜单显示，“神威·太湖之光”超级计算机系统登顶榜单之首，不仅速度比第二名“天河二号”快出近两倍，其效率也提高3倍；[1] 11月14日，在美国盐湖城公布的新一期TOP500榜单中，“神威·太湖之光”以较大的运算速度优势轻松蝉联冠军；11月18日，我国科研人员依托“神威·太湖之光”超级计算机的应用成果首次荣获“戈登·贝尔”奖，实现了我国高性能计算应用成果在该奖项上零的突破。2017年5月，中华人民共和国科学技术部高技术中心在无锡组织了对“神威·太湖之光”计算机系统课题的现场验收。专家组经过认真考察和审核，一致同意其通过技术验收；11月13日，全球超级计算机500强榜单公布，“神威·太湖之光”以每秒9.3亿亿次的浮点运算速度第四次夺冠。2018年11月12日，新一期全球超级计算机500强榜单在美国达拉斯发布，中国超算“神威·太湖之光”位列第三名。

最权威的超级计算机排名的参考网址

http://www.top500.org

TOP 5 Sites for JUNE 2020:

Rank	System	Cores	Rmax (TFlop/s)	Rpeak (TFlop/s)	Power (kW)
1	**Supercomputer Fugaku** - Supercomputer Fugaku, A64FX 48C 2.2GHz, Tofu interconnect D, Fujitsu RIKEN Center for Computational Science Japan	7,299,072	415,530.0	513,854.7	28,335
2	**Summit** - IBM Power System AC922, IBM POWER9 22C 3.07GHz, NVIDIA Volta GV100, Dual-rail Mellanox EDR Infiniband, IBM DOE/SC/Oak Ridge National Laboratory United States	2,414,592	148,600.0	200,794.9	10,096
3	**Sierra** - IBM Power System AC922, IBM POWER9 22C 3.1GHz, NVIDIA Volta GV100, Dual-rail Mellanox EDR Infiniband, IBM / NVIDIA / Mellanox DOE/NNSA/LLNL United States	1,572,480	94,640.0	125,712.0	7,438
4	**Sunway TaihuLight** - Sunway MPP, Sunway SW26010 260C 1.45GHz, Sunway, NRCPC National Supercomputing Center in Wuxi China	10,649,600	93,014.6	125,435.9	15,371
5	**Tianhe-2A** - TH-IVB-FEP Cluster, Intel Xeon E5-2692v2 12C 2.2GHz, TH Express-2, Matrix-2000, NUDT National Super Computer Center in Guangzhou China	4,981,760	61,444.5	100,678.7	18,482

超级计算机前5名介绍

https://zhuanlan.zhihu.com/p/161667856

https://dxtol.com/arm-chips-are-inside-the-worlds-most-powerful-supercomputer/

Summit

Summit - IBM Power System AC922, IBM POWER9 22C 3.07GHz, NVIDIA Volta GV100, Dual-rail Mellanox EDR Infiniband

Site:	DOE/SC/Oak Ridge National Laboratory
System URL:	http://www.olcf.ornl.gov/olcf-resources/compute-systems/summit/
Manufacturer:	IBM
Cores:	2,414,592
Memory:	2,801,664 GB
Processor:	IBM POWER9 22C 3.07GHz
Interconnect:	Dual-rail Mellanox EDR Infiniband
Performance	
Linpack Performance (Rmax)	148,600 TFlop/s
Theoretical Peak (Rpeak)	200,795 TFlop/s
Nmax	16,473,600
HPCG [TFlop/s]	2,925.75
Power Consumption	
Power:	10,096.00 kW (Submitted)
Power Measurement Level:	3
Measured Cores:	2,397,824
Software	
Operating System:	RHEL 7.4
Compiler:	XLC, nvcc
Math Library:	ESSL, CUBLAS 9.2
MPI:	Spectrum MPI

Sunway TaihuLight - Sunway MPP, Sunway SW26010 260C 1.45GHz, Sunway

Site:	National Supercomputing Center in Wuxi
Manufacturer:	NRCPC
Cores:	10,649,600
Memory:	1,310,720 GB
Processor:	Sunway SW26010 260C 1.45GHz
Interconnect:	Sunway
Performance	
Linpack Performance (Rmax)	93,014.6 TFlop/s
Theoretical Peak (Rpeak)	125,436 TFlop/s
Nmax	12,288,000
HPCG [TFlop/s]	480.848
Power Consumption	
Power:	15,371.00 kW (Submitted)
Power Measurement Level:	2
Software	
Operating System:	Sunway RaiseOS 2.0.5

国家超级计算无锡中心
National Supercomputing Center in Wuxi
登录
电信
联通
移动
教育网
中文 | EN
中心概况
中心新闻
中心资源
中心业务
上机指南
典型案例
开拓创新、团结协作、开放共享、追求卓越
神威
太湖之光
世界上首台峰值运行速度超过十亿亿次的超级计算机
中国第一台全部采用国产处理器构建的世界第一的超级计算机

“神威·太湖之光”高效能计算系统

系统峰值性能：125.436PFlops

实测持续运算性能：93.015PFlops

处理器型号："申威26010" 众核处理器

整机处理器个数：40960个

整机处理器核数：10649600个

系统总内存：1310720 GB

操作系统：Raise Linux

编程语言：C、C++、Fortran

并行语言及环境：MPI、OpenMP、OpenACC等

SSD存储：230TB

在线存储：10PB，带宽288GB/s

近线存储：10PB，带宽32GB/s

天河2号（银河2号）超级计算机

- 国防科技大学研制的天河2号超级计算机以33.86千万亿次/秒（petaflop/s）的双精度浮点运算速度成为全球最快的超级计算机，2013年位居top500榜首。2014年天河2号比第二名Titan快近一倍。
- 天河2号（银河2号）有16000个节点，每个节点由两个Intel Xeon IvyBridge处理器和三个Xeon Phi协处理器组成，总共3120000个计算核心，部署在广州的国家超级计算机中心。

天河一号二期工程——天河1A

陈田老师和天河一号

Sierra

Sierra, Livermore's latest advanced technology high performance computing system, joined LLNL's lineup of supercomputers in 2018. The new system provides computational resources that are essential for nuclear weapon scientists to fulfill the National Nuclear Security Administration's stockpile stewardship mission through simulation in lieu of underground testing. **Advanced Simulation and Computing (ASC) Program** scientists and engineers will use Sierra to assess the performance of nuclear weapon systems as well as nuclear weapon science and engineering calculations. These calculations are necessary to understand key issues of physics, the knowledge of which later makes its way into the integrated design codes. This work on Sierra has important implications for other global and national challenges such as nonproliferation and counterterrorism.

Sequoia

- Sequoia的建造主要是为美国国家核安全管理局进行复杂的模拟运算，这对于国家核武器储备的安全管理十分重要。

- 走鹃为BladeCenter QS22/LS21 集群，由IBM PowerXCell 3.2GHz和AMD Opteron DC 1.8GHz两种处理器组成，拥有122400个计算核心。
- 其中，皓龙处理器负责标准的运算处理，如文件系统的I/O，而PowerXCell 芯片主要加速数学和CPU密集型运算。

Roadrunner的使用

花费1.33亿美元的Roadrunner是在纽约州的IBM Poughkeepsie研究中心进行建造、检测、调试和运行Linpack benchmark的。这套重达250吨的宠然大物的最终安装地点——新墨西哥州北部美国能源部洛斯阿拉莫斯国家实验室。在这里，它会被美国国家核安全局（National Nuclear Security Administration，NNSA) 用来确保美国核武器储备的持续发展、安全和可靠性，包括模拟核爆炸后零点几秒的行为状态。自上一次物理核试验以来，已经过去了15年，在这期间，NNSA一直在使用计算机模拟来测试国家的核武器。

“跟我所知道的其他任何一家科技企业相比，这些模拟试验用到更多物理学、化学和材料科学方面的技术。”NNSA研究开发和模拟实验室主任Demitri Kusnezov说，“它每年需要使用最大型的计算机系统来模拟非常复杂的问题。Roadrunner是我们在这方面最新的一套工具，相信它是一个有里程碑意义的成就。”

在执行武器模拟之余，Roadrunner也会用于航天、能源、人类基因、纳米和气候方面的研究。IBM的一个应用小组已经在模拟人类大脑方面达到了petaflop的性能水平。2009年早期在Los Alamos实验室的一些测试项目的应用包括模拟生物燃料纤维素分子碎裂（molecular breakdown of cellulose for biofuels）、超新星光变曲线（supernova light curves）、等离子物理的三维磁体重联（3D magnetic reconnection in plasma physics）和金属纳米线的时间演化（time evolution of metallic nanowires）

超级计算机进行核试验

1996年9月10日的联合国大会，以158票赞成、3票反对通过了《全面禁止核试验条约》。不过善良的人们，不要过于高兴，因为该条约的通过只能说明核竞赛在实爆方面的结束，并不等于所有形式的核试验的终结，更不等于核战争的阴影从此消失。计算机模拟仿真就是一种有效的核试验方式。

过去，核爆炸试验曾经是研制核武器不可缺少的一环，但情况有了较大的改变。科学技术的发展已经降低了核爆炸试验在研制核武器过程中的作用。从技术上讲，计算机仿真核试验对于研制和完善新型核武器将具有重大意义。　俄罗斯军事专家认为，“许多西方发达国家即便不进行核试验，也能运用高速大规模电子计算机，在三维空间对核爆炸的全过程进行全方位模拟”。一语道破天机：未来的核试验，将不用核弹用电脑。

- 目前美国使用的三种核武器试验方式
  - 超级计算机仿真核爆炸
  - 在实验室用特种设备进行物理模拟
  - 进行亚临界核试验

亚临界核试验对计算机技术要求很高，只有美、俄、英、法等国家拥有这种能力。

第三种方法

- 通过SSP （*The Stockpile Stewardship Program*）计划不进行核实验，我们在全面禁止实验下能应对保持美国核威胁的挑战吗?
  - We can meet the challenge of maintaining our nuclear deterrent under a [comprehensive test ban] through a Stockpile Stewardship Program without nuclear testing?
- 要建立领先的计算建模与仿真能力
  - 迅速推进从基于核实验的方法到基于计算的方法的转变
    - to promptly shift from nuclear test-based methods to computationally based methods,
  - 将储存维护单元集成起来，从而减少核危险
    - to integrate stockpile stewardship elements and thus reduce the nuclear danger.

Simulation -- A 21st Century Discipline

Gilbert Weigand， US Department of Energy，1999

第三种方法

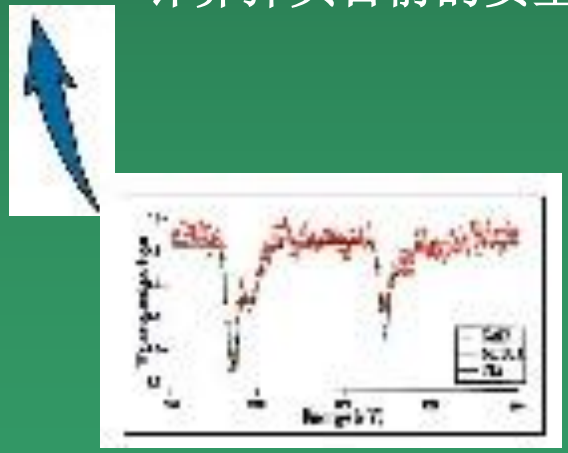
SSP（*The Stockpile Stewardship Program*）必须保证核武器继续安全而可靠

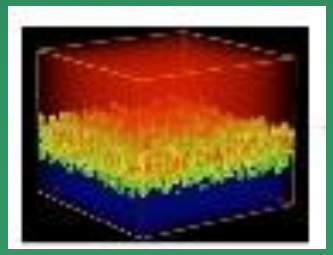
潜在的保存问题

计算弹头目前的安全系数

利用AGEX 数据

验证高级计算仿真





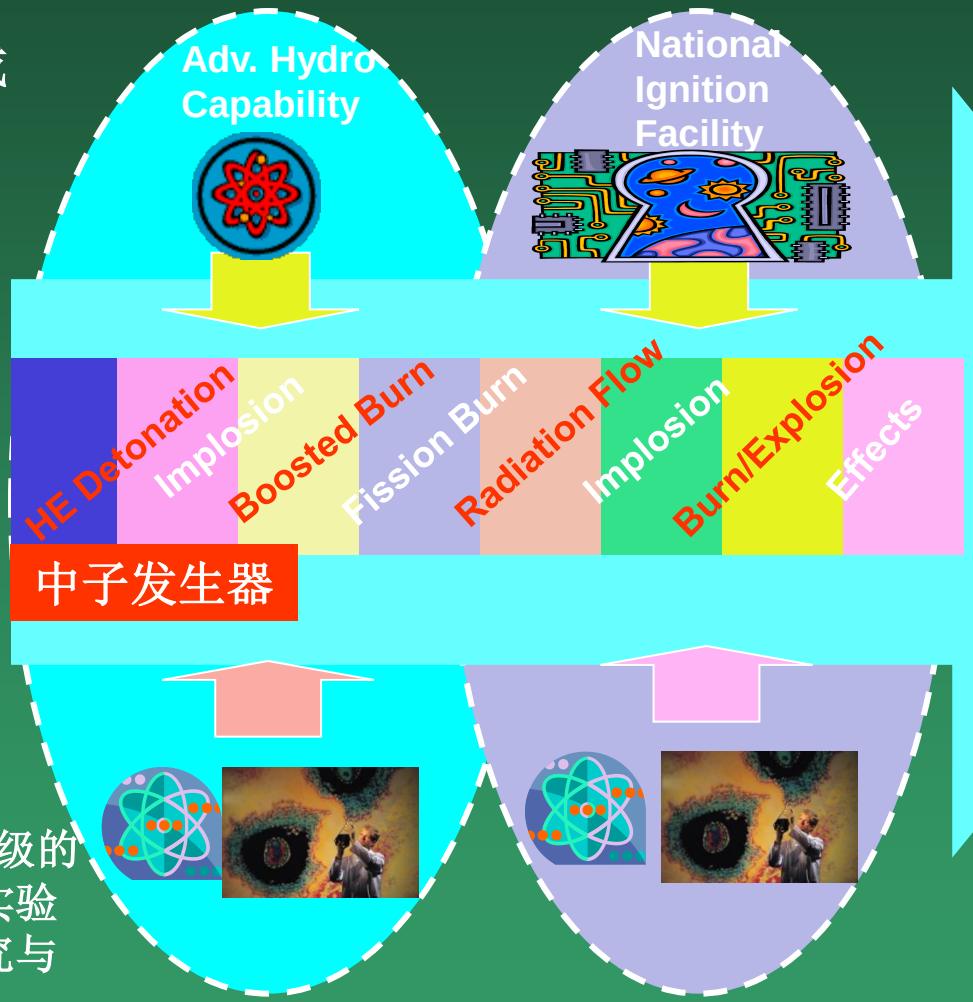
利用验证过的仿真数据重复地下核试验

Shifting promptly from test-based methods to science-based methods

(1) 全规模或全能量密度研究与实验

(2)仿真和理论

(3) 世界领先级的学术机构与实验室的科学研究与实验



存档的武器数据

评价与确认

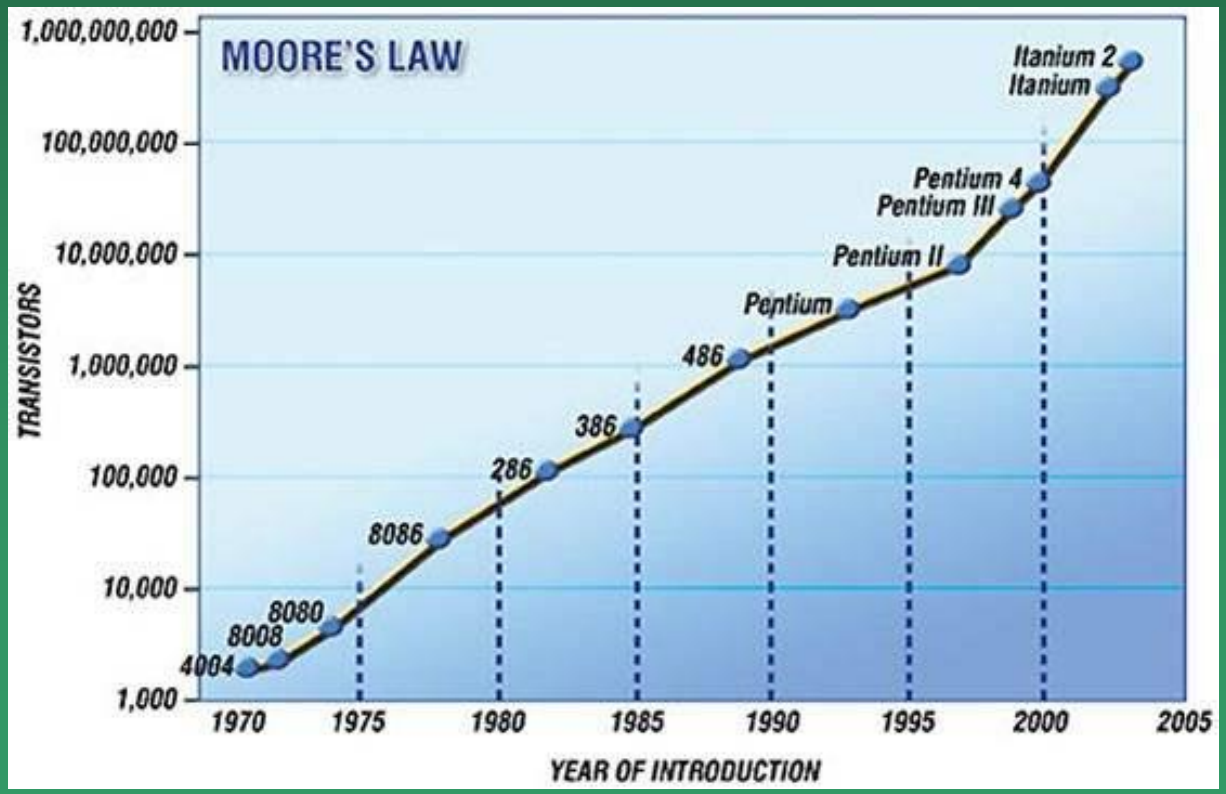
核输出
- 安全性
- 可靠性
- 性能

二、微型计算机的出现和发展

Moore’s Law:

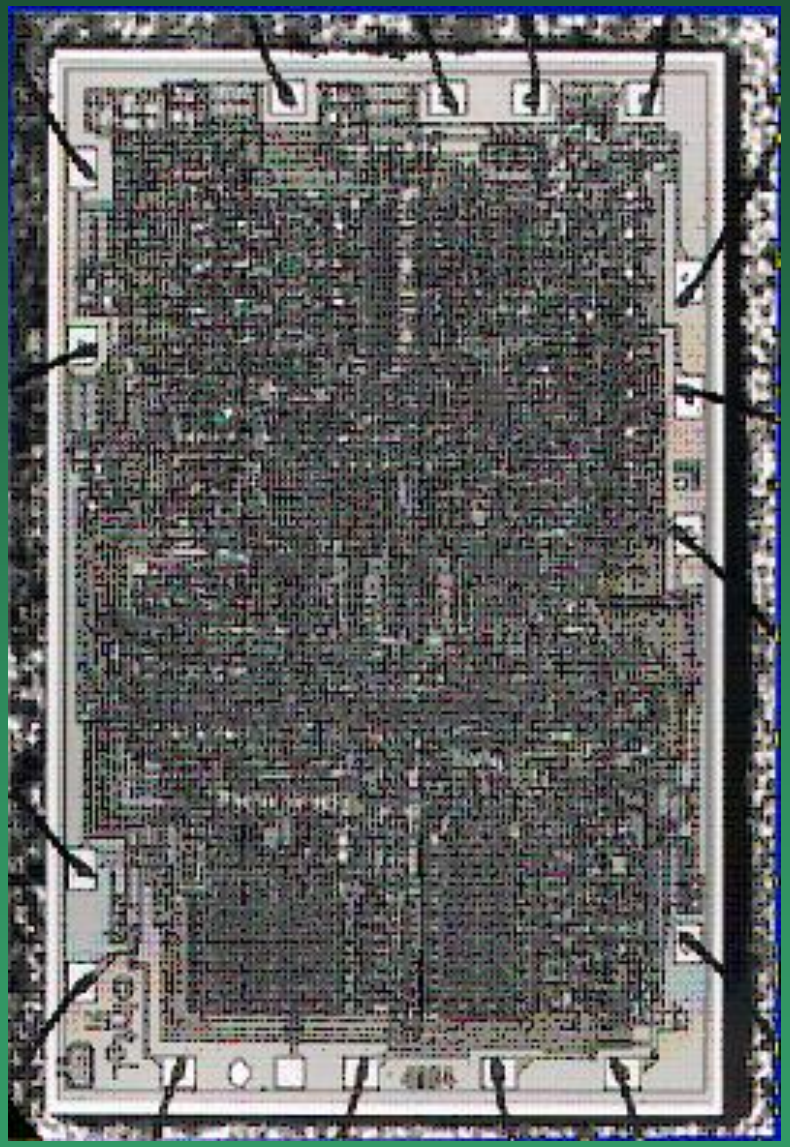
Intel 公司的缔造者之一
Gordon Moore 1965年发现

微芯片上集成的晶体管数目，每隔18-24个月翻一番



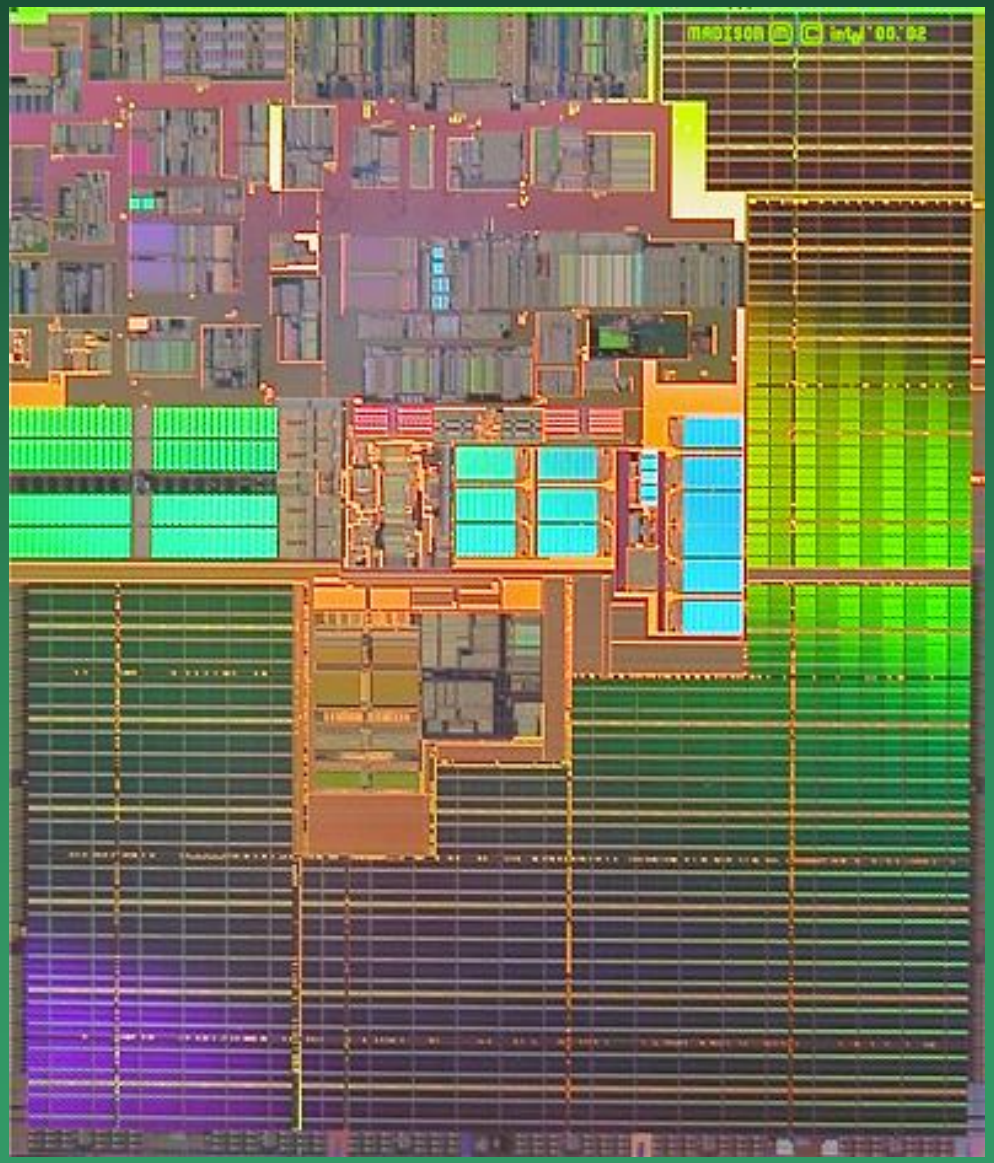
FIRST MICROPROCESSOR (4004 by Intel 1971)

(2300 Transistors 12 mm^2 8μm 750 kHz)



ITANIUM MICROPROCESSOR

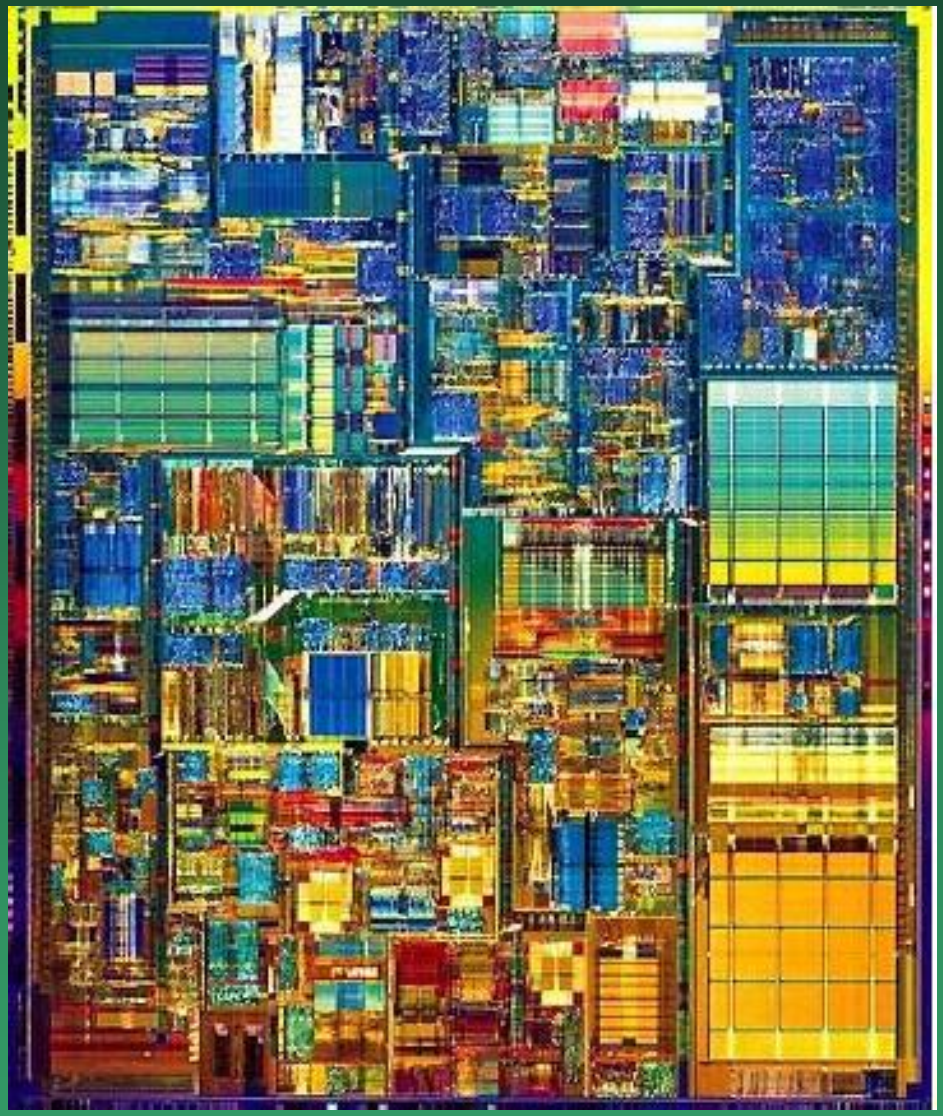
(410 Million Transistors 374 mm2 0.13µm 1.5 GHz)



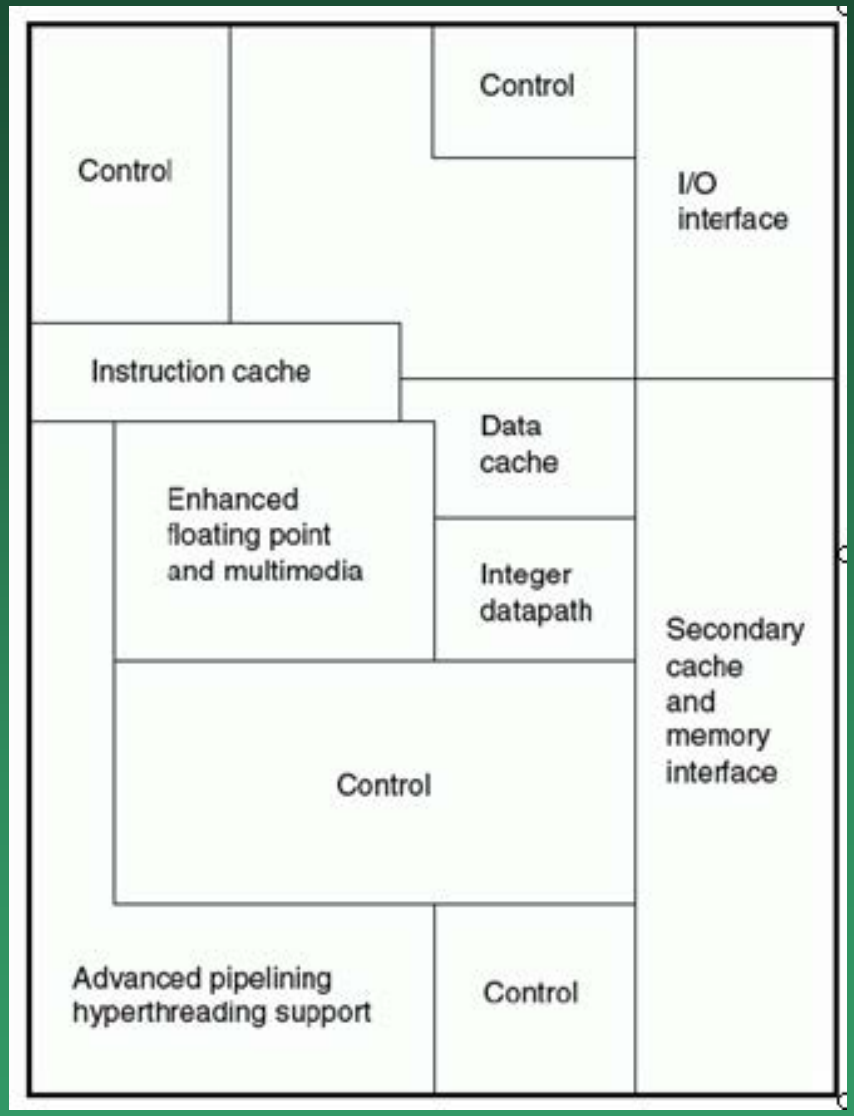
Intel 公司的典型微处理器产品

4040	4位	1971年	2300
8080	8位	1974年	
8086	16位	1979年	2.9 万
80286	16位	1982年	13.4 万
80386	32位	1985年	27.5 万
80486	32位	1989年	120.0 万
Pentium	64位（准）	1993年	310.0 万
Pentium pro	64位（准）	1995年	550.0 万
Pentium II	64位（准）	1997年	550.0 万以上
Pentium III	64位（准）	1999年	800.0 万以上
Pentium 4	64位	2000年	4 200.0 万

Intel 公司的典型微处理器产品



Pentium 4



Pentium 4内部布局

Loongson 3B

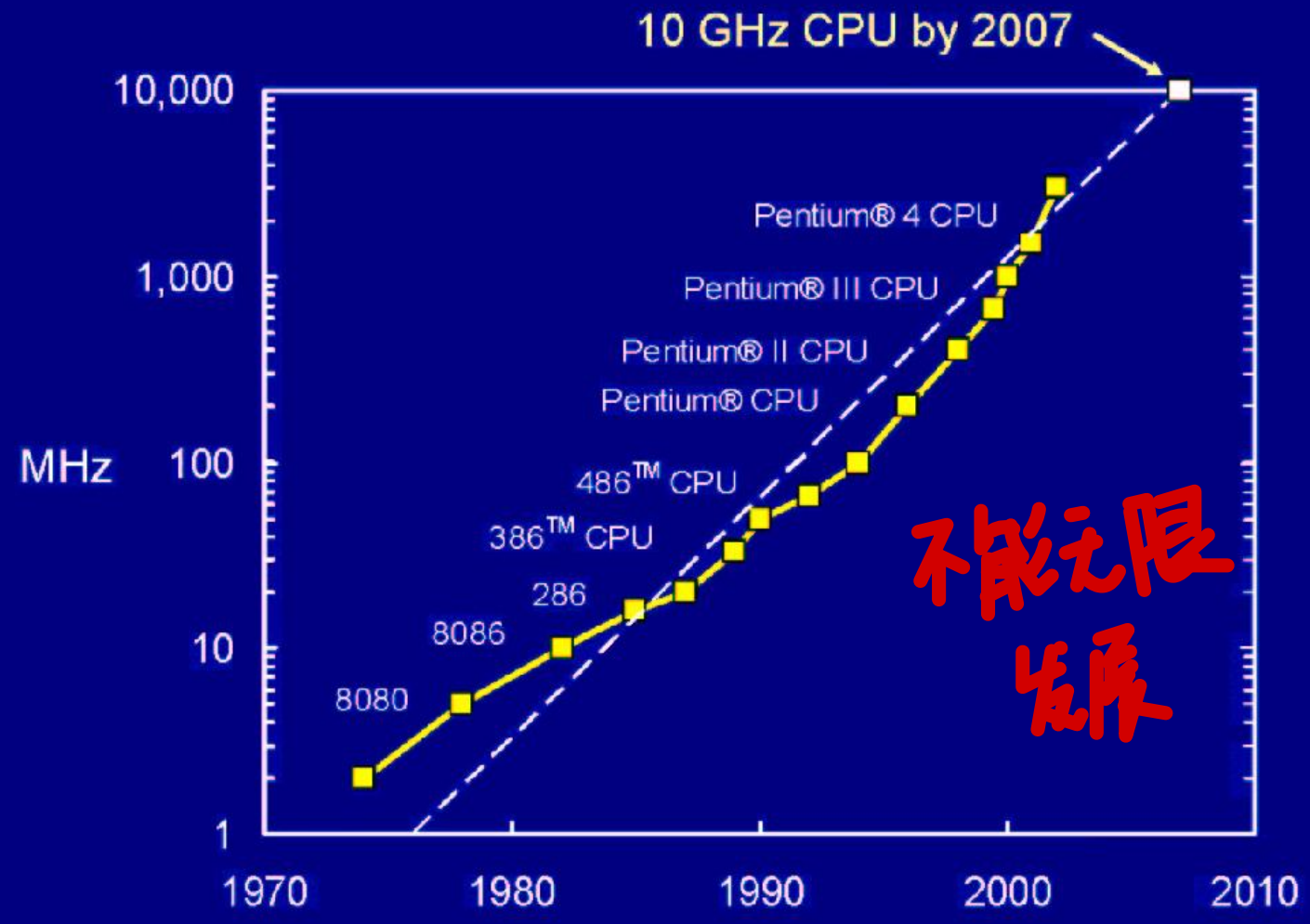
*曙光*公司此次推出的具有完全自主知识产权的国内首款基于“*龙芯*”3B处理器的L620-G15、L640-G15、*TC4600L*三款服务器产品和L300-G10桌面办公终端产品，均采用国产的“*龙芯*”3B处理器及国产操作系统，进一步巩固了*曙光*在信息安全自主可控战略的长期规划。

据介绍，“龙芯”3B处理器采用28纳米工艺制造，拥有11亿个晶体管，在设计的复杂度上与国际主流相近，这也是我国第一个超过10亿的晶体管的产品，其运算性能实现了大幅飞跃，国产龙芯处理器与市场主流产品之间的差距正在大幅缩短.

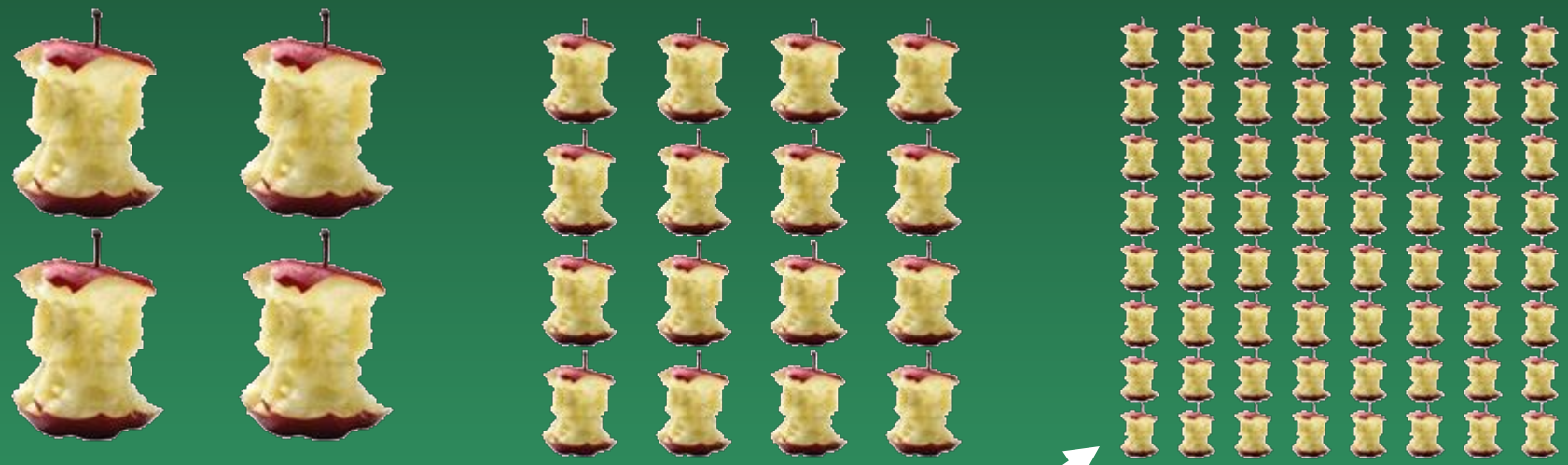
有别于其他可信计算服务器，L620R-S是曙光基于“龙芯”平台开发的一款高性能双路可信服务器，通过采用国产“龙芯”多核处理器和国产可信芯片，实现全自主安全可控的软硬件系统，具有独特的安全性，构建从硬件开始，到BIOS、操作系统、应用的信任链，为业务应用提供安全可信运行环境，全面实现计算系统的主动安全机制。

“之所以基于自主可控的平台搭建可信计算的平台，是因为我们并不能保证非自主可控的CPU一定是安全可靠的。事关国家信息安全，拥有核心技术和自主知识产权的国产处理器是当仁不让的第一选择。”有行业分析师表示，曙光此举实现了我国在信息安全领域真正的全自主可控。

CPU MHz TREND



处理器的发展将以增加芯片内的核数为主要方式



并行技术

如何有效使用？？

Courtesy: M. McLaren

50

David Patterson

"From my perspective, parallelism is the biggest challenge since high-level programming languages. It's the biggest thing in 50 years because industry is betting its future that parallel programming will be useful. "

ACM Queue Interview with John Hennessy and David Patterson, Jan 2007
http://www.acmqueue.com/modules.php?name=Content&pa=showpage&pid=445&page=1

ACM&IEEE院士，ACM前任主席，提出RISC, RAID, NOW等

三、软件技术的兴起和发展

1. 各种语言

机器语言　面向机器

汇编语言　面向机器

高级语言　面向问题

FORTRAN　科学计算和工程计算

PASCAL　结构化程序设计

C + +　面向对象

Java　适应网络环境

2. 系统软件

语言处理程序	汇编程序 编译程序 解释程序
操作系统	DOS UNIX Windows
服务性程序	装配 调试 诊断 排错
数据库管理系统	数据库和数据库管理软件
网络软件	

3. 软件发展的特点

(1) 开发周期长

(2) 制作成本高

(3) 检测软件产品质量的特殊性

软件是程序以及开发、使用和维护程序所需要的所有文档

2.2 计算机的应用

一、科学计算和数据处理

二、工业控制和实时控制

三、网络技术

四、CAD与虚拟现实

五、CAD/CAM/CIMS

六、多媒体技术

七、人工智能

一、科学计算和数据处理

- 科学计算（数值计算）

早期的计算机主要用于科学计算。目前，科学计算仍然是计算机应用的一个重要领域，如高能物理、工程设计、地震预测、气象预报、航天技术等。

在近代科学和工程技术中常常会遇到大量复杂的科学问题，因此，科学研究、工程技术的计算是计算机应用的一个基本方面，也是计算机最早应用的领域。科学计算的特点是计算公式复杂，计算量大和数值变化范围大，原始数据相应较少。这类问题只有具有高速运算和信息存储能力以及高精度的计算机系统才能完成。

一、科学计算和数据处理

- 数据处理（信息管理）

世界上的计算机80％以上主要用于数据处理。数据处理是对数值、文字、图表等信息数据及时地加以记录、整理、检索、分类、统计、综合和传递，得出人们所要求的有关信息。它是目前计算机最广泛的应用领域。

数据处理的特点是原始数据多，时间性强，计算公式相应比较简单。例如财贸、交通运输、石油勘探、电报电话、医疗卫生等方面的计划统计、财务管理、物资管理、人事管理、行政管理、项目管理、购销管理、情况分析、市场预测等工作。

二、工业控制和实时控制

- 过程控制

过程控制是指利用计算机进行生产过程、实时过程的控制，它要求很快的反应速度和很高的可靠性，以提高产量和质量，节约原料消耗，降低成本，达到过程的最优控制。过程控制的特点是要求实时性强，即计算机做出反应的时间必须与被控过程的实际时间相适应。因此，计算机广泛应用于石油化工、水电、冶金、机械加工、交通运输及其他国民经济部门中生产过程的控制以及导弹、火箭和航天飞船等的自动控制。

三、网络技术的应用

- 电子商务
- 网络教育
- 敏捷制造

敏捷制造，是指制造企业采用现代通信手段，通过快速配置各种资源(包括技术、管理和人员)，以有效和协调的方式响应用户需求，实现制造的敏捷性。

敏捷制造主要包括三个要素：生产技术、管理和人力资源。

案例：

https://wenku.baidu.com/view/ca3026af492fb4daa58da0116c175f0e7cd119a7.html

四、CAD与VR

- CAD，即计算机辅助设计(Computer Aided Design)，是利用计算机及其图形设备帮助设计人员进行设计工作。
- 虚拟现实技术（Virtual Reality, VR）是一种可以创建和体验虚拟世界的计算机仿真系统，它利用计算机生成一种模拟环境，使用户沉浸到该环境中。
- VR应用于影视、教育、设计、医学、军事等方面。

五、CAM与3D打印

- CAM，计算机辅助制造（Computer Aided Making）：是将计算机应用于制造生产过程的过程或系统。数控的特征是由程序指令来控制机床。
- 3D打印：即快速成型技术的一种，又称增材制造，它是一种以数字模型文件为基础，运用粉末状金属或塑料等可粘合材料，通过逐层打印的方式来构造物体的技术。

六、多媒体技术

- 多媒体技术是指通过计算机对文字、数据、图形、图像、动画、声音等多种媒体信息进行综合处理和管理，使用户可以通过多种感官与计算机进行实时信息交互的技术，又称为计算机多媒体技术。

七、人工智能

- 模式识别——文字/语音/图像
- 专家系统
- 机器人
- 吴恩达 Andrew Ng

——《机器学习》、《深度学习》

2.3 计算机的展望

一、计算机具有类似人脑的一些超级智能功能

二、芯片集成度的提高受以下三方面的限制

- 芯片集成度受物理极限的制约
- 按几何级数递增的制作成本
- 芯片的功耗、散热、线延迟

温度墙和功耗墙

三、替代传统的硅芯片

2.3

1. 光计算机

利用光子取代电子进行运算和存储

2. DNA生物计算机

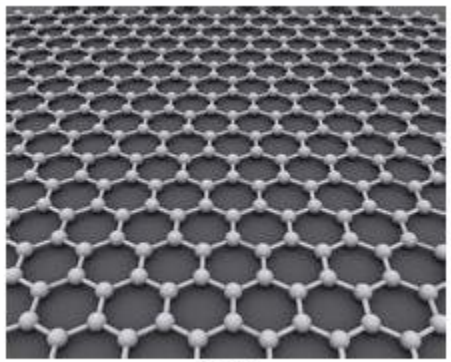
通过控制DNA分子间的生化反应

3. 量子计算机

利用原子所具有的量子特性

三、？替代传统的硅芯片

2.3

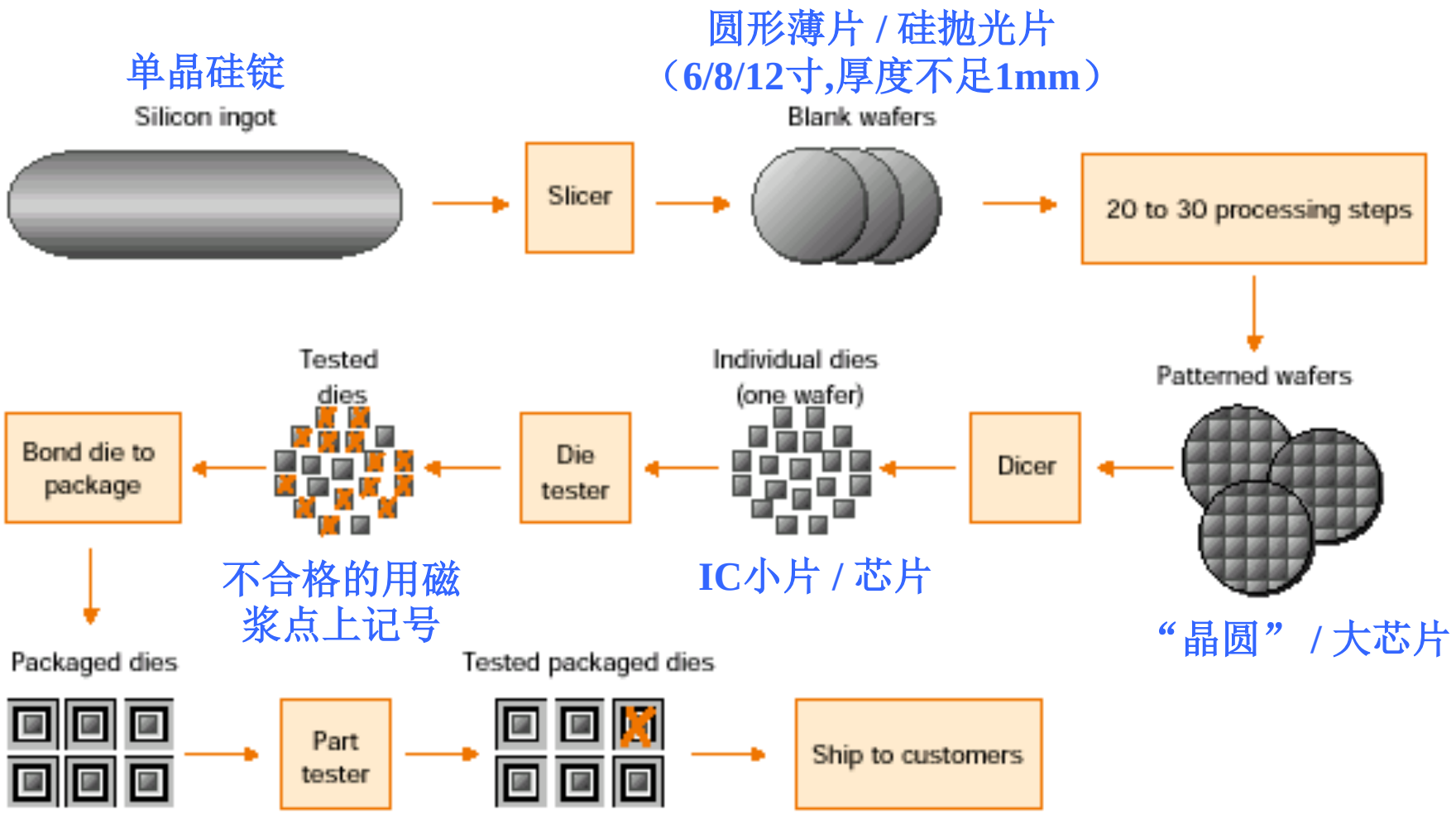

石墨烯

- 石墨烯不仅是已知材料中最薄的一种，还非常牢固坚硬；作为单质，它在室温下传递电子的速度比已知导体都快。

石墨烯（Graphene）是一种由碳原子构成的单层片状结构的新材料。是一种由碳原子以sp2杂化轨道组成六角型呈蜂巢晶格的平面薄膜，只有一个碳原子厚度的二维材料[1]。石墨烯一直被认为是假设性的结构，无法单独稳定存在[1]，直至2004年，英国曼彻斯特大学物理学家安德烈·海姆和康斯坦丁·诺沃肖洛夫，成功地在实验中从石墨中分离出石墨烯，而证实它可以单独存在，两人也因“在二维石墨烯材料的开创性实验”为由，共同获得2010年诺贝尔物理学奖[2]。

石墨烯目前是世上最薄却也是最坚硬的纳米材料[3]，它几乎是完全透明的，只吸收2.3%的光"[4]；导热系数高达5300 W/m·K，高于碳纳米管和金刚石，常温下其电子迁移率*超过15000 cm²/V·s，又比纳米碳管或硅晶体*高，而电阻率只约10-6 Ω·cm，比铜或银更低，为目前世上电阻率最小的材料[1]。因为它的电阻率极低，电子迁移的速度极快，因此被期待可用来发展出更薄、导电速度更快的新一代电子元件或晶体管。由于石墨烯实质上是一种透明、良好的导体，也适合用来制造透明触控屏幕、光板、甚至是太阳能电池。

Integrated Circuits manufacturing process



封装：将芯片固定在塑胶或陶瓷基座上，把芯片上蚀刻出来的引线与基座底部伸出的引脚连接，盖上盖板并封焊成芯片

约需400多道工序！

海阔凭鱼跃，
天高任鸟飞。

计算机组成

第六章　计算机的运算方法

合肥工业大学
计算机与信息学院
阙夏

Contents

6.1 无符号数和有符号数

6.2 数的定点表示和浮点表示

6.3 定点运算

6.4 浮点四则运算

6.5 算术逻辑单元

大　纲

(一)数制与编码

1. 进位计算制及其相互转换
2. 真值和机器数
3. BCD码
4. 字符与字符串
5. *校验码*

(二)定点数的表示和运算

1. 定点数的表示
   - 无符号数的表示
   - 有符号数的表示
2. 定点数的运输
   - 定点数的移位运算;
   - 原码定点数的加/减运算;
   - 补码定点数的加/减运算;
   - 定点数的乘/除运算;
   - 溢出概念和判别方法

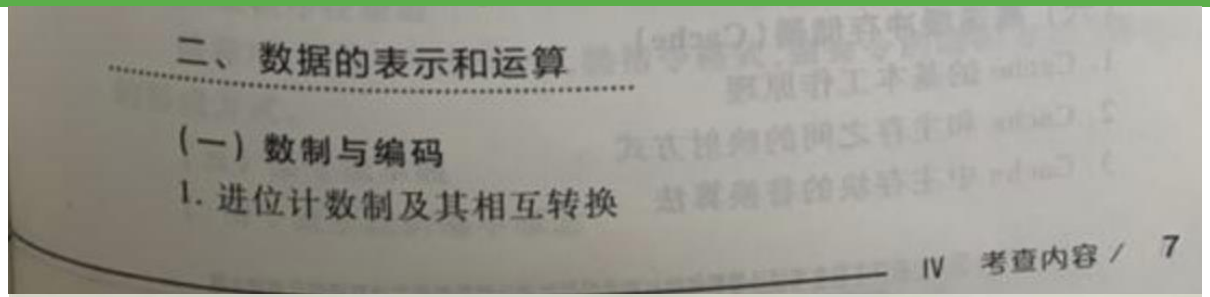
(三)浮点数的表示和运算

1. 浮点数的表示
   - IEEE754标准
2. 浮点数的加/减运算

(四)算术逻辑单元ALU

1. 串行加法器和并行加法器
2. 算术逻辑单元ALU的功能和结构

大　纲



二、数据的表示和运算

（一）数制与编码

1. 进位计数制及其相互转换

2. 真值和机器数

3. 字符与字符串

（二）定点数的表示和运算

1. 定点数的表示

无符号数的表示，带符号整数的表示。

2. 定点数的运算

定点数的移位运算，原码定点数的加/减运算，补码定点数的加/减运算，定点数的乘/除运算，溢出概念和判别方法。

（三）浮点数的表示和运算

1. 浮点数的表示

IEEE 754 标准。

2. 浮点数的加/减运算

（四）算术逻辑单元 ALU

1. 串行加法器和并行加法器

2. 算术逻辑单元 ALU 的功能和结构

数制问题

- **100+100=?**

- **生活中常见的进制有哪些?**

10进制，12/24进制，60进制，7进制，4进制，2进制……

数制问题

- **基（数）**：某种数制所使用的全部符号的个数。
- **位**：每个符号在数中的位置。
- **位权**：每个数位对应的单位值。

例如：8进制的所有符号集合：{0，1，2，3，4，5，6，7}

8进制的基（数）是8

8进制数：30675. 214

8进制数中第i数位的位权：8^i

数制问题

计算机中常用的4种进位数制

二进制：r=2，基本符号：01

八进制：r=8，基本符号：01234567

十进制：r=10，基本符号：0123456789

十六进制：r=16，基本符号：

0123456789ABCDEF

其中A～F表示十进制数10～15

4种进位数制之间的关系：二进制用于计算机内部，八和十六进制是二进制的缩写，十进制用于人员。

数制问题

- **R进制数 => 十进制数**

  按“权”展开 (a power of R)

  例1: $(10101.01)_2=1\times2^4+1\times 2^2+1\times2^0+1\times2^{-2}=(21.25)_{10}$

  例2: $(307.6)_8=3\times8^2+7\times8^0+6\times8^{-1}=(199.75)_{10}$

  例1: $(3A.\ 1)_{16}=3\times16^1+10\times16^0+1\times16^{-1}=(58.0625)_{10}$

- **十进制数 => R进制数**

  整数部分和小数部分分别转换

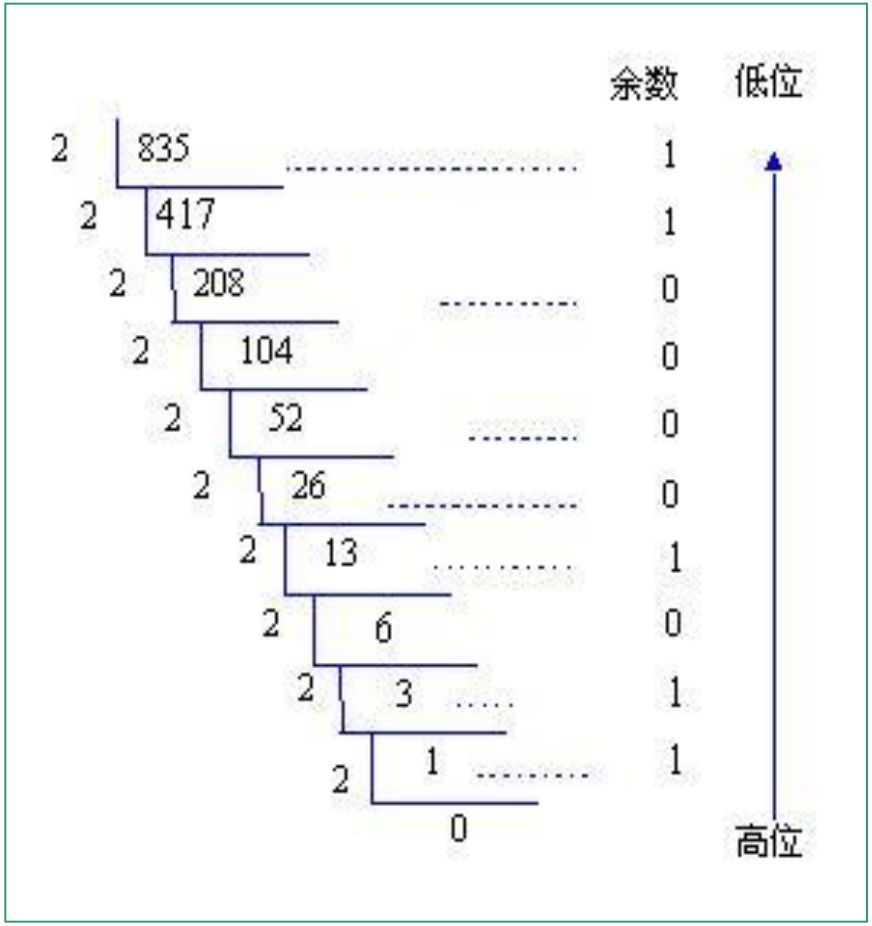
  ① 整数(integral part)----“除基取余，上右下左”

  ② 小数(fractional part)----“乘基取整，上左下右”

十进制向其他进制转换

例1:$(835.6785)_{10}=(1101000011.1011)_2$

整数----“除基取余，上右下左”



小数----“乘基取整，上左下右”

0.6875×2=1.375　整数部分=1　(高位)

0.375×2=0.75　整数部分=0　↓

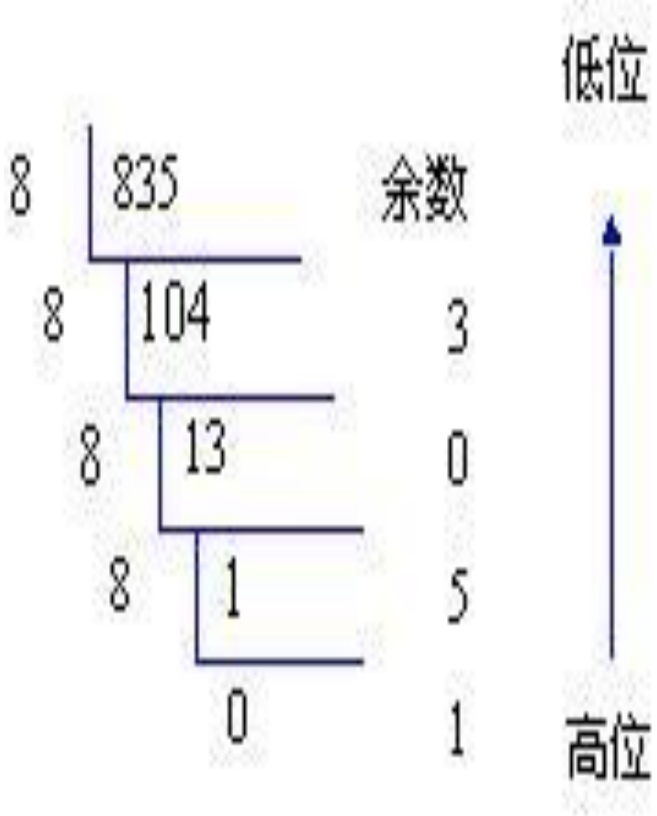
0.75×2=1.5　整数部分=1　↓

0.5×2=1.0　整数部分=1　(低位)

十进制向其他进制转换

例2:$(835.63)_{10}=(1503.50243...)_8$

整数----“除基取余，上右下左”



小数----“乘基取整，上左下右”

有可能乘积的小数部分总得不到0，此时得到一个近似值。

0.63×8=5.04 整数部分=5 (高位)

0.04×8=0.32 整数部分=0

0.32×8=2.56 整数部分=2

0.56×8=4.48 整数部分=4

0.48×8=3.84 整数部分=3 (低位)

其他进制的相互转换

(3) 二/八/十六进制数的相互转换

① 八进制数转换成二进制数

$(13.724)_8=(\ 001\ 011\ .\ 111\ \ 010\ \ 100\ )_2=(1011.1110101)_2$

② 十六进制数转换成二进制数

$(2B.5E)_{16} = (00101011\ .\ 01011110)_2 = (101011.0101111)_2$

③ 二进制数转换成八进制数

$(0.10101)_2 = (\ 000\ .\ 101\ \ 010\ )_2 = (\ 0.52)_8$

④ 二进制数转换成十六进制数

$(11001.11)_2 = (\ 0001\ \ 1001\ .\ 1100\ )_2 = (\ 19.C\ )_{16}$

其他进制的相互转换

任意进制之间的转换可以用**除基取余（乘基取整）法**。

比如，想把M进制转换成K进制，可以用输入数据连续去除K，直到商为零为止，然后把每次所得的余数倒看成一个数，就是相应的K进制数。

单选题 0.5分

设置

十进制数60转化为二进制结果为:（　　）

A 0111000

B 0111110

C 0111100

D 0111101

提交

编码

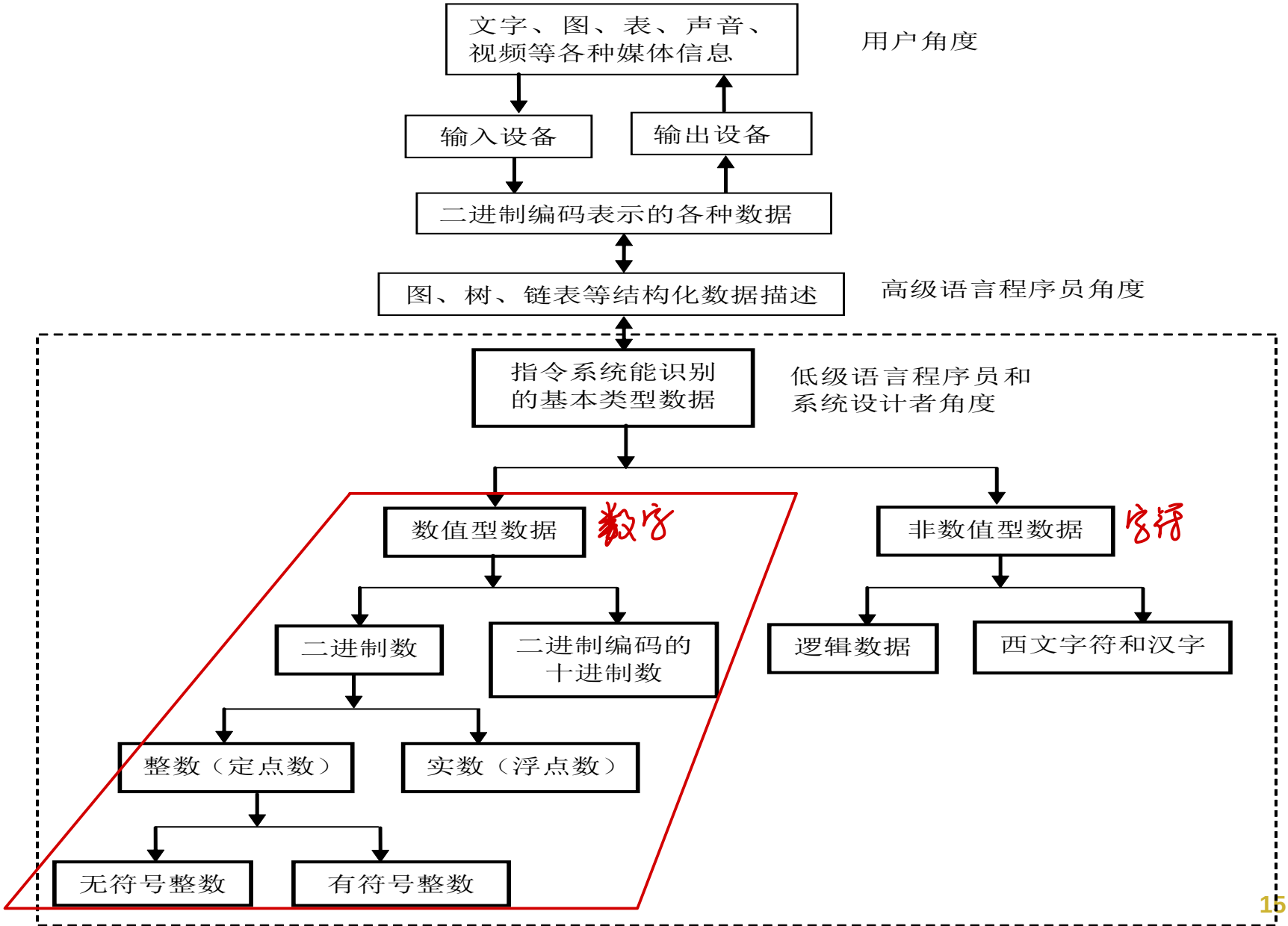
编码：就是按指定的方法，将信息从一种形式（格式），转换成另一种形式（格式）。

为什么计算机要编码?

编码是计算机交流的语言。

当人与计算机之间交流时，因为人和计算机都各自有自己的语言，彼此不懂对方，就需要一套规则，来约定彼此之间的交流，编码技术就是这样产生的。

计算机的外部信息与内部机器级数据



常见的编码方式

有以下几大类：

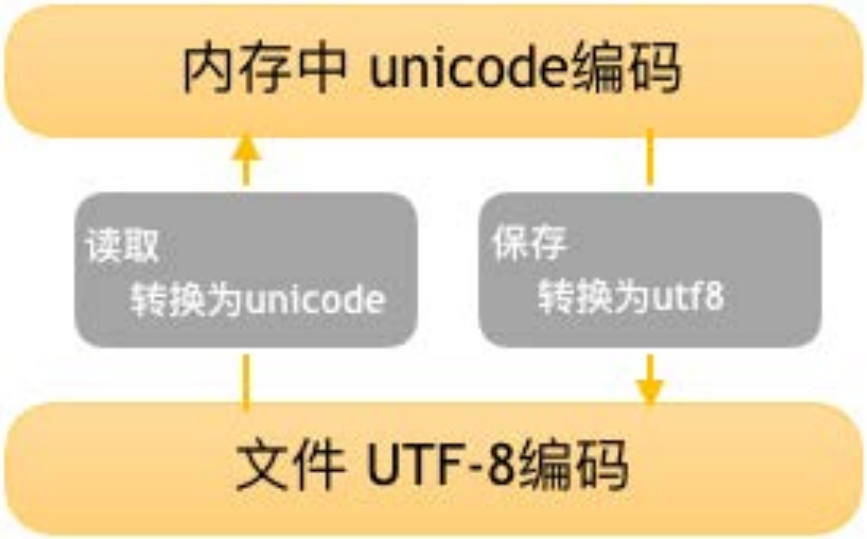
- 数字编码：原、补、反、BCD码，汉明码等；
- 字符编码：ASCII（128个字符）、Unicode、UTF-8/16、GBK、GB2312等；
- 图片编码：bmp，jpg，png等
- 视频编码：MPEG系列、H.26X系列等；
- 音频编码： PCM编码的WAV、 MP3、OGG、MPC、WMA等

不管是数字，字符还是图片或视频，在计算机存储上都是一视同仁，全都是**字节流**。0/1

常见的编码方式

现在计算机系统通用的字符编码工作方式：

在计算机内存中，统一使用Unicode编码，当需要保存到硬盘或者需要传输的时候，就转换为UTF-8编码。

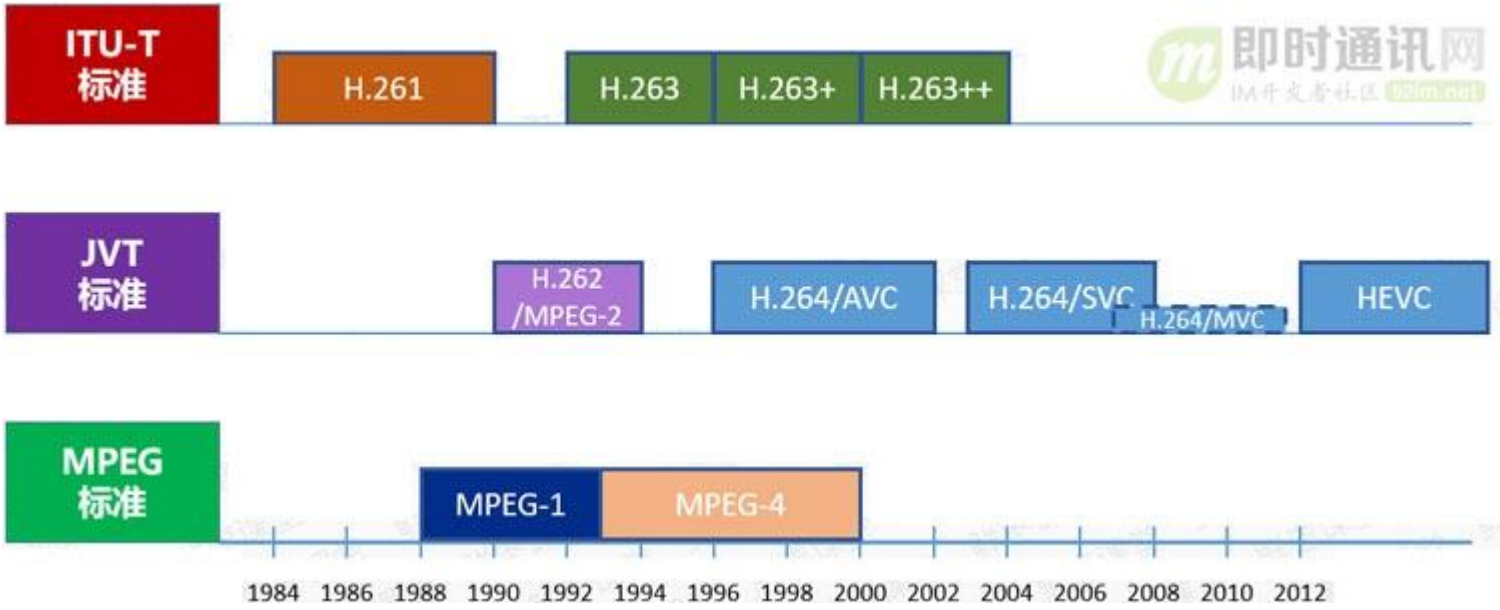


常见的编码方式

常用编码介绍一览表

编码	制定时间	作用	所占字节数
ASCII	1967年	表示英语及西欧语言	8bit/1bytes
GB2312	1980年	国家简体中文字符集，兼容ASCII	2bytes
Unicode	1991年	国际标准组织统一标准字符集	2bytes
GBK	1995年	GB2312的扩展字符集，支持繁体字，兼容GB2312	2bytes
UTF-8	1992年	不定长编码	1-3bytes

https://blog.csdn.net/qq_34745204



即时通讯网

BCD （Binary-Coded Decimal）码

- **BCD码**:用4位二进制代码的不同组合来表示一个十进制数码的编码方法。

- **编码思想**： 每个十进数位至少有4位二进制表示。而4位二进制位可组合成16种状态，去掉10种状态后还有6种冗余状态。

用BCD码表示十进制数

十进制数	8421码	5421码	2421码	余3码	余3循环码
0	0000	0000	0000	0011	0010
1	0001	0001	0001	0100	0110
2	0010	0010	0010	0101	0111
3	0011	0011	0011	0110	0101
4	0100	0100	0100	0111	0100
5	0101	1000	1011	1000	1100
6	0110	1001	1100	1001	1101
7	0111	1010	1101	1010	1111
8	1000	1011	1110	1011	1110
9	1001	1100	1111	1100	1010

用BCD码表示十进制数

- **编码方案**

  1. 十进制有权码
  2. 十进制无权码
  3. 其他编码方案：如独热编码（One-Hot Encoding）等

- **符号位的表示：**
  - “+”：1100；“-”：1101
  - 例：+236=(1100 0010 0011 0110)$_{8421}$ (占2个字节)

    - 2369=(1101 0000 0010 0011 0110 1001)$_{8421}$ (占3个字节)

补0以使数占满一个字节

西文字符的编码表示*

- 特点
  - 是一种拼音文字，用有限几个字母可拼写出所有单词
  - 只对有限个字母和数学符号、标点符号等辅助字符编码
  - 所有字符总数不超过256个，使用7或8个二进位可表示
- 表示（常用编码为7位ASCII码）

  要求必须熟悉数字、字母和空格(SP)的表示
  - 十进制数字：0/1/2…/9
  - 英文字母：A/B/…/Z/a/b/…/z
  - 专用符号：+/-/%/*/&/……
  - 控制字符（不可打印或显示）
- 操作
  - 字符串操作，如:传送/比较　等

字符串的表示与存储*

字符串是指连续的一串字符，它们占据主存中连续的多个字节，每个字节存放一个字符，对一个主存字的多个字节，有按从低位到高位字节次序存放的，也有按从高位到低位字节次序存放的。表示字符串数据要给出串存放的主存起始地址和串的长度。如：

IF A>B THEN READ(C) 就可以有如下不同存放方式：

I	F		A
>	B		T
H	E	N	
R	E	A	D
(	C	)	

A		F	I
T		B	>
	N	E	H
D	A	E	R
	)	C	(

假定每个字
由 4 个字节组成

字符串的表示与存储*

按从低位到高位字节次序存放

I	F		A
>	B		T
H	E	N	
R	E	A	D
(	C	)	

31 24 23 16 15 8 7 0

49	46	20	41
3e	42	20	54
48	45	4e	20
52	45	41	44
28	43	29	20

16进制数据

汉字及国际字符的编码表示*

- 特点
  - 汉字是表意文字，一个字就是一个方块图形。
  - 汉字数量巨大，总数超过6万字，给汉字在计算机内部的表示、汉字的传输与交换、汉字的输入和输出等带来了一系列问题。
- 编码形式
  - 有以下几种汉字代码：

    输入码：对汉字用相应按键进行编码表示，用于输入

    内码：用于在系统中进行存储、查找、传送等处理

    字模点阵或轮廓描述：描述汉字字模点阵或轮廓，用于显示/打印

汉字的输入码*

向计算机输入汉字的方式：

① 手写汉字联机识别输入，或者是印刷汉字扫描输入后自动识别，这两种方法现均已达到实用水平。

② 用语音输入汉字，虽然简单易操作，但离实用阶段还有距离。

③ 利用英文键盘输入汉字：每个汉字用一个或几个键表示，这种对每个汉字用相应按键进行的编码称为汉字“输入码”，又称外码。输入码的码元为按键。是最简便、最广泛的汉字输入方法。

常用的方法有：搜狗拼音、五笔字型、智能ABC、微软拼音等

使用汉字输入码的原因：

① 键盘面向西文设计，一个或两个西文字符对应一个按键，非常方便。

② 汉字是大字符集，专门的汉字输入键盘由于键多、查找不便、成本高等原因而几乎无法采用。

字符集与汉字的内码*

问题：西文字符常用的内码是什么？其内码就是ASCII码。

对于汉字内码的选择，必须考虑以下几个因素：

① 不能有二义性，即不能和ASCII码有相同的编码。

② 尽量与汉字在字库中的位置有关，便于汉字查找和处理。

③ 编码应尽量短。

国标码（国标交换码）

1981年我国颁布了《信息交换用汉字编码字符集·基本集》(GB2312—80)。该标准选出6763个常用汉字，为每个汉字规定了标准代码，以供汉字信息在不同计算机系统间交换使用

可在汉字国标码的基础上产生汉字机内码

GB2312-80字符集*

- 由三部分组成：
  - ① 字母、数字和各种符号，包括英文、俄文、日文平假名与片假名、罗马字母、汉语拼音等共687个
  - ② 一级常用汉字，共3755个，按汉语拼音排列
  - ③ 二级常用汉字，共3008个 ，不太常用，按偏旁部首排列
- 汉字的区位码
  - 码表由94行、94列组成，行号为区号，列号为位号，各占7位
  - 指出汉字在码表中的位置，共14位，区号在左、位号在右
- 汉字的国标码
  - 每个汉字的区号和位号各自加上32（20H），得到其“国标码”
  - 国标码中区号和位号各占7位。在计算机内部，为方便处理与存储，前面添一个0，构成一个字节

汉字内码*

- 至少需2个字节才能表示一个汉字内码。
  - 由汉字的总数决定!
- 可在GB2312国标码的基础上产生汉字内码
  - 为与ASCII码区别，将国标码的两个字节的第一位置“1”后得到一种汉字内码

例如，汉字“大”在码表中位于第20行、第83列。因此区位码为0010100 1010011，国标码为00110100 01110011，即3473H。前面的34H和字符“4”的ACSII码相同，后面的73H和字符“s”的ACSII码相同，将每个字节的最高位各设为“1”后，就得到其内码：B4F3H (1011 0100 1111 0011B)，因而不会和ASCII码混淆。

国际字符集*

国际字符集的必要性

- 不同地区使用不同字符集内码，如中文GB2312 / Big5、日文Shift-JIS / EUC-JP等。在安装中文系统的计算机中打开日文文件，会出现乱码。
- 为使所有国际字符都能互换，必须创建一种涵盖全部字符的多字符集。

国际多字符集

- 通过对各种地区性字符集规定使用范围来唯一定义各字符的编码。
- 国际标准ISO/IEC 10646提出了一种包括全世界现代书面语言文字所使用的所有字符的标准编码，有4个字节编码(UCS-4)和2字节编码(UCS-2) 。
- 我国（包括香港、台湾地区）与日本、韩国联合制订了一个统一的汉字字符集（CJK编码），共收集了上述不同国家和地区共约2万多汉字及符号，采用2字节编码 (即：UCS-2)，已被批准为国家标准(GB13000)。
- Windows操作系统(中文版)已采用中西文统一编码，收集了中、日、韩三国常用的约2万汉字，称为“Unicode”，采用2字节编码，与UCS-2一致 。

UNICODE编码*

Unicode是完全双字节表示的多国文字编码体系，编码空间 0x0000-0xFFFF。

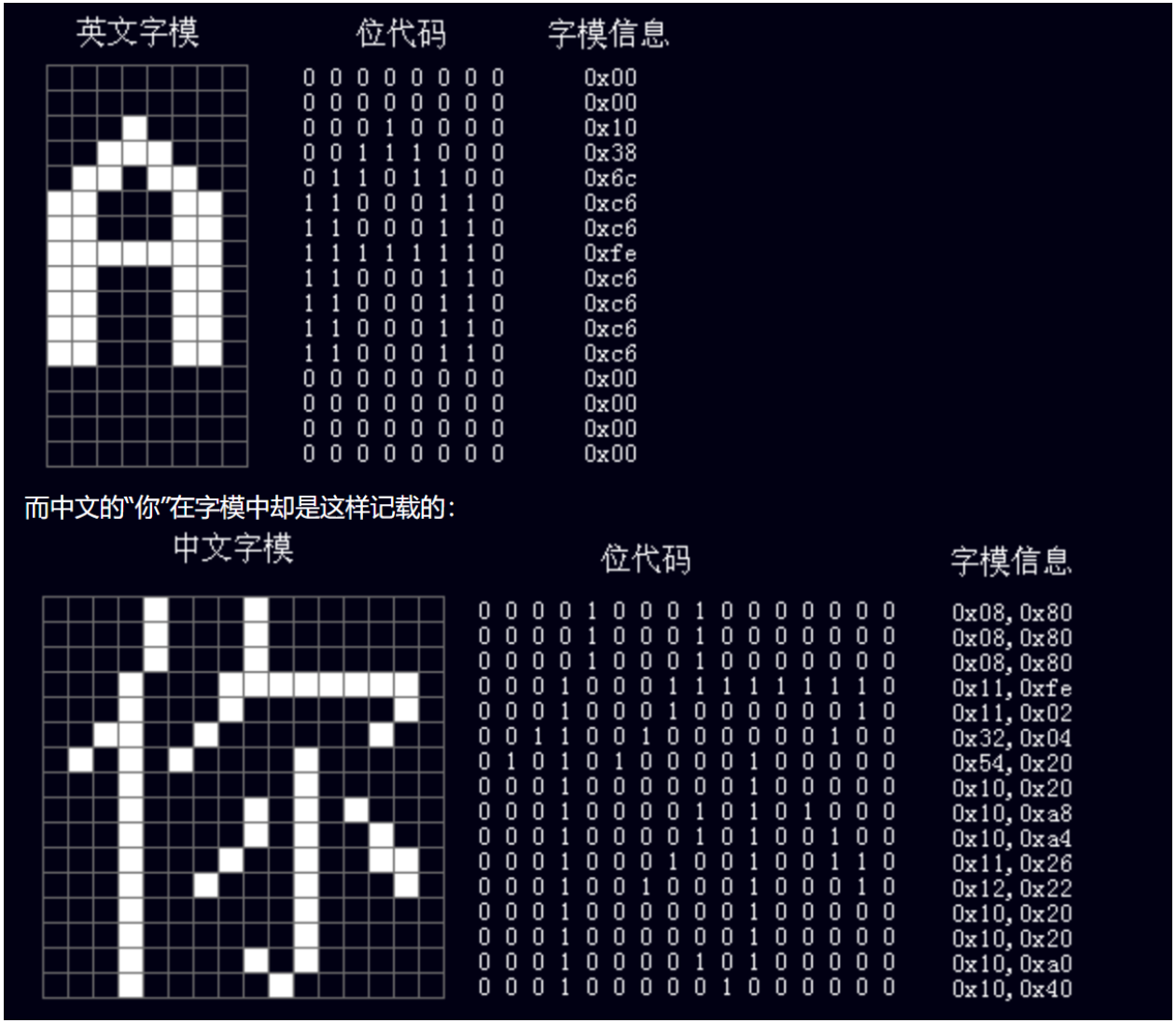
可以表示 65536 个字符。

汉字的字模点阵码和轮廓描述*

- 为便于打印、显示汉字，汉字字形必须预先存在机内
  - 字库 (font)：所有汉字形状的描述信息集合
  - 不同字体 (如宋体、仿宋、楷体、黑体等) 对应不同字库
  - 从字库中找到字形描述信息，然后送设备输出
- 字形主要有两种描述方法：
  - 字模点阵描述（图像方式）
  - 轮廓描述（图形方式）
    - 直线向量轮廓
    - 曲线轮廓（True Type字形）

字模点阵描述*

- 字模编码
  - 字模编码是以点阵方式用来描述汉字字形的代码，它是汉字的输出形式。



数据的基本宽度

- 比特（bit）是计算机中处理、存储、传输信息的最小单位
- 二进制信息的计量单位是“字节”(Byte)，也称“位组”
  - 现代计算机中，存储器按字节编址
  - 字节是最小可寻址单位 *(addressable* unit *)*

  一般8位 基本单位
- 除比特和字节外，还经常使用“字”(word)作为单位
- “字”和 “字长”的概念不同

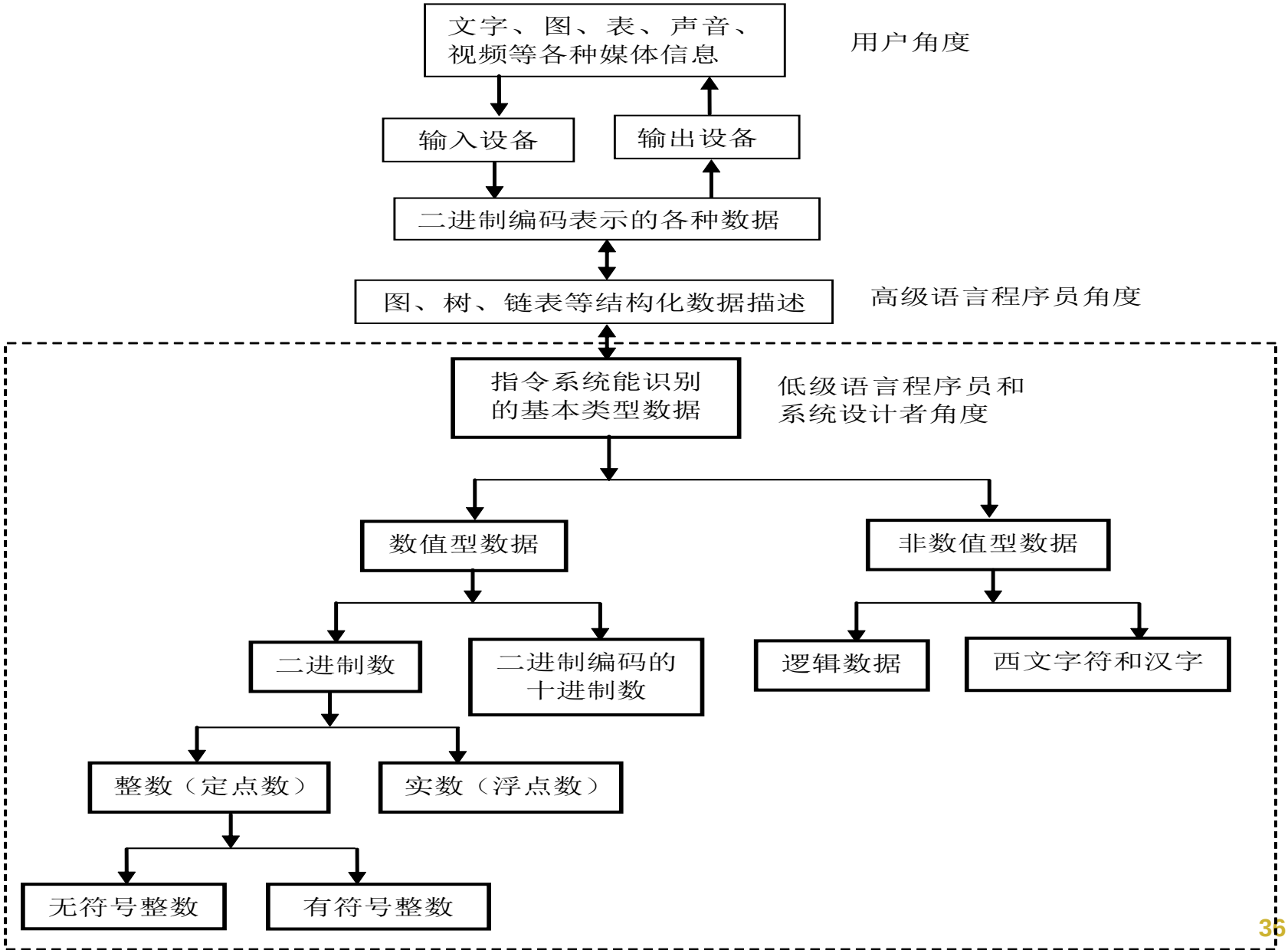
数据的基本宽度

- “字”和“字长”的概念不同
  - “字长”指数据通路的宽度。

    (数据通路指CPU内部数据流经的路径以及路径上的部件，主要是CPU内部进行数据运算、存储和传送的部件，这些部件的宽度基本上要一致，才能相互匹配。因此，“字长”等于CPU内部总线的宽度、运算器的位数、通用寄存器的宽度等。)
  - “字”表示被处理信息的单位，用来度量数据类型的宽度。
  - 字和字长的宽度可以一样，也可不同。

    例如，x86体系结构定义“字”的宽度为16位，但从386开始字长就是32位了。

计算机的外部信息与内部机器级数据



信息的二进制编码

- **机器级数据分两大类：**
  - 数值数据：无符号整数、带符号整数、浮点数（实数）、十进制数
  - 非数值数据：逻辑数（包括位串）、西文字符和汉字
- **计算机内部所有信息都用二进制（即：0和1）进行编码**
- **用二进制编码的原因：**
  - 技术上容易实现，用双稳态电路表示二进制数字0和1是很容易的事情
  - 可靠性高，用二进制表示数据抗干扰能力强
  - 二进制编码、计数、运算规则简单
  - 二进制只有两个数码，正好与逻辑代数中的“真”和“假”相吻合。
  - 二进制数与十进制数之间的转换相当容易

设计上最简单，工程上最可靠，商业上最便宜

数值数据的表示

- **数值数据表示的三要素**
  - 进位计数制
  - 定、浮点表示
  - 如何用二进制编码

  即：要确定一个数值数据的值必须先确定这三个要素。

  例如，机器数 01011001的值是多少？ 答案是：不知道！
- **进位计数制**
  - 十进制、二进制、十六进制、八进制数及其相互转换
- **定/浮点表示（解决小数点问题）**
  - 定点整数、定点小数
  - 浮点数（可用一个定点小数和一个定点整数来表示）
- **定点数的编码（解决正负号问题）**
  - 原码、补码、反码、移码 （反码很少用）

逻辑数据的编码表示

- 表示
  - 用一位表示 真：1 / 假：0
  - N位二进制数可表示N个逻辑数据，或一个位串
- 运算
  - 按位进行
  - 如:按位与 / 按位或 / 逻辑左移 / 逻辑右移 等
- 识别
  - 逻辑数据和数值数据在形式上并无差别，也是一串0/1序列，机器靠指令来识别。

史上导致重大损失的10大计算机漏洞

https://www.freebuf.com/vuls/124449.html

为什么要研究机器内的数据表示

为什么研究机器内的数据表示

1)目的：组织数据,方便计算机硬件直接使用。

2)要考虑的因素

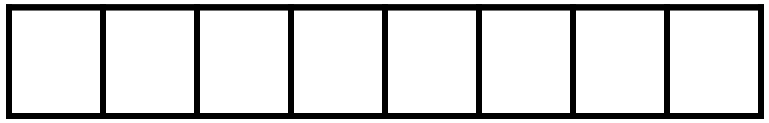
- 支持的数据类型；
- 能表示的数据范围；
- 能表示的数据精度；
- 存储和处理的代价；
- 是否有利于软件的移植等...

6.1 无符号数和有符号数

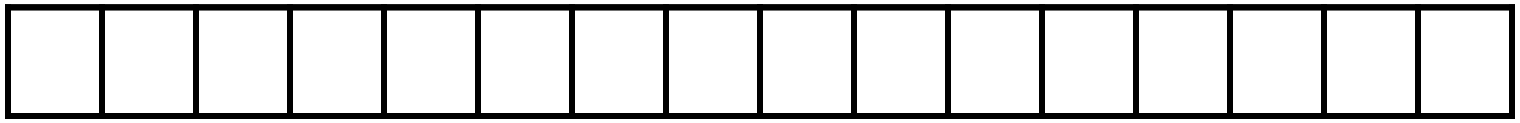
一、无符号数

寄存器的位数（机器字长）

反映无符号数的表示范围



8 位　　0 ~ 255



16 位　　0 ~ 65535

$2^{16}-1$

二、有符号数

1. 机器数与真值

真值	机器数
带符号的数	符号数字化的数
+ 0.1011	0.1011
– 0.1011	1.1011
+ 1100	01100
– 1100	11100

约定，并未存储

小数点的位置

小数点的位置

小数点的位置

小数点的位置

2. 原码表示法

(1) 定义

整数

$$[x]_{原}=\begin{cases}0,\ x & 0\le x<2^n\\ 2^n-x & -2^n<x\le 0\end{cases}$$

x 为真值　　n 为整数的位数

已知

如　$x=+1110$　　$[x]_{原}=0,\ 1110$

符号位

这里的逗号 是
人为加上便于阅读

$x=-1110$　　$[x]_{原}=2^4+1110=1,\ 1110$

符号位为1

带符号的绝对值表示

2. 原码表示法

小数

$$[x]_{原}=\begin{cases} x & 1>x\geq 0 \\ 1-x & 0\geq x>-1 \end{cases}$$

x 为真值

如

$x=+0.1101$　　$[x]_{原}=0.1101$

这里的小数点是人为加上便于阅读

$x=-0.1101$　　$[x]_{原}=1-(-0.1101)=1.1101$

$x=+0.1000000$　　$[x]_{原}=0.1000000$

这里的小数点是人为加上便于阅读

$x=-0.1000000$　　$[x]_{原}=1-(-0.1000000)=1.1000000$

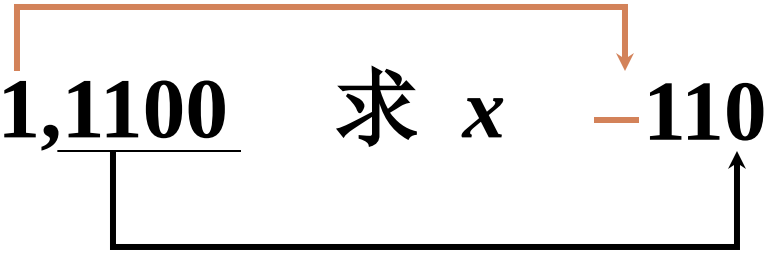
(2) 举例

例 6.1　已知 $[x]_{原} = 1.0011$　　求 x $-$ 0.0011

解：由定义得

$$x = 1-[x]_{原} = 1-1.0011 = -0.0011$$

例 6.2　已知 $[x]_{原} = 1,1100$　　求 x -1100



解：由定义得

$$x = 2^4-[x]_{原} = 10000-1,1100 = -1100$$

(2) 举例

例 6.3　已知 $[x]_{原} = 0.1101$　　求 x

解：由定义　$\because [x]_{原} = 0.1101$

$\therefore x = +0.1101$

例 6.4　求$x = 0$ 的原码（数值长度4位）。

解：　设 $x = +0.0000$　　$[+0.0000]_{原} = 0.0000$

$x = -0.0000$　　$[-0.0000]_{原} = 1.0000$

同理对于整数　　$[+0]_{原} = 0,0000$

$[-0]_{原} = 1,0000$

正零负零不一样

$\therefore [+0]_{原} \neq [-0]_{原}$

原码的特点：　简单、直观

但是用原码作加法时，会出现如下问题：

要求	数1	数2	实际操作	结果符号
加法	正	正	加	正
加法	正	负	减	可正可负
加法	负	正	减	可正可负
加法	负	负	加	负

能否 只作加法？

找到一个与负数等价的正数 来代替这个负数

就可使 减 ——→ 加　补

3. 补码表示法

(1) 补的概念

- 时钟

逆时针

$$\begin{array}{r} 11 \\ -\ 8 \\ \hline 3 \end{array}$$

顺时针

$$\begin{array}{r} 11 \\ +\ 4 \\ \hline 15 \\ -\ 12 \\ \hline 3 \end{array}$$

可见 -8 可用 $+4$ 代替 减法 ⟶ 加法

称 $+4$ 是 -8 以 12 为模的补数

记作 $-8 \equiv +4 \pmod{12}$

同理 $-3 \equiv +9 \pmod{12}$

$-5 \equiv +7 \pmod{12}$

时钟以12为模

结论

- 一个负数加上“模”即得该负数的补数
- 两个互为补数的数 它们绝对值之和即为模数
  - 计数器（模 16） 1011⟶0000 ？

11

$$\begin{array}{r} 1011 \\ -\ 1011 \\ \hline 0000 \end{array} \qquad \begin{array}{r} 1011 \\ +\ 0101 \\ \hline 10000 \end{array}$$

自然去掉

可见 −1011 可用 + 0101 代替

记作 $-1011 \equiv +0101 \pmod{2^4}$

同理 $-011 \equiv +101 \pmod{2^3}$

$-0.1001 \equiv +1.0111 \pmod{2}$

(2) 正数的补数即为其本身

两个互为补数的数 $-1011 \equiv +0101 \pmod{2^4}$

分别加上模 $+10000 \qquad +10000$

结果仍互为补数 $+0101 \equiv +10101$

丢掉

$\therefore +0101 \equiv +0101 \pmod{2^4}$

可见 $+0101 \rightarrow +0101$

? -1011

? 0,0101 $\rightarrow +0101$

? 1,0101 $\rightarrow -1011$

$2^{4+1} - 1011 = 100000$

$\pmod{2^{4+1}}$

-1011

1,0101

用 逗号 将符号位和数值位隔开

(3) 补码定义

整数

$$[x]_{补} = \begin{cases} 0, \ x & 2^n > x \geq 0 \\ 2^{n+1} + x & 0 > x \geq -2^n \ (\mathrm{mod}\ 2^{n+1}) \end{cases}$$

x 为真值　　n 为整数的位数

如　　$x = +1010$　　$x = -1011000$

$[x]_{补} = 0,1010$

用 逗号 将符号位和数值位隔开

$$[x]_{补} = 2^{7+1} + (-1011000)$$
$$= 100000000$$
$$-1011000$$
$$1,0101000$$

(3) 补码定义

小数

$$[x]_{补} = \begin{cases} x & 1 > x \geq 0 \\ 2 + x & 0 > x \geqslant -1 \ (\mathrm{mod}\ 2) \end{cases}$$

x 为真值

如 $x = +0.1110$ $x = -0.1100000$

$[x]_{补} = 0.1110$

$[x]_{补} = 2+(-0.1100000)$

$= 10.0000000$

$-\ 0.1100000$

1.0100000

用 小数点 将符号位和数值位隔开

(4) 求补码的快捷方式

设 $x = -1010$ 时

则 $[x]_{补} = 2^{4+1} - 1010$

$= 100000 - 1010$

$= 1,0110$

$= 11111 + 1 - 1010$

$= 11111 + 1 - 1010$

$= 10101 + 1$

$= 1,0110$

又 $[x]_{原} = 1,1010$

当真值为 负 时，补码 可用 原码除符号位外每位取反，末位加 1 求得

(5) 举例

例 6.5　已知 $[x]_{补} = 0.0001$

求 x

解：由定义得　$x = +0.0001$

例 6.6　已知 $[x]_{补} = 1.0001$

求 x

解：由定义得

$x = [x]_{补} - 2$

$= 1.0001 - 10.0000$

$= -0.1111$

$[x]_{补} \xrightarrow{?} [x]_{原}$

$[x]_{原} = 1.1111$

$\therefore x = -0.1111$

例6.7 已知 $[x]_{补} = 1,1110$

求 x

解：由定义得

$$x = [x]_{补} - 2^{4+1}$$

$$= 1,1110 - 100000$$

$$= -0010$$

$$[x]_{补} \xrightarrow{?} [x]_{原}$$

$$[x]_{原} = 1,0010$$

$$\therefore x = -0010$$

当真值为 负 时，原码 可用 补码除符号位外

每位取反，末位加 1 求得

练习　求下列真值的补码

真值		$[x]_{补}$	$[x]_{原}$
$x = +70 =$	1000110	0, 1000110	0,1000110
$x = -70 = -$	1000110	1, 0111010	1,1000110
$x =$	0.1110	0.1110	0.1110
$x = -$	0.1110	1.0010	1.1110
$x =$	0.0000	0.0000	0.0000
$x = -$	0.0000	0.0000	1.0000
$x = -$	1.0000	1.0000	不能表示

$[+0]_{补} = [-0]_{补}$

由小数补码定义

$$[x]_{补} = \begin{cases} x & 1 > x \geq 0 \\ 2 + x & 0 > x \geq -1 \ (\text{mod } 2) \end{cases}$$

$[-1.0000]_{补} = 2 + x = 10.0000 - 1.0000 = 1.0000$

求特殊数的补码（要求背诵）

假定机器数有n位（不含符号位）

① $[-2^n]_{补} = 2^{n+1} - 2^n = 10\ldots0(n个0) \pmod{2^{n+1}}$ 整数负边界

② $[-1]_{补} = 2^{n+1} - 0\ldots01(n-1个0) = 11\ldots1(n个1) \pmod{2^{n+1}}$ 整数补码

③ $[-1.0\ldots0]_{补}(n个0) = 2 - 1.0\ldots0 = 1.0\ldots0(n个0) \pmod{2}$ 小数补码

④ $[+0]_{补} = [-0]_{补} = 00\ldots0(n个0)$

补码总结

- 补码最高一位是符号位，符号 0 正 1 负
- 小数补码表示为：2×符号位 + 数的真值
- 零的补码只有一个，故补码能表示 -1
- 补码能很好地用于加减运算，运算时，符号位与数值位一样参加运算。

4. 反码表示法

(1) 定义

整数

$$[x]_{反}=\begin{cases}0,\ x & 2^n > x \geq 0\\ (2^{n+1}-1)+x & 0 \geq x > -2^n \ (\mathrm{mod}\ 2^{n+1}-1)\end{cases}$$

x 为真值　　n 为整数的位数

如　　$x = +1101$　　$x = -1101$

$[x]_{反} = 0,1101$

$[x]_{反} = (2^{4+1}-1) - 1101$

$= 11111 - 1101$

$= 1,0010$

用 逗号 将符号位和数值位隔开

4. 反码表示法

小数

$$[x]_{反}=\begin{cases} x & 1>x\geq 0 \\ (2-2^{-n})+x & 0\geq x>-1\ (\mathrm{mod}\ 2-2^{-n}) \end{cases}$$

x 为真值

如

$x = +0.1101$　　$x = -0.1010$

$[x]_{反} = 0.1101$

$[x]_{反} = (2-2^{-4}) -0.1010$

$= 1.1111 -0.1010$

$= 1.0101$

用 小数点 将符号位和数值位隔开

(2) 举例

例6.8　已知 $[x]_{反} = 0,1110$　求 x

解：　由定义得　$x = +1110$

例6.9　已知 $[x]_{反} = 1,1110$　求 x

解：　由定义得　$x = [x]_{反} - (2^{4+1} - 1)$

$= 1,1110 - 11111$

$= -0001$

例 6.10　求 0 的反码

解：　设 $x = +0.0000$　$[+0.0000]_{反} = 0.0000$

$x = -0.0000$　$[-0.0000]_{反} = 1.1111$

同理，对于整数　$[+0]_{反} = 0,0000$　$[-0]_{反} = 1,1111$

$\therefore \quad [+0]_{反} \neq [-0]_{反}$

三种机器数的小结

- 最高位为符号位，书写上用“,” （整数）

  或“.” （小数）将数值部分和符号位隔开

- 对于正数，原码 = 补码 = 反码

- 对于负数，符号位为 1，其 数值部分

  原码除符号位外每位取反末位加 1⟶补码

  原码除符号位外每位取反⟶反码

三种机器数的小结

$$[x]_{原}=\begin{cases}0,\ x & 0\le x<2^n\\ 2^n-x & -2^n<x\le 0\end{cases}$$

$$[x]_{原}=\begin{cases}x & 1>x\ge 0\\ 1-x & 0\ge x>-1\end{cases}$$

	整数范围	小数范围
原码	$-2^n<[x]_{原}<2^n$	$-1<[x]_{原}<+1$
补码	$-2^n\le[x]_{补}<2^n$	$-1\le[x]_{补}<+1$
反码	$-2^n<[x]_{反}<2^n$	$-1<[x]_{反}<+1$

$$[x]_{补}=\begin{cases}0,\ x & 2^n>x\ge 0\\ 2^{n+1}+x & 0>x\ge -2^n\ (\mathrm{mod}\ 2^{n+1})\end{cases}$$

$$[x]_{补}=\begin{cases}x & 1>x\ge 0\\ 2+x & 0>x\ge -1\ (\mathrm{mod}\ 2)\end{cases}$$

假设 n=3（即包含符号位机器数共 4 位）

真值 x	$[x]_{原}$	$[x]_{补}$	$[x]_{反}$
+0	0000	0000	0000
-0	1000	0000	1111
-8	无法表示	1000	无法表示
-1.0	无法表示	1000	无法表示

$$[x]_{反}=\begin{cases}0,\ x & 2^n>x\ge 0\\ (2^{n+1}-1)+x & 0\ge x>-2^n\ (\mathrm{mod}\ 2^{n+1}-1)\end{cases}$$

$$[x]_{反}=\begin{cases}x & 1>x\ge 0\\ (2-2^{-n})+x & 0\ge x>-1\ (\mathrm{mod}\ 2-2^{-n})\end{cases}$$

例如

假设n=3（即包含符号位机器数共4位）

	正数
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111

补码

	负数
~~0~~	~~0000~~
-1	1111
-2	1110
-3	1101
-4	1100
-5	1011
-6	1010
-7	1001
-8	1000

反码

	负数
-0	1111
-1	1110
-2	1101
-3	1100
-4	1011
-5	1010
-6	1001
-7	1000

原码

	负数
-0	1000
-1	1001
-2	1010
-3	1011
-4	1100
-5	1101
-6	1110
-7	1111

0	+0	0	+0
/	/	/	/
2	2	2	2
⋮	⋮	⋮	⋮
255	255	255	255
256	-0	-256	-0
257	-1	-255	-254
⋮			

255 511 -

例6.11 设机器数字长为 8 位（其中一位为符号位）

对于整数，当其分别代表无符号数、原码、补码和反码时，对应的真值范围各为多少？

二进制代码	无符号数对应的真值	原码对应的真值	补码对应的真值	反码对应的真值
00000000	0	+0	±0	+0
00000001	1	+1	+1	+1
00000010	2	+2	+2	+2
⋮	⋮	⋮	⋮	⋮
01111111	127	+127	+127	+127
10000000	128	-0	-128	-127
10000001	129	-1	-127	-126
⋮	⋮	⋮	⋮	⋮
11111101	253	-125	-3	-2
11111110	254	-126	-2	-1
11111111	255	-127	-1	-0

例6.12 已知 $[y]_{补}$ 求 $[-y]_{补}$

解：设 $[y]_{补} = y_0.\, y_1\, y_2 \cdots y_n$

$[y_{原}] = 0.\, y_1\, y_2 \cdots y_n$

$y=$

$-y=$

<I> $[y]_{补} = 0.\, y_1 y_2 \cdots y_n$

$[y]_{补}$连同符号位在内，每位取反，末位加 1

即得$[-y]_{补}$

$$[-y]_{补} = 1.\overline{y_1}\ \overline{y_2} \cdots \overline{y_n} + 2^{-n}$$

<II> $[y]_{补} = 1.\, y_1 y_2 \cdots y_n$

$[y]_{补}$连同符号位在内，每位取反，末位加 1

即得$[-y]_{补}$

$$[-y]_{补} = 0.\overline{y_1}\ \overline{y_2} \cdots \overline{y_n} + 2^{-n}$$

5. 移码表示法

补码表示很难直接判断其真值大小

如	十进制	二进制	补码	
	$x = +21$	+10101	010101	错
	$x = -21$	−10101	101011	大
	$x = +31$	+11111	011111	错
	$x = -31$	−11111	100001	大

$x + 2^5$ 2^n 得 2^{n+1}

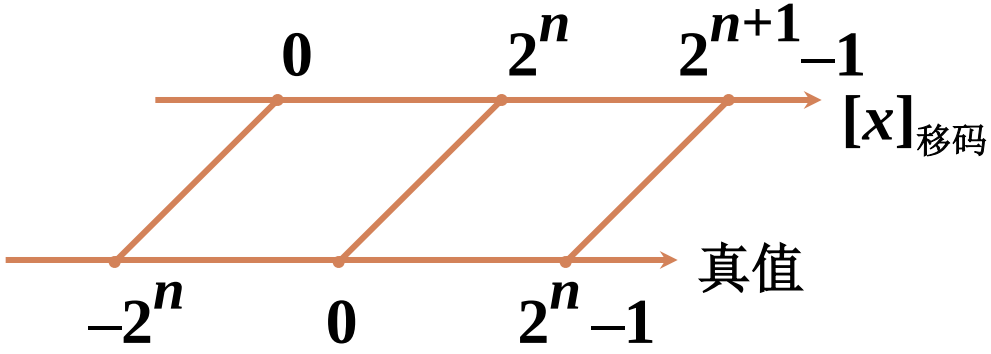
+10101 + 100000 = 110101	大	正确
−10101 + 100000 = 001011		
+11111 + 100000 = 111111	大	正确
−11111 + 100000 = 000001		

(1) 移码定义

$$[x]_{移} = 2^n + x \quad (2^n > x \geq -2^n)$$

x 为真值，n 为 整数的位数

移码在数轴上的表示



如　　$x = 10100$

$[x]_{移} = 2^5 + 10100 = 1,10100$

$x = -10100$

$[x]_{移} = 2^5 - 10100 = 0,01100$

用 逗号 将符号位和数值位隔开

(2) 移码和补码的比较

设　　$x = +1100100$

$[x]_{移} = 2^7 + 1100100 = 1,1100100$

$[x]_{补} = 0,1100100$

设　　$x = -1100100$

$[x]_{移} = 2^7 - 1100100 = 0,0011100$

$[x]_{补} = 1,0011100$

原↔反
补↔移

补码与移码只差一个符号位

(3) 真值、补码和移码的对照表

真值 x (n=5)	$[x]_{补}$	$[x]_{移}$	$[x]_{移}$对应的十进制整数
-1 0 0 0 0 0	1 0 0 0 0 0	0 0 0 0 0 0	0
- 1 1 1 1 1	1 0 0 0 0 1	0 0 0 0 0 1	1
- 1 1 1 1 0	1 0 0 0 1 0	0 0 0 0 1 0	2
⋮	⋮	⋮	⋮
- 0 0 0 0 1	1 1 1 1 1 1	0 1 1 1 1 1	31
± 0 0 0 0 0	0 0 0 0 0 0	1 0 0 0 0 0	32
+ 0 0 0 0 1	0 0 0 0 0 1	1 0 0 0 0 1	33
+ 0 0 0 1 0	0 0 0 0 1 0	1 0 0 0 1 0	34
⋮	⋮	⋮	⋮
+ 1 1 1 1 0	0 1 1 1 1 0	1 1 1 1 1 0	62
+ 1 1 1 1 1	0 1 1 1 1 1	1 1 1 1 1 1	63

(4) 移码的特点

- 当 $x = 0$ 时 $[+0]_{移} = 2^5 + 0 = 1,00000$

  $[-0]_{移} = 2^5 - 0 = 1,00000$

  $\therefore [+0]_{移} = [-0]_{移}$

- 当 $n = 5$ 时 最小的真值为 $-2^5 = -100000$

  $[-100000]_{移} = 2^5 - 100000 = 000000$

  可见，最小真值的移码为全 0

用移码表示浮点数的阶码

能方便地判断浮点数的阶码大小

例题1

例：无符号数在C语言中对应**unsigned short、unsigned int(unsigned)、unsigned long**，带符号整数表示为**short、int、long**类型，求以下程序段在一个**32**位机器上运行时输出的结果，并说明为什么。

```
#include <stdio.h>

int main(){
    int  x=-1;
    unsigned y=2147483648UL;
    printf (" x=%u=%d\n", x , x);
    printf (" y=%u=%d\n", y , y);
    return 0;
}
```

说明：

•其中**printf**为输出函数，指示符**%u、%d**分别表示以无符号整数和有符号整数形式输出十进制数的值，

• 2147483648=2^{31}

解：因为现代计算机中带符号整数都是用补码表示的，**-1**的补码整数表示为“**11…1**”（共**32**位），当作**32**位无符号数时，因此，十进制表示为 2^{32}**-1=4294967296-1= 4294967295**

2^{31}的无符号整数在计算机中表示为“**10…0**”，当作为有符号整数输出时，其值为最小负数**-**2^{32-1}**=-** 2147483648

输出结果：

```
x=4294967295=-1
y=2147483648=-2147483648
```

例题2

例：以下为C语言程序，用来计算一个数组a中每个元素的和，当参数len为0时，返回值应该为0，却发生了存储器访问异常。请问这是什么原因造成的？说明如何修改。

```
#include <stdio.h>

float  sum_elements  ( float a[ ],unsigned len){
    float  result=0;
    for ( int i=0; i<=len-1; i++ )
        result += a[ i ] ;
    return result;
}
int main(){

    int len=0;
    float x,a[]={0.1,0.2,0.3};

    x = sum_elements(a,len);
    printf (" x=%f", x);
}
```

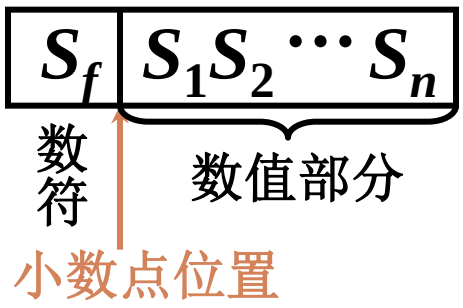
x=0. 000000

解：存储器访问异常是由于对数组a访问时产生了越界错误造成的。循环变量i是int型，而len是unsigned型，当len为0时，执行len-1的结果为32个1，是最大可表示的32位无符号数，任何无符号数都比它小，使循环体不断被执行，导致数组访问越界，因而发生存储器访问异常，应当将len声明为int型。

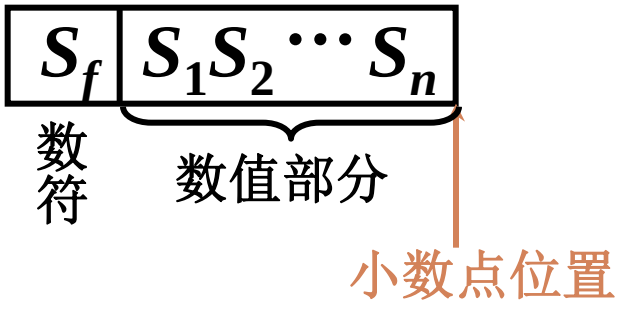
6.2 数的定点表示和浮点表示

小数点按约定方式标出

一、定点表示



或



定点机	小数定点机	整数定点机
原码	$-(1-2^{-n}) \sim +(1-2^{-n})$	$-(2^n-1) \sim +(2^n-1)$
补码	$-1 \sim +(1-2^{-n})$	$-2^n \sim +(2^n-1)$
反码	$-(1-2^{-n}) \sim +(1-2^{-n})$	$-(2^n-1) \sim +(2^n-1)$

二、浮点表示

$N = S \times r^j$　　浮点数的一般形式

S 尾数　j 阶码　r 基数（基值）

计算机中 r 取 2、4、8、16 等

当 $r = 2$　　N = 11.0101

√ = 0.110101×2^{10}

= 1.10101×2^{1}

= 1101.01×2^{-10}

= $0.00110101 \times 2^{100}$

二进制表示

规格化数

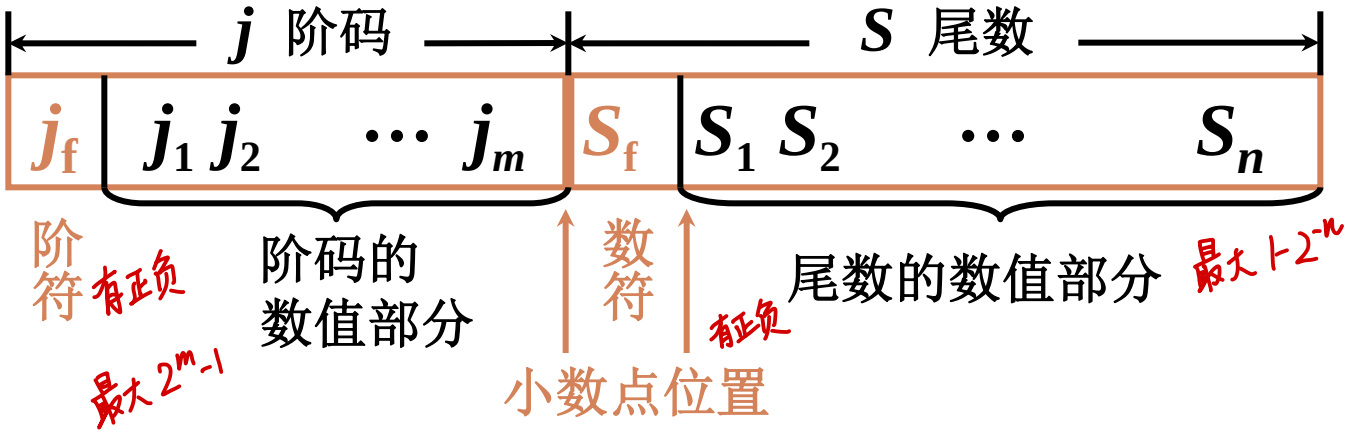
尾数为纯小数

计算机中　S　小数、可正可负

j　整数、可正可负

定点机表示浮点数

1. 浮点数的表示形式

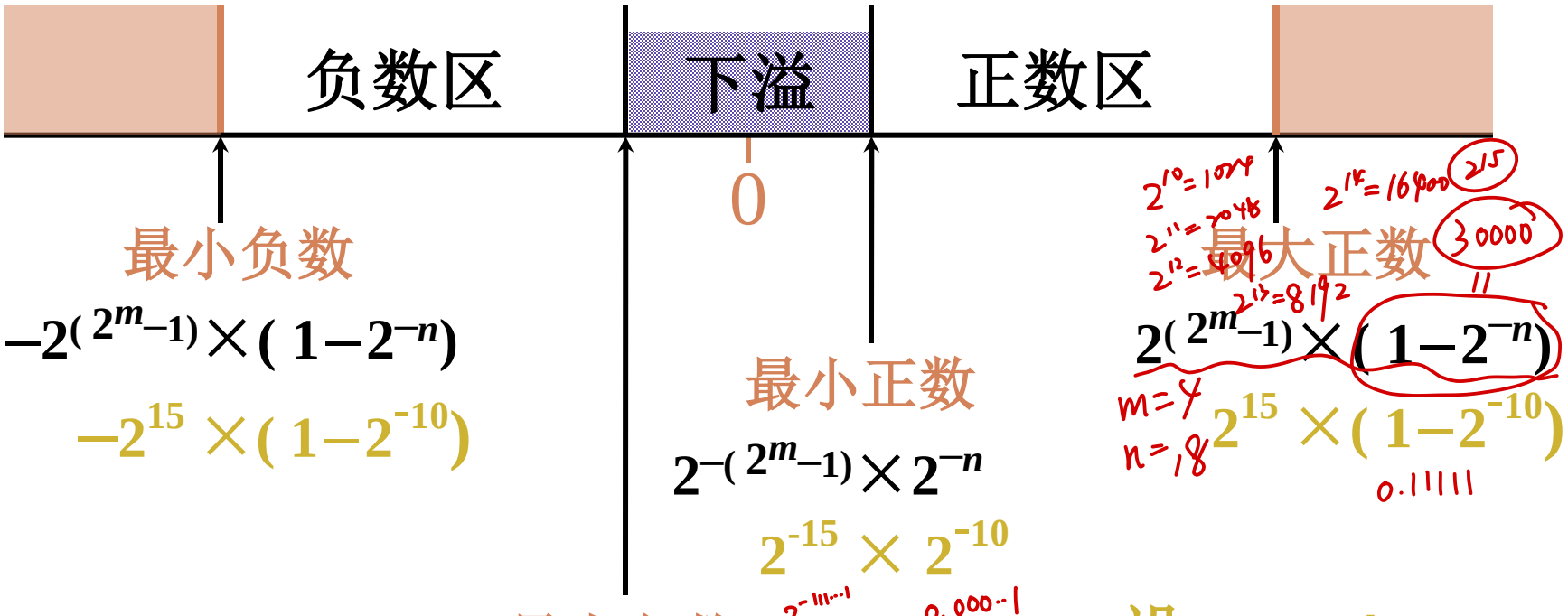


S_f	代表浮点数的符号
n	其位数反映浮点数的精度
m	其位数反映浮点数的表示范围
j_f 和 m	共同表示小数点的实际位置

2. 浮点数的表示范围(原码)

上溢　阶码 > 最大阶玛

下溢　阶码 < 最小阶码　按 机器零 处理



填空题 1分

设置

设机器数字长为 24 位，欲表示±3万的十进制数，试问在保证数的表示范围的前提下，能达到最大精度，除阶符、数符各 取1 位外，阶码 [填空1] 、尾数 [填空2] 各取几位？

22

正常使用填空题需3.0以上版本雨课堂

作答

练习 6.2

设机器数字长为 24 位，欲表示±3万的十进制数，试问在保证数的表示范围的前提下，能达到最大精度，除阶符、数符各 取1 位外，阶码、尾数各取几位？

解：∵ $2^{14} = 16384$ $2^{15} = 32768$

∴ 15 位二进制数可反映 ±3 万之间的十进制数

$2^{15} \times 0.\times\times\times \ldots \ldots \times\times$

18位

$2^4 = 16$ $m = 4、5、6 \cdots$

满足 最大精度 可取 $m = 4$，$n = 18$

3. 浮点数的规格化形式

$|s| \geqslant 1/r$

进制

$r = 2$　尾数最高位为 1

$r = 4$　尾数最高 2 位不全为 0

$r = 8$　尾数最高 3 位不全为 0

基数不同，浮点数的规格化形式不同

$0.\times\times\times\times\times\times\times\times$

$2^{-1}\ 2^{-2}\ 2^{-3}\ 2^{-4}\ \ldots$

$r = 2$		√ 0.0101110	√
$r = 4$	√	√ 0.1000101	√
$r = 8$		0.0010101	√

4. 浮点数的规格化

移数 不移小数点

$r = 2$	左规	尾数左移 1 位，	阶码减 1
	右规	尾数右移 1 位，	阶码加 1
$r = 4$	左规	尾数左移 2 位，	阶码减 1
	右规	尾数右移 2 位，	阶码加 1
$r = 8$	左规	尾数左移 3 位，	阶码减 1
	右规	尾数右移 3 位，	阶码加 1

基数 r 越大，可表示的浮点数的范围越大

基数 r 越大，浮点数的精度降低

例如：　设 $m = 4$，$n = 10$

尾数规格化后的浮点数表示范围

最大正数　$2^{+1111} \times 0.\underbrace{1111111111}_{10\text{ 个 }1} = 2^{15} \times (1 - 2^{-10})$

$2^{-15} \times 2^{-10} = 2^{-25}$

最小正数　$2^{-1111} \times 0.\underbrace{1000000000}_{9\text{ 个 }0} = 2^{-15} \times 2^{-1} = 2^{-16}$

$-2^{-15} \times 2^{-10} = -2^{-25}$

最大负数　$2^{-1111} \times (-0.\underbrace{1000000000}_{9\text{ 个 }0}) = -2^{-15} \times 2^{-1} = -2^{-16}$

最小负数　$2^{+1111} \times (-0.\underbrace{1111111111}_{10\text{ 个 }1}) = -2^{15} \times (1 - 2^{-10})$

三、举例

例 6.13　将 $+\frac{19}{128}$ 写成二进制定点数、浮点数及在定点机和浮点机中的机器数形式。其中数值部分均取 10 位，数符取 1 位，浮点数阶码取 5 位（含1位阶符）。

解：　设 $x = +\frac{19}{128}$ ？ $\frac{19}{128} = \frac{1+16+2}{27} = 2^{-7}+2^{-3}+2^{-6}$ 0.0010011

二进制形式　$x = 0.0010011$

定点表示　$x = 0.0010011000$

浮点规格化形式　$x = 0.1001100000\times 2^{-10}$

定点机中　$[x]_{原} = [x]_{补} = [x]_{反} = 0.0010011000$

浮点机中　$[x]_{原} = 1,\ 0010;\ \ 0.\ 1001100000$

$[x]_{补} = 1,\ 1110;\ \ 0.\ 1001100000$

$[x]_{反} = 1,\ 1101;\ \ 0.\ 1001100000$

例 6.14　将 –58 表示成二进制定点数和浮点数，并写出它在定点机和浮点机中的三种机器数及阶码为移码，尾数为补码的形式（其他要求同上例）。

解：　设 $x = -58$

$-58 = -(32+16+8+2) = -(2^5+2^4+2^3+2^1) = 1111010$

$[x]_{原} = 0,0110;\ 1.1110100000$

$[x]_{补} = 0,0110;$

二进制形式　$x = -111010$

定点表示　$x = -0000111010$

浮点规格化形式　$x = -(0.1110100000) \times 2^{110}$

定点机中

$[x]_{原} = 1,\ 0000111010$

$[x]_{补} = 1,\ 1111000110$

$[x]_{反} = 1,\ 1111000101$

浮点机中

$[x]_{原} = 0,\ 0110;\ 1.\ 1110100000$

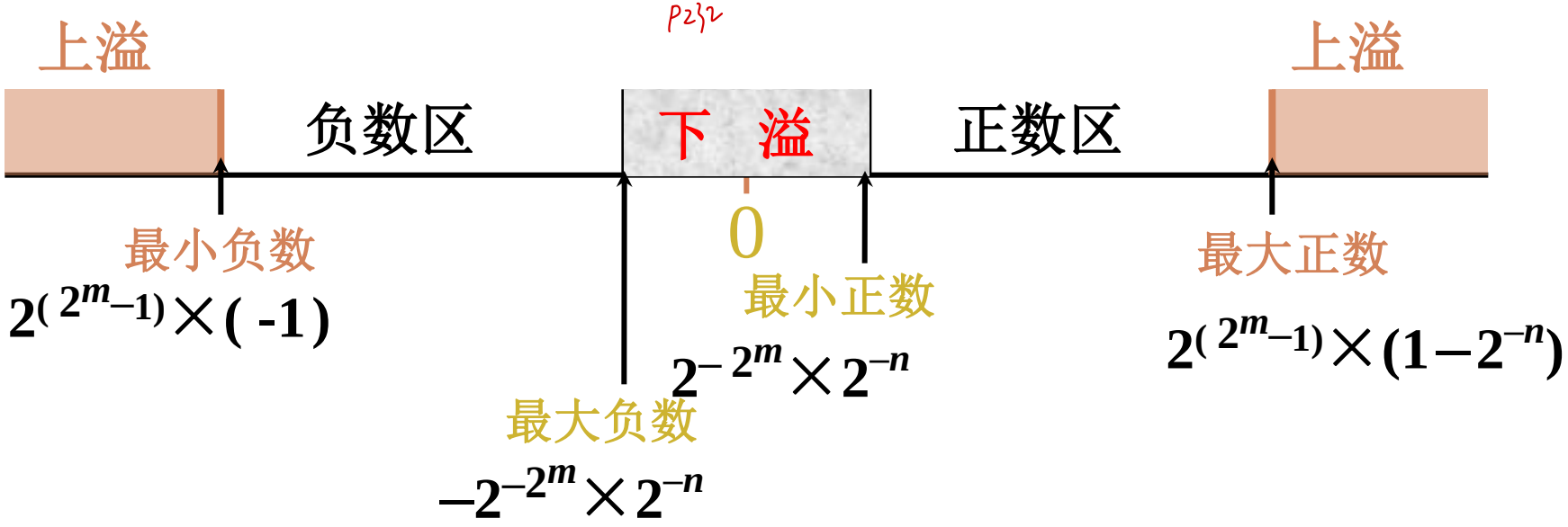
$[x]_{补} = 0,\ 0110;\ 1.\ 0001100000$

$[x]_{反} = 0,\ 0110;\ 1.\ 0001011111$

$[x]_{阶移、尾补} = 1,\ 0110;\ 1.\ 0001100000$

例6.15 写出补码浮点数正数和负数真值范围。

设 $n = 10$，$m = 4$， 阶符、数符各取 1位。



解：

	真值	补码
最大正数	$2^{15} \times (1-2^{-10})$	0,1111; 0.1111111111
最小正数	$2^{-16} \times 2^{-10}$	1,0000; 0.0000000001
最大负数	$-2^{-16} \times 2^{-10}$	1,0000; 1.1111111111
最小负数	$2^{15} \times (-1.0)$	0,1111; 1.0000000000

机器零

- 当浮点数尾数为 0 时，不论其阶码为何值按机器零处理
- 当浮点数 阶码等于或小于它所表示的最小数时，不论尾数为何值，按机器零处理

如 $m = 4$　　$n = 10$

当阶码和尾数都用补码表示时，机器零为

×,××××；　0.0 0 …… 0

（阶码 = −16）　1, 0 0 0 0；　×.×× …… ×

当阶码用移码，尾数用补码表示时，机器零为

0, 0 0 0 0；0.0 0 …… 0

有利于机器中“ 判 0 ” 电路的实现

“Father” of the IEEE 754 standard

直到80年代初，各个机器内部的浮点数表示格式还没有统一
因而相互不兼容，机器之间传送数据时，带来麻烦

1970年代后期，IEEE成立委员会着手制定浮点数标准

1985年完成浮点数标准IEEE754的制定

现在所有计算机都采用IEEE754来表示浮点数

This standard was primarily the work of one person, UC Berkeley math professor William Kahan.

1989
ACM Turing
Award Winner!

www.cs.berkeley.edu/~wkahan/ieee754status/754story.html

Prof. William Kahan

四、IEEE 754 标准

$N = S \times r^j$ 浮点数的一般形式

$x = -(0.1110100000) \times 2^{110}$

$x = -(1.1101000000) \times 2^{101}$ 省略一位

符号Sign	阶码Exponent（含阶符）	尾数Mantissa

符号位S： 0——正数，1——负数。

阶码E: 移码，存在偏移量 与之前不同 边界平滑

尾数M: 1.xxx……x 只是1 原码

在实际表示中整数位的1省略，称隐藏位（临时浮点数不采用隐藏位方案）

阶码部分也与一般移码不同，对短实数而言，

$[X]_{移}=2^7-1+X = 127+X$

四、IEEE 754 标准

Single Precision	符号位 S	阶码	尾数	总位数
短实数	1	8	23	32
长实数	1	11	52	64
临时实数	1	15	64	80

Sign bit: 1 表示negative ; 0表示 positive

Exponent（阶码 / 指数）： 全0和全1用来表示特殊值！

- SP规格化数阶码范围为0000 0001 (-126) ~ 1111 1110 (127)
- bias为127 (single), 1023 (double)

课后思考：偏移量为什么用127？
若用128, 则阶码范围为多少？

Mantissa（尾数）：

- 规格化尾数最高位总是1，所以隐含表示，省1位
- 1 + 23 bits （ single），1 + 52 bits （double）

SP: $(-1)^S \times (1 + \text{Mantissa}) \times 2^{(\text{Exponent}-127)}$

DP: $(-1)^S \times (1 + \text{Mantissa}) \times 2^{(\text{Exponent}-1023)}$

0000 0001 (-127) ～
1111 1110 (126)

四、IEEE 754 标准

- 例1: 将-178.125转化成32位单精度IEEE754格式表示。

解：$(-178.125)_{10}$ => $(-\ 10110010.001\ )_2$

$-\ 10110010.001 = -1.0110010001*2^7 = = -1.0110010001*2^{111}$

S(1位)=1

E(8位)=$(7+127)_{10}$=111+01111111 =$(10000110\ )_2$

1.M(23位)=1.011 0010 0010 0000 0000 0000

数符Sign	阶码Exponent（含阶符）								尾数Mantissa																			
1	1	0	0	0	0	1	1	0	0	1	1	0	0	1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0

隐含小数点

四、IEEE 754 标准

- 例2：若浮点数 x 的IEEE754标准存储格式为：41360000H，求其浮点数 x 的十进制数值。

解：把41360000H转换为二进制：

0100 0001 0011 0110 0000 0000 0000 0000

符号S　　阶码E　　尾数M

0　　10000010　　011 0110 0000 0000 0000 0000

$E = 130-127 = (3)_{10}$　　$M = (0.011011)_2$

$x = + (1.M) * 2^E$

$= + 1.011011 * 2^3$

$= (1011.011)_2$

$= (11.375)_{10}$

1 8 23

0100 0001 0011 0110 0000 0000

$+1.011011 \times 2^{11}$

$= (1011.011)_2$

$= (11.375)_{10}$

四、IEEE 754 标准

- 特殊情况

单精度浮点数各种极值情况:

类别	正负号	实际指数	有偏移指数	指数域	尾数域	数值
零	0	-127	0	0000 0000	000 0000 0000 0000 0000 0000	0.0
负零	1	-127	0	0000 0000	000 0000 0000 0000 0000 0000	−0.0
1	0	0	127	0111 1111	000 0000 0000 0000 0000 0000	1.0
-1	1	0	127	0111 1111	000 0000 0000 0000 0000 0000	−1.0
最小的非规约数	*	-126	0	0000 0000	000 0000 0000 0000 0000 0001	$\pm 2^{-23} \times 2^{-126} = \pm 2^{-149} \approx \pm 1.4 \times 10^{-45}$
中间大小的非规约数	*	-126	0	0000 0000	100 0000 0000 0000 0000 0000	$\pm 2^{-1} \times 2^{-126} = \pm 2^{-127} \approx \pm 5.88 \times 10^{-39}$
最大的非规约数	*	-126	0	0000 0000	111 1111 1111 1111 1111 1111	$\pm(1-2^{-23}) \times 2^{-126} \approx \pm 1.18 \times 10^{-38}$
最小的规约数	*	-126	1	0000 0001	000 0000 0000 0000 0000 0000	$\pm 2^{-126} \approx \pm 1.18 \times 10^{-38}$ 门平滑
最大的规约数	*	127	254	1111 1110	111 1111 1111 1111 1111 1111	$\pm(2-2^{-23}) \times 2^{127} \approx \pm 3.4 \times 10^{38}$
正无穷	0	128	255	1111 1111	000 0000 0000 0000 0000 0000	+∞
负无穷	1	128	255	1111 1111	000 0000 0000 0000 0000 0000	−∞
NaN	*	128	255	1111 1111	non zero	NaN

* 符号位可以为0或1 .

四、IEEE 754 标准

IEEE754中规定：

1：若阶码为全0，尾数为全0，表示的是0；根据符号的不同，可表示正0，负0；

2：若阶码为全0，尾数为非0。表示正负非规格化数。

3：若阶码为全1，尾数为全0，表示根据阶符不同，表示正负无穷大。

4：若阶码为全1 ，尾数为非0，表示 NaN (Not a Number)，不是一个数。

形式	指数	小数部分
零	0	0
非规约形式	0	非0
规约形式	1到 2^e-2	任意
无穷	2^e-1	0
NaN	2^e-1	非零

四、IEEE 754 标准-扩展*

直到 20 世纪 80 年代（即在没有制定 IEEE 754 标准之前），业界还没有一个统一的浮点数标准。相反，很多计算机制造商根据自己的需要来设计自己的浮点数表示规则，以及浮点数的执行运算细节。另外，他们常常并不太关注运算的精确性，而把实现的速度和简易性看得比数字的精确性更重要，而这就给代码的可移植性造成了重大的障碍。

直到 1976 年，Intel 公司打算为其 8086 微处理器引进一种浮点数协处理器时，意识到作为芯片设计者的电子工程师和固体物理学家也许并不能通过数值分析来选择最合理的浮点数二进制格式。于是，他们邀请加州大学伯克利分校的 William Kahan 教授（当时最优秀的数值分析家）来为 8087 浮点处理器（FPU）设计浮点数格式。而这时，William Kahan 教授又找来两个专家协助他，于是就有了 KCS 组合（Kahn、Coonan和Stone），并共同完成了 Intel 公司的浮点数格式设计。

由于 Intel 公司的 KCS 浮点数格式完成得如此出色，以致 IEEE（Institute of Electrical and Electronics Engineers，电子电气工程师协会）决定采用一个非常接近 KCS 的方案作为 IEEE 的标准浮点格式。于是，IEEE 于 1985 年制订了二进制浮点运算标准 IEEE 754（IEEE Standard for Binary Floating-Point Arithmetic，ANSI/IEEE Std 754-1985），该标准限定指数的底为 2，并于同年被美国引用为 ANSI 标准。目前，几乎所有的计算机都支持 IEEE 754 标准，它大大地改善了科学应用程序的可移植性。

考虑到 IBM System/370 的影响，IEEE 于 1987 年推出了与底数无关的二进制浮点运算标准 IEEE 854，并于同年被美国引用为 ANSI 标准。1989 年，国际标准组织 IEC 批准 IEEE 754/854 为国际标准 IEC 559：1989。后来经修订后，标准号改为 IEC 60559。现在，几乎所有的浮点处理器完全或基本支持 IEC 60559。同时，C99 的浮点运算也支持 IEC 60559。

四、IEEE 754 标准-扩展*

这种方式提供了一种从非规格方式值平滑转换到规格化值的办法。 ——《深入理解计算机系统》

非规格数

先来看看非规格化数值的的表示形式与规格数的表示形式相一致，但是计算的数值会有所差异，

$(-1)^{S} * F * 2^{E}$

指数域为全零，实际的指数值E=1-bias，而有效数值M=F（而不是规格数中的M=1+F）

为什么非规格化的值的指数值为1-bias而不是-bias？

这种形式的最小规格化数同样有 $E=1-7=-6$，并且小数取值范围也为 $0, \frac{1}{8}, \cdots, \frac{7}{8}$。然而，有效数在范围 $1+0=1 \sim 1+\frac{7}{8}=\frac{15}{8}$ 之间，得出数 V 在范围 $\frac{8}{512} \sim \frac{15}{512}$ 之间。

可以观察到最大非规格化数 $\frac{7}{512}$ 和最小规格化数 $\frac{8}{512}$ 之间的平滑转变。这种平滑性归功于我们对非规格化数 E 的定义。通过将 E 定义为 $1-Bias$，而不是 $-Bias$，这样我们可以补偿非规格化数的有效数没有隐含的开头的 1。

当我们增大指数时，我们成功地得到更大的规格化值，经过 1.0，然后得到最大的规格化数。这个数具有指数 $E=7$，得到一个权 $2^{E}=128$。小数等于 $\frac{7}{8}$ 得到有效数 $M=\frac{15}{8}$。因此，数值是 $V=240$。超出这个值就会溢出到+∞。

http://blog.csdn.net/programmer_at

但是这种平滑处理与“IEEE 754标准中，阶的偏置值为什么是127，而不是128？”关系不大，因为即使阶的偏置值是128一样可以实现非规格数向规格数的平滑转变（此时，最大非规格数为7/1024，最小规格数为8/1024）。因此，“非规格数向规格数的平滑转变”归功于“非规格化的值的指数值为1-bias而不是-bias”，而与IEEE754标准阶的偏置值是127还是128关系不大。

描述	位表示	e	E	f	M	V	
0	0 0000 000	0	-6	0	0	0	
最小的非规格化数	0 0000 001	0	-6	1/8	1/8	1/512	
	0 0000 010	0	-6	2/8	2/8	2/512	
	0 0000 011	0	-6	3/8	3/8	3/512	
	...						
	0 0000 110	0	-6	6/8	6/8	6/512	
最大的非规格化数	0 0000 111	0	-6	7/8	7/8	7/512	非规格数像规格数平滑过渡
最小的规格化数	0 0001 000	1	-6	0	8/8	8/512	
	0 0001 001	1	-6	1/8	9/8	9/512	
	...						
	0 0110 110	6	-1	6/8	14/8	14/16	
	0 0110 111	6	-1	7/8	14/8	15/16	
1	0 0111 000	7	0	0	8/8	1	
	0 0111 001	7	0	1/8	9/8	9/8	
	0 0111 010	7	0	2/8	10/8	10/8	
	...						
	0 1110 110	14	7	6/8	14/8	224	
最大的规格化数	0 1110 111	14	7	7/8	15/8	240	
无穷大	0 1111 000	—	—	—	—	+∞	

图 2.23 8 位浮点格式的非负值示例

有 $k=4$ 的指数位的和 $n=3$ 的有效数位。偏置量是 7。

http://blog.csdn.net/programmer_at

https://blog.csdn.net/programmer_at/article/details/54882503#t18

6.3 定点运算

一、移位运算

1. 移位的意义

15.米 = 1500.厘米

小数点右移 2 位

机器用语　　15 相对于小数点 左移 2 位

（ 小数点不动 ）

左移　　绝对值扩大

右移　　绝对值缩小

在计算机中，移位与加减配合，能够实现乘除运算

2. 算术移位规则

符号位不变

不讨论移码

	码　制	添补代码
正数	原码、补码、反码	0
负数	原　码	0
	补　码	左移 添 0
		右移 添 1
	反　码	1

例6.16

设机器数字长为 8 位（含一位符号位），写出 $A = +26$时，三种机器数左、右移一位和两位后的表示形式及对应的真值，并分析结果的正确性。

解：　　$A = +26 = +11010$

则 $[A]_{原} = [A]_{补} = [A]_{反} = 0,0011010$

移位操作	机　器　数	对应的真值
	$[A]_{原}=[A]_{补}=[A]_{反}$	
移位前	00011010	+26
←1	00110100	+52
←2	01101000	+104
→1	00001101	+13
→2	00000110	+6

例6.17

设机器数字长为 8 位（含一位符号位），写出 $A = -26$时，三种机器数左、右移一位和两位后的表示形式及对应的真值，并分析结果的正确性。

解：　　$A = -26 = -11010$

原码

移位操作	机　器　数	对应的真值
移位前	10011010	– 26
←1	10110100	– 52
←2	11101000	– 104
→1	10001101	– 13
→2	10000110	– 6

$A = -26 = -11010$

补码

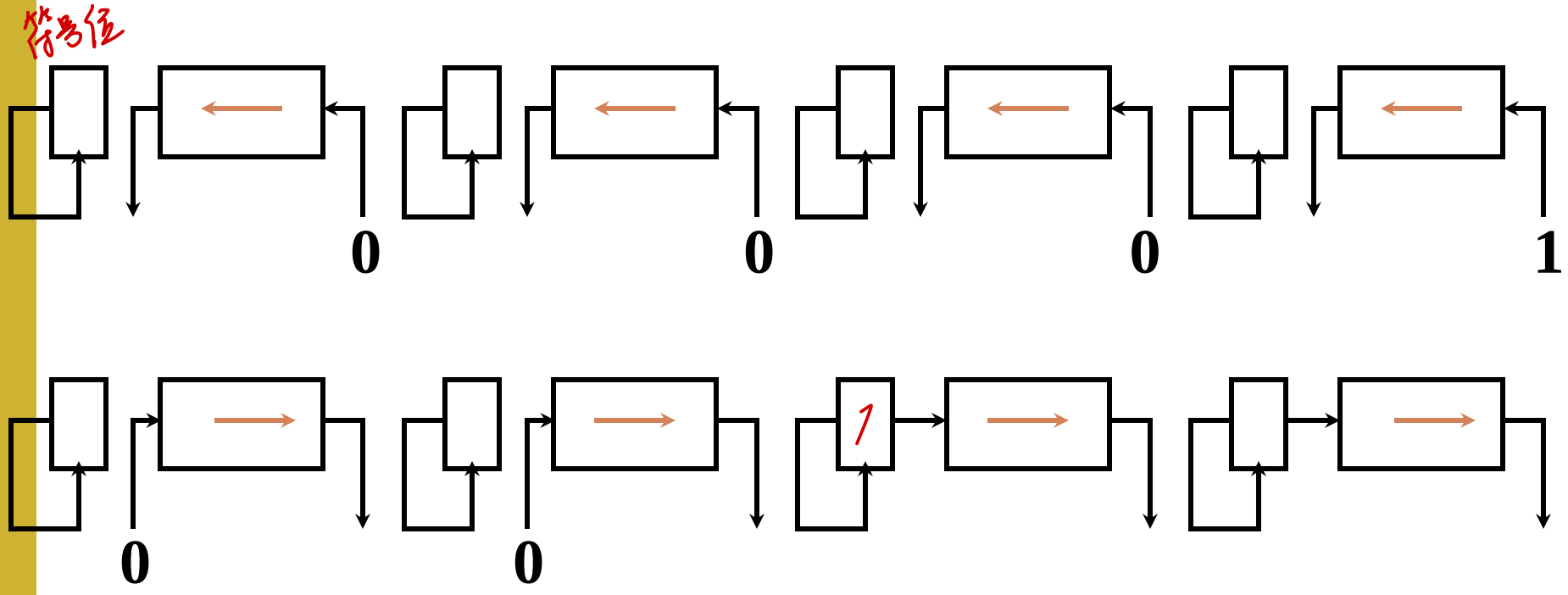
移位操作	机　器　数	对应的真值
移位前	11100110	– 26
←1	11001100	– 52
←2	10011000	– 104
→1	11110011	– 13
→2	11111001	– 7

反码

移位操作	机　器　数	对应的真值
移位前	11100101	– 26
←1	11001011	– 52
←2	10010111	– 104
→1	11110010	– 13
→2	11111001	– 6

3. 算术移位的硬件实现

6.3



	（a）真值为正	（b）负数的原码	（c）负数的补码	（d）负数的反码
←丢 1	出错	出错	正确	正确
→丢 1	影响精度	影响精度	影响精度	正确

丢0影响精度

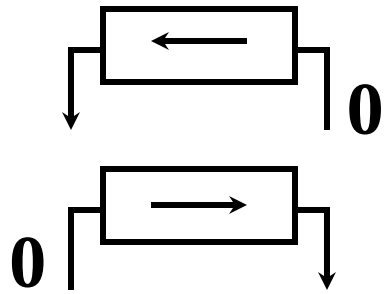
4. 算术移位和逻辑移位的区别 6.3

算术移位　　有符号数的移位

逻辑移位　　无符号数的移位

逻辑左移　　低位添 0，高位移丢

逻辑右移　　高位添 0，低位移丢



例如　　11010011　　10110010

逻辑左移　　10100110　　逻辑右移　　01011001

算术左移　　10100110　　算术右移　　11011001（补码）

高位 1 移丢

二、加减法运算

但是用原码作加法时，会出现如下问题：

要求	数1	数2	实际操作	结果符号
加法	正	正	加	正
加法	正	负	减	可正可负
加法	负	正	减	可正可负
加法	负	负	加	负

能否 只作加法？

找到一个与负数等价的正数 来代替这个负数

就可使 减 ——→ 加

二、加减法运算

6.3

1. 补码加减运算公式

(1) 加法

整数 $[A]_{补} + [B]_{补} = [A+B]_{补}$ （$\mathbf{mod}\ 2^{n+1}$）

小数 $[A]_{补} + [B]_{补} = [A+B]_{补}$ （$\mathbf{mod}\ 2$）

(2) 减法

$$A-B = A+(-B)$$

整数 $[A-B]_{补} = [A+(-B)]_{补} = [A]_{补} + [-B]_{补}\ (\mathbf{mod}\ 2^{n+1})$

小数 $[A-B]_{补} = [A+(-B)]_{补} = [A]_{补} + [-B]_{补}\ (\mathbf{mod}\ 2)$

连同符号位一起相加，符号位产生的进位自然丢掉

证明

(1) 设A＞0，B＜0

①｜B｜＞A （ A+B ＜0 ）

$$[A+B]_{补} \overset{负}{=} 2+A+B$$

$$=A+2+B$$

$$=[A]_{补}+[B]_{补}$$

②｜B｜ ＜ A （ A+B ＞ 0 ）

$$[A+B]_{补} \overset{正}{=} A+B$$

$$=A+2+B\ (\mathrm{mod}\ 2)$$

$$=[A]_{补}+[B]_{补}$$

2. 举例

例 6.18　设 $A = 0.1011$，$B = -0.0101$

求 $[A + B]_{补}$　　验证

解：

$$\begin{array}{rl} [A]_{补} & = \quad 0.1011 \\ +\,[B]_{补} & = \quad 1.1011 \\ \hline [A]_{补} + [B]_{补} & = 10.0110 = [A + B]_{补} \end{array}$$

$$\begin{array}{r} 0.1011 \\ -\,0.0101 \\ \hline 0.0110 \end{array}$$

$\therefore A + B = 0.0110$

例 6.19　设 $A = -9$，$B = -5$

求 $[A+B]_{补}$　　验证

解：

$$\begin{array}{rl} [A]_{补} & = \quad 1,0111 \\ +\,[B]_{补} & = \quad 1,1011 \\ \hline [A]_{补} + [B]_{补} & = 11,0010 = [A + B]_{补} \end{array}$$

$$\begin{array}{r} -1001 \\ +\,-0101 \\ \hline -1110 \end{array}$$

$\therefore A + B = -1110$

例 6.20 设机器数字长为 8 位（含 1 位符号位） 6.3

且 $A = 15$，$B = 24$，用补码求 $A - B$

解：$A = 15 = 0001111$

$B = 24 = 0011000$

$[A]_{补} = 0, 0001111$　　$[B]_{补} = 0, 0011000$

$+ [-B]_{补} = 1, 1101000$

$[A]_{补} + [-B]_{补} = 1, 1110111 = [A - B]_{补}$

$\therefore A - B = -1001 = -9$

练习 1　设 $x = \frac{9}{16}$　$y = \frac{11}{16}$　用补码求 $x+y$

$$\begin{array}{r} 0.1001 \\ +\ 0.1011 \\ \hline 1.0100 \\ -0.1100 \end{array}$$

$x + y = -0.1100 = -\frac{12}{16}$　溢出　错

练习 2　设机器数字长为 8 位（含 1 位符号位）

且 $A = -97$，$B = +41$，用补码求 $A - B$

$A - B = +1110110 = +118$　错

3. 溢出判断

(1) 一位符号位判溢出

参加操作的两个数（减法时即为被减数和“求补”以后的减数）符号相同，其结果的符号与原操作数的符号不同，即为溢出　一正一负不会溢出

硬件实现

最高有效位的进位 ⊕ 符号位的进位 = 1　　溢出

如

$1 \oplus 0 = 1$
$0 \oplus 1 = 1$
} 有溢出

$0 \oplus 0 = 0$
$1 \oplus 1 = 0$
} 无溢出

(2) 两位符号位判溢出

$$[x]_{补'} = \begin{cases} x & 1 > x \geq 0 \\ 4 + x & 0 > x \geq -1 \ (\text{mod } 4) \end{cases}$$

$$[x]_{补'} + [y]_{补'} = [\, x + y \,]_{补'} \ (\text{mod } 4)$$

$$[x - y]_{补'} = [x]_{补'} + [-y]_{补'} \ (\text{mod } 4)$$

结果的双符号位 相同　　未溢出

结果的双符号位 不同　　溢出

00, ×××××
11, ×××××

规格化处理 → 右移

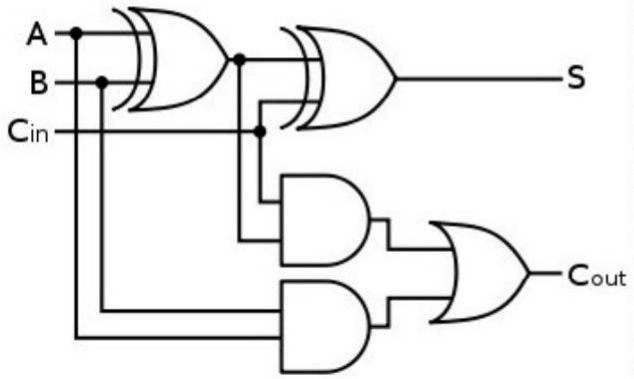
10, ×××××
01, ×××××

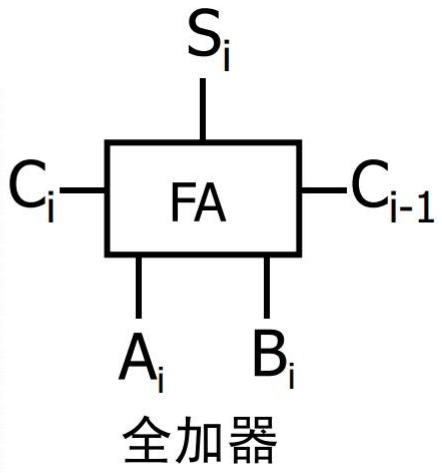
最高符号位 代表其 真正的符号

(3) 进位判溢法

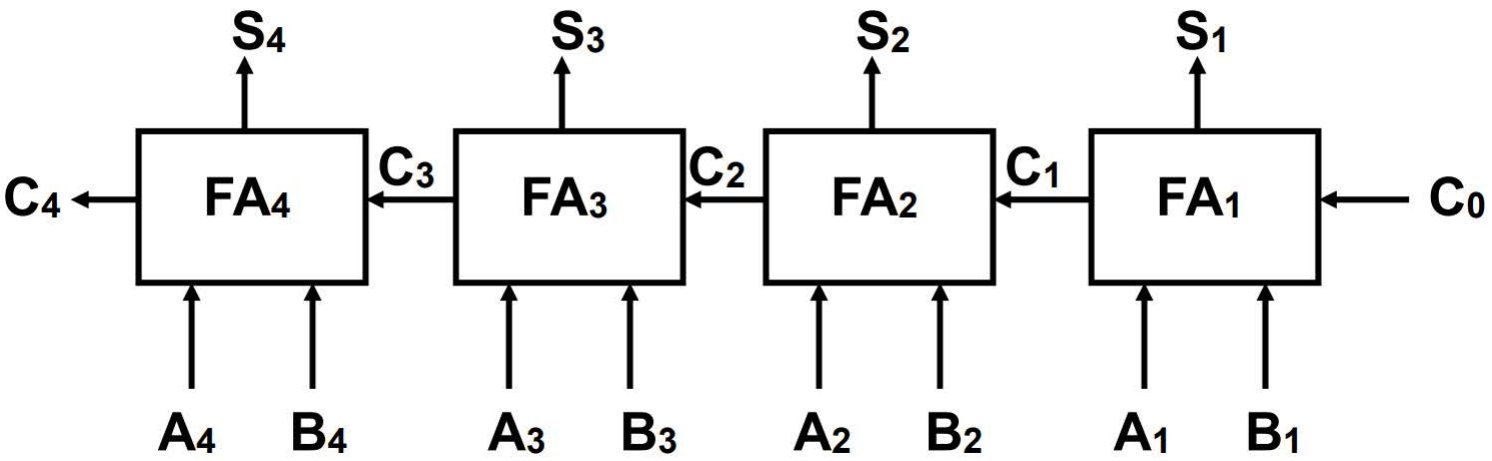
- 数值位有向符号位的进位，但符号位不产生向更高位的进位，数值位没有向符号位的进位，但符号位产生向更高位的进位，则有溢出。

4. 补码加减法的硬件配置



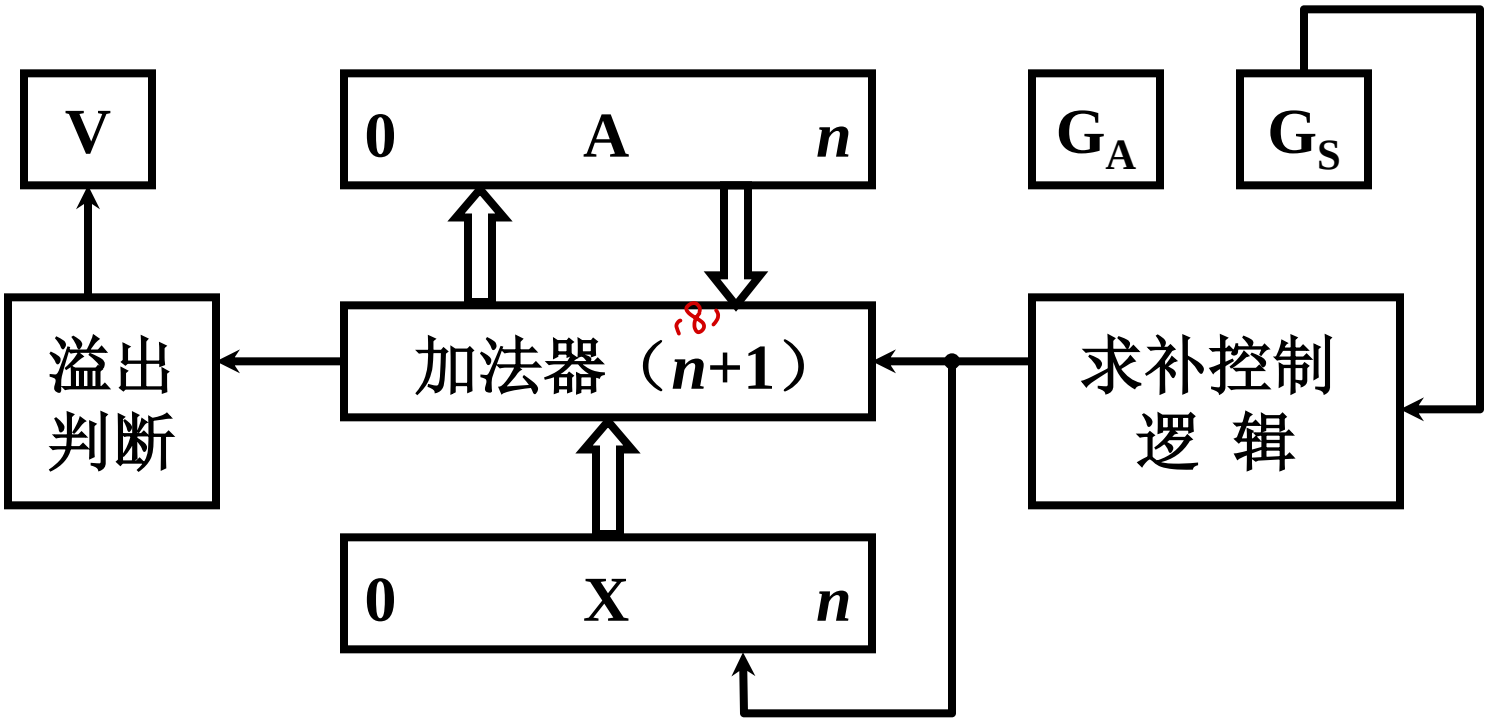


全加器



4. 补码加减法的硬件配置

6.3



A、X 均 $n+1$ 位

用减法标记 G_S 控制求补逻辑

三、乘法运算 6.3

1. 分析笔算乘法

$A = -0.1101$　　$B = 0.1011$

$A \times B = -0.10001111$　乘积的符号心算求得

```
  0.1101
 ×0.1011
 --------
  0 0 0 0 1 1 0 1
  0 0 0 1 1 0 1 0
  0 0 0 0 0 0 0 0
  0 1 1 0 1 0 0 0
 --------------------
 0.10001111
```

移与运算

- ✓ 符号位单独处理 ⊕
- ✓ 乘数的某一位决定是否加被乘数　双路选择器
- ? 4个位积一起相加
- ✓ 乘积的位数扩大一倍

ACC　MQ　两个寄存器

右移更方便

2. 笔算乘法改进

6.3

$$A \cdot B = A \cdot 0.1011$$

$$= 0.1A + 0.00A + 0.001A + 0.0001A$$

$$= 0.1A + 0.00A + 0.001(A + 0.1A)$$

$$= 0.1A + 0.01[0 \cdot A + 0.1(A + 0.1A)]$$

$$= 0.1\{A + 0.1[0 \cdot A + 0.1(A + 0.1A)]\}$$ 迭代

右移一位

$$= 2^{-1}\{A + 2^{-1}[0 \cdot A + 2^{-1}(A + 2^{-1}(A + 0))]\}$$

第一步　被乘数A + 0

第二步　→ 1，得新的部分积

第三步　部分积 + 被乘数

⋮

第八步　→ 1，得结果

① ② ③ ⑧

3. 改进后的笔算乘法过程（竖式） 6.3

$A = -0.1101 \quad B = 0.1011$

两个寄存器

部分积	乘数	说明
0.0000 0.1101	1011	初态，部分积 = 0 乘数为 1，加被乘数
0.1101 0.0110 0.1101	→ 1101	→1，形成新的部分积 乘数为 1，加被乘数
1.0011 0.1001 0.0000	1 → 1110	→1，形成新的部分积 乘数为 0，加 0
0.1001 0.0100 0.1101	11 → 1111	→1，形成新的部分积 乘数为 1，加 被乘数
1.0001 0.1000	111 1111	→1，得结果

逻辑右移

逻辑右移

不考虑符号位

连符号位

好1个→算术移位

小结 6.3

- 乘法运算 ⟶ 加和移位。$n = 4$，加 4 次，移 4 次
- 由乘数的末位决定被乘数是否与原部分积相加，然后⟶1, 形成新的部分积，同时乘数 ⟶ 1（末位移丢），空出高位存放部分积的低位。
- 被乘数只与部分积的高位相加

硬件　3个寄存器，具有移位功能

一个全加器

4. 原码乘法

(1) 原码一位乘运算规则

以小数为例

设$[x]_{原} = x_0.\ x_1x_2 \cdots x_n$

$[y]_{原} = y_0.\ y_1y_2 \cdots y_n$

$[x \cdot y]_{原} = (x_0 \oplus y_0).(0.\ x_1x_2 \cdots x_n)(0.\ y_1y_2 \cdots y_n)$

$= (x_0 \oplus y_0).\ x^*y^*$

式中 $x^* = 0.\ x_1x_2 \cdots x_n$ 为 x 的绝对值

$y^* = 0.\ y_1y_2 \cdots y_n$ 为 y 的绝对值

乘积的符号位单独处理 $x_0 \oplus y_0$

数值部分为绝对值相乘 $x^* \cdot y^*$

(2) 原码一位乘递推公式

$$x^*\cdot y^* = x^*(0.y_1y_2 \quad \cdots y_n)$$

$$= x^*(y_1 2^{-1} + y_2 2^{-2} + \quad \cdots + y_n 2^{-n})$$

$$= 2^{-1}(y_1 x^* + 2^{-1}(y_2 x^* + \cdots 2^{-1}(y_n x^* + 0) \cdots))$$

z_0

z_1

$\cdots\cdots$

z_n

$z_0 = 0$

$z_1 = 2^{-1}(y_n x^* + z_0)$ 0/1

$z_2 = 2^{-1}(y_{n-1} x^* + z_1)$

$\vdots$

$z_n = 2^{-1}(y_1 x^* + z_{n-1})$

例6.21 已知 $x = -0.1110$ $y = 0.1101$ 求$[x \cdot y]_{原}$

解：数值部分的运算

部分积	乘数	说明
0.0000 0.1110	1101	部分积 初态 $z_0 = 0$
0.1110 0.0111 0.0000	0110	→1，得 z_1
0.0111 0.0011 0.1110	0 1011	→1，得 z_2
1.0001 0.1000 0.1110	10 1101	→1，得 z_3
1.0110 0.1011	110 0110	→1，得 z_4

逻辑右移

逻辑右移

例6.21 已知 $x = -0.1110$ $y = 0.1101$ 求$[x \cdot y]_{原}$

解：数值部分的运算

部分积	乘数	说明
00.0000 00.1110	1101	部分积 初态 $z_0 = 0$
00.1110 00.0111 00.0000	0110	→1，得 z_1
00.0111 00.0011 00.1110	0 1011	→1，得 z_2
01.0001 00.1000 00.1110	10 1101	→1，得 z_3
01.0110 00.1011	110 0110	→1，得 z_4

真正符号位在最高位

算术右移

算术右移

例6.21 结果

① 乘积的符号位 $x_0 \oplus y_0 = 1 \oplus 0 = 1$

② 数值部分按绝对值相乘

$$x^* \cdot y^* = 0.1\,0\,1\,1\,0\,1\,1\,0$$

则 $[x \cdot y]_{原} = 1.1\,0\,1\,1\,0\,1\,1\,0$

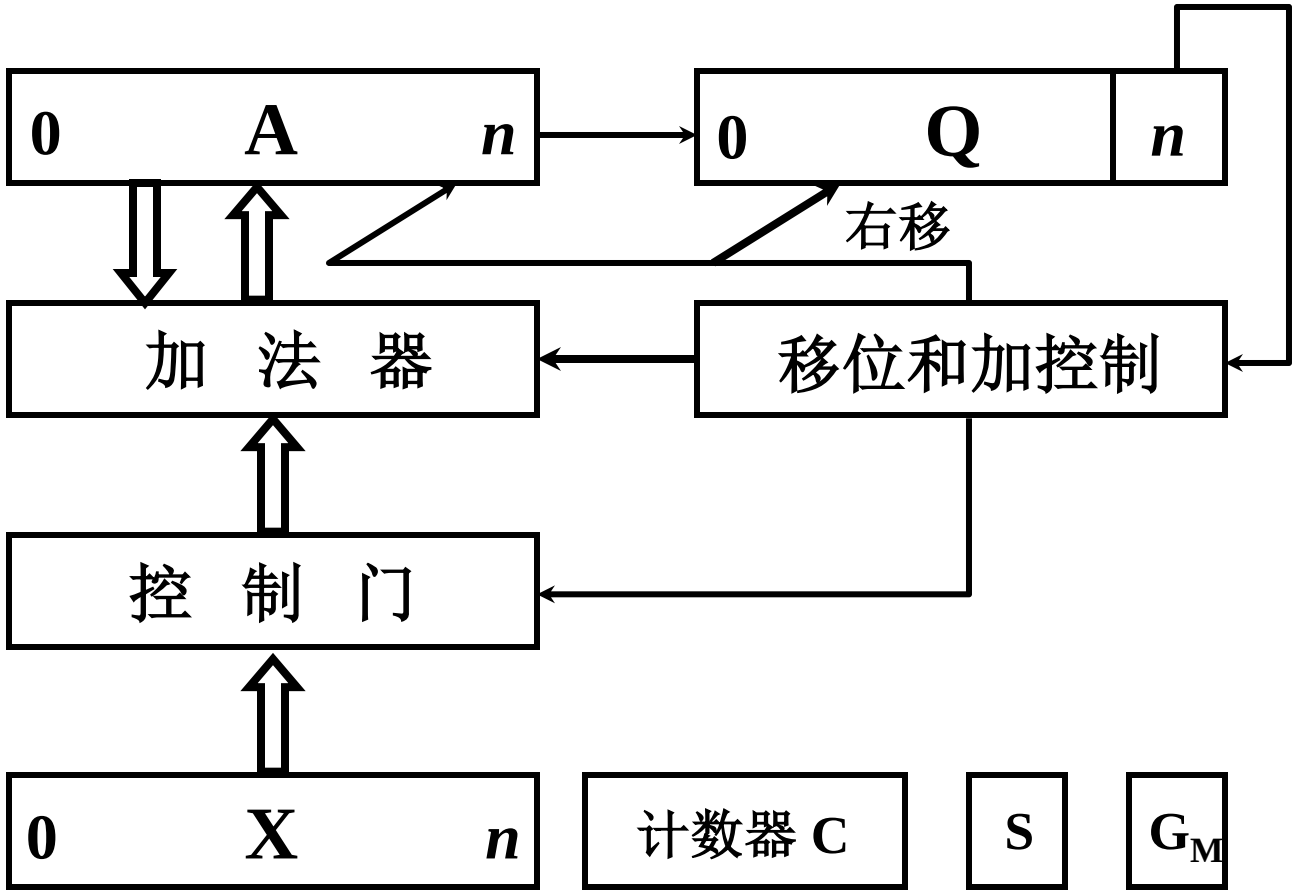
特点 绝对值运算

用移位的次数判断乘法是否结束

逻辑移位

(3) 原码一位乘的硬件配置

6.3



A、X、Q 均 *n*+1 位

移位和加受末位乘数控制

(4) 原码两位乘

6.3

原码乘　　符号位 和 数值位 部分 分开运算

两位乘　　每次用 乘数的 2 位判断 原部分积 是否加 和 如何加 被乘数

乘数 $y_{n-1}\ y_n$	新的部分积
0 0	加 “0” 后 ⟶2
0 1	x 加 1 倍的被乘数后 ⟶2
1 0	$2x$ 加 2 倍的被乘数后 ⟶2
1 1	$3x$ 加 3 倍的被乘数后 ⟶2

3 ?

$$\begin{array}{r} 4 \\ -\ 1 \\ \hline 3 \end{array} \qquad \begin{array}{r} 100 \\ -\ 01 \\ \hline 11 \end{array}$$

先 减 1 倍 的被乘数
再 加 4 倍 的被乘数

移位

(5) 原码两位乘运算规则

6.3

乘数判断位$y_{n-1}y_n$	标志位C_j	操 作 内 容
0 0	0	$z \rightarrow 2, y^* \rightarrow 2, C_j$ 保持 “0”
0 1	0	$z+x^* \rightarrow 2, y^* \rightarrow 2, C_j$ 保持 “0”
1 0	0	$z+2x^* \rightarrow 2, y^* \rightarrow 2, C_j$ 保持 “0”
1 1	0	$z-x^* \rightarrow 2, y^* \rightarrow 2,$ 置 “1” C_j
0 0	1	$z+x^* \rightarrow 2, y^* \rightarrow 2,$ 置 “0” C_j
0 1	1	$z+2x^* \rightarrow 2, y^* \rightarrow 2,$ 置 “0” C_j
1 0	1	$z-x^* \rightarrow 2, y^* \rightarrow 2, C_j$ 保持 “1”
1 1	1	$z \rightarrow 2, y^* \rightarrow 2, C_j$ 保持 “1”

共有操作 $+x^*$ $+2x^*$ $-x^*$ $\longrightarrow 2$

实际操作 $+[x^*]_{补}$ $+[2x^*]_{补}$ $+[-x^*]_{补}$ $\longrightarrow 2$ 补码移

例6.22 已知 $x = 0.111111$　$y = -0.111001$　求$[x \cdot y]_{原}$ 6.3

数值部分的运算

解：

部分积 z	乘数y MQ	C_j	说明
000.000000	00.111001	0	初态 $z_0 = 0$
000.111111			+ x^*，　$C_j = 0$
000.111111			
000.001111	11 001110	0	→ 2，得 z_1
001.111110			+ $2x^*$，$C_j = 0$
010.001101	11		
000.100011	0111 0011	0	→ 2，得 z_2
111.000001			− x^*，　$C_j = 1$
111.100100	0111		
111.111001	000111 00	1	→ 2，得 z_3
000.111111			+ x^*，　$C_j = 0$
000.111000	000111		

补码右移

补码右移

欠$4x^*$

例6.22 结果

① 乘积的符号位 $x_0 \oplus y_0 = 0 \oplus 1 = 1$

② 数值部分的运算

$$x^* \cdot y^* = 0.1\,1\,1\,0\,0\,0\,0\,0\,0\,1\,1\,1$$

则 $[x \cdot y]_{原} = 1.1\,1\,1\,0\,0\,0\,0\,0\,0\,1\,1\,1$

特点 绝对值的补码运算（用补码加减）

中间结果不负

用移位的次数判断乘法是否结束

算术移位

(6) 原码两位乘和原码一位乘比较 6.3

	原码一位乘	原码两位乘
符号位	$x_0 \oplus y_0$	$x_0 \oplus y_0$
操作数	绝对值	绝对值的补码
移位	逻辑右移	算术右移
移位次数	n	$\frac{n}{2}$（n为偶数）
最多加法次数	n	$\frac{n}{2}+1$（n为偶数）

思考　n 为奇数时，原码两位乘 移？次 最多加？次

5. 补码乘法（以小数为例） 6.3

(1) 补码一位乘运算规则

以小数为例　　设被乘数 $[x]_{补} = x_0.x_1x_2\cdots x_n$

乘数 $[y]_{补} = y_0.y_1y_2\cdots y_n$

① 被乘数任意，乘数为正

与原码乘相似　但 加 和 移位 按 补码规则 运算

乘积的符号自然形成

② 被乘数任意，乘数为负

乘数$[y]_{补}$，去掉符号位，操作同 ①

最后 加$[-x]_{补}$，校正

教材P251例6.19和例6.20

③ Booth 算法（被乘数、乘数符号任意） 6.3

设$[x]_{补} = x_0.x_1x_2 \cdots x_n$　　$[y]_{补} = y_0.y_1y_2 \cdots y_n$

$[x \cdot y]_{补} = [x\ (y_0.y_1 \cdots y_n)]_{补}$

$-[x]_{补} = +[-x]_{补}$　证明见教材P253

$= [x]_{补}(\ 0.y_1 \cdots y_n\ ) - [x]_{补} \cdot y_0$

$= [x]_{补}(y_1\ 2^{-1} + y_2 2^{-2} + \cdots + y_n 2^{-n}) - [x]_{补} \cdot y_0$

$2^{-1} = 2^0 - 2^{-1}$

$= [x]_{补}(-y_0 + y_1\ 2^{-1} + y_2 2^{-2} + \cdots + y_n 2^{-n})$

$2^{-2} = 2^{-1} - 2^{-2}$

$= [x]_{补}[-y_0 + (y_1 - y_1 2^{-1}) + (y_2 2^{-1} - y_2 2^{-2}) + \cdots + (y_n 2^{-(n-1)} - y_n 2^{-n})]$

$= [x]_{补}[(y_1 - y_0) + (y_2 - y_1)2^{-1} + \cdots + (y_n - y_{n-1})2^{-(n-1)} + (0 - y_n)2^{-n})]$

$= [x]_{补}[(y_1 - y_0) + (y_2 - y_1)2^{-1} + \cdots + (y_{n+1} - y_n)2^{-n}]$

附加位 y_{n+1}

④ Booth 算法递推公式

6.3

$[z_0]_{补} = 0$

0 / 1 / -1

$[z_1]_{补} = 2^{-1}\{(y_{n+1} - y_n)[x]_{补} + [z_0]_{补}\}$ $\quad y_{n+1} = 0$

$\vdots$

$[z_n]_{补} = 2^{-1}\{(y_2 - y_1)[x]_{补} + [z_{n-1}]_{补}\}$

$[x \cdot y]_{补} = [z_n]_{补} + (y_1 - y_0)[x]_{补}$ 最后一步不移位

如何实现 $y_{i+1} - y_i$?

y_i y_{i+1}	$y_{i+1} - y_i$	操作
0 0	0	→1
0 1	1	$+[x]_{补}$ →1
1 0	-1	$+[-x]_{补}$ →1
1 1	0	→1

例6.23 已知 $x = +0.0011$ $y = -0.1011$ 求$[x \cdot y]_{补}$

6.3

解：

部分积z	乘数y_n	附加位y_{n+1}	说明
00.0000	1.0101	0	
11.1101			$+[-x]_{补}$
11.1101			
11.1110	1 1010	1	→1
00.0011			$+[x]_{补}$
00.0001	1		
00.0000	11 101	0	→1
11.1101			$+[-x]_{补}$
11.1101	11		
11.1110	111 10	1	→1
00.0011			$+[x]_{补}$
00.0001	111		
00.0000	1111	0	→1
11.1101			$+[-x]_{补}$
11.1101	1111		最后一步不移位

算术移位

双符号位

$[x]_{补} = 0.0011$

$[y]_{补} = 1.0101$

$[-x]_{补} = 1.1101$

$\therefore [x \cdot y]_{补}$

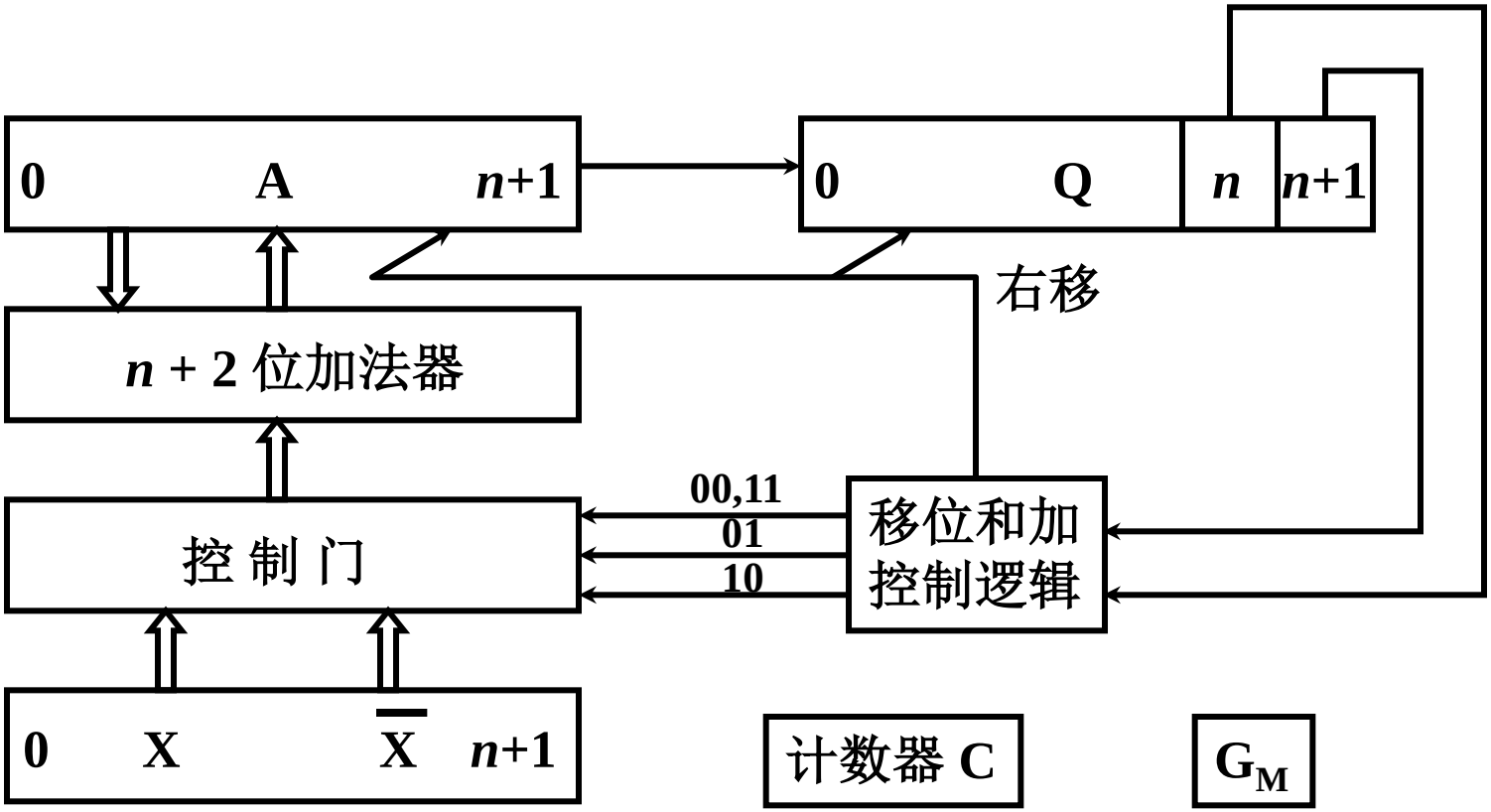
$=1.11011111$

$x \cdot y = -0.00100001$

Booth 算法

(2) Booth 算法的硬件配置

6.3



A、X、Q　均 $n+2$ 位

移位和加受末两位乘数控制

乘法小结

6.3

- 整数乘法与小数乘法完全相同
  可用 逗号 代替小数点

- 原码乘　符号位 单独处理
  补码乘　符号位 自然形成

除法亦同

- 原码乘去掉符号位运算　即为无符号数乘法

- 不同的乘法运算需有不同的硬件支持

拓展*

6.3

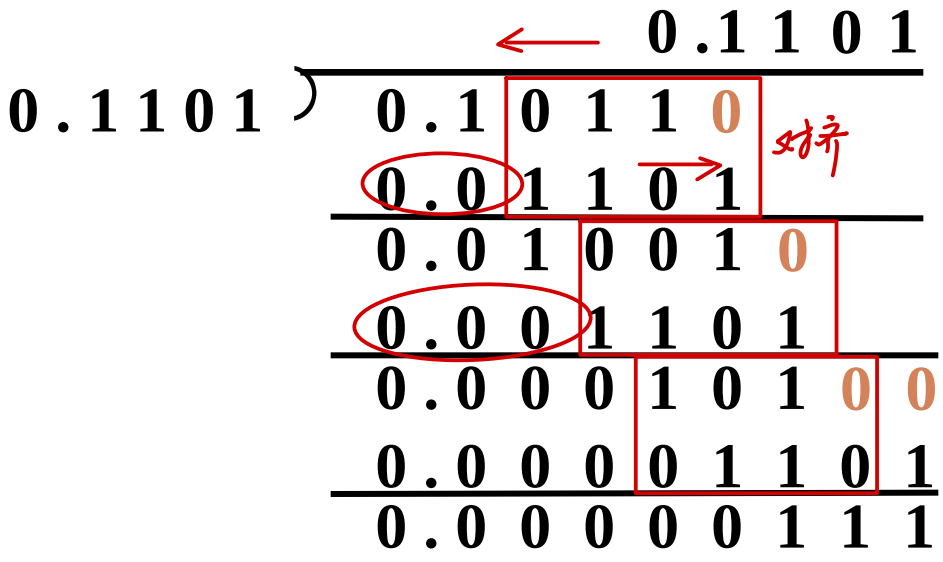
世界最难的乘法：大数乘法与人工智能
https://www.sohu.com/a/330965377_100273862

电脑硬件入门——基础之CPU架构解读
https://zhuanlan.zhihu.com/p/65840506

四、除法运算 6.3

1. 分析笔算除法

$x = -0.1011$　$y = 0.1101$　求 $x \div y$



✓商符单独处理

? 心算上商

? 余数不动低位补“0”
减右移一位的除数

? 上商位置不固定

$x \div y = -0.1101$　商符心算求得

余数为：$-0.0000\,0111$

与被除数符号一致

2. 笔算除法和机器除法的比较 6.3

笔算除法	机器除法
商符单独处理	符号位异或形成
心算上商	$\|x\|-\|y\|>0$ 上商 1
	$\|x\|-\|y\|<0$ 上商 0 恢复
余数 不动 低位补“0” 前面部分不参与运算	余数 左移一位 低位补“0”
减右移一位 的除数	减 除数
2 倍字长加法器	1 倍字长加法器 可做减法
上商位置 不固定	在寄存器 最末位上商

3. 原码除法

6.3

以小数为例

$$[x]_{原} = x_0.x_1x_2 \quad \cdots x_n$$

$$[y]_{原} = y_0.y_1y_2 \quad \cdots y_n$$

$$\left[\frac{x}{y}\right]_{原} = (x_0 \oplus y_0).\ \frac{x^*}{y^*}$$

式中 $x^* = 0.x_1x_2 \cdots x_n$ 为 x 的绝对值

$y^* = 0.y_1y_2 \cdots y_n$ 为 y 的绝对值

商的符号位单独处理 $x_0 \oplus y_0$

数值部分为绝对值相除 $\frac{x^*}{y^*}$

约定

小数定点除法 $x^* < y^*$　　整数定点除法 $x^* > y^*$

被除数不等于 0

除数不能为 0

进一步说明

- **除前预处理**

  ①若被除数=0且除数≠0，或定点整数除法中出现|被除数|<|除数|，则商为0，不再继续

  ②若被除数≠0、除数=0，则发生“除数为0”异常

  ③若被除数和除数都为0，则有些机器产生一个不发信号的NaN，即“quiet NaN”

  当被除数和除数都≠ 0，且商≠ 0时，才进一步进行除法运算。

(1) 恢复余数法

有减法 → 补码

6.3

例6.24 $x = -0.1011$ $y = -0.1101$ 求 $[\frac{x}{y}]_{原}$

解：$[x]_{原} = 1.1011$ $[y]_{原} = 1.1101$ $[y^*]_{补} = 0.1101$ $[-y^*]_{补} = 1.0011$

① $x_0 \oplus y_0 = 1 \oplus 1 = 0$

②

被除数（余数）	商	说　明
0.1011	0.0000	
1.0011		$+[-y^*]_{补}$
1.1110	0	余数为负，上商0
0.1101		恢复余数 $+[y^*]_{补}$
0.1011	0	恢复后的余数
1.0110	0	← 1
1.0011		$+[-y^*]_{补}$
0.1001	01	余数为正，上商 1
1.0010	01	← 1
1.0011		$+[-y^*]_{补}$

逻辑左移

逻辑左移

被除数（余数）	商	说　明
0.0101	011	余数为正，上商 1
0.1010	011	← 1
1.0011		$+[-y^*]_{补}$
1.1101	0110	余数为负，上商 0
0.1101		恢复余数 $+[y^*]_{补}$
0.1010	0110	恢复后的余数
1.0100	0110	← 1
1.0011		$+[-y^*]_{补}$
0.0111	01101	余数为正，上商 1

逻辑左移

$\frac{x^*}{y^*} = 0.1101$　$\therefore [\frac{x}{y}]_{原} = 0.1101$

上商 5 次

第一次上商判溢出

移 4 次

余数为：-0.00000111

余数为正　上商 1

余数为负　上商 0，恢复余数　　最终余数需要 $*2^{-n}$

(2) 不恢复余数法　加减交替法　6.3

- 恢复余数法运算规则

余数　$R_i>0$　上商“1”，　$R_{i+1}=2R_i-y^*$

余数　$R_i<0$　上商“0”，　R_i+y^*　　恢复余数

$$R_{i+1}=2(R_i+y^*)-y^*=2R_i+y^*$$

- 不恢复余数法运算规则

$R_i>0$　上商“1”　　$R_{i+1}=2R_i-y^*$

$R_i<0$　上商“0”　　$R_{i+1}=2R_i+y^*$

当前余数

加减交替

已恢复余数

当n+1步余数为负时，需加上 y^* 得到第n+1步正确的余数，

最后的余数为：$R_n\times 2^{-n}$

例6.25 $x = -0.1011$ $y = -0.1101$ 求 $[\frac{x}{y}]_{原}$ 6.3

解：

0.1011	0.0000	
1.0011		$+[-y^*]_{补}$
1.1110	0	余数为负，上商 0
1.1100	0	← 1
0.1101		$+[y^*]_{补}$
0.1001	01	余数为正，上商 1
1.0010	01	← 1
1.0011		$+[-y^*]_{补}$
0.0101	011	余数为正，上商 1
0.1010	011	← 1
1.0011		$+[-y^*]_{补}$
1.1101	0110	余数为负，上商 0
1.1010	0110	← 1
0.1101		$+[y^*]_{补}$
0.0111	01101	余数为正，上商 1

逻辑左移

$[x]_{原} = 1.1011$

$[y]_{原} = 1.1101$

$[y^*]_{补} = 0.1101$

$[-y^*]_{补} = 1.0011$

加减交替

例6.25 结果

① $x_0 \oplus y_0 = 1 \oplus 1 = 0$

② $\frac{x^*}{y^*} = 0.1101$

$\therefore \ [\frac{x}{y}]_{原} = 0.1101$

[余数]$_{原}$为：$[-0.0111*2^{-4}]_{原}=1.00000111$

特点：上商 $n+1$ 次

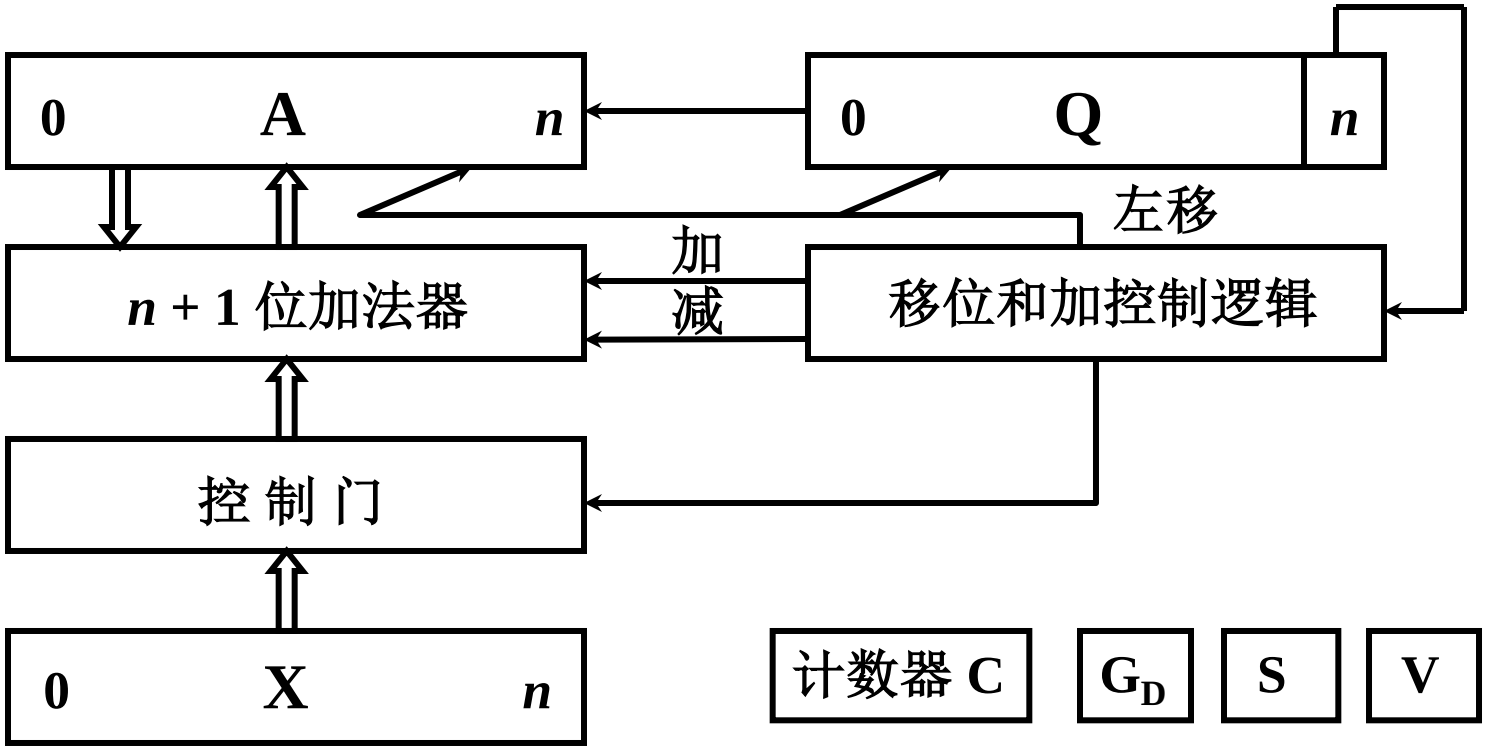
第一次上商判溢出

移n次，加$n+1$次

用移位的次数判断除法是否结束

(3) 原码加减交替除法硬件配置

6.3



A、X、Q 均 $n+1$ 位

用 Q_n 控制加减交替

4. 补码除法

6.3

(1) 商值的确定

① 比较被除数和除数绝对值的大小

补码负数上商

➢ x 与 y 同号　用减法

$x = 0.1011$　$[x]_{补} = 0.1011$　$[x]_{补} = 0.1011$　$x^* > y^*$

$y = 0.0011$　$[y]_{补} = 0.0011$　$+[-y]_{补} = 1.1101$　$[R_i]_{补}$与$[y]_{补}$同号

$[R_i]_{补} = 0.1000$　“够减”

$x = -0.0011$　$[x]_{补} = 1.1101$　$[x]_{补} = 1.1101$　$x^* < y^*$

$y = -0.1011$　$[y]_{补} = 1.0101$　$+[-y]_{补} = 0.1011$　$[R_i]_{补}$与$[y]_{补}$异号

$[R_i]_{补} = 0.1000$　“不够减”

➢x 与 y 异号　用加法

$x = 0.1011$　$[x]_{补} = 0.1011$

$y = -0.0011$　$[y]_{补} = 1.1101$

$$\begin{array}{rl} & [x]_{补} = 0.1011 \\ + & [y]_{补} = 1.1101 \\ \hline & [R_i]_{补} = 0.1000 \end{array}$$

$x^* > y^*$

$[R_i]_{补}$与$[y]_{补}$异号

“够减”

$x = -0.0011$　$[x]_{补} = 1.1101$

$y = 0.1011$　$[y]_{补} = 0.1011$

$$\begin{array}{rl} & [x]_{补} = 1.1101 \\ + & [y]_{补} = 0.1011 \\ \hline & [R_i]_{补} = 0.1000 \end{array}$$

$x^* < y^*$

$[R_i]_{补}$与$[y]_{补}$同号

“不够减”

小结

$[x]_{补}$和$[y]_{补}$	求 $[R_i]_{补}$	$[R_i]_{补}$与 $[y]_{补}$
同号	$[x]_{补} - [y]_{补}$	同号，“够减”
异号	$[x]_{补} + [y]_{补}$	异号，“够减”

② 商值的确定　末位恒置“1”法

$\times.\underbrace{\times\times\times\times}1$

$[x]_{补}$与 $[y]_{补}$同号 正商	0．	原码	1	按原码上商	“够减”上“1” “不够减”上“0”
$[x]_{补}$与 $[y]_{异}$异号 负商	1．	反码	1	按反码上商	“够减”上“0” “不够减”上“1”

小 结

$[x]_{补}$与 $[y]_{补}$	商	$[R_i]_{补}$与 $[y]_{补}$	商　值
同　号	正	够减（同号） 不够减（异号）	1 0　原码上商
异　号	负	够减（异号） 不够减（同号）	0 1　反码上商

简 化 为

$[R_i]_{补}$与$[y]_{补}$	商值
同　号	1
异　号	0

(2) 商符的形成

除法过程中自然形成

$[x]_{补}$和$[y]_{补}$同号 → $[x]_{补}-[y]_{补}$ → 比较$[R_i]_{补}$和$[y]_{补}$

同号(够)“1”

异号(不够)“0”

原码上商

小数除法 第一次“不够”上“0”

正商

$[x]_{补}$和$[y]_{补}$异号 → $[x]_{补}+[y]_{补}$ → 比较$[R_i]_{补}$和$[y]_{补}$

异号(够)“0”

同号(不够)“1”

反码上商

小数除法 第一次“不够”上“1”

负商

(3) 新余数的形成

加减交替

$[R_i]_{补}$和 $[y]_{补}$	商	新余数
同 号	1	$2[R_i]_{补} + [-y]_{补}$
异 号	0	$2[R_i]_{补} + [\ y\ ]_{补}$

补码加减交替法的算法规则

1. 符号位参加运算，商的符号自然形成。
2. 被除数与除数同号，被除数减去除数。

   被除数与除数异号，被除数加上除数。
3. 余数与除数同号，商上**1**，余数左移**1**位减去除数/余数与除数异号，商上**0**，余数左移**1**位加上除数。（余数左移加上或减去除数就得到新余数）
4. 采用校正法包括符号位在内，应重复步骤**3** **n+1**次.

商的校正原则

- 精度要求不高时，将商的最低位恒置1，最大误差2^{-n} 反码商加补码商

例6.26

设 $x = -0.1011$　$y = 0.1101$

求 $[\frac{x}{y}]_{补}$ 并还原成真值

解：$[x]_{补} = 1.0101$　$[y]_{补} = 0.1101$　$[-y]_{补} = 1.0011$

$\therefore [\frac{x}{y}]_{补} = 1.0011$

则 $\frac{x}{y} = -0.1101$

余数R	商	说明
1.0101	0.0000	$[x]_{补}$
0.1101		$[x]_{补}$与$[y]_{补}$异号做加法 $+[y]_{补}$
0.0010	1	$[R]_{补}$与 $[y]_{补}$同号上“1”
0.0100	1	←1
1.0011		$[R]_{补}$与$[y]_{补}$同号做减法 $+[-y]_{补}$
1.0111	10	$[R]_{补}$与 $[y]_{补}$异号上“0”
0.1110	10	←1
0.1101		$[R]_{补}$与$[y]_{补}$异号做加法 $+[y]_{补}$
1.1011	100	$[R]_{补}$与 $[y]_{补}$异号上“0”
1.0110	100	←1
0.1101		$[R]_{补}$与$[y]_{补}$异号做加法 $+[y]_{补}$
0.0011	1001	$[R]_{补}$与 $[y]_{补}$同号上“1”
0.0110	10011	←1 末位恒置“1”

逻辑左移

$[余数]_{补}$为：$[-0.0110*2^{-4}]_{补} = 1.11111010$

例6.26

设 $x = -0.1011$　$y = 0.1101$

求 $[\frac{x}{y}]_{补}$ 并还原成真值

解：$[x]_{补} = 11.0101$　$[y]_{补} = 0.1101$　$[-y]_{补} = 11.0011$

11.0101	0.0000	
00.1101		异号做加法
00.0010	1	同号上“1”
00.0100	1	←1
11.0011		$+[-y]_{补}$
11.0111	10	异号上“0”
10.1110	10	←1
00.1101		$+[y]_{补}$
11.1011	100	异号上“0”
11.0110	100	←1
00.1101		$+[y]_{补}$
00.0011	1001	同号上“1”
00.0110	10011	←1 末位恒置“1”

算术左移

$\therefore [\frac{x}{y}]_{补} = 1.0011$

则 $\frac{x}{y} = -0.1101$

$[余数]_{补}$为：

$[-0.0110*2^{-4}]_{补}$

$=1.11111010$

(4) 补码除法小结

- 除数和被除数用补码表示
- 直接用补码除，求出反码商
- 再修正为近似的补码商
- 补码除法共上商 $n+1$ 次（末位恒置 1）
  第一次为商符
- 加 n 次　移 n 次
- 第一次商可判溢出
- 精度误差最大为 2^{-n}

6.4 浮点四则运算

一、浮点加减运算

$$x = S_x \cdot 2^{j_x} \qquad y = S_y \cdot 2^{j_y}$$

1. 对阶

(1) 求阶差

$$\Delta j = j_x - j_y = \begin{cases} = 0 & j_x = j_y \quad \text{已对齐} \\ > 0 & j_x > j_y \begin{cases} x \text{ 向 } y \text{ 看齐} & S_x \leftarrow 1,\ j_x - 1 \\ y \text{ 向 } x \text{ 看齐} & S_y \rightarrow 1,\ j_y + 1 \end{cases} \\ < 0 & j_x < j_y \begin{cases} x \text{ 向 } y \text{ 看齐} & S_x \rightarrow 1,\ j_x + 1 \\ y \text{ 向 } x \text{ 看齐} & S_y \leftarrow 1,\ j_y - 1 \end{cases} \end{cases}$$

可能溢出

(2) 对阶原则

小阶向大阶看齐

为什么？防止溢出

6.4 浮点四则运算

一、浮点加减运算

$$x = S_x \cdot 2^{j_x} \qquad y = S_y \cdot 2^{j_y}$$

2. 尾数相加

尾数相加与定点数加减法相同

例如 $x = 0.1101 \times 2^{01}$ $y = (-0.1010) \times 2^{11}$

求 $[x+y]_{补}$

解： $[x]_{补} = 00, 01; 00.1101$ $[y]_{补} = 00, 11; 11.0110$

1. 对阶

① 求阶差 $[\Delta j]_{补} = [j_x]_{补} - [j_y]_{补}$

$$\begin{array}{rr} & 00, 01 \\ + & 11, 01 \\ \hline & 11, 10 \end{array}$$

阶差为负（-2） $\therefore S_x \longrightarrow 2$ $j_x + 2$

② 对阶 $[x]_{补'} = 00, 11; 00.0011$

2. 尾数求和

$$\begin{array}{rll} & [S_x]_{补'} & = 00.0011 \\ + & [S_y]_{补} & = 11.0110 \\ \hline & & 11.1001 \end{array}$$

对阶后的$[S_x]_{补'}$

$x+y=(-0.0111) \times 2^{11}$

$\therefore [\,]_{补} = 00, 11; 11.1001$

3. 规格化

6.4

(1) 规格化数的定义

$$r = 2 \qquad \frac{1}{2} \le |S| < 1$$

对阶
加减
规格化

(2) 规格化数的判断

$S>0$	规格化形式	$S<0$	规格化形式
真值	0.1×× …×	真值	–0.1×× …×
原码	0.1×× …×	原码	1.1×× …×
补码	0.1×× …×	补码	1.0×× …×
反码	0.1×× …×	反码	1.0×× … ×

原码　不论正数、负数，第一数位为1

补码　符号位和第 1 数位不同（反码亦然）

特例

$$S = -\frac{1}{2} = -0.100\cdots0$$

$$[S]_{原} = 1.100\cdots0$$

$$[S]_{补} = 1.100\cdots0$$

$\therefore$ $[-\frac{1}{2}]_{补}$ 不是规格化的数

$$S = -1$$

机器判别方便

$$[S]_{补} = 1.000\cdots0$$

$\therefore$ $[-1]_{补}$ 是规格化的数

(3) 左规

尾数←1，阶码减 1，直到数符和第一数位不同为止

上例　$[x+y]_{补} = 00,\ 11;\ 11.\ 1001$　$x+y=(-0.0111)\times 2^{11}$

左规后　$[x+y]_{补} = 00,\ 10;\ 11.\ 0010$

$\therefore\ x + y = (-0.1110)\times 2^{10}$

(4) 右规

当 尾数溢出（ >1）时， 需 右规

即尾数出现 01.××…×或 10.××…×时

尾数→1，阶码加 1

例6.27 $x = 0.1101\times 2^{10}$ $y = 0.1011\times 2^{01}$

一般2位

求 $x+y$（除阶符、数符外，阶码取 3 位，尾数取 6 位）

解：$[x]_{补} = 00, 010; 00.\ 110100$

$[y]_{补} = 00, 001; 00.\ 101100$

① 对阶

$[\Delta j]_{补} = [j_x]_{补} - [j_y]_{补}$ = 00, 010

\+ 11, 111

100, 001

阶差为 +1　　$\therefore S_y \longrightarrow 1,\ j_y+1$

$\therefore [y]_{补'} = 00, 010; 00.\ 010110$

② 尾数求和

$[S_x]_{补}$ = 00. 110100

\+ $[S_y]_{补'}$ = 00. 010110　　对阶后的$[S_y]_{补'}$

01. 001010　　尾数溢出需右规

③ 右规

$[x+y]_{补} = 00,010;\ 01.001010$

右规后

$[x+y]_{补} = 00,011;\ 00.100101$

$\therefore\ x+y = 0.100101 \times 2^{11}$

4. 舍入

在 对阶 和 右规 过程中，可能出现 尾数末位丢失 引起误差，需考虑舍入

(1) 0 舍 1 入法： 移掉的最高位为1，尾数末尾加1；为0，舍掉。

(2) 恒置 “1” 法： 右移时，丢掉低位值，就将结果最低位置1。

例 6.28 $x=(-\frac{5}{8})\times 2^{-5}$ $y=(\frac{7}{8})\times 2^{-4}$

求 $x-y$（除阶符、数符外，阶码取 3 位，尾数取 6 位）

解：$x=(-0.101000)\times 2^{-101}$　　$y=(0.111000)\times 2^{-100}$

$[x]_{补}=11,011;\ 11.011000$　　$[y]_{补}=11,100;\ 00.111000$

① 对阶

$$[\Delta j]_{补}=[j_x]_{补}-[j_y]_{补}$$

$$\begin{array}{r} 11,011 \\ +\ 00,100 \\ \hline 11,111 \end{array}$$

阶差为 -1　　$\therefore S_x \longrightarrow 1,\quad j_x+1$

$\therefore [x]_{补'}=11,100;\ 11.101100$

② 尾数求和

$$[S_x]_{补'} = 11.\ 101100$$
$$+\ [-S_y]_{补} = 11.\ 001000$$
$$110.\ 110100$$

③ 右规

$$[x+y]_{补} = 11,\ 100;\ 10.\ 110100$$

右规后

$$[x+y]_{补} = 11,\ 101;\ 11.\ 011010$$

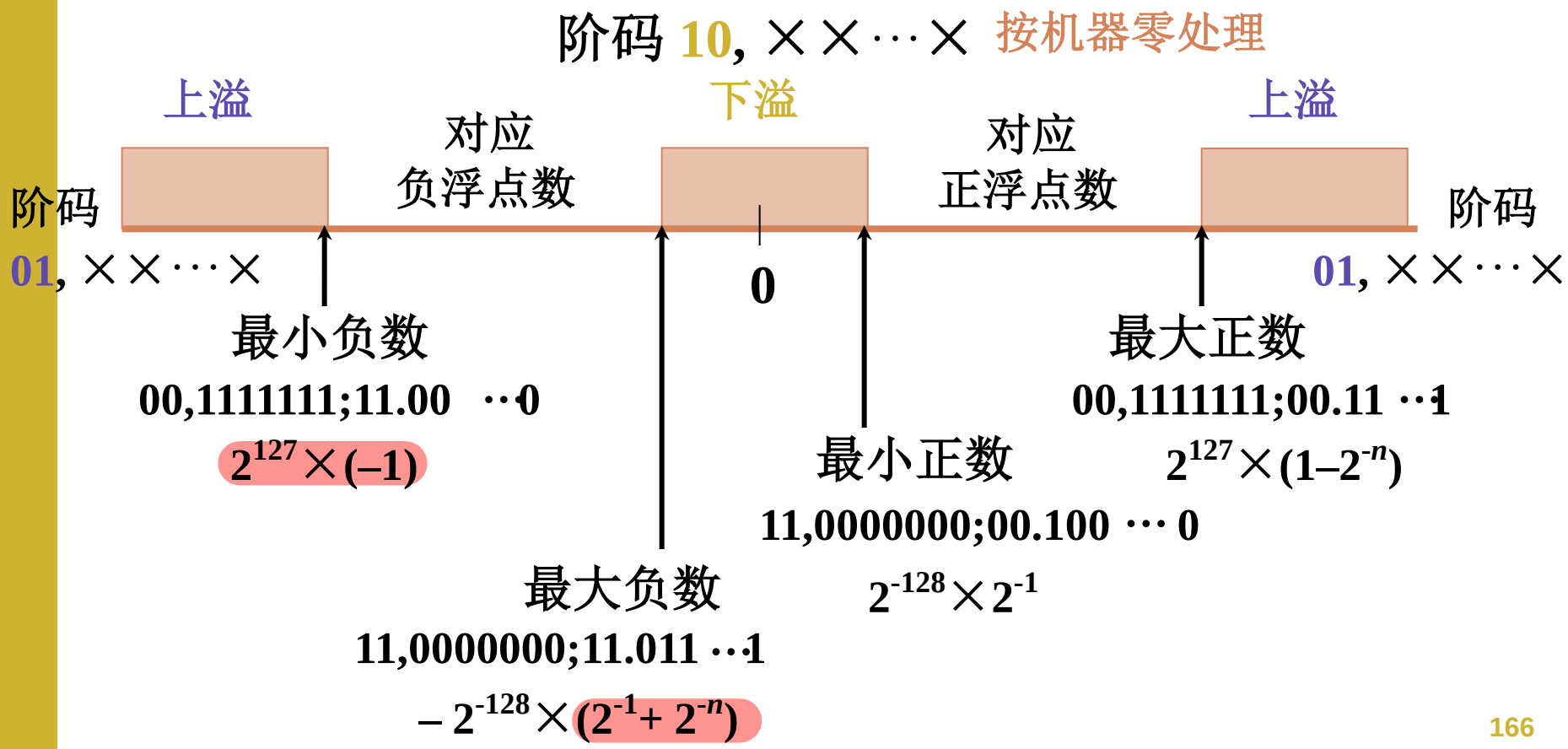
$$\therefore\ x - y = (-0.100110) \times 2^{-11}$$

$$= \left(-\frac{19}{32}\right) \times 2^{-3}$$

5. 溢出判断

6.4

设机器数为补码，尾数为规格化形式，并假设阶符取 2 位，阶码取 7 位，数符取 2 位，尾数取 n 位，则该补码在数轴上的表示为



二、浮点乘除运算

6.4

$$x = S_x \cdot 2^{j_x} \qquad y = S_y \cdot 2^{j_y}$$

1. 乘法

$$x \cdot y = (S_x \cdot S_y) \times 2^{j_x + j_y}$$

2. 除法

$$\frac{x}{y} = \frac{S_x}{S_y} \times 2^{j_x - j_y}$$

3. 步骤

(1) 阶码运算

(2) 尾数乘除运算

(3) 规格化

(4) 舍入

(5) 判溢出

4. 浮点运算部件

阶码运算部件，尾数运算部件

浮点数运算总结

- 加减运算流程

  对阶 → 尾数相加减 → 规格化

  → 舍入操作 → 判断是否溢出

- 乘除运算流程

  阶码相加(减) →尾数相乘(除) →规格化

  →舍入操作 → 判断是否溢出

十进制数的加减运算

- 有的机器有十进制加减法指令，用于对BCD码进行加减运算。所以这些机器中必须要有相应的十进制加减运算逻辑。
- 以NBCD码（8421码）为例，讨论十进制整数的加减运算。
- 一般规定数符在最高位 1100：正，1101：负

  或 0：正， 1：负

例如：+2039 1100 0010 0000 0011 1001

或 0 0010 0000 0011 1001

-1265 1101 0001 0010 0110 0101

1 0001 0010 0110 0101

符号和数值部分分开处理！

十进制加法运算举例

例1 25+31=56

```
  0010  0101
+ 0011  0001
-----------
  0101  0110
```

例2 25+39=64

```
  0010  0101
+ 0011  1001
-----------
  0101  1110
        0110
-----------
  0110  0100
```

$(1110)_2>9$ 需“+6”校正

例3 27+39=66

```
  0010  0111
+ 0011  1001
-----------
  0101  0000
      1 0110
-----------
  0110  0110
```

低位有进位，则进到高位，同时该低位“+6”校正

例4 99+99=198

```
  1001  1001
+ 1001  1001
-----------
1 0010  0010
      1 0110
-----------
1 0011  1000
  0110
-----------
1 1001  1000
```

问题：本位和在什么范围内需“+6”校正？

- 结果<=9时，不需校正；大于9或有进位时， 需“+6”校正
- 最高位有进位时，发生溢出

大于9：1010,1011,1100,1101,1110,1111

有进位：10000,10001,10010和10011

最大为19: 2x9+1=19, 范围为10~19

移码加/减运算*

- 用于浮点数阶码运算
- 符号位和数值部分可以一起处理
- 运算公式（假定在一个n位ALU中进行加法运算）

$$[E1]_{移}+[E2]_{移}=2^{n-1}+E1+2^{n-1}+E2=2^n+E1+E2=[E1+E2]_{补} \quad (\text{mod } 2^n)$$

$$[E1]_{移}-[E2]_{移}=[E1]_{移}+[-[E2]_{移}]_{补}=2^{n-1}+E1+2^n-[E2]_{移}$$

$$=2^{n-1}+E1+2^n-2^{n-1}-E2$$

$$=2^n+E1-E2=[E1-E2]_{补} \quad (\text{mod } 2^n)$$

结论：移码的和、差等于和、差的补码！

- 运算规则　　**补码和移码的关系：符号位相反、数值位相同！**

① 加法：直接将$[E1]_{移}$和$[E2]_{移}$进行模2^n加，然后对结果的符号取反。

② 减法：先将减数$[E2]_{移}$求补（各位取反，末位加1），然后再与被减数 $[E1]_{移}$进行模2^n相加，最后对结果的符号取反。

③ 溢出判断：进行模2^n相加时，如果两个加数的符号相同，并且与和数的符号也相同，则发生溢出。

移码加/减运算*

例1： 用四位移码计算“–7+（– 6）”和“–3 + 6”的值。

解：$[-7]_{移}$ = 0001　　$[-6]_{移}$= 0010　　$[-3]_{移}$= 0101　　$[6]_{移}$= 1110

$[-7]_{移}$ + $[-6]_{移}$ = 0001 + 0010 = 0011 （两个加数与结果符号都为0，溢出）

$[-3]_{移}$ + $[6]_{移}$ = 0101 + 1110 = 0011， 符号取反后为 1011，其真值为+3

问题：$[-7+(-6)]_{移}$=?　　$[-3+(6)]_{移}$ =?

例2： 用四位移码计算“–7 –（– 6）”和“–3 – 5”的值。

解：$[-7]_{移}$ = 0001　　$[-6]_{移}$= 0010　　$[-3]_{移}$= 0101　　$[5]_{移}$= 1101

$[-7]_{移}$ – $[-6]_{移}$ = 0001 + 1110 = 1111， 符号取反后为 0111，其真值为–1。

$[-3]_{移}$ – $[5]_{移}$ = 0101 + 0011 = 1000，符号取反后为 0000，其真值为– 8。

(1) 阶码运算规则*

①补码运算规则

②移码运算规则　$[x]_{移} = 2^n + j_x \quad (2^n > j_x \geq -2^n)$

$$[j_x]_{移} + [j_y]_{移} = 2^n + j_x + 2^n + j_y$$

$$= 2^n + (2^n + j_x + j_y) = 2^n + [j_x + j_y]_{移}$$

又：$[j_y]_{补} = 2^{n+1} + j_y$

$$[j_x]_{移} + [j_y]_{补} = 2^n + j_x + 2^{n+1} + j_y = 2^{n+1} + [j_x + j_y]_{移}$$

$$= [j_x + j_y]_{移} \quad (\mathrm{mod}\ 2^{n+1})$$

同理有　$[j_x]_{移} + [-j_y]_{补} = [j_x - j_y]_{移}$

移码和补码的数值位相同、符号位相反

为防止溢出，采用双符号位的阶码加法器*

规定：移码的最高符号位

恒用 “ 0 ”参加运算

结果符号：

00	正常	正
01		负
?		
10	上溢	
11	下溢	

例： $x=011$ $y=110$ 求$[x\pm y]_{移}$

$[x]_{移}=01,011$ $[y]_{补}=00,110$ $[-y]_{补}=11,010$

$[x+y]_{移}=[x]_{移}+[y]_{补}=01,011+00,110$

$=10,001$ （上溢，9）

$[x-y]_{移}=[x]_{移}+[-y]_{补}=01,011+11,010$

$=00101$ $(\mathrm{mod}\ 2^5, -3)$

例：已知$x=0.1011\times2^{0110}$ $y=-0.0101\times2^{0011}$，试用浮点数运算方法计算x*y。要求浮点数的格式为：阶码6位（2位阶符），补码表示，尾数5位（1位数符），补码表示，并要求为规格化浮点数。

解 ① 对x及y按所要求的浮点数格式编码

x已是规格化数，可直接编码

$$[x]_{浮}=00,0110;0.1011$$

Y不是规格化数，先将y化为规格化数再编码

$$y=-0.0101\times2^{0011}=-0.1010\times2^{0010}$$

$$[y]_{浮}=00,0010;1.0110$$

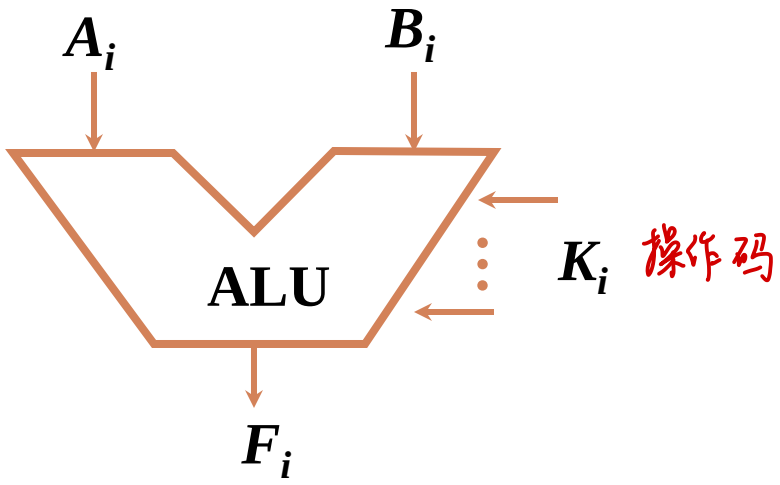
② 求x*y

- 阶码相加　　E=Ex+Ey=00,0110+00,0010=00,1000
- 尾数相乘　　$[M]_{补}=[Mx*My]_{补}=11.10010010$
- 　规格化　左规
  - 积的尾数M左移一位　　$[M]_{补}=11.0010010$
  - 积的阶码E减1　　E=00,1000+11,1111=00,0111
- 舍入　按“0”舍“1”入法，$[M]_{补}=1.0010$
- 检查是否溢出　　阶符为00,无溢出

∴ [x*y]浮=00,0111;1,0010　即$x*y=-0.1110\times2^{7}$

6.5 算术逻辑单元

一、ALU 电路



组合逻辑电路

K_i 不同取值

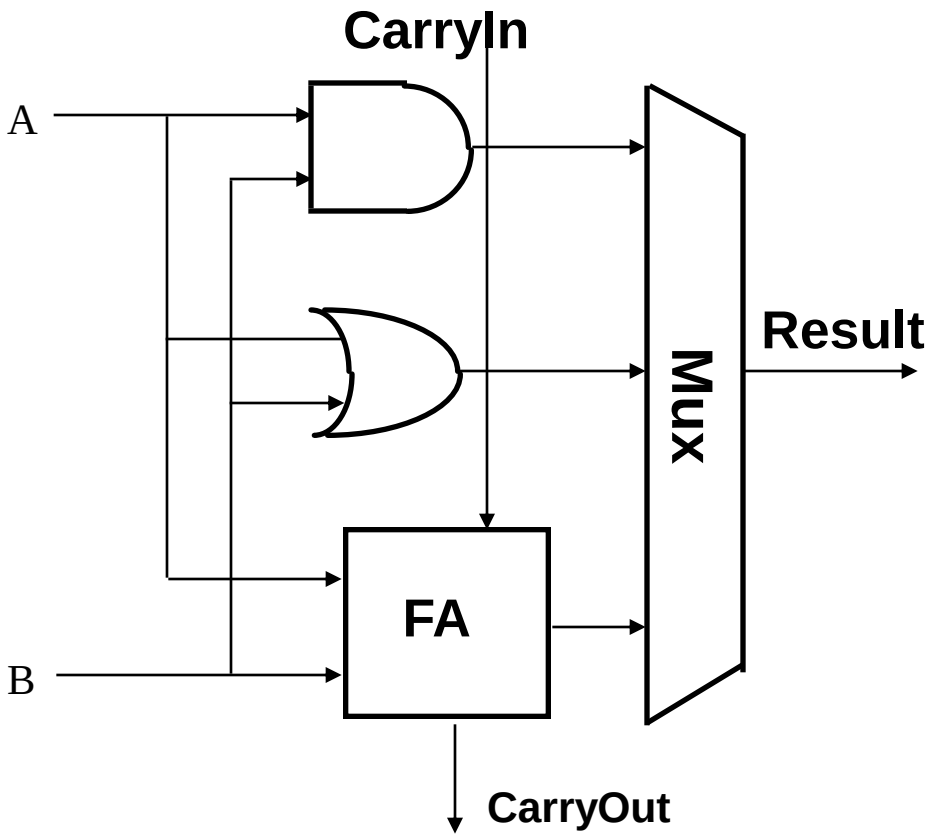
F_i 不同

四位 ALU　74181

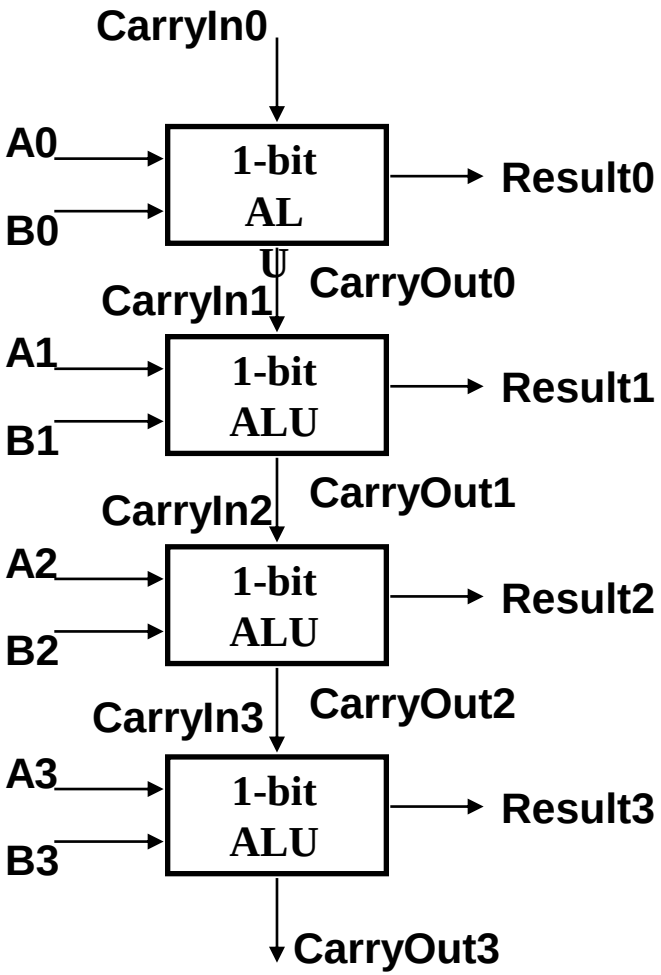
A 4-bit ALU

进位链

1-bit ALU



4-bit串行 ALU

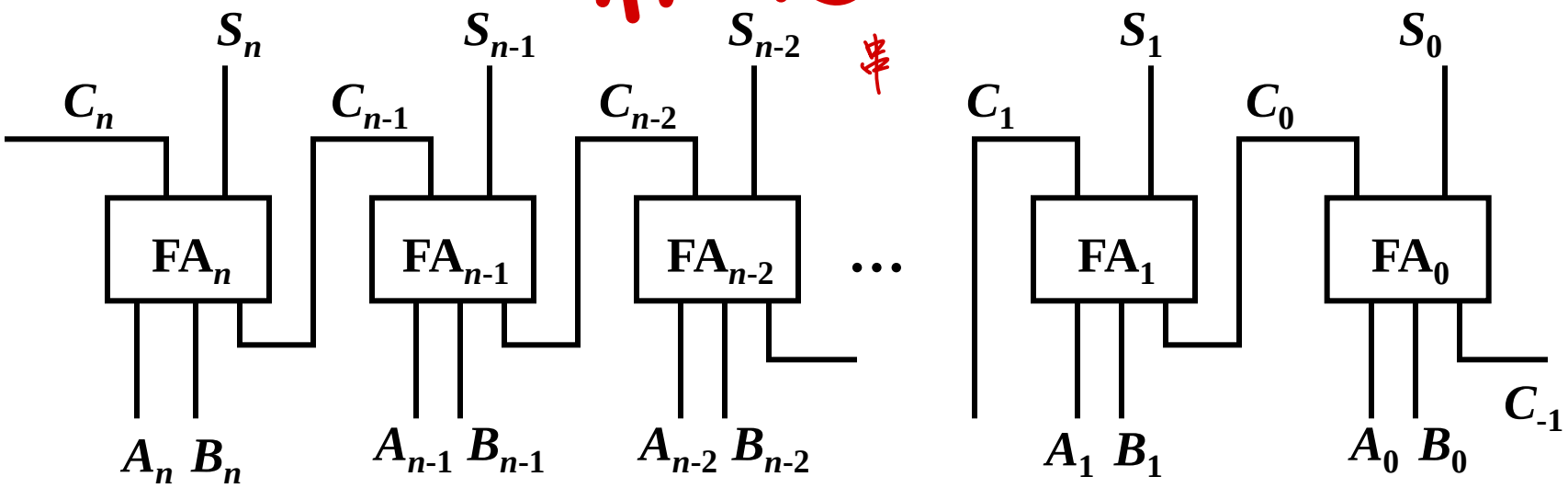


MUX是什么？（数字电路课学过）

关键路径延迟长，速度慢！

二、快速进位链

1. 并行加法器



和 $S_i = \overline{A_i}\,\overline{B_i}\,C_{i-1} + \overline{A_i}\,B_i\,\overline{C_{i-1}} + A_i\,\overline{B_i}\,\overline{C_{i-1}} + A_i\,B_i\,C_{i-1}$

进位 $C_i = \overline{A_i}\,B_i\,C_{i-1} + A_i\,\overline{B_i}\,C_{i-1} + A_i\,B_i\,\overline{C_{i-1}} + A_i\,B_i\,C_{i-1}$

$= A_i\,B_i + (A_i + B_i)C_{i-1}$

$d_i = A_i\,B_i$ 本地进位　　$t_i = A_i + B_i$ 传送条件

则 $C_i = d_i + t_i C_{i-1}$

2. 串行进位链

进位链　　传送进位的电路

串行进位链　　进位串行传送

以 4 位全加器为例，每一位的进位表达式为

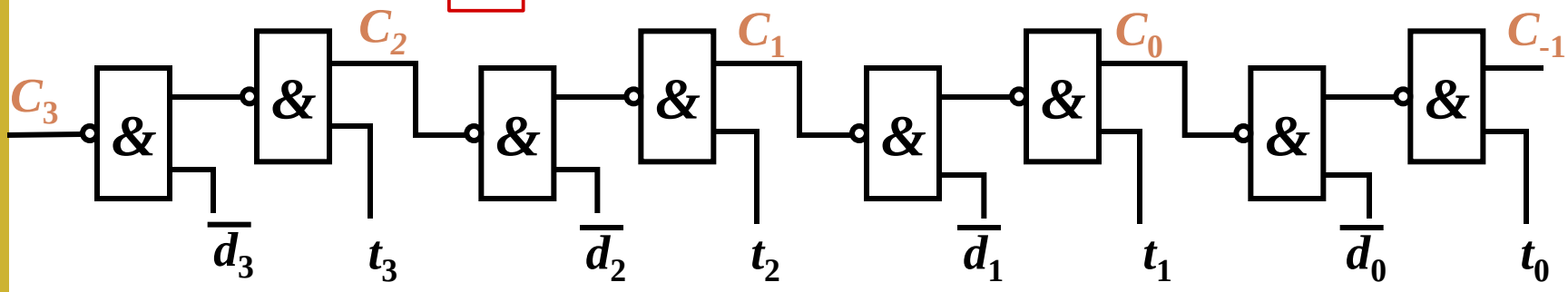
$$C_0 = d_0 + t_0 C_{-1} = \overline{\overline{d_0} \cdot \overline{t_0 C_{-1}}}$$

$$C_1 = d_1 + t_1 C_0$$

$$C_2 = d_2 + t_2 C_1$$

$$C_3 = d_3 + t_3 C_2$$

设与非门的级延迟时间为t_y



4位 全加器产生进位的全部时间为 $8t_y$

n 位全加器产生进位的全部时间为 $2nt_y$

3. 并行进位链（先行进位，跳跃进位）

n 位加法器的进位同时产生　　以 4 位加法器为例

$$C_0 = d_0 + t_0 C_{-1}$$

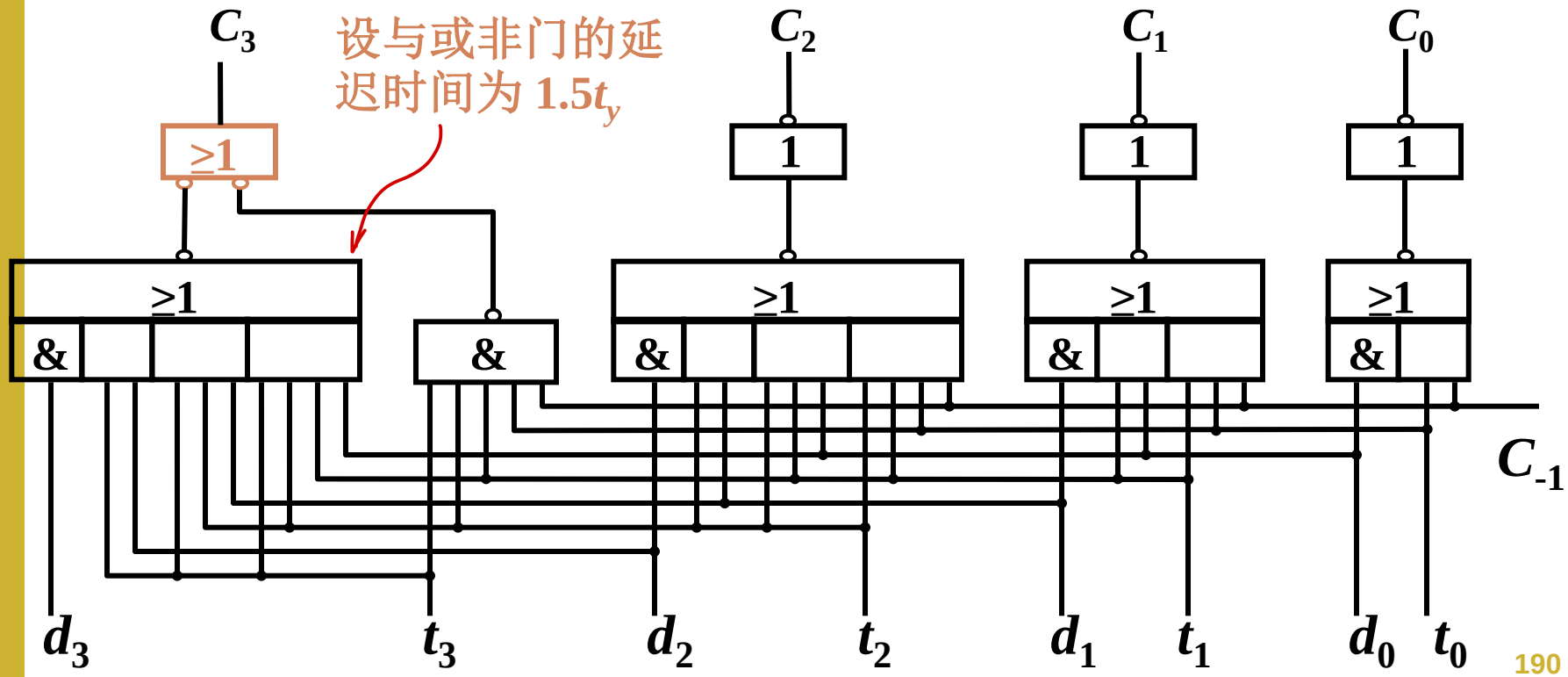
$$C_1 = d_1 + t_1 C_0 = d_1 + t_1 d_0 + t_1 t_0 C_{-1}$$

$$C_2 = d_2 + t_2 C_1 = d_2 + t_2 d_1 + t_2 t_1 d_0 + t_2 t_1 t_0 C_{-1}$$

$$C_3 = d_3 + t_3 C_2 = d_3 + t_3 d_2 + t_3 t_2 d_1 + t_3 t_2 t_1 d_0 + t_3 t_2 t_1 t_0 C_{-1}$$

当 $d_i\ t_i$ 形成后，只需 $2.5t_y$ 产生全部进位

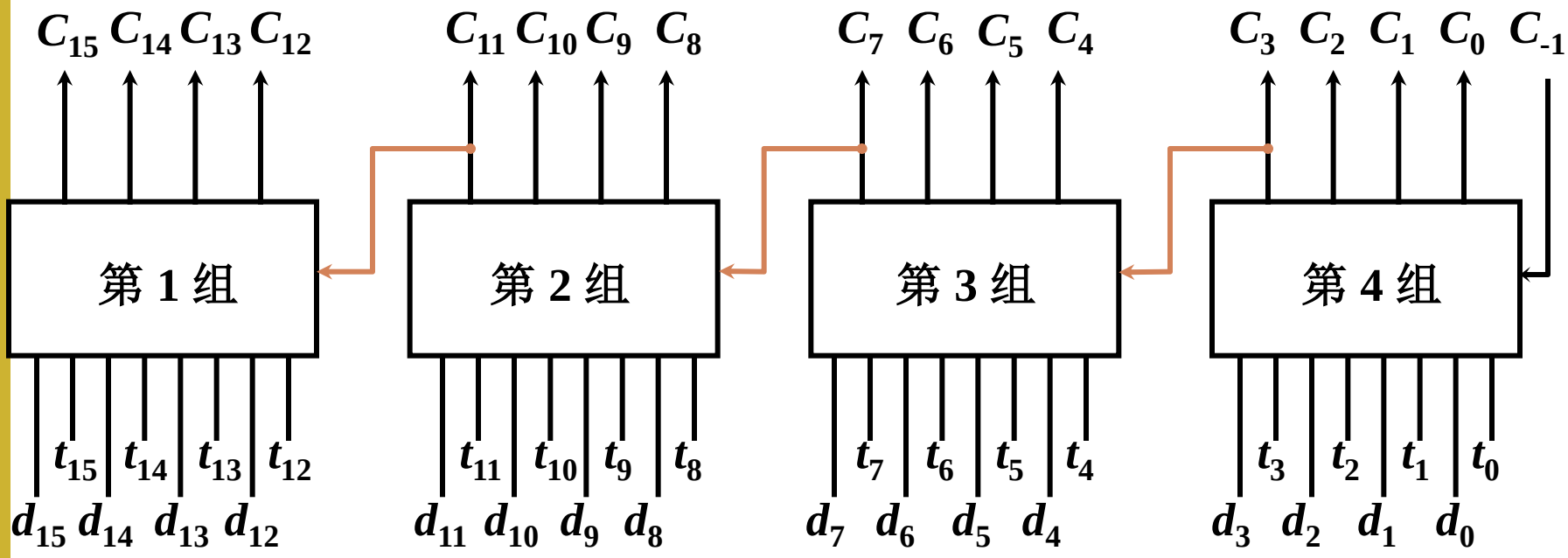
$8t_y$



(1) 单重分组跳跃进位链

n 位全加器分若干小组，小组中的进位同时产生，
小组与小组之间采用串行进位

以 $n = 16$ 为例



当 d_i t_i 形成后

经 2.5 t_y 产生 $C_3 \sim C_0$

5 t_y 产生 $C_7 \sim C_4$

7.5 t_y 产生 $C_{11} \sim C_8$

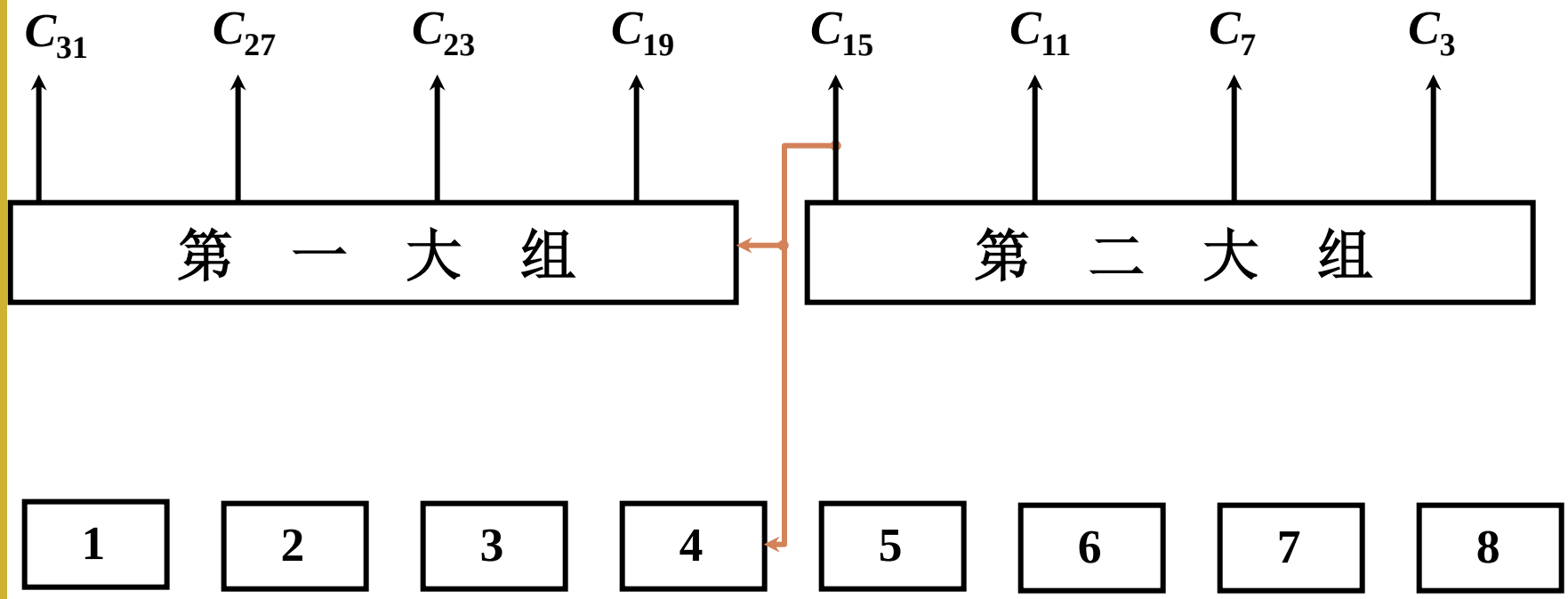
10 t_y 产生 $C_{15} \sim C_{12}$

32 t_y

(2) 双重分组跳跃进位链

n 位全加器分若干大组，大组中又包含若干小组。每个大组中小组的最高位进位同时产生。大组与大组之间采用串行进位。

以 $n = 32$ 为例



（3) 双重分组跳跃进位链 大组进位分析

以第 8 小组为例

$$C_3 = d_3 + t_3C_2 = \underbrace{d_3 + t_3d_2 + t_3t_2d_1 + t_3t_2t_1d_0}_{D_8} + \underbrace{t_3t_2t_1t_0}_{T_8}C_{-1}$$

D_8 小组的本地进位　与外来进位无关

T_8 小组的传送条件　与外来进位无关　传递外来进位

同理　第 7 小组　$C_7 = D_7 + T_7C_3$

第 6 小组　$C_{11} = D_6 + T_6C_7$

第 5 小组　$C_{15} = D_5 + T_5C_{11}$

进一步展开得

$C_3 = D_8 + T_8C_{-1}$

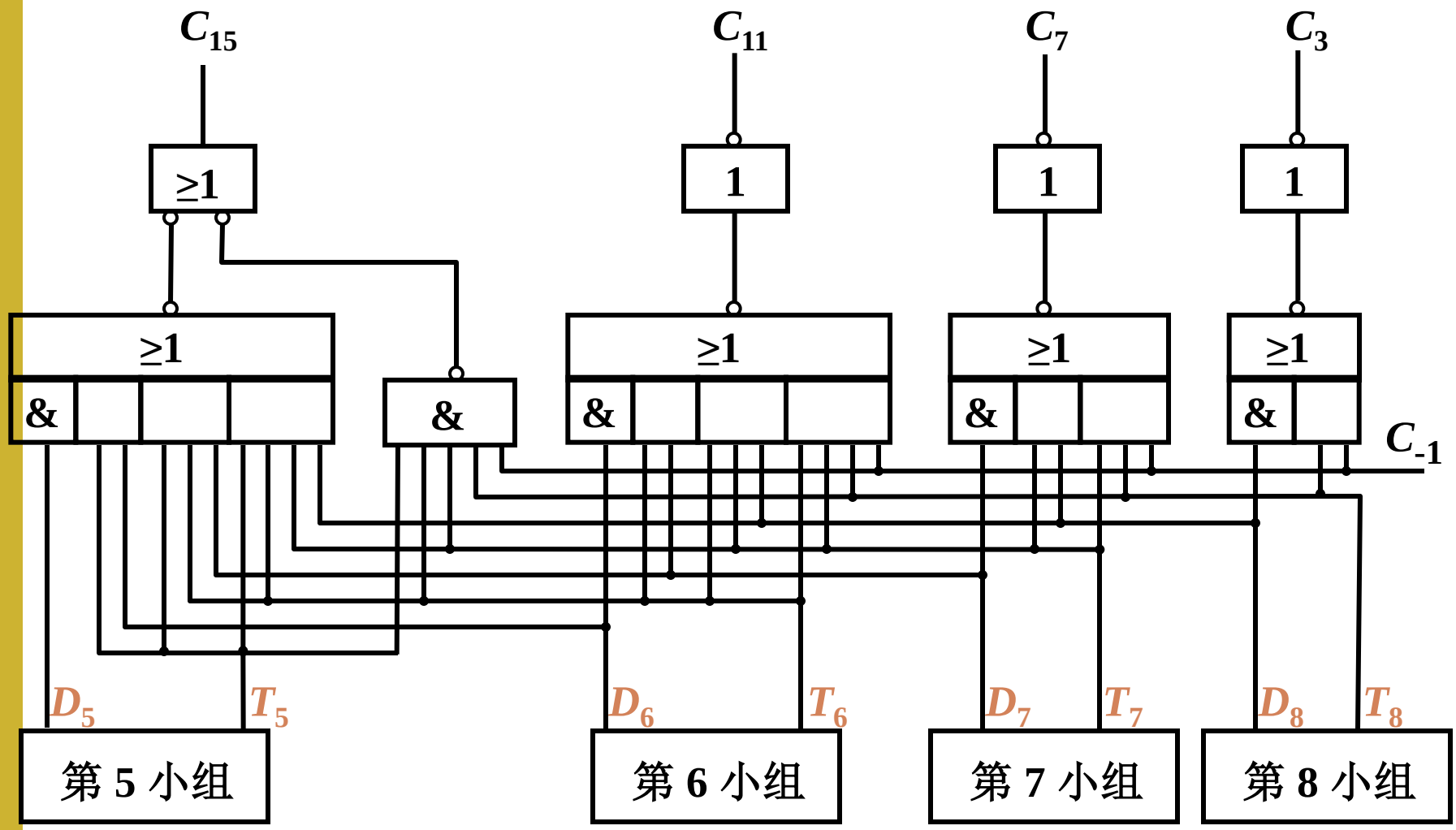
$C_7 = D_7 + T_7C_3 = D_7 + T_7D_8 + T_7T_8C_{-1}$

$C_{11} = D_6 + T_6C_7 = D_6 + T_6D_7 + T_6T_7D_8 + T_6T_7T_8C_{-1}$

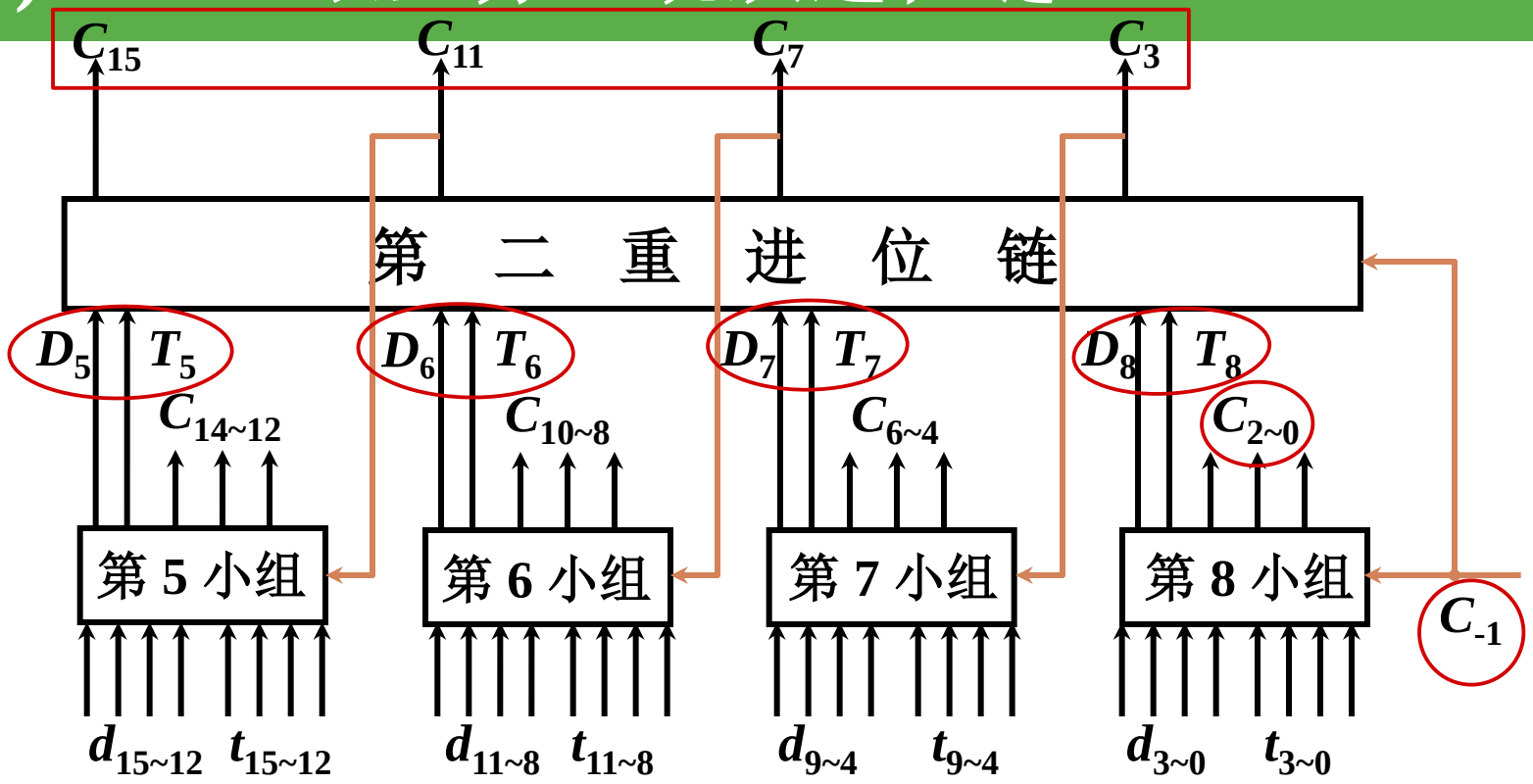
$C_{15} = D_5 + T_5C_{11} = D_5 + T_5D_6 + T_5T_6D_7 + T_5T_6T_7D_8 + T_5T_6T_7T_8C_{-1}$

(4) 双重分组跳跃进位链的 大组 进位线路

以第 2 大组为例



(6) n =16 双重分组跳跃进位链



当 $d_i\ t_i$ 和 C_{-1} 形成后　经 2.5 t_y　产生 C_2、C_1、C_0、$D_5\sim D_8$、$T_5\sim T_8$

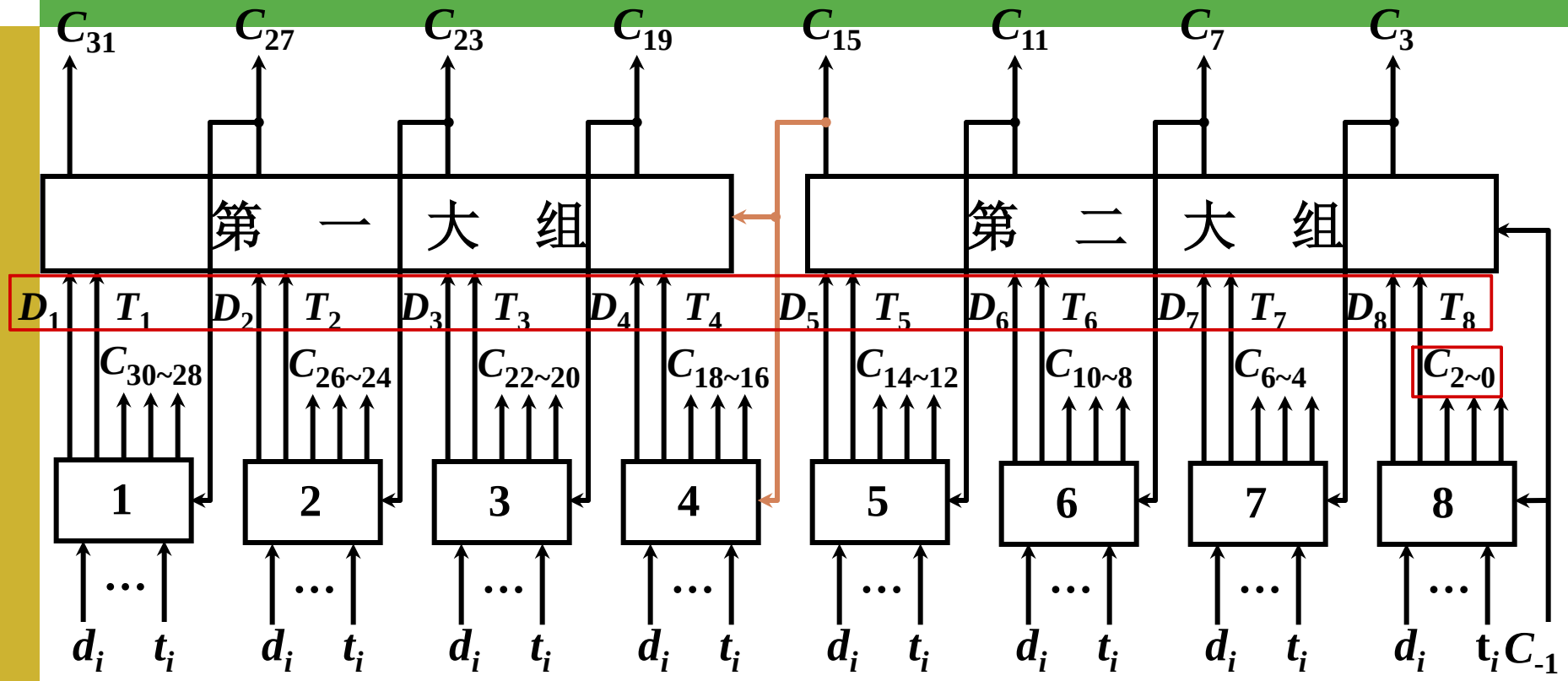
经 5 t_y　产生 C_{15}、C_{11}、C_7、C_3

经 7.5 t_y　产生 $C_{14}\sim C_{12}$、$C_{10}\sim C_8$、$C_6\sim C_4$

串行进位链　经 3 2 t_y　产生　全部进位

单重分组跳跃进位链　经 1 0 t_y　产生　全部进位

(7) n =32 双重分组跳跃进位链



当 d_i t_i 形成后

经 2.5 t_y 产生 C_2、C_1、C_0、$D_1 \sim D_8$、$T_1 \sim T_8$

5 t_y 产生 C_{15}、C_{11}、C_7、C_3

7.5 t_y 产生 $C_{18} \sim C_{16}$、$C_{14} \sim C_{12}$、$C_{10} \sim C_8$、$C_6 \sim C_4$
C_{31}、C_{27}、C_{23}、C_{19}

10 t_y 产生 $C_{30} \sim C_{28}$、$C_{26} \sim C_{24}$、$C_{22} \sim C_{20}$

计算机组成

TIME
TO
21:10 B. S. BEGAWAN
22:25 MELBOURNE
21:35 LONDON-GATWK
21:45 KUALA LUMPUR
22:45 LONDON-HEATH
22:05 TORONTO
22:05 TOWNSVILLE
22:10 ADELAIDE
22:15 FRANKFURT
22:30 CAIRO
SIA SQ 11
BA 12
AC 859
QANTAS QF 62
QANTAS QF 8
SIA SQ 26
SIA SQ 24
B26
B23
B25
C42
C31
GATE CHANGE
GATE CHANGE
GATE CHANGE
Vs BELOW, OR PROCEED TO THE NEXT LOUNGE

任重而道远

第四章　存储器

计算机与信息学院

Contents

1 4.1 概 述

2 √4.2 主存储器

3 √4.3 高速缓冲存储器

4 4.4 辅助存储器

大　纲

四. 存储器

(一)存储器的分类

(二)存储器的层次化结构

(三)半导体随机存取存储器

1. SRAM存储器的工作原理

2.DRAM存储器的工作原理

(四)只读存储器

(五)主存储器与CPU的连接

(六)双口RAM和多模块存储器

(七)高速缓冲存储器(Cache)

1. 程序访问的局部性原理

2. Cache的基本工作原理

3.Cache和主存之间的映射方式

4. Cache中主存块的替换算法

5.Cache写策略

（八） 虚拟存储器

1. 虚拟存储器的基本概念

2. 页式虚拟存储器

3. 段式虚拟存储器

4. 段页式虚拟存储器

5. TLB（快表）

大　纲

三、存储器层次结构

（一）存储器的分类

（二）存储器的层次化结构

（三）半导体随机存取存储器

1. SRAM 存储器

2. DRAM 存储器

3. 只读存储器

4. Flash 存储器

（四）主存储器与 CPU 的连接

（五）双口 RAM 和多模块存储器

（六）高速缓冲存储器（Cache）

1. Cache 的基本工作原理

2. Cache 和主存之间的映射方式

3. Cache 中主存块的替换算法

4. Cache 写策略

（七）虚拟存储器

1. 虚拟存储器的基本概念

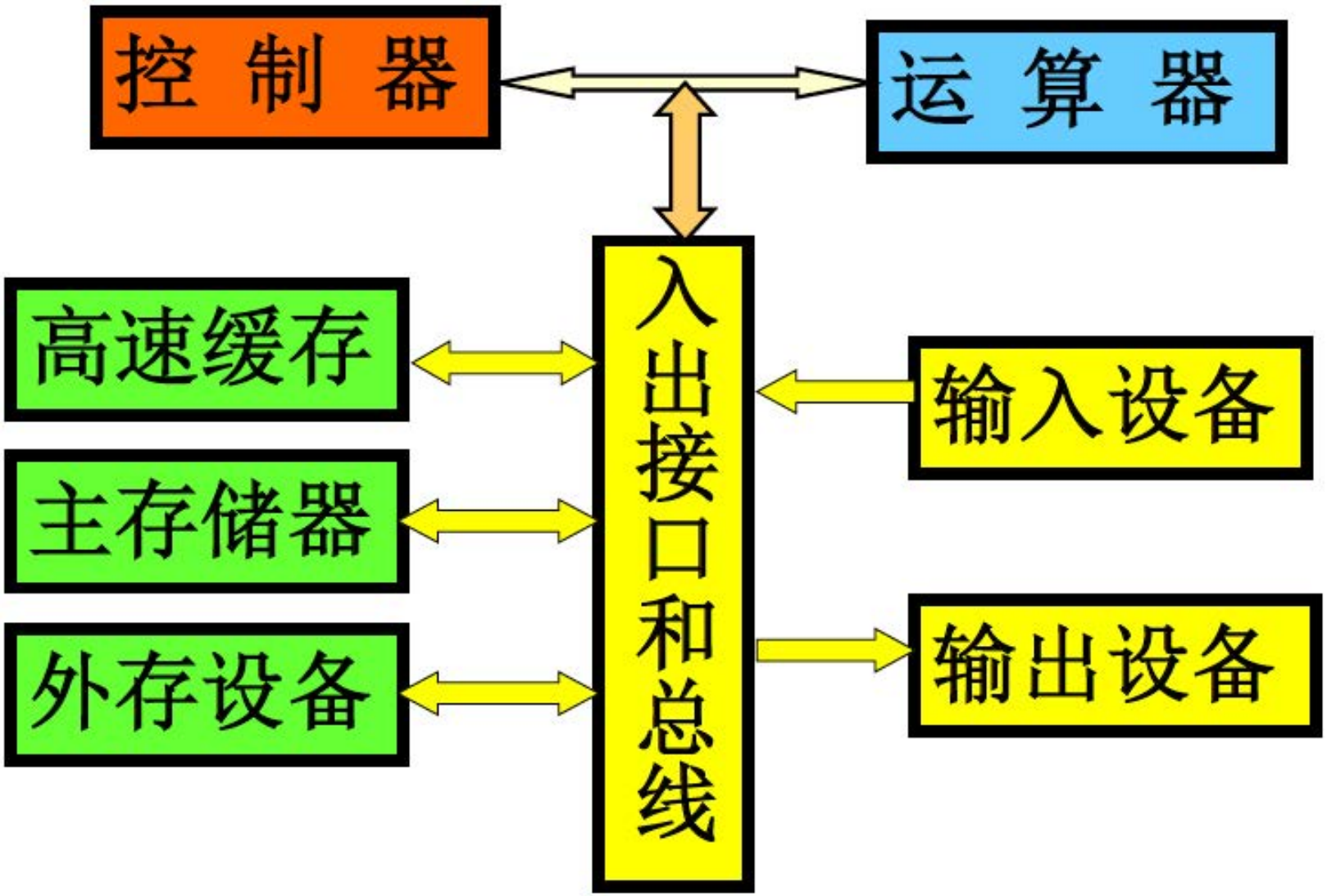
2. 页式虚拟存储器

3. 段式虚拟存储器

4. 段页式虚拟存储器

5. TLB（快表）

计算机硬件系统



存储器地位和作用

- 存储程序使计算机走向通用。
- 计算机中用来存放程序和数据的部件，是Von Neumann结构计算机的重要组成，是计算机的中心。

以前以运算器为中心

- 程序和数据的特点
  - 源程序、汇编程序、机器语言程序
  - 各种类型的数据
  - 共同点：二进制数据

对存储介质的基本要求

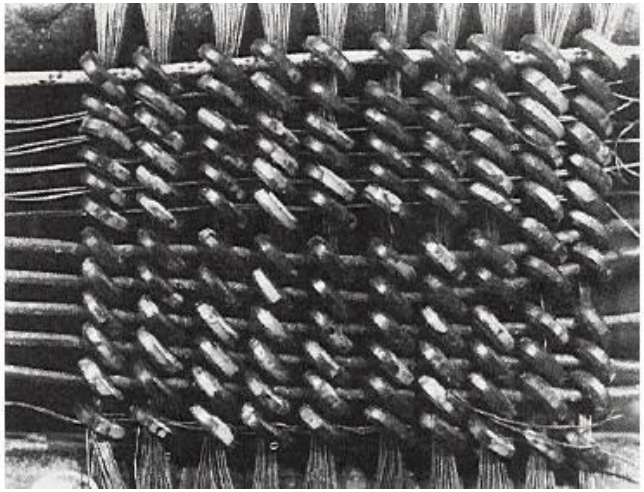
- 能够有两个稳定状态来表示二进制中的“0”和“1”
- 容易识别
- “0”和“1”两个状态能方便地进行转换
- 几种常用的存储方式
  - 磁颗粒、半导体(电平/电容)、光

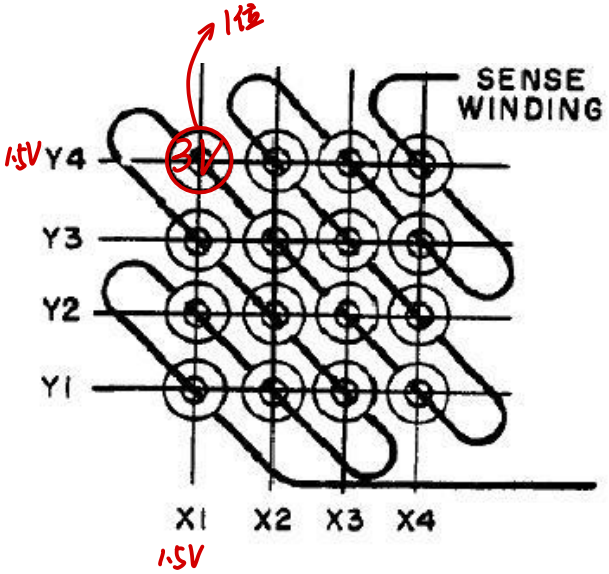
早期存储器

- 水银延迟线存储器
- EDSAC,1949
- Maurice Wilkes
- 1967年Turing奖
- 存储原理
  - 水波

磁芯存储器

- 圆柱型陶瓷上涂磁粉
- 手工穿线，水手结
- 消磁后重写





半导体存储器

- 存储原理
  - MOS管寄生电容
  - 触发器
- 访问机制
  - 随机访问
- 分类
  - ROM、RAM
  - SRAM、DRAM

4.1 概 述

回顾：存储器基本术语

- 记忆单元 （存储基元 / 存储元 / 位元） （Cell)
  - 具有两种稳态的能够表示二进制数码0和1的物理器件
- 存储单元 / 编址单位（Addressing Unit)
  - 主存中具有相同地址的位构成一个存储单元，也称为一个编址单位
- 存储体/ 存储矩阵 / 存储阵列（Bank)
  - 所有存储单元构成一个存储阵列
- 编址方式（Addressing Mode)
  - 字节编址、按字编址
- 存储器地址寄存器（Memory Address Register - MAR)
  - 用于存放主存单元地址的寄存器
- 存储器数据寄存器（ Memory Data Register-MDR (MBR) ）
  - 用于存放主存单元中的数据的寄存器

投票

最多可选1项

设置

现代计算机主存是_____。

A 按字编址

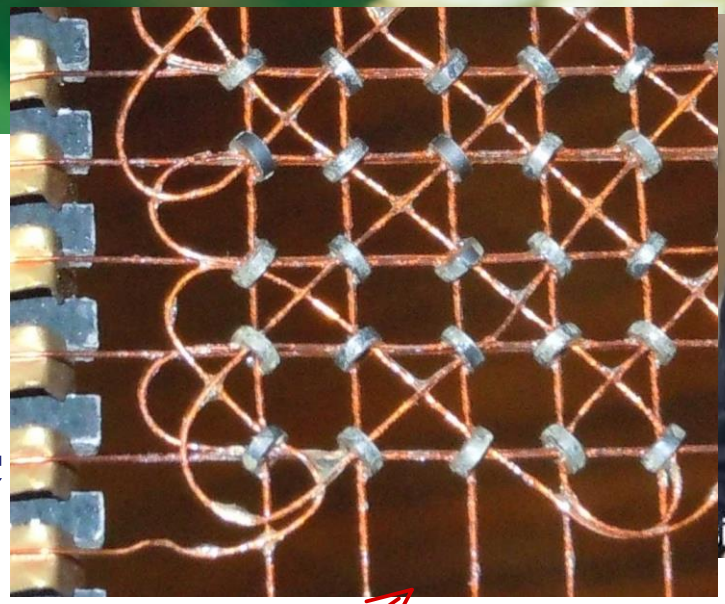
B 按字节编址

提交

4.1 概 述

- 存储器可以分成哪些类型？
- 现代计算机为什么要对存储分层？

4.1 概 述



一、存储器分类

1. 按存储介质分类

(1) 半导体存储器　　TTL，MOS　　易失

(2) 磁表面存储器　　磁盘（Disk）、磁带 （Tape）

(3) 磁芯存储器　　硬磁材料、环状元件
（破坏性读出）

(4) 光盘存储器　　激光、磁光材料

非易失

4.1 概 述

2. 按存取方式分类

① 随机存储器（Random Access Memory,RAM）

- 静态RAM：以触发器原理寄存信息
- 动态RAM：以电容充电原理寄存信息

② 只读存储器（Read Only Memory,ROM）

- 掩膜型只读存储器（Masked ROM，MROM）
- 可编程只读存储器（Programmable ROM，PROM）
- EPROM、EEPROM，Flash Memory……

4.1 概 述

③ 顺序存取存储器(Sequential Access Memory, SAM)

- 数据按顺序从存储载体的始端读出或写入，因而存取时间的长短与信息所在位置有关。例如：磁带。

④ 直接存取存储器 (Direct Access Memory, DAM)

- 直接定位到要读写的数据块，在读写某个数据块时按顺序进行。例如：磁盘。

⑤ 相联存储器(Associate Memory/Content Addressed Memory CAM)

- 按内容检索到存储位置进行读写。例如：快表。

3. 按在计算机中的作用分类

4.1

- 存储器
  - 主存储器
    - RAM
      - 静态 RAM（Statle RAM）
      - 动态 RAM（Dynamic RAM）
    - ROM
      - MROM
      - PROM
      - EPROM
      - EEPROM
  - Flash Memory
  - 辅助存储器　磁盘 磁带 光盘
  - 缓存Cache

施敏　Non-Volatile Memory
1967年　NAND flash 关键技术
非挥发MOS场效应记忆晶体管（NVSM）的发明者

4. 按断电后信息的可保存性分类

- **非易失（不挥发）性存储器(Non-volatile Memory)**
  - **信息可一直保留， 不需电源维持。**

  （如： ROM、磁表面存储器、光存储器等）
- **易失（挥发）性存储器(Volatile Memory)**
  - **电源关闭时信息自动丢失。（如：RAM、Cache等）**

5. 按功能/容量/速度/所在位置分类

- 寄存器(Register)
  - 封装在CPU内，用于存放当前正在执行的指令和使用的数据
  - 用触发器实现，速度快，容量小（几十K）
- 高速缓存(Cache)
  - 位于CPU内部或附近，用来存放当前要执行的局部程序段和数据
  - 用SRAM实现，速度可与CPU匹配，容量小（几MB）
- 内存储器MM（主存储器Main (Primary) Memory）
  - 位于CPU之外，用来存放已被启动的程序及所用的数据
  - 用DRAM实现，速度较快，容量较大（几GB）
- 外存储器AM (辅助存储器Auxiliary / Secondary Storage)
  - 位于主机之外，用来存放暂不运行的程序、数据或存档文件
  - 用磁表面或光存储器实现，容量大而速度慢

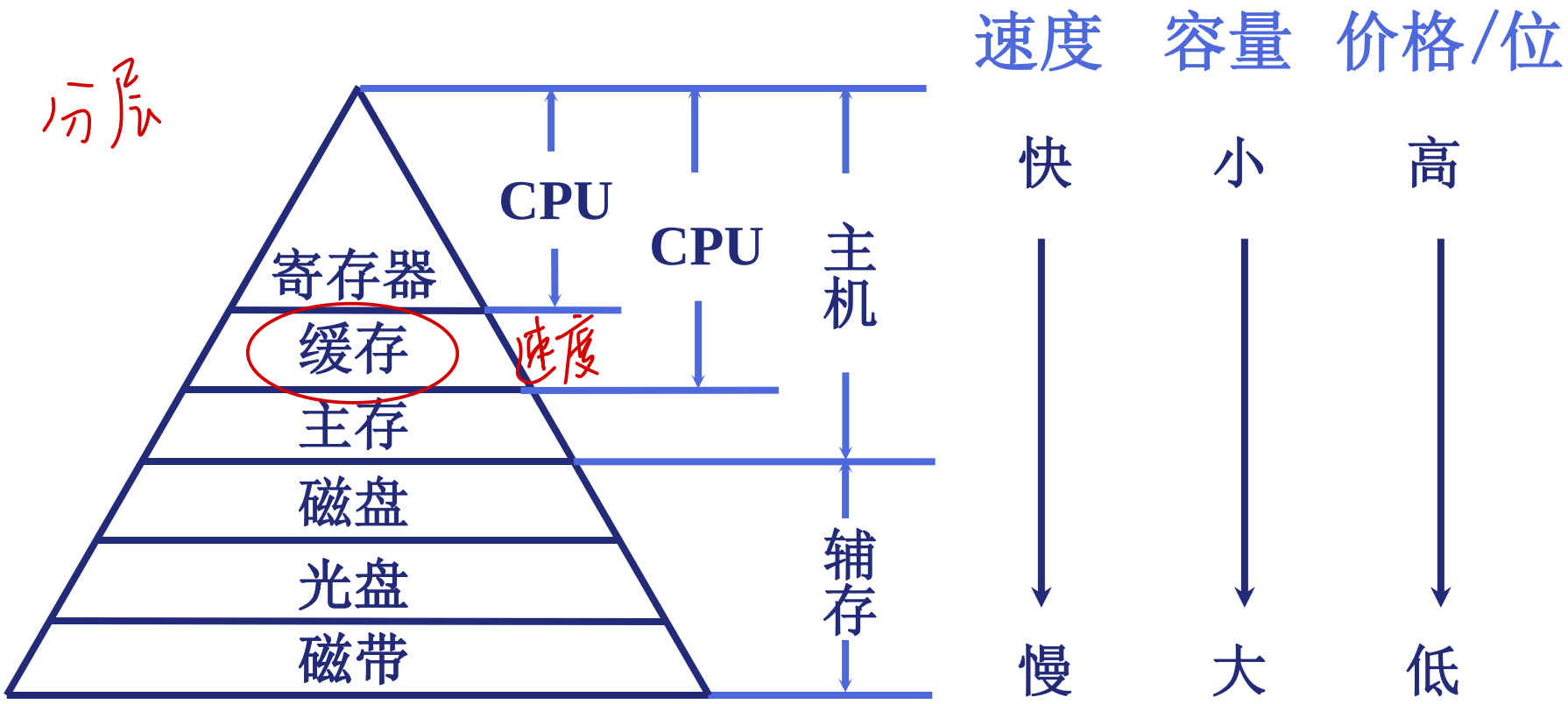
存储器设计目标

CPU 速度快
存储器 容量大

- 目标
  - 大容量、高速度、低成本、高可靠性
- 目前现实
  - 大容量存储器速度慢
  - 快速存储器容量小
- 如何实现我们的目标呢?
  - 层次存储器系统

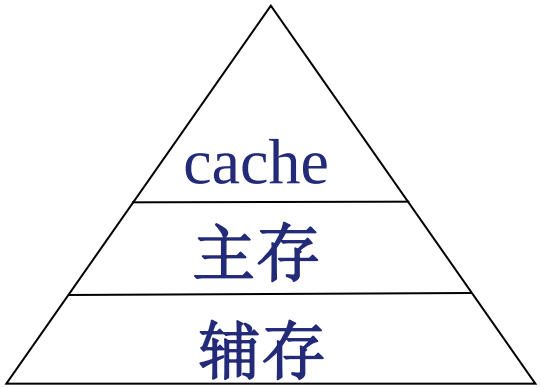
二、存储器的层次结构

1. 存储器三个主要特性的关系



2. 三级存储器结构

- 由高速缓冲存储器、主存储器和外存储器组成
  - 内存储器：CPU能直接访问的存储器
    - 包括高速缓冲存储器和主存储器
  - 外存储器：CPU不能直接访问的存储器，外存储器的信息必须调入内存储器后才能由CPU进行处理

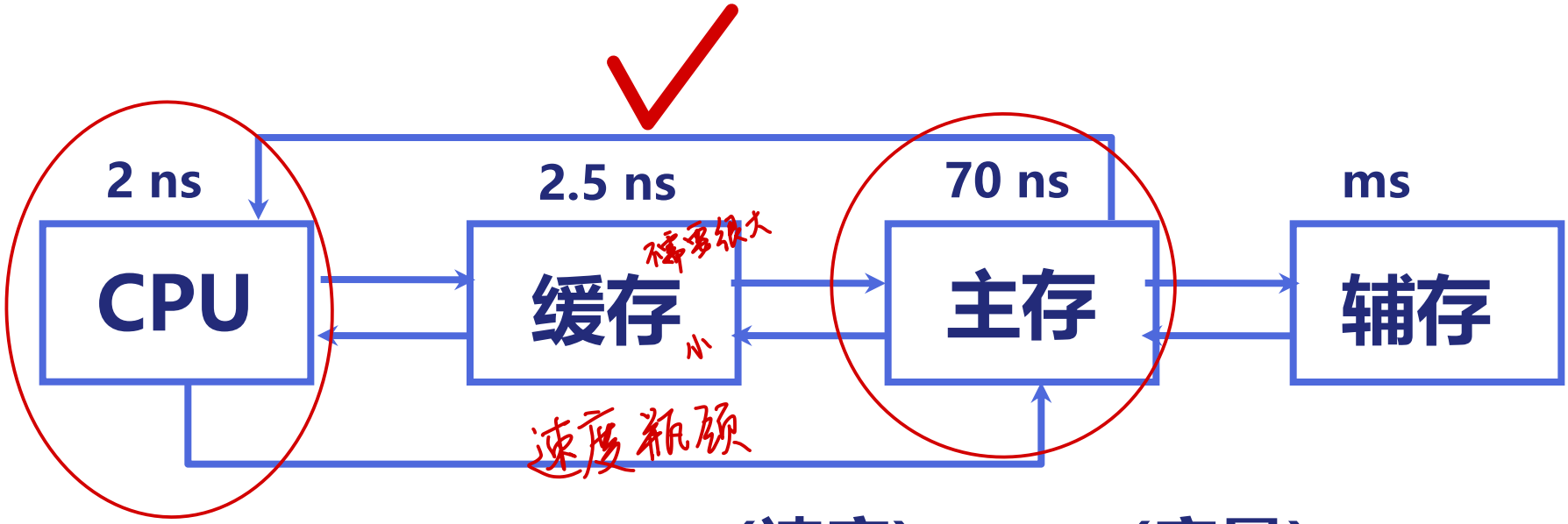


问题

- CPU clock rates ~0.33ns –2ns (3GHz-500MHz)

Memory technology	Access time in nanosecs (ns)	Access time in cycles	$ per GB in 2012	Capacity
SRAM (on chip) 片上	0.5-2.5 ns	1-3 cycles	$4k	256 KB
SRAM (off chip) 片下	1.5-30 ns	5-15 cycles	$4k	32 MB
DRAM 内存条	50-70 ns	150-200 cycles	$10-$20	8 GB
SSD (Flash)	5k-50k ns	Tens of thousands	$0.75-$1	512 GB
Disk	5M-20M ns	Millions	$0.05-$0.1	4 TB

缓存—主存层次和主存—辅存层次

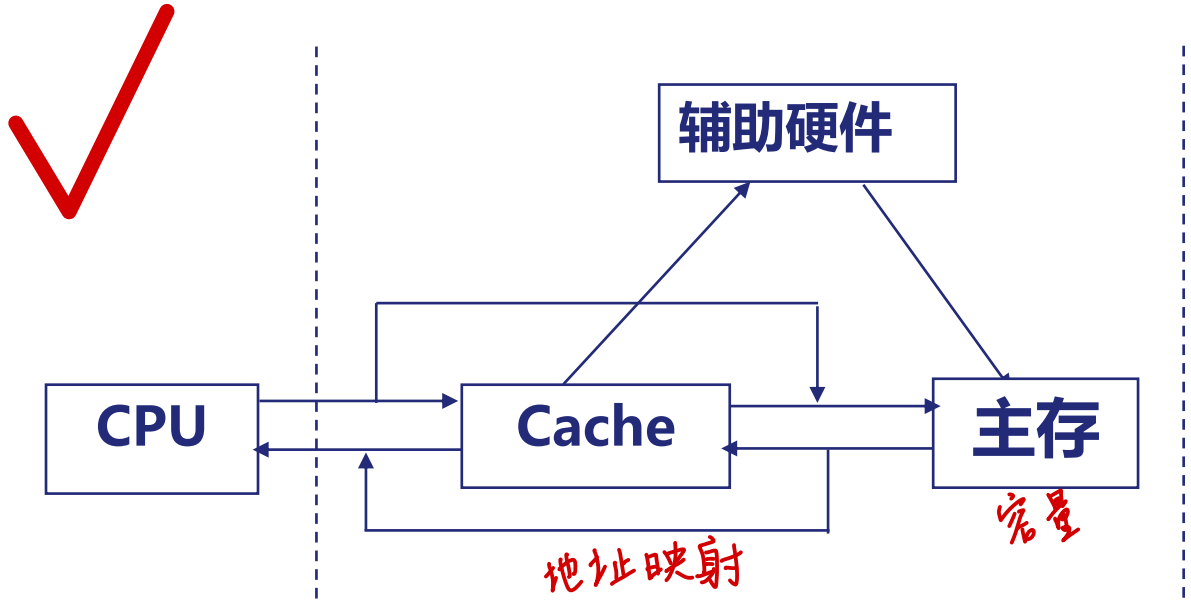


（速度）　　（容量）

缓存—主存　　主存—辅存

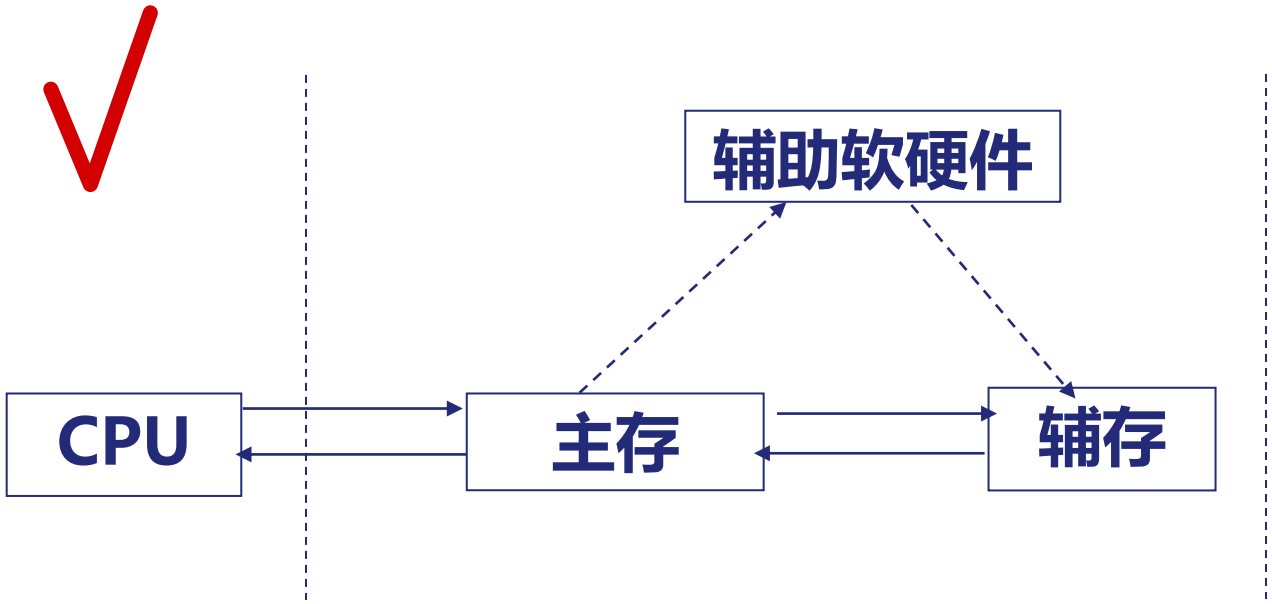
主存储器	虚拟存储器
实地址	虚地址
物理地址	逻辑地址

Cache-主存存储层次（Cache存储系统）



- Cache存储系统是为了解决主存速度不足而提出来的。
- 在Cache和主存之间增加辅助硬件，让它们构成一个整体
- 从CPU看，速度接近Cache，容量是主存的容量，位价格接近于主存价格
- 由于Cache存储系统全部采用硬件来调度，因此对系统程序员和应用程序员是透明的 写代码时不必考虑 Cache

主存 - 辅存存储层次（虚拟存储系统）



- 虚拟存储器是为了解决主存容量不足而提出来的。
- 在主存和辅存之间，增加辅存的软硬件，让它们构成一个整体
- 从CPU看，速度接近主存的速度，容量是虚拟的地址空间，每位价格是接近辅存的价格
- 由于虚拟存储系统需要通过操作系统来调度，因此对系统程序员是不透明的，但对应用程序员是透明的

层次存储器系统

- 高速度
  - 静态存储器速度高
  - 设置较小容量的高速缓冲存储器 Cache
- 大容量 focus
  - 动态存储器价格适中，速度适中
  - 可作为主存储器
- 低成本
  - 磁盘存储器价格低廉
  - 作为辅助存储器，暂存CPU访问频率不高的数据和程序
  - 作为虚拟存储器的载体

4.2 主存储器

一、概述

二、半导体存储芯片简介

三、随机存取存储器（RAM）

四、只读存储器（ROM）

五、存储器与CPU的连接

六、存储器的校验

七、提高访存速度的措施

4.2 主存储器

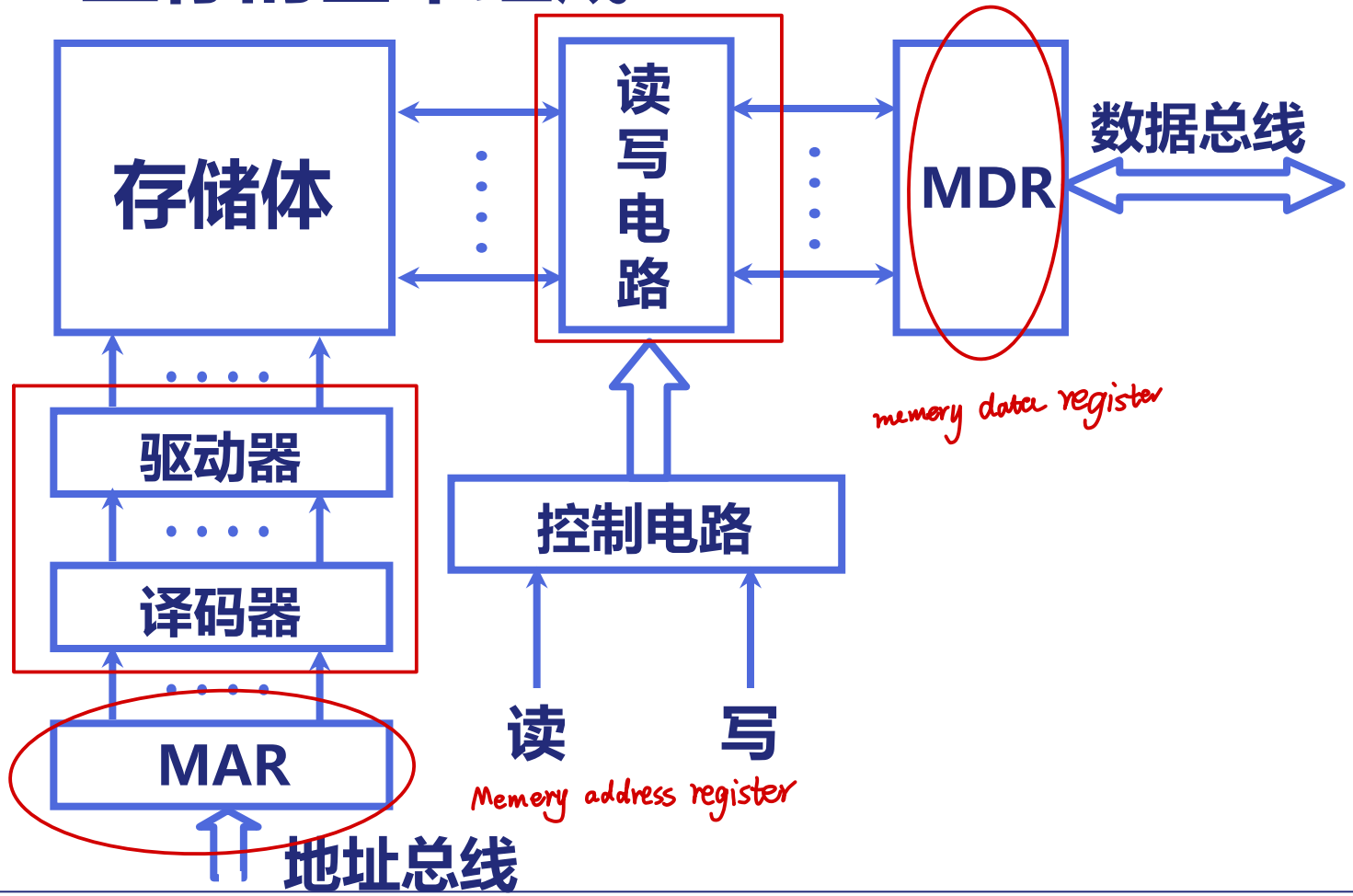
一、概述

四个方面问题：

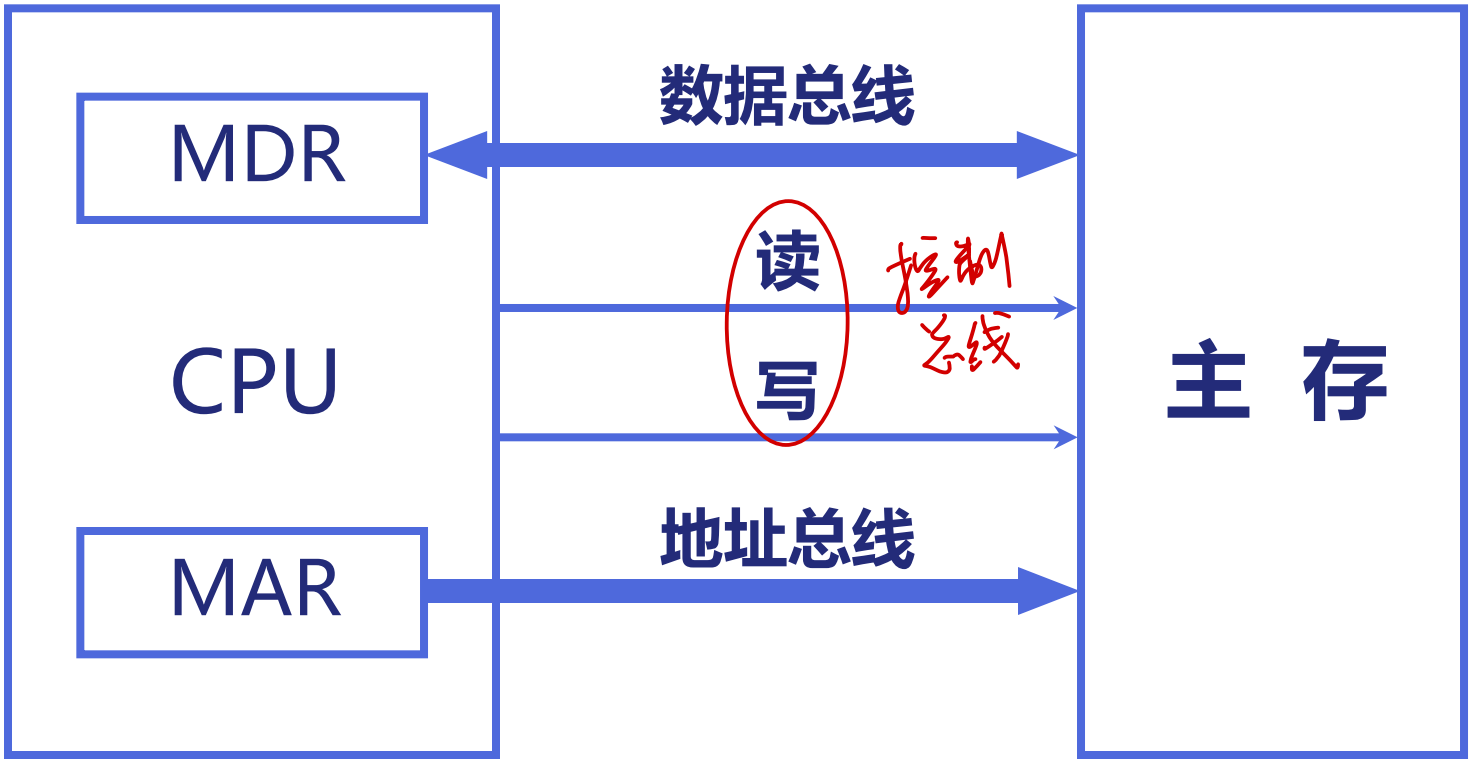
1. 主存的基本组成
2. 主存与CPU之间的联系
3. 主存中存储地址的分配
4. 主存的技术指标

一、概 述

1. 主存的基本组成



2. 主存和 CPU 的联系

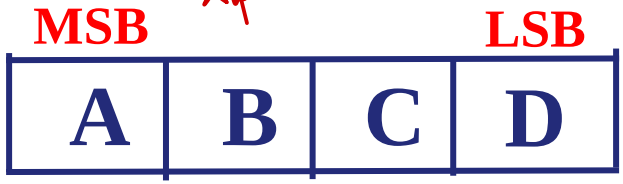


3. 主存中存储单元地址的分配

大端和小端存储方式

① Big-endian: 最高字节地址(MSB)是数据地址

大端



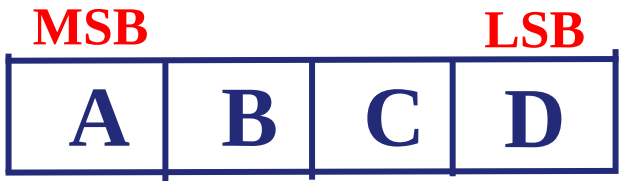
IBM 360/370, Motorola 68k, MIPS, Sparc

高位字节 地址为字地址

字地址	字节地址			
0	A	B	C	D
4	4	5	6	7
8	8	9	10	11

② Little-endian: 最低字节地址(LSB)是数据地址

小端



Intel 80x86, DEC， VAX

低位字节 地址为字地址

字地址	字节地址			
0	D	C	B	A
4	4	5	6	7
8	8	9	10	11

3. 主存中存储单元地址的分配

高位字节 地址为字地址

字地址	字节地址 MSB			LSB
0	0 (1B)	1 (8 bit)	2	3
4	4	5	6	7
8	8	9	10	11

低位字节 地址为字地址

字地址	字节地址 MSB			LSB
0	3	2	1	0
4	7	6	5	4
8	11	10	9	8

设地址线 24 根　　按 字节 寻址　2^{24} = 16 M 个地址

若字长为 16 位　　按 字 寻址　8 M个字地址

若字长为 32 位　　按 字 寻址　4 M 个字地址

数据的存储和排列顺序

- 80年代开始，几乎所有机器都用字节编址
- ISA指令集设计时要考虑的两个问题：
  - **如何从一个字节地址中取到一个32位的字?**
  - **字的存放问题** 大端 小端
  - **一个字能否存放在任何字节边界?**
  - **字的边界对齐问题**

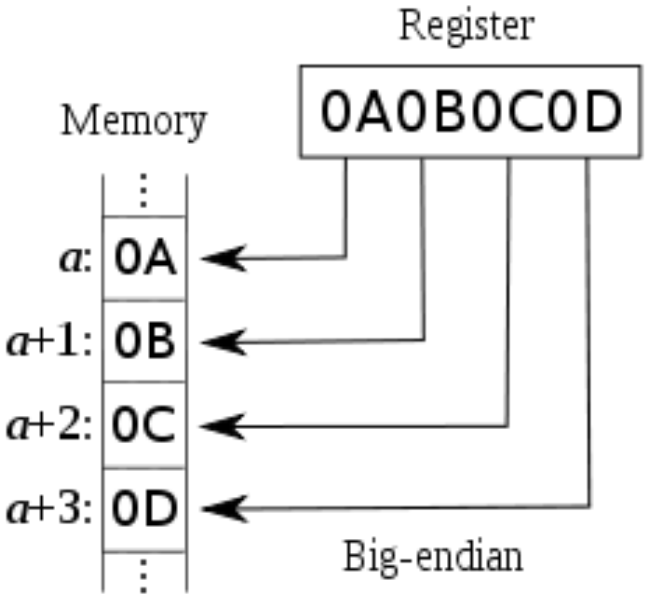
BIG Endian versus Little Endian

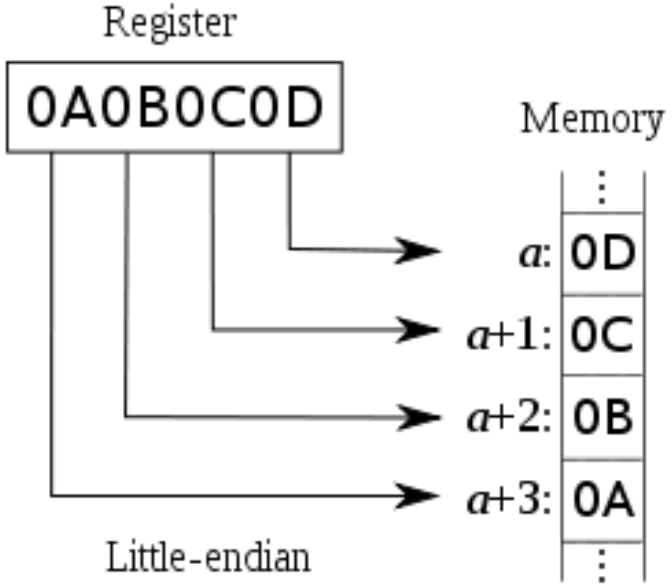
大端Big-Endian

低地址存放最高有效位（**MSB**），既高位字节排放在内存的低地址端，低位字节排放在内存的高地址端。

小端Little-Endian

低地址存放最低有效位（LSB），既低位字节排放在内存的低地址端，高位字节排放在内存的高地址端。





Alignment(对齐)

Alignment: 要求数据的地址是相应的边界地址

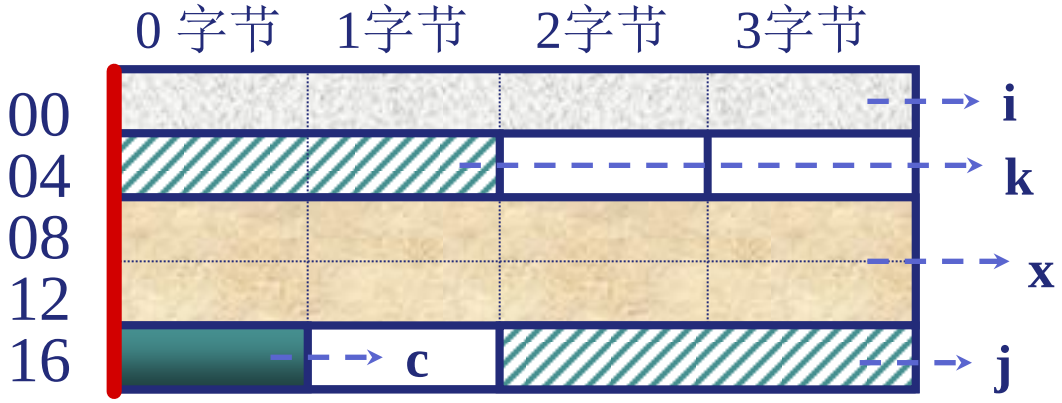
- 目前机器字长一般为32位或64位，而存储器地址按字节编址
- 指令系统支持对字节、半字、字及双字的运算，也有位处理指令很少
- 各种不同长度的数据存放时，有两种处理方式：
  - 按边界对齐 √
  - 不按边界对齐

    坏处：可能会增加访存次数！

Alignment(对齐)

示例　假设数据顺序：字-半字-双字-字节-半字-……

如：int i, short k, double x, char c, short j,……（32位系统）

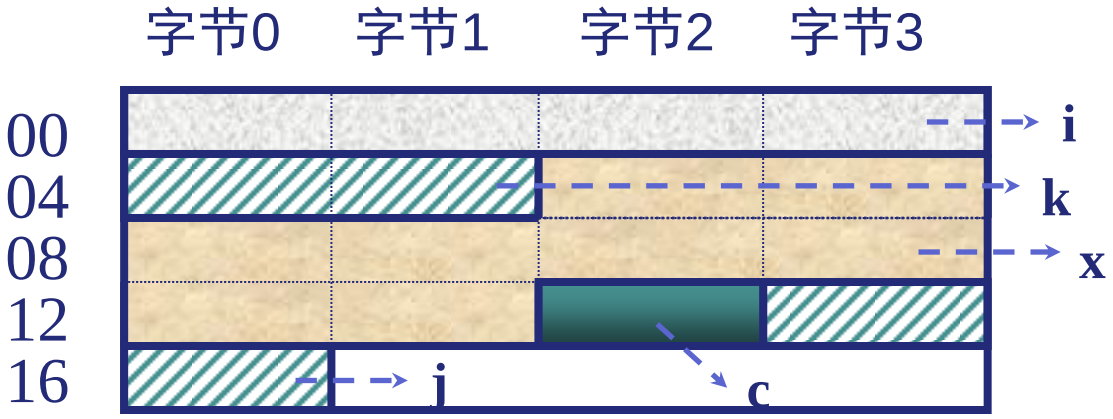


按边界对齐

x：2个周期

j：1个周期

则：&i=0; &k=4; &x=8; &c=16; &j=18;……



边界不对齐

x：3个周期

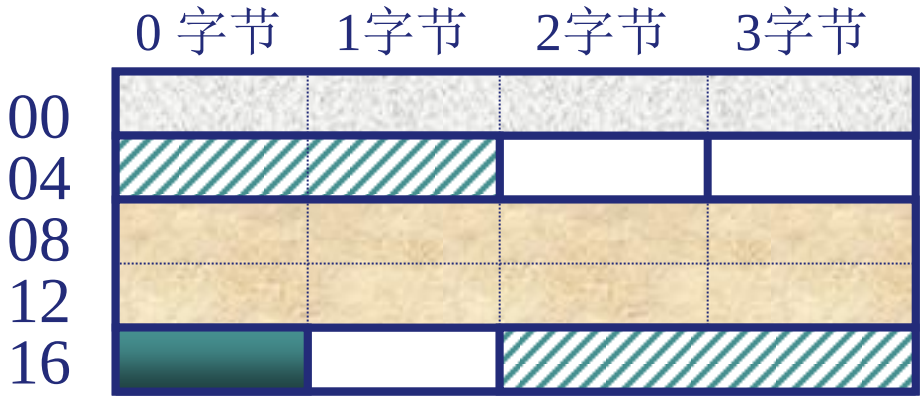
j：2个周期

增加了访存次数！

则：&i=0; &k=4; &x=6; &c=14; &j=15;……

Alignment(对齐)

- 边界对齐与存储地址的关系（以32位为例）



- 双字长数据边界对齐的起始地址的最末三位为000(8字节整数倍)；
- 单字长边界对齐的起始地址的末二位为00(4字节整数倍)；
- 半字长边界对齐的起始地址的最末一位为0(2 字节整数倍)。

4. 主存的性能指标

(1) 存储容量　　主存 存放二进制代码的总数量

- 以字编址的计算机：字数与字长的乘积

存储容量 = 存储单元个数×存储字长

64K×16位：64K个存储单元，每个存储单元字长为16位

- 以字节编址的计算机，以字节数来表示

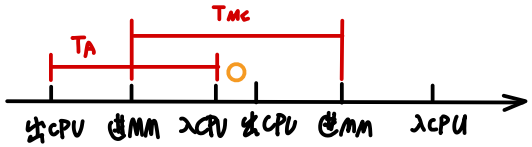
存储容量 = 存储单元个数×存储字长/8

64K×16位用字节数表示：128K字节（128KB）

- 地址线位数反映存储单元数，数据线的位数反映存储单元字长
- 地址线10根，数据线4根，则主存容量为1K×4位

4. 主存的技术指标

(2) 存储速度



- 存取时间T_A（Memory Access Time）：从CPU送出内存单元的地址码开始，到主存读出数据并送到CPU（或者是把CPU数据写入主存）所需要的时间（单位：ns，1 ns = 10^{-9} s）

读出时间　写入时间

- 存取周期T_{MC}（Memory Cycle Time）：连读两次访问存储器所需的最小时间间隔，它应等于存取时间加上下一存取开始前所要求的附加时间，因此，T_{MC}比T_A大（因为存储器由于读出放大器、驱动电路等都有一段稳定恢复时间，所以读出后不能立即进行下一次访问。）

读周期　写周期

- $T_{MC} > T_A$，因为存储器在读写操作后，要有一段恢复内部状态的复原时间。对于破坏性读出的RAM，存取周期往往比存取时间要大的多，甚至可达到$T_{MC} = 2T_A$，这是因为存储器中的信息读出后需要马上进行再生（重写）。

4. 主存的技术指标

(3) 存储器的带宽：单位时间内存储器存取的信息量

单位：字/秒，字节/秒，位/秒

- 提高主存的带宽的措施
  - 缩短存取周期 cache
  - 增加存储字长
  - 增加存储体

4. 主存的技术指标

(4) 可靠性：指在规定时间内，存储器无故障读写的概率。

- 平均无故障时间（Mean Time Between Failures，MTBF）用于衡量可靠性。
- MTBF为两次故障之间的平均时间间隔。MTBF越长，说明存储器可靠性越高。

(5) 功耗：功耗既反映耗电又反映其发热程度。

4.2 主存储器

一、概述

二、半导体存储芯片简介

三、随机存取存储器（RAM）

四、只读存储器（ROM）

五、存储器与CPU的连接

六、存储器的校验

七、提高访存速度的措施

二、半导体存储芯片简介

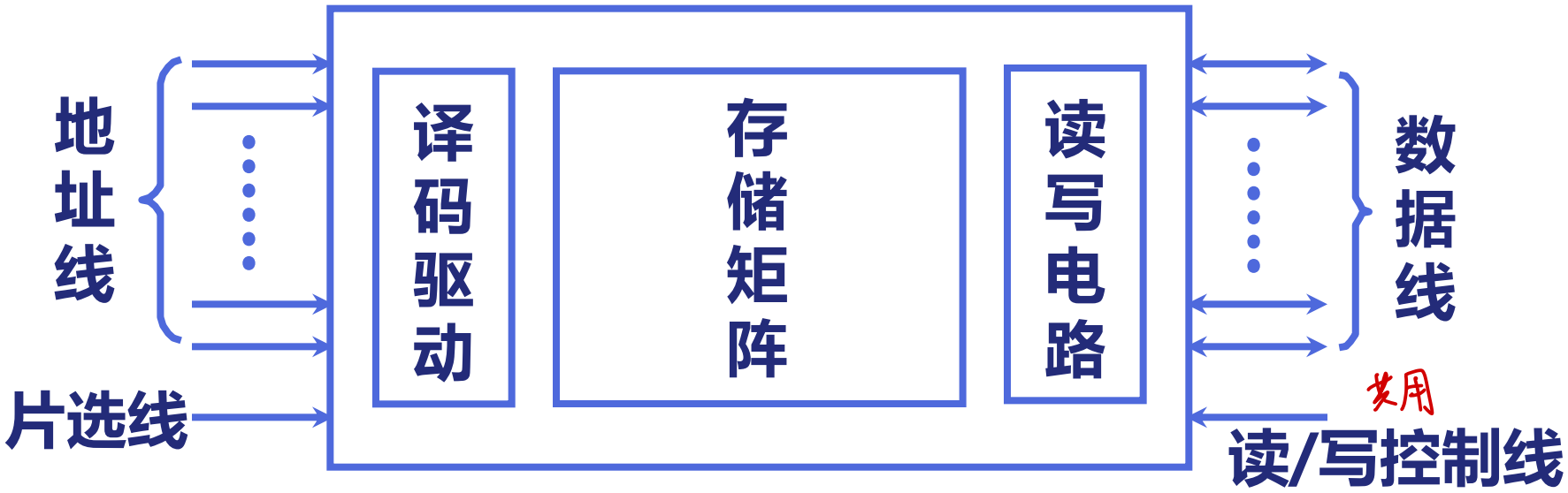
1.半导体存储芯片的基本结构

2.半导体存储芯片的译码驱动方式

- 线选法
- 重合法

二、半导体存储芯片简介

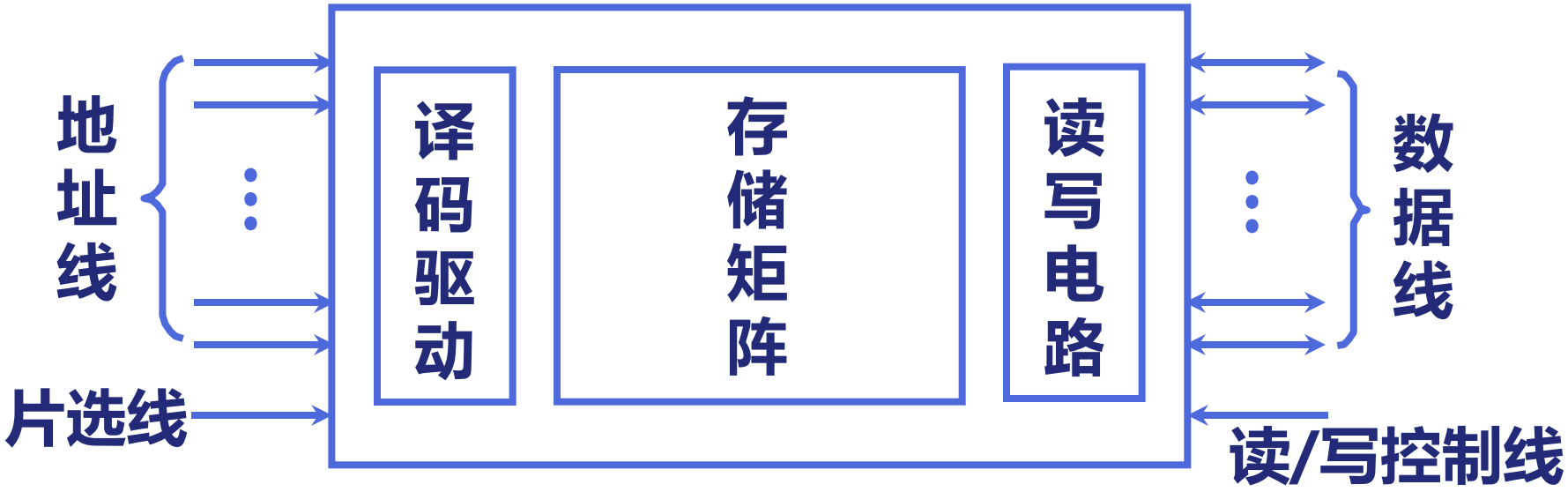
1. 半导体存储芯片的基本结构



地址线（单向）	数据线（双向）	芯片容量
10	4	1K × 4位
14	1	16K × 1位
13	8	8K × 8位

二、半导体存储芯片简介

1．半导体存储芯片的基本结构



片选线 chip select $\overline{CS}$ chip enable $\overline{CE}$

读/写控制线 $\overline{WE}$（低电平写 高电平读）

$\overline{OE}$（允许读） $\overline{WE}$（允许写）

存储芯片片选线的作用

- 由于一块集成芯片容量有限，要组成一个大的存储器一般需要将多个芯片连接使用。
- 在使用存储器某个地址时，某些芯片需要使用，其它芯片暂时不用，即片选。
- 当芯片的片选信号（$\overline{CS}$）有效时，芯片被选中，此片所连的地址线才有效，才能对它进行读写操作

存储芯片片选线的作用

14 地址线

用 16K × 1位 的存储芯片组成 64K × 8位 的存储器

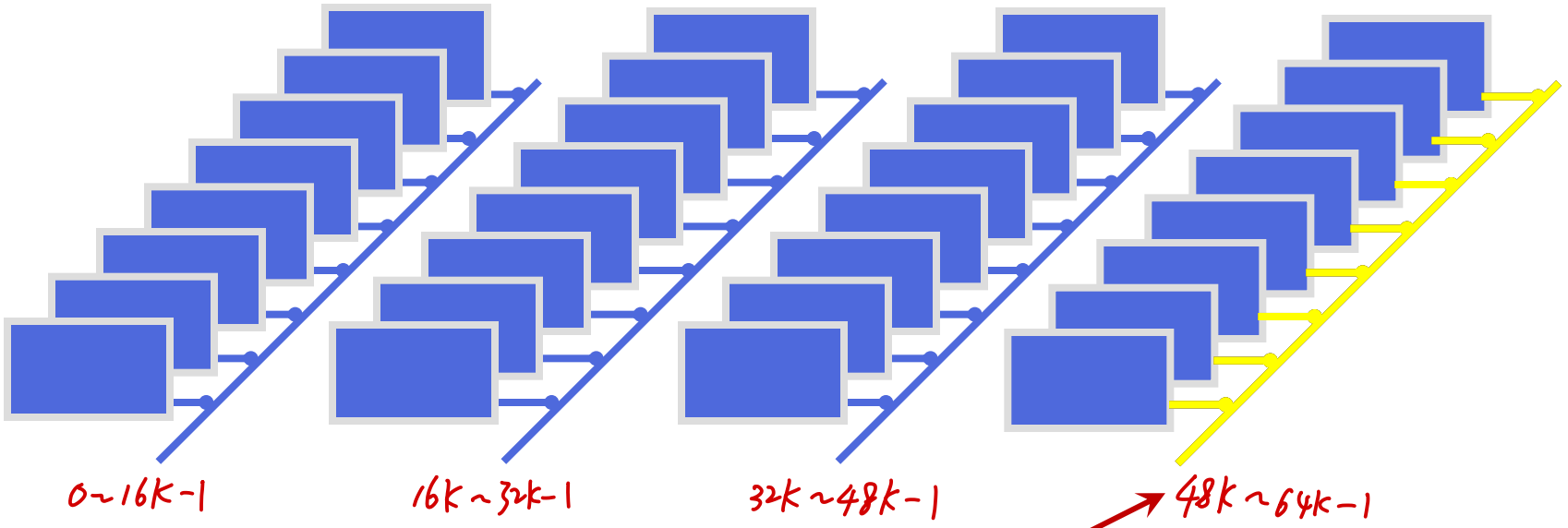
32片

8片 16K × 1位

8片 16K × 1位

8片 16K × 1位

8片 16K × 1位



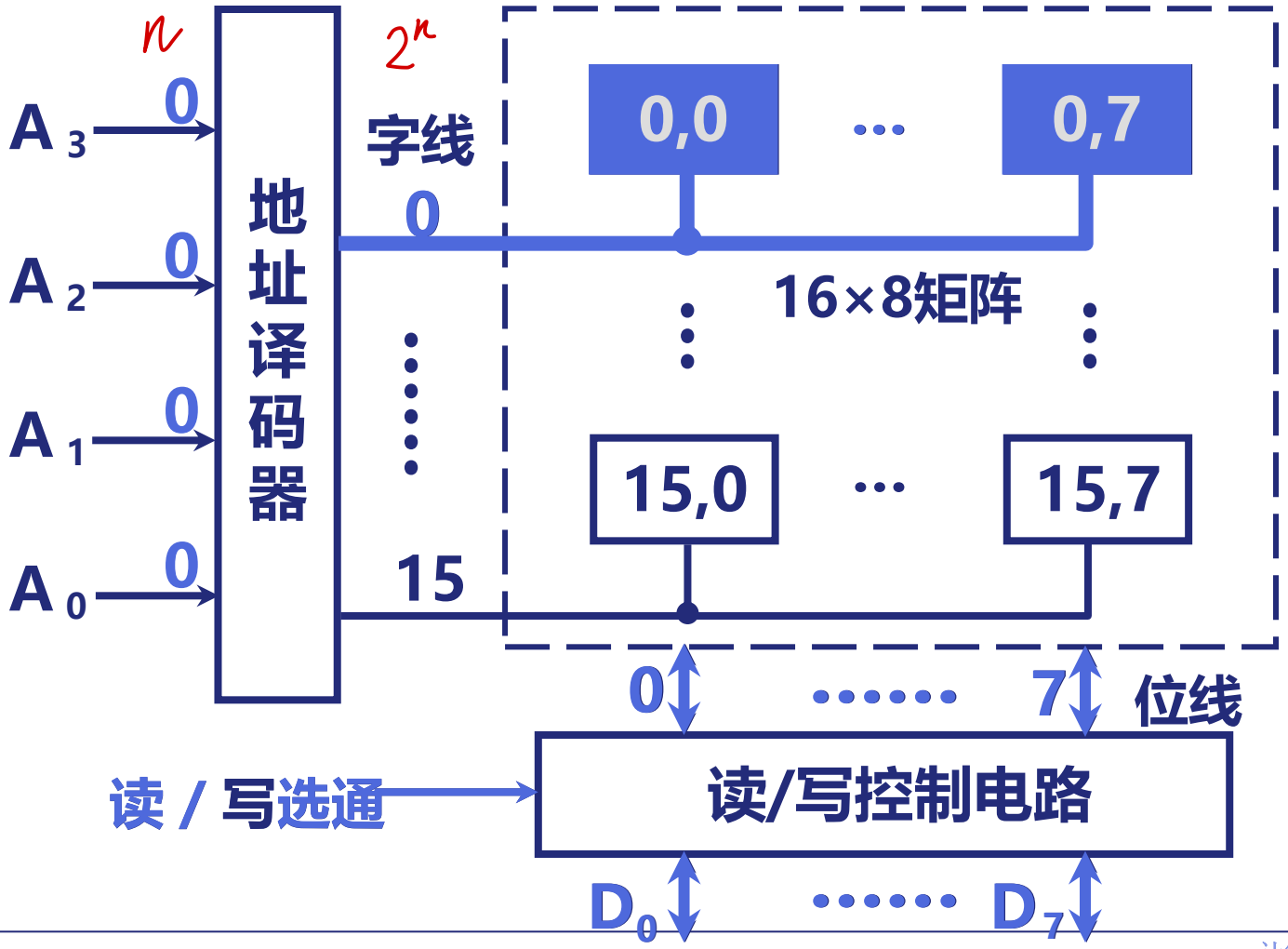
当地址为 2^{16}-1时

此 8 片的片选有效

2. 半导体存储芯片的译码驱动方式

(1) 线选法

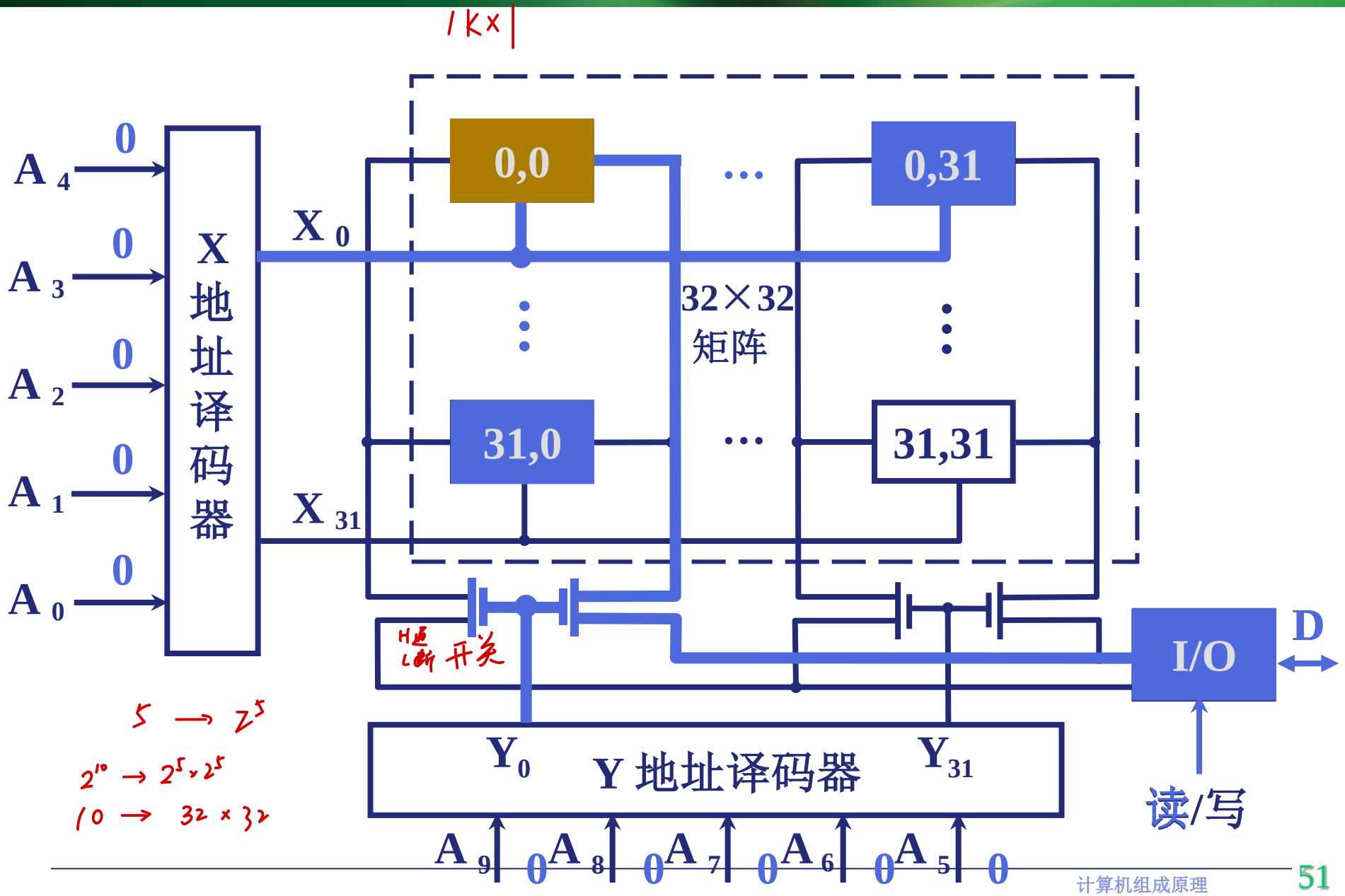
存储容量 16 B



2. 半导体存储芯片的译码驱动方式

- 地址译码器设计方案
  - 线选法（单译码）：地址译码器只有一个，用一根字线直接选中一个存储单元各位（如一个字节）
    - 当地址线数较大，译码器复杂、开销大，使得存储器成本迅速增加，性能下降
  - 重合法（双译码）：X地址译码器、Y地址译码器

(2) 重合法


1Kx1
A4
A3
A2
A1
A0
0
0
0
0
0
X地址译码器
X0
X31
0,0
0,31
31,0
31,31
32×32矩阵
H通 L断 开关
I/O
D
读/写
Y0
Y地址译码器
Y31
A9 0 A8 0 A7 0 A6 0 A5 0
5 → 2^5
2^10 → 2^5 × 2^5
10 → 32 × 32

2. 半导体存储芯片的译码驱动方式

- **线选法（单译码）：地址译码器只有一个，用一根字线直接选中一个存储单元各位（如一个字节）**

N位地址，寻址2^n个存储单元，2^n根译码线

- **重合法（双译码）：X地址译码器、Y地址译码器**

N位地址，寻址2^n个存储单元，$2^{n/2+1}$根译码线

$2^{n/2} \cdot 2$

FURY DDR4
DDR4雷电系列骇客神条
DDR4
HYPERX
FURY
DDR4
高性价比
的DDR4内存
PnP
无需设置
即可实现理想频率
32GB
大容量
单条容量高达32GB

Apacer

msi
HYPERX
FURY
HYPERX
FURY
EZ Debug LED
2019/12/7 16:11

4.2 主存储器

一、概述

二、半导体存储芯片简介

三、随机存取存储器（RAM）

四、只读存储器（ROM）

五、存储器与CPU的连接

六、存储器的校验

七、提高访存速度的措施

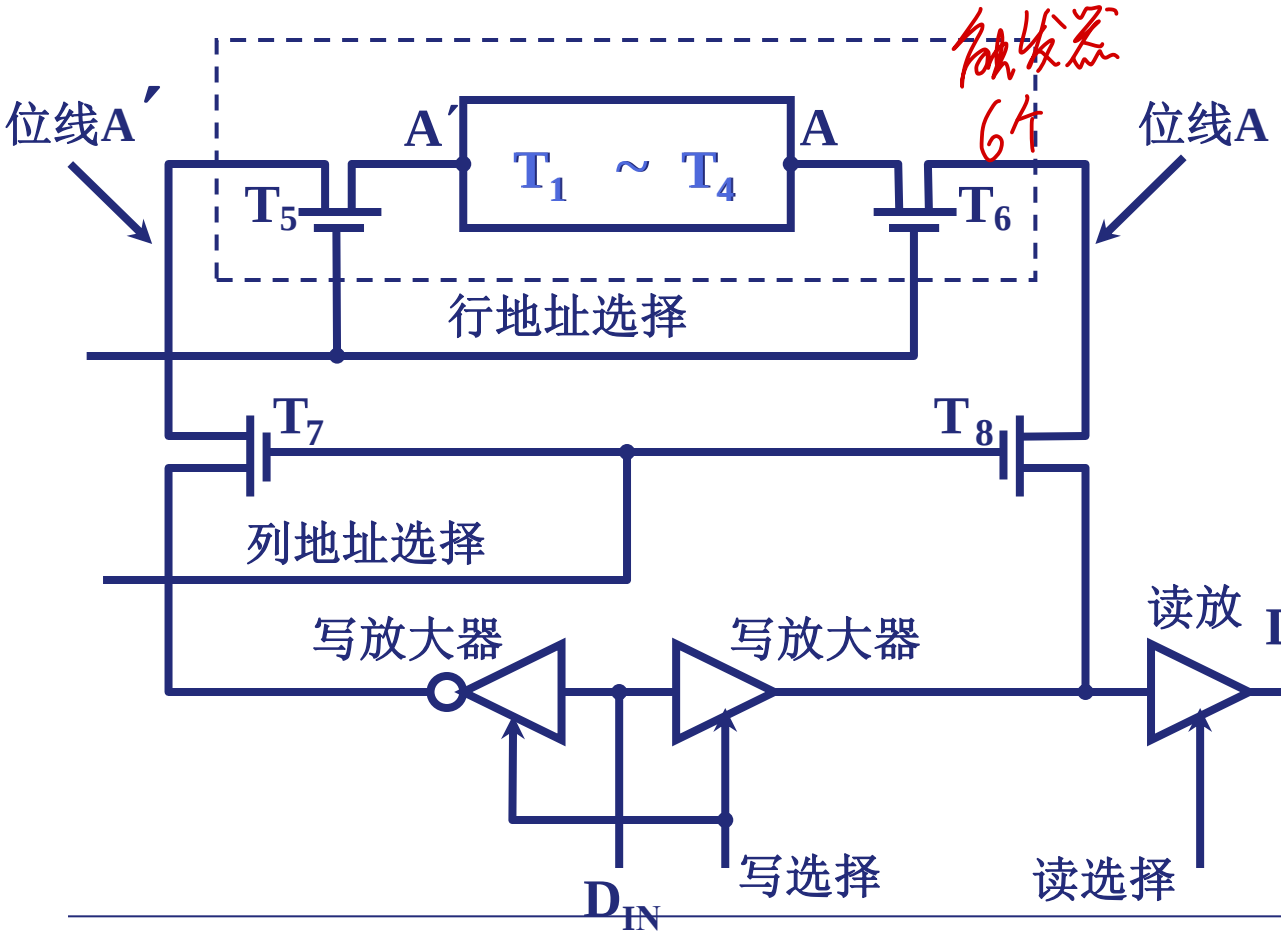
三、随机存取存储器（RAM）

- 1. 静态 RAM (SRAM，Static RAM)
  - 保存0和1的原理是什么?
  - 基本单元电路的构成是什么?
  - 对单元电路如何读出和写入?
  - 典型芯片的结构是什么样子的?
  - 静态RAM芯片如何进行读出和写入操作?
- 2. 动态 RAM (DRAM，Dynamic RAM)
- 3. 动态RAM和静态RAM的比较

三、随机存取存储器（RAM）

1. 静态 RAM (SRAM)

(1) 静态 RAM 基本电路



$T_1 \sim T_4$ 触发器

T_5、T_6 行开关

T_7、T_8 列开关

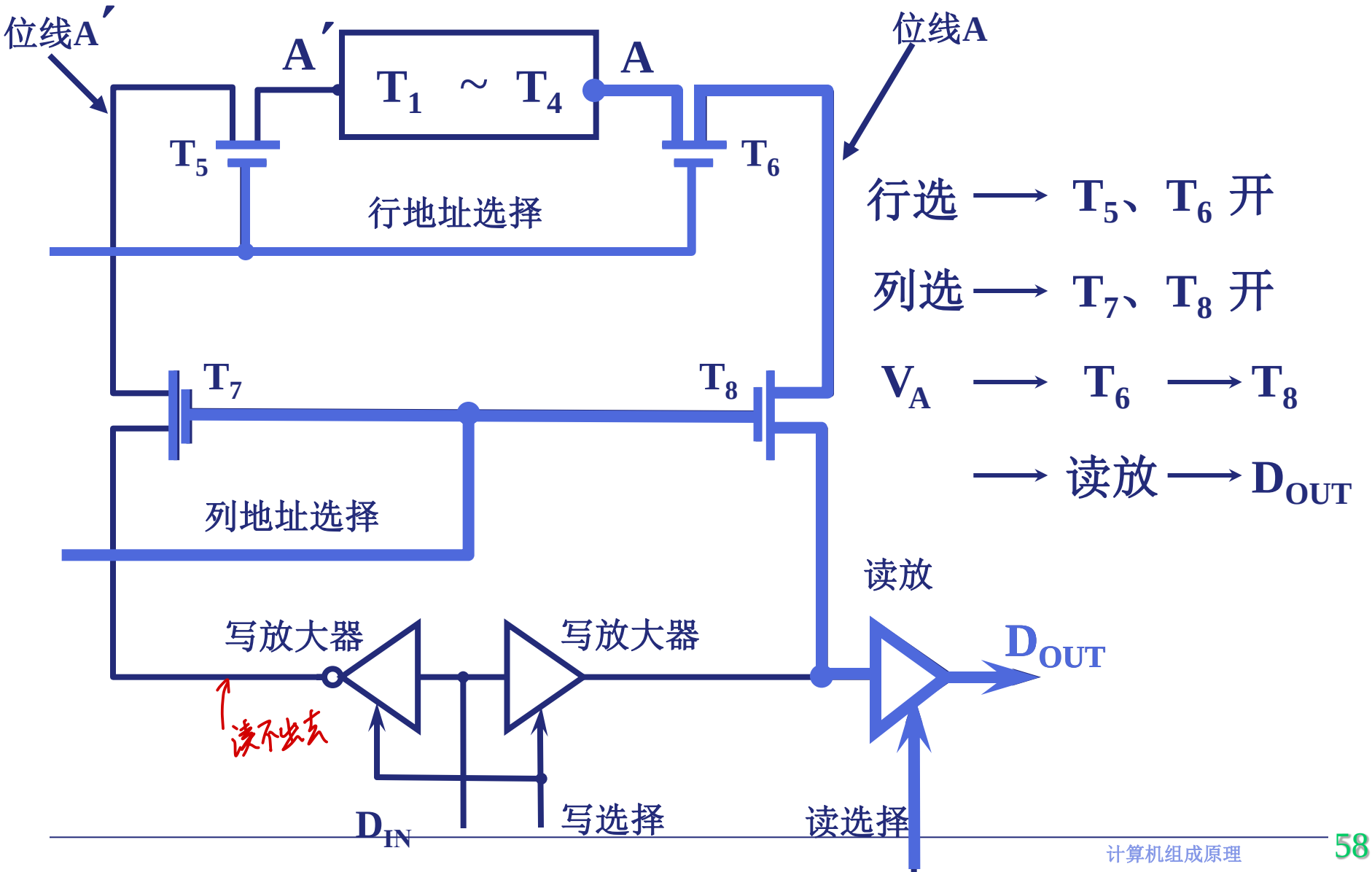
T_7、T_8 一列共用

A 触发器原端

A′ 触发器非端

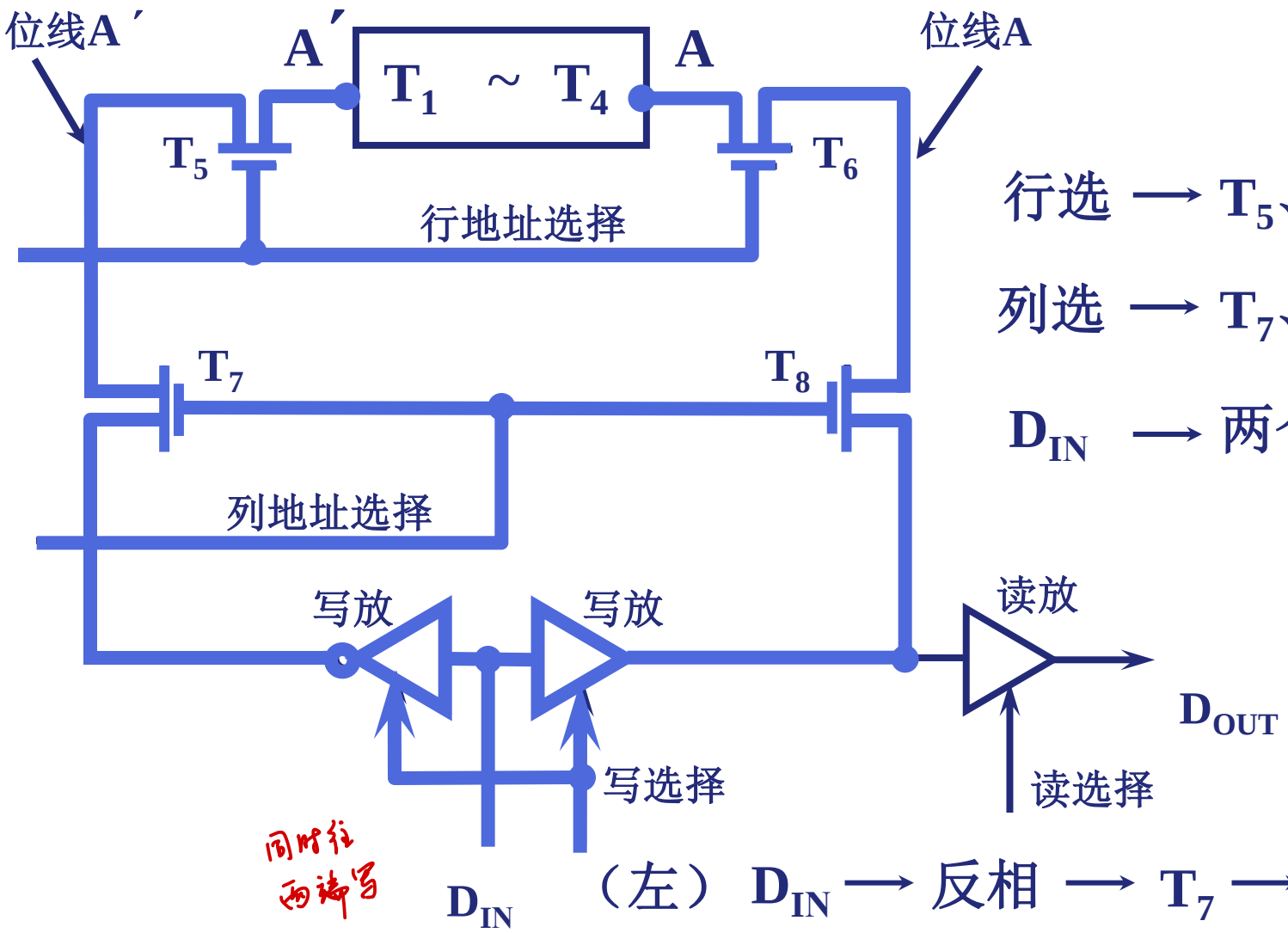
① 静态 RAM 基本电路的 读 操作

4.2



② 静态 RAM 基本电路的 写 操作

4.2



行选 → T_5、T_6 开

列选 → T_7、T_8 开

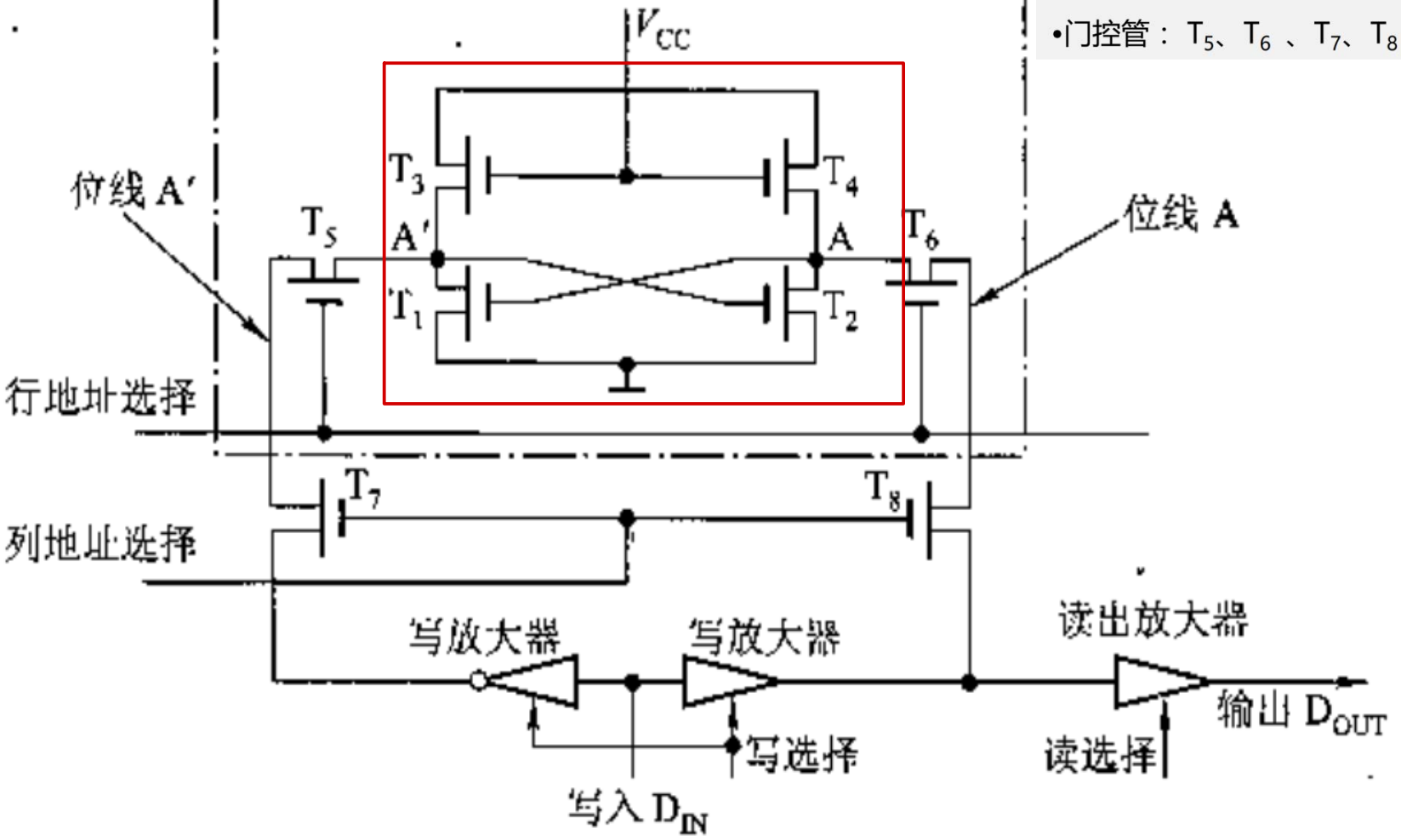
D_{IN} → 两个写放

（左）D_{IN} → 反相 → T_7 → T_5 → A′

（右）D_{IN} → T_8 → T_6 → A

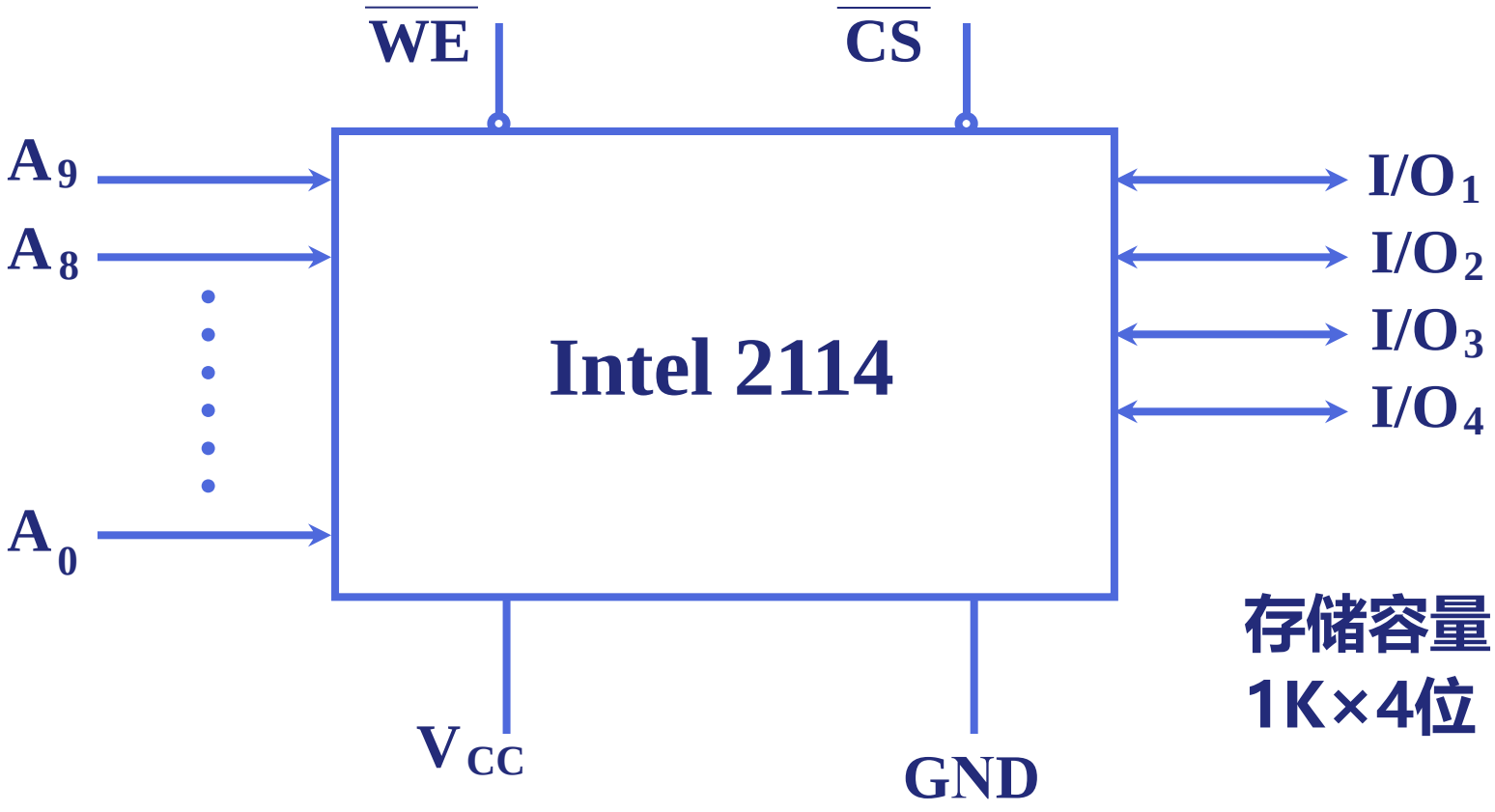
课本P77 图4.11

- 工作管： T_1、 T_2 (保存数据)
- 负载管： T_3、 T_4 （补充电荷）
- 门控管： T_5、T_6 、T_7、T_8 （开关作用）

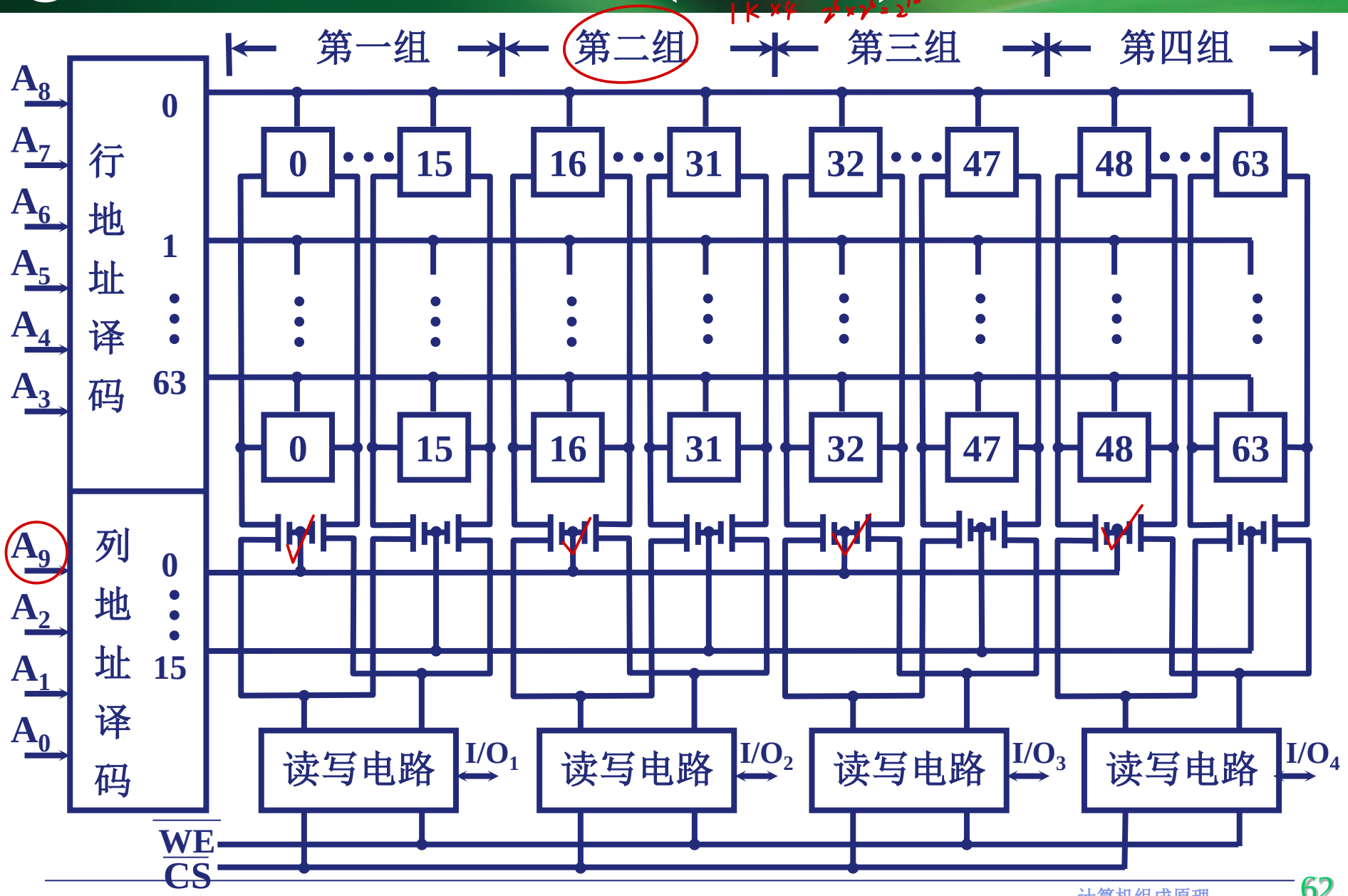


(2) 静态 RAM 芯片举例

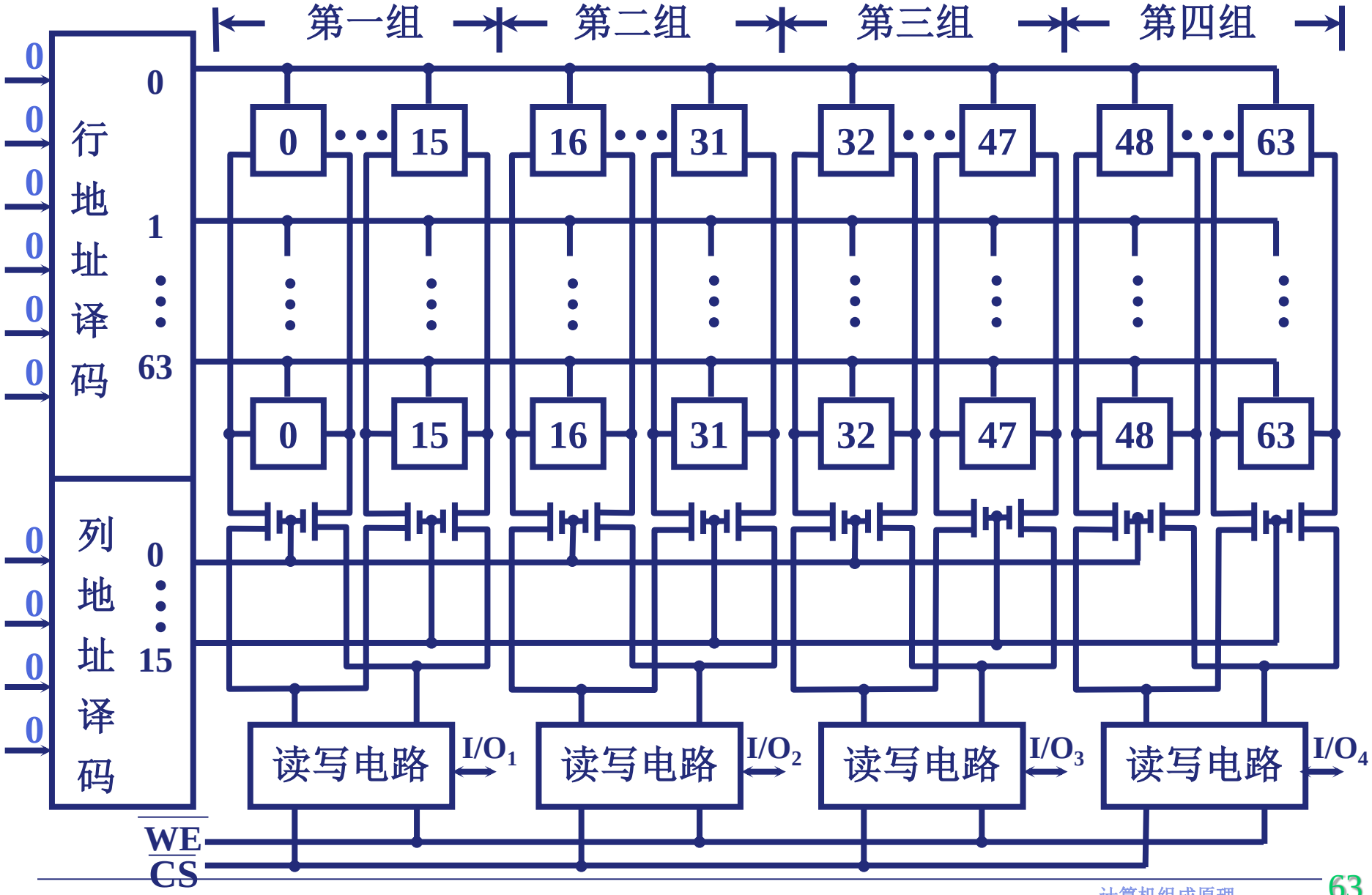
① Intel 2114 外特性



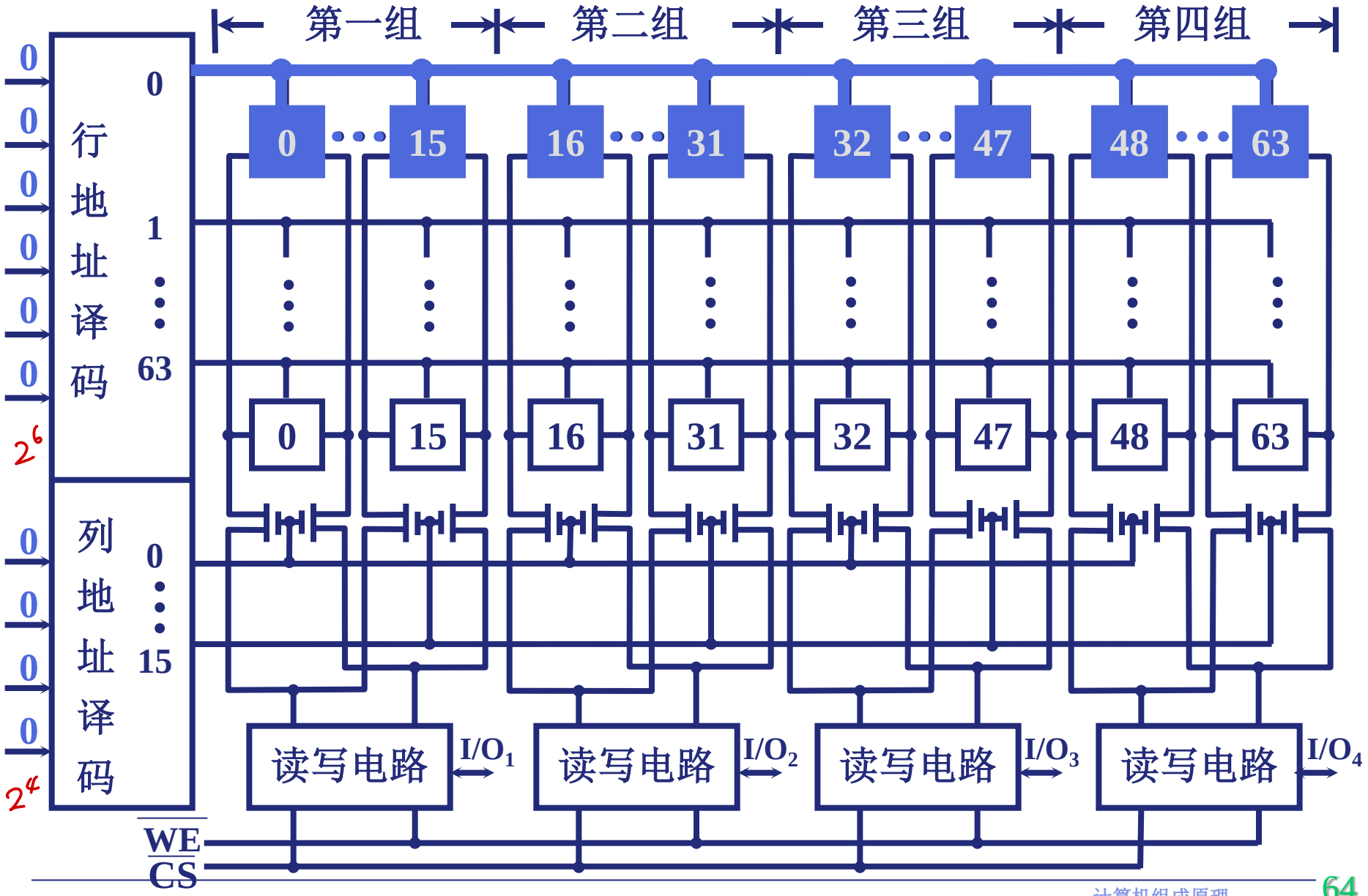
② Intel 2114 RAM 矩阵 (64 × 64) 读



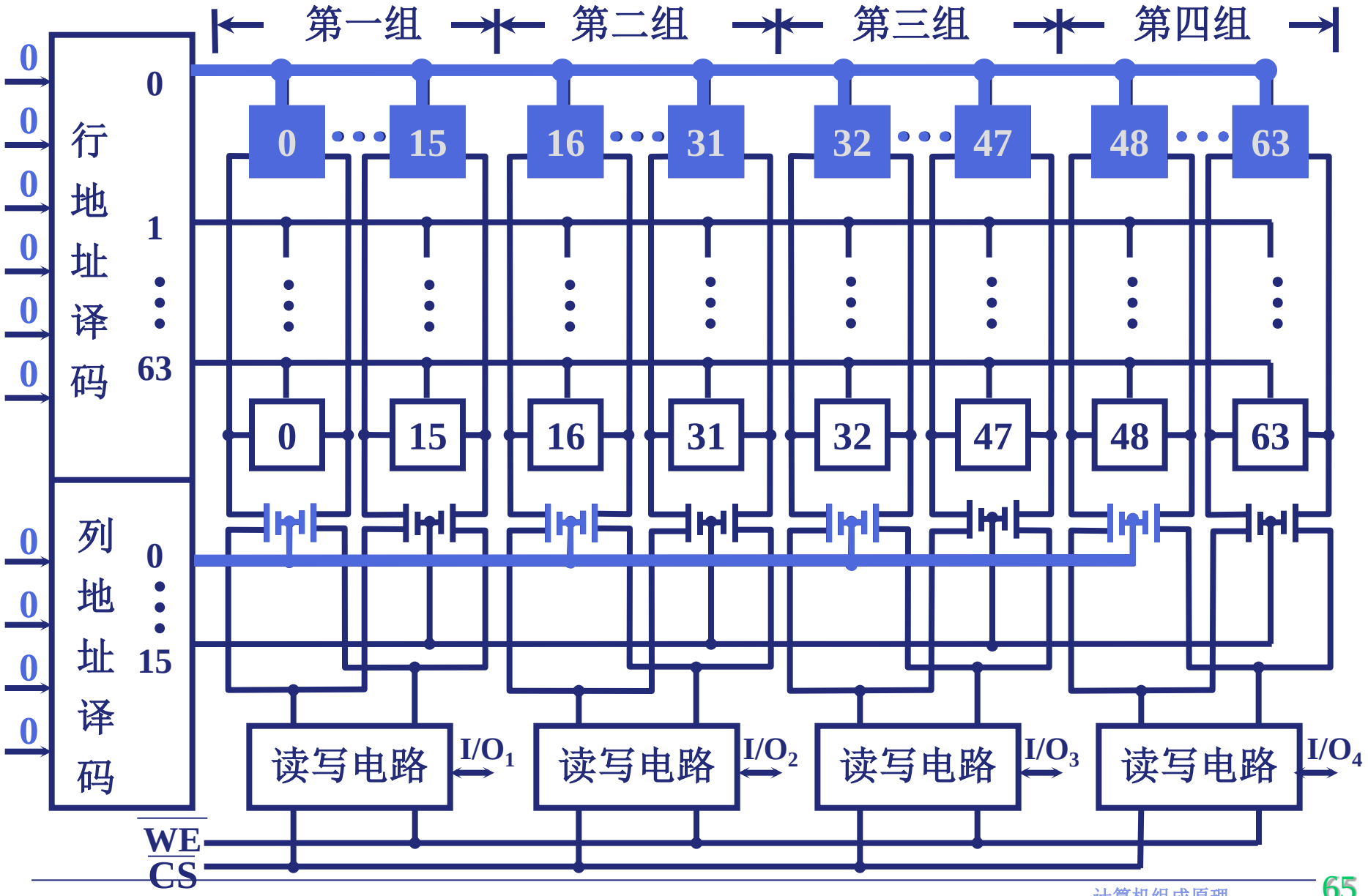
② Intel 2114 RAM 矩阵 (64 × 64) 读



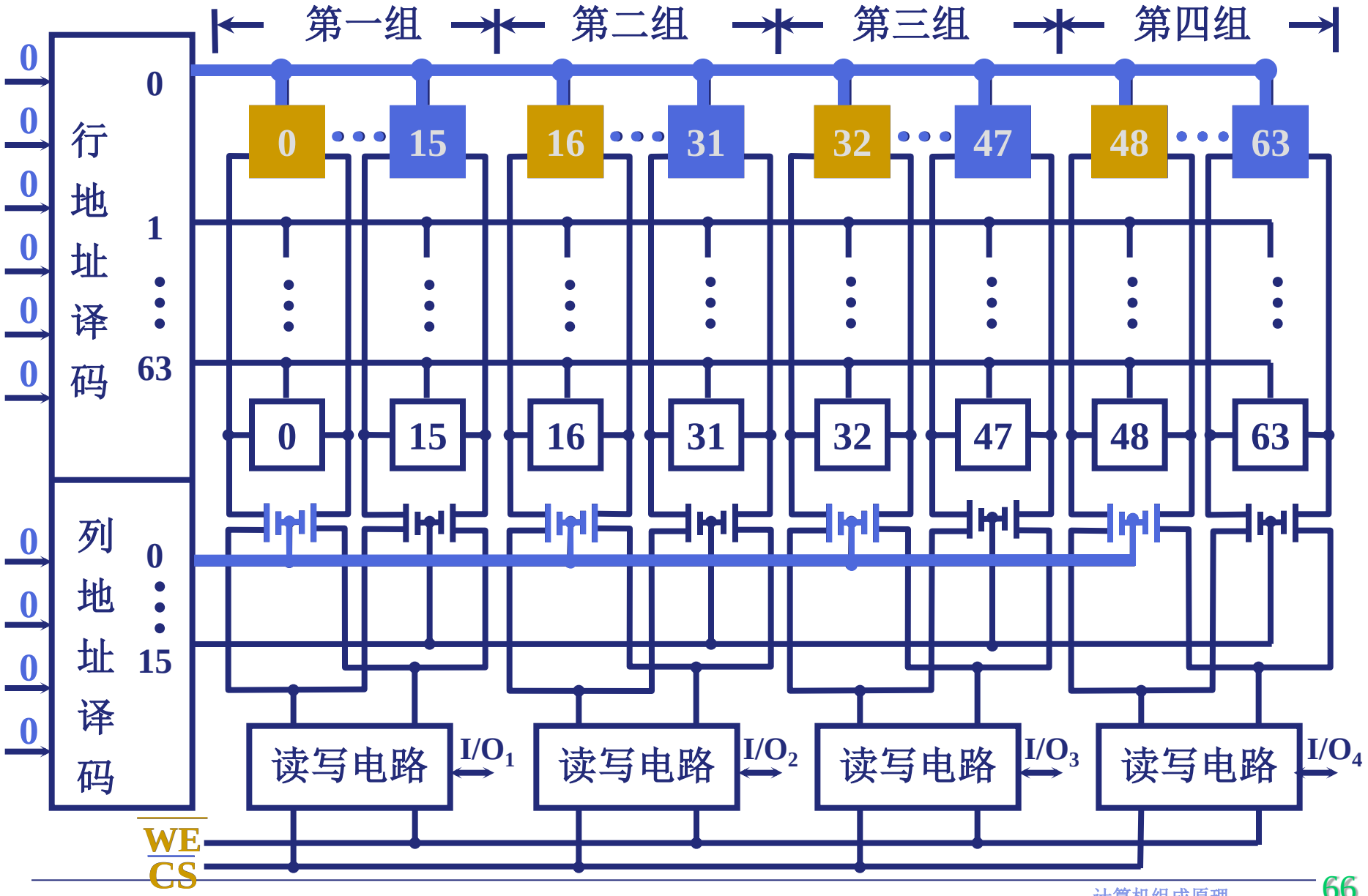
② Intel 2114 RAM 矩阵 (64 × 64) 读



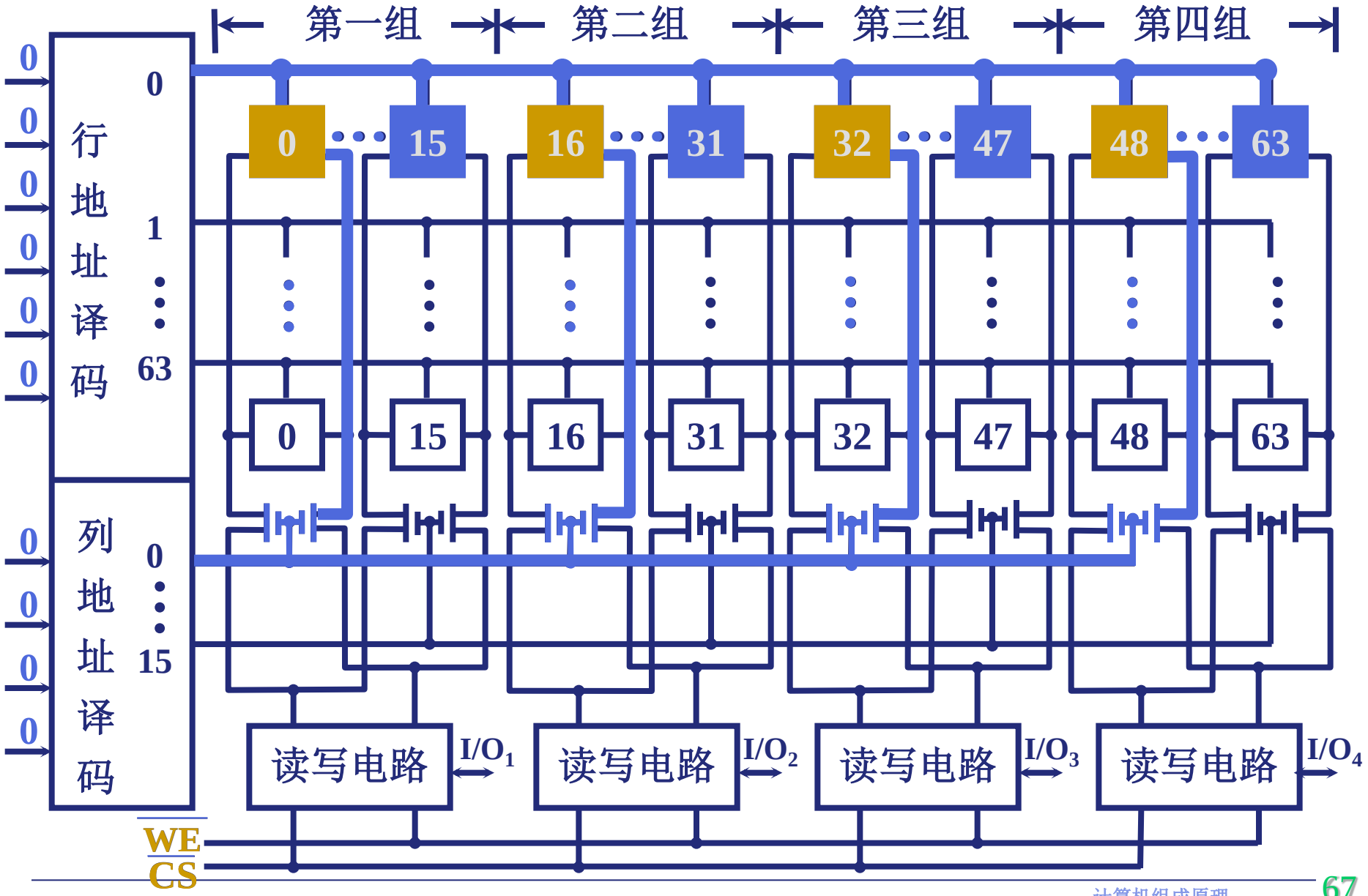
② Intel 2114 RAM 矩阵 (64 × 64) 读



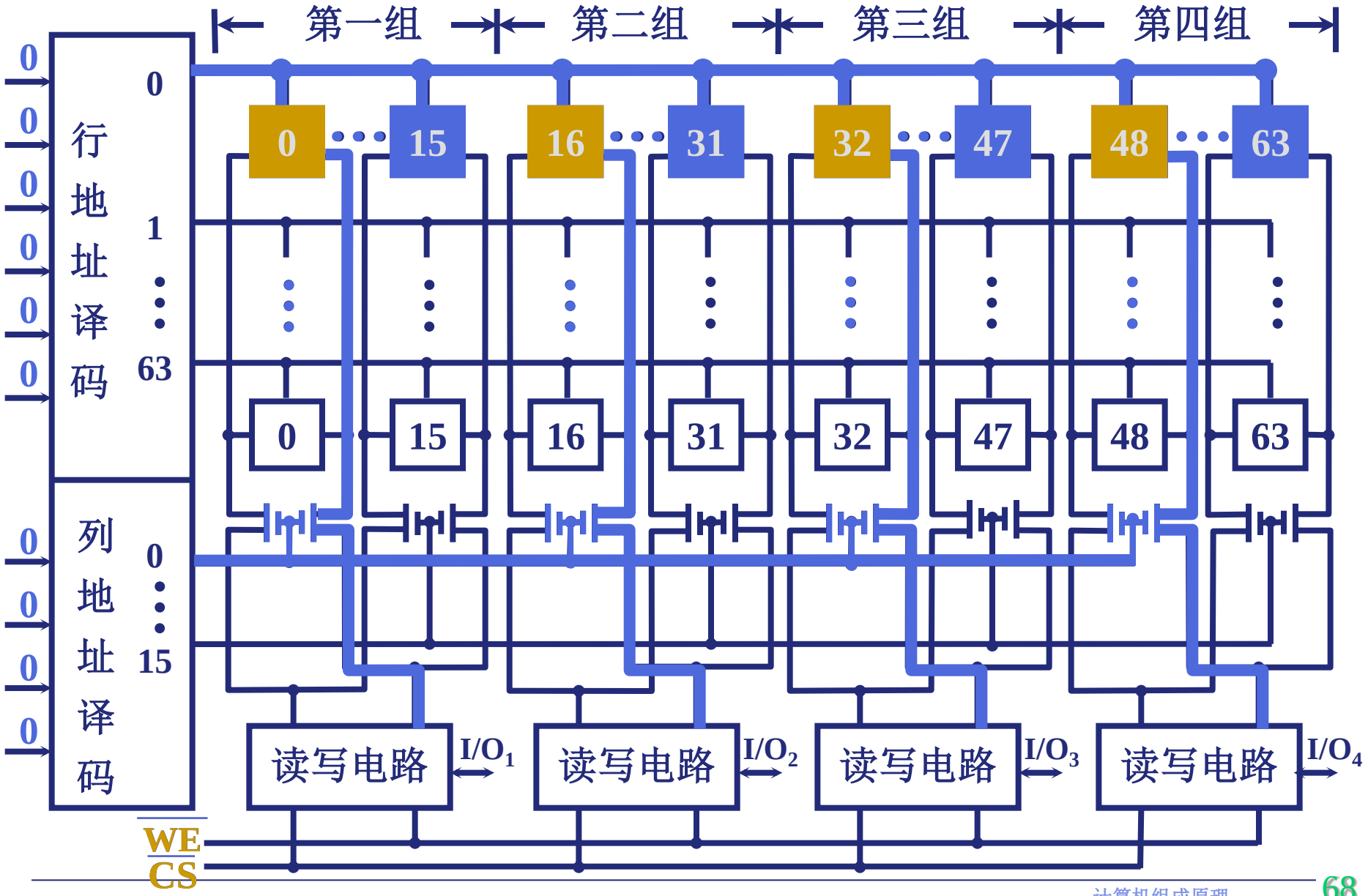
② Intel 2114 RAM 矩阵 (64 × 64) 读



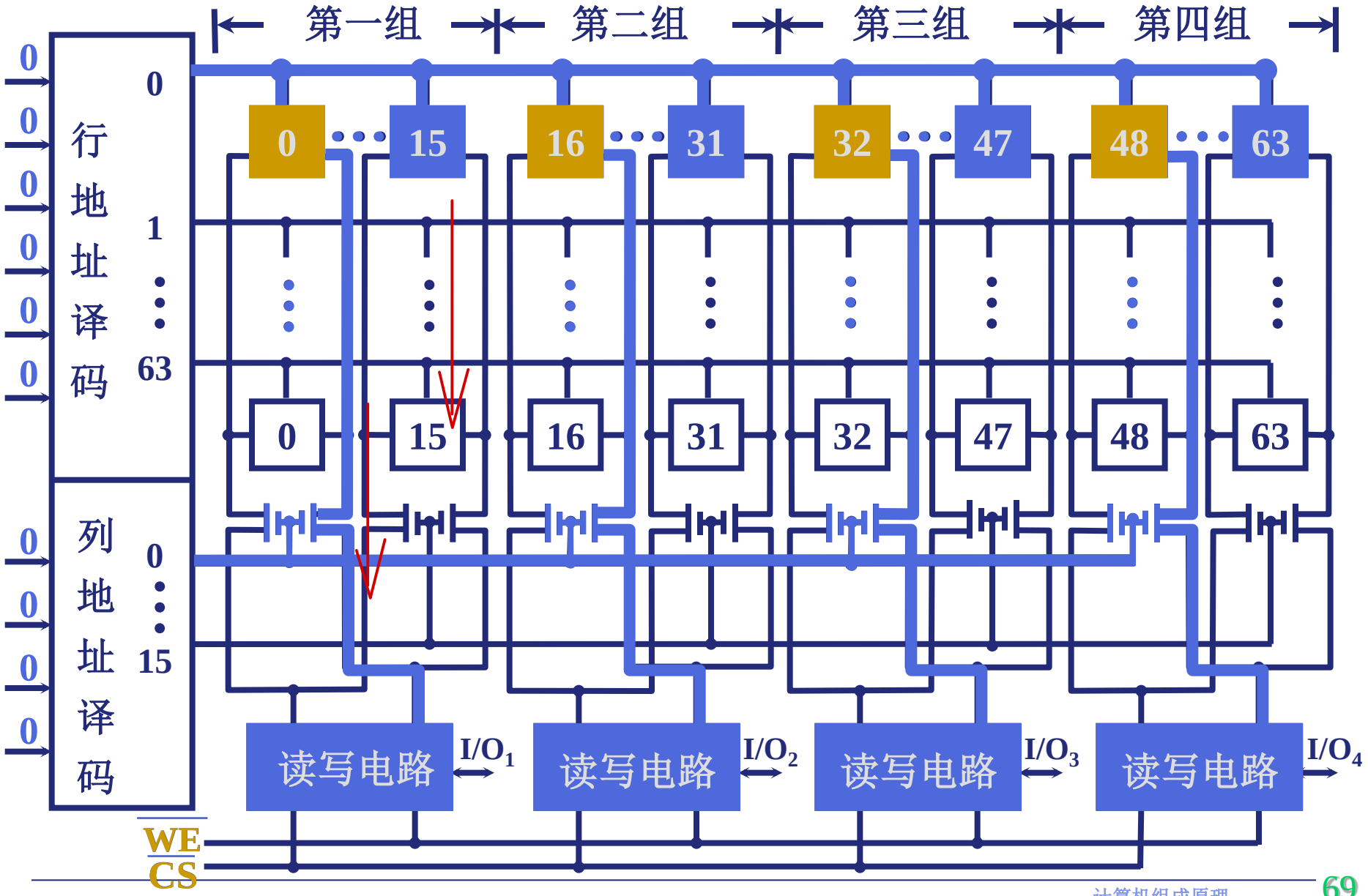
② Intel 2114 RAM 矩阵 (64 × 64) 读



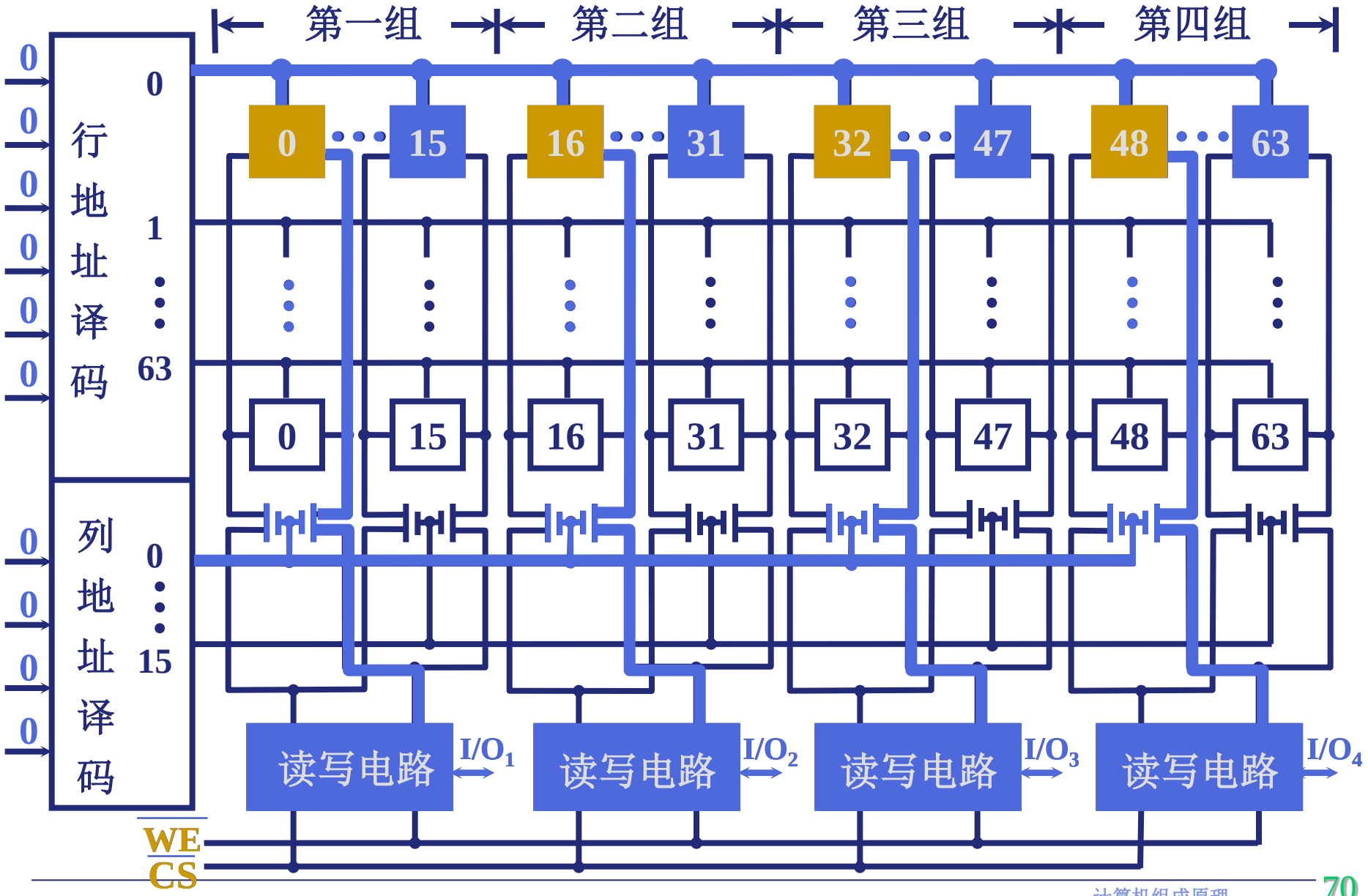
② Intel 2114 RAM 矩阵 (64 × 64) 读



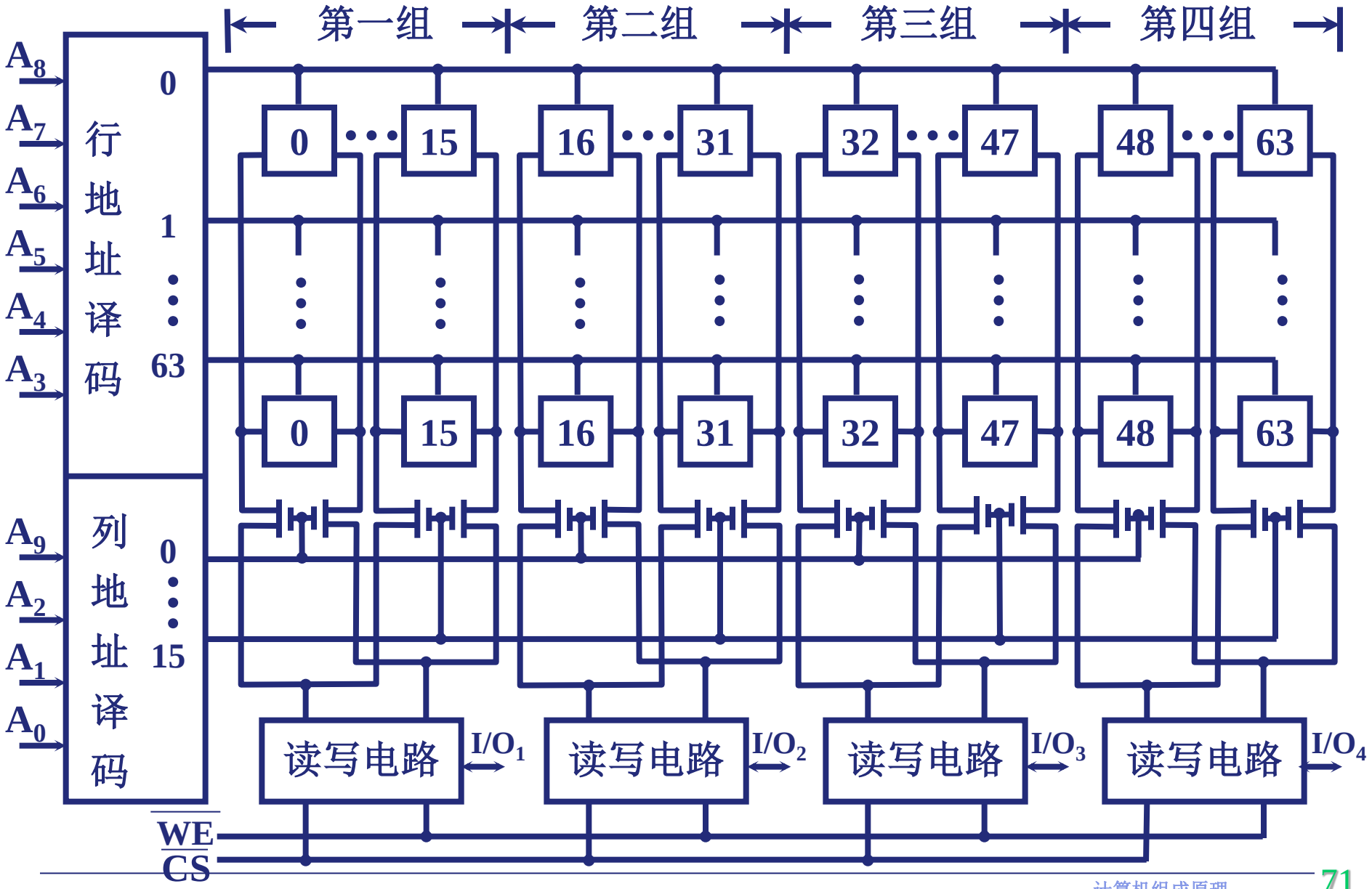
② Intel 2114 RAM 矩阵 (64 × 64) 读



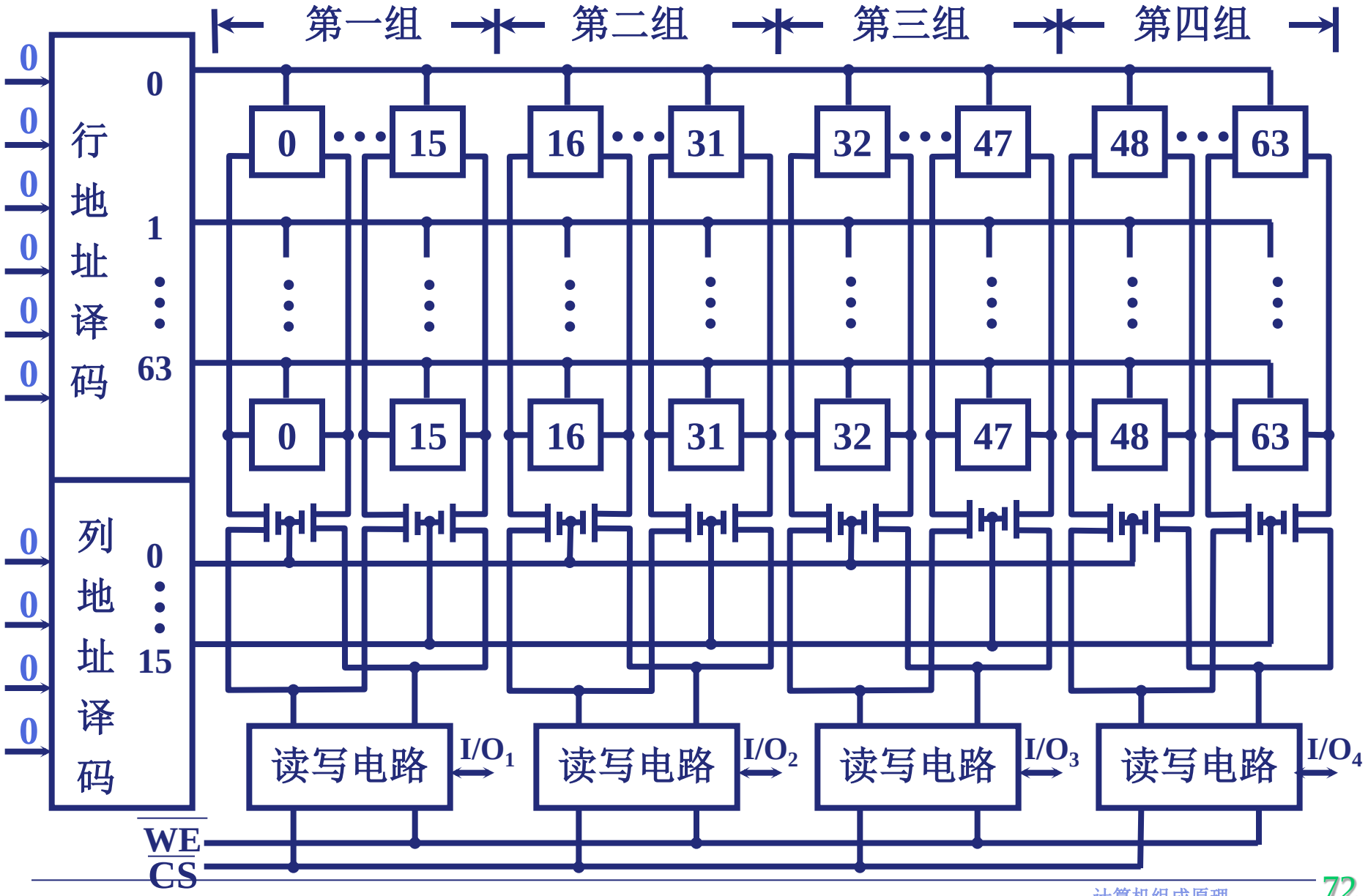
② Intel 2114 RAM 矩阵 (64 × 64) 读



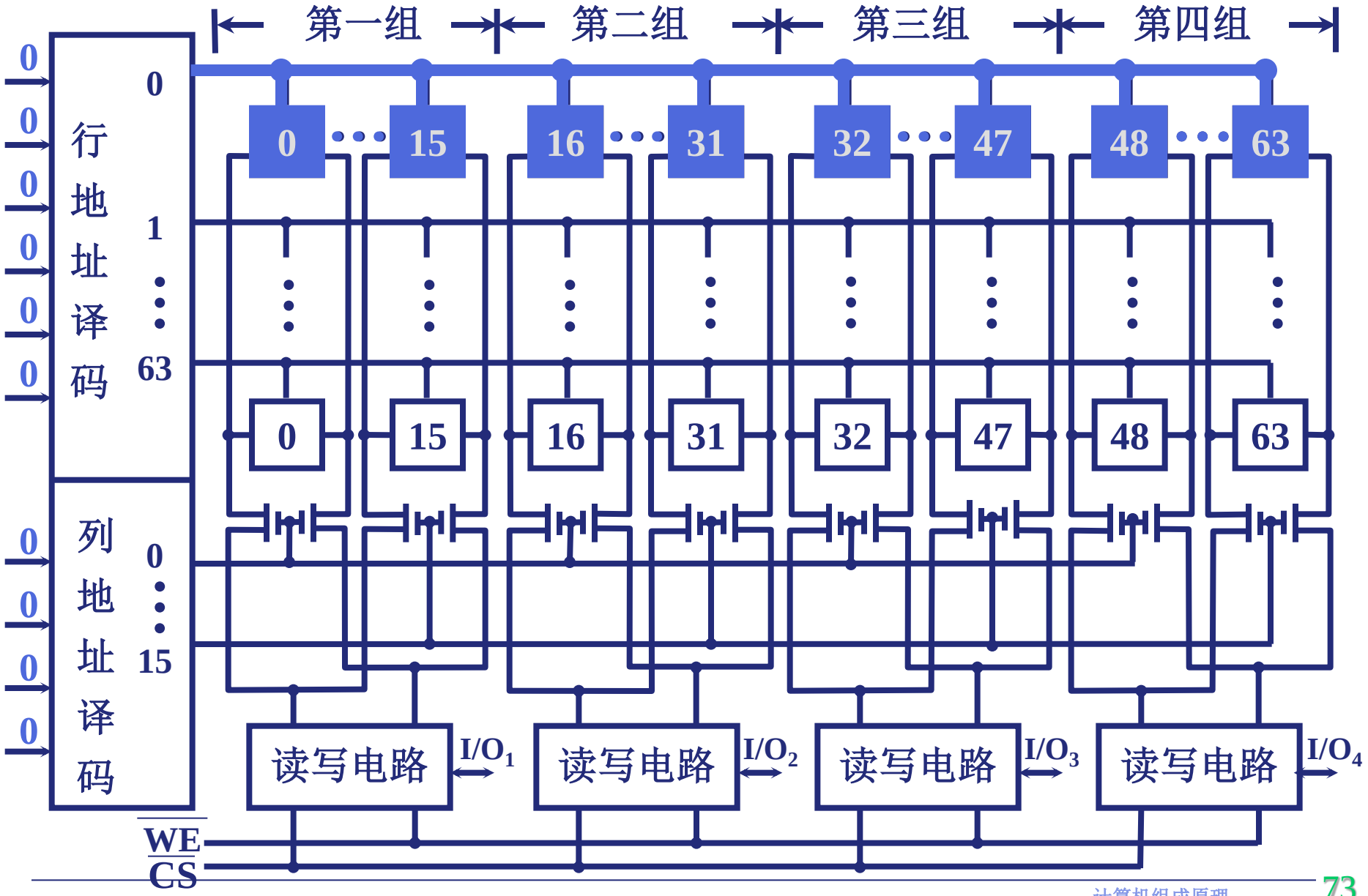
③ Intel 2114 RAM 矩阵 (64 × 64) 写



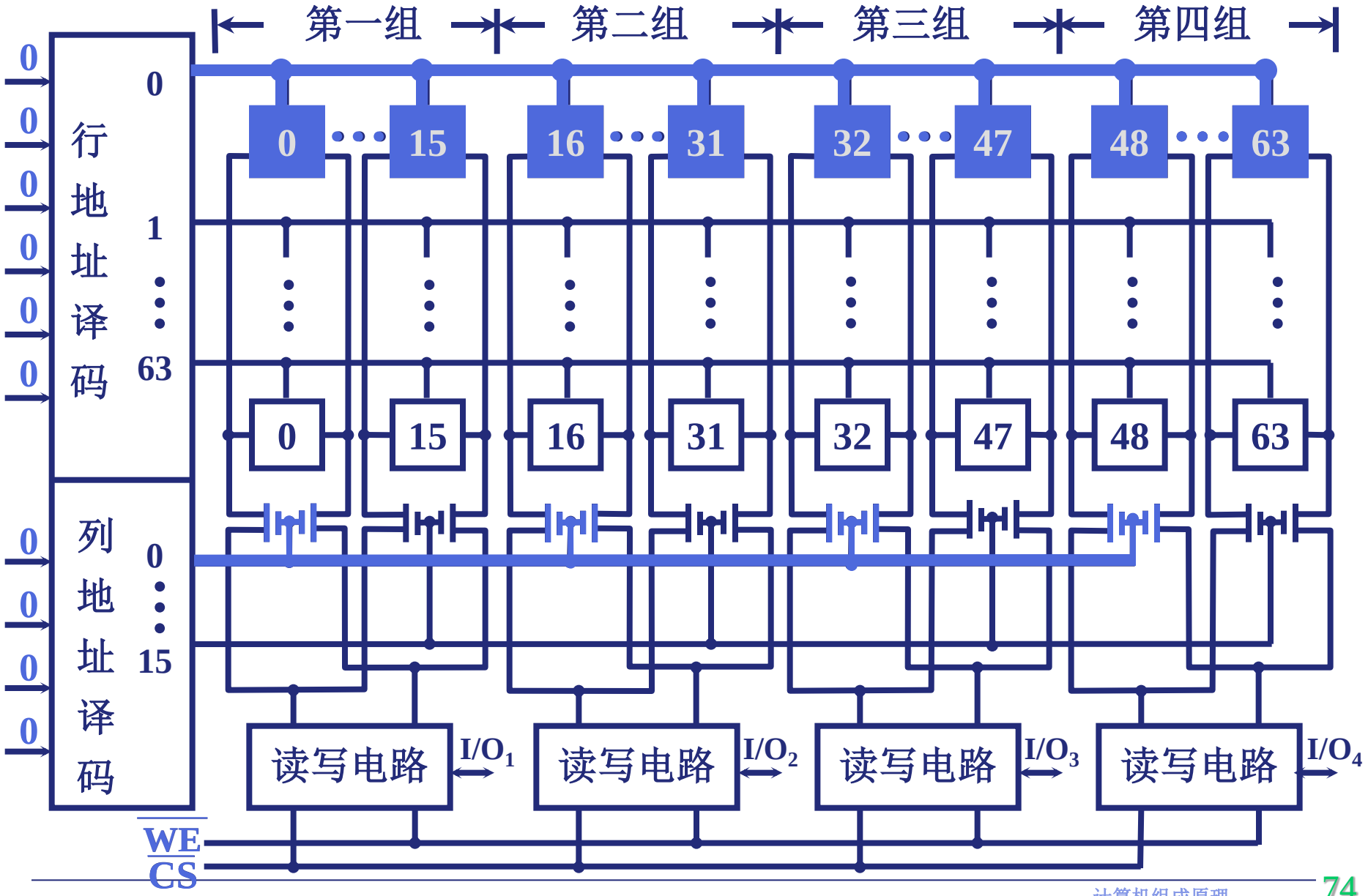
③ Intel 2114 RAM 矩阵 (64 × 64) 写



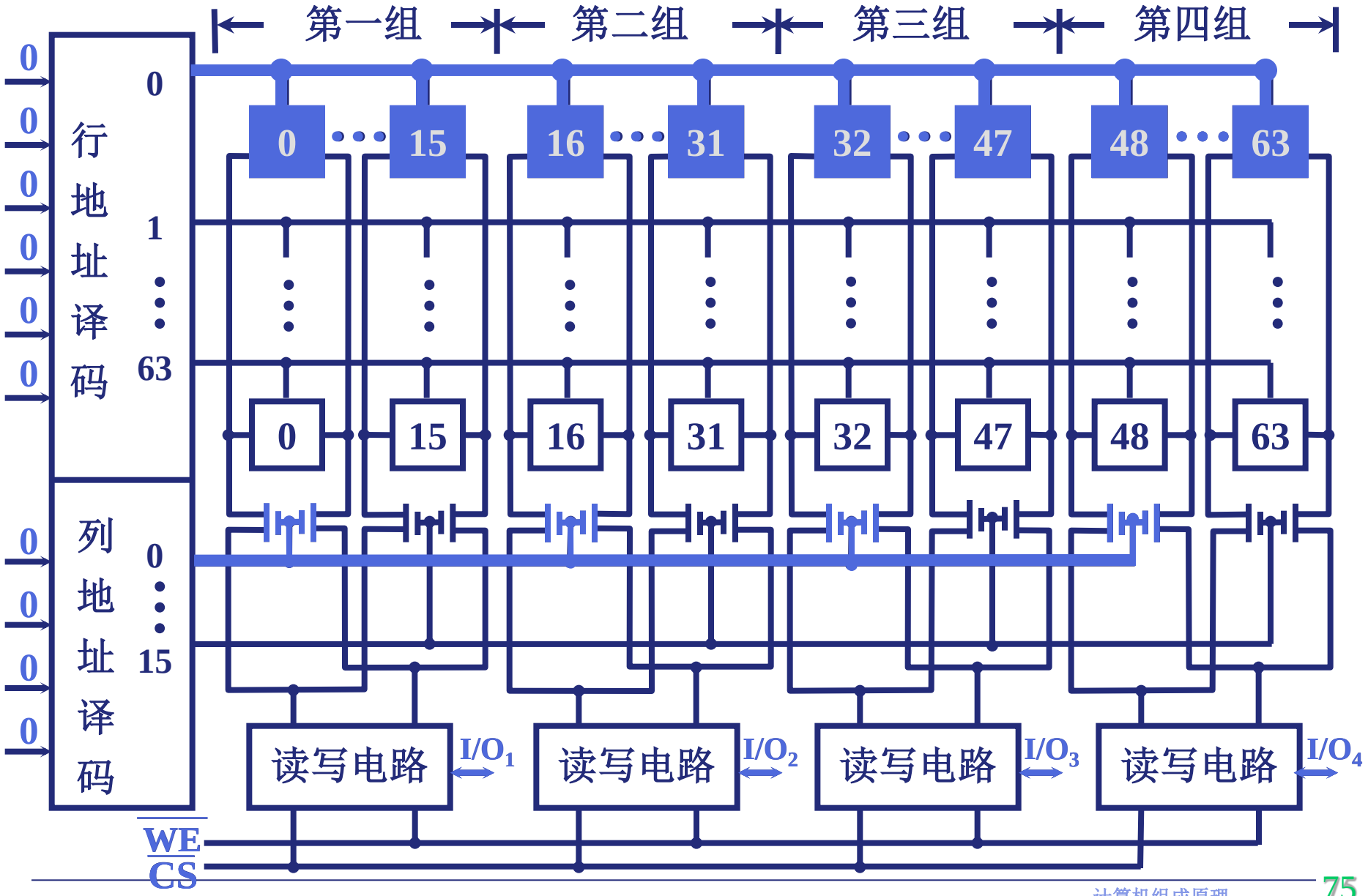
③ Intel 2114 RAM 矩阵 (64 × 64) 写



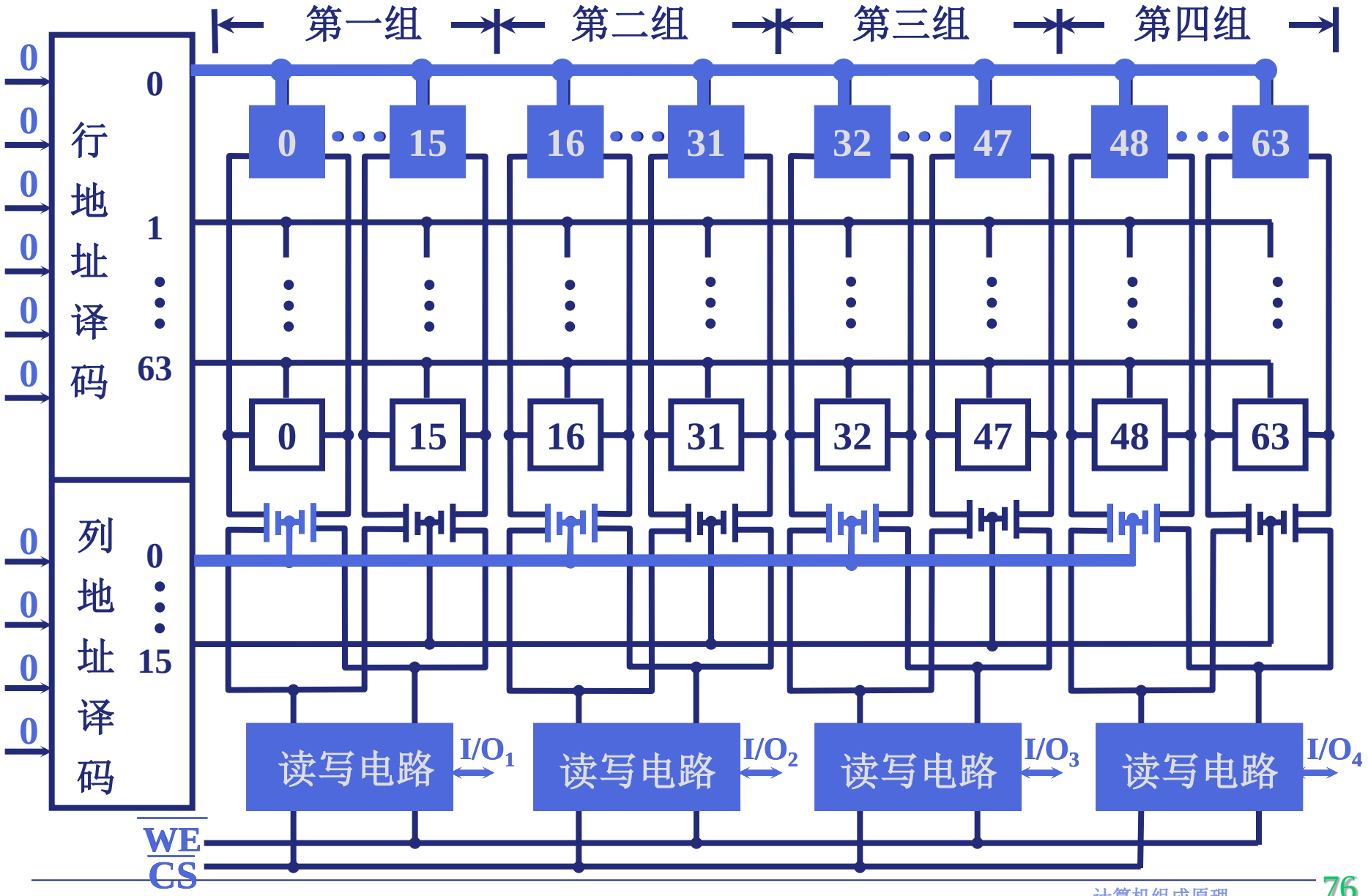
③ Intel 2114 RAM 矩阵 (64 × 64) 写



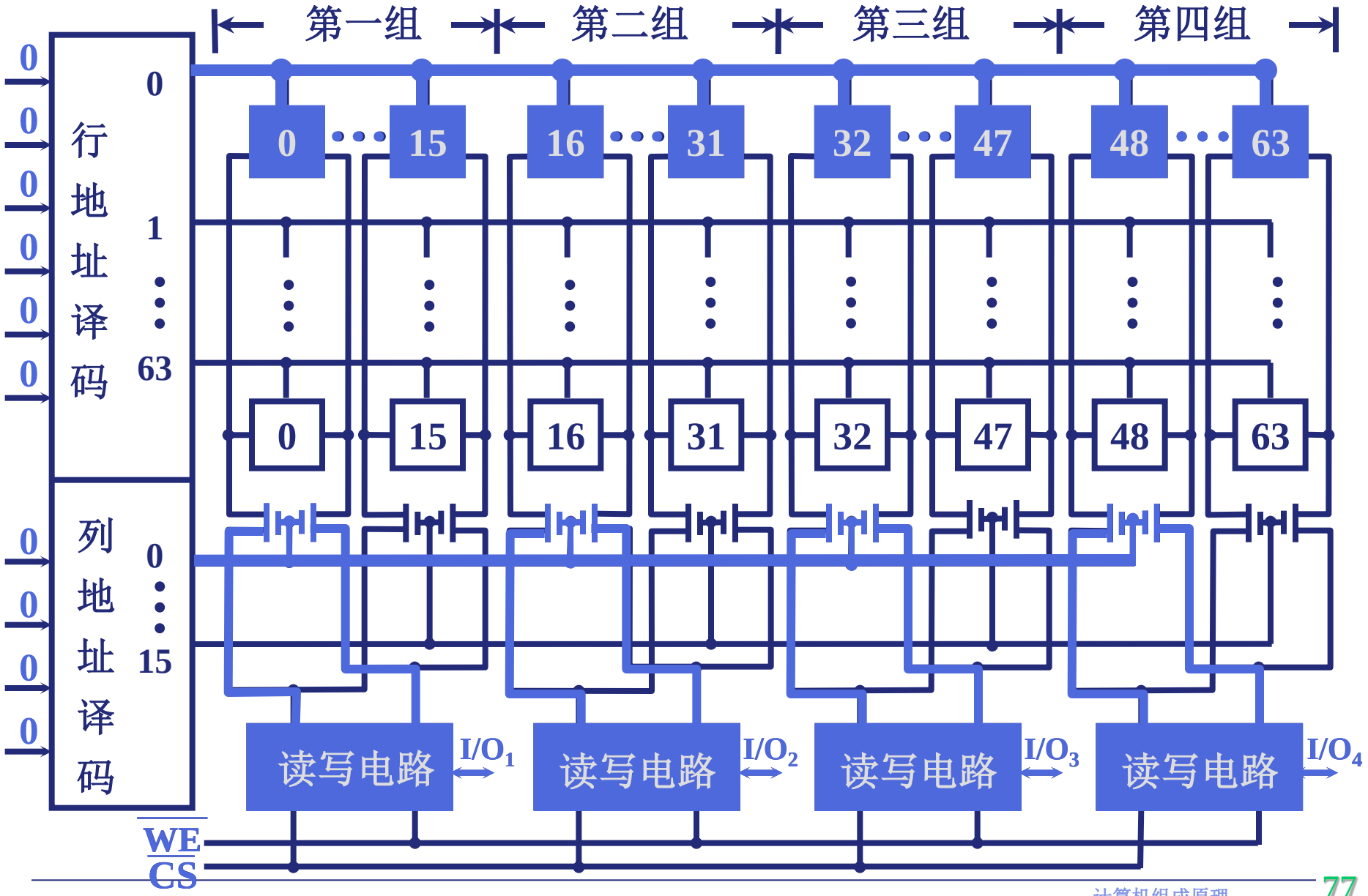
③ Intel 2114 RAM 矩阵 (64 × 64) 写



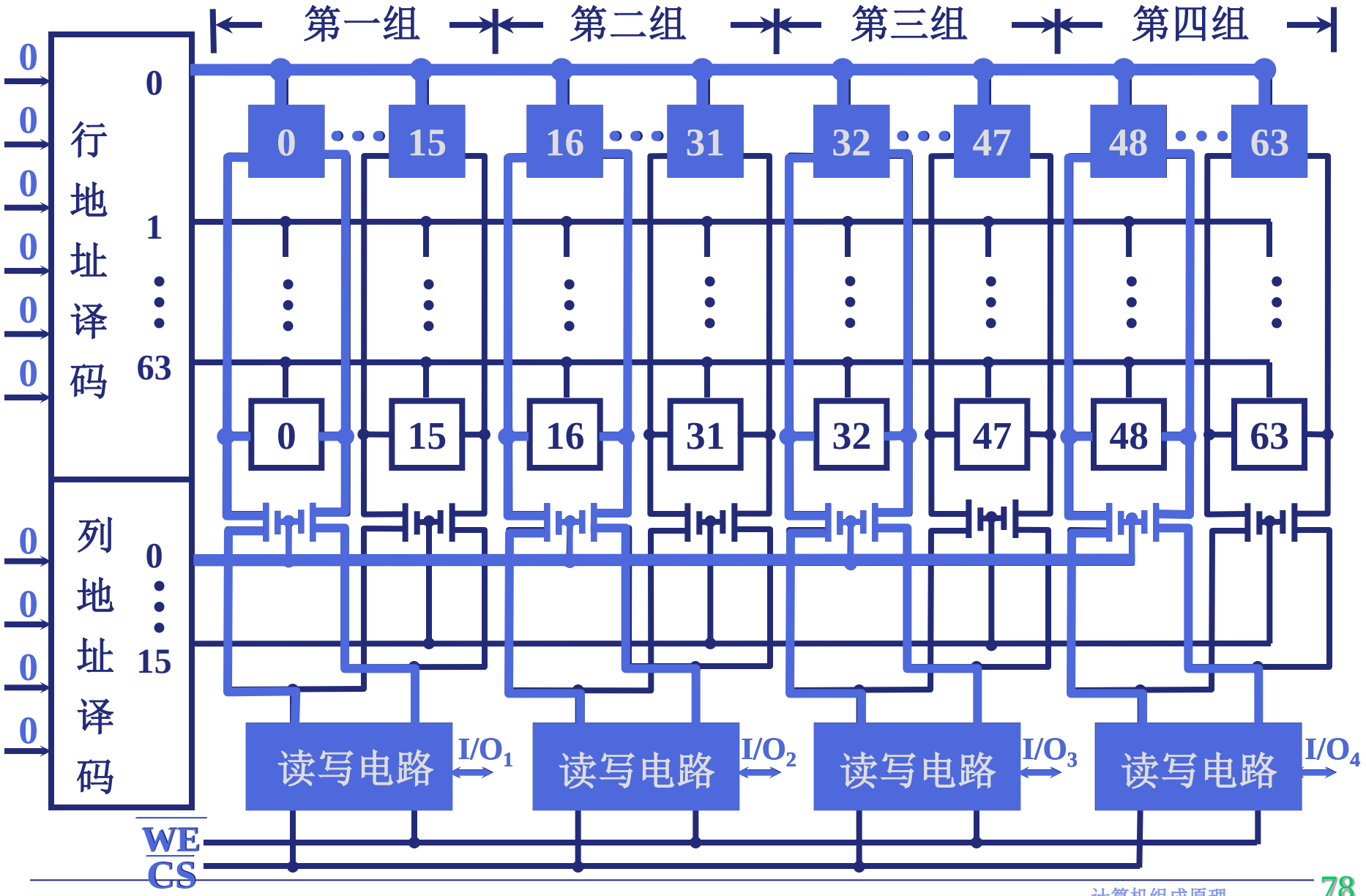
③ Intel 2114 RAM 矩阵 (64 × 64) 写



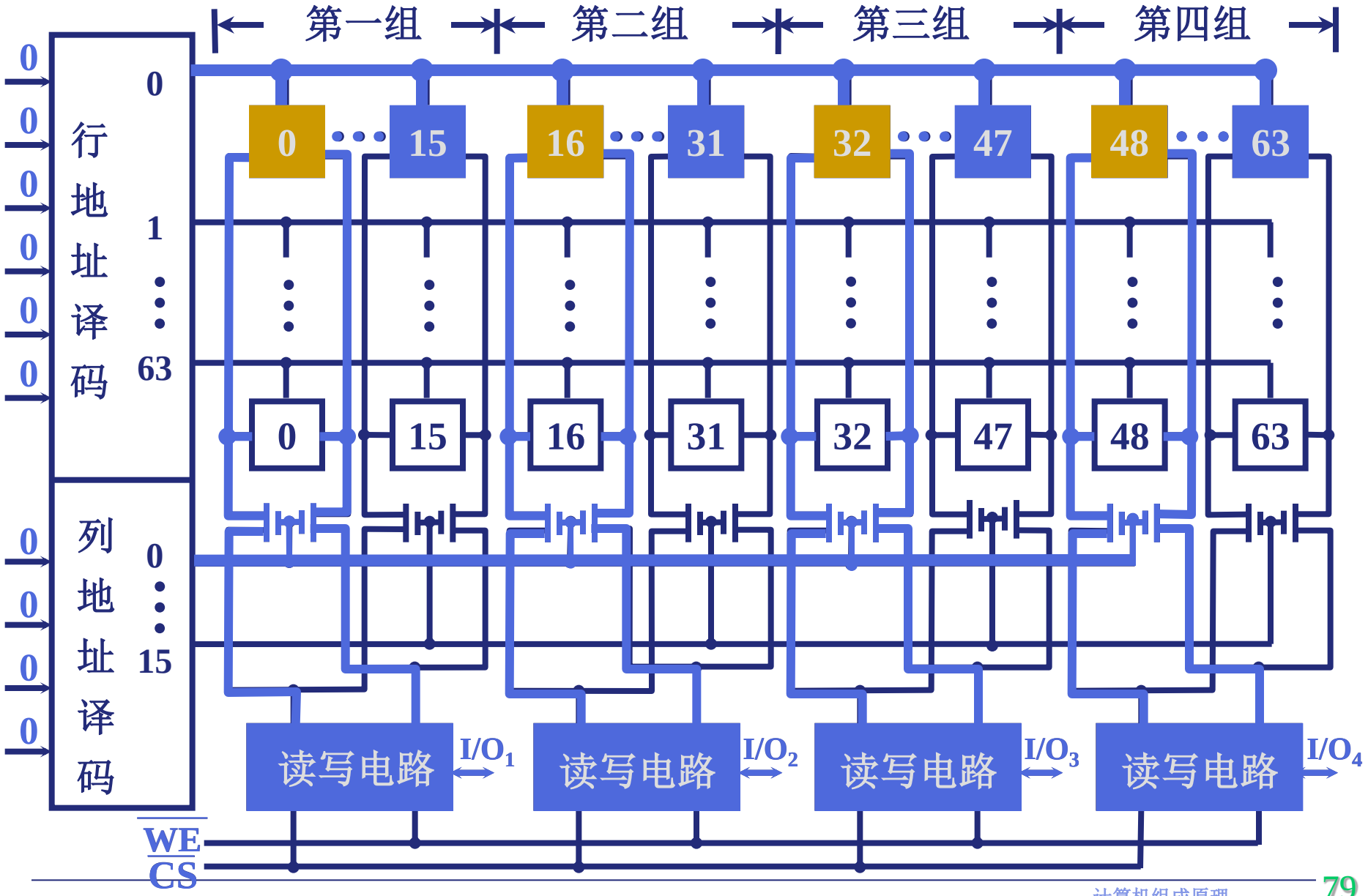
③ Intel 2114 RAM 矩阵 (64 × 64) 写



③ Intel 2114 RAM 矩阵 (64 × 64) 写



③ Intel 2114 RAM 矩阵 (64 × 64) 写

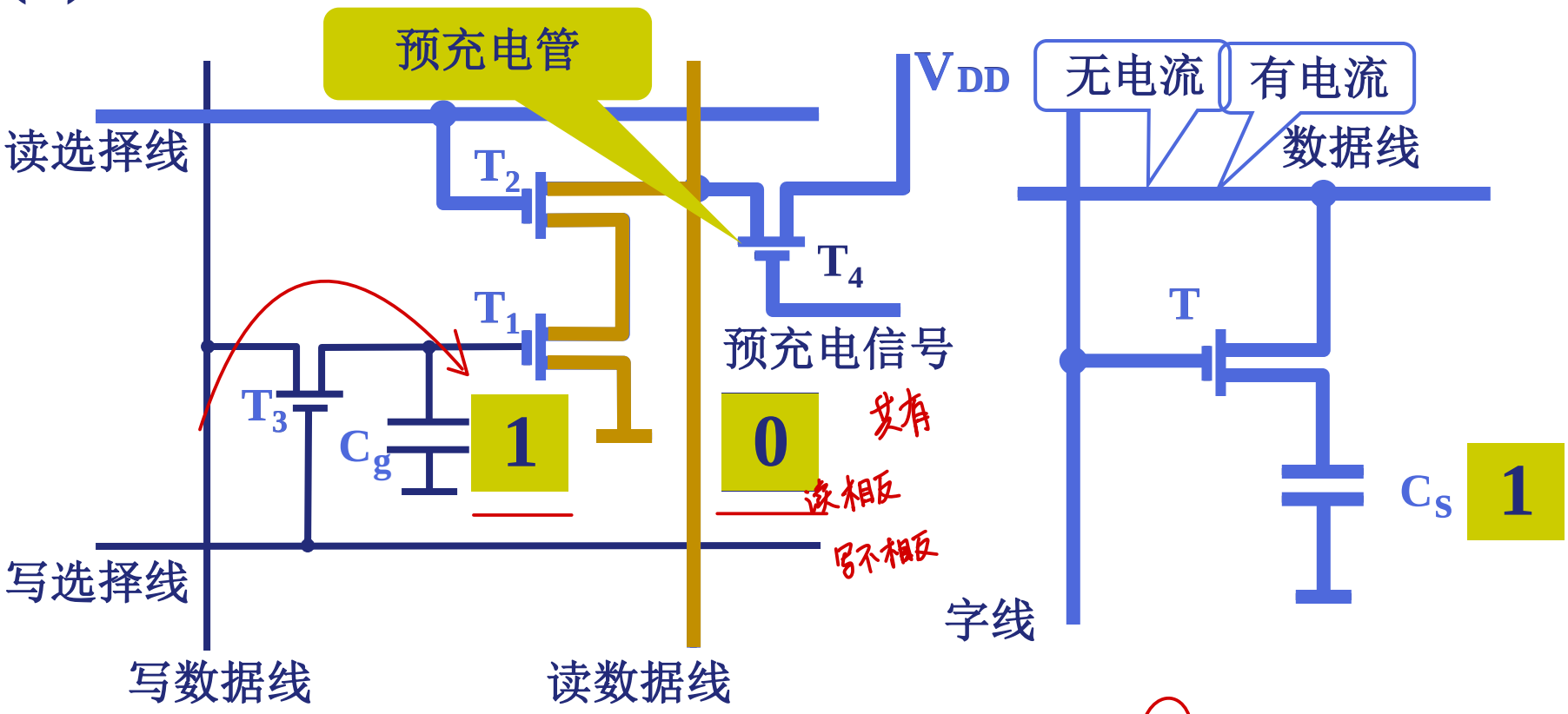


三、随机存取存储器（RAM）

- 1. 静态 RAM (SRAM，Static RAM)
- 2. 动态 RAM (DRAM，Dynamic RAM)
  - 保存0和1的原理是什么?
  - 基本单元电路的构成是什么?
  - 对单元电路如何读出和写入?
  - 典型芯片的结构是什么样子的?
  - 动态RAM芯片的如何进行读出和写入操作?
  - 动态RAM为什么要刷新，刷新方法
- 3. 动态RAM和静态RAM的比较

2. 动态 RAM (DRAM)

(1) 动态 RAM 基本单元电路



读出与原存信息相反

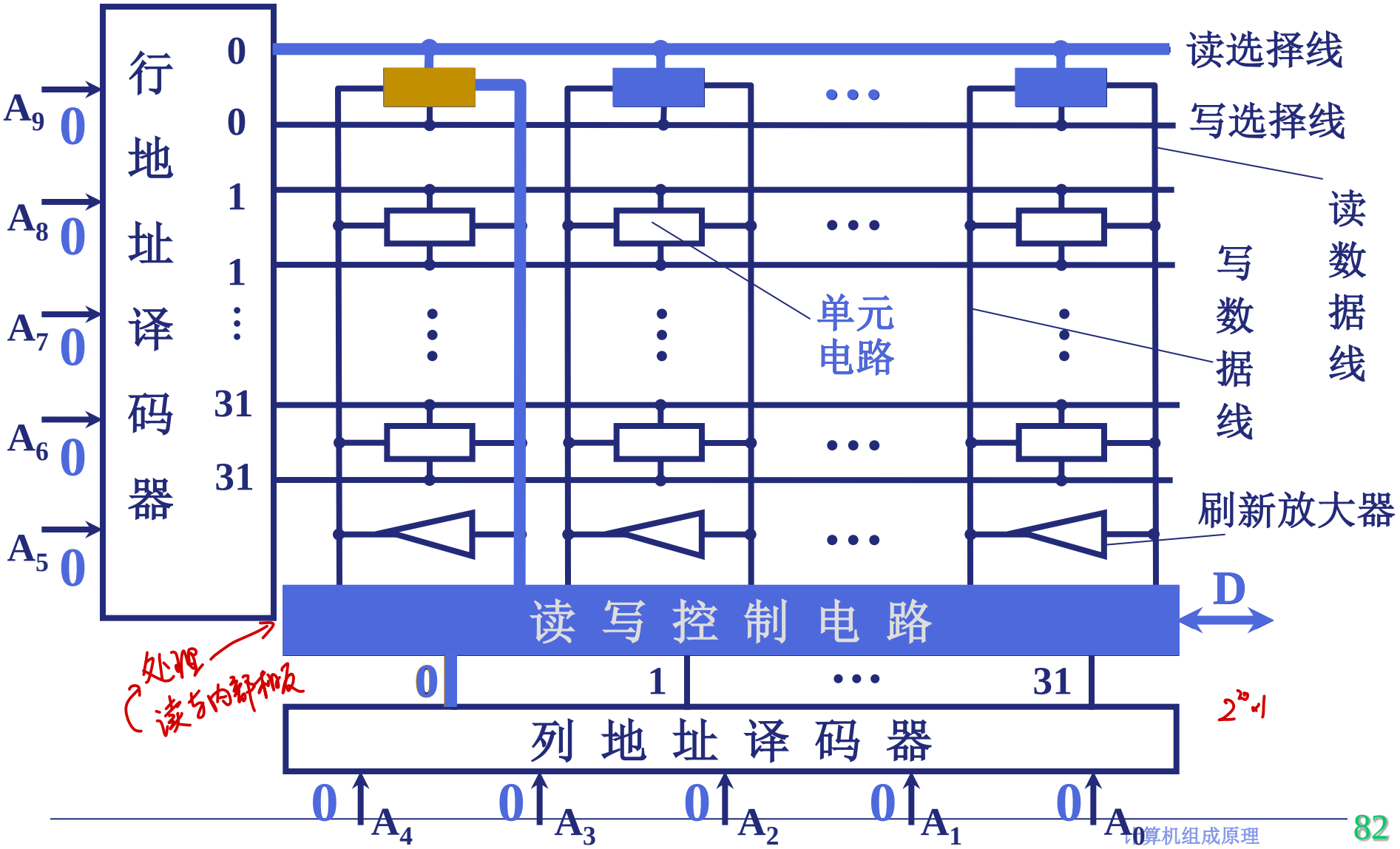
写入与输入信息相同

读出时数据线有电流 为 “1”

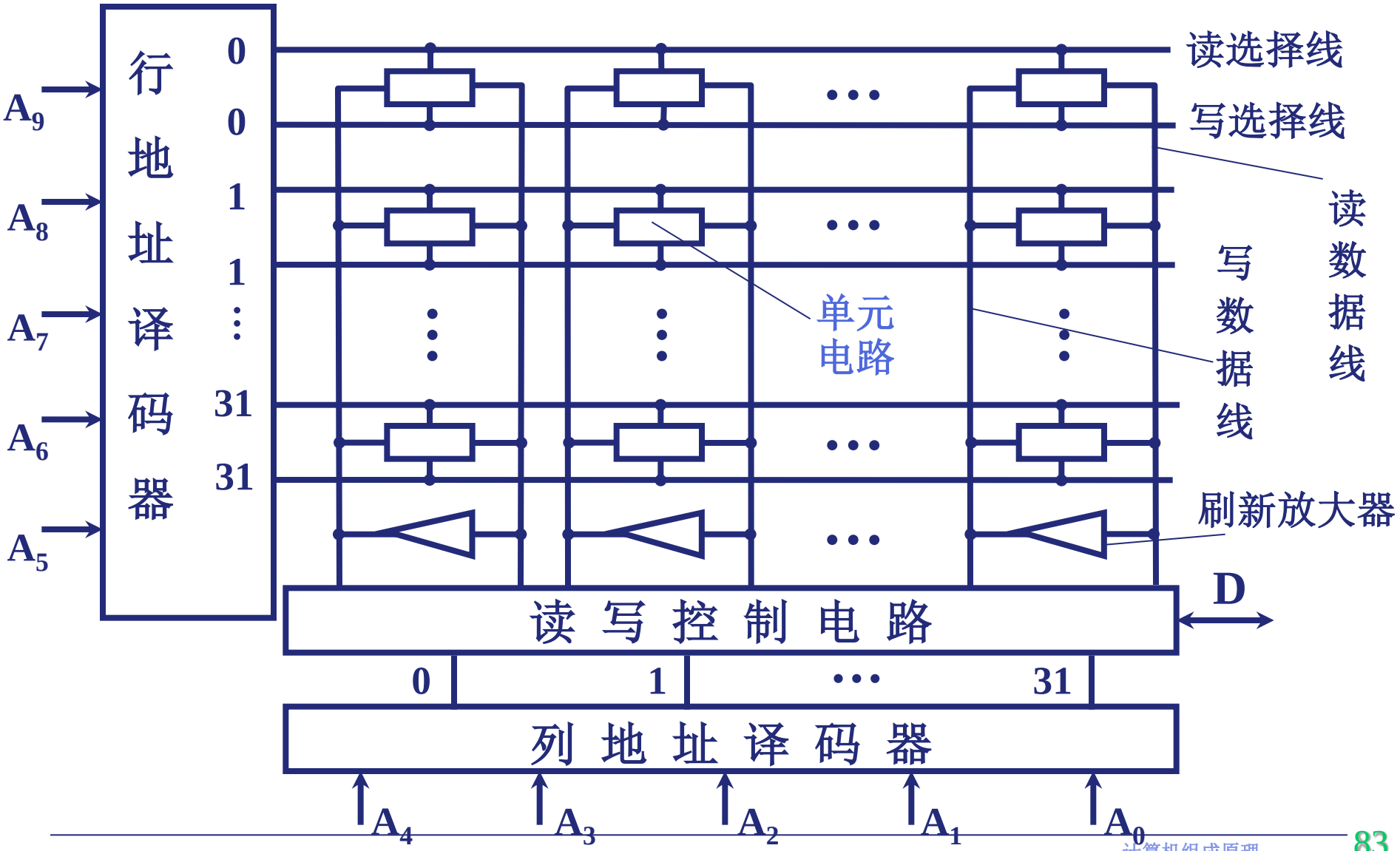
写入时C_S充电 为 “1” 放电 为”0”

(2) 动态 RAM 芯片举例

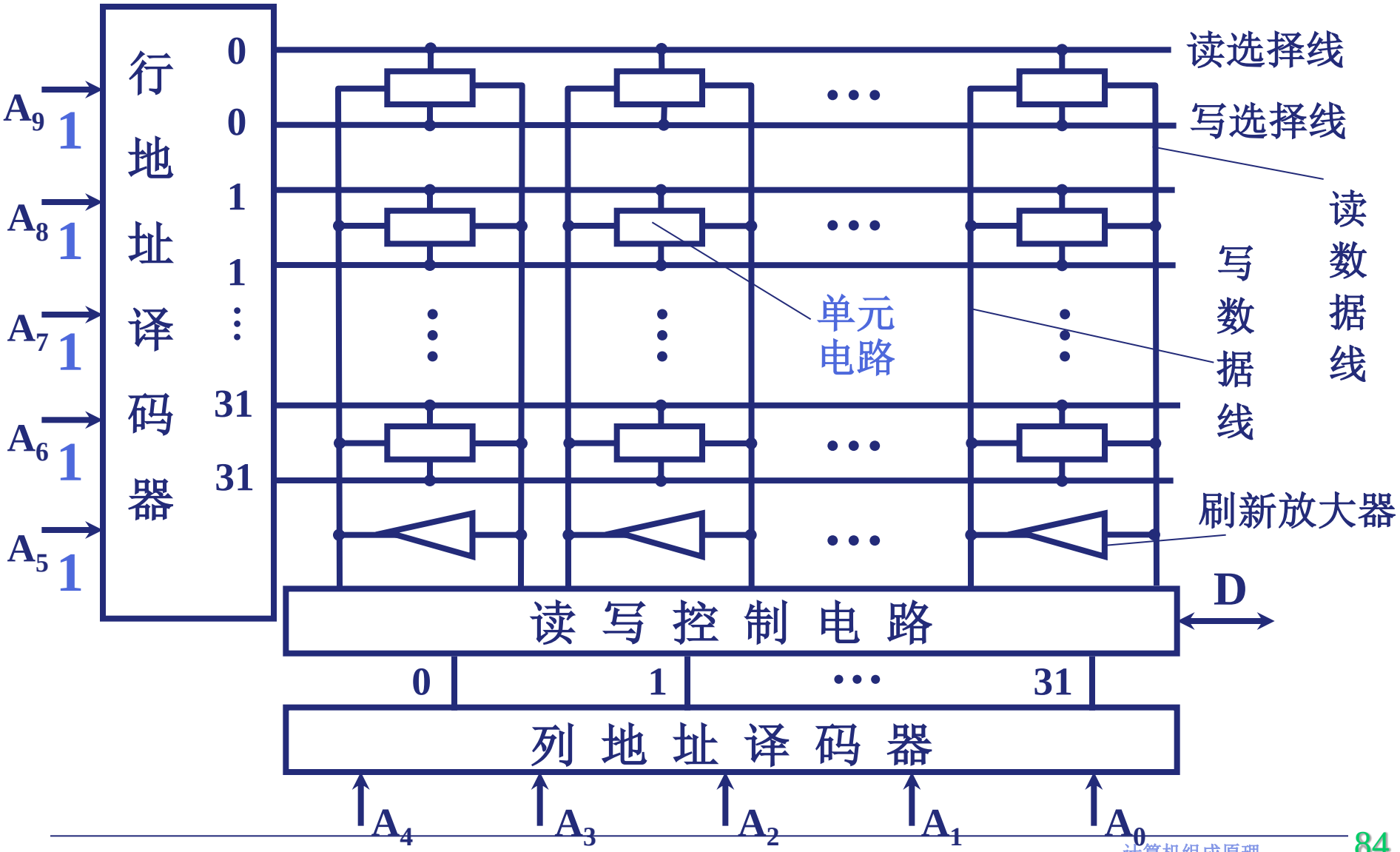
① 三管动态 RAM 芯片 (Intel 1103) 读



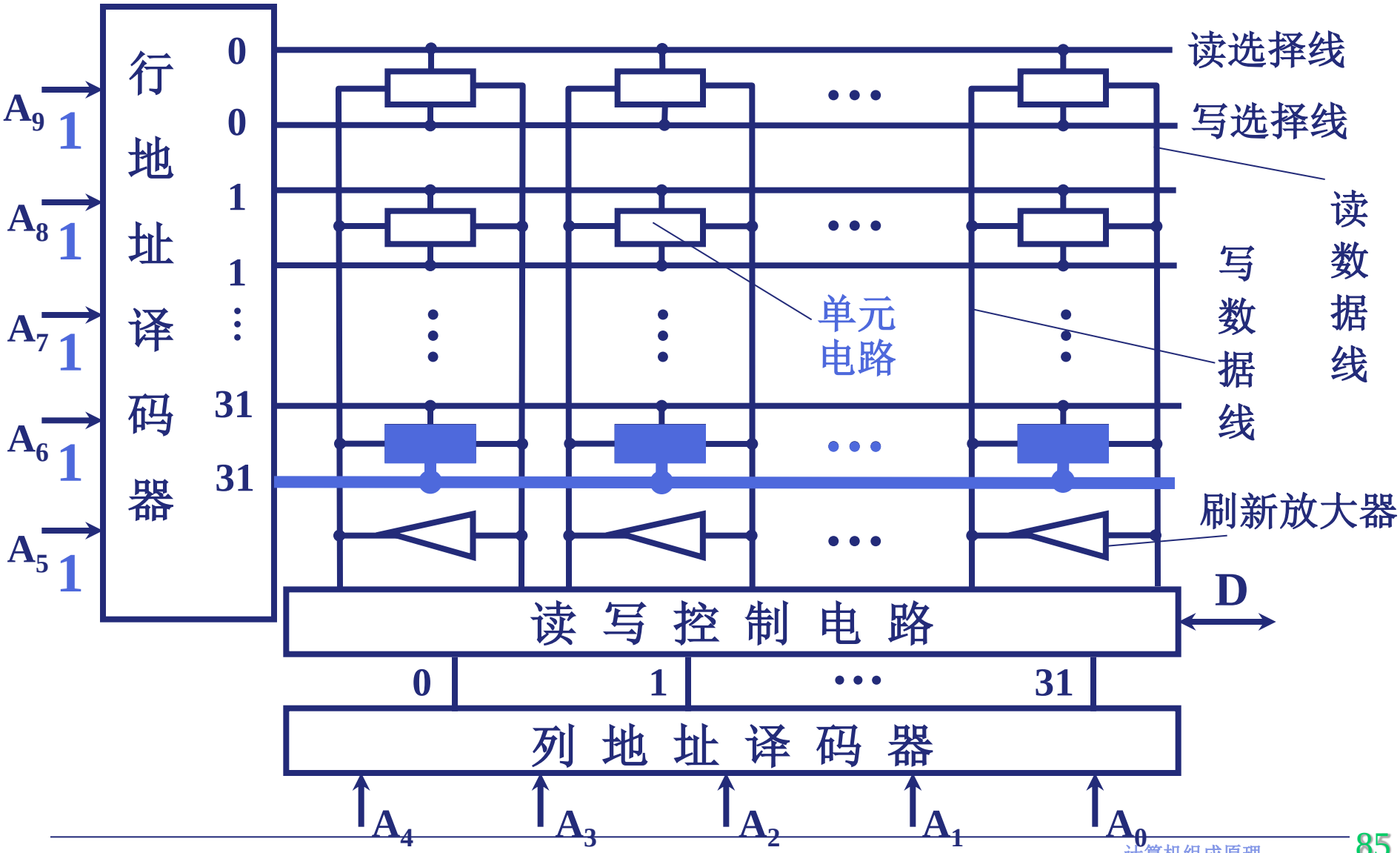
② 三管动态 RAM 芯片 (Intel 1103) 写



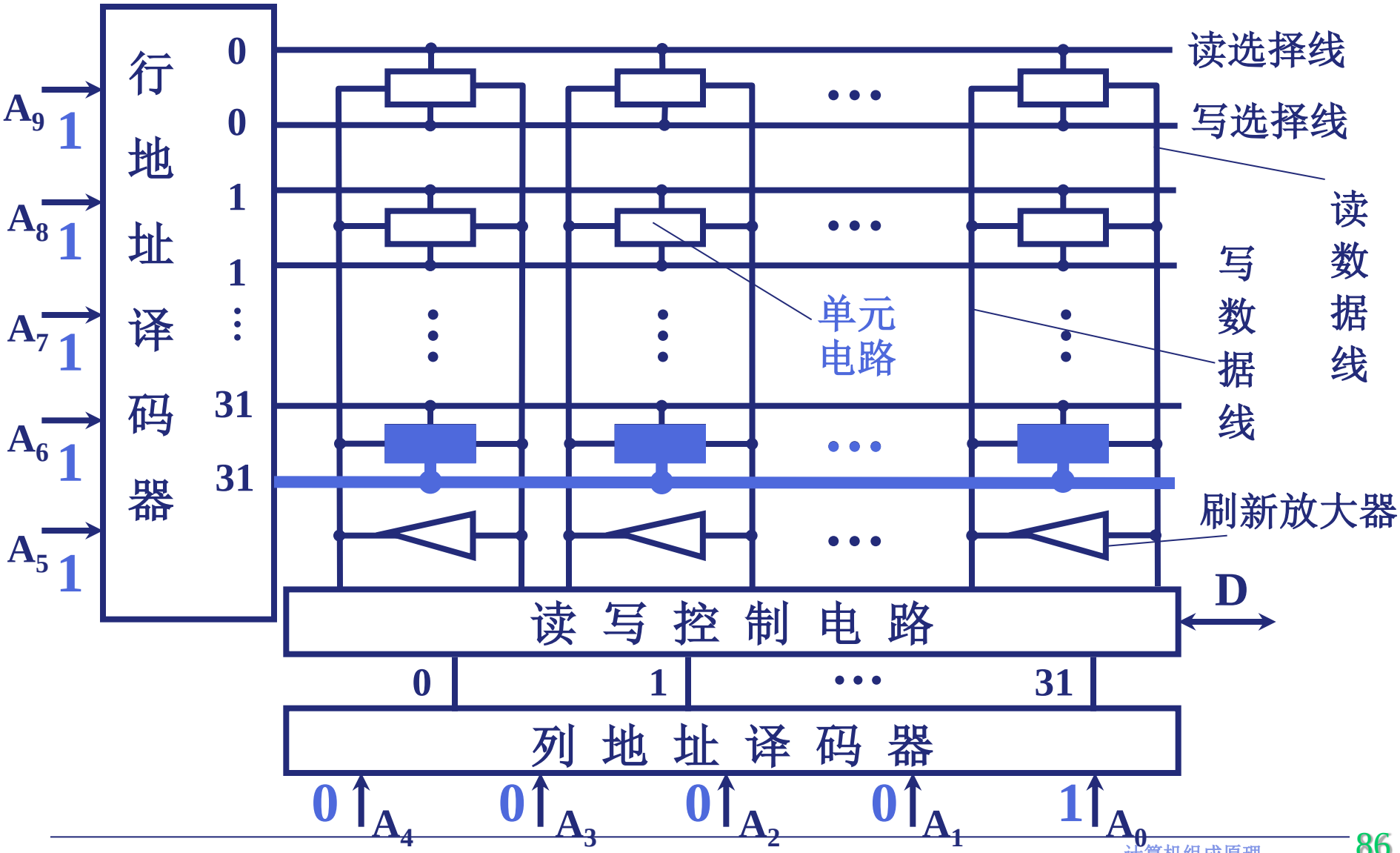
② 三管动态 RAM 芯片 (Intel 1103) 写



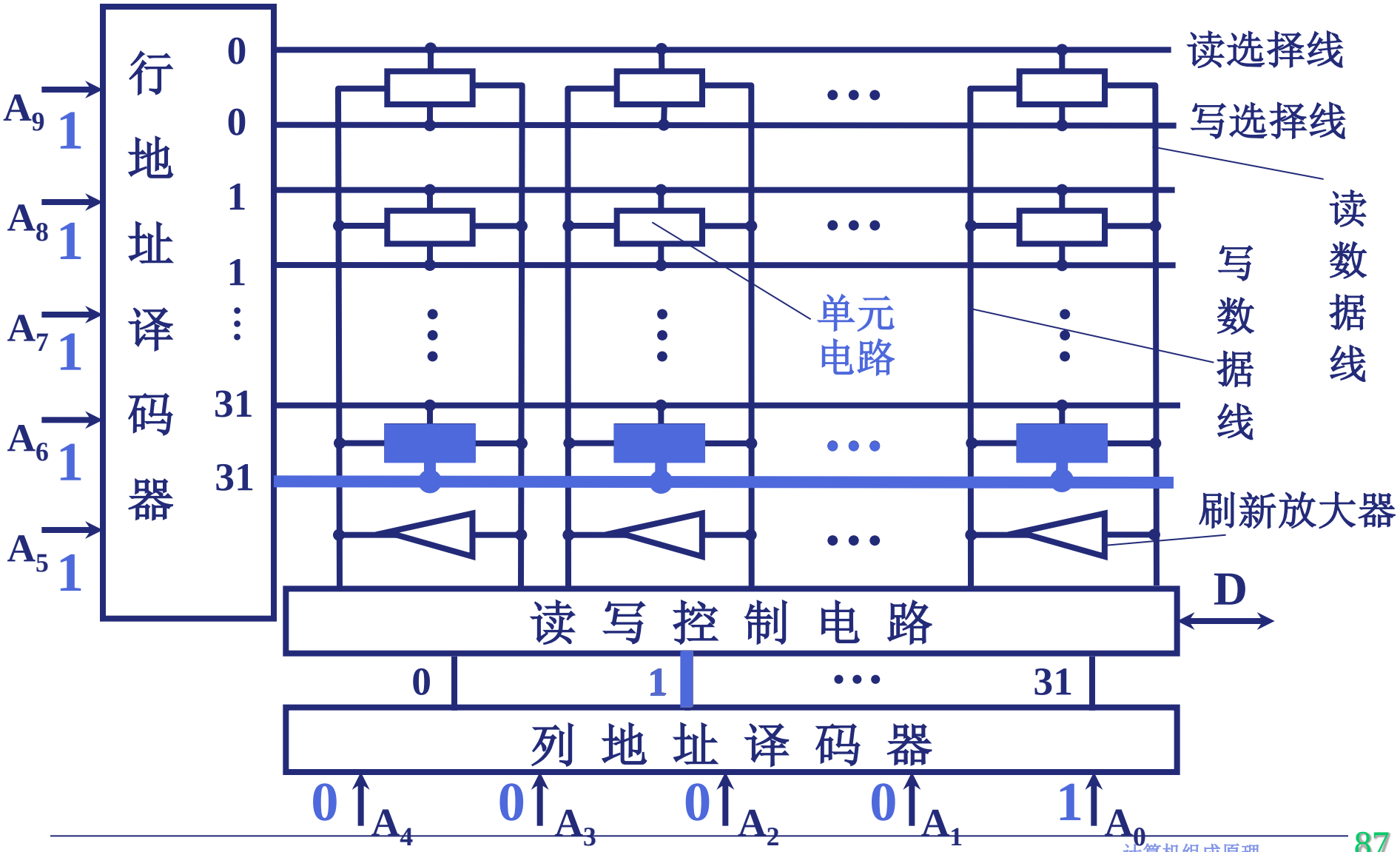
② 三管动态 RAM 芯片 (Intel 1103) 写



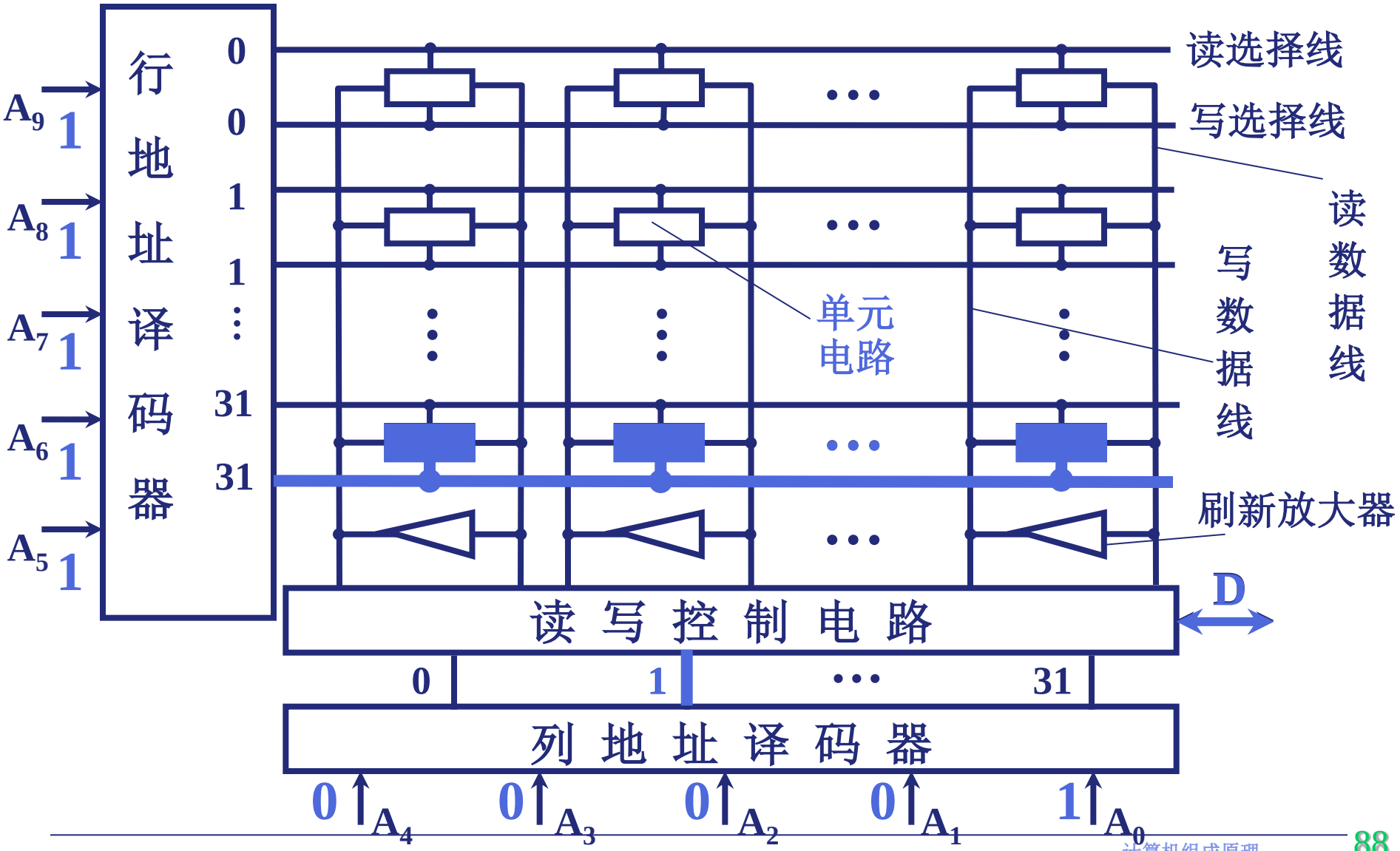
② 三管动态 RAM 芯片 (Intel 1103) 写



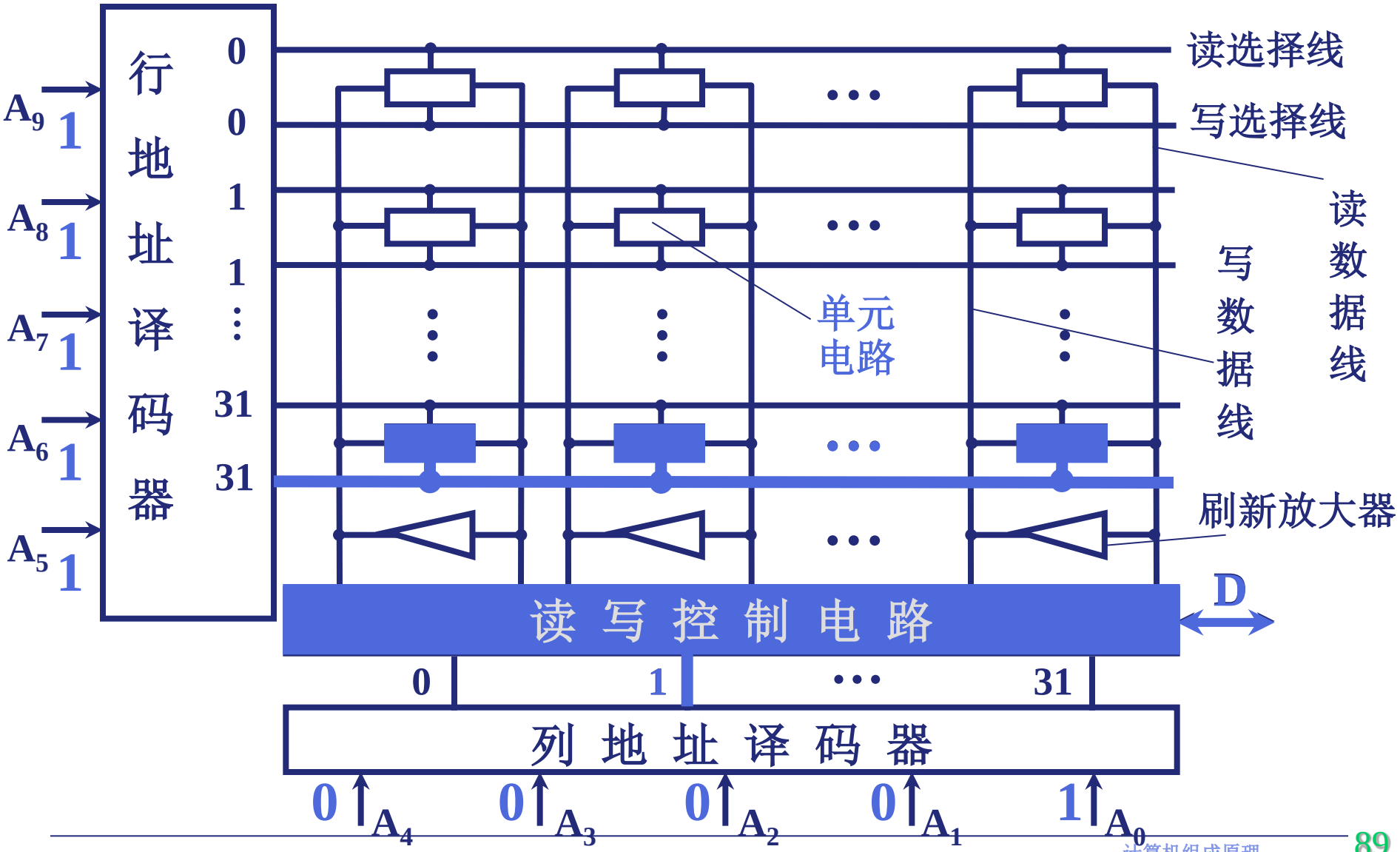
② 三管动态 RAM 芯片 (Intel 1103) 写



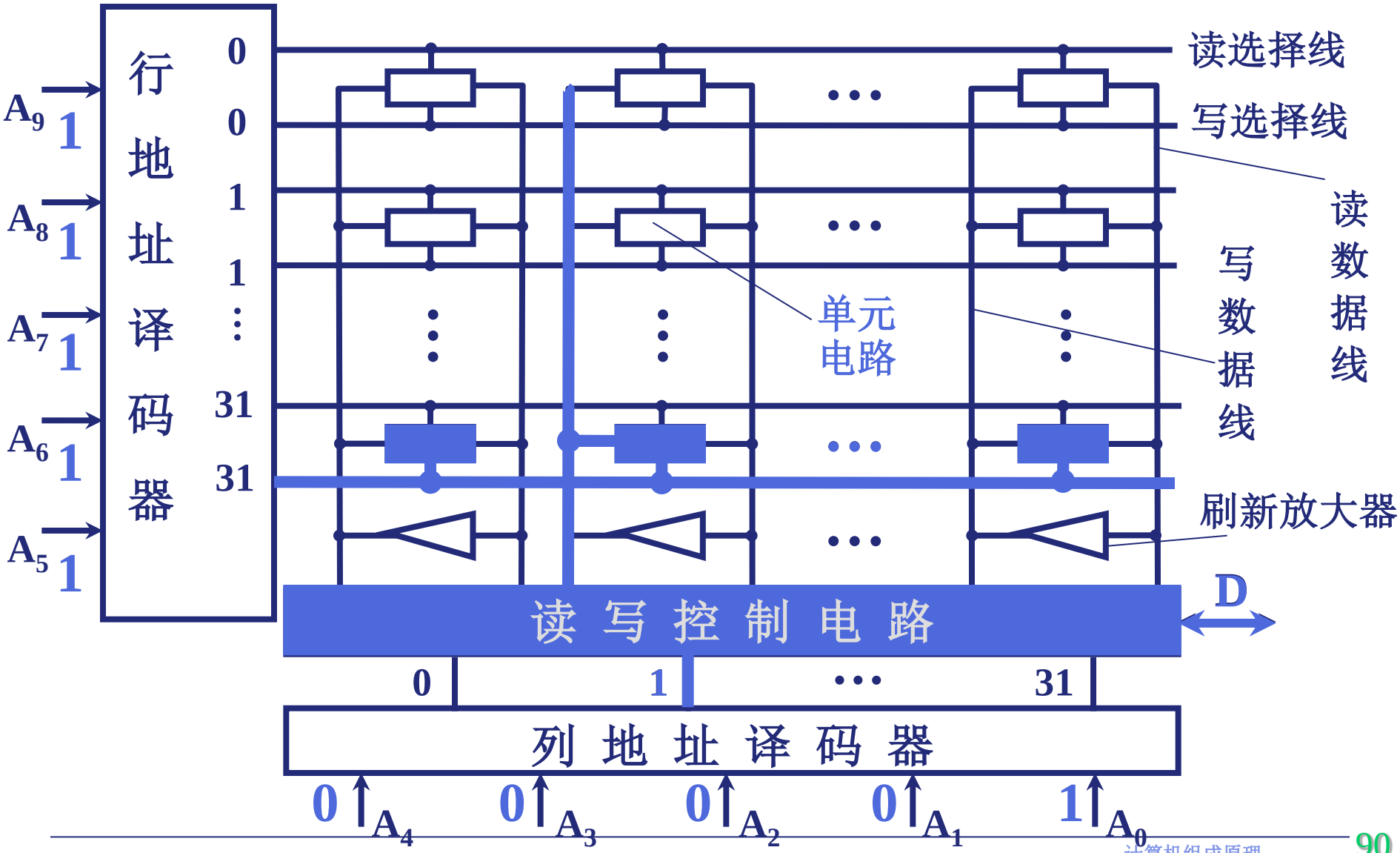
② 三管动态 RAM 芯片 (Intel 1103) 写



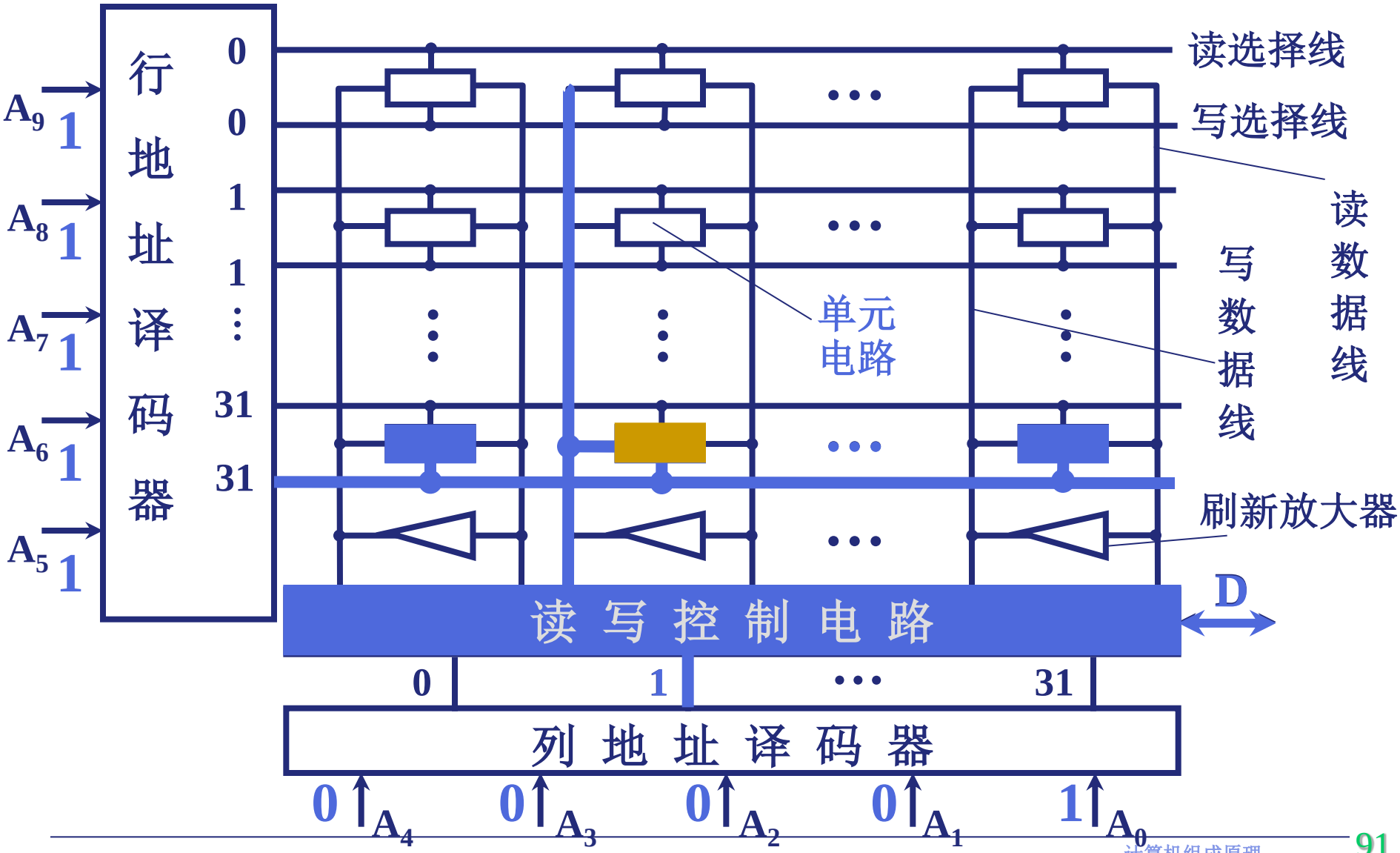
② 三管动态 RAM 芯片 (Intel 1103) 写



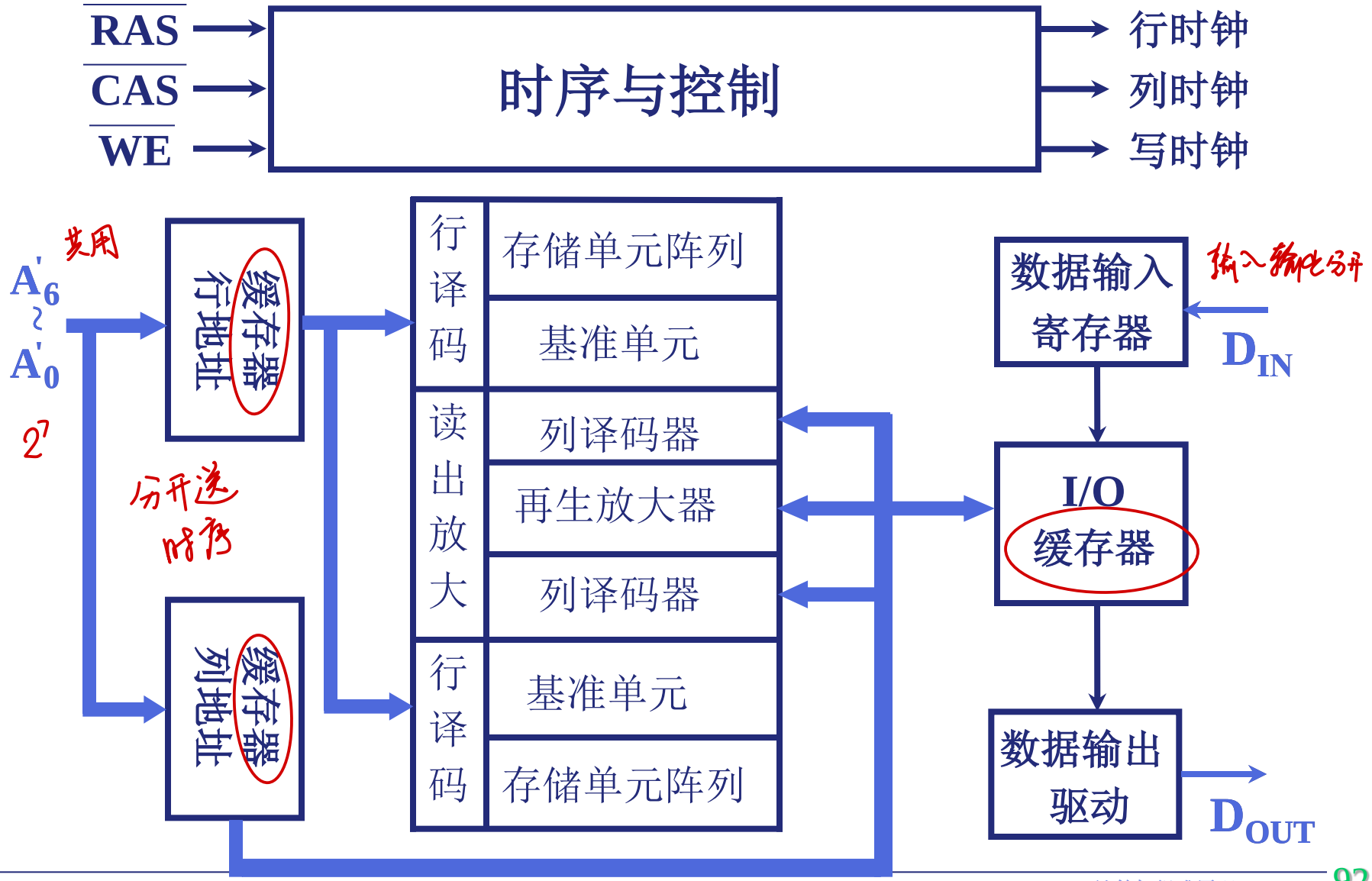
② 三管动态 RAM 芯片 (Intel 1103) 写



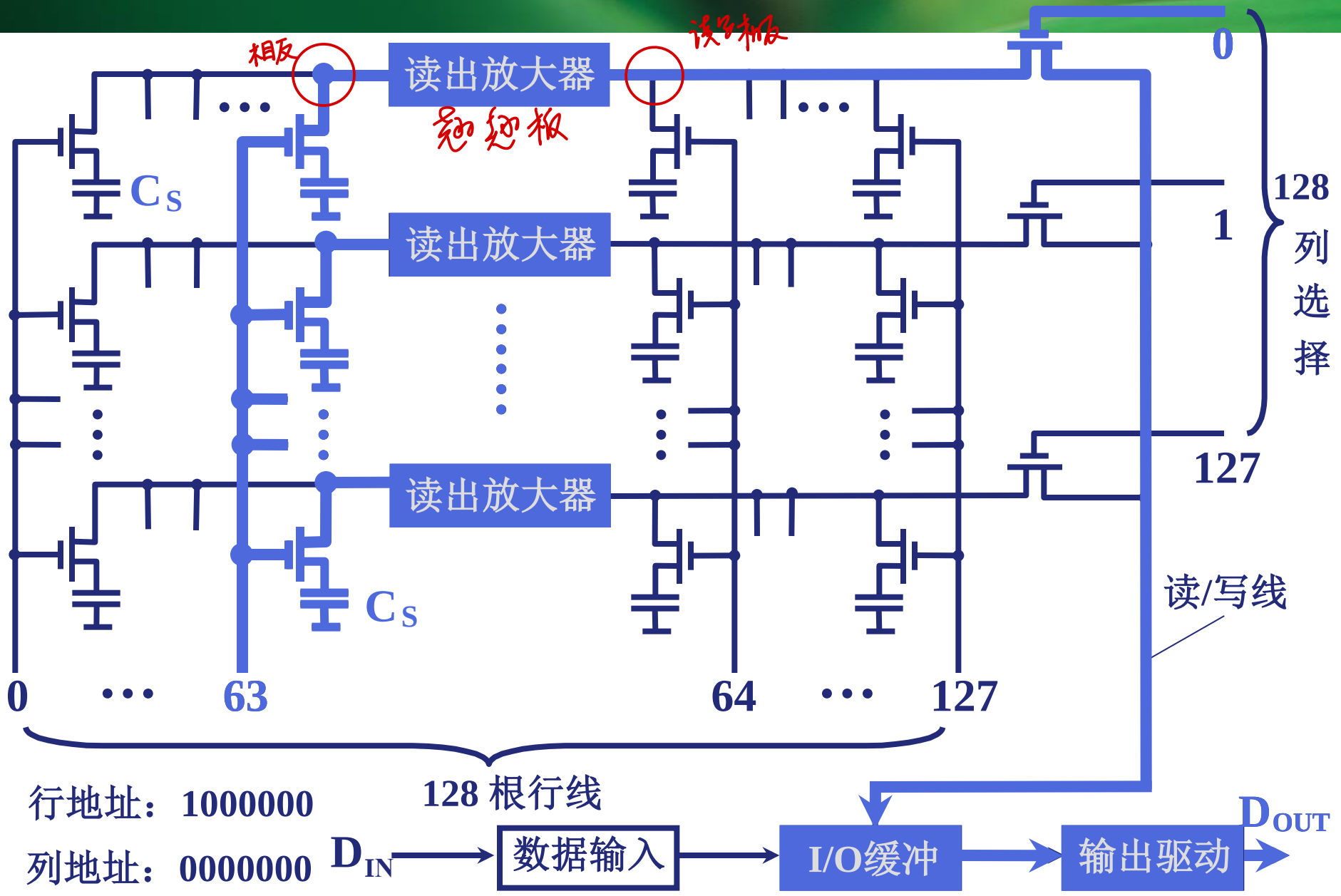
② 三管动态 RAM 芯片 (Intel 1103) 写



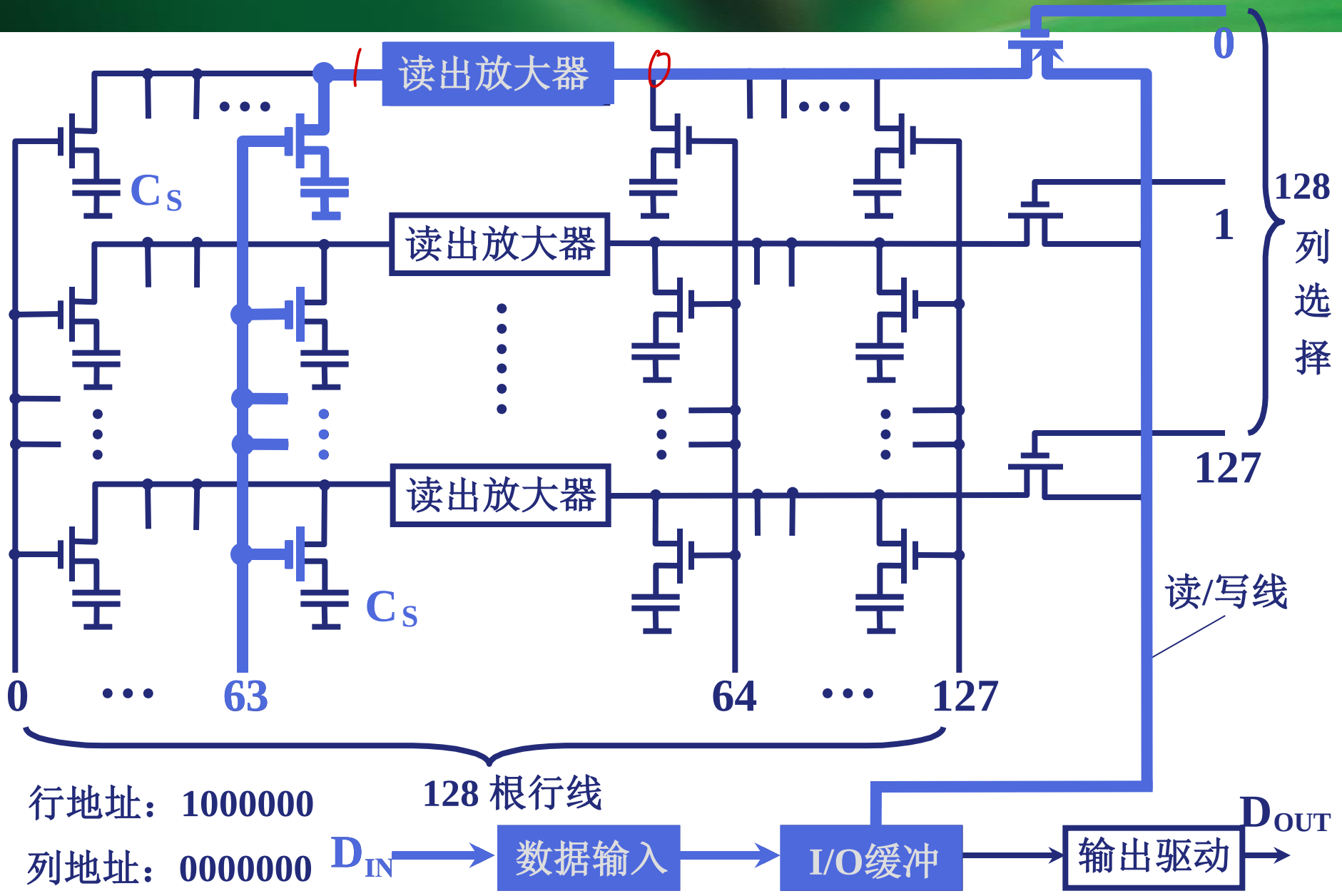
③ 单管动态 RAM 4116 (16K × 1位) 外特性



④ 4116 (16K × 1位) 芯片 读 原理



④ 4116 (16K × 1位) 芯片 写 原理



(3) 动态 RAM 时序

行、列地址分开传送 （教材P85）

读时序	写时序
行地址 $\overline{\text{RAS}}$ 有效	行地址 $\overline{\text{RAS}}$ 有效
写允许 $\overline{\text{WE}}$ 有效(高)	写允许 $\overline{\text{WE}}$ 有效(低)
列地址 $\overline{\text{CAS}}$ 有效	数据 D_{IN} 有效
数据 D_{OUT} 有效	列地址 $\overline{\text{CAS}}$ 有效

(4) 动态 RAM 刷新

电容内容易消失

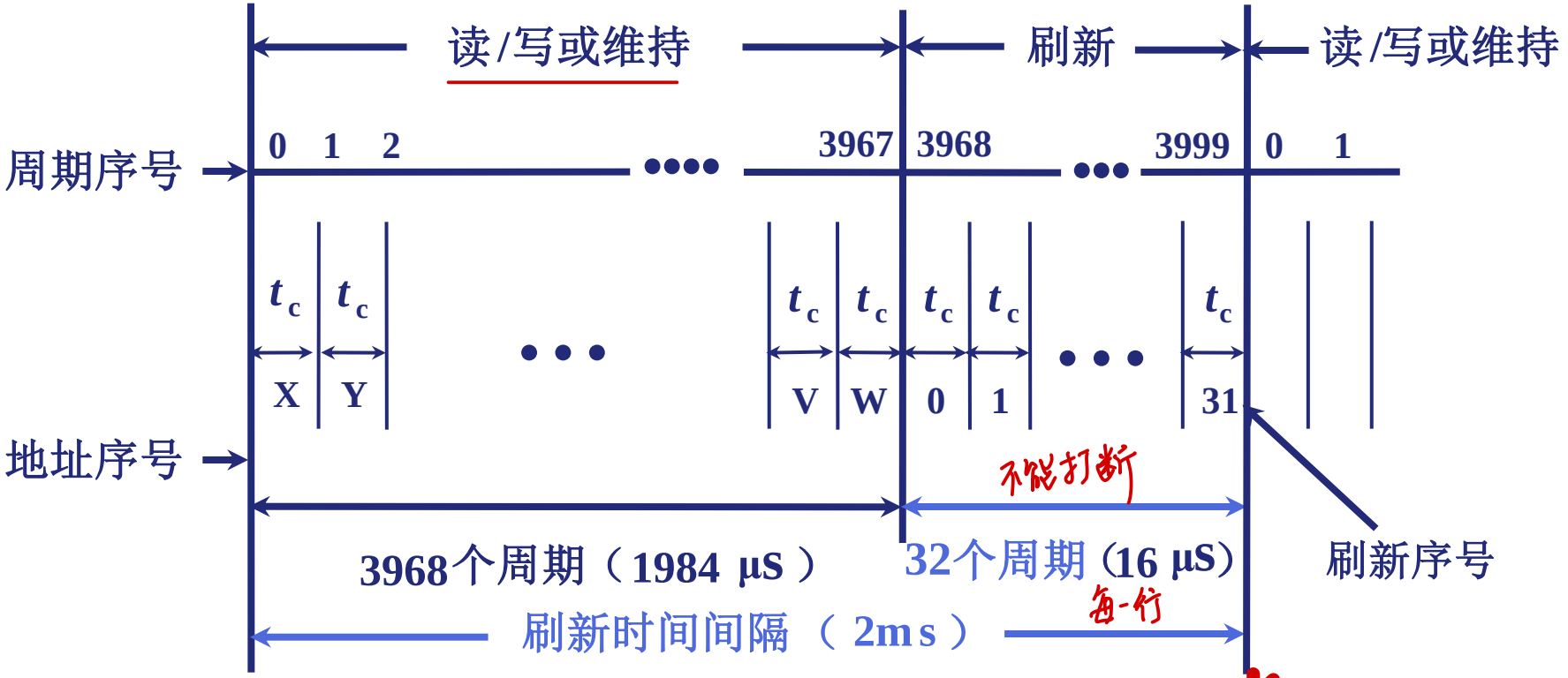
- 刷新是先将原存信息读出，再由刷新放大器形成原信息并重新写入的再生过程。
  - 刷新过程对CPU是透明的
  - 刷新是按行进行的，每行中的记忆单元同时被刷新，故刷新操作时仅仅需要行地址，不需要列地址。

不能按列，需重新设计芯片

  - 刷新类似读操作，但有所不同。刷新操作只给电容补充电荷，不需要信息输出。另外，刷新不用加片选，整个存储器中所有芯片同时被刷新。

(4) 动态 RAM 刷新

① 集中刷新（存取周期为0.5μs） 以 32× 32矩阵为例



“死区”为 0.5 μs ×32 = 16 μs

“死时间率”为 32/4000 ×100% = 0.8%

② 分散刷新（存取周期为1μs）

以 128 ×128 矩阵为例

$t_C = t_M + t_R$

读写 刷新

无“死区”

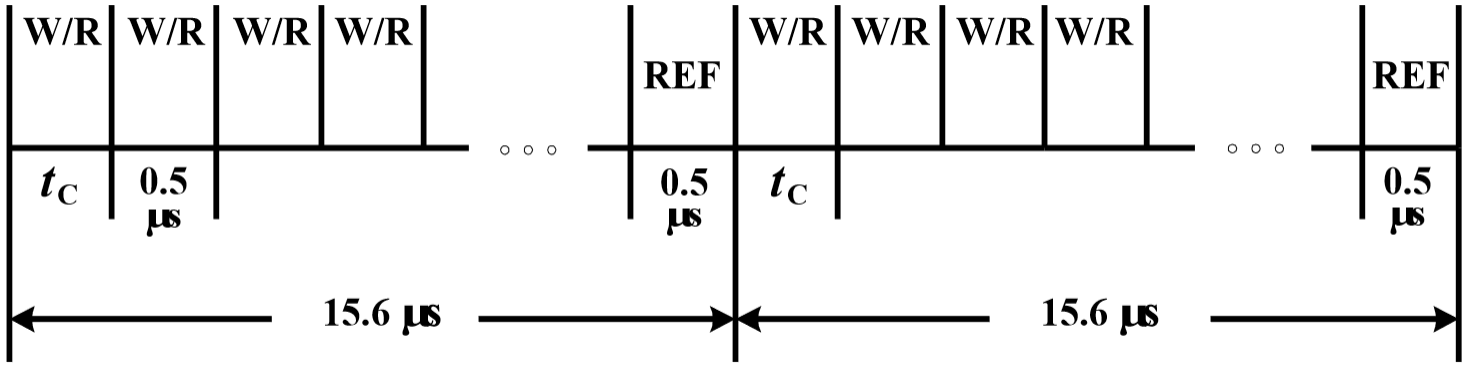
2ms中，每行被刷新15.625次

(存取周期为 0.5 μs + 0.5 μs)

③ 异步刷新

- 是分散刷新与集中刷新的结合

对于 128 ×128 的存储芯片（存取周期为 0.5μs）



若每隔 2 ms 集中刷新一次　　“死区” 为 64 μs

若每隔 15.6 μs 刷新一行　　“死区” 为 0.5 μs

而且每行每隔 2 ms 刷新一次

将刷新安排在指令译码阶段，不会出现 “死区”

(4) 动态 RAM 刷新小结

- 刷新周期：从上次对整个存储器刷新结束到下次对整个存储器全部刷新一遍为止的时间间隔，为电容数据有效保存期的上限（64ms）。
- 有三种刷新方式：集中式、分散式、异步刷新。

① 集中刷新：

前一段时间正常读/写，后一段时间停止读/写，集中逐行刷新。

特点：集中刷新时间长，不能正常读/写（死区），很少使用。

② 分散刷新：

一个存储周期分为两段: 前一段用于正常读/写操作，后一段用于刷新操作。

特点：不存在死区，但每个存储周期加长。很少使用。

③ 异步刷新：

结合上述两种方式。以4096行为例，在64ms时间内必须轮流对每一行刷新一次，即每隔64ms/4096=15.625μs刷新一行。

特点：结合前两种，效率高，用得较多。

3. 动态 RAM 和静态 RAM 的比较

主存

缓存

存储原理	DRAM 电容	SRAM 触发器
集成度	高	低
芯片引脚	少	多
功耗	小	大
价格	低	高
速度	慢	快
刷新	有	无

四、只读存储器（ROM）

1. 掩膜 ROM (MROM)

标题

- 以元件的“有/无”表示“1”、“0”。
- 内容由半导体制造厂按用户提出的要求在芯片的生产过程中直接写入的，写入后内容无法改变。

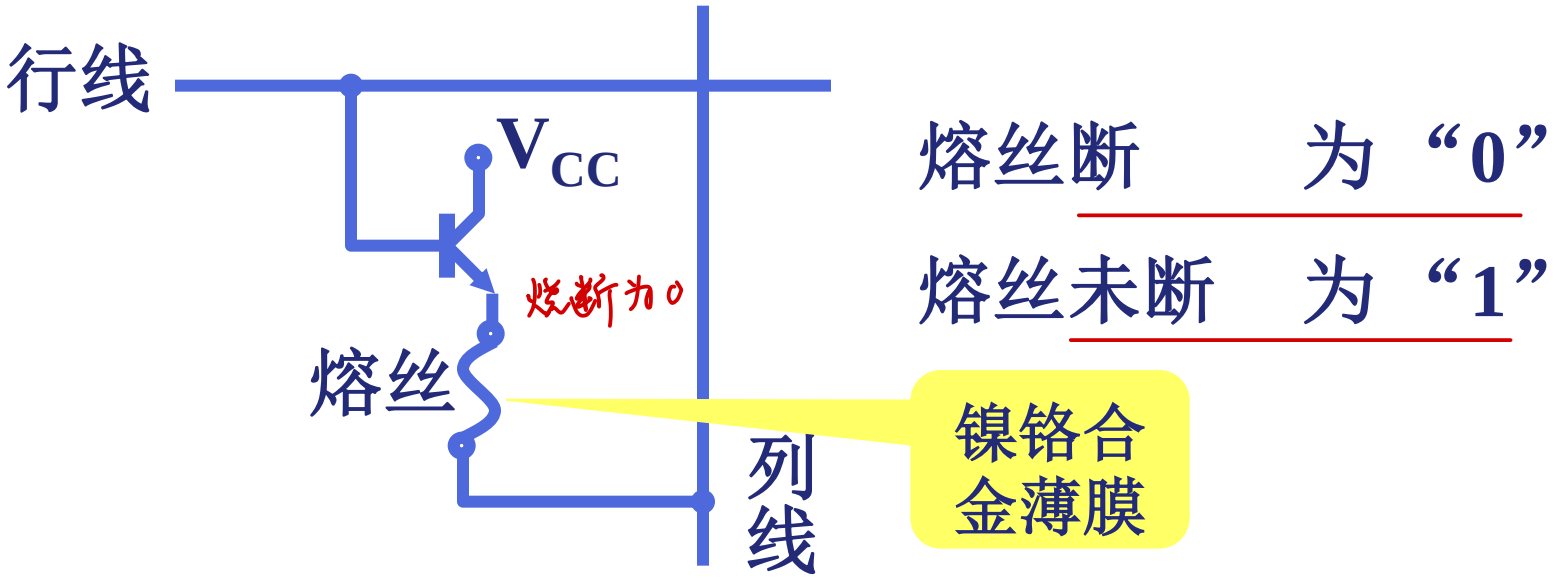
行列选择线交叉处有 MOS 管为“1”

行列选择线交叉处无 MOS 管为“0”

- 优点：可靠性高，形成批量之后价格便宜；
- 缺点：用户对制造厂的依赖性过大，灵活性差。

2. PROM (一次性编程)

- 允许用户利用专门的设备（编程器）写入自己的程序，但一旦写入后，其内容将无法改变。



- 产品出厂时，所有记忆单元均制成“0”（或制成“1”），用户根据需要可自行将其中某些记忆单元改为“1”（或改为“0”）。
- 双极型PROM有两种结构，一种是熔丝烧断型，另一种是PN结击穿型，

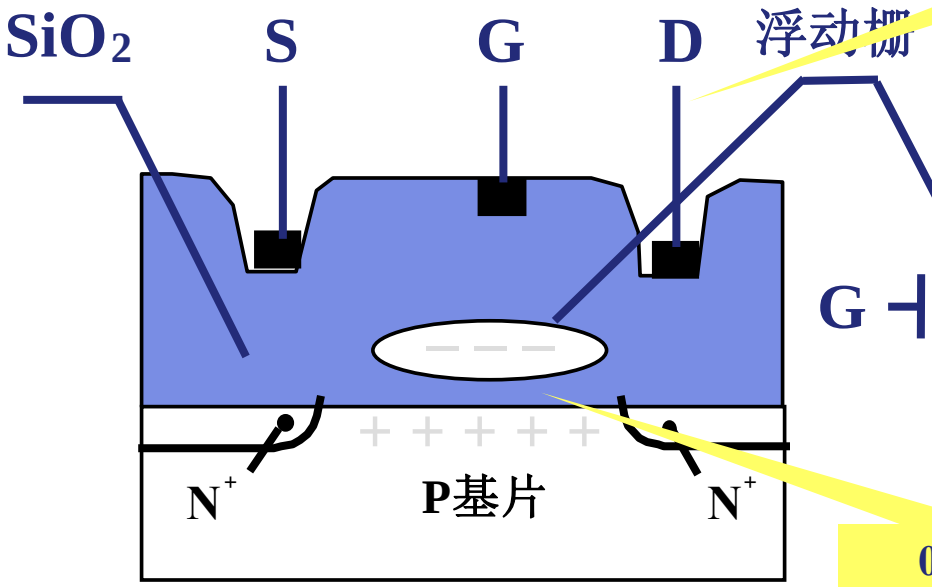
3. EPROM (多次性编程)

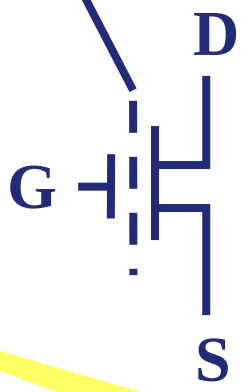
浮动栅雪崩注入型MOS管

(1) N型沟道浮动栅 MOS 电路

25V（写入电压）、50ms宽正脉冲

埋在氧化物绝缘层中，不与外部导通。写入信息前，浮栅上没有电荷，D、S间不导通，MOS呈“1”态





S 源

D 漏

紫外线全部擦洗

0.05~0.1μm

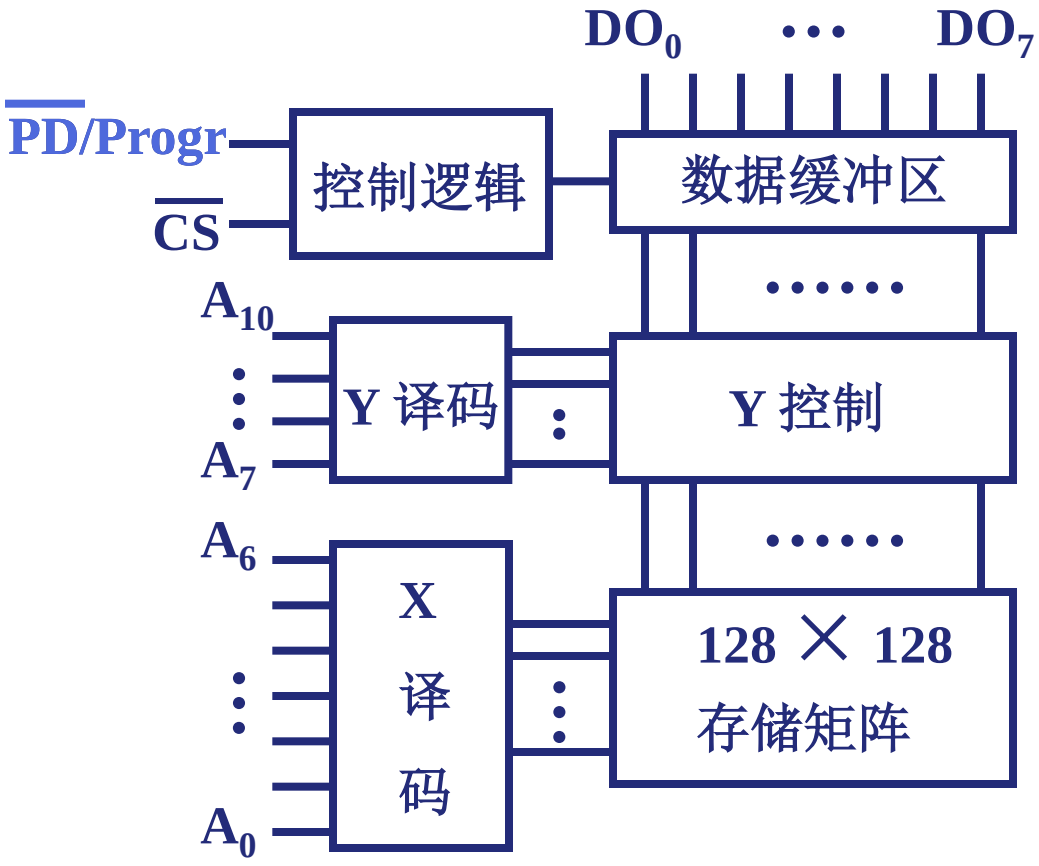
D 端加正电压　　形成浮动栅　　S 与 D 不导通为 “0”

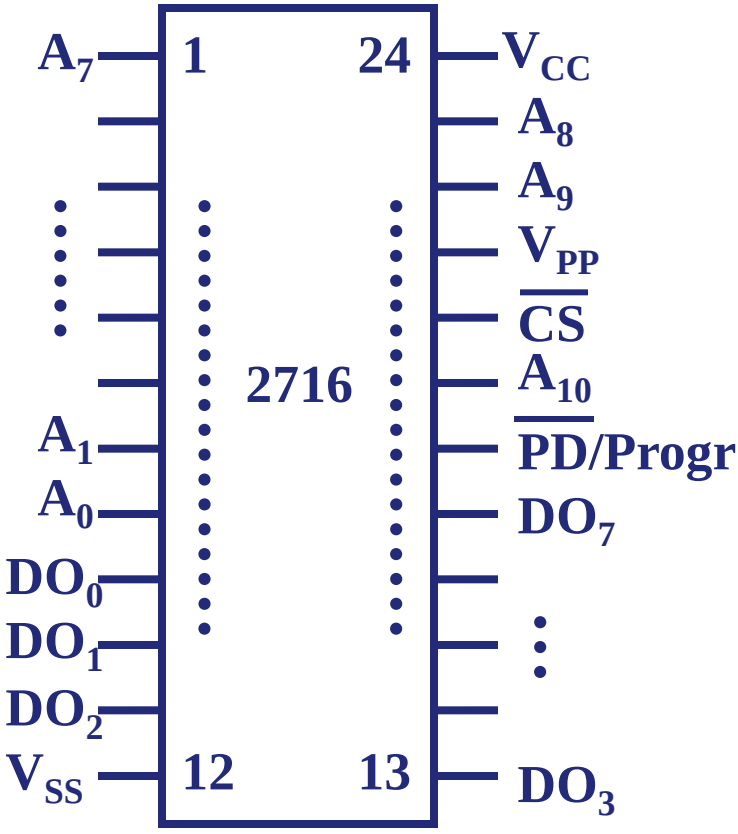
D 端不加正电压　　不形成浮动栅　　S 与 D 导通为 “1”

3. EPROM (多次性编程)

- 可由用户利用编程器写入信息，可以多次改写。
- EPROM出厂时，存储内容为全“1”，用户可以根据需要将其中某些记忆单元改为“0”。当需要更新存储内容时可以将原存储内容擦除（恢复全“1”），以便再写入新的内容。
- EPROM可分为：紫外线擦除和电擦除
  - UVEPROM：用紫外线灯制作的擦除器照射存储器芯片上的透明窗口，使芯片中原存内容被擦除。
    - 由于是用紫外线灯进行擦除，所以只能对整个芯片擦除，而不能对芯片中个别需要改写的存储单元单独擦除。
    - 另外，为了防止存储的信息受日光中紫外线成分的作用而缓慢丢失，在UVEPROM芯片写入完成后，必须用不透明的黑纸将芯片上的透明窗口封住。
  - EEPROM是采用电气方法来进行擦除的。
    - 在联机条件下既可以用字擦除方式擦除，也可以用数据块擦除方式擦除。以字擦除方式操作时，能够只擦除被选中的那个存储单元的内容；在数据块擦除操作时，可擦除数据块所有单元的内容。

(2) 2716 EPROM 的逻辑图和引脚





$\overline{PD}$/Progr 功率下降 / 编程输入端　读出时 为 低电平

EPROM不能取代RAM

- 原因

①EPROM的编程次数（寿命）是有限的：

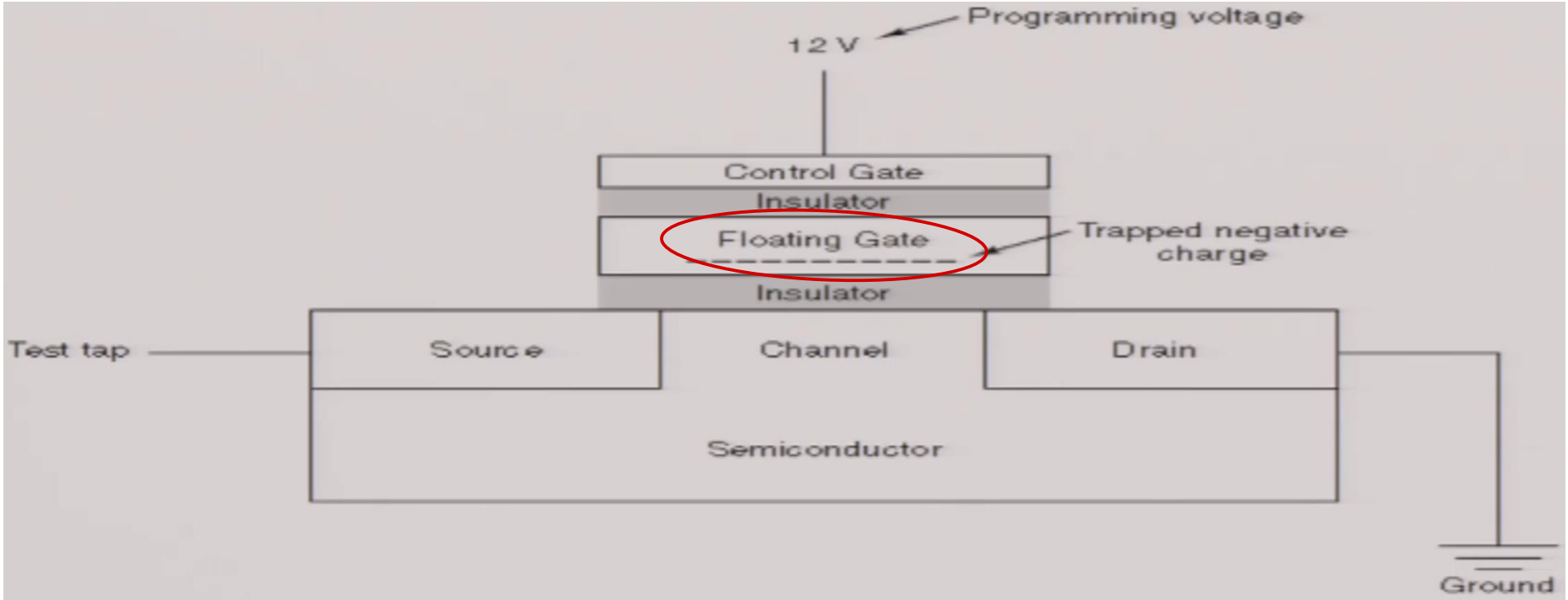
EEPROM因为氧化层被磨损，重复改写次数一般10万次

②写入时间过长，即使对于EEPROM，擦除一个字节大约需要10ms，写入一个字节大约需要10us，比SRAM或DRAM的时间长100－1000倍。

4. Flash Memory (快擦型存储器)

- 闪速存储器是20世纪80年代中期出现的一种快擦写型存储器，它的主要特点是既可在不加电的情况下长期保存信息，又能在线进行快速擦除与重写，兼备了EEPROM和RAM的优点。
- 目前，大多数微型计算机的主板采用闪速存储器来存储BIOS程序，由于BIOS的数据和程序非常重要，不允许修改，故早期主板BIOS芯片多采用PROM或EPROM。

Flash Memory 原理



- 写0，控制极加写电压
  - 往浮动栅极上注入电子
- 擦除（写1）
  - 漏极接擦除电压
- 读：控制极加读电压
  - 晶体管不导通，读出0
  - 晶体管导通，读出1

Flash Memory 特点

- 非易失性（10年~100年）
- 电可擦除
- 写入次数有限（老化）
- NOR型
  - 代替EPROM、EEPROM
  - 按地址进行访问
  - 用于存放程序
- NAND型
  - 作为硬盘的替代品
  - 按扇区访问、按页擦除和写入

HISTORICAL DEVELOPMENT OF FLOATING-GATE NVSM

YEAR	DEVICE	INVENTOR(S)/AUTHOR(S)	ORGANIZATION
1967	Floating-Gate Concept	D. Kahng, S.M. Sze	Bell Labs
1971	EPROM-FAMOS	D. Frohman-Bentchkowsky	Intel
1976	EEPROM-SAMOS	H. Iizuka et al.	Toshiba
1984	Flash Memory	F. Masuoka et al.	Toshiba
1985	FLOTOX Transistor	S.K. Lai et al	Intel
1985	NOR Flash	F. Masuoka et al.	Toshiba
1987	NAND Flash	F. Masuoka et al.	Toshiba
1994	Room-Temperature Single-Electron Transistor	K. Yano et al.	Hitachi
1995	Multilevel Cell	M. Bauer et al.	Intel
2010	Multi-layer integration	S.J Whang et al.	Hynix
2013	128 Gb 3b/cell NAND Flash	G.Naso et al.	Micron

固态硬盘*

- 固态硬盘没有机械结构，没有移动的部分
- 安静，低功耗，高性能，不怕摔，低发热
- 价格比硬盘高（将来会下降），有限的擦除次数

可靠

固态硬盘

坏后没数据

机械硬盘

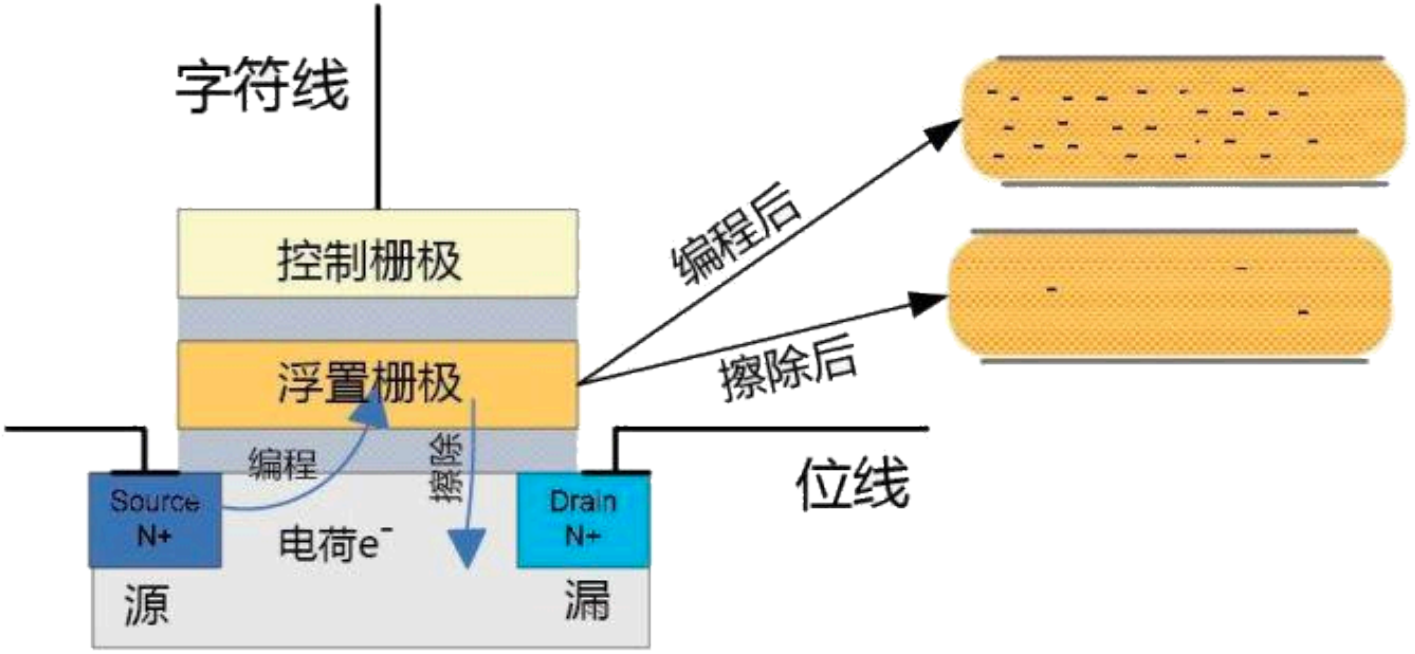
可找回

固态硬盘存储*

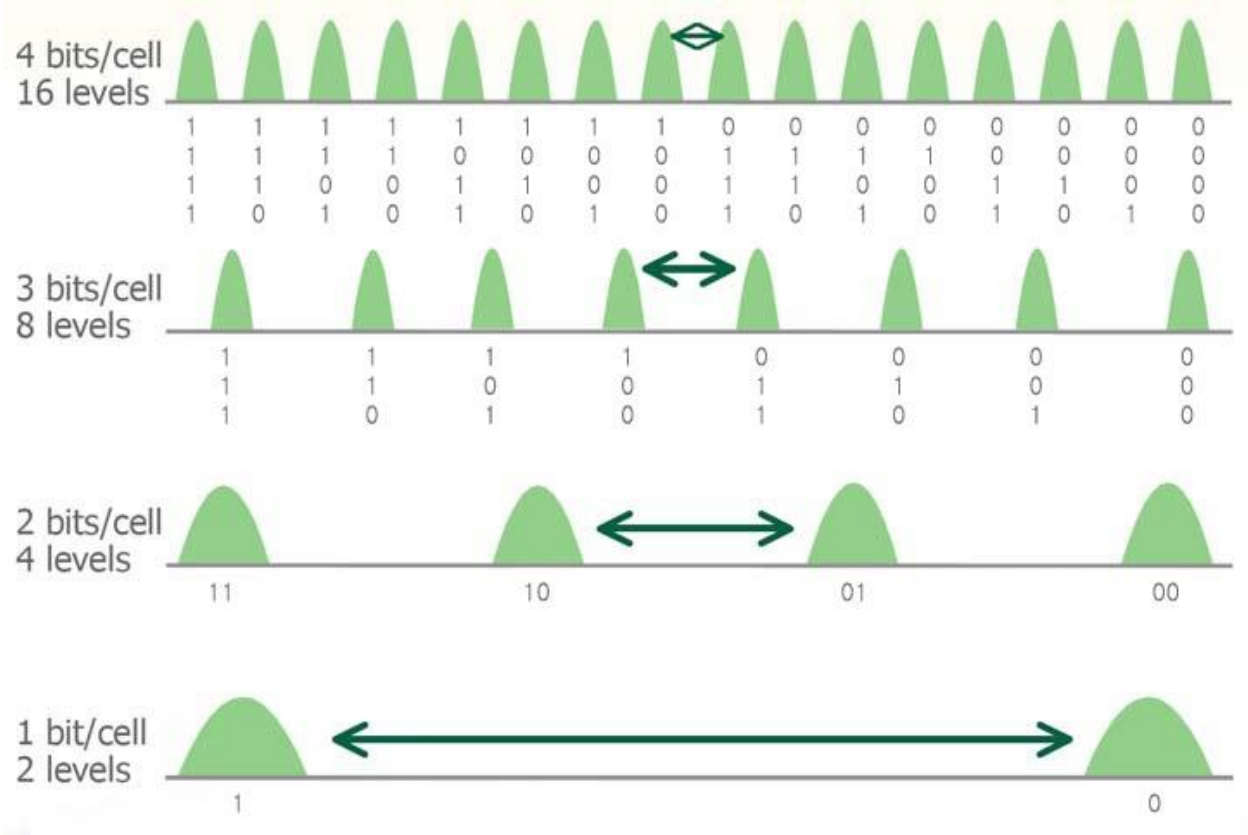
- 固态硬盘（Solid State Drives），用固态电子存储芯片阵列而制成的硬盘，由控制单元和存储单元（FLASH芯片、DRAM芯片）组成。

固态硬盘的存储单元*

- 闪存的内部存储结构是金属-氧化层-半导体-场效应管(MOSFET)：源极，漏极和栅极，增加了浮置栅极
- 对于闪存的写入，即控制栅极去充电，对栅极加压，使得浮置栅极存储的电荷越多，超过阈值，就表示0
- 对于闪存的擦除，即对浮置栅极进行放电，低于阈值，就表示1



SLC，MLC，TLC，QLC*



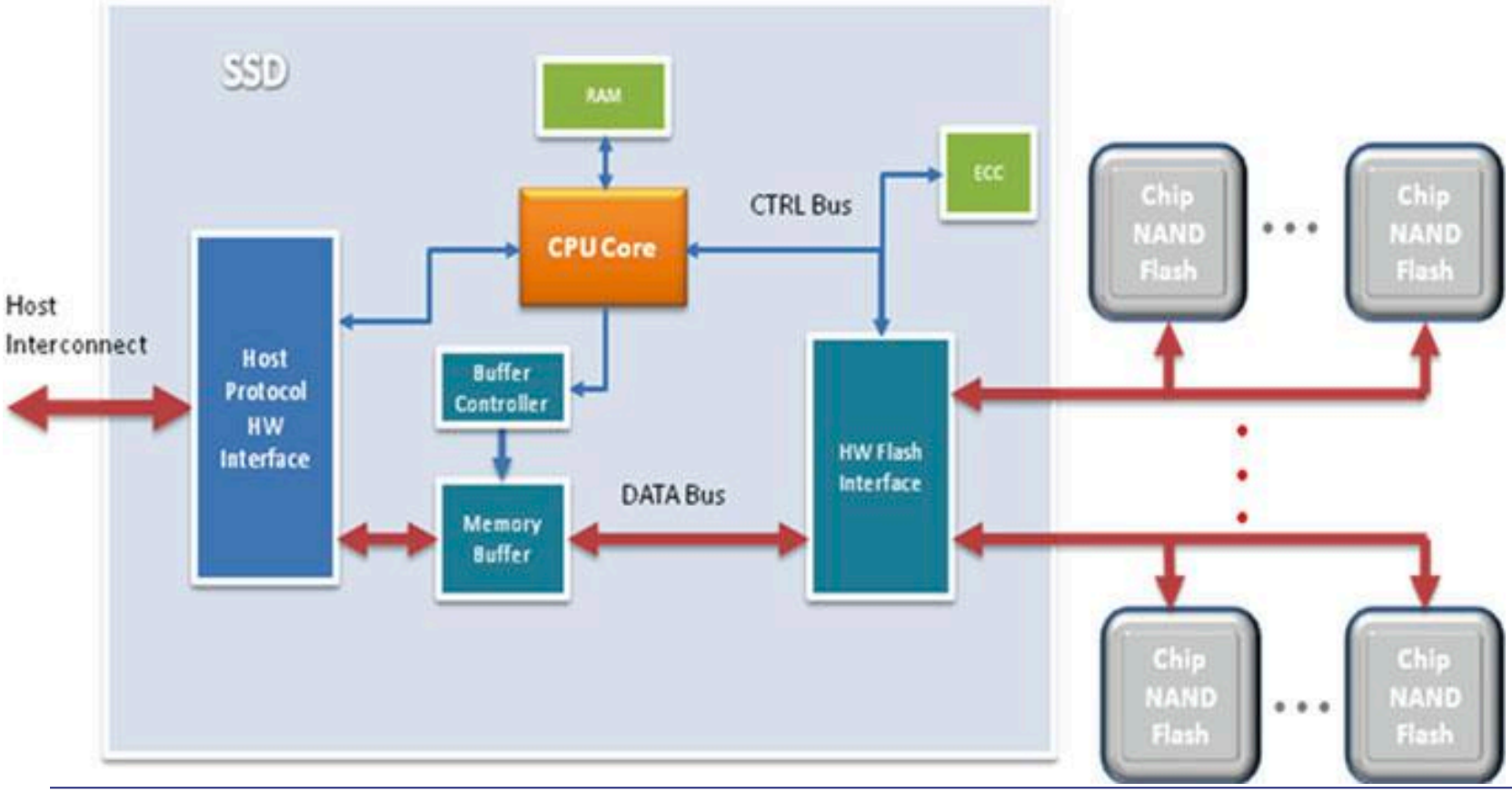
- 按照每个存储单元能够存储的位数分为SLC，MLC，TLC，QLC
- 多比特单元采用格雷码编码

固态硬盘存储单元的擦除次数*

- 有限次擦除
  - 随着擦除次数的增加，存储单元不能可靠地保持状态（存储数据）
    - 耐久性 **Endurance**
    - 保持力 **Retention**
- **SLC：100,000次**
- **MLC：10,000次**
- **TLC：1,000次**

从存储器件到固态硬盘*

- SSD主要由SSD控制器，FLASH存储阵列，板上DRAM（可选），以及跟HOST接口（诸如SATA，SAS, PCIe等）组成。
- 三个重要组成部分：主存芯片，闪存芯片，固件算法



典型的SSD性能数据*

- 依据不同SSD的型号具有不同的性能数据
- 读数据的性能：
  - 20-100微秒的延迟
  - 100-500MB/s的带宽
- 擦除的性能：
  - 2ms，即毫秒级
- 写数据的性能：
  - 200微秒的延迟
  - 100-200MB/s的带宽

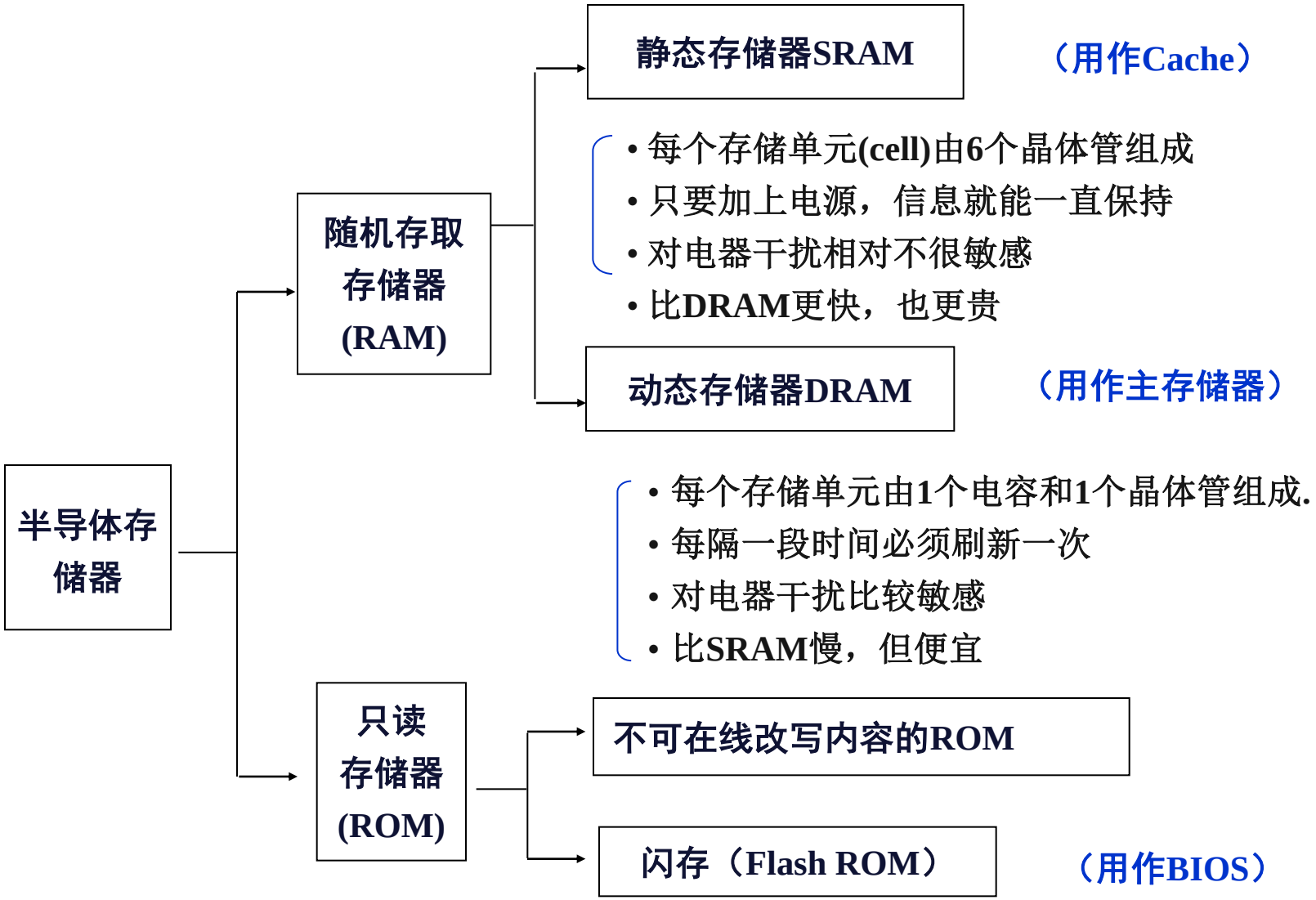
2019年的性能数据*

- DELTA MAX SSD硬盘为SATA 6Gbps接口
- 有250GB、500B、1TB及2TB容量
- 其中250GB的读取、写入速度是560、500MB/s之外
- 其他容量的读取、写入都是560MB/s、510MB/s
- 4K随机读写最高90K、80K IOPS

SSD总结*

- 机械式存储设备转化为电子式存储设备
- 不同的操作粒度
  - 以页的方式读写
  - 以块的方式擦除
- 擦除次数有限：需要磨损均衡
- **FTL：**
  - 逻辑地址到物理地址的转换
  - 磨损均衡
  - 垃圾回收

内存芯片的多种类型


半导体存储器
随机存取存储器(RAM)
静态存储器SRAM
（用作Cache）
• 每个存储单元(cell)由6个晶体管组成
• 只要加上电源，信息就能一直保持
• 对电器干扰相对不很敏感
• 比DRAM更快，也更贵
动态存储器DRAM
（用作主存储器）
• 每个存储单元由1个电容和1个晶体管组成.
• 每隔一段时间必须刷新一次
• 对电器干扰比较敏感
• 比SRAM慢，但便宜
只读存储器(ROM)
不可在线改写内容的ROM
闪存（Flash ROM）
（用作BIOS）

五、存储器与 CPU 的连接

1. 存储器容量的扩展：由于单个芯片的存储器容量有限，为了组成较大容量的存储器，需要多个芯片进行组合。

- 位扩展（增加存储字长）：由$mK\times n_1$的存储器芯片组成$mK\times n_2$的存储器，需（n_2/n_1）片$mK\times n_1$的存储器芯片。
- 字扩展（增加存储字数量）：由$m_1K\times n$的存储器芯片$m_2K\times n$的存储器，需（m_2/m_1）片$m_1K\times n$的存储器芯片。
- 字位同时扩展：由$m_1K\times n_1$的存储器芯片组成$m_2K\times n_2$的存储器，需（m_2/m_1）×（n_2/n_1）片$m_1K\times n_1$的存储器芯片。

五、存储器与 CPU 的连接

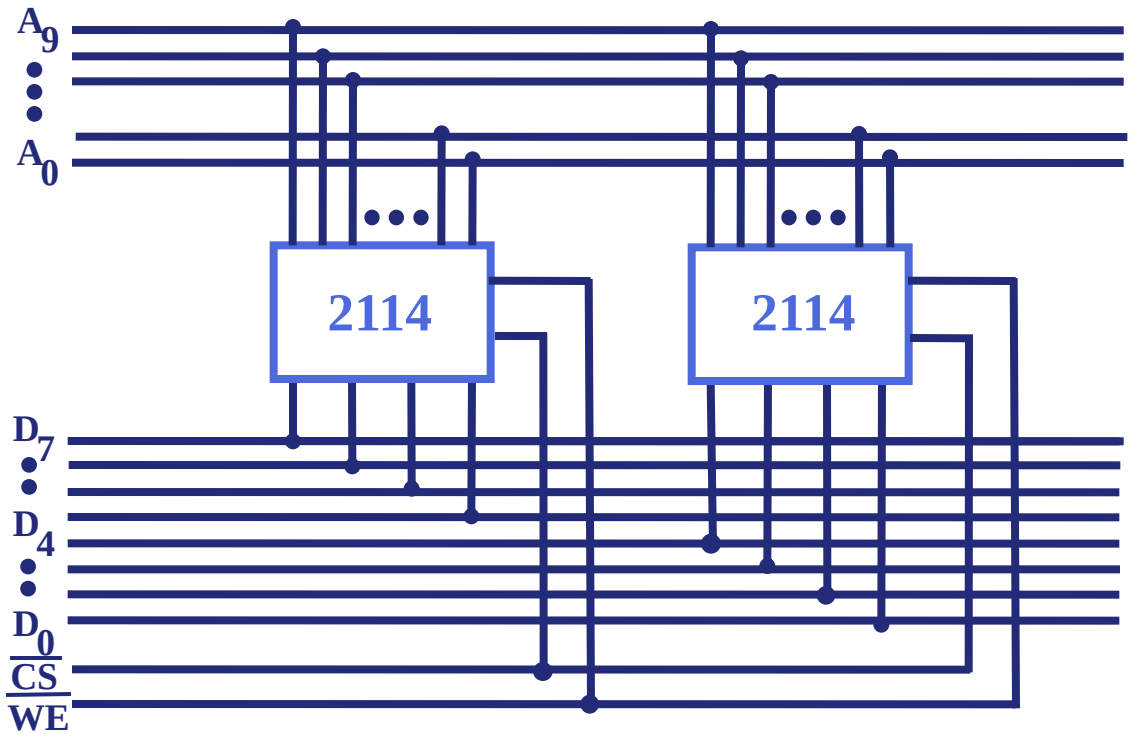
(1) 位扩展 (增加存储字长)

10根地址线

2^{10}

用 2片 1K×4位 存储芯片组成 1K×8位 的存储器

8根数据线



(2) 字扩展（增加存储字的数量）

$2^{10} \times 2$

11根地址线

用 2片 1K×8位 存储芯片组成 2K×8位 的存储器

8根数据线

$A_{10}\ A_9 \cdots\cdots A_0$

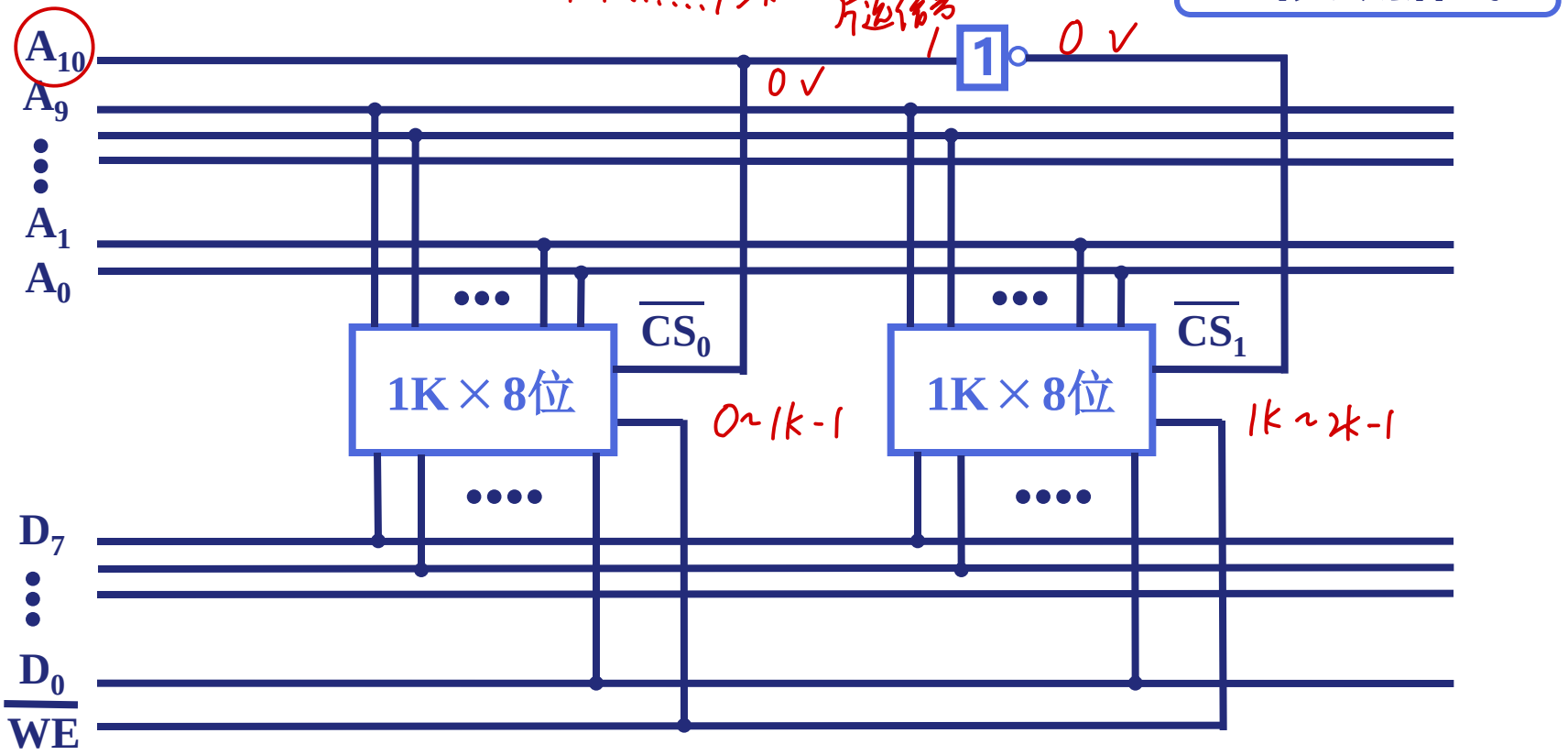
0 0 ……0 >1K

0 1 ……1 >1K

1 0 ……0 >1K

1 1 ……1 >1K

片选信号

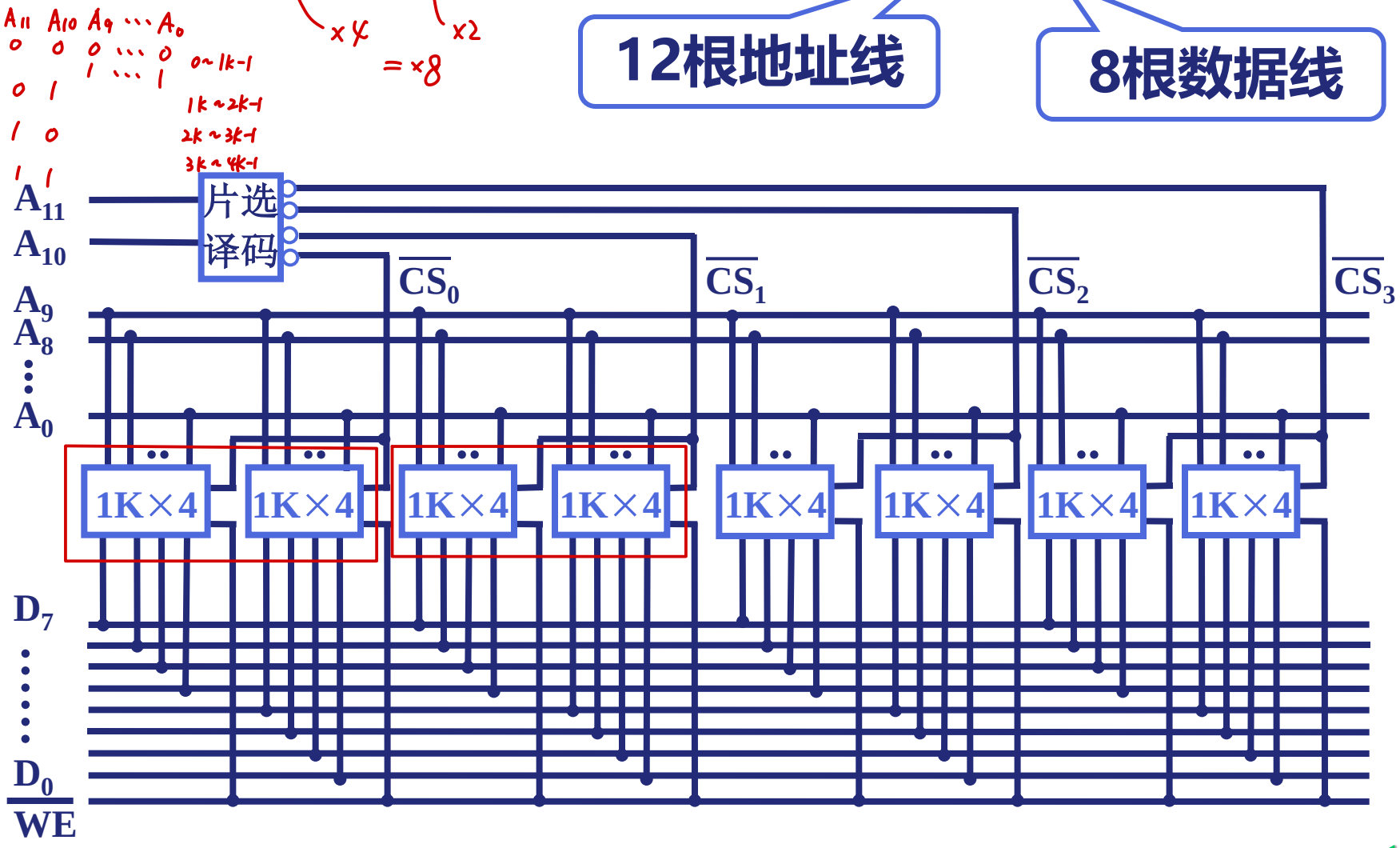


(3) 字、位扩展 (通常)

用 8片 1K×4位 存储芯片组成 4K×8位 的存储器

12根地址线

8根数据线



2. 存储器与 CPU 的连接

(1)地址线的连接:内部地址线、芯片选择线

CPU地址线往往比存储器地址线多，所以将CPU地址线低位与存储芯片地址相连，CPU地址线高位或做存储芯片扩充，或做他用。

(2) 数据线的连接:数据线对应连接。

(3) 读/写线的连接：数据线对应连接。

(4) 合理选用芯片 片选信号

(5) 其他：时序、负载

例4.1 （教材P94）

设CPU有16根地址线、8根数据线，并用$\overline{\text{MREQ}}$作为访存控制信号（低电平有效），用WR作为读/写控制信号（高电平为读，低电平为写）。现有下列存储芯片：

1 K×4位RAM、 4 K×8位RAM 、 8 K×8位RAM、

2 K×8位ROM、 4 K×8位ROM、 8 K×8位ROM

及74138译码器和各种门电路，画出CPU与存储器连接图，要求：

①主存地址空间分配

6000H~67FFH为系统程序区

6800H~6BFFH为用户程序区

②合理选用上述存储芯片，说明各选几片

③ 详细画出存储芯片的片选逻辑图

例4.1 解:

(1) 写出对应的二进制地址码

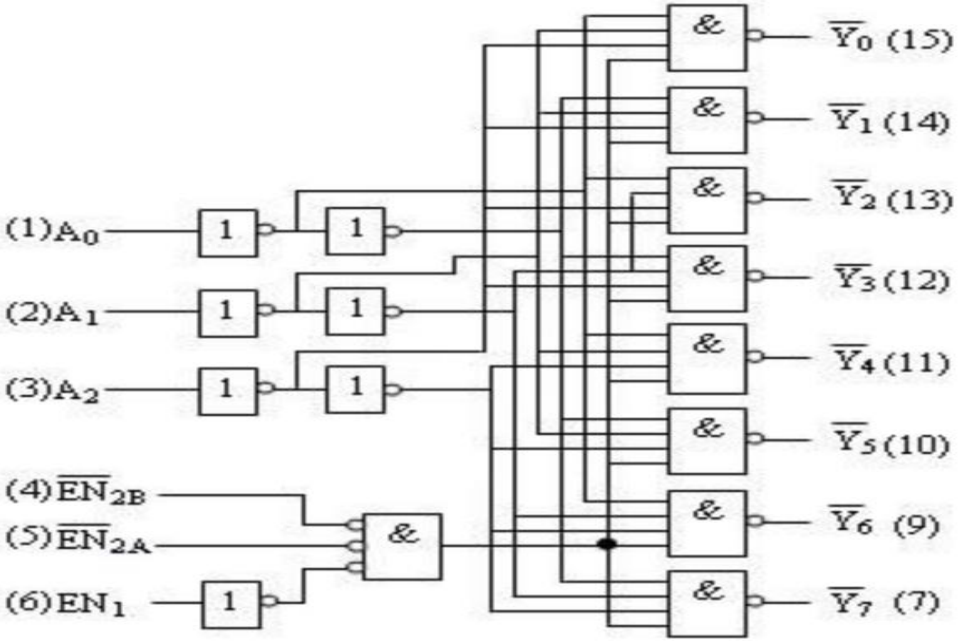
A_{15}	A_{14}	A_{13}	A_{12}	A_{11}	A_{10}	…	A_7	…	A_4	A_3	…	A_0	
0	1	1	0	0	0	0	0	0	0	0	0	0	ROM 2K×8位
⋮													
0	1	1	0	0	1	1	1	1	1	1	1	1	
0	1	1	0	1	0	0	0	0	0	0	0	0	RAM 1K×8位
⋮													
0	1	1	0	1	0	1	1	1	1	1	1	1	

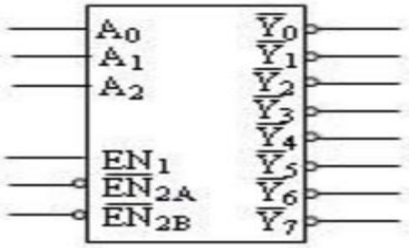
1片2K×8位

2片1K×4位

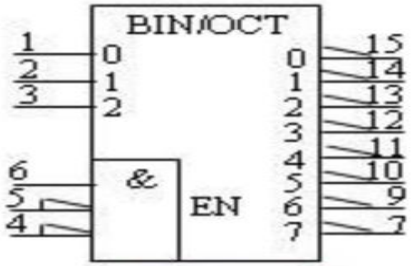
(2) 确定芯片的数量及类型

74138译码器真值表及引脚图功能





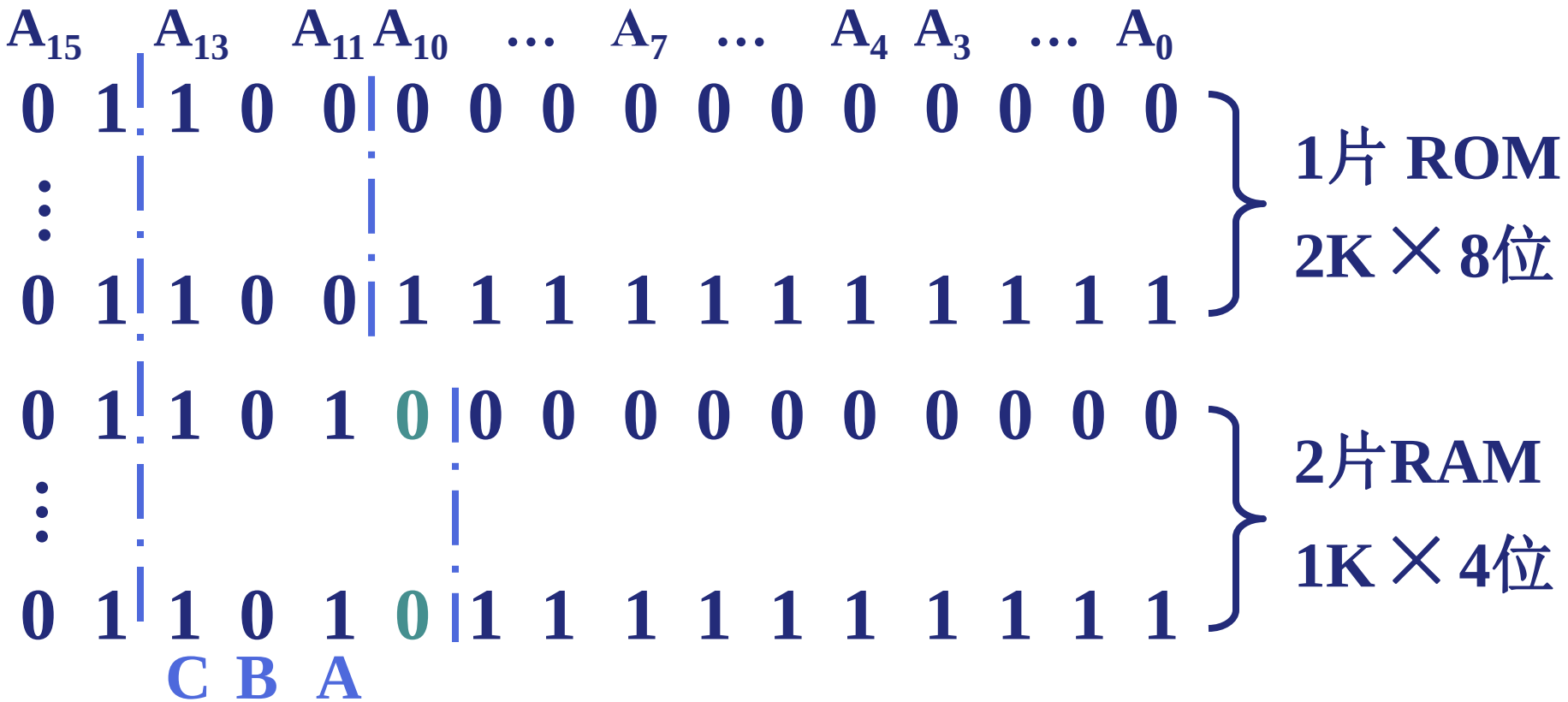
(b)



EN_1	$\overline{EN_{2A}}$	$\overline{EN_{2B}}$	A_2	A_1	A_0	$\overline{Y_7}$	$\overline{Y_6}$	$\overline{Y_5}$	$\overline{Y_4}$	$\overline{Y_3}$	$\overline{Y_2}$	$\overline{Y_1}$	$\overline{Y_0}$
0	X	X	X	X	X	1	1	1	1	1	1	1	1
X	1	X	X	X	X	1	1	1	1	1	1	1	1
X	X	1	X	X	X	1	1	1	1	1	1	1	1
1	0	0	0	0	0	1	1	1	1	1	1	1	0
			0	0	1	1	1	1	1	1	1	0	1
			0	1	0	1	1	1	1	1	0	1	1
			0	1	1	1	1	1	1	0	1	1	1
			1	0	0	1	1	1	0	1	1	1	1
			1	0	1	1	1	0	1	1	1	1	1
			1	1	0	1	0	1	1	1	1	1	1
			1	1	1	0	1	1	1	1	1	1	1

DLTDL.com

(3) 分配地址线

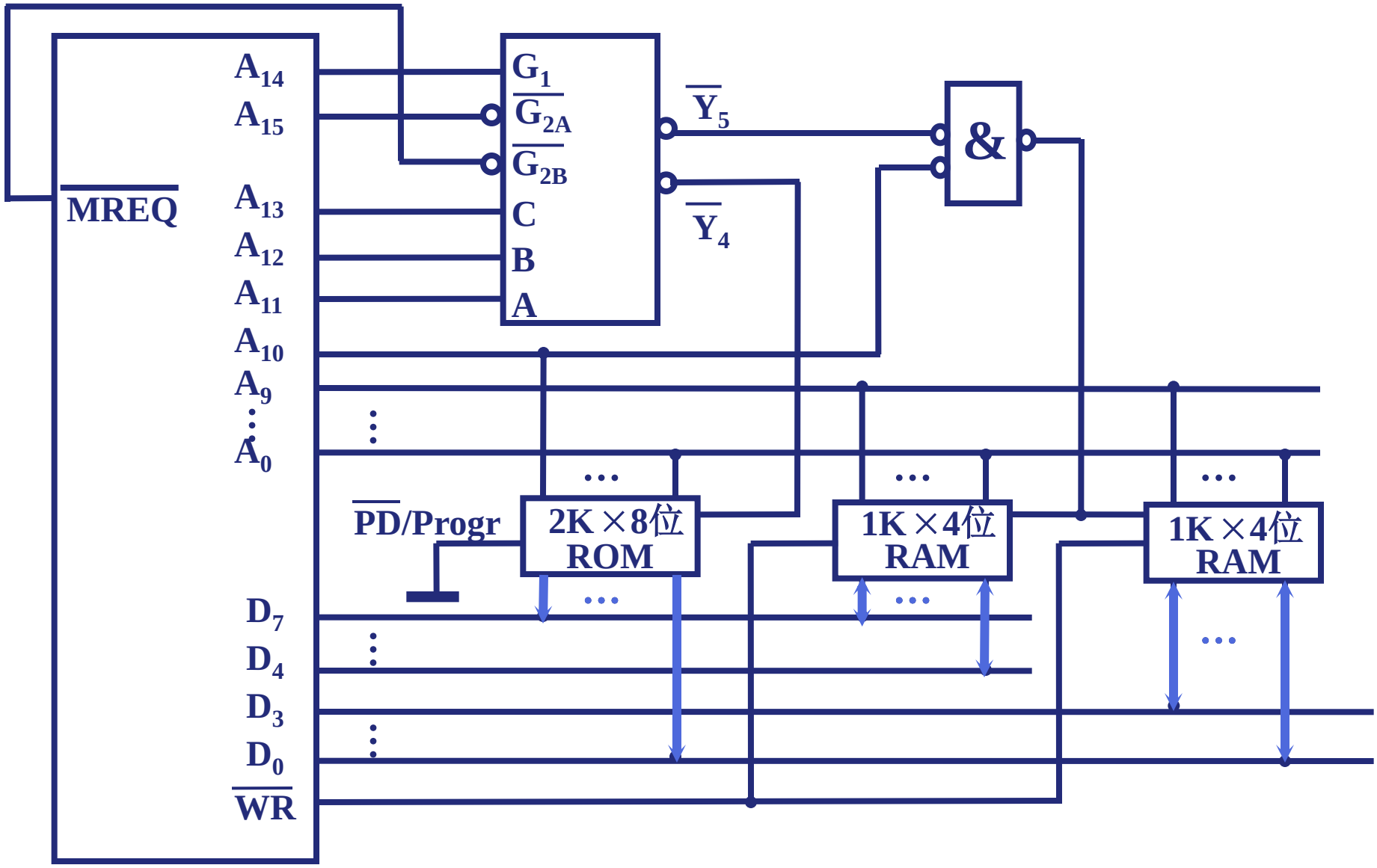


A_{10}~A_0 接 2K×8位 ROM 的地址线

A_9 ~ A_0 接 1K×4位 RAM 的地址线

(4) 确定片选信号

例 4.1 CPU 与存储器的连接图



例4.2 （教材P95）

例4.2 假设同前，要求最小 8K为系统程序区，相邻16K为用户程序区。最大 4K为系统程序区工作区。

(1) 写出对应的二进制地址码

(2) 确定芯片的数量及类型

1片 8K × 8位 ROM 2片 8K × 8位 RAM

1片 4K × 8位 RAM

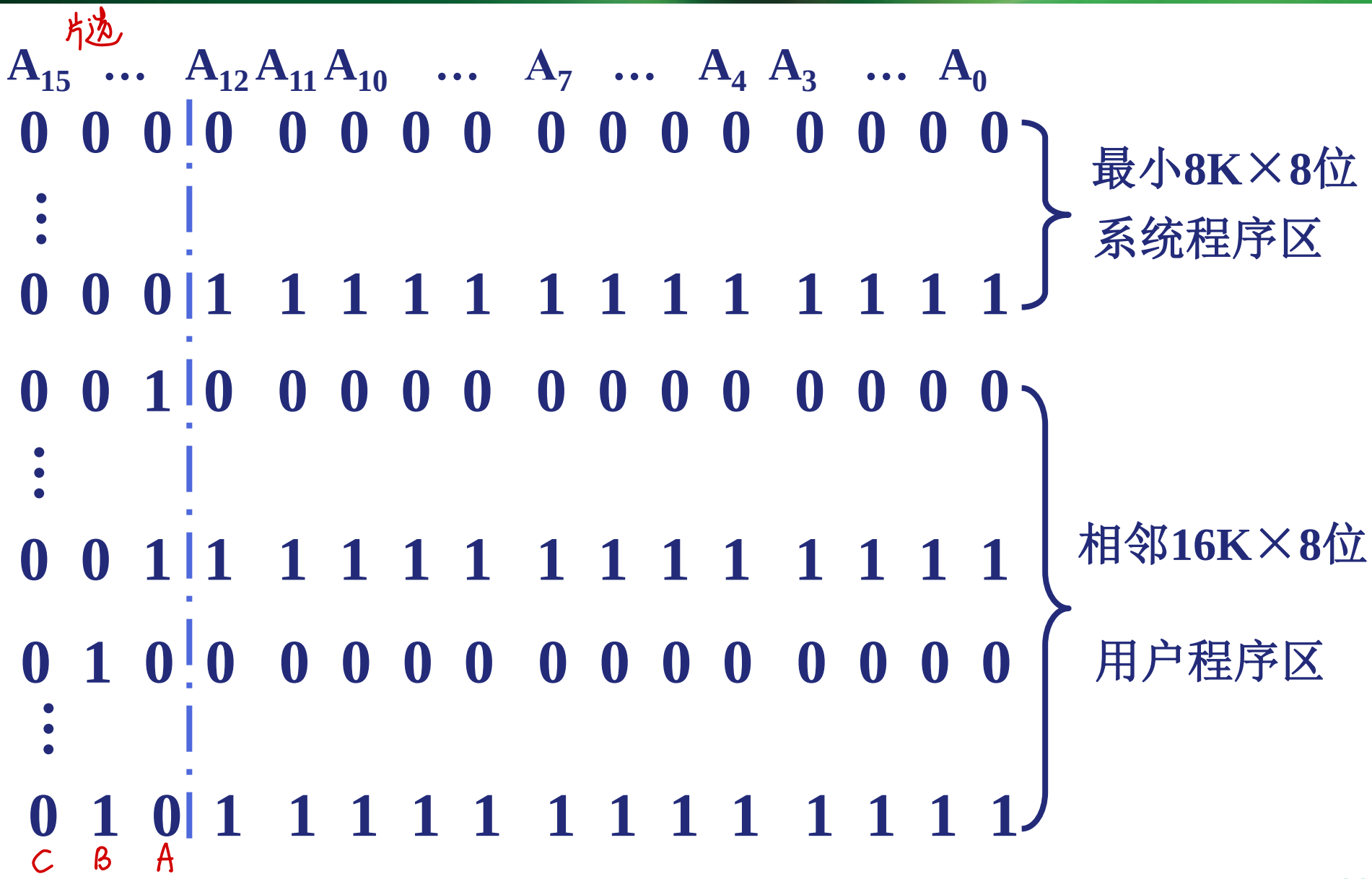
(3) 分配地址线

A_{12}~ A_0 接 ROM 和 8K × 8位 RAM 的地址线

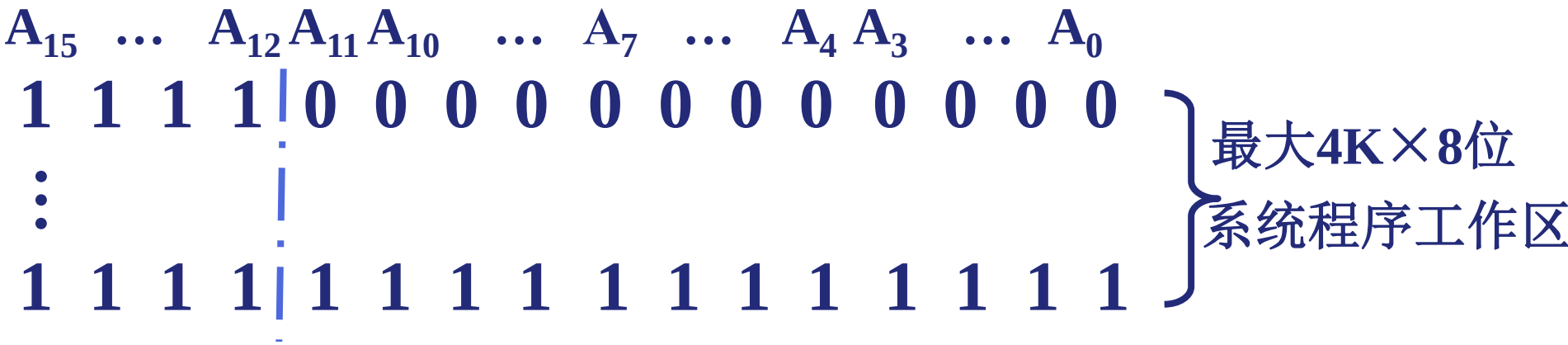
A_{11}~ A_0 接 4K × 8位 RAM 的地址线

(4) 确定片选信号

(1) 写出对应的二进制地址码



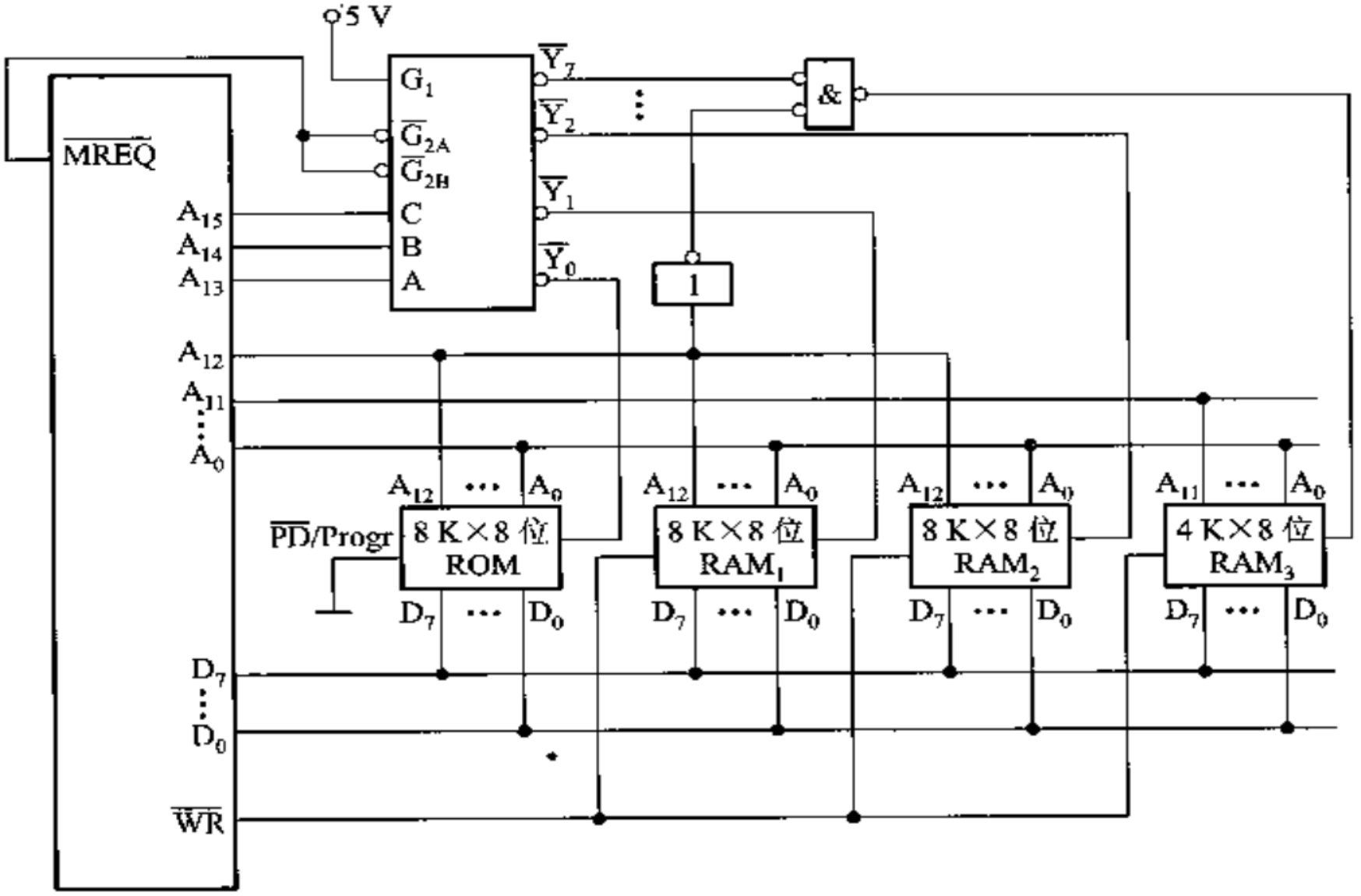
(1) 写出对应的二进制地址码



(2) 确定芯片的数量及类型

- 最小8KB系统程序区：1片8K×8位ROM;
- 与其相邻的16KB用户程序区：选择2片8K×8位RAM;
- 最大4KB系统程序工作区：选择1片4K×8位RAM。

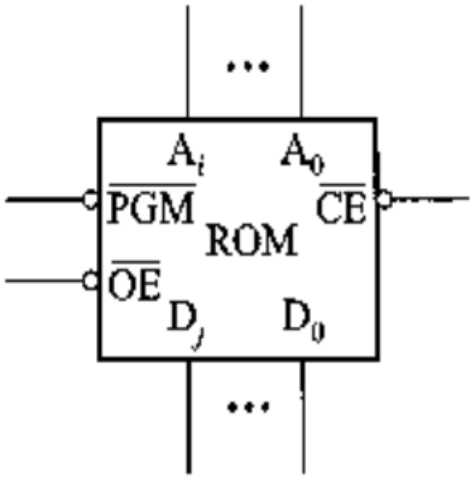
例 4.2 CPU 与存储器的连接图



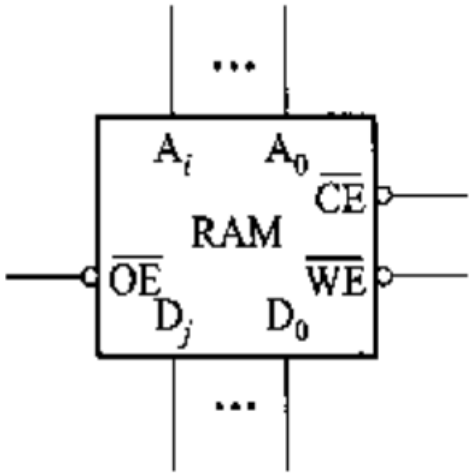
例4.3 （教材P97）

例 4.3　设 CPU 有 20 根地址线，16 根数据线。并用 $\overline{IO/M}$ 作访存控制信号。$\overline{RD}$ 为读命令，$\overline{WR}$ 为写命令。CPU可采用BHE和A0控制字节和字两种形式访存。要求采用以下芯片，门电路自定。

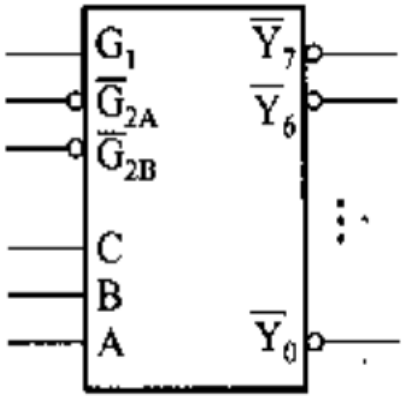
字节编址　字地址



64 K×8 位
32 K×8 位
32 K×16 位



64 K×8 位
32 K×8 位
32 K×16 位



74138

例4.3 （教材P97）

（1）CPU按字节访问和按字访问的地址范围各多大？

（2）CPU按字节访问时需分奇偶体，且最大64KB为系统程序区，与其相邻的64KB为用户程序区，写出每片存储芯片对应的二进制地址码。

（3）画出CPU与存储芯片连接图。

约定方式

BHE	A0	访问形式
0	0	字
0	1	奇字节
1	0	偶字节
1	1	不访问

*例 4.3

奇数字节和偶数字节

解：(1) CPU 按字节访问的地址范围为 1 M，CPU 按字访问的地址范围是 512 K。

(2) 由于 CPU 按字节访存时需区分奇偶体，并且还可以按字访问，因此如果选择 64 K × 8 位的芯片，按字节访问时体现不出奇偶分体；如果选择 32 K × 16 位的芯片，虽然能按字访问，但不能满足以字节为最小单位。故一律选择 32 K × 8 位的存储芯片，其中系统程序区 64 KB 选择 2 片32 K × 8 位 ROM，用户程序区 64 KB 选择 2 片 32 K × 8 位 RAM。它们对应的二进制地址范围如下：

C 作为 CBA 中一个输入

A

字 $A_{19}A_{18}A_{17}A_0$ = 100 (CBA)

A_{19} …	A_{16}	A_{15} … A_{12}	A_{11} … A_8	A_7 … A_4	A_3 … A_0	
1 1 1 1	1	1 1 1 1	1 1 1 1	1 1 1 1	1	64 K × 8 位 ROM，其中 1 片 32 K × 8 位(奇)
…						
1 1 1 1	0	0 0 0 0	0 0 0 0	0 0 0 0	0	1 片 32 K × 8 位(偶)
A_{19} …	A_{16}	A_{15} … A_{12}	A_{11} … A_8	A_7 … A_4	A_3 … A_0	
1 1 1 0	1	1 1 1 1	1 1 1 1	1 1 1 1	1	64 K × 8 位 RAM，其中 1 片 32 K × 8 位(奇)
…						
1 1 1 0	0	0 0 0 0	0 0 0 0	0 0 0 0	0	1 片 32 K × 8 位(偶)

101

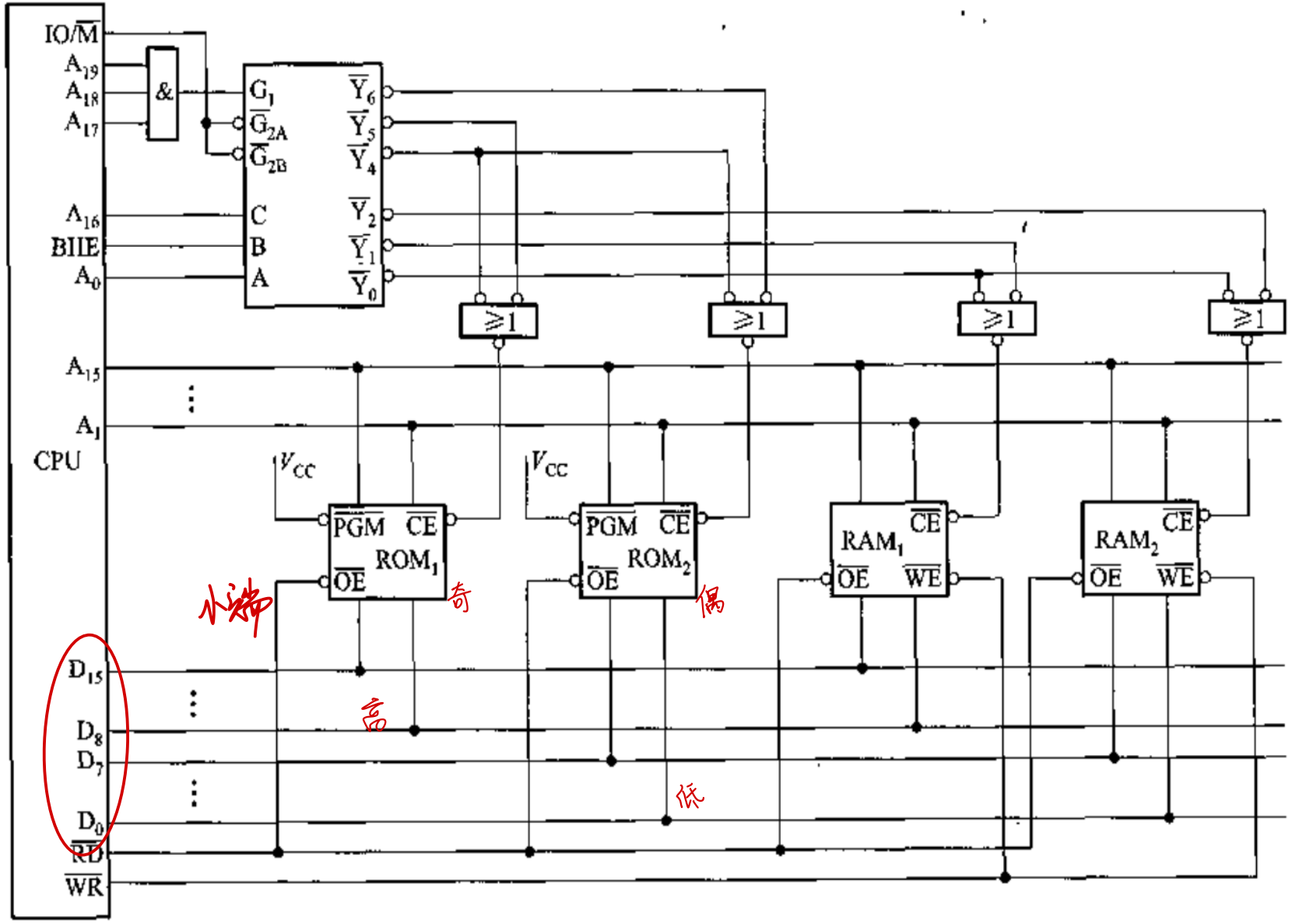
110

001

010

字 000

例 4.3 CPU 与存储器的连接图



*例3：CPU和主存的连接

CPU地址线A15～A0，数据线D7～D0，$\overline{WR}$为读/写信号，$\overline{MREQ}$为访存请求信号。0000H～3FFFH为BIOS区，4000H～FFFFH为用户程序区。用8K×4位ROM芯片和16K×8位RAM芯片构成该存储器，要求说明地址译码方案，并将ROM芯片、RAM芯片与CPU连接。

解：因为0000H～3FFFH为BIOS，故ROM区高两位总是00，低14位为全译码。

ROM区大小为：2^{14}×8位=16K×8位=16KB，ROM芯片数为：

16K×8位 / 8K×4位 = 2×2 = 4，字方向扩展2倍，位方向扩展2倍。

ROM芯片内地址位数为13位，连到CPU低13位地址线A12～A0。

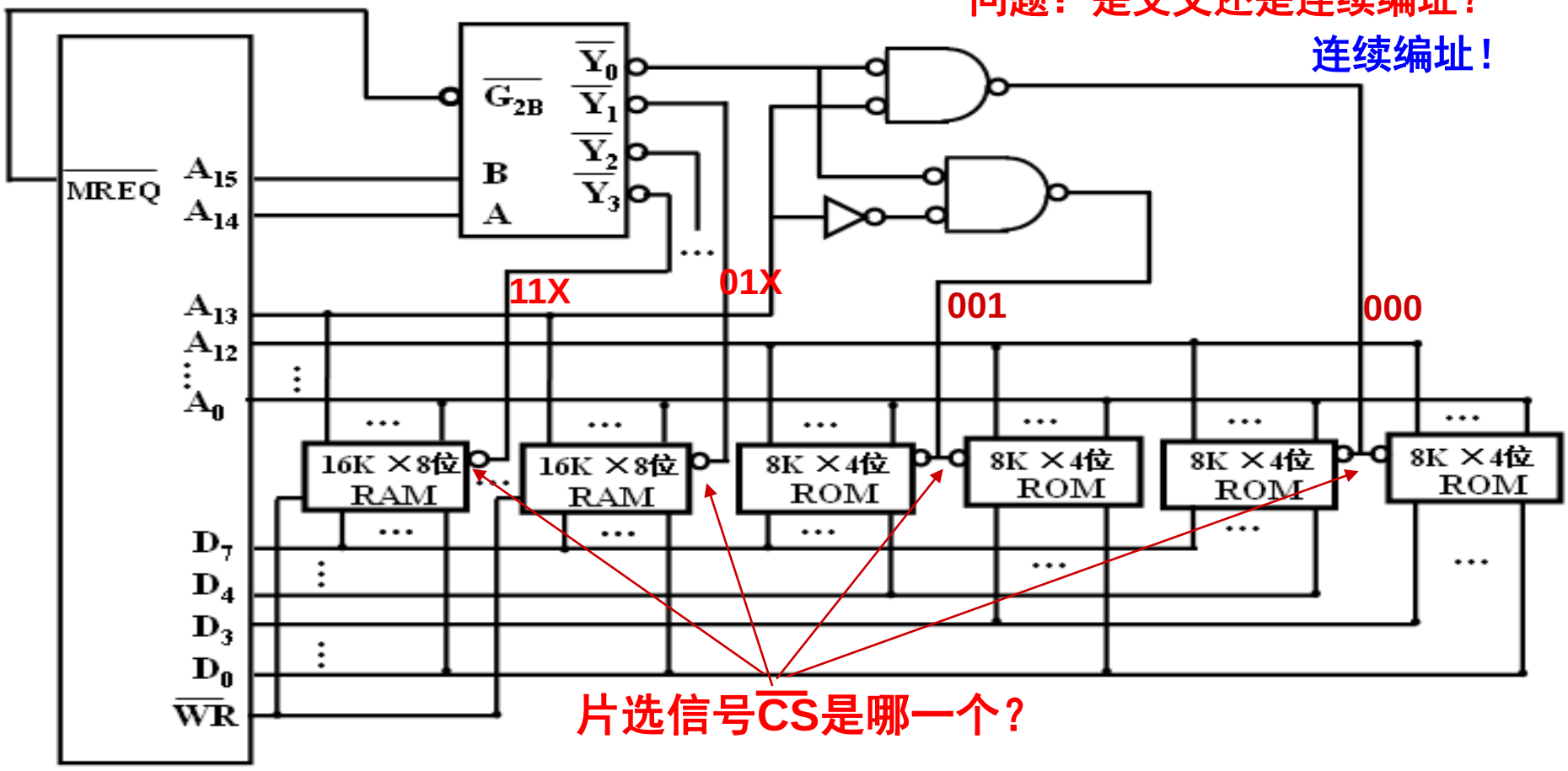
因为4000H～FFFFH为用户程序区，故RAM区高两位是01、10、11，低14位为全译码。RAM区大小为：3×2^{14}×8位=3×16K×8位= 48KB。

RAM芯片数为：

48K×8位 / 16K×8位 = 3×1 = 3，字方向上扩展3倍，位方向上不扩展。

RAM芯片内地址位数为14位，连到CPU低14位地址线A13～A0。

* ROM片选由最高三位确定。
RAM片选由最高两位确定。



问题：为什么$\overline{WR}$不连到ROM芯片上？ ROM芯片只读不写

问题：$\overline{MREQ}$信号的作用是什么？ 有效(低电平)时，表示选中主存读写。

*练习

CPU有16根地址线（A15～A0），8根数据线（D7～D0），$\overline{\text{MREQ}}$作为访问存储器的控制电平（低电平有效），$\overline{\text{WE}}$作为读写控制电平（$\overline{\text{WE}}$=0时，写允许，$\overline{\text{WE}}$＝1时，读允许）。现有芯片：2114（1K×4位），需要扩展为2KB内存，地址范围为2000H~27FFH，片选信号由74LS138（3－8译码器）译码进行，试画出CPU与3－8译码器及存储芯片的连接。

*存储器接口

(1) 8位存储器接口

如果数据总线为8位，则存储器只能按字节编址

(2) 16位存储器接口

16位存储器系统由2个存储体组成，存储体选择通过选择信号BHE实现。

如果要传送一个16位数，则2个存储器被选中

如果要传送一个8位数，则1个存储器被选中

(3) 32位存储器接口

32位存储器系统由4个存储体组成，存储体选择通过选择信号$\overline{BE_3}$~$\overline{BE_0}$实现。

如果要传送一个32位数，则4个存储器被选中

如果要传送一个16位数，则2个存储器被选中

如果要传送一个8位数，则1个存储器被选中

*存储器接口

(4) 64位存储器接口

64位存储器系统由8个存储体组成，存储体选择通过选择信号BE7~BE0实现。

如果要传送一个64位数，则8个存储器被选中

如果要传送一个32位数，则4个存储器被选中

如果要传送一个16位数，则2个存储器被选中

如果要传送一个8位数，则1个存储器被选中

六、存储器的校验

- 为什么要对存储器的信息进行校验?
- 为了能够校验出信息是否正确，如何进行编码?
- 纠错或检错能力与什么因素有关?
- 校验出信息出错后是如何进行纠错?
- 除了我们教材上讲的校验码，你还知道哪些容错编码?，原理是什么?

六、存储器的校验

- 合法代码集合

检0位错、纠0位错

- 1、{000，001，010，011，100，101，110，111}

所有无纠错能力

- 2、{000， 011，101，110}

100 检1位错，纠0位错

- 3、{000，111}

100 110 检1位错，纠1位错

- 4、{0000，1111}

1000 1100 检2位错，纠1位错

- 5、{00000，11111}

11000 11100 检2位错，纠2位错

编码的检测能力和纠错能力与什么有关呢?

任意两组合法代码之间二进制位的最少差异数。 码距

码距与检错、纠错能力的关系

编码的纠错 、检错能力与编码的最小距离L有关

$$L-1=D+C\ (D\geq C)$$

L： 编码的最小距离

D： 检测错误的位数

C： 纠正错误的位数

L = 3

具有 一位纠错能力

码距与检错、纠错能力的关系（当d≤4）

① 若码距L为奇数，则能发现L-1位错，或能纠正(L-1)/2位错。

② 若码距L为偶数，则能发现L/2位错，并能纠正(L/2-1)位错。

海明码是具有发现**两位**错和纠正**一位**错能力的编码。

六、存储器的校验

- 数据校验码：是一种常用的带有发现某些错误或带有自动纠错能力的数据编码系统。
- 数据校验码检错纠错方法：“冗余校验”。
- 由若干位代码组成的一个字叫“码字”。
- 码距：一种码制中，任意两个合法码字之间的最小距离（不同的二进制位数）称为**码距**。
- 合理增大码距能提高检错和纠错能力。
- 常用的数据校验码有**奇偶校验码**、**海明校验码**和**循环冗余校验码**（CRC）。

1、奇偶校验码

- 奇校验：每个码字（包括校验位）中1的个数为奇数
- 偶校验：每个码字（包括校验位）中1的个数为偶数

1、奇偶校验码

基本思想：增加一位奇（偶）校验位并一起存储或传送，根据终部件得到的相应数据和校验位，再求出新校验位，最后根据新校验位确定是否发生了错误。

实现原理：假设数据$B=b_{n-1}b_{n-2}...b_1b_0$从源部件传送至终部件。在终部件接收到的数据为$B'=b_{n-1}'\ b_{n-2}'\ ...b_1'\ b_0'$

第一步：在源部件求出奇（偶）校验位P。

若采用奇校验，则$P=b_{n-1}\oplus b_{n-2}\oplus...\oplus b_1\oplus b_0\oplus 1$

若采用偶校验，则$P=b_{n-1}\oplus b_{n-2}\oplus...\oplus b_1\oplus b_0$

第二步：在终部件求出奇（偶）校验位P'。

若采用奇校验，则$P'=b_{n-1}'\oplus b_{n-2}'\oplus...\oplus b_1'\oplus b_0'\oplus 1$

若采用偶校验，则$P'=b_{n-1}'\oplus b_{n-2}'\oplus...\oplus b_1'\oplus b_0'$

第三步：计算最终的校验位P*，并根据其值判断有无奇偶错。

假定P在终部件接受到的值为P"，则$P^*=P'\oplus P''$

① 若P*=1，则表示终部件接受的数据有奇数位错。

② 若P*=0，则表示终部件接受的数据正确或有偶数个错。

奇偶校验码的特点

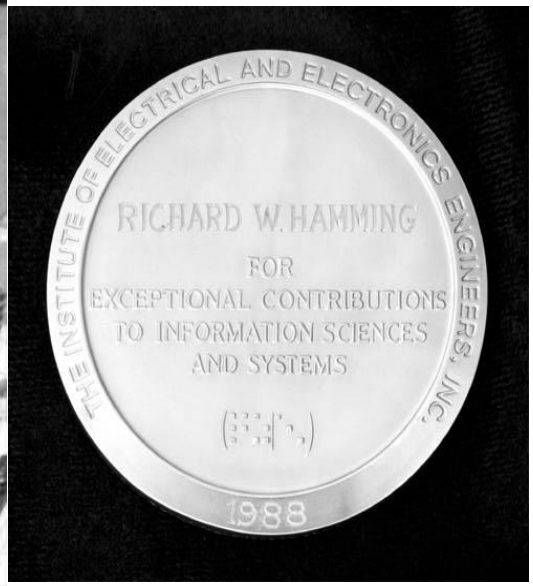
- **问题：奇偶校验码的码距是几？为什么？**
  - 码距d=2。

    在奇偶校验码中，若两个数中有奇数位不同，则它们相应的校验位就不同；若有偶数位不同，则虽校验位相同，但至少有两位数据位不同。因而任意两个码字之间至少有两位不同。
- **特点**
  - 根据码距和纠/检错能力的关系，它只能发现奇数位出错，不能发现偶数位出错，而且也不能确定发生错误的位置，不具有纠错能力。
  - 开销小，适用于校验一字节长的代码，故常被用于存储器读写检查或按字节传输过程中的数据校验

    因为一字节长的代码发生错误时，1位出错的概率较大，两位以上出错则很少，所以可用奇偶校验。

Richard Hamming (1915.2.11-1998.1.7)

- **1968年Richard W.Hamming因在数值方法，自动编码系统，错误检测和错误纠正码方面的贡献被授予图灵奖**
- **1988年因为在信息科学与系统方面的杰出贡献，IEEE授予R.H.一枚奖章，并将此奖章命名为 "the Hamming Medal"**



2、海明（汉明）校验码

成检2位纠1位

- 海明码：是一种能够纠正一位错误的线性编码系统。
  - 1950年由贝尔实验室的Richard Hamming首先提出
  - 主要用于存储器校验
- 基本思想：分组校验（**多重奇偶校验**）
  - 将有效信息按某种规律分为若干组，每组安排一个校验位进行奇偶校验。
  - 在数据位组中加入几个校验位，增加了数据代码间的码距。
  - 当某一位发生变化时会引起校验结果发生变化，不同代码位上的错误会得出不同的校验结果。

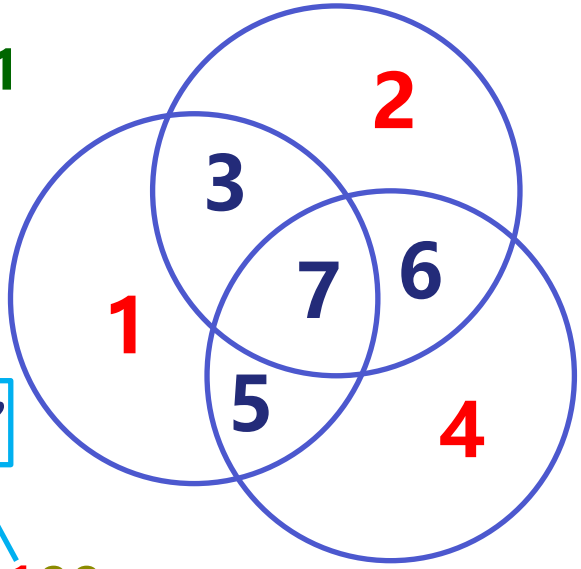
2、海明（汉明）校验码

偶校验

00100011　　100100011　　1001000011

检验位

海明码的分组是非划分方式，组间有交叉。



1	2	3	4	5	6	7

校验位位置固定 2^i

分组看“1”

第1组: 1 3 5 7

第2组: 2 3 6 7

第3组: 4 5 6 7

1: 001	2: 010	4: 100
3: 011	3: 011	5: 101
5: 101	6: 110	6: 110
7: 111	7: 111	7: 111

$P_3P_2P_1$	
0 0 0	无差错
0 0 1	1
1 0 1	5
1 1 1	7

给出错误位置

分成3组，每组有1位校验位，共包括4位数据位

3、海明（汉明）码的组成

① 确定校验位位数：

设欲检测的二进制代码为n位，r为校验位位数，则应满足不等式：

$$2^r \geq n+r+1$$

例：若欲检测8位二进制代码，则需要添加几个校验位?

设r = 3，则$2^3 = 8$， n+r+1 = 8 + 3+1 = 12，不满足

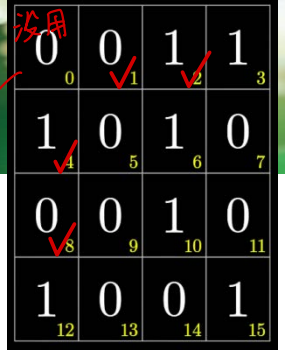
设r = 4，则$2^4 = 16$，n+r+1 = 8 + 4+1 = 13，满足

所以，r最小取4。

② 校验位应该放在哪些位置呢?

2^i (i = 0，1，2 ，3 ，……)

3、海明（汉明）码的组成



整个信息校验

③ 校验位的形成（取值）（偶校验）：

设欲检测的二进制代码为：01101110

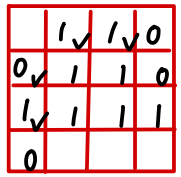
C_1 = 第一组中所有位（除C_1外）求异或 = $b_7 \oplus b_6 \oplus b_4 \oplus b_3 \oplus b_1 = 0 \oplus 1 \oplus 0 \oplus 1 \oplus 1 = 1$

C_2 = 第二组中所有位（除C_2外）求异或 = $b_7 \oplus b_5 \oplus b_4 \oplus b_2 \oplus b_1 = 0 \oplus 1 \oplus 0 \oplus 1 \oplus 1 = 1$

C_3 = 第三组中所有位（除C_3外）求异或 = $b_6 \oplus b_5 \oplus b_4 \oplus b_0 = 1 \oplus 1 \oplus 0 \oplus 0 = 0$

C_4 = 第四组中所有位（除C_4外）求异或 = $b_3 \oplus b_2 \oplus b_1 \oplus b_0 = 1 \oplus 1 \oplus 1 \oplus 0 = 1$

∴ 海明码为：110011011110



	1	2	3	4	5	6	7	8	9	10	11	12
	C_1	C_2	b_7	C_3	b_6	b_5	b_4	C_4	b_3	b_2	b_1	b_0
	1	1	0	0	1	1	0	1	1	1	1	0
第一组(C_1)1	√		√		√		√		√		√	
第二组(C_2)2		√	√			√	√			√	√	
第三组(C_3)4				√	√	√	√					√
第四组(C_4)8								√	√	√	√	√

3、海明（汉明）码的组成

④ 校验原理

在数据输出存储器时分别求P_1,P_2,P_3,P_4

$P_1=C_1\oplus$第一组中所有位求异或$=C_1\oplus b_7\oplus b_6\oplus b_4\oplus b_3\oplus b_1$

$P_2=C_2\oplus$第二组中所有位求异或$=C_2\oplus b_7\oplus b_5\oplus b_4\oplus b_2\oplus b_1$

$P_3=C_3\oplus$第三组中所有位求异或$=C_3\oplus b_6\oplus b_5\oplus b_4\oplus b_0$

$P_4=C_4\oplus$第四组中所有位求异或$=C_4\oplus b_3\oplus b_2\oplus b_1\oplus b_0$

检错

当$P_4P_3P_2P_1$=0000时，则从存储器中取出的数据无错。

否则，$P_4P_3P_2P_1$的二进制编码即为出错位号。

例如：$P_4P_3P_2P_1$=1001，

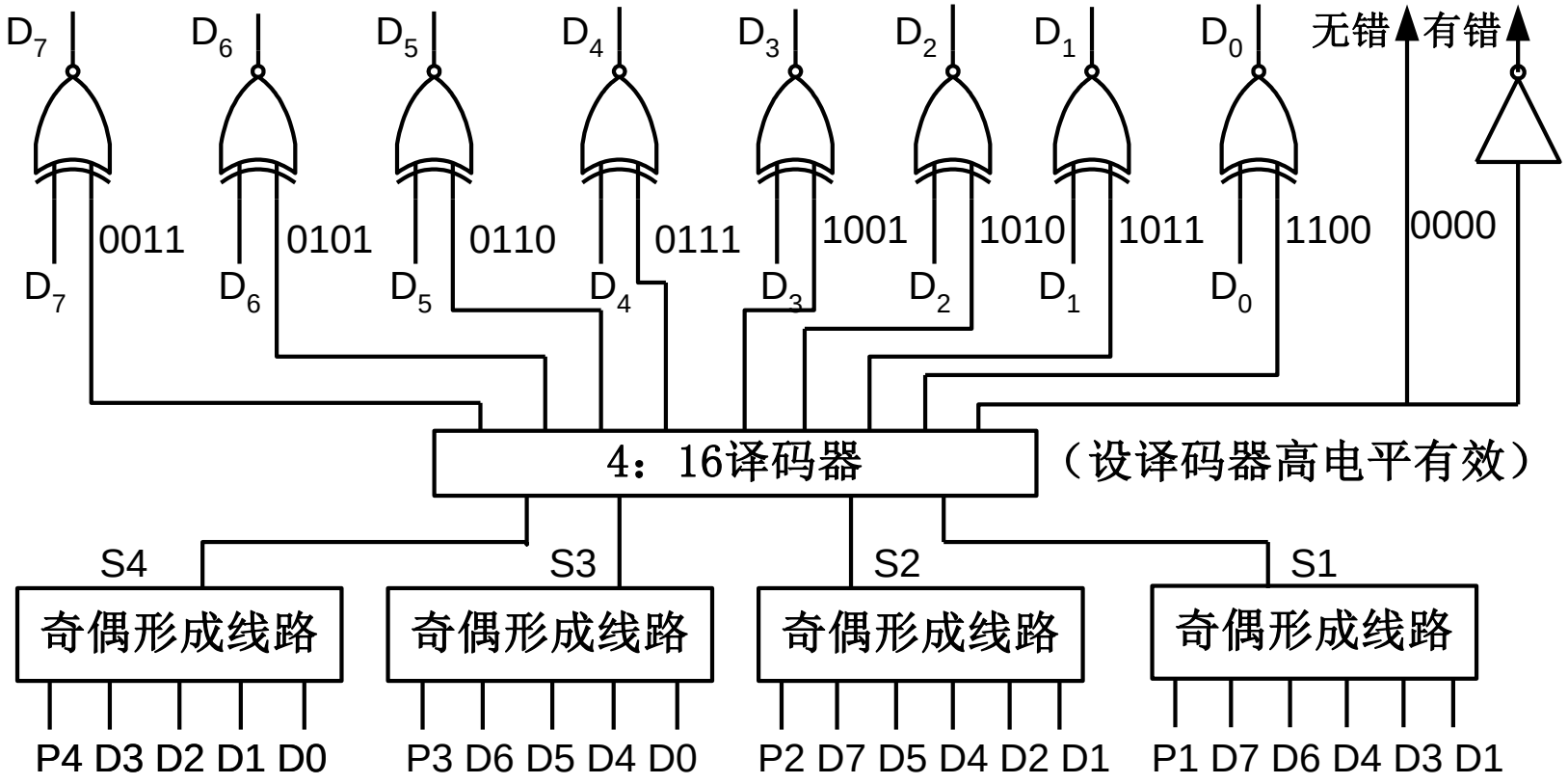
1	2	3	4	5	6	7	8	9	10	11	12
C_1	C_2	b_7	C_3	b_6	b_5	b_4	C_4	b_3	b_2	b_1	b_0

P_4为1

说明第9位出错，即b_3错，将其取反，即可纠错。

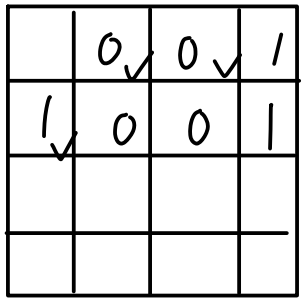
3、海明（汉明）码的组成

- 纠正一位出错位的海明校验逻辑电路



设置

按配偶原则配置1001的汉明码为__。



A 0011001

B 0111001

C 1110001

D 0010001

提交

例4.4 P102

已知接收到的汉明码为 0110101 （按配偶原则配置）试问要求传送的信息是什么?

解： 0110101

纠错过程如下：

	0	1	1
0	1	0	1

$P_1=1$
$P_2=1$
$P_3=0$
$P_3P_2P_1=011$

$P_1=1⊕3⊕5⊕7=0⊕1⊕1⊕1=1$

$P_2=2⊕3⊕6⊕7=1⊕1⊕0⊕1=1$

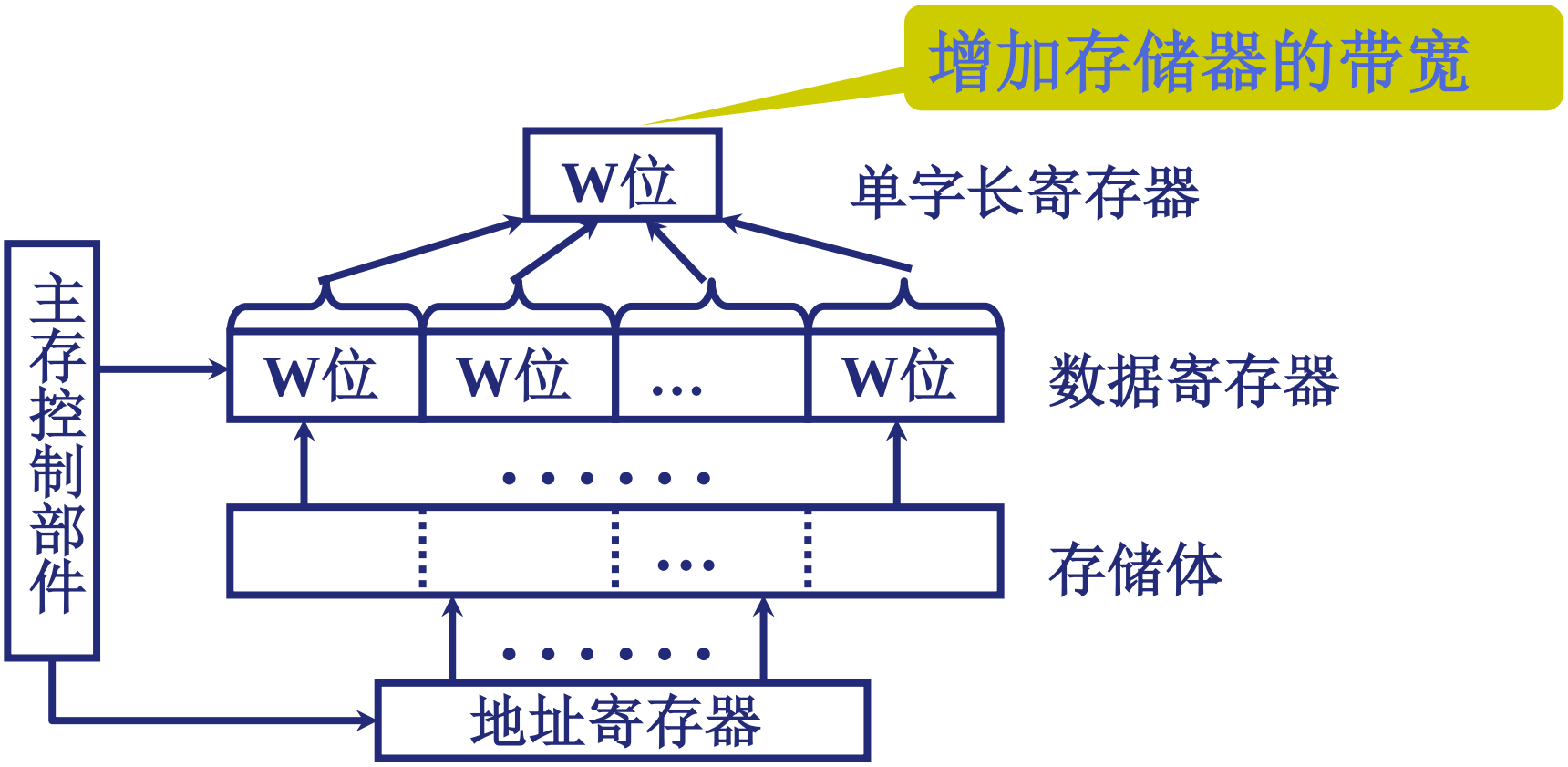
$P_3=4⊕5⊕6⊕7=0⊕1⊕0⊕1=0$

∴ $P_3P_2P_1=011$，即第3位出错，可纠正为0100101.

七、提高访存速度的措施

- 采用高速器件
- 采用cache（高速缓冲存储器）
- 调整主存结构
- 采用并行存储器（多体交叉存储器）

1. 单体多字系统



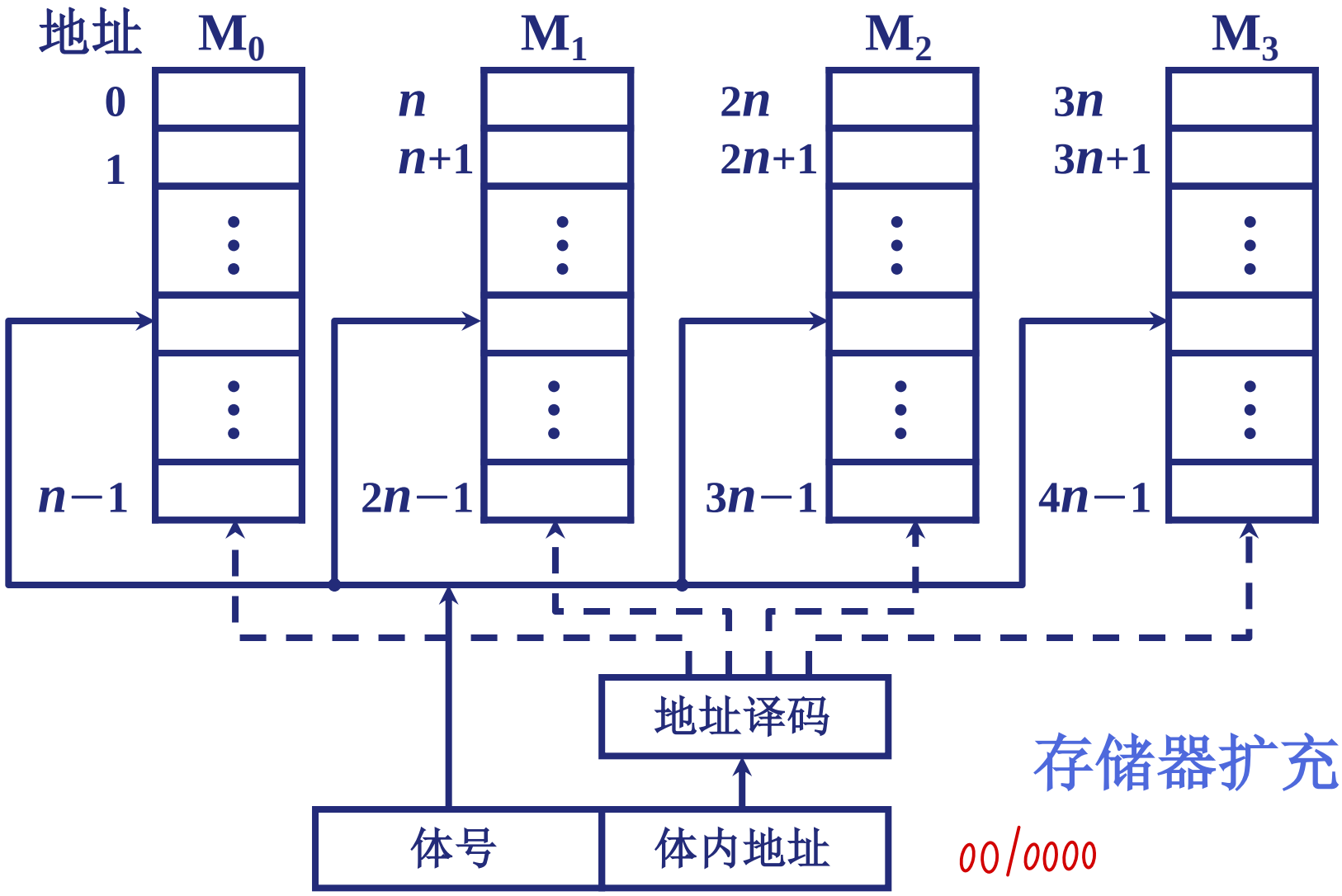
前提：指令和数据在主存中必须是连续存放的，一旦遇到转移指令，或者操作数不能连续存放，这种方法的效果就不明显了。

2. 多体并行系统

- 计算机中大容量的主存，可由多个存储体组成，每个存储体都具有自己的读写线路、地址寄存器和数据寄存器，称为“存储模块”。这种多模块存储器可以实现重叠与交叉存取。

(1) 高位交叉

如果不同请求源访问不同的体，可实现各个体并行工作。



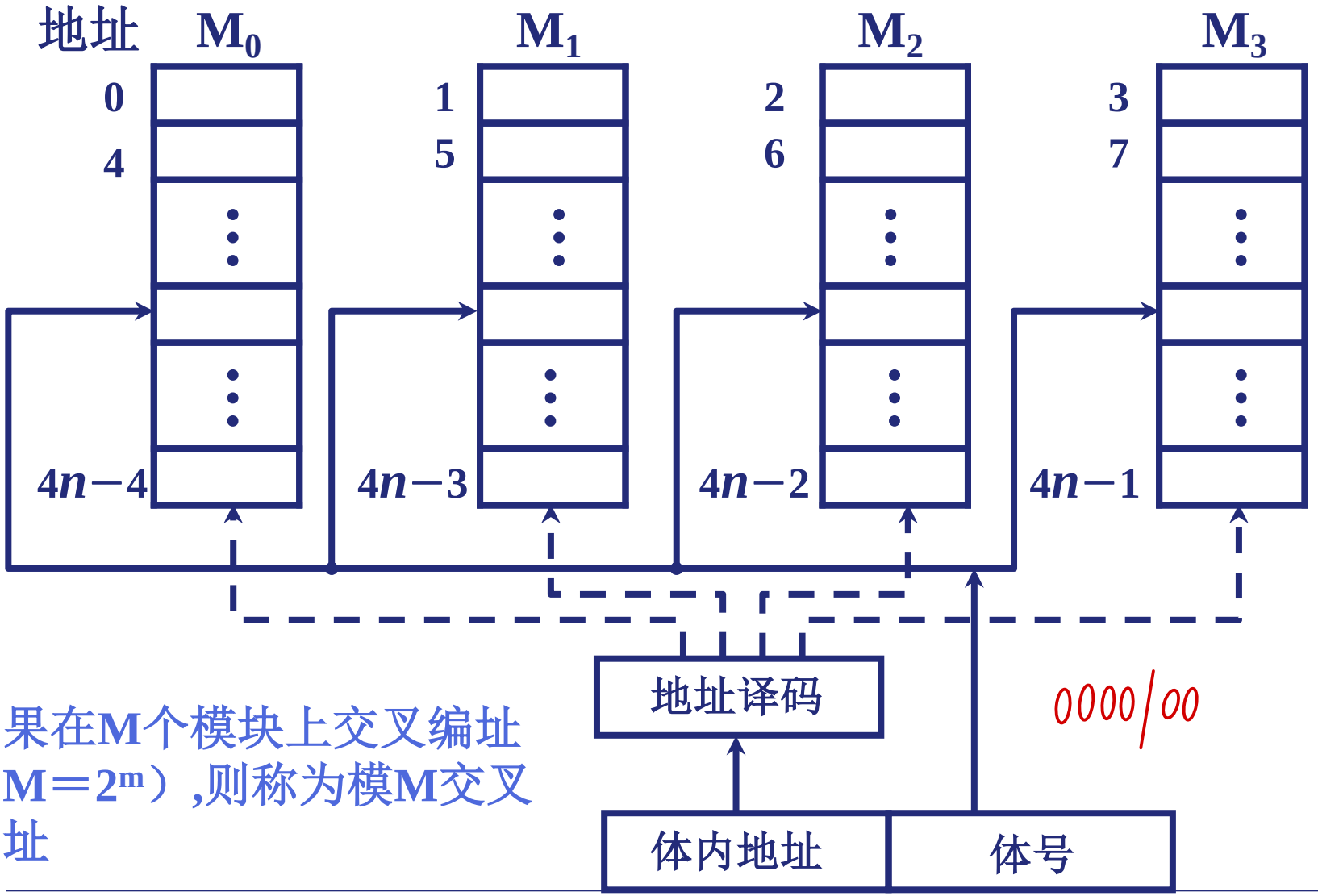
存储器扩充容易。

(2) 低位交叉（各个体轮流编址）

4.2

并行读取

连续地址分布在相邻的不同模块内，同一模块内的地址都不连续

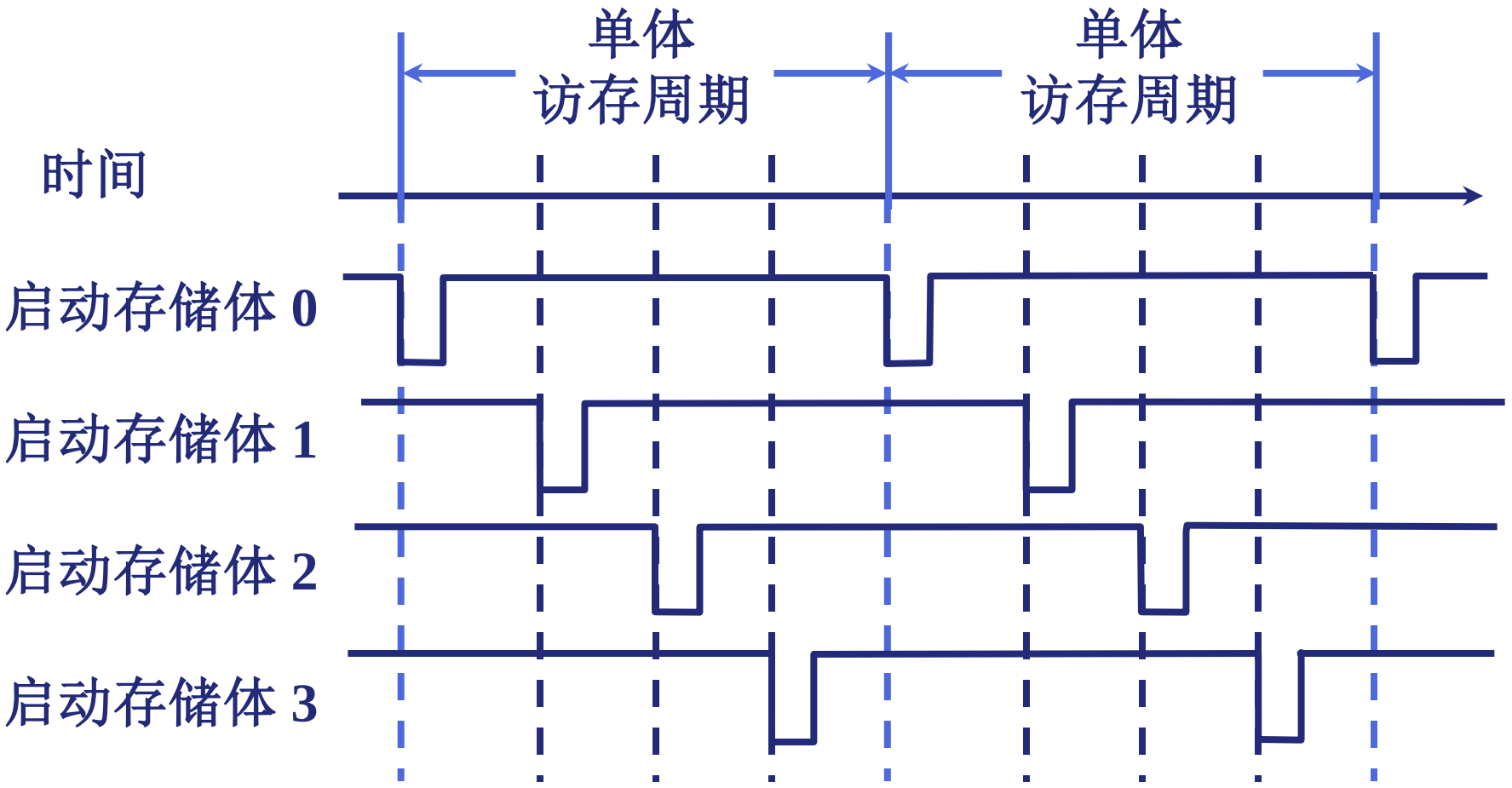


如果在M个模块上交叉编址（$M=2^m$），则称为模M交叉编址

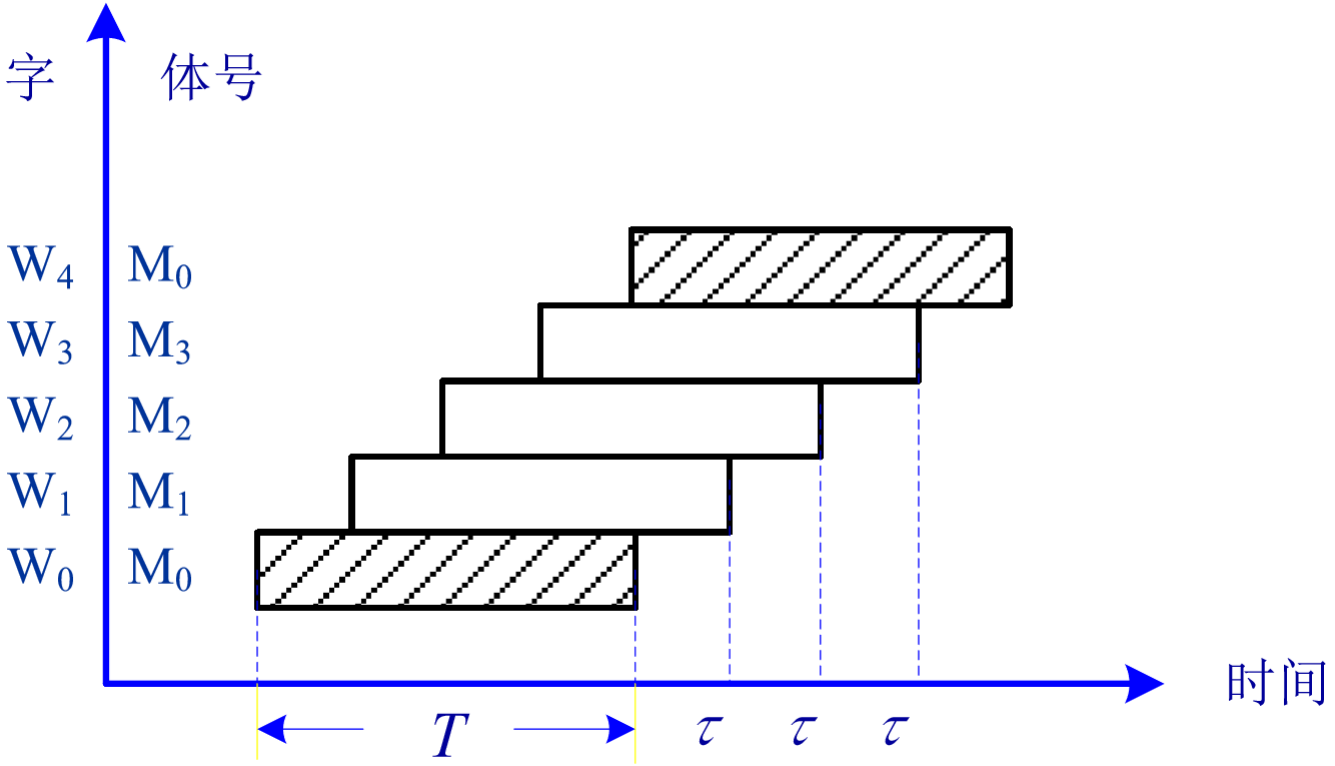
0000/00

低位交叉的特点

在不改变存取周期的前提下，增加存储器的带宽



设四体低位交叉存储器，存取周期为T，总线传输周期为τ，为实现流水线方式存取，应满足 T = 4τ。 一般 τ<T



连续读取 4 个字所需的时间为 T + (4 - 1)τ

3.高性能芯片

- **(1) SDRAM (同步 DRAM)**

  在系统时钟的控制下进行读出和写入

  CPU 无须等待

- **(2) RDRAM**

  由 Rambus 开发，主要解决 **存储器带宽** 问题

- **(3) 带 Cache 的 DRAM**

  在 DRAM 的芯片内 集成 了一个由 SRAM 组成的 Cache，

  有利于 **猝发式读取**

4.3 高速缓冲存储器

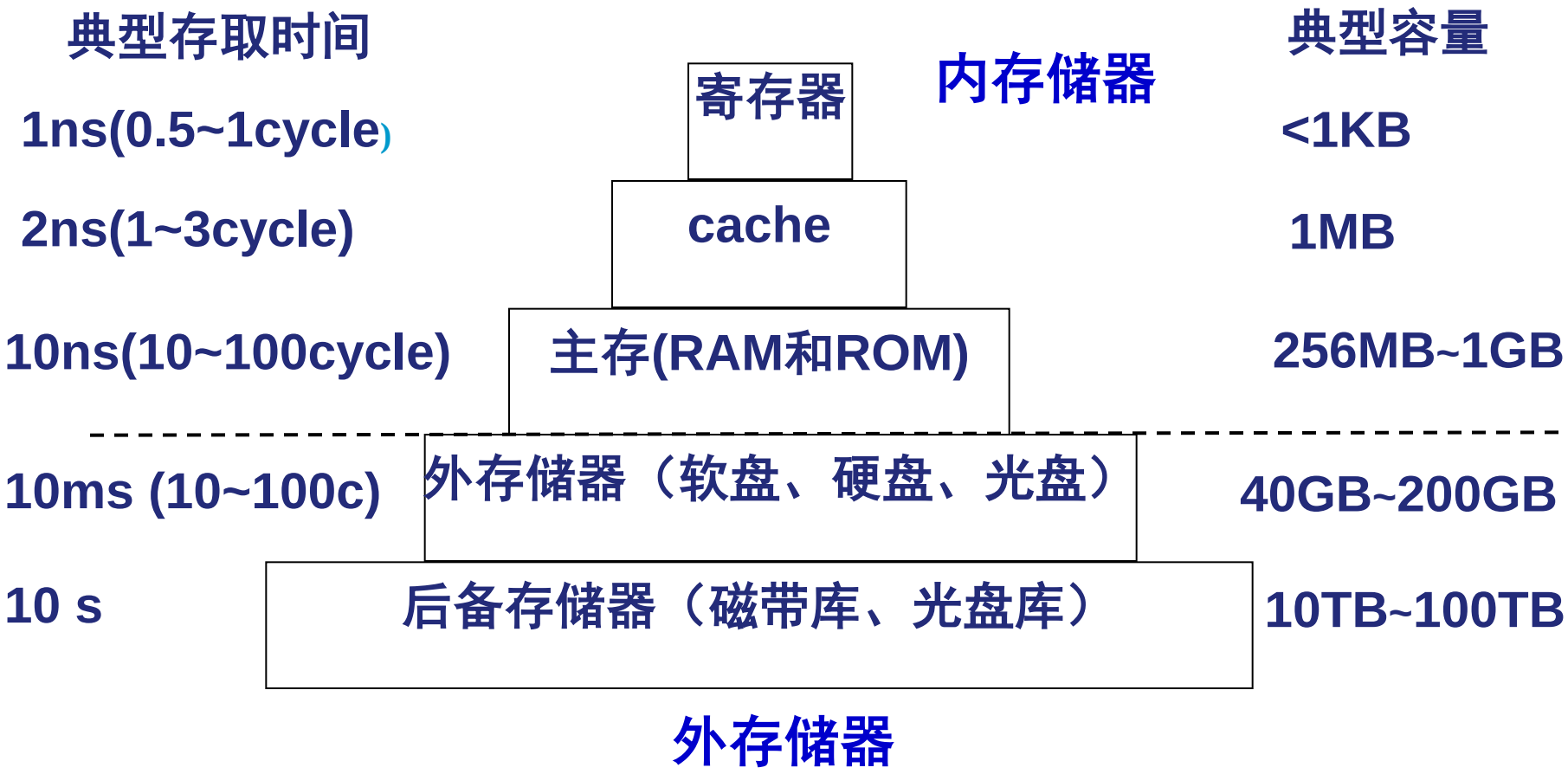
一、概述

- 1、为什么用Cache
- 2、Cache的工作原理
  - 主存和缓存的编址
  - 命中与未命中
  - Cache的命中率
  - Cache-主存系统的效率
- 3、Cache的基本结构
- 4、Cache的读写操作
- 5、Cache的改进

二、Cache-主存的地址映射

三、替换算法

计算机中存储器的层次结构



给出的时间和容量会随时间变化，但数量级相对关系不变。

4.3 高速缓冲存储器

一、概述

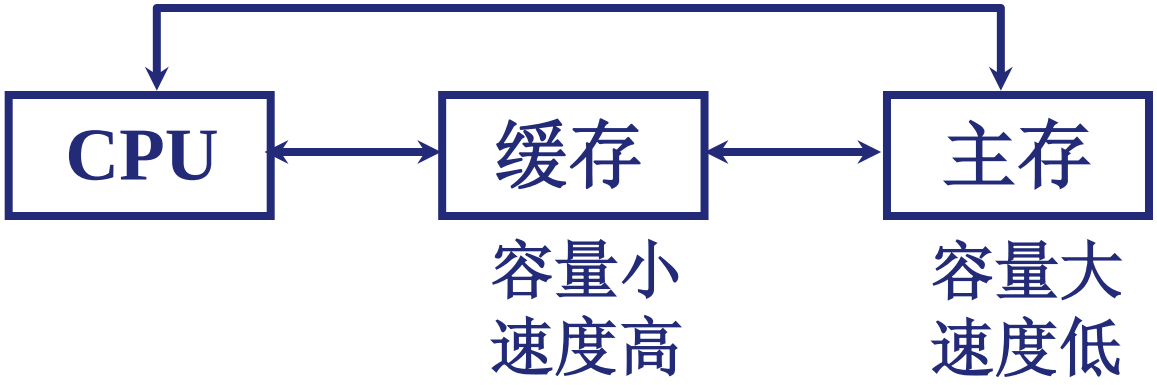
多体并行系统中I/O访存级别高于CPU访存级别，CPU等待 外设 > CPU

1. 问题的提出

主存速度发展低于CPU速度发展，CPU与DRAM速度间隙每年增加50％

避免 CPU “空等” 现象

CPU 和主存（DRAM）的速度差异



理论依据：程序访问的局部性原理。即：在一个较短的时间间隔内，CPU对局部范围的存储器地址频繁访问，而对此地址范围之外的地址访问很少。

程序访问的局部性原理

- **引入Cache的理论依据：程序访问的局部性原理。**
  - **大量典型程序的运行情况分析结果表明，在一个较短的时间间隔内，CPU对局部范围的存储器地址频繁访问，而对此地址范围之外的地址访问很少。**
- **程序具有访问局部性特征的原因**
  - **指令：指令按序存放，地址连续，循环程序段或子程序段重复执行**
  - **数据：连续存放，数组元素重复、按序访问**

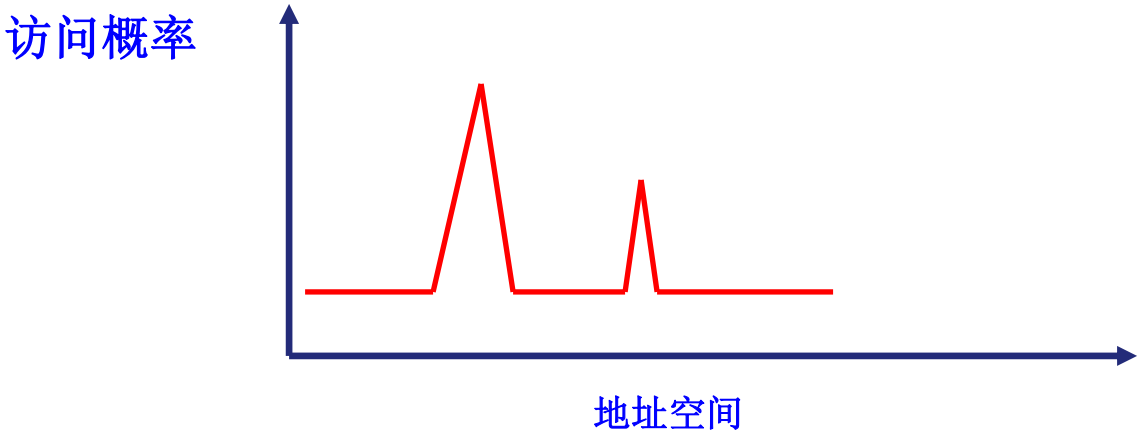
程序的局部性原理

```
for (i=0; i<1000; i++) {
   for (j=0; j<1000; j++ ) {
      a[i] = b[i] + c[i];
   }
}

If err { ……}
   else for (i=0; i<1000; i++) {
      for (j=0; j<1000; j++ ) {
         e[i] = d[i] * a[i];
      }
   }
……
```

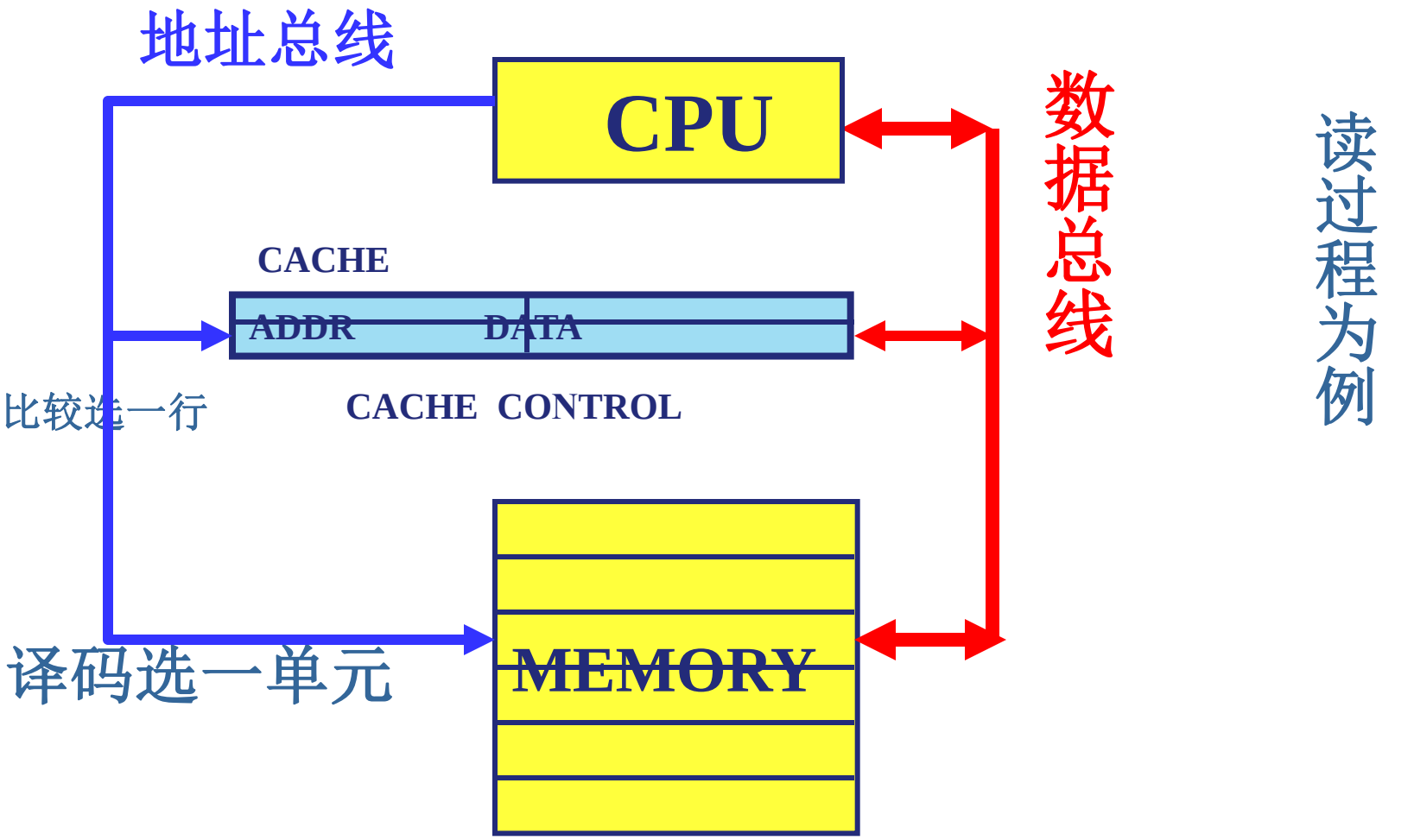
- 数据流访问的局部性
- 指令访问的局部性
- 不同的程序段可能访问不同的内存空间

程序的局部性原理



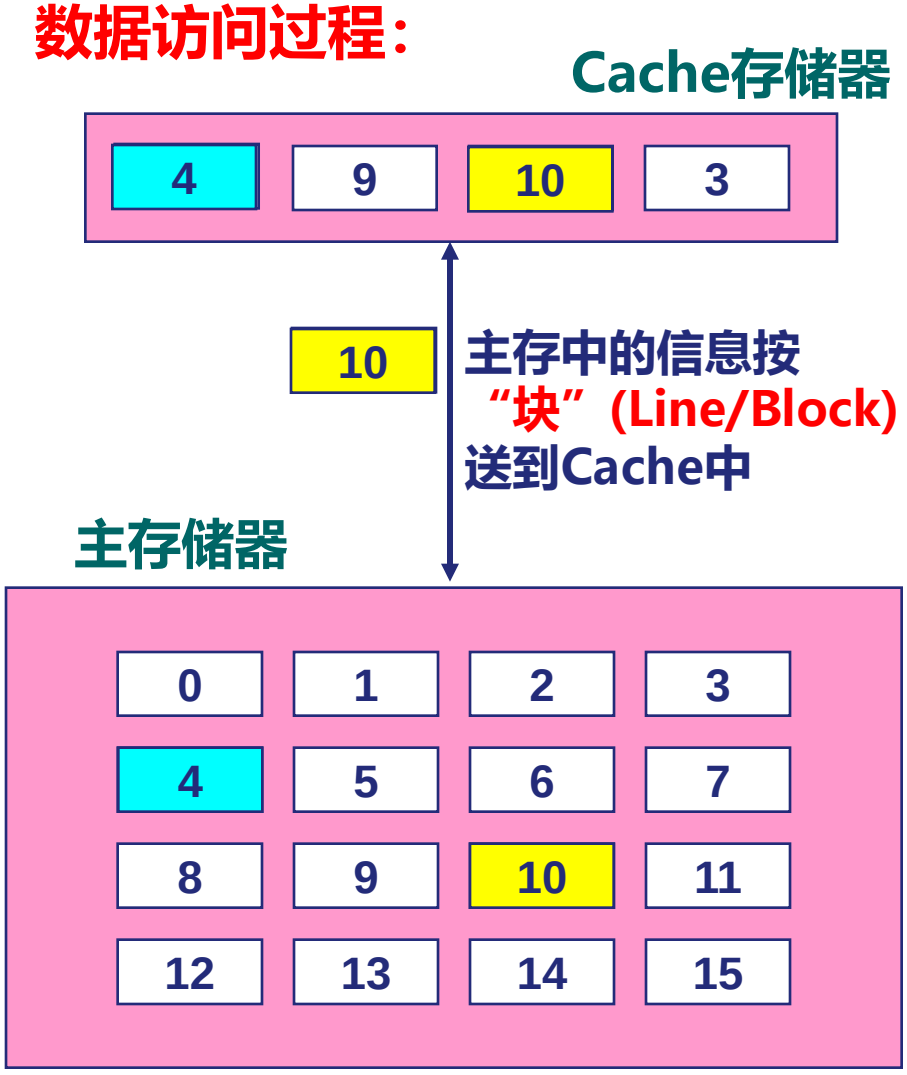
- 程序在一定的时间段内通常只访问较小的地址空间
- 两种局部性：
  - 时间局部性（Temporal Locality）
    - 定义：刚被访问过的单元很可能不久又被访问
    - 做法：让最近被访问过的信息保留在靠近CPU的存储器中
  - 空间局部性（Spatial Locality）
    - 含义：刚被访问过的单元的邻近单元很可能不久被访问
    - 做法：将刚被访问过的单元的邻近单元调到靠近CPU的存储器中

Cache的基本运行原理



Cache(高速缓存)是什么样的？

- Cache是一种小容量高速缓冲存储器，它由SRAM组成。
- Cache直接制作在CPU芯片内，速度几乎与CPU一样快。
- 程序运行时，CPU使用的一部分数据/指令会预先成批拷贝在Cache中，Cache的内容是主存储器中部分内容的映象。
- 当CPU需要从内存读(写)数据或指令时，先检查Cache，若有，就直接从Cache中读取，而不用访问主存储器。



要解决的问题

- 1. 地址和Cache行之间的映射关系：
- 如何根据主存地址得到Cache中的数据？

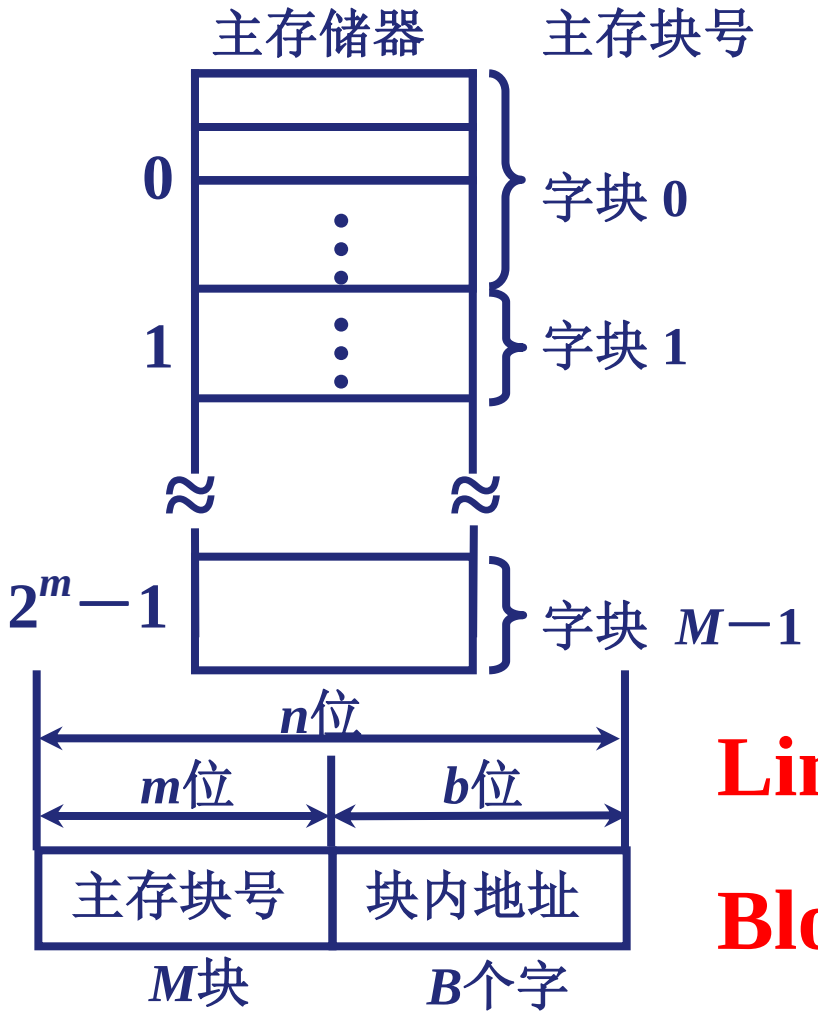
- 2. 数据之间一致性：
- Cache中的内容是否已经是主存对应地址的内容？

- 3. 数据交换的粒度：
- Cache中的内容与主存内容以多大的粒度交换？

- 4. Cache内容装入和替换策略
- 如何提高Cache的命中率？

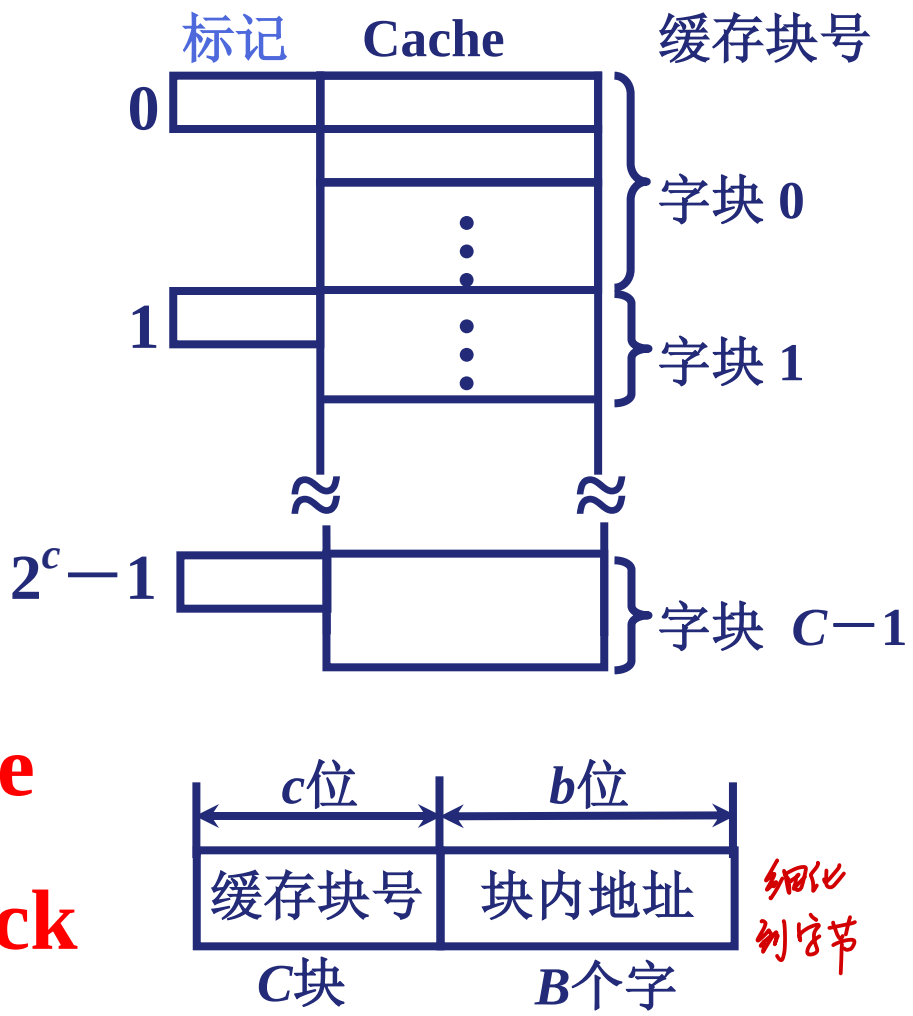
2. Cache 的工作原理

(1) 主存和缓存的编址



Line

Block



主存和缓存按块交换信息　　块的大小相同　B 为块长

(2) 命中与未命中

缓存共有 C 块

主存共有 M 块　$M >> C$

命中(hit)　主存块 调入 缓存

主存块与缓存块建立了对应关系

用 标记记录 与某缓存块建立了对应关系的主存块块号

未命中
(缺失 miss)　主存块 未调入 缓存

主存块与缓存块未建立对应关系

(3) Cache 的命中率（hit rate）

变换

CPU 欲访问的信息在 Cache 中的 比率

在程序执行期间，设*Nc*为访问Cache命中的次数，*Nm*为访问主存总次数，命中率为*h*，则

$$h = \frac{Nc}{Nc + Nm}$$

命中率 与 Cache 的 容量 与 块长 有关

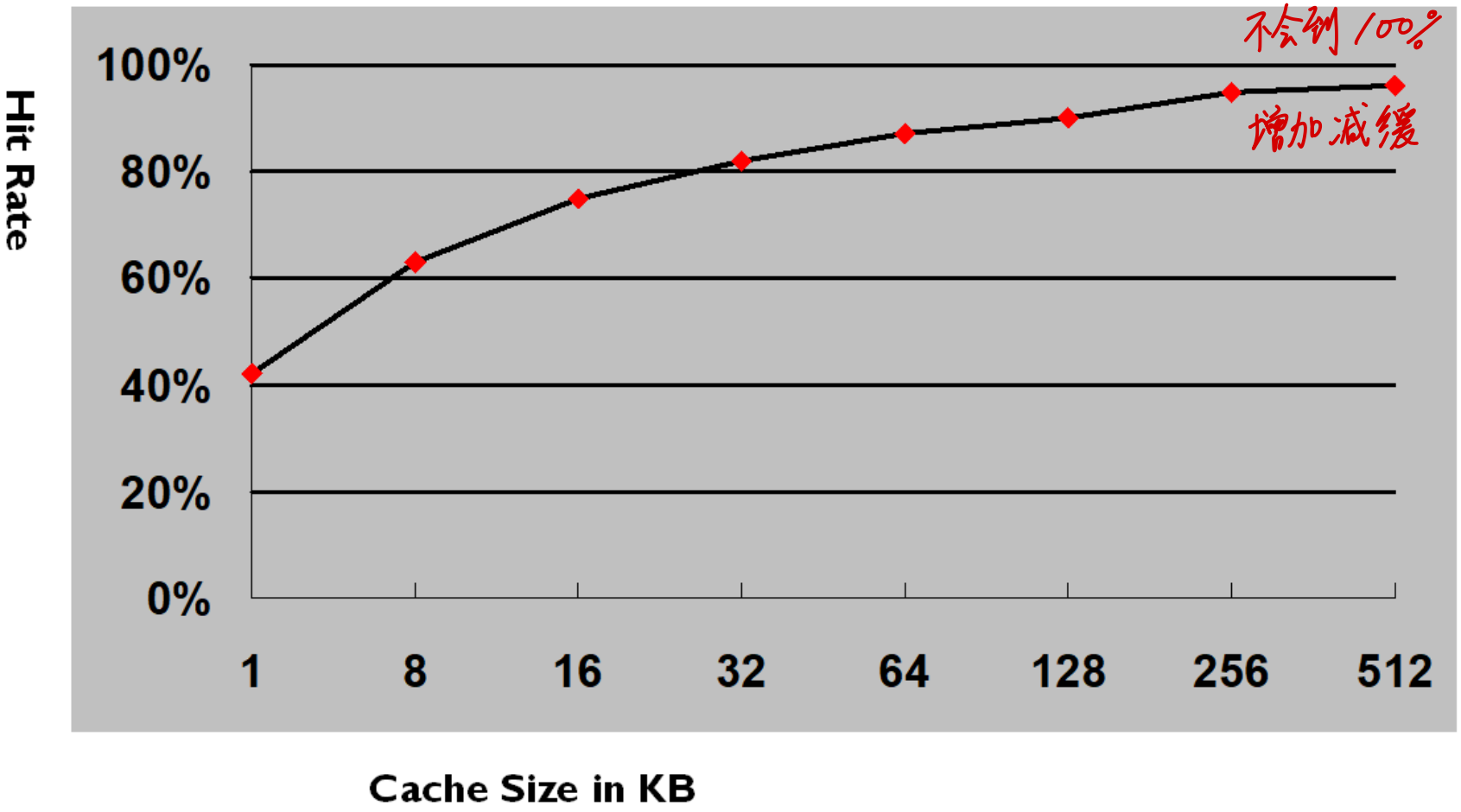
一般每块可取 4 至 8 个字　块长取一个存取周期内从主存调出的信息长度

CRAY_1　16体交叉　块长取 16 个存储字

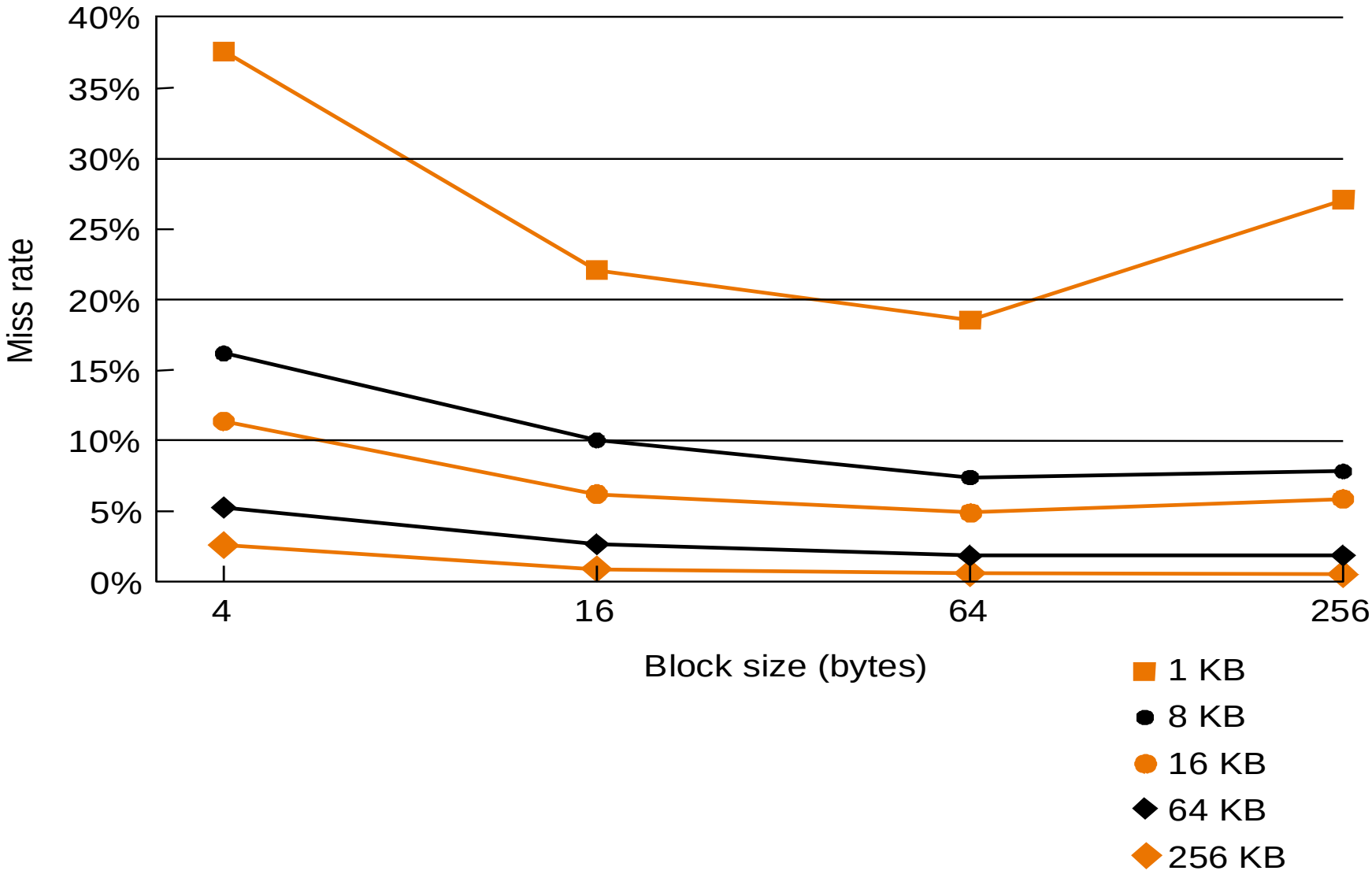
IBM 370/168　4体交叉　块长取 4 个存储字

（64位×4 = 256位）

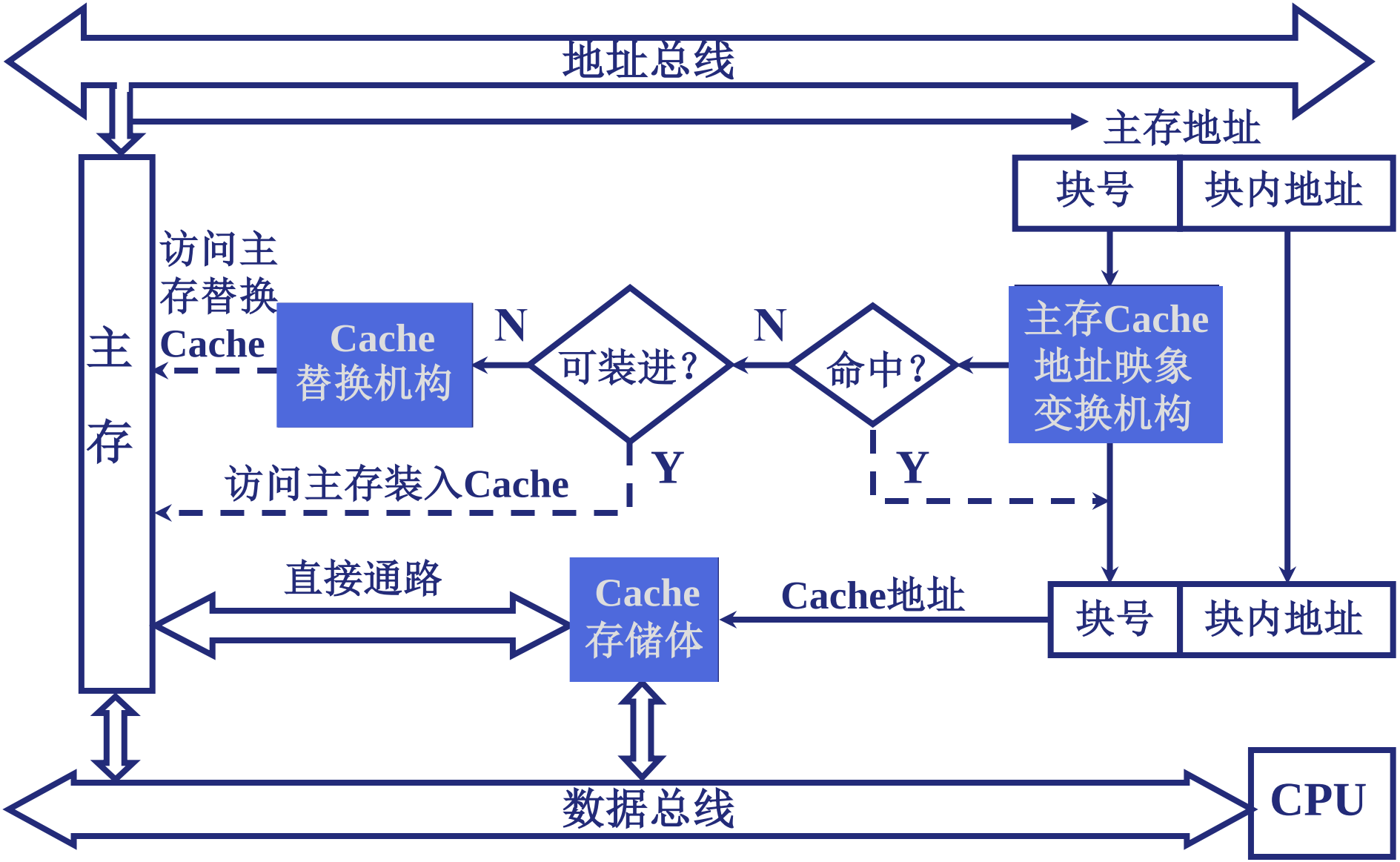
命中率和Cache容量的关系



块（Line，Block）大小和缺失率的关系

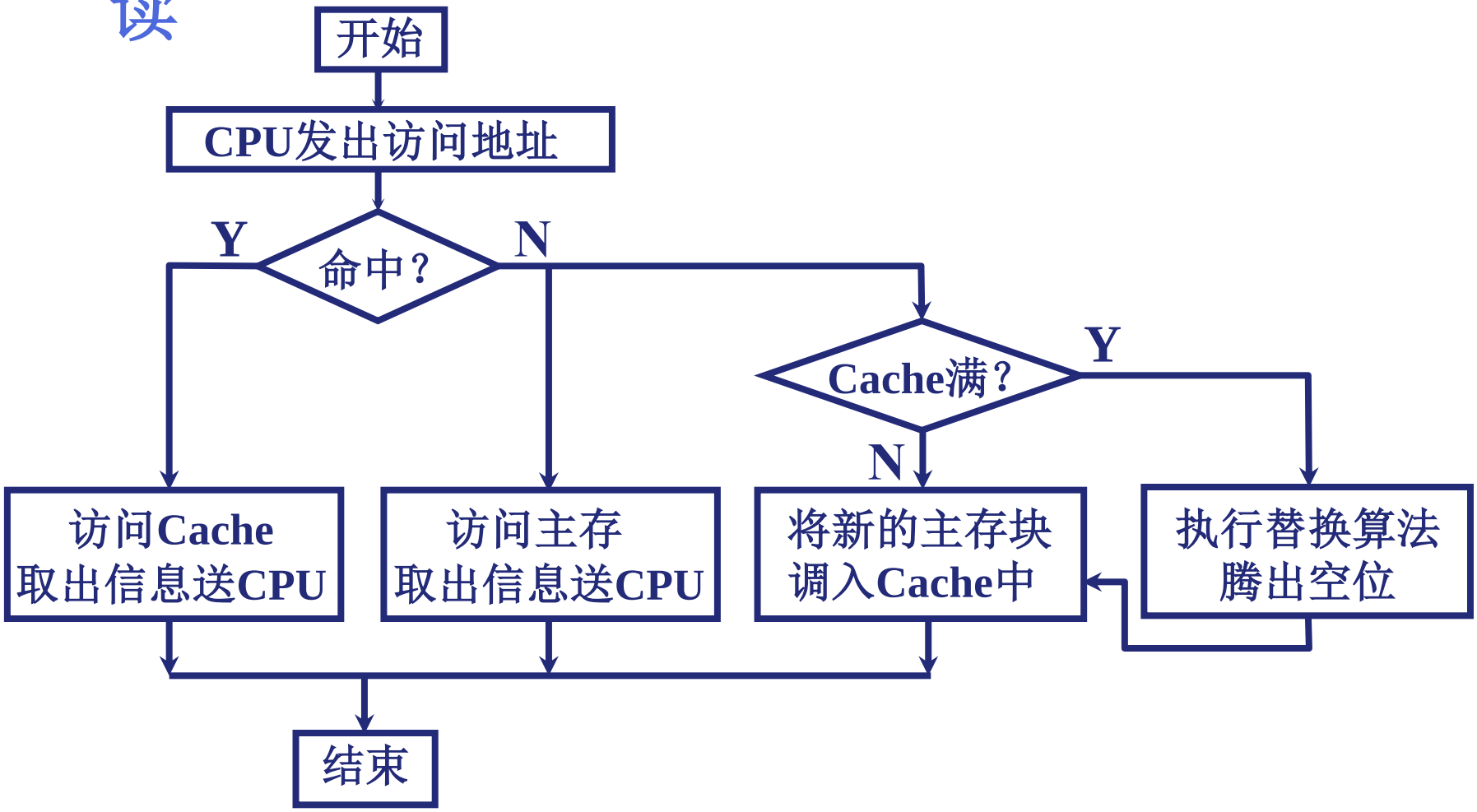


3. Cache 的基本结构



4. Cache 的 读写 操作

读



写　Cache 和主存的一致性

4. Cache 的 写 操作（一致性保证） 4.3

- 对Cache块内写入的信息，必须与被映象的主存块内的信息完全一致。
  - **写直达法/写穿**（write-through，或称store-through）：

    写操作时数据既写入Cache又写入主存
    - 但有可能会增加访存次数。
  - **写回法/拖后写**（write-back）又称标志交换式（Flag-Swap）：

    写操作时只把数据写入 Cache 而不写入主存

    当 Cache 数据被替换出去时才写回主存
    - 这种方法，其速度快，但因主存中的字块未经随时修改，可能失效。

外设改动 同时 Cache 改动

5. Cache 的改进

(1) 增加 Cache 的级数

片载（片内）Cache

片外 Cache

(2) 统一缓存和分开缓存

指令 Cache　　数据 Cache

与主存结构有关

与指令执行的控制方式有关　　是否流水

Pentium	8K 指令 Cache	8K 数据 Cache
PowerPC620	32K 指令 Cache	32K 数据 Cache

二、Cache 映射(Cache Mapping)

- 如何进行映射/映象
  - 把主存空间划分成大小相等的主存块（Block）
  - Cache中存放一个主存块的对应单位称为槽（Slot）或行（Line）或块（Block）
  - 将主存块和Cache行按照以下三种方式进行映射
    - 直接映射(Direct)：每个主存块映射到Cache的固定块中
    - 全相联映射(Full Associate)：每个主存块映射到Cache的任意块中
    - 组相联映射(Set Associate)：每个主存块映射到Cache的固定组中的任意一块中

1. 直接映射（direct mapped）

- 直接映射：把主存的每一块映射到Cache中的一个固定的Cache块的方式。
- 最简单的地址映射方式，地址变换速度快，但块冲突的概率较高，当程序往返访问两个相互冲突的块中的数据时，Cache的命中率将急剧下降。
- 映射公式：

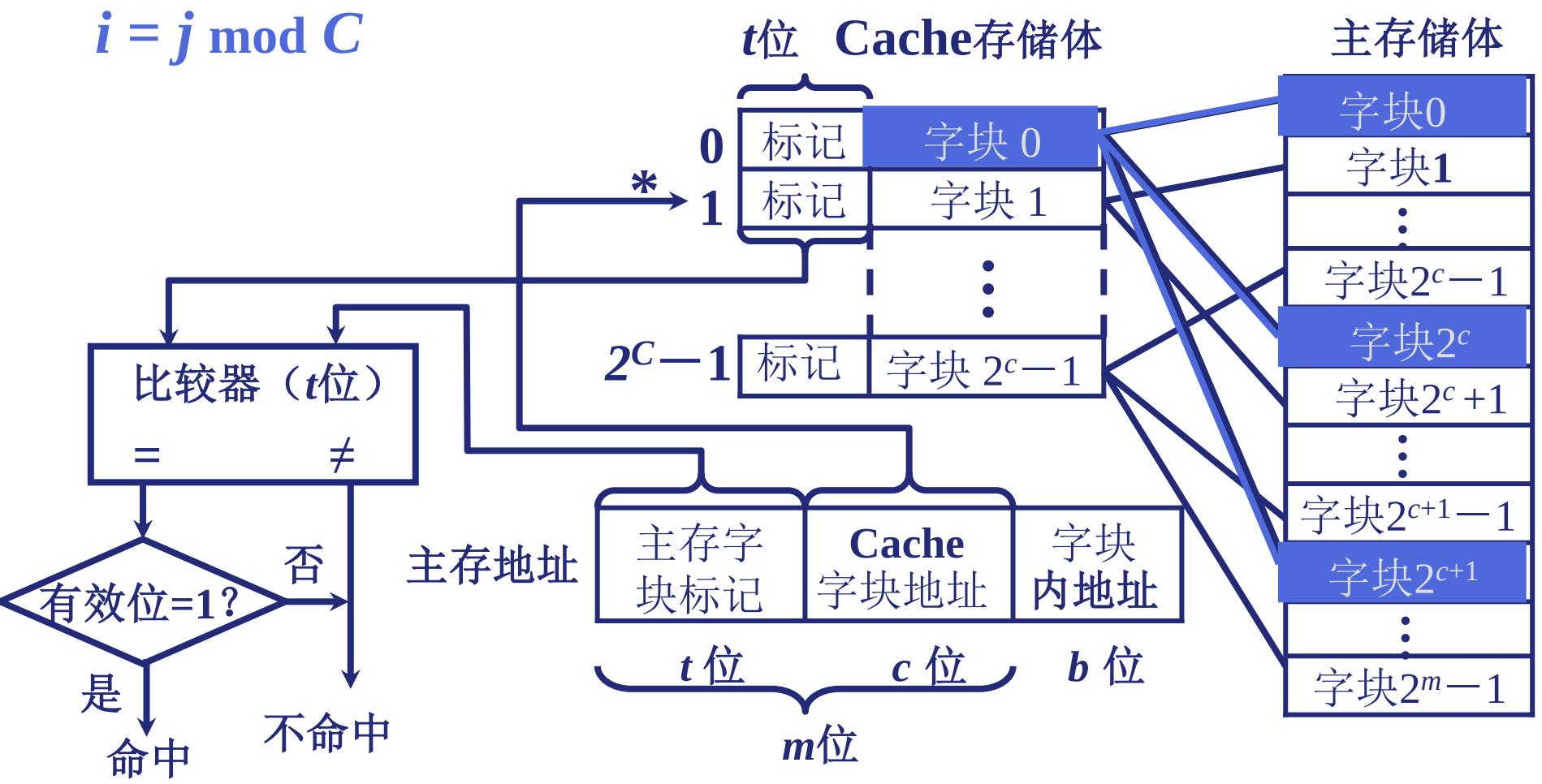
Cache块号=主存块号 mod Cache块数

举例：4=100 mod 16 （假定Cache共有16块）

(说明：主存第100块应映射到Cache的第4块中。)

块（行）都从0开始编号 无特殊说明

1. 直接映射



每个缓存块 i 可以和 若干 个 主存块 对应

每个主存块 j 只能和 一 个 缓存块 对应

1. 直接映像

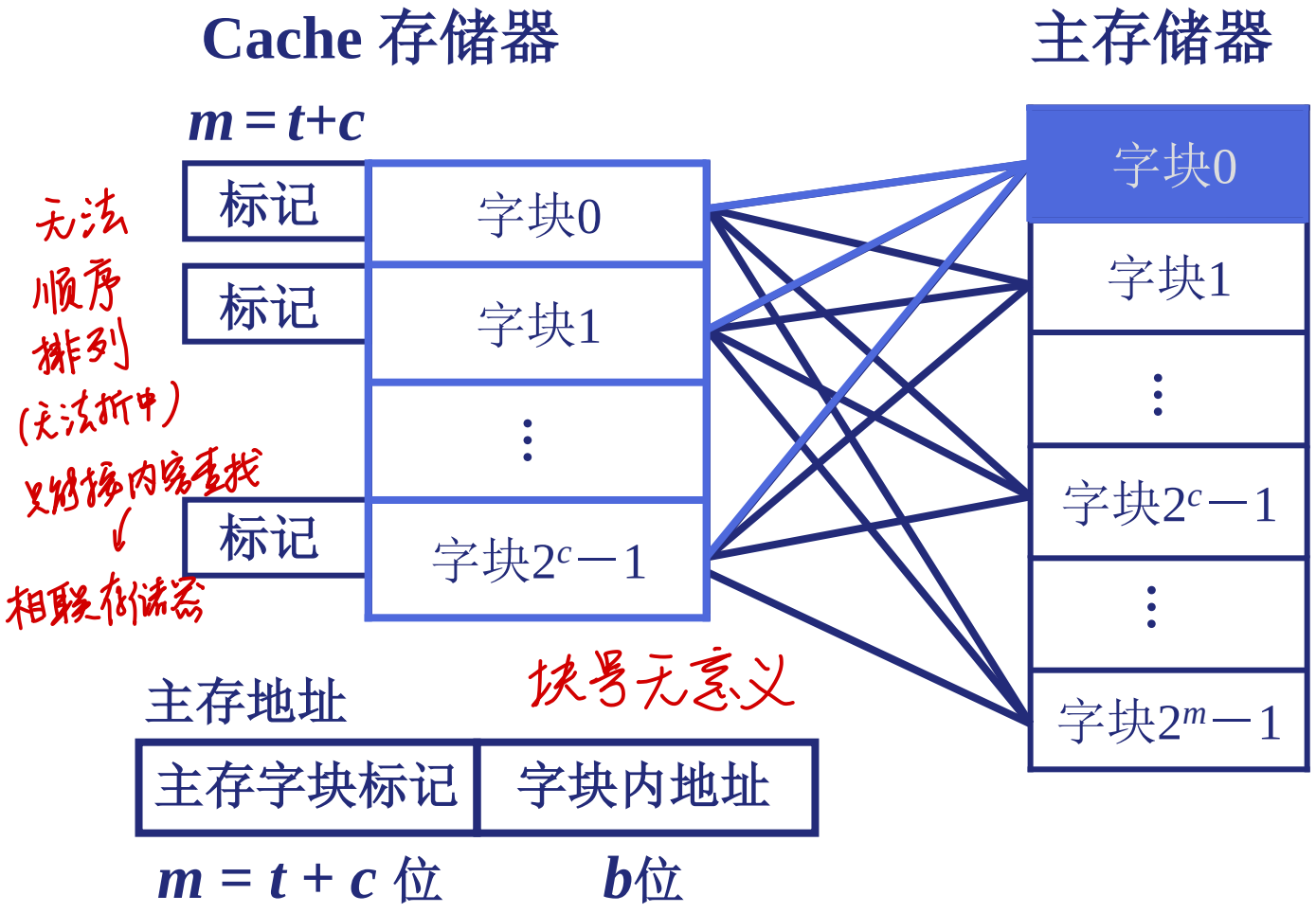
- 特点：
  - 容易实现，命中时间短
  - 无需考虑淘汰（替换）问题
  - 但不够灵活，Cache存储空间得不到充分利用，命中率低
  - 例如，需将主存第0块与第16块同时复制到Cache中时，由于它们都只能复制到Cache第0行，即使Cache其它块空闲，也有一个主存块不能写入Cache。这样就会产生频繁的 Cache装入。

2. 全相联映射（fully associative）

最复杂

- 每个主存块都可映射到任何Cache块的地址映射方式。
- 在全相联映射方式下，主存中存储块的数据可调入Cache中的任意块。
- 命中率实现比较复杂。

2. 全相联映射

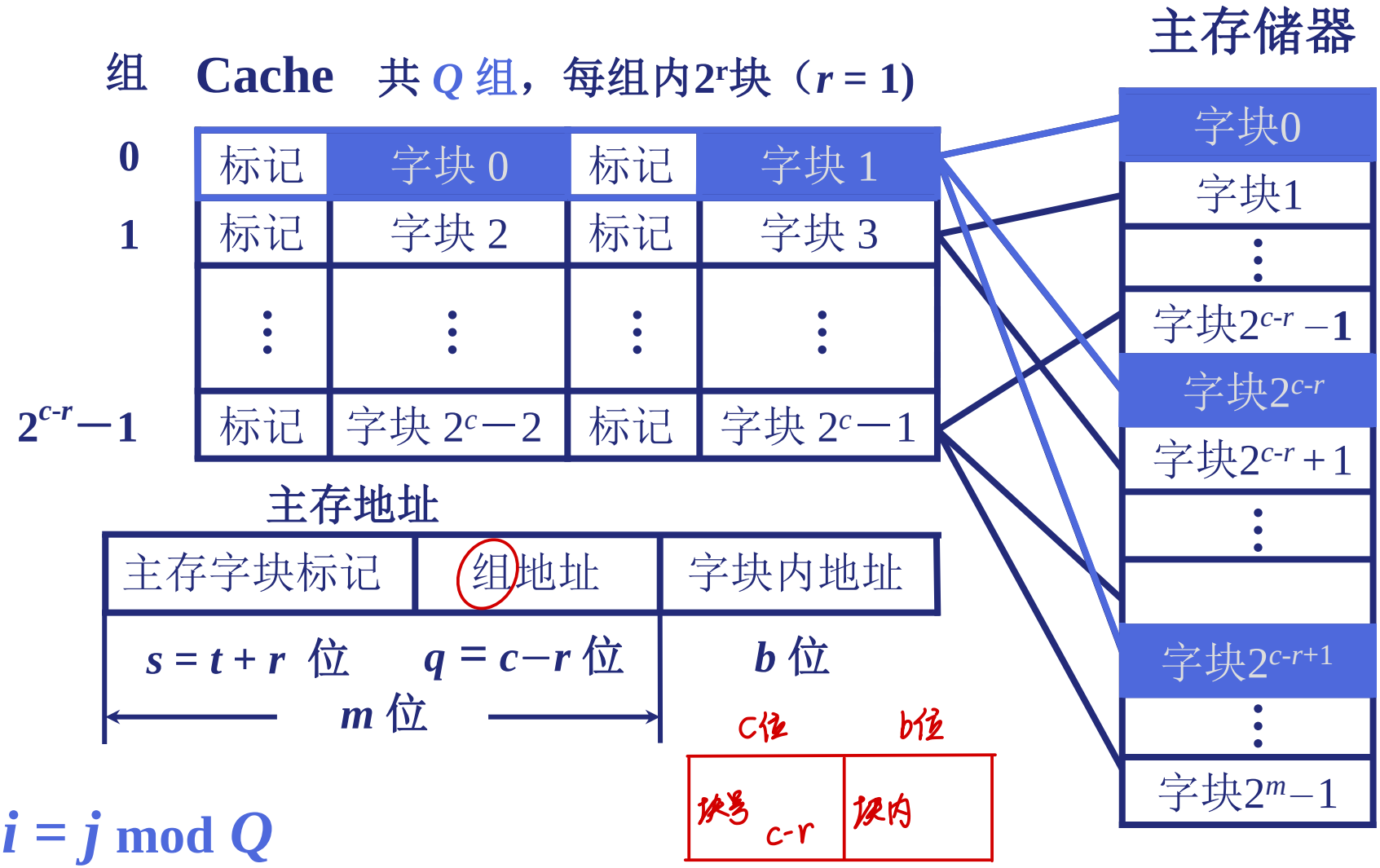


主存 中的 任一块 可以映射到 缓存 中的 任一块

3、组相联映射（Set Associative）

- 特点：
  - 结合直接映射和全相联映射的优点。当Cache组数为1时，变为全相联映射；当每组只有一个槽时，变为直接映射。
  - 每组2或4块（称为2-路或4-路组相联）较常用。通常每组4块以上很少用。在较大容量的L2 Cahce和L3 Cahce中使用4-路以上。

3. 组相联映像（set associative）



某一主存块 *j* 按模 *Q* 映射到 缓存 的第 *i* 组中的 任一块

书上例题

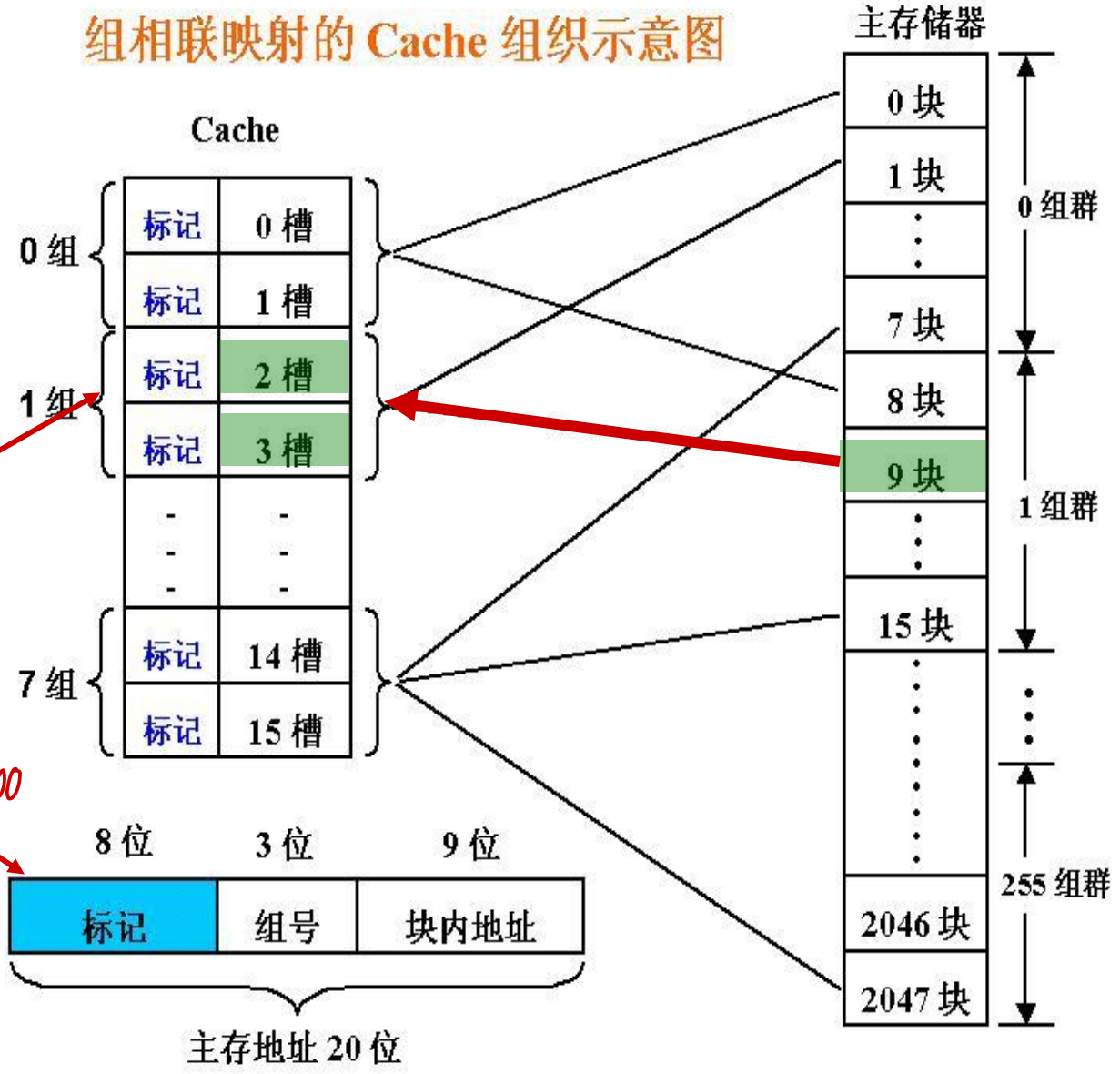
- 例4.8 教材p120
- 例4.9 教材p120
- 例4.10 教材p121

4.11 p122

假定数据在主存和Cache间的传送单位为512字节。

Cache大小：2^{13}字节=8K字节=16行 x 512字节/ 块

主存大小：2^{20}字节=1024K字节=2048块 x 512字节/ 块



指出对应块取自哪个主存组群

指出对应地址位于哪个主存组群中

0000 0001 0010 0000 1100

例：如何对0120CH单元进行访问？

0000 0001 0010 0000 1100B是第1组群中的001块（即第9块）中第12个单元。

所以，映射到第一组中。

将主存地址标记和对应Cache组中每个Cache标记进行比较！

主存地址

直接映射：

标记	Cache块号	块内地址

全相联映射：

标记（主存块号）	块内地址

组相联映射：

标记	Cache组号	块内地址

一个Cache块的内容

有效位	标记	数据

直接映射到全相联（实际就是组相联）

One-way set associative
(direct mapped)

Block	Tag	Data
0		
1		
2		
3		
4		
5		
6		
7		

Two-way set associative

Set	Tag	Data	Tag	Data
0				
1				
2				
3				

Four-way set associative

Set	Tag	Data	Tag	Data	Tag	Data	Tag	Data
0								
1								

Eight-way set associative (fully associative)

Tag	Data	Tag	Data	Tag	Data	Tag	Data	Tag	Data	Tag	Data	Tag	Data	Tag	Data

三、替换算法

4.3

- 常用替换算法有：
  - 先进先出FIFO （first-in-first-out）
  - 最近最少使用LRU （ least-recently used）
  - 最不经常使用LFU （ least-frequently used）
  - 随机替换算法（Random）

  等等

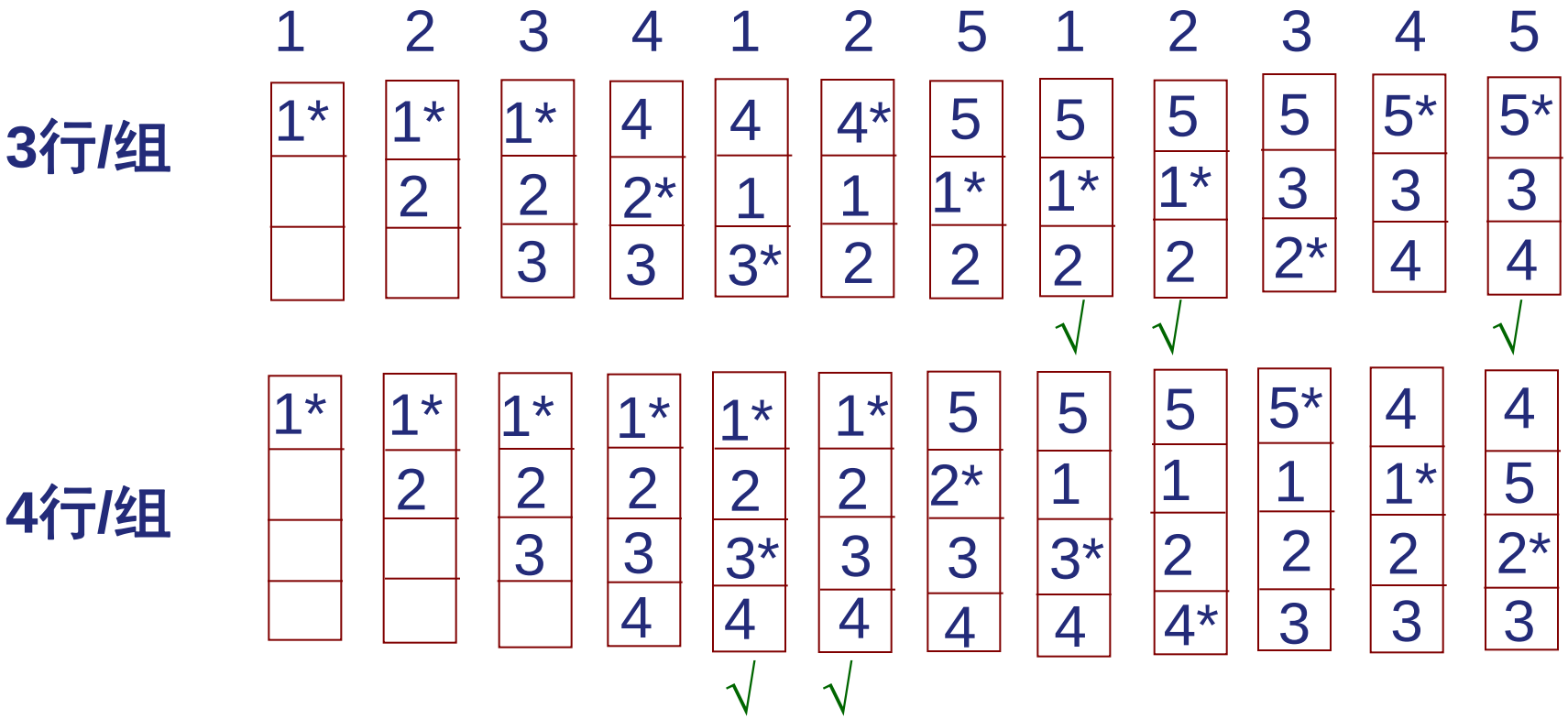
- 选择替换算法的依据是存储器总体的性能，主要是Cache的访问命中率。

替换算法-先进先出（FIFO）

- 总是把最先进入的那一块淘汰掉。

不是越大越高

例：假定主存中的5块{1,2,3,4,5}同时映射到Cache同一组中，对于同一地址流，考察3行/组、 4行/组的情况。



由此可见，FIFO不是一种堆栈算法，即命中率并不随组的增大而提高。

替换算法-最近最少用(LRU)

- 总是把最近最少用的那一块淘汰掉。

例：假定主存中的5块{1,2,3,4,5}同时映射到Cache同一组中，对于同一地址流，考察3行/组、 4行/组、 5行/组的情况。

	1	2	3	4	1	2	5	1	2	3	4	5
	1	2	3	4	1	2	5	1	2	3	4	5
		1	2	3	4	1	2	5	1	2	3	4
			1	2	3	4	1	2	5	1	2	3
				1	2	3	4	4	4	5	1	2
							3	3	3	4	5	1
3行/组							√	√				
4行/组					√	√	√	√				
5行/组					√	√	√	√	√	√	√	

4.4 辅助存储器

一、概述

二、磁记录原理

三、硬磁盘存储器

四、软磁盘存储器

五、光盘存储器

4.4 辅助存储器

一、概述

了解概念

机械硬盘比固态硬盘可靠

磁带：①读取慢 ②顺序存储 容错✓

1. 特点：不直接与 CPU 交换信息

2. 磁表面存储器的技术指标 p124

(1) 记录密度 道密度 D_t 位密度 D_b

磁道 里越大 道上长度 外疏

(2) 存储容量 $C = n \times k \times s$

(3) 平均寻址时间 寻道时间 + 等待时间

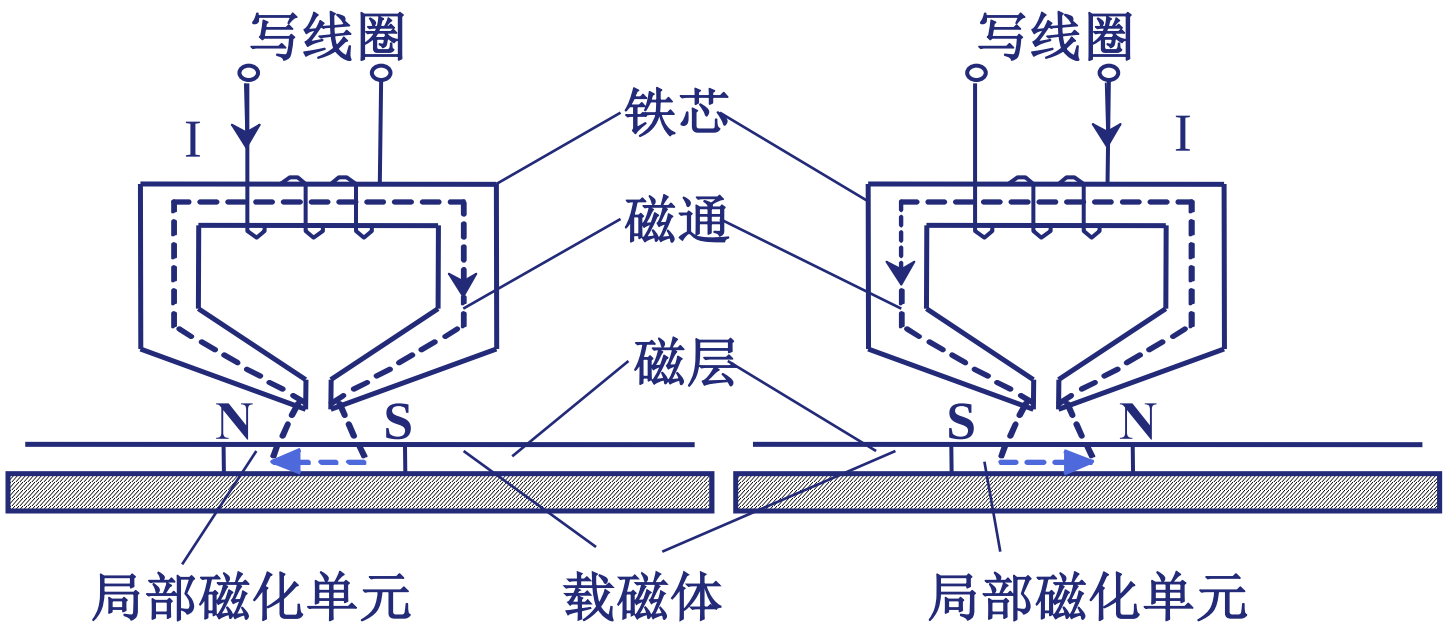
辅存的速度 { 寻址时间 / 磁头读写时间 }

(4) 数据传输率 与位密度有关

(5) 误码率 出错信息位数与读出信息的总位数之比

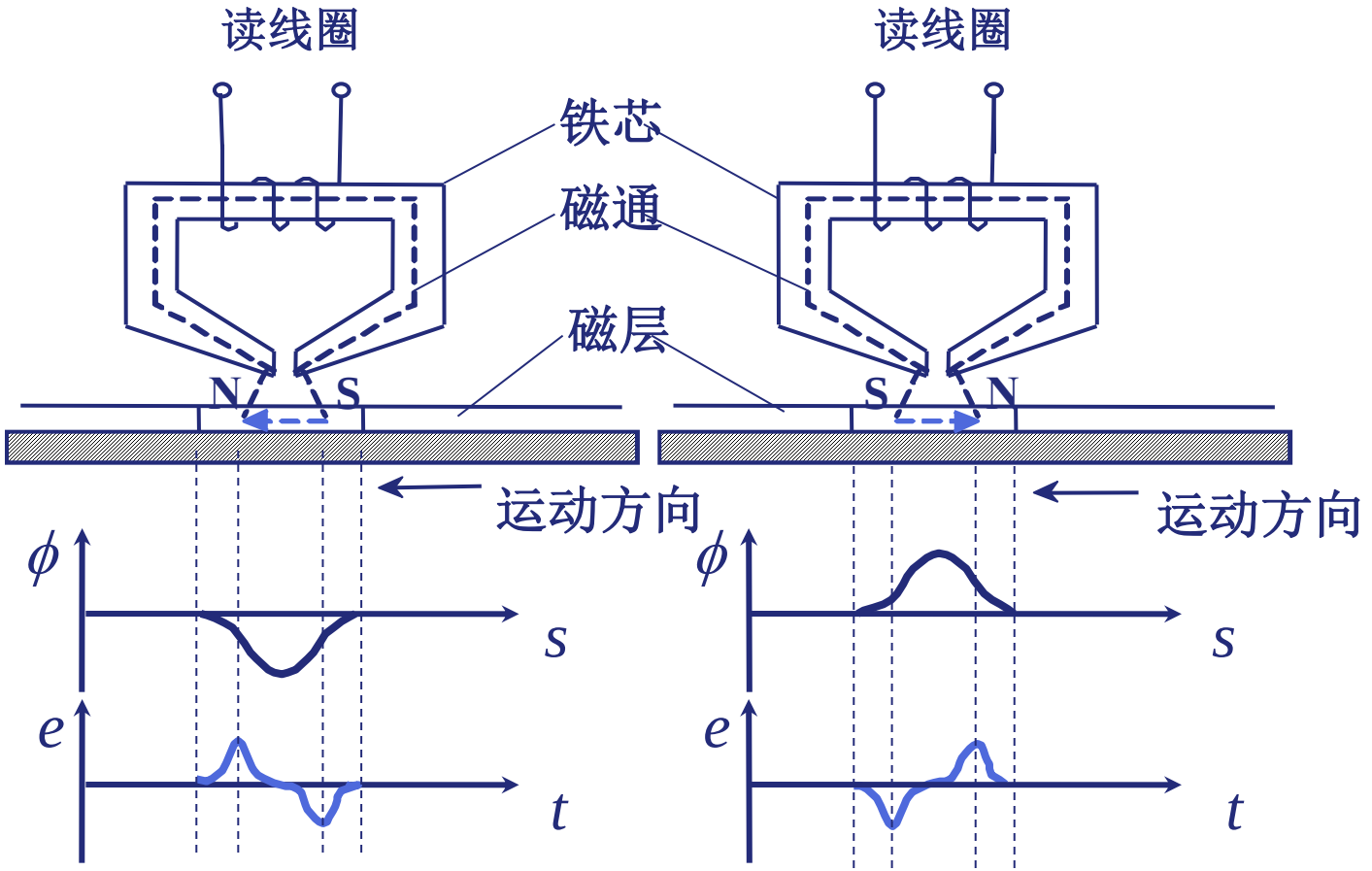
1. 磁记录原理

写



写入“0”　　写入“1”

读



读出“0”　　读出“1”

2. 磁记录方式 p127

(1) 归零制（RZ）

(2) 不归零制（NRZ）

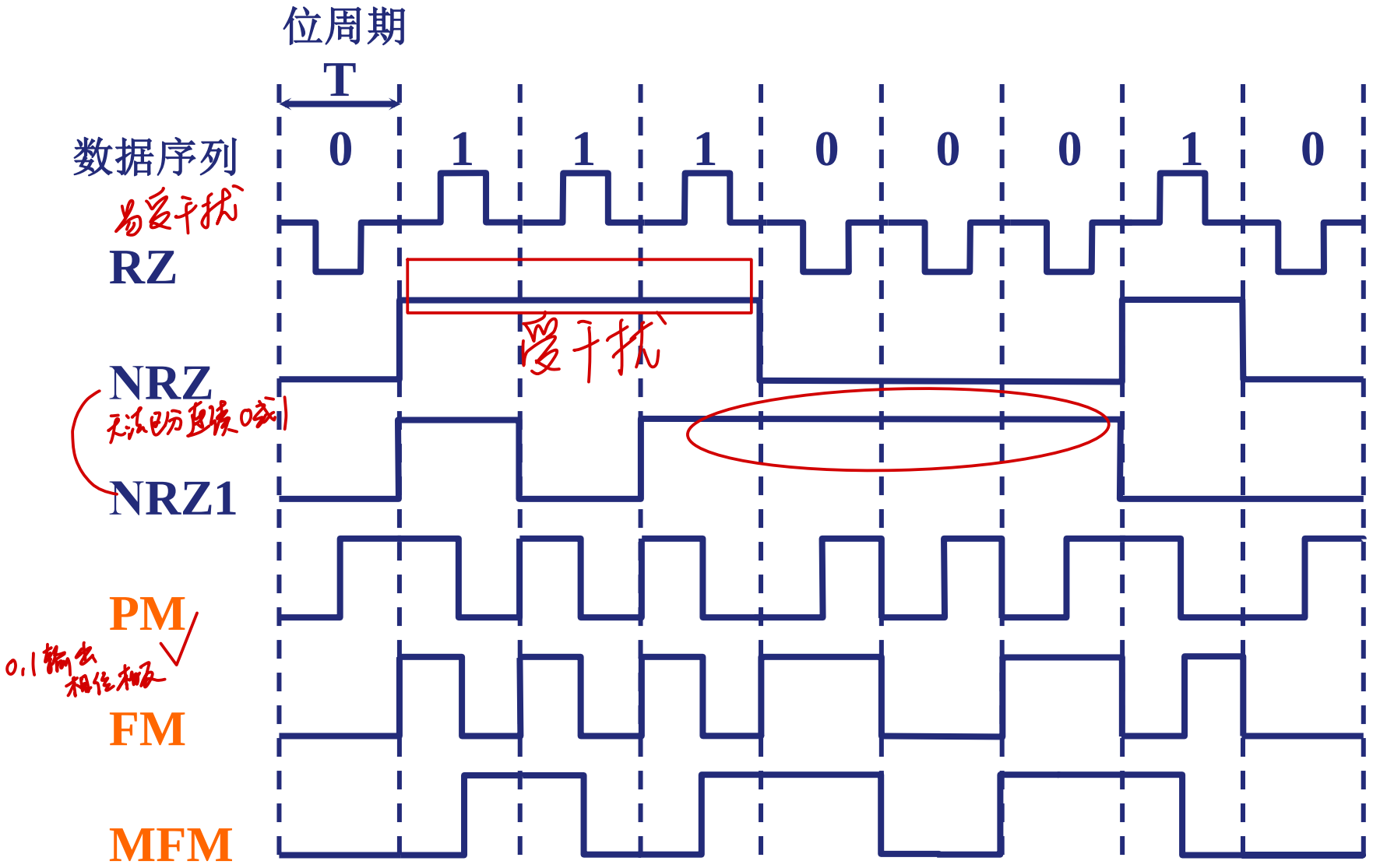
(3)“见1就翻” 的不归零制（NRZ1）

(4) 调相制（PM）

(5) 调频制（FM）

(6) 改进型 调频制（MFM）

二、磁记录原理和记录方式



三、硬磁盘存储器

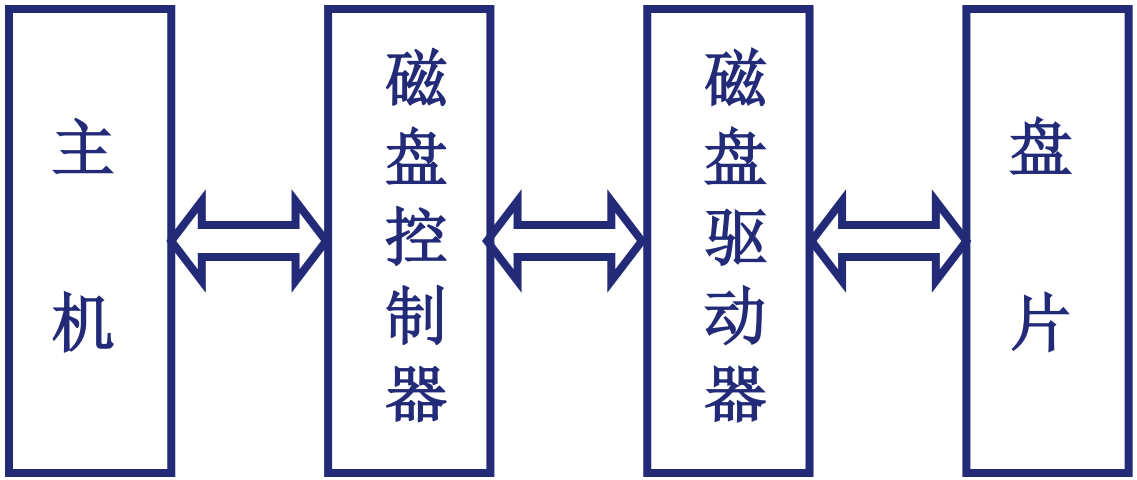
4.4

1. 硬磁盘存储器的类型

(1) 固定磁头和移动磁头

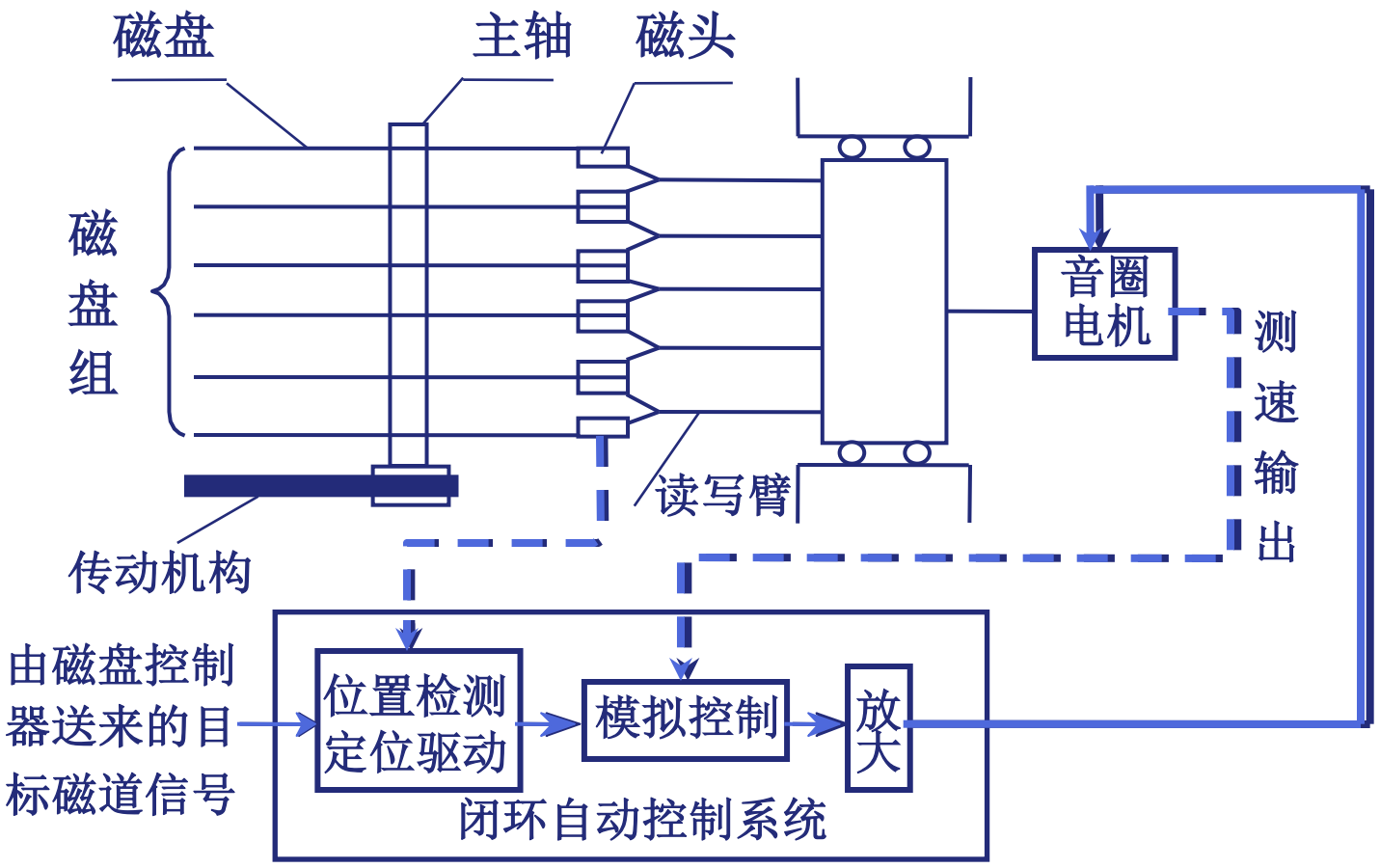
(2) 可换盘和固定盘

2. 硬磁盘存储器结构

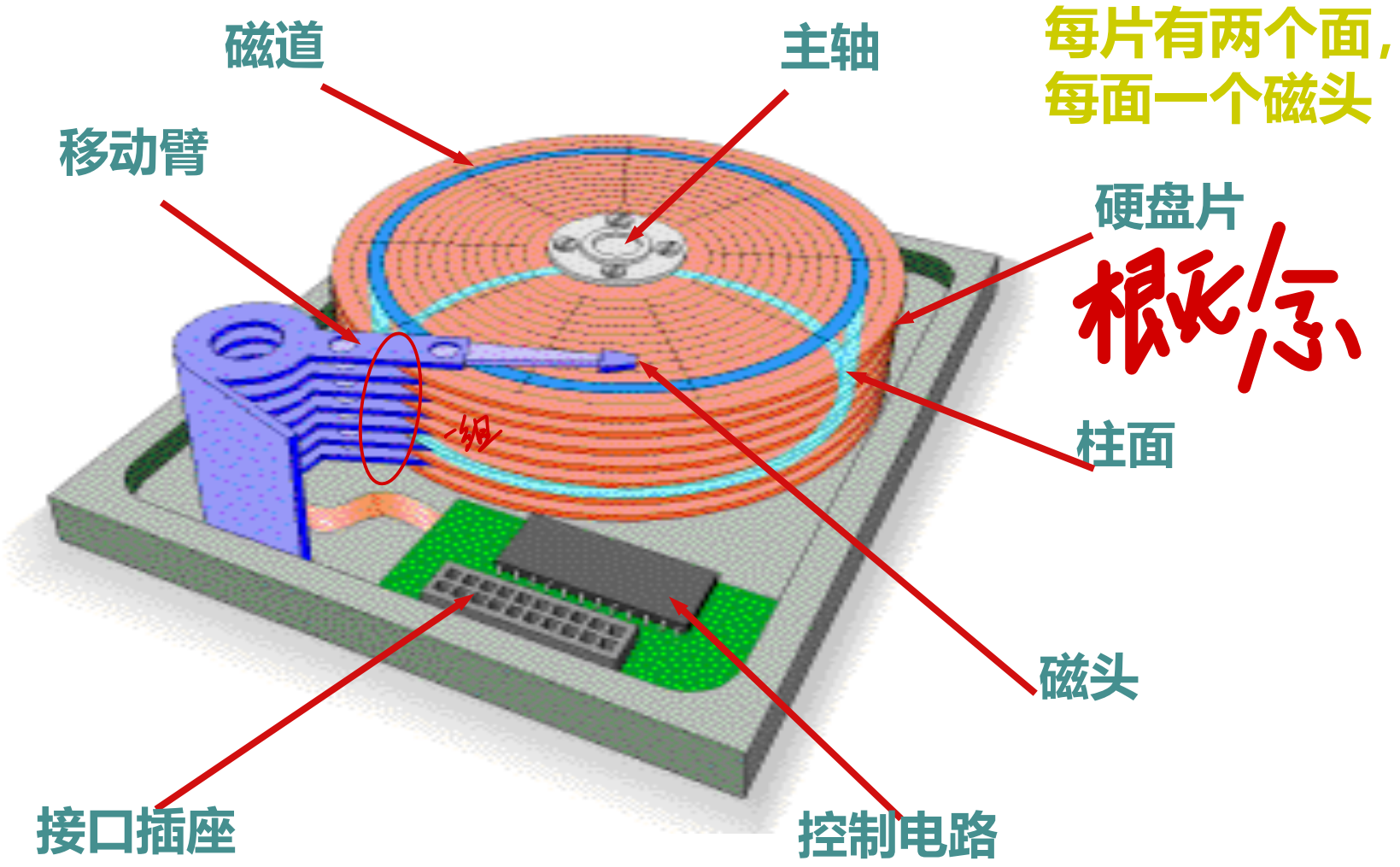


(1) 磁盘驱动器

主轴　　定位驱动　　数据控制



磁盘驱动器



磁道号就是柱面号、磁头号就是盘面号

(2) 磁盘控制器

磁盘控制器 是

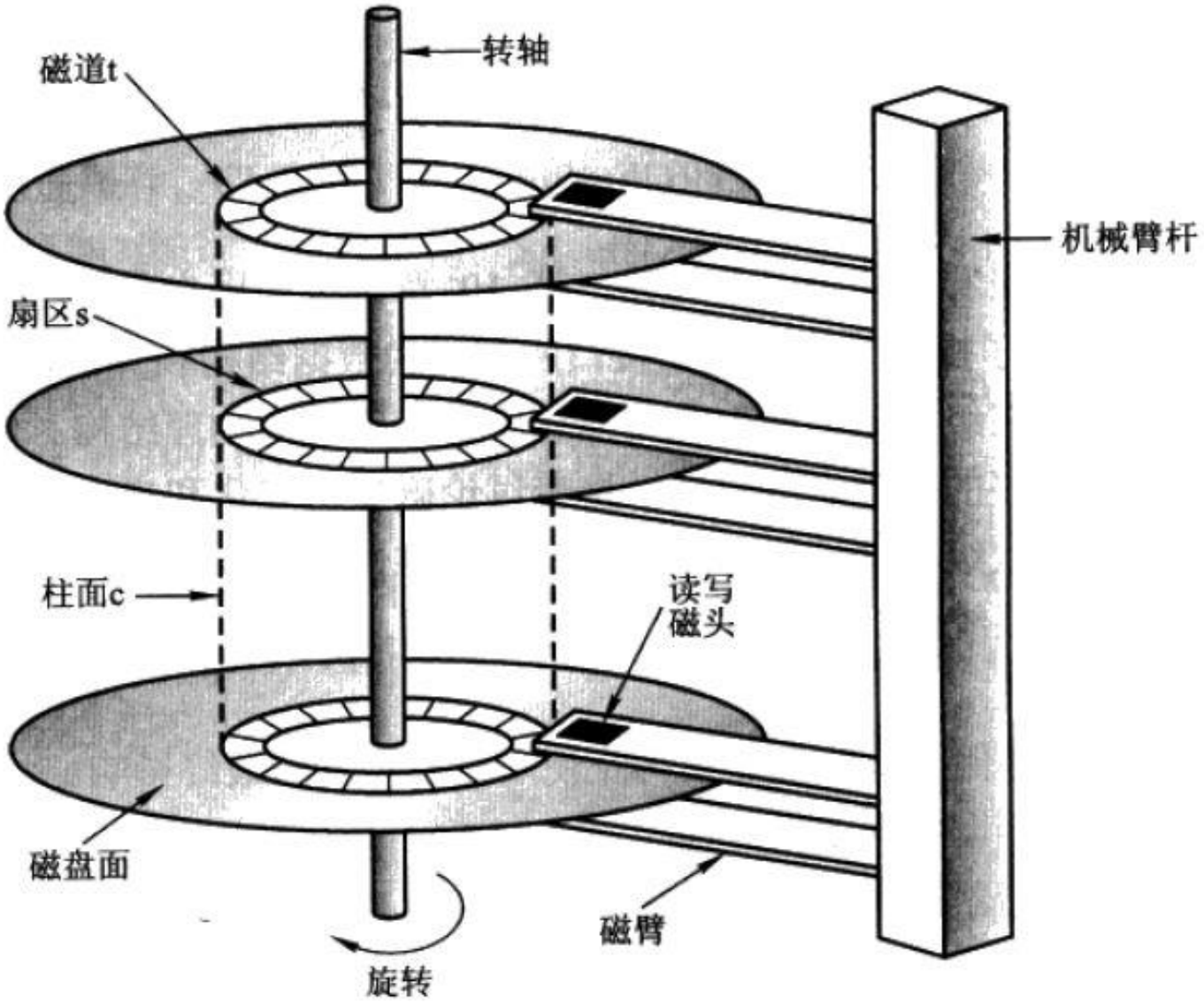
主机与磁盘驱动器之间的 接口 { 对主机 通过总线
对硬盘（设备）

- 接受主机发来的命令，转换成磁盘驱动器的控制命令
- 实现主机和驱动器之间的数据格 式转换
- 控制磁盘驱动器读写

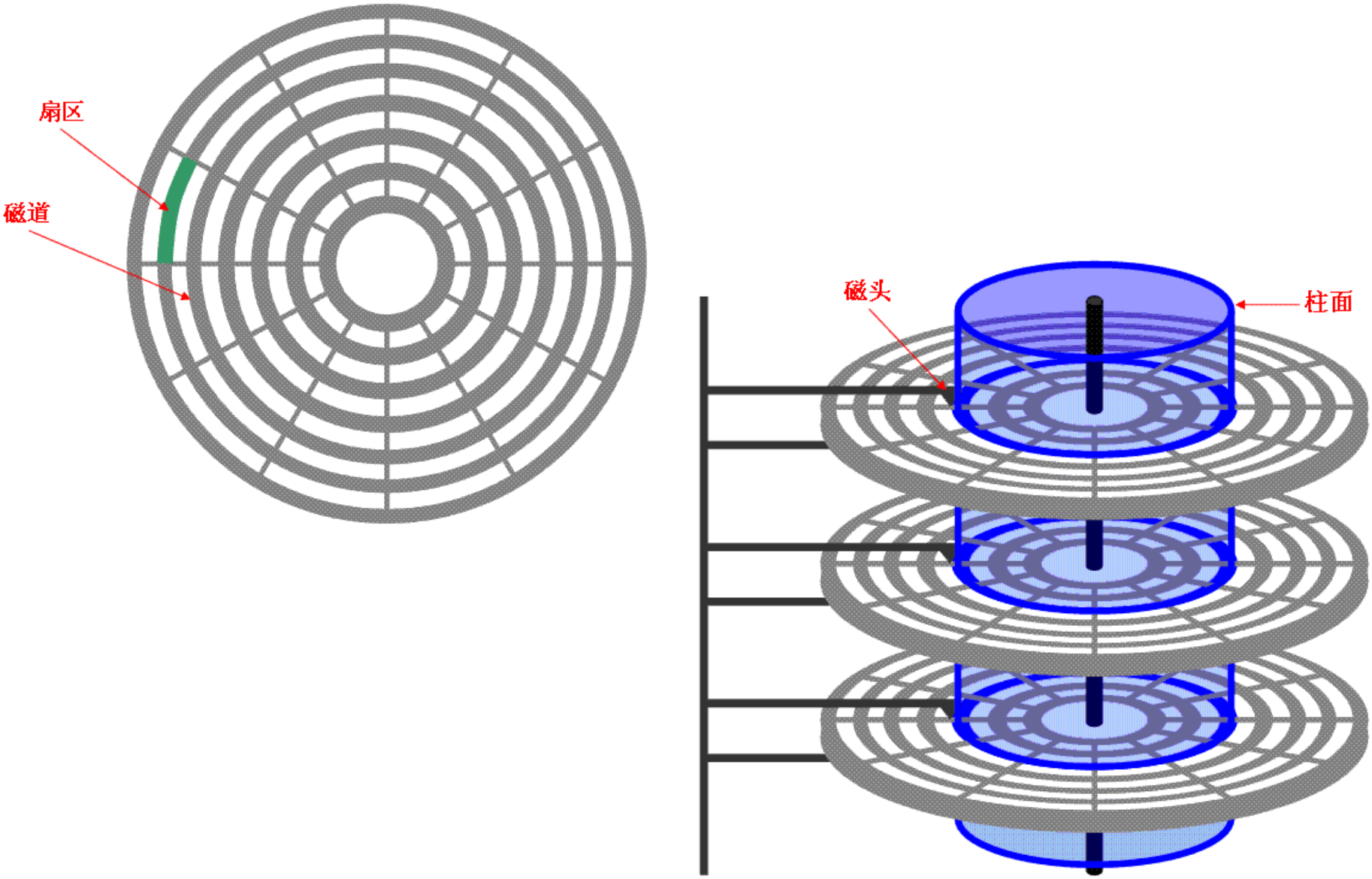
(3) 盘片

- 磁层由非矩形剩磁特性的导磁材料（如氧化铁、镍钴合金）构成。导磁材料制成的磁胶涂敷或镀在载磁体上，厚度通常在0.1~5μm。
- 载磁体可以是金属（铝）合金（硬质载磁体）或者塑料（软质载磁体）

几个概念



几个概念



四、软磁盘存储器

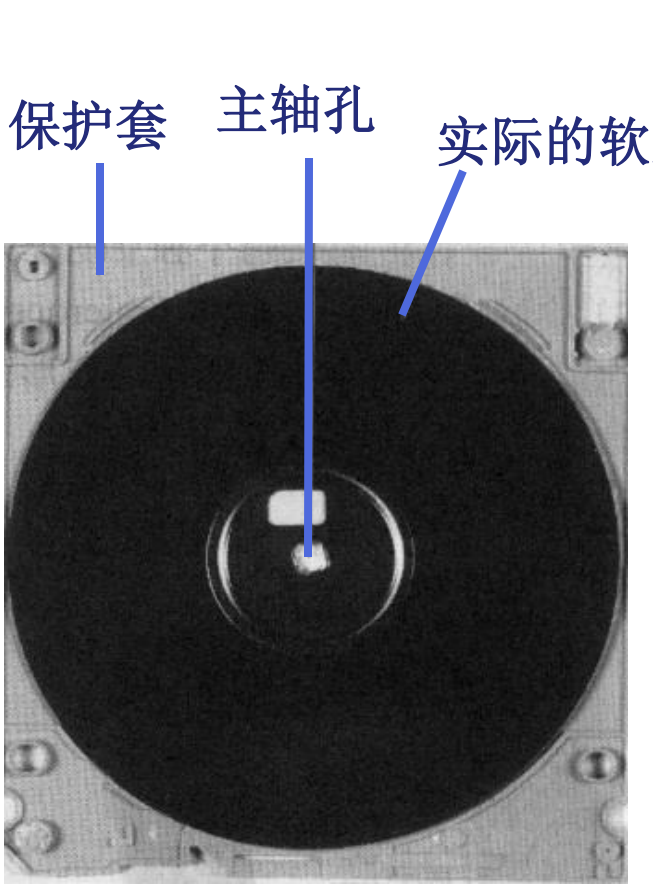
4.4

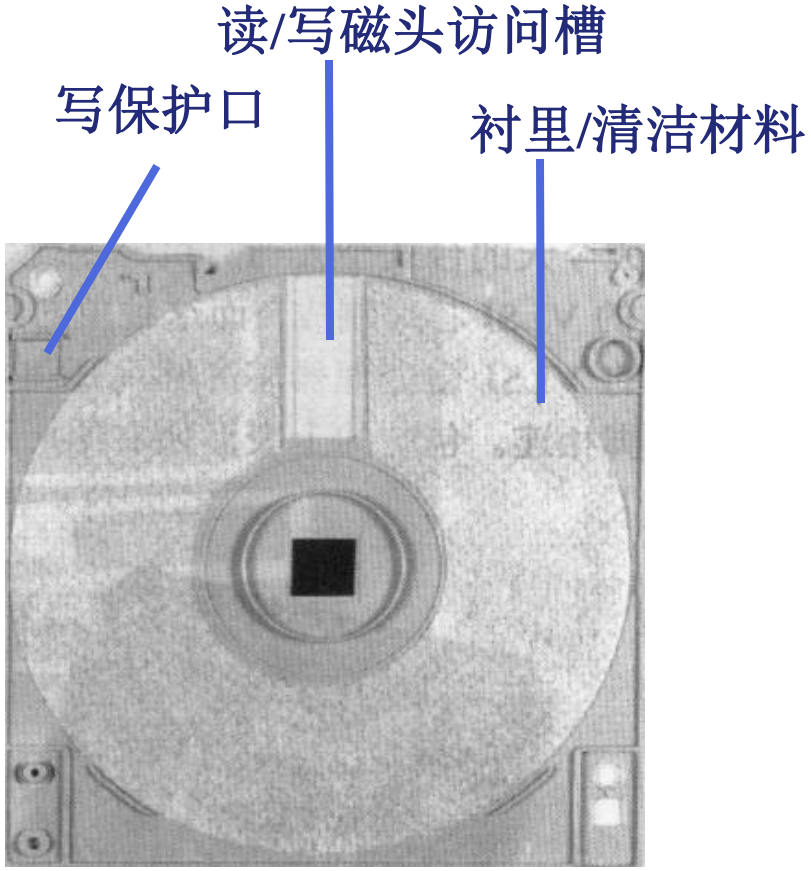
1. 概述

	硬盘	软盘
速度	高	低
磁头	固定、活动	活动
	浮动	接触盘片
盘片	固定盘、盘组 大部分不可换	可换盘片
价格	高	低
环境	苛刻	

2. 软盘片

由聚酯薄膜制成





软盘

容易坏

五、光盘

4.4

1. 概述

采用光存储技术　　利用激光写入和读出

- 第一代光存储技术　　采用非磁性介质　　不可擦写
- 第二代光存储技术　　采用磁性介质　　可擦写

2. 光盘的存储原理

只读型和只写一次型　热作用（物理或化学变化）

可擦写光盘　　热磁效应

业精于勤，荒于嬉

行成于思，毁于随

计算机与信息学院

RAID的提出

磁盘阵列

- CPU性能在过去的十年中有了极大地提高，几乎是每18个月翻一番。但磁盘的性能却没能跟上。在70年代，小型机磁盘的平均查找时间为50到100毫秒，现在是10毫秒。在许多行业（如汽车或航空业），如果性能的提高能达到这个速度，即20年内提高5到10倍，那就会是头条新闻，但对计算机行业，这却成了一个障碍。因为CPU性能和磁盘性能间的差距这些年来越来越大。

RAID的提出

- 在提高CPU性能方面，并行处理技术已得到广泛使用。这些年来，许多人意识到，并行I/O也是一个提高磁盘性能的好办法。1988年，Patterson et al.在他的一篇文章中建议用6个特定的磁盘组织来提高磁盘的性能或可用性，或两方面都同时提高。这个建议很快就被采用，并导致了一种新的I/O设备的诞生，这就是RAID盘。Patterson et al.把RAID定义为廉价磁盘的冗余阵列（Redundant Array of Inexpensive Disks），但工业界把“I”由“廉价的（Inexpensive）”替换成“独立的（Independent）”。

RAID

- **RAID定义**
  - 廉价磁盘的冗余阵列（Redundant Arrays of Inexpensive Disks）
  - 用N 个低价磁盘构成一个统一管理的阵列，以取代特贵单一磁盘
- **RAID目标**
  - N个磁盘的容量
  - 1/N的访问时间
  - 更高的性价比
  - 采用冗余技术提高存储信息的可用性

RAID0：DataStriping

RAID1：DriveMirroring

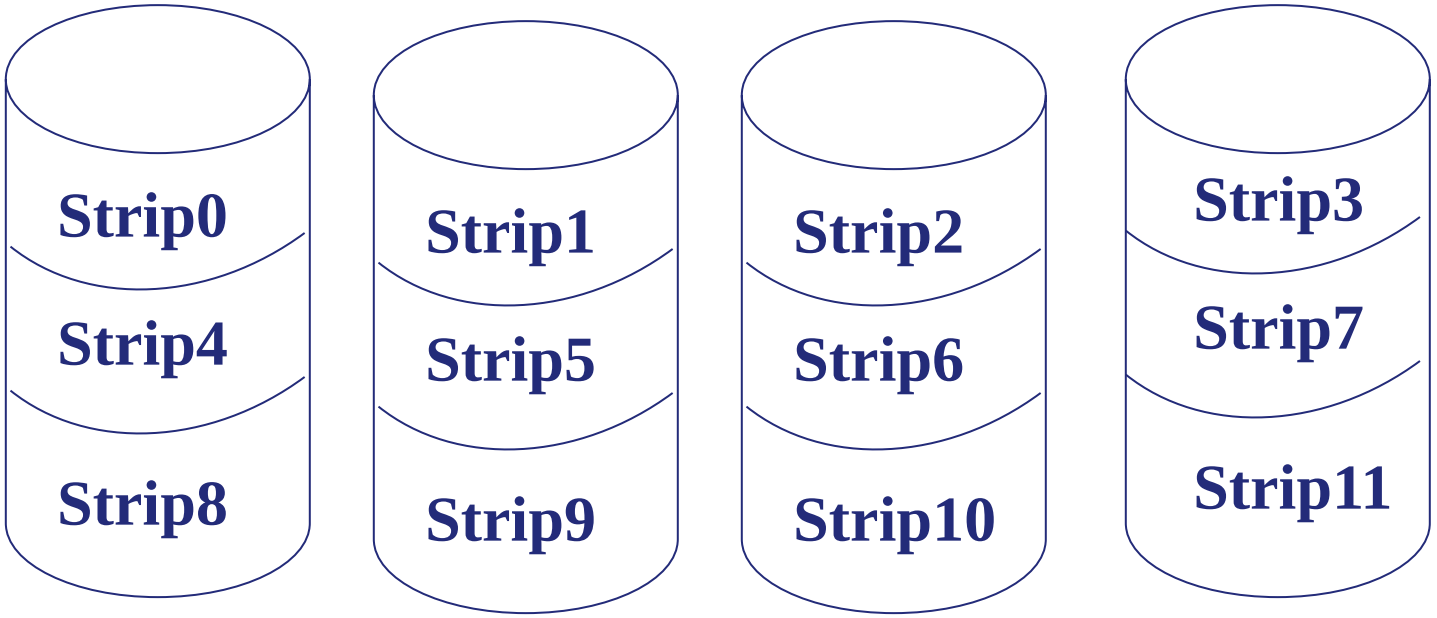
RAID4：Data Guarding

RAID5：Distributed Data Guarding

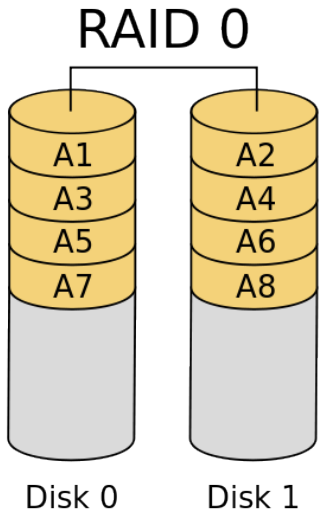
RAID0

- RAID0将由RAID模拟的单个虚拟磁盘划分成带（strip），每带k个扇区。第0带为第0到第$k-1$扇区，第1带为第k扇区到第$2k-1$扇区，等等。对k=1，每个带为1个扇区；对k=2，每带有2个扇区；等等。RAID 0以交叉循环的方式将数据写到连续的带中，下图描述的就是有4个磁盘驱动器的RAID盘。这种在多个驱动器上分布数据的方式叫作分带。如果软件发出从带的边界开始读四个连续带的数据块的命令，RAID控制器将把这个命令分解成四个单独的读命令，四个驱动器每个一个，让它们并行执行。这样，就实现了对软件透明的并行I/O操作。

RAID0



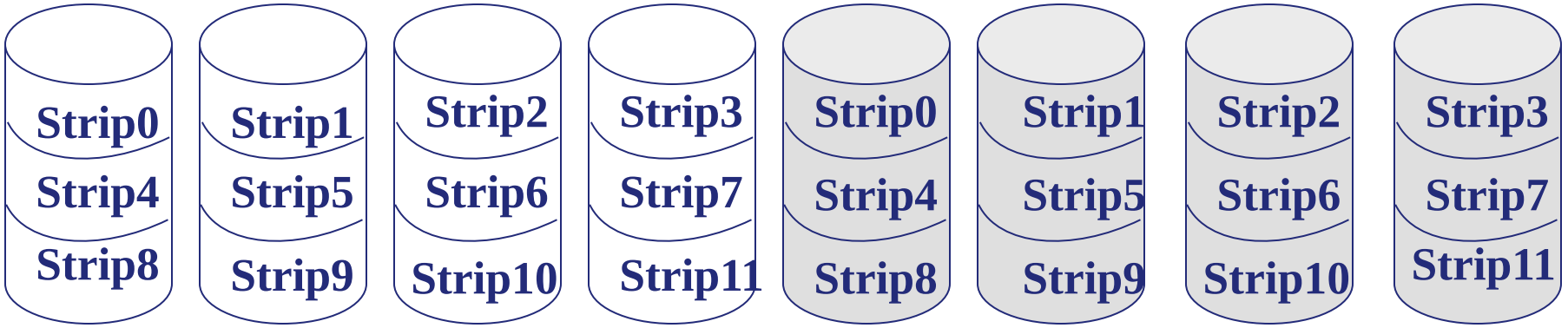
1. 适合数据请求量比较大的情况
2. 没有冗余，可靠性差，不算真正的RAID



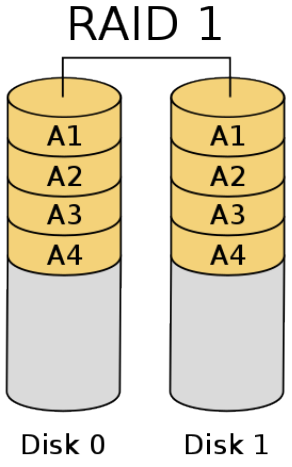
RAID1

- 它复制了所有的磁盘，所以有四块主磁盘和四块辅助磁盘。每个对磁盘的写操作都进行两次，而每次读操作则可以读任意一个备份，把负载均衡分布到不同的驱动器上。这样，写操作的性能并不比单个磁盘好，但读磁盘的性能却比单个磁盘高了两倍。容错性能就更好了，如果一个驱动器崩溃的话，只要简单的用备份驱动器代替就行了。恢复整个磁盘的操作包括两个步骤：装上一个新的驱动器，然后将整个备份驱动器的内容拷贝到新的驱动器上。

RAID1



1. 冗余备份，可靠性高
2. 写性能不高，但读性能却提高了两倍
3. 成本较高

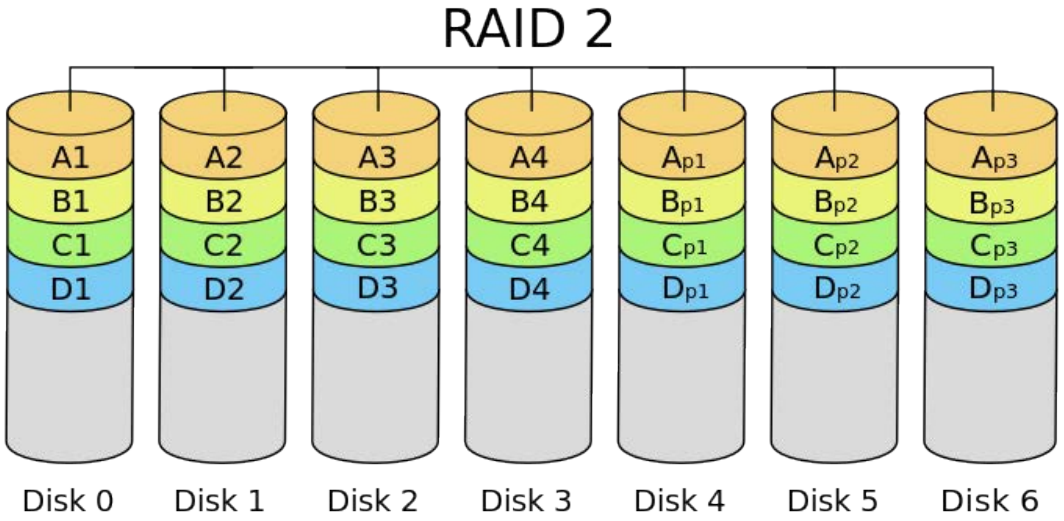


RAID2

- **RAID 2的工作单位为字，可能的话甚至可以是字节。首先我们可以想象将单个虚拟磁盘上的字节分解成一对4位的半字节，对每个半字节加上3位海明码形成7位字，即其中1、2、4位做校验位。然后，用下图所示的七个驱动器的磁头和旋转同步，就可能将整个海明码字写在七个驱动器上，每个驱动器一位。**

RAID2

1. 驱动器必须同步旋转
2. 驱动器个数要足够多
3. 需要多个控制器



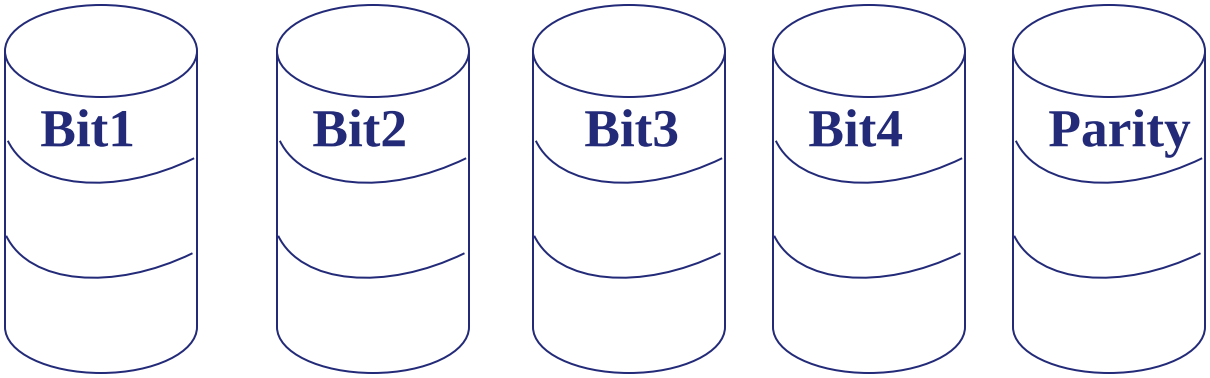
RAID3

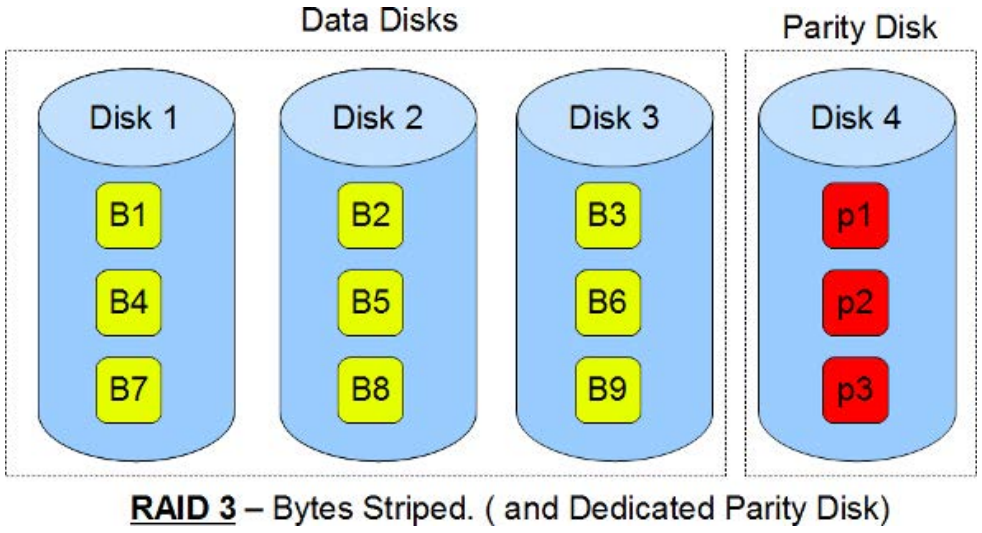
- **RAID 3是RAID 2的一个简化版本，它只需对每个字计算一个校验位，写到一个校验驱动器上。和RAID 2相同，驱动器之间必须严格同步，因为一个字被分布到多个驱动器中。**

RAID3

1. 驱动器之间要严格同步

2. 对整个磁盘崩溃的错误，能够进行恢复





RAID 3 – Bytes Striped. (and Dedicated Parity Disk)

RAID4

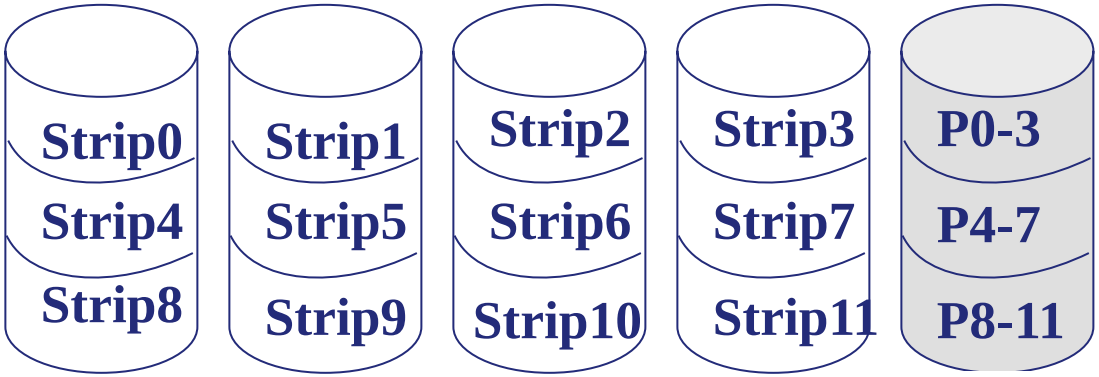
- **RAID 4和RAID 0类似，将对带的校验写在额外的驱动器上。例如，若带的长度是*k*个字节，将所有的带异或到一起，产生一个*k*字节长的校验带。如果其中一块磁盘崩溃的话，它的内容可以从校验磁盘上重新计算出来。**

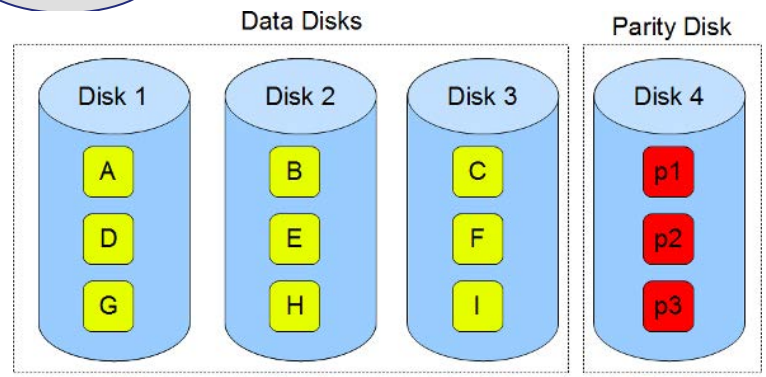
RAID4

1. 不对字进行校验，也不需要驱动器同步

2. 可以防止整块盘崩溃，但对盘上部分字节数据出错的纠错性能相当差

3. 校验盘负载沉重



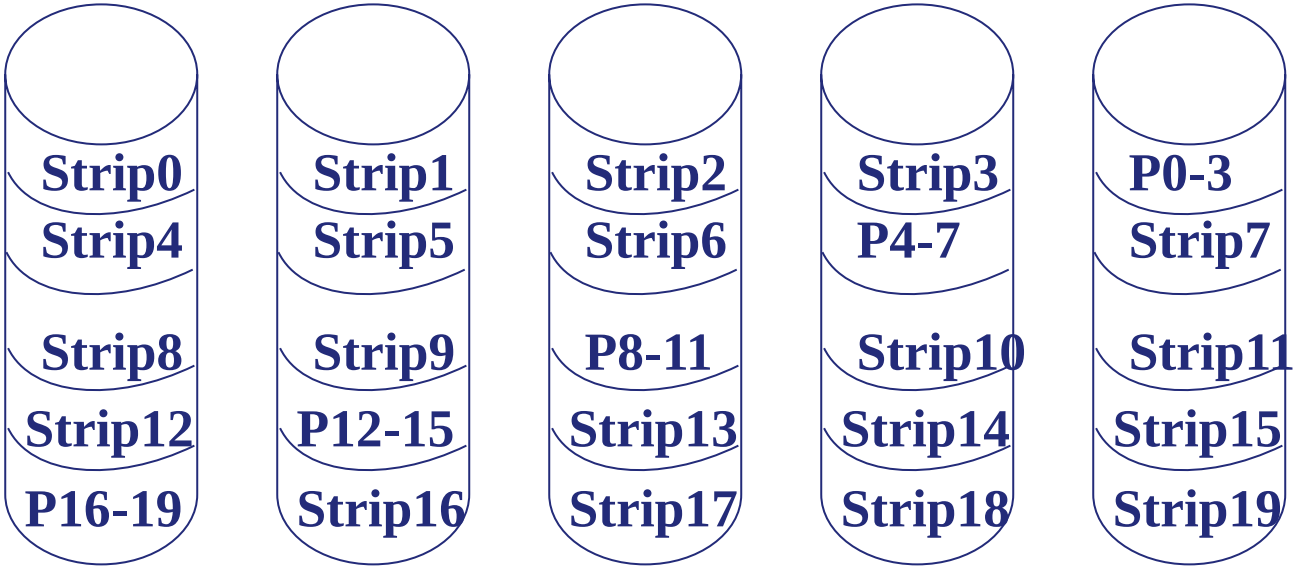


RAID 4 – Blocks Striped. (and Dedicated Parity Disk)

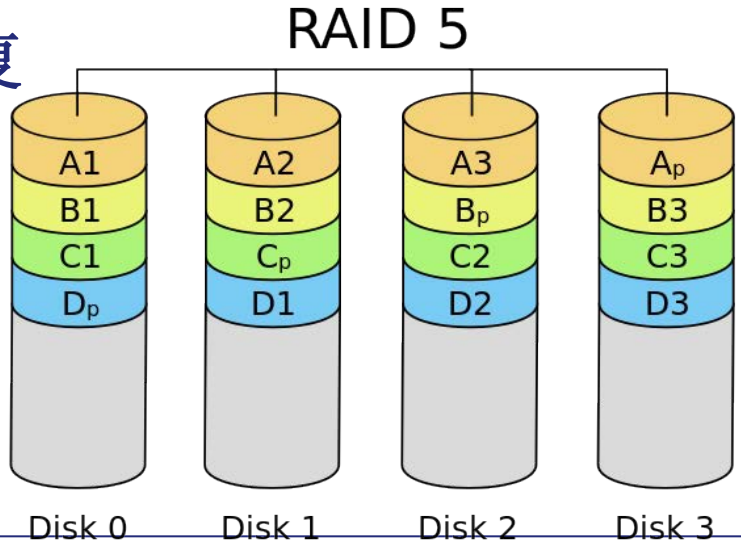
RAID5

- **RAID 5为减少校验盘的负载，将校验位循环均匀分布到所有的驱动器上。**

RAID5

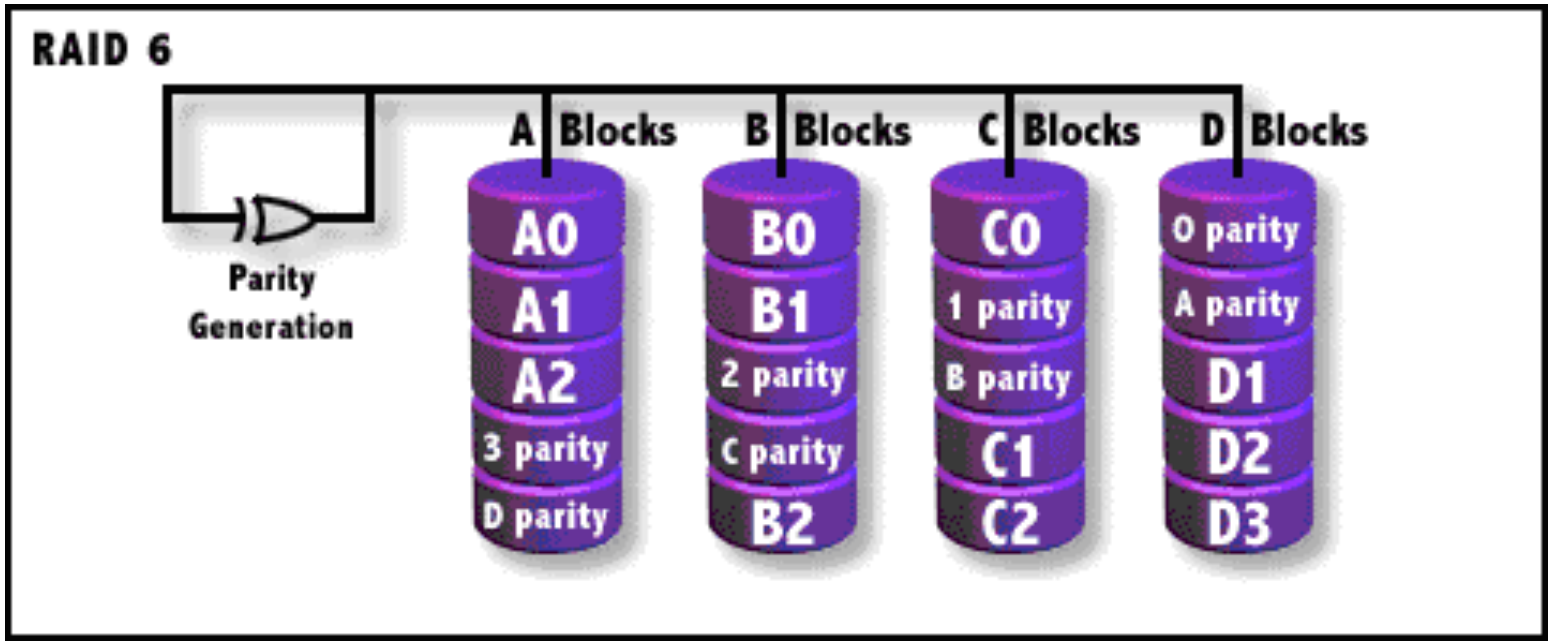


1.如果RAID 5的磁盘崩溃的话，修复磁盘内容的将是一个复杂的过程。

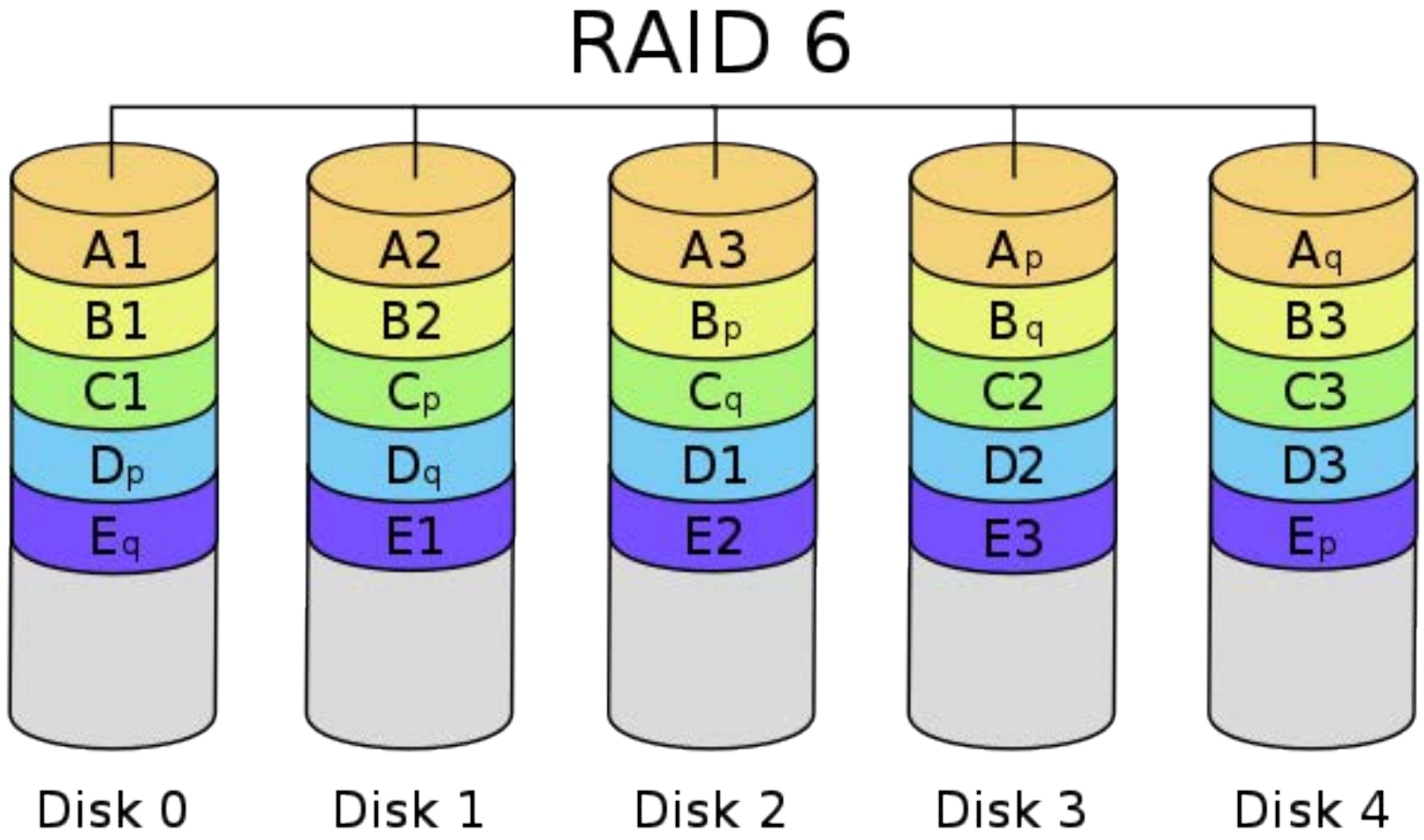


RAID6

- 如果两个磁盘出错呢？
  - Independent Data disks with two independent distributed parity schemes
  - 二维校验



RAID6


RAID 6
A1
B1
C1
Dp
Eq
A2
B2
Cp
Dq
E1
A3
Bp
Cq
D1
E2
Ap
Bq
C2
D2
E3
Aq
B3
C3
D3
Ep
Disk 0
Disk 1
Disk 2
Disk 3
Disk 4

计算机组成原理

第三章 系统总线

阙夏
计算机与信息学院

2020/10/14

第三章 系统总线

1 3.1 总线的基本概念
2 3.2 总线的分类
3 3.3 总线特性及性能指标
4 3.4 总线结构
5 3.5 总线控制

大纲要求

- 教学要求：

了解总线的分类及性能指标；

了解总线标准。

理解总线判优的概念，掌握总线判优方式；

掌握总线定时方式；

- 教学重点：

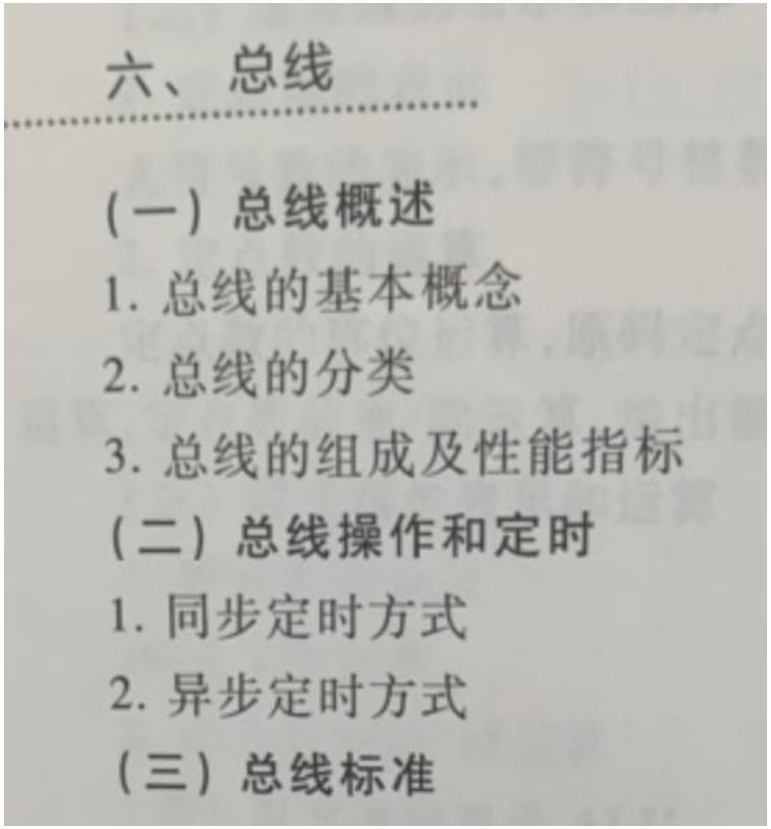
总线的分类及性能指标；

总线判优方式；

总线定时方式

- 教学难点：

总线判优方式。


六、总线

（一）总线概述

1. 总线的基本概念

2. 总线的分类

3. 总线的组成及性能指标

（二）总线操作和定时

1. 同步定时方式

2. 异步定时方式

（三）总线标准

前言

计算机五大部件的连接方式：

- 分散连接
- 总线连接

3.1 总线的基本概念

一、为什么要用总线

- 减少各部件之间的连接线；
- 方便系统的构成、扩充或更新。

各部件和总线采用挂接形式，增加一个部件或去掉一个部件对整个系统的结构不会造成什么影响。

总线技术使计算机的设计、生产走向标准化。

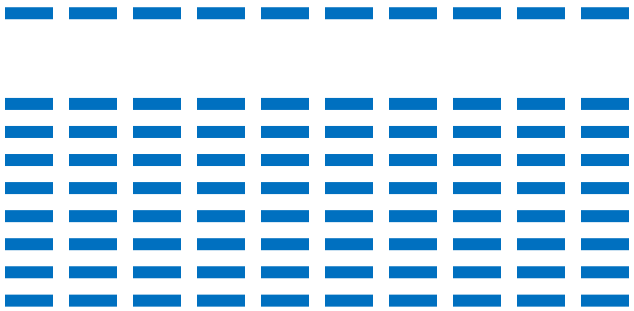
二、什么是总线（bus）

总线是连接各个部件的信息传输线，

是各个部件**共享的**传输介质。

三、总线上信息的传送

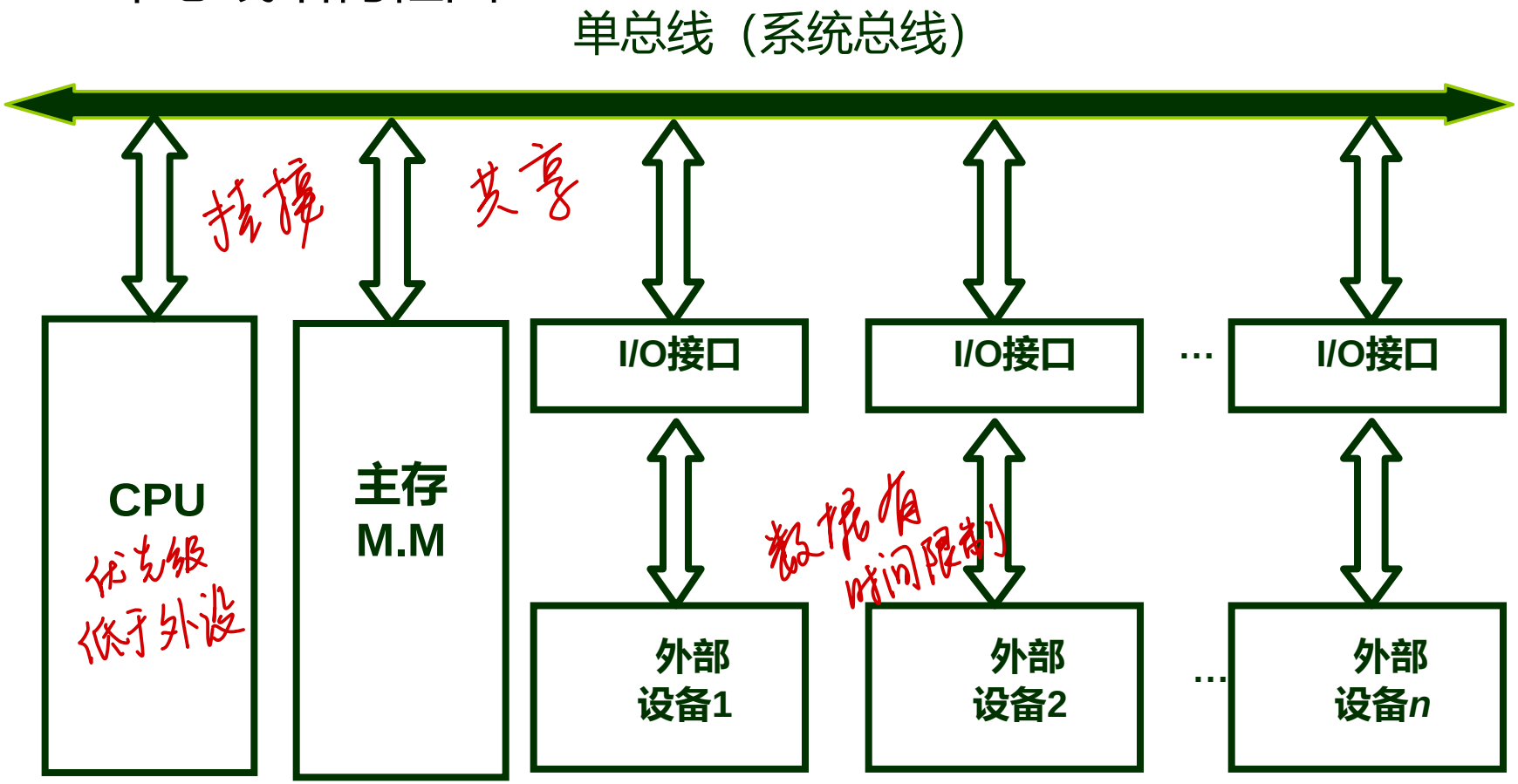
串行



并行

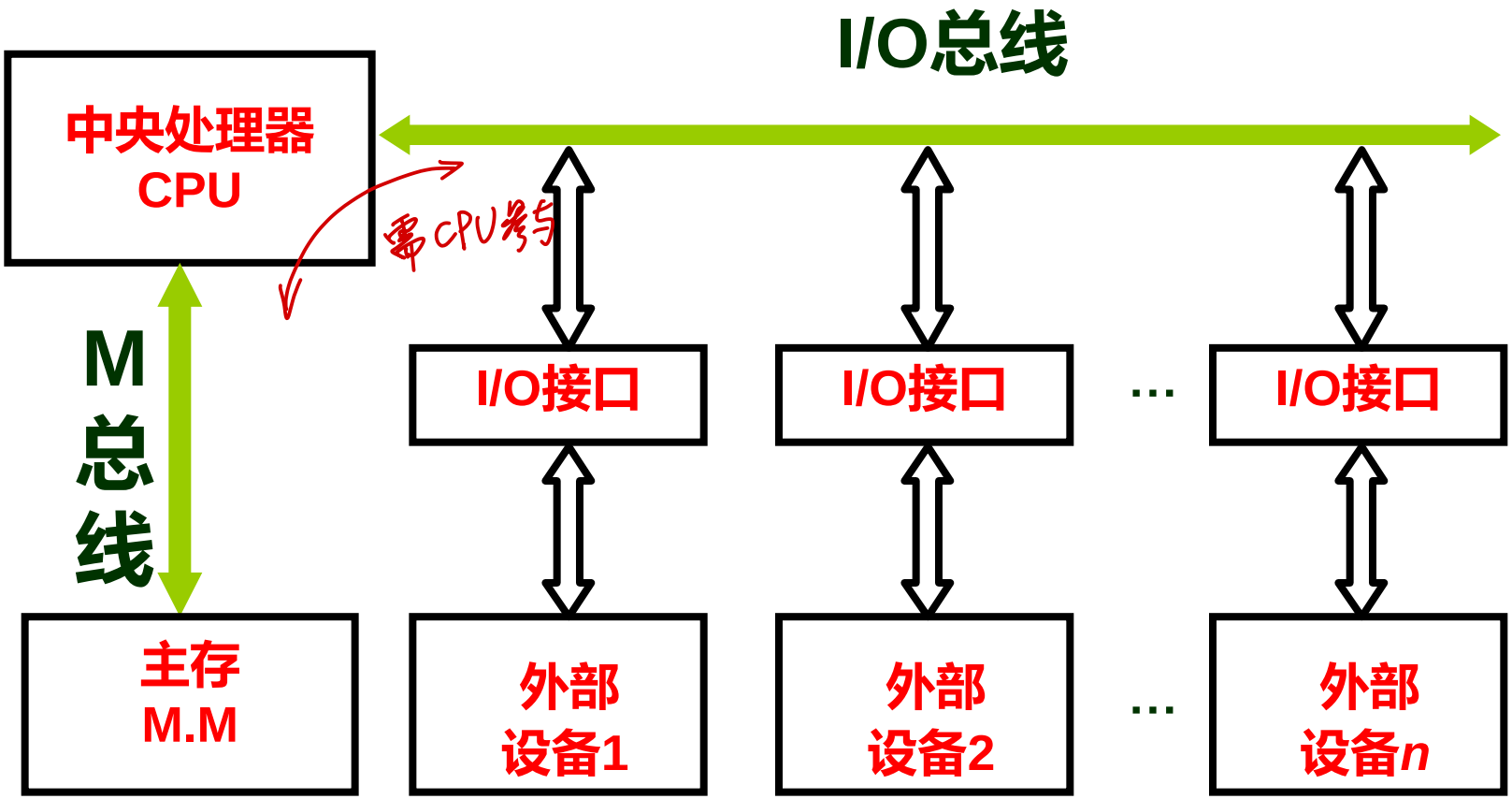
四、总线结构计算机举例

1. 单总线结构框图



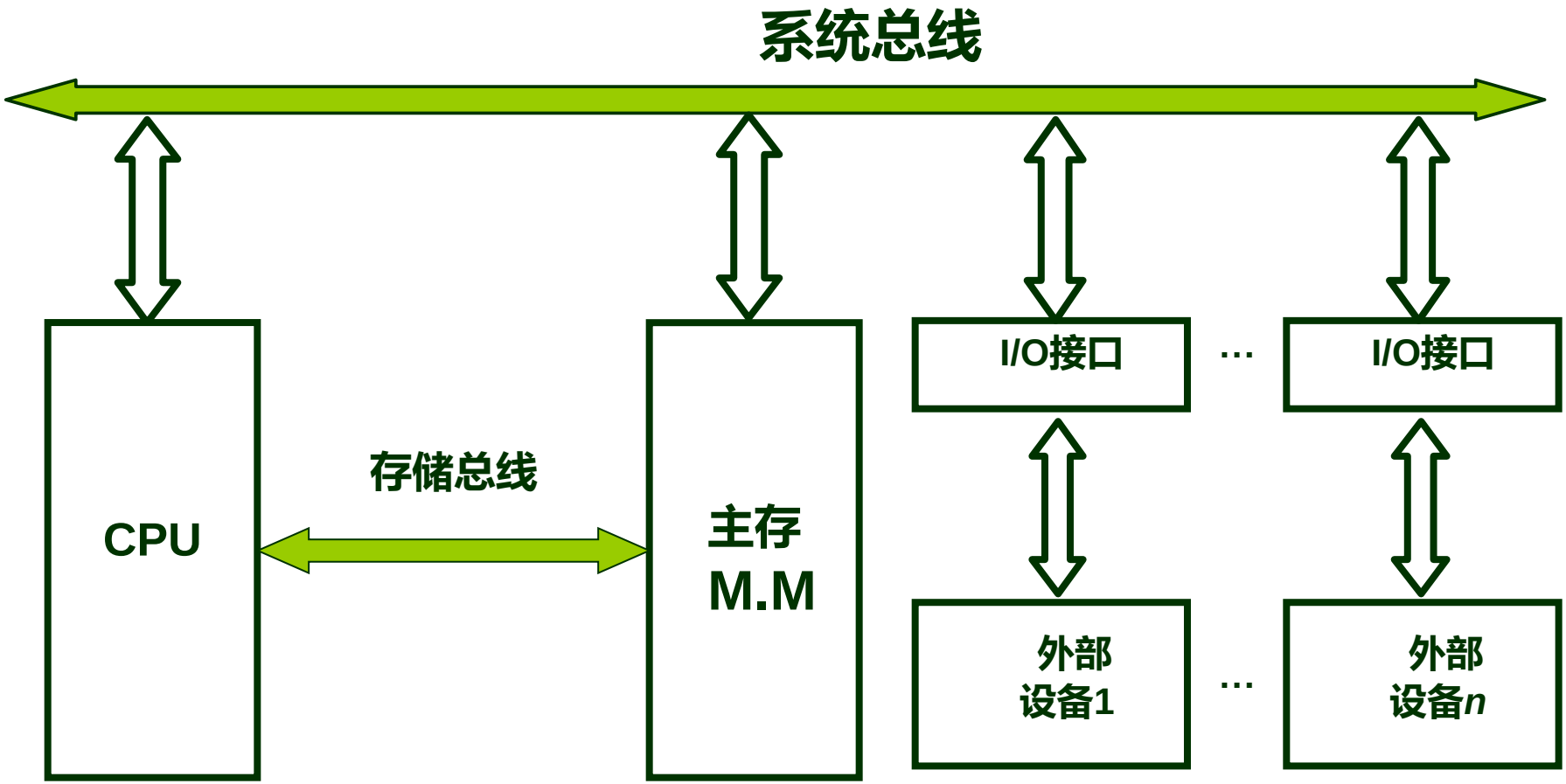
3.1 总线的基本概念

2. 面向 CPU 的双总线结构框图



3.1 总线的基本概念

3. 以存储器为中心的双总线结构框图



3.2 总线的分类

1.片内总线　　芯片内部的总线。课本P43

2.系统总线　　计算机各部件之间的信息传输线。

- 数据总线　　与机器字长、存储字长有关/双向
- 地址总线　　与存储地址、 I/O地址有关/单向　　**存储单元个数=$2^{地址总线根数}$**
- 控制总线　　发出各种控制信号，监视各部件状态，有入 / 有出

中断请求、
总线请求

存储器读、存储器写
总线允许、中断确认

3.通信总线　　用于**计算机系统之间**或**计算机系统与其他系统**（如控制仪表、移动通信等）之间的通信。

3.2 总线的分类-系统总线

CPU 从 3 号单元中读取数据的过程(见图 1.3)如下：

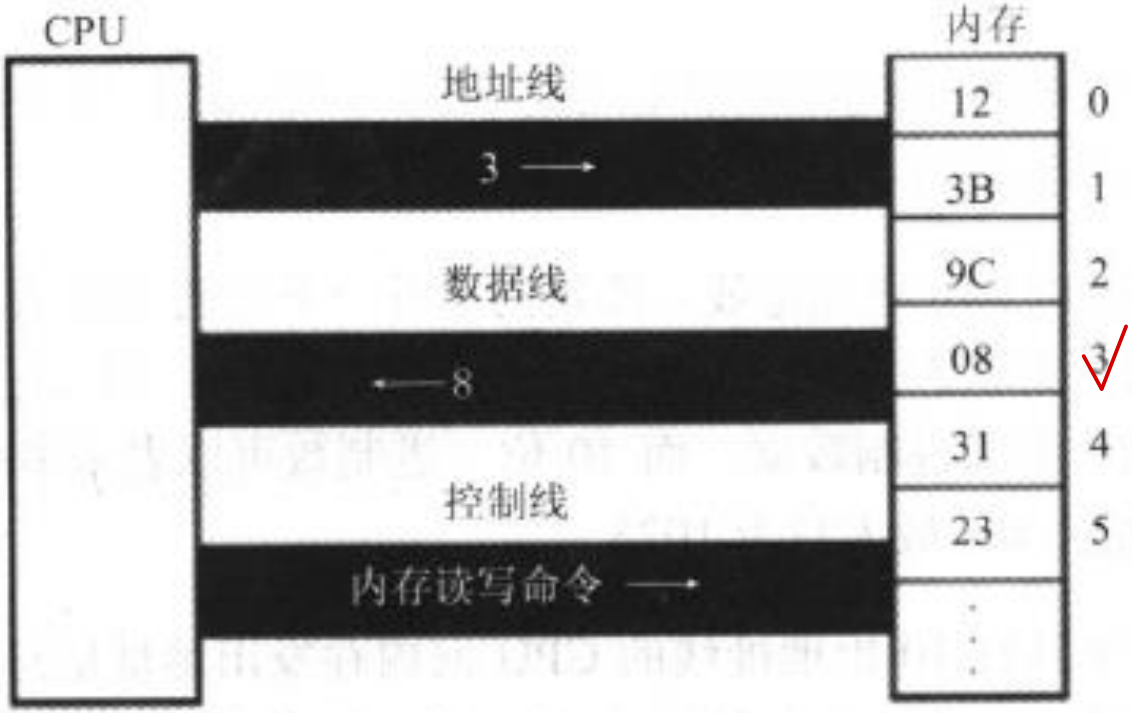


图 1.3　CPU 从内存中读取数据的过程

3.2 总线的分类-系统总线

计算机内部每个有效信息，必须具有3个基本属性：内容，指向和行为；这三个属性要通过三个总线实现：数据总线，地址总线，控制总线；

读/写

- 数据总线：决定传输的数据值。（字长大小）
  - ①决定一次数据传输的数据大小。
  - ②数据总线的宽度决定了CPU和外界的数据传送速度。
  - ③每条传输线只能传递1位二进制数据。
  - ④数据总线是数据线条数之和

3.2 总线的分类-系统总线

- 地址总线：决定内存数据操作的地址

  ①CPU是通过地址总线来指定存储单元的。

  ②地址总线决定了CPU能访问的最大内存空间的大小。

  ③地址总线是地址线数量之和。

- 控制总线：反映了数据的状态和传输方式。

  ① CPU通过控制总线对外部器件进行控制。

  ② 控制总线的宽度决定了CPU对外器件的控制能力。

  ③ 控制总线是控制线数量之和。

3.2 总线的分类-通信总线

- **通信总线**（外部总线、 I/O 总线）：

  用于计算机系统之间

  或计算机系统与其他系统（如控制仪表、移动通信等）之间的通信。

-通常是电缆式总线，如SCSI、RS-232、USB等

传输方式
- 串行通信总线
- 并行通信总线

Intel 体系结构中特指的“系统总线”

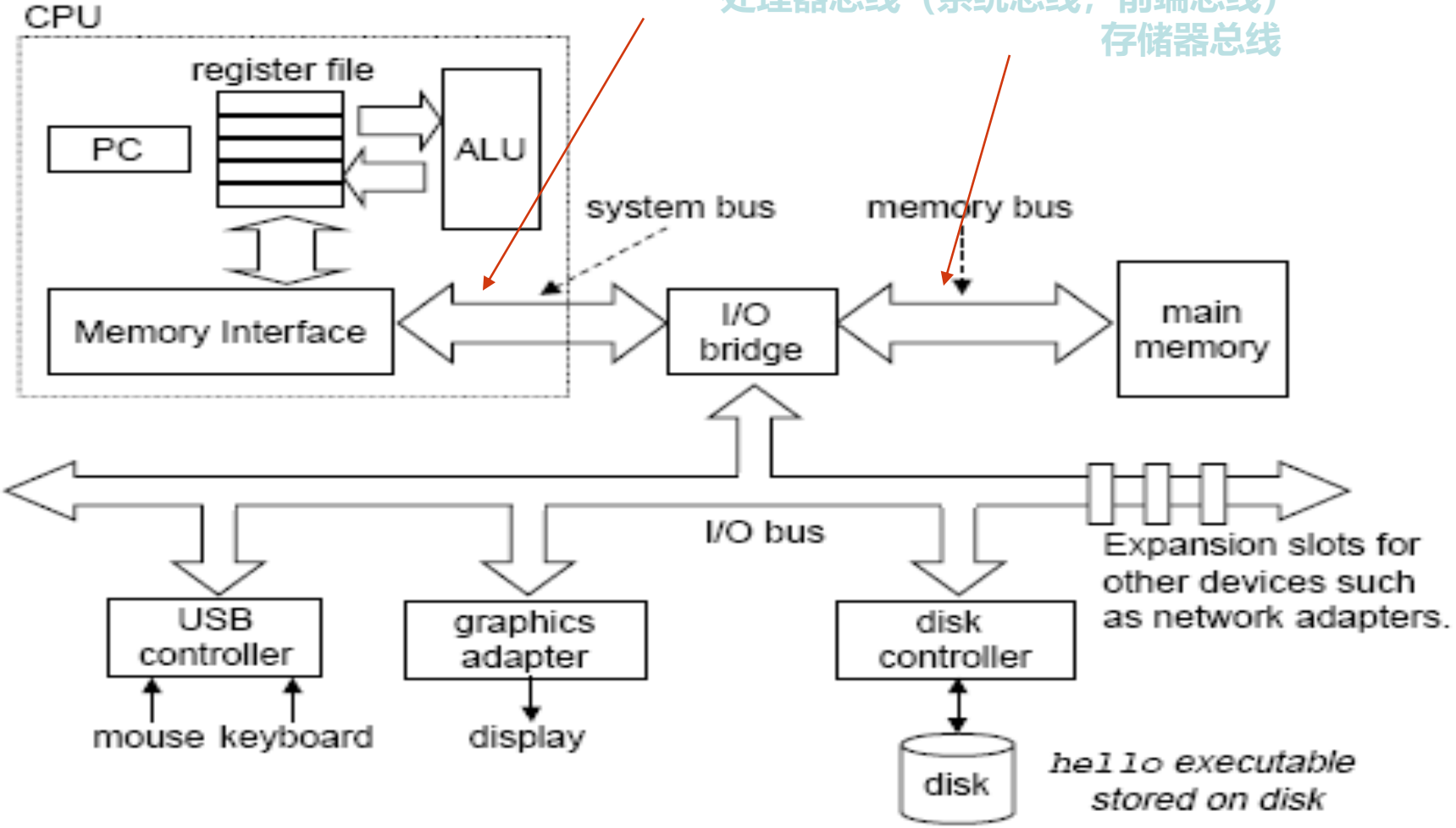
注：Intel公司在推出845、850等芯片组时，对“System Bus”有专门的定义，将处理器总线称为前端总线(Front Bus)或系统总线。

Intel 体系结构中特指的“系统总线”

北桥芯片组把处理器–存储器总线分成了两个总线：

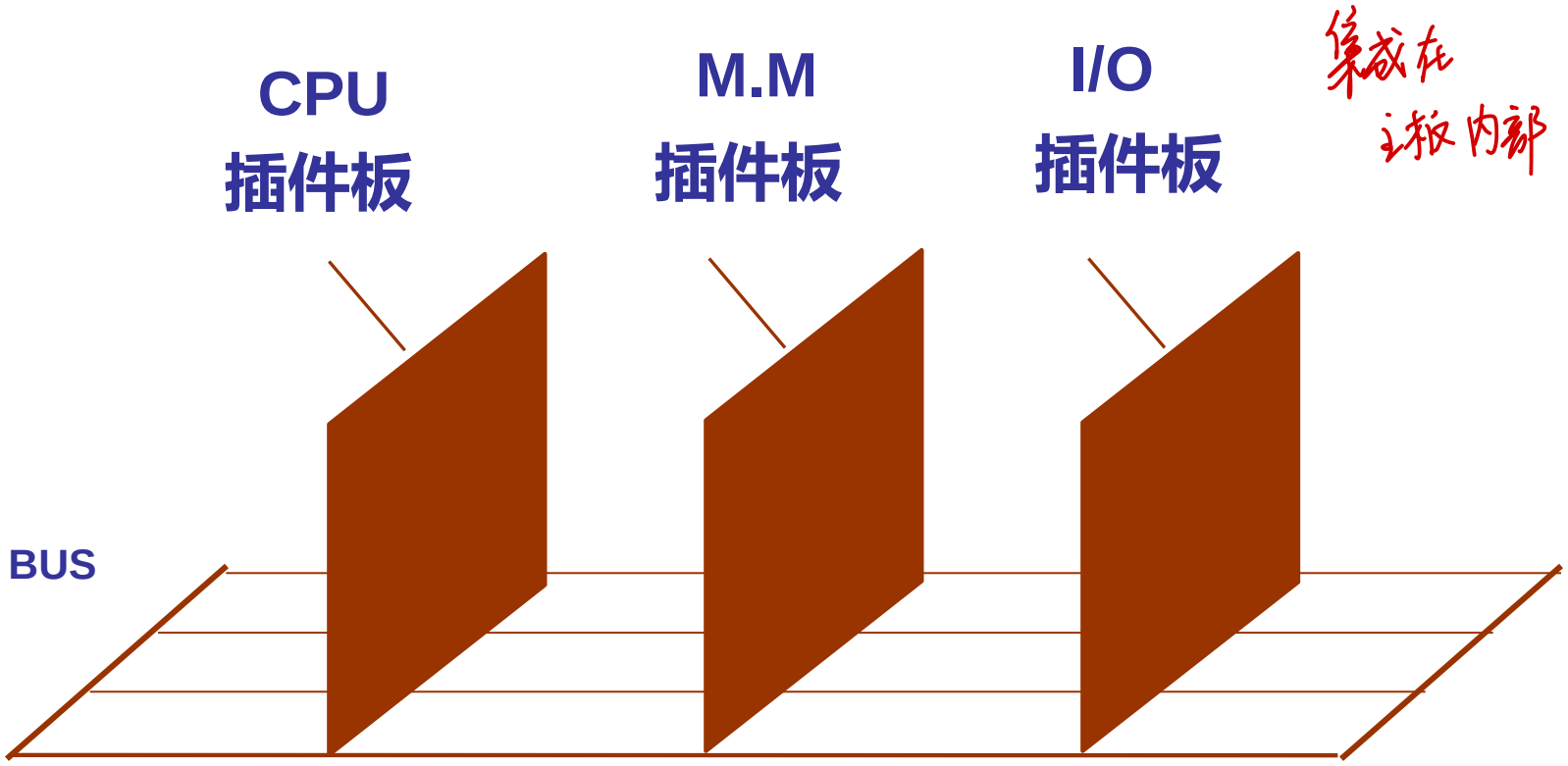
处理器总线（系统总线，前端总线）

存储器总线



3.3 总线特性及性能指标

一、总线物理实现



3.3 总线特性及性能指标

二、总线特性

1. 机械特性 机械连接方式上的一些性能，尺寸 形状

厂家控制 国内标准基本统一

2. 电气特性 每根传输线上的信号的传输方向和有效的电平范围

3. 功能特性 每根传输线的功能
- 地址
- 数据
- 控制

4. 时间特性 信号的时序关系

3.3 总线特性及性能指标

三、总线的性能指标

1. 总线宽度 数据线的根数，用bit表示

2. 总线带宽 (Bus Bandwidth) 每秒传输的最大字节数（MB/s）

（总线最大数据传输率、传输带宽）

总线带宽 = （总线宽度/8）×总线工作频率（MHz）

3. 时钟同步/异步 同步、不同步

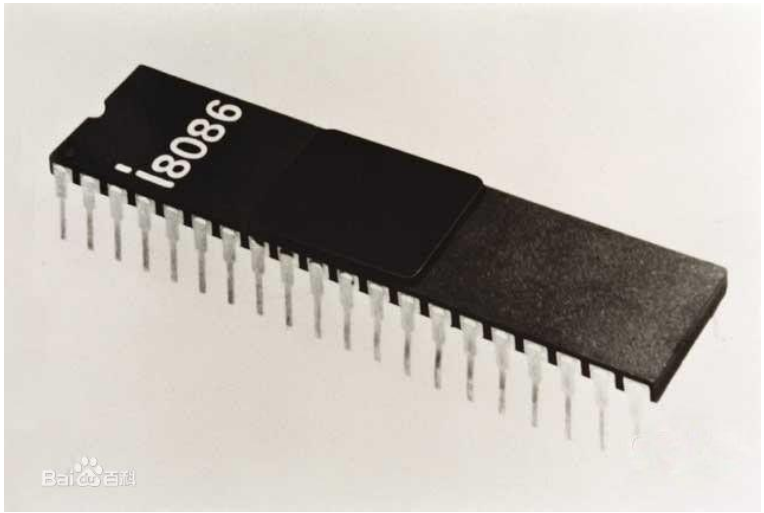
3.3 总线特性及性能指标

4. 总线复用 地址线 与 数据线 复用

5. 信号线数 地址线、数据线和控制线的 总和

6. 总线控制方式 突发工作、自动配置、仲裁方式等

7. 其他指标 负载能力、电源电压（5 or 3.3）、总线宽度扩展性



3.3 总线特性及性能指标

例：1、 某总线在一个总线周期中并行传送4个字节的数据，假设一个总线周期等于一个时钟周期，总线时钟频率为33MHz，求总线带宽。

4*33=132 MB/s（MBps）

2、如果一个总线中并行传送64位数据，总线时钟频率升为66MHz，求总线带宽。

64÷8*66=528 MB/s（MBps）

分析那些因素影响带宽。宽度、频率

3.3 总线特性及性能指标

名词解释

- **波特率**：每秒钟通过信道传输的码元数（二进制位数），单位为bps(位/秒），记为波特。(每秒内线路状态的改变次数)
- **比特率**：每秒钟通过信道传输的信息量（有效数据位数）去除校验位
- **猝发式数据传输**（burst mode）：是一种总线传输方式，即在一个总线周期传输存储地址连续的多个消息。
- **消息传输**：将总线需要传送的数据信息、地址信息和控制信息等合成一个固定的数据结构，以猝发方式进行传输。
- **总线协议**：总线通信同步方式规定了实现总线数据传输的定时规则，即总线协议。
- **总线事务**(Bus Transaction)：从请求总线到完成总线使用的操作序列。如：

  存储器读、存储器写、I/O读、I/O写、读指令、中断响应等

3.3 总线特性及性能指标

3.3 总线特性及性能指标

四、总线的标准

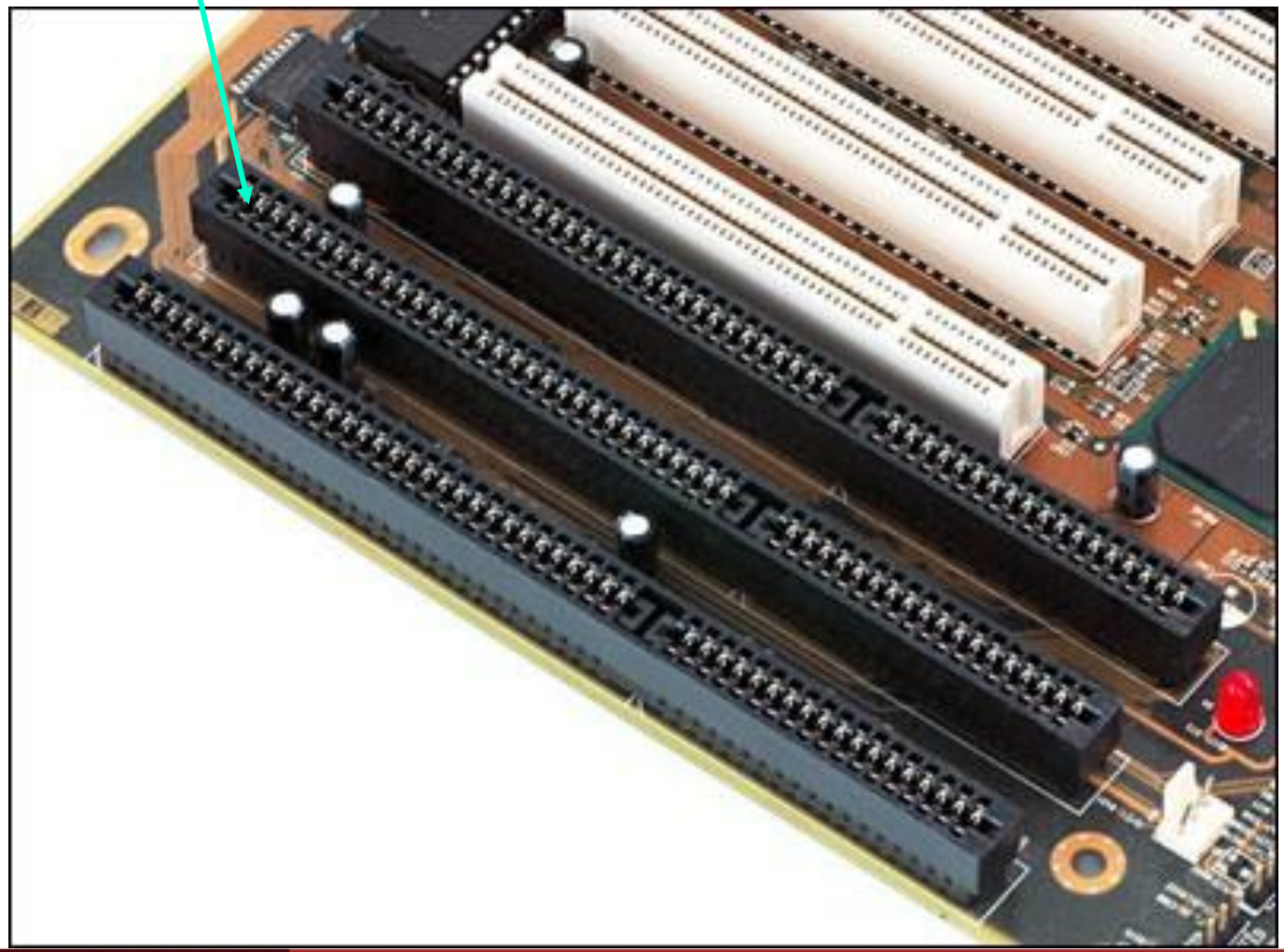
频率、带宽不同

总线标准	数据线	总线时钟	带宽
ISA	16	8MHz(独立)	16MBps
EISA	32	8MHz(独立)	33MBps
VESA	32	32MHz(CPU)	133MBps
PCI 并	32/64	33/66MHz(独立)	132/528MBps
AGP 连接 串	32	66.7/133MHz(独立)	266MBps(AGP 1x) 533MBps(AGP 2x) 1GBps(AGP 4x) 2.1GBps(AGP 8x)
RS-232	串行通信		数据终端设备和数据通信设备之间的接口
USB	串行接口		1.5~12Mbps(USB1.0) 480Mbps(USB2.0)
PCI Express	8,点对点串行连接的设备连接方式	2.5 GHz	支持双向多通道，最大带宽可达256Gbps

https://blog.csdn.net/CocoWu892

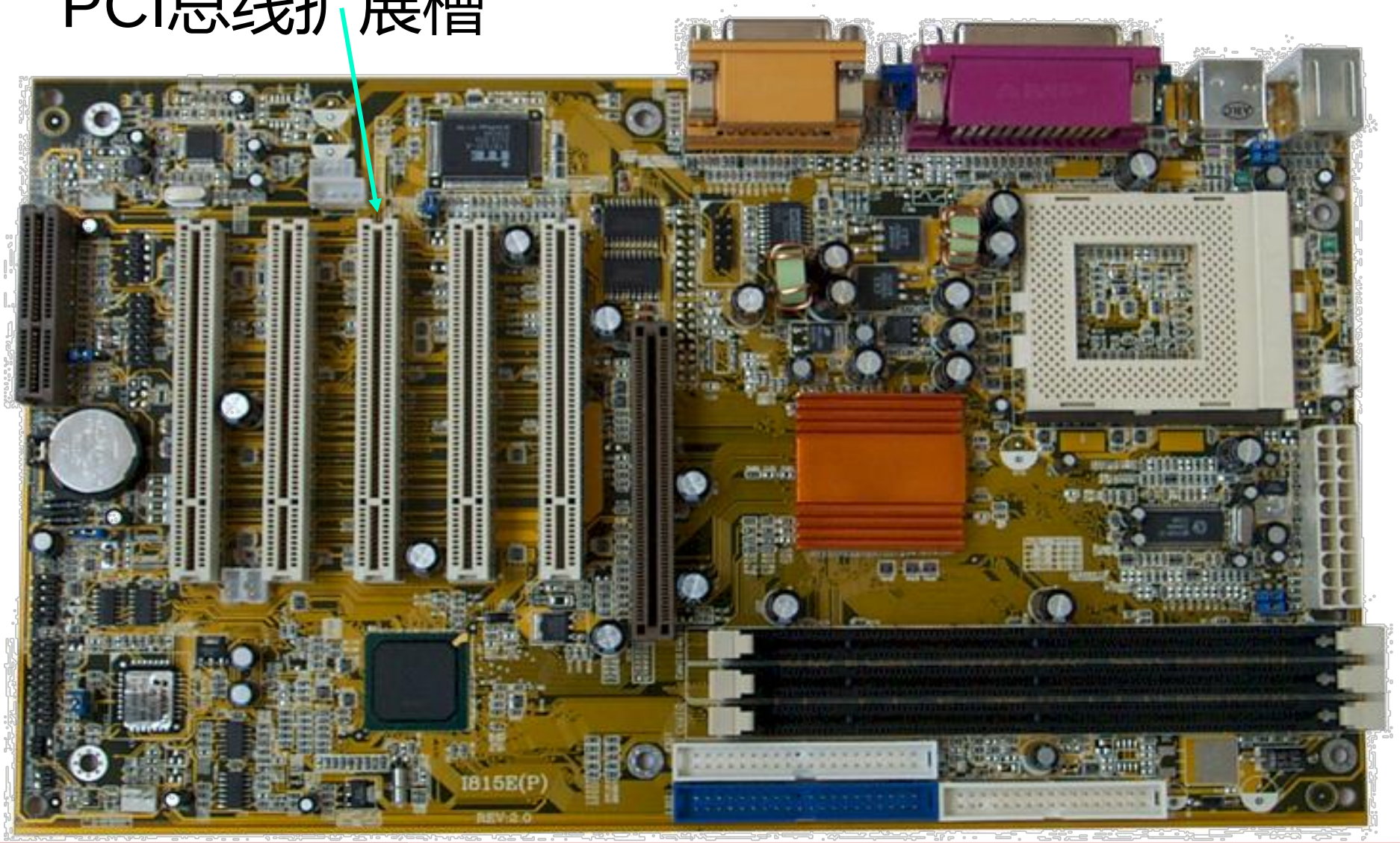
3.3 总线特性及性能指标

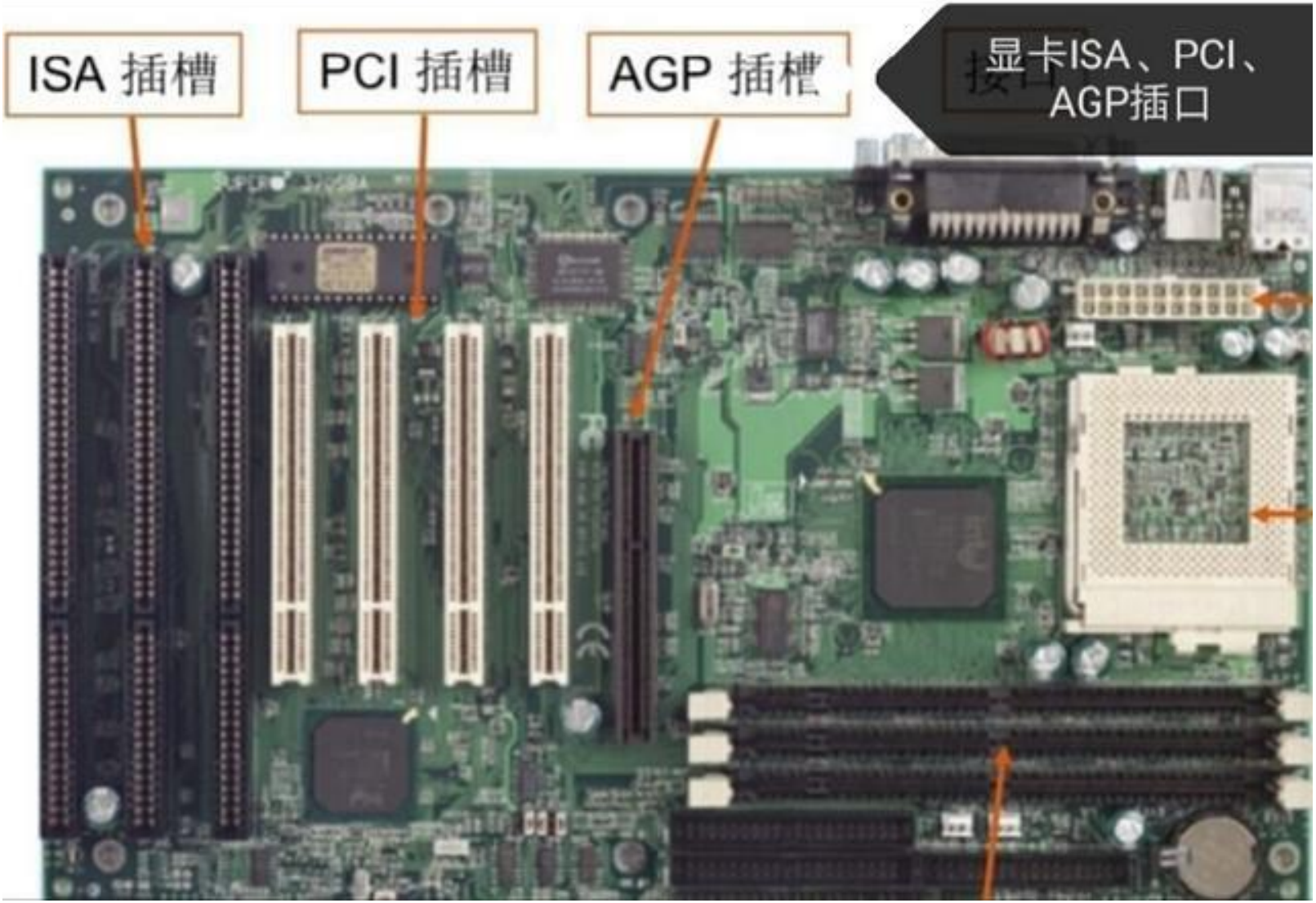
ISA总线



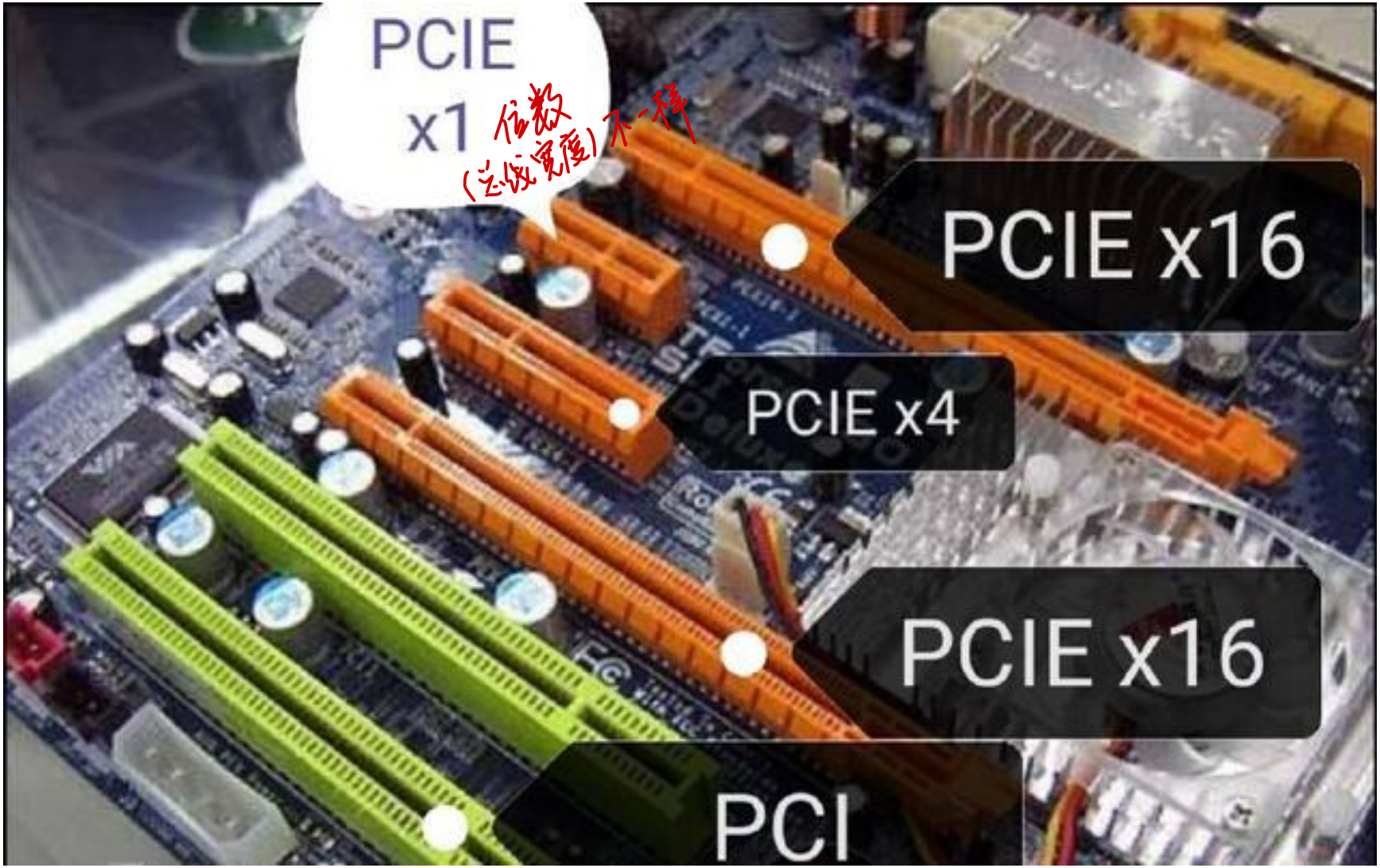
3.3 总线特性及性能指标

PCI总线扩展槽



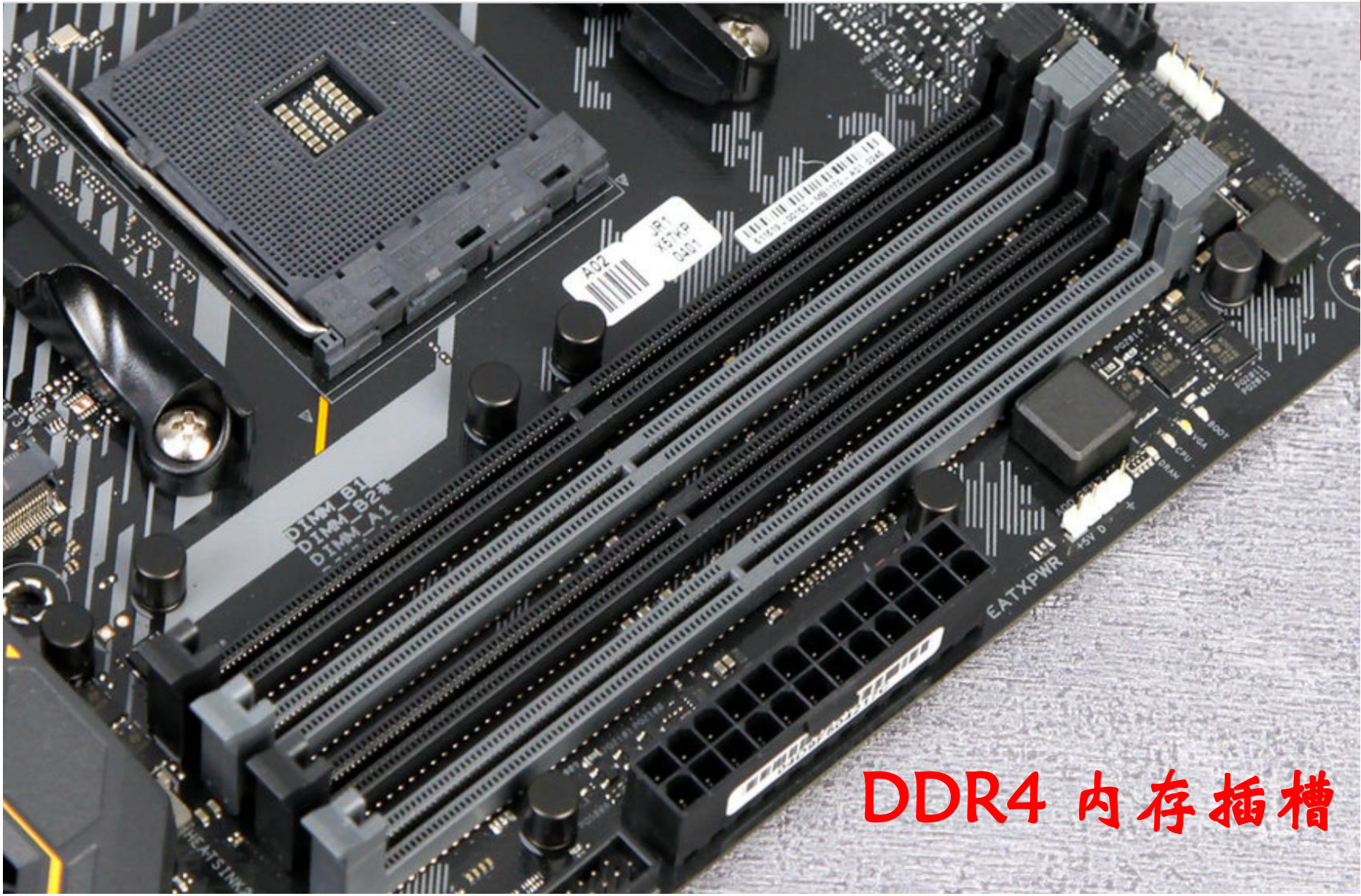

ISA 插槽
PCI 插槽
AGP 插槽
显卡ISA、PCI、
AGP插口

3.3 总线特性及性能指标



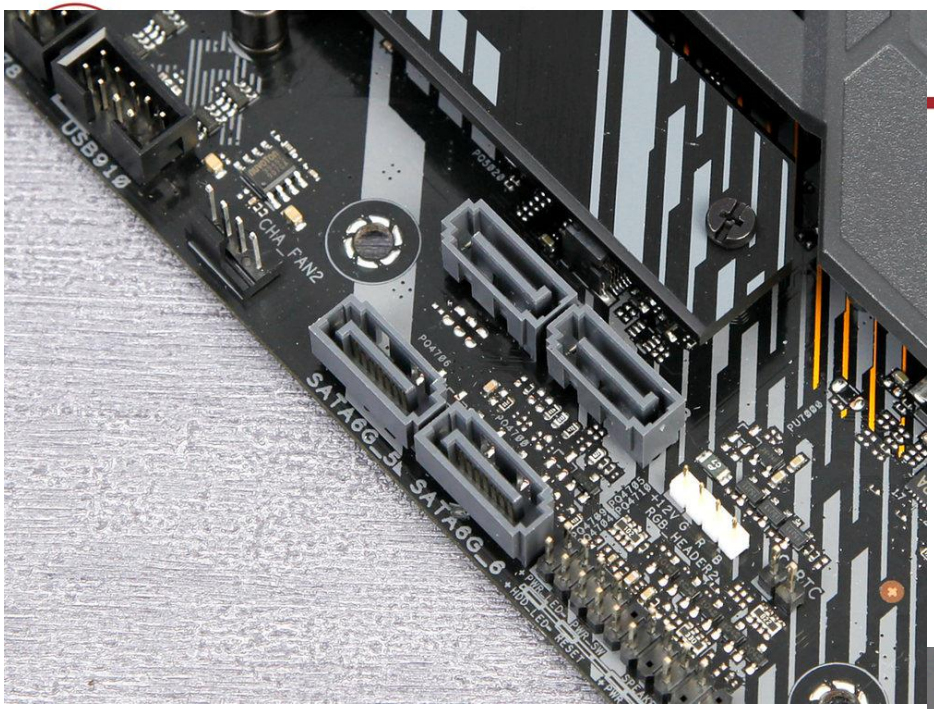
华硕TUF GAMING X570-PLUS (WI-FI)

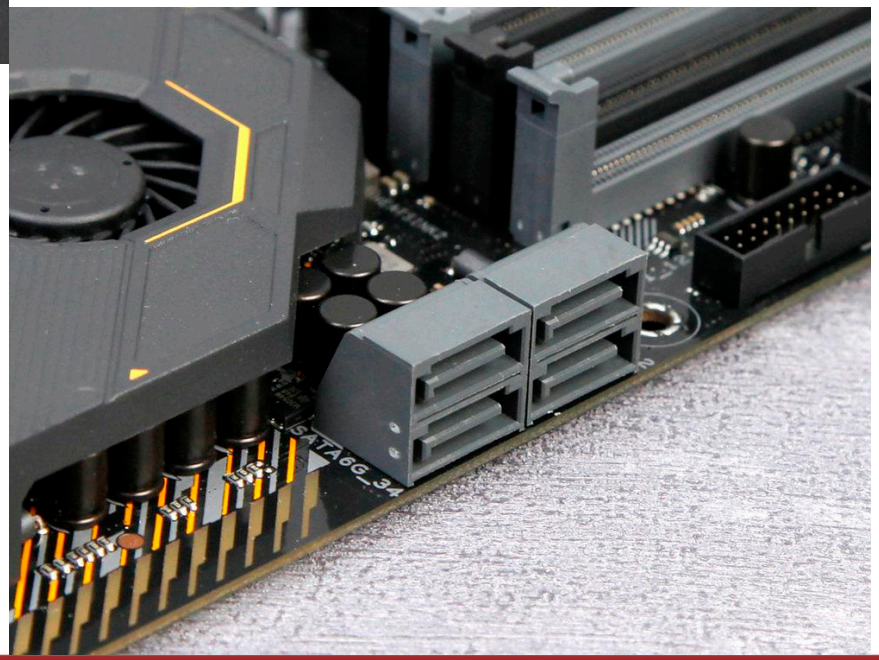
Socket AM4（1331）接口

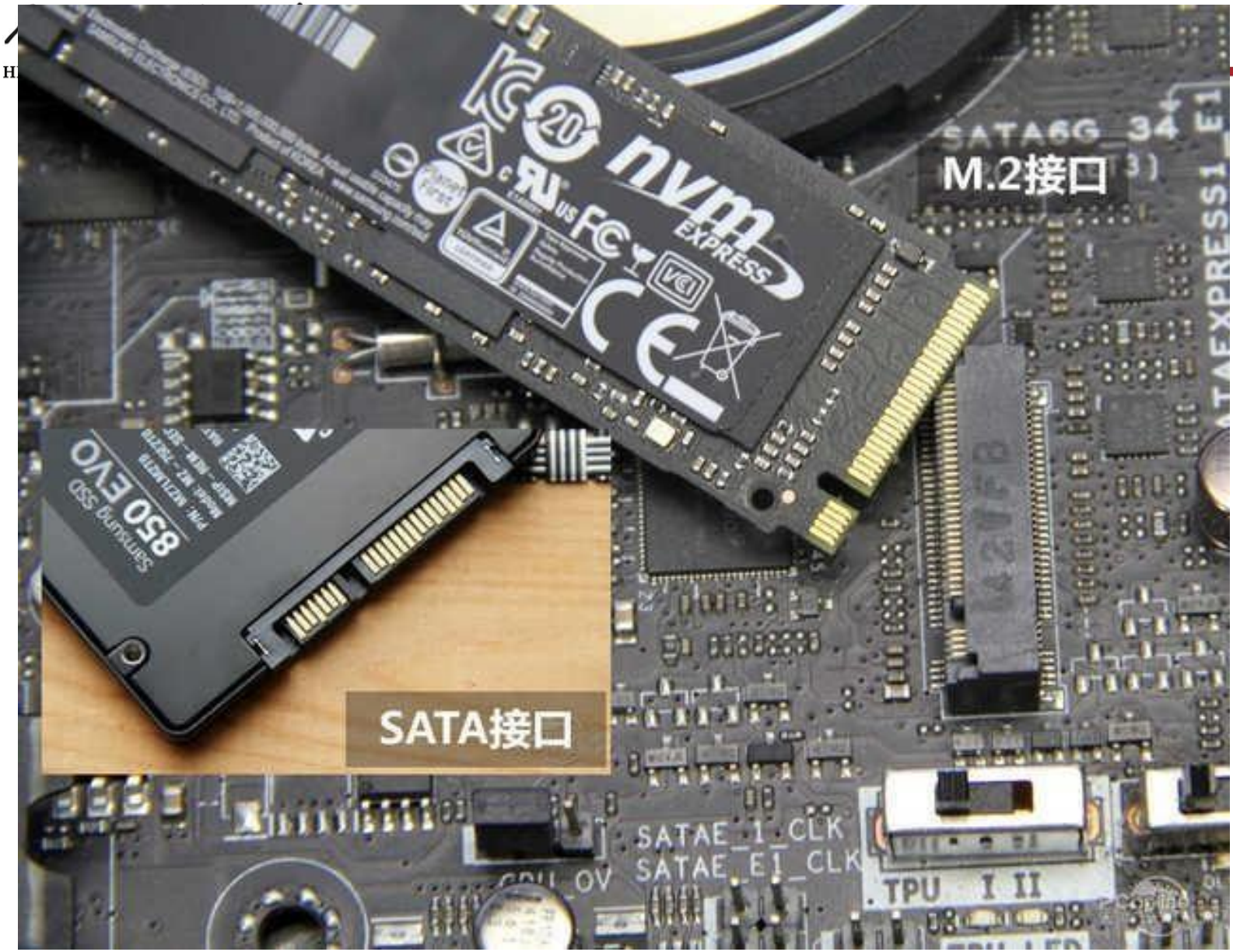


DDR4 内存插槽

SATA 3.0



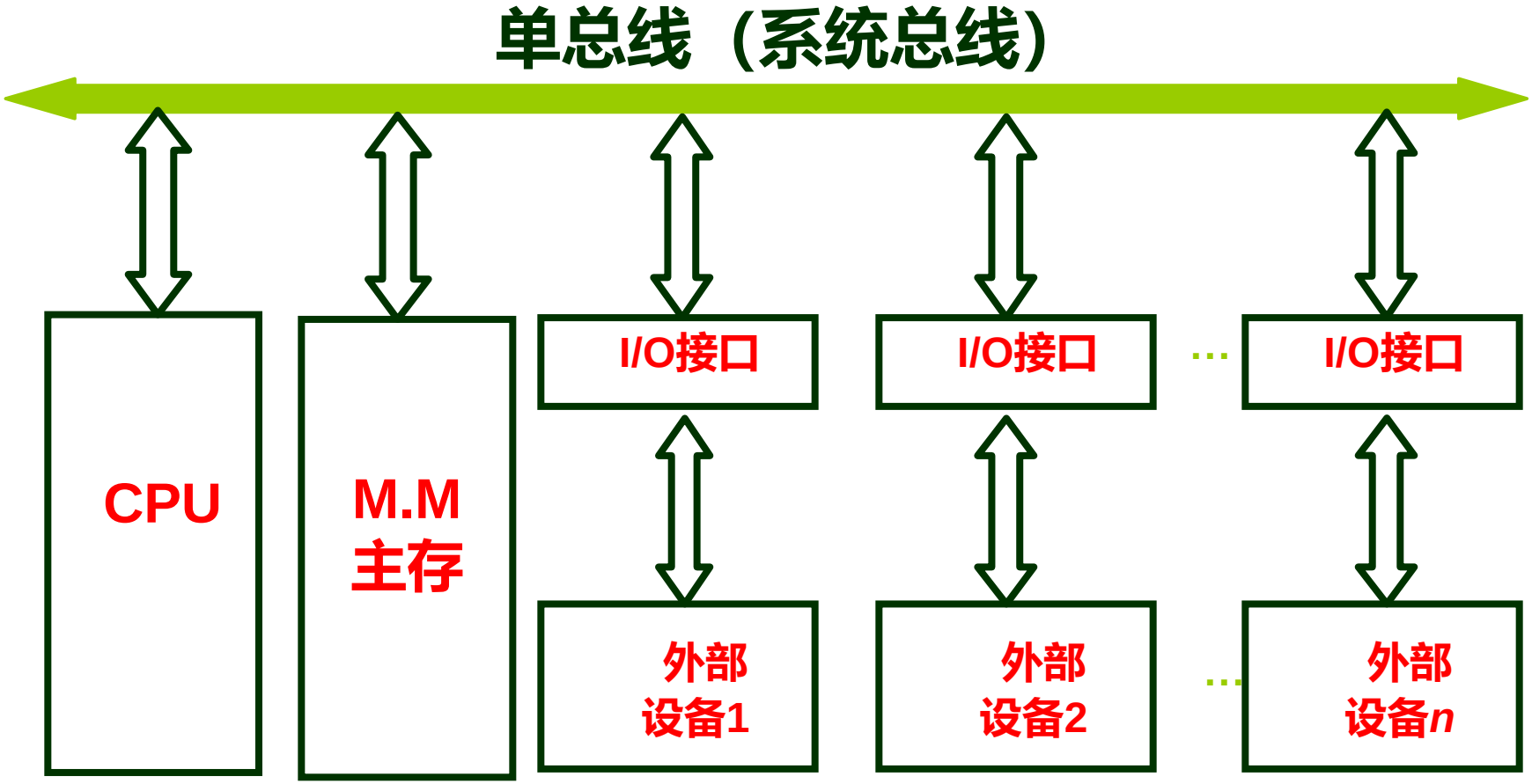



M.2接口
SATA接口
SATA6G_34
SATAE_I_CLK
SATAE_EI_CLK
TPU I II

I/O接口

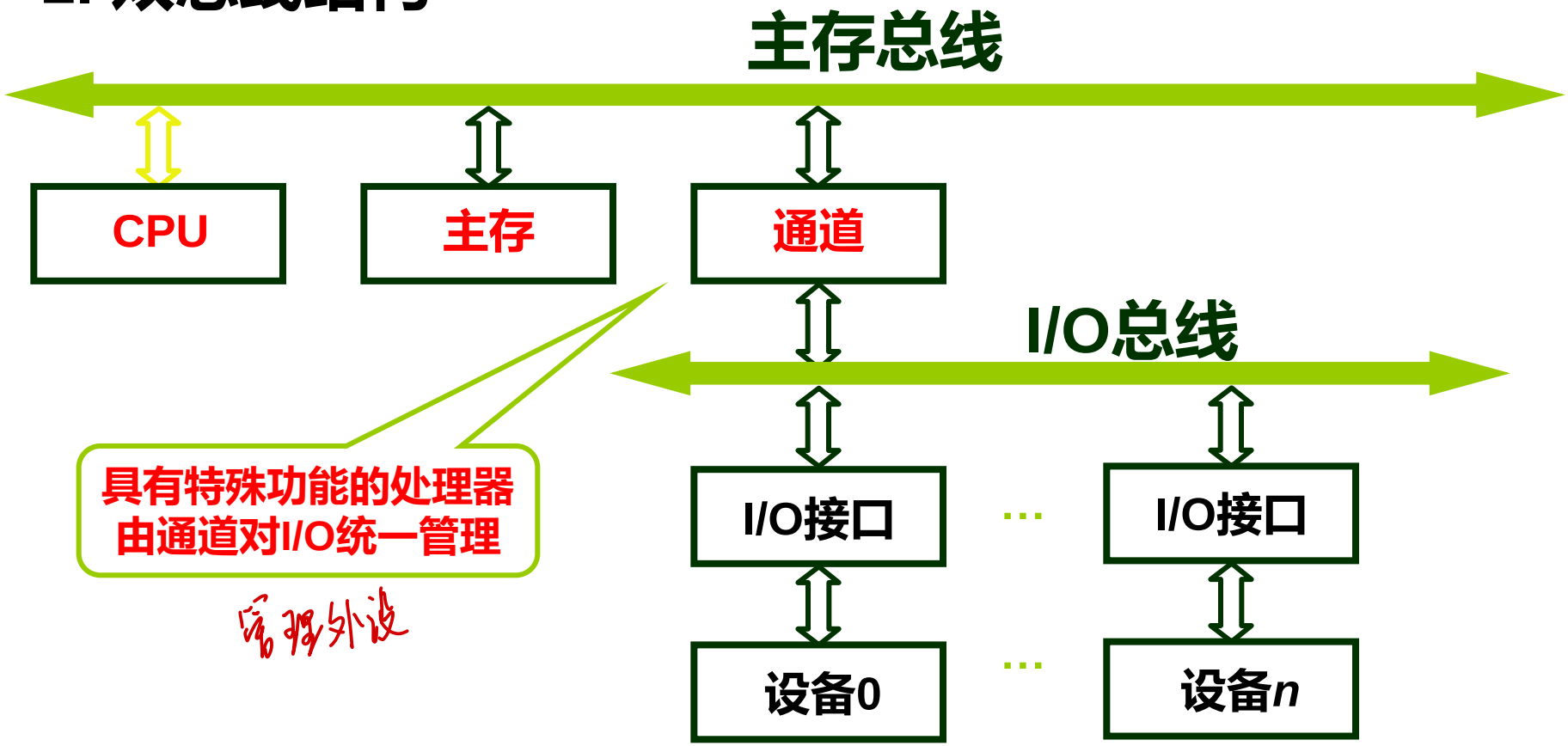
3.4 总线结构

一、单总线结构



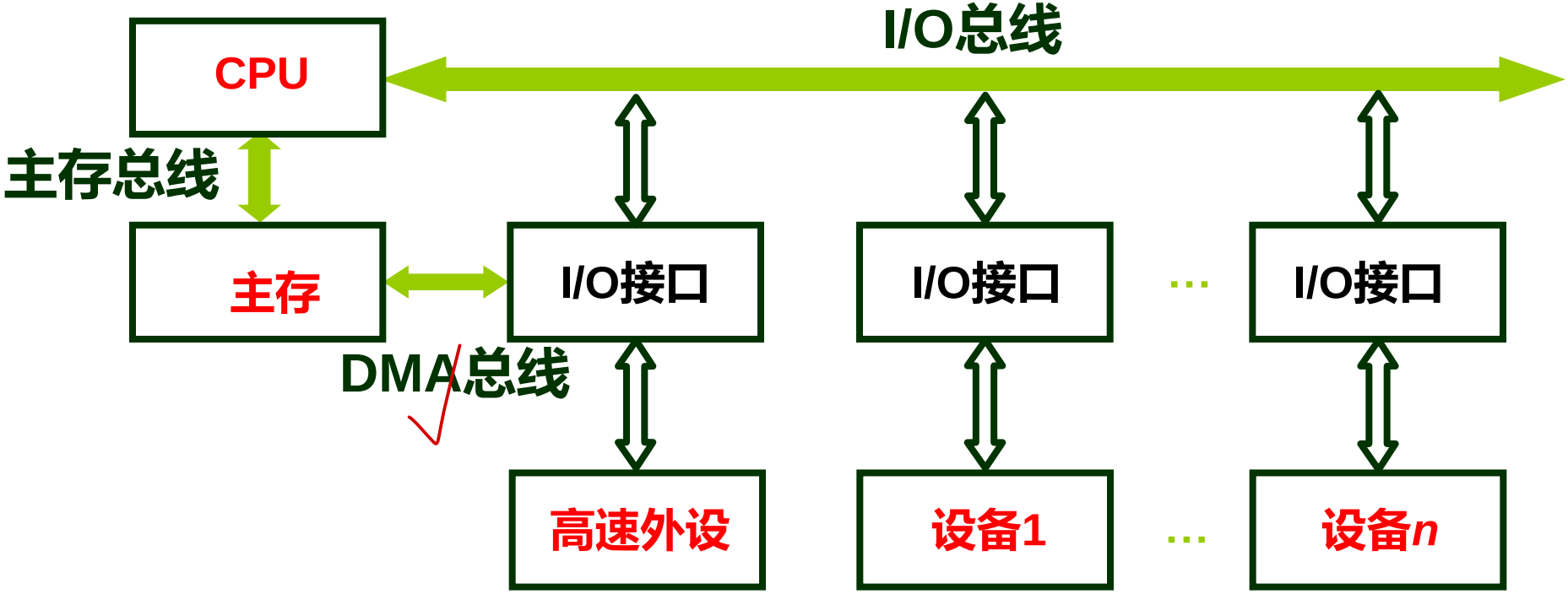
二、多总线结构

1. 双总线结构



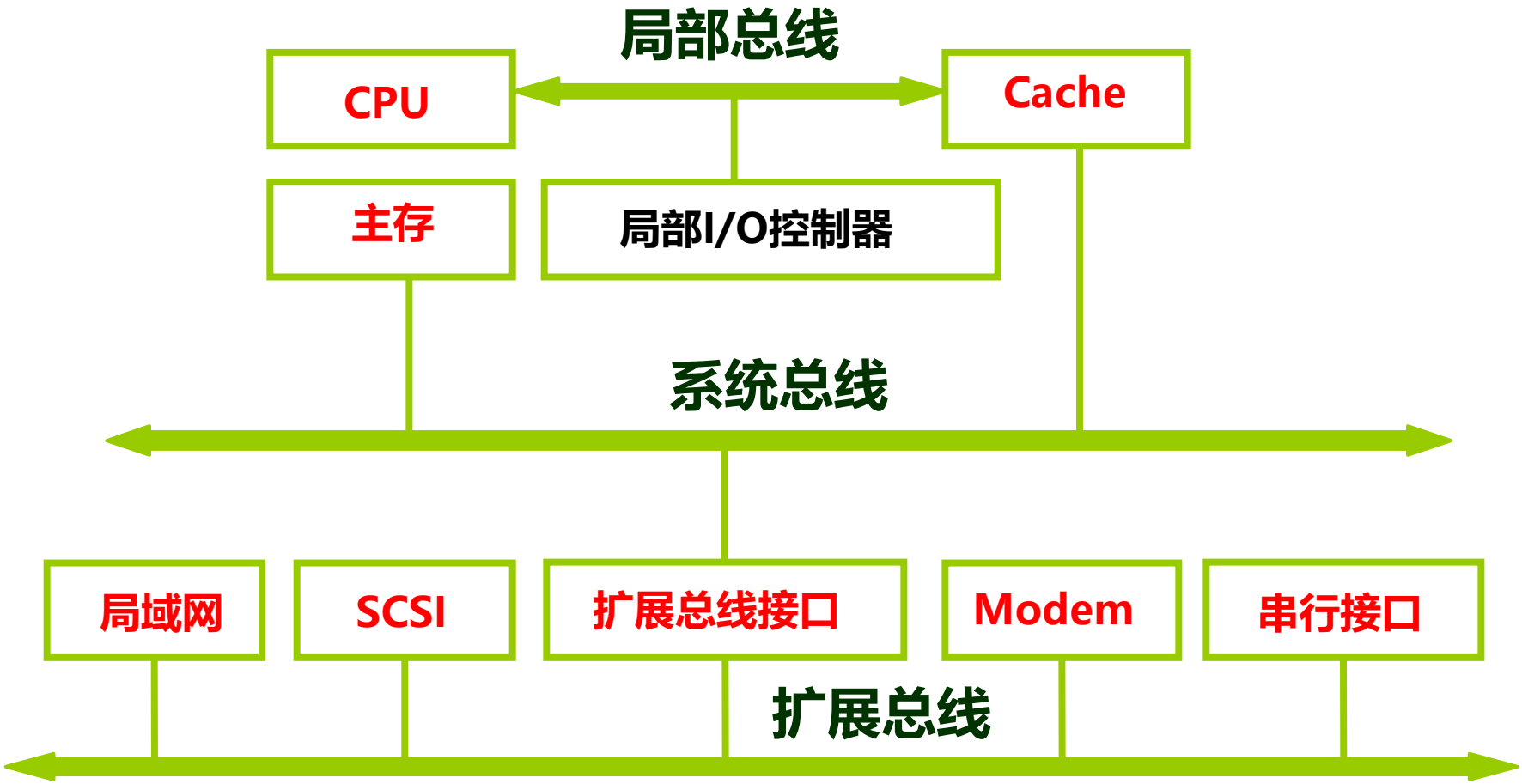
3.4 总线结构

2. 三总线结构



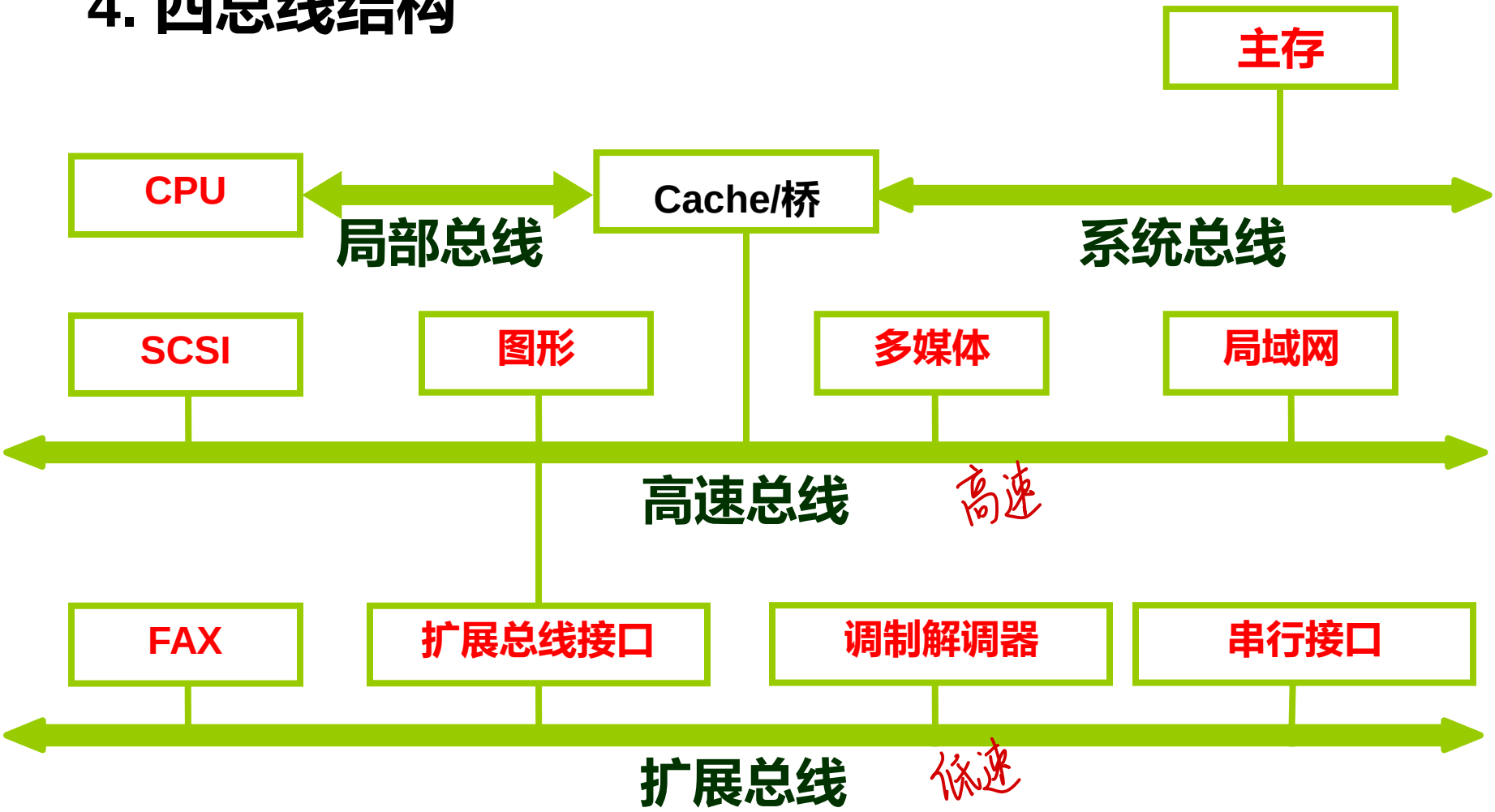
3.4 总线结构

3. 三总线结构的又一形式



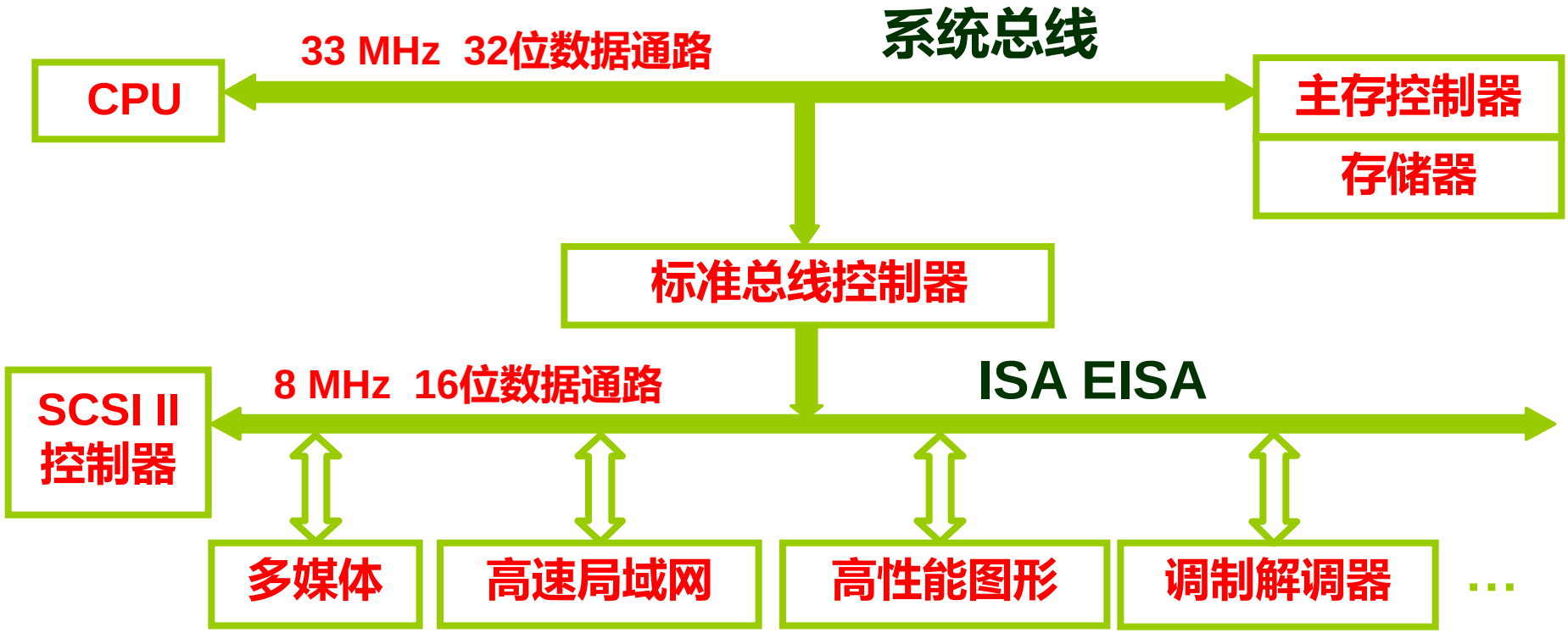
3.4 总线结构

4. 四总线结构



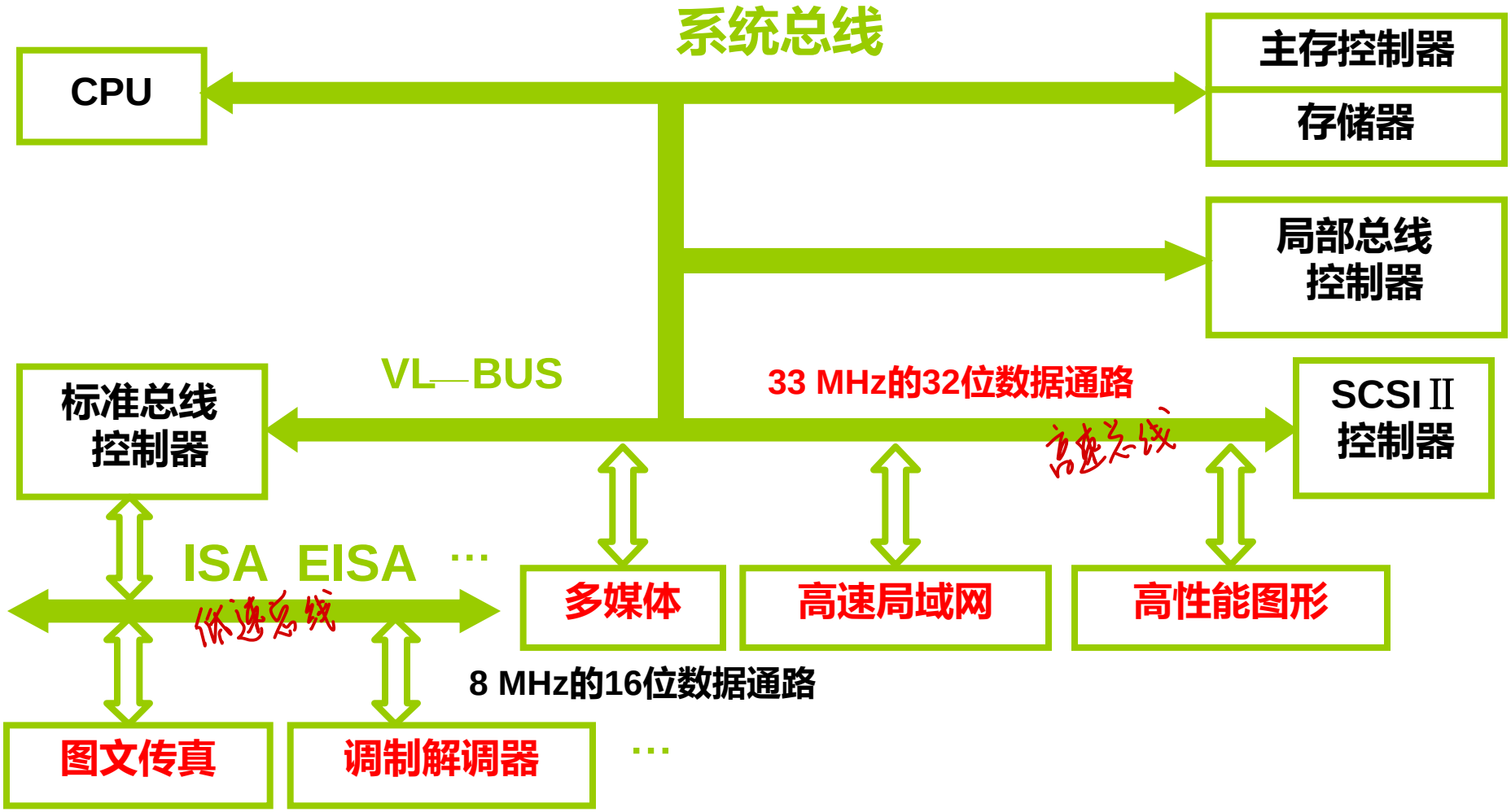
三、总线结构举例

1. 传统微型机总线结构

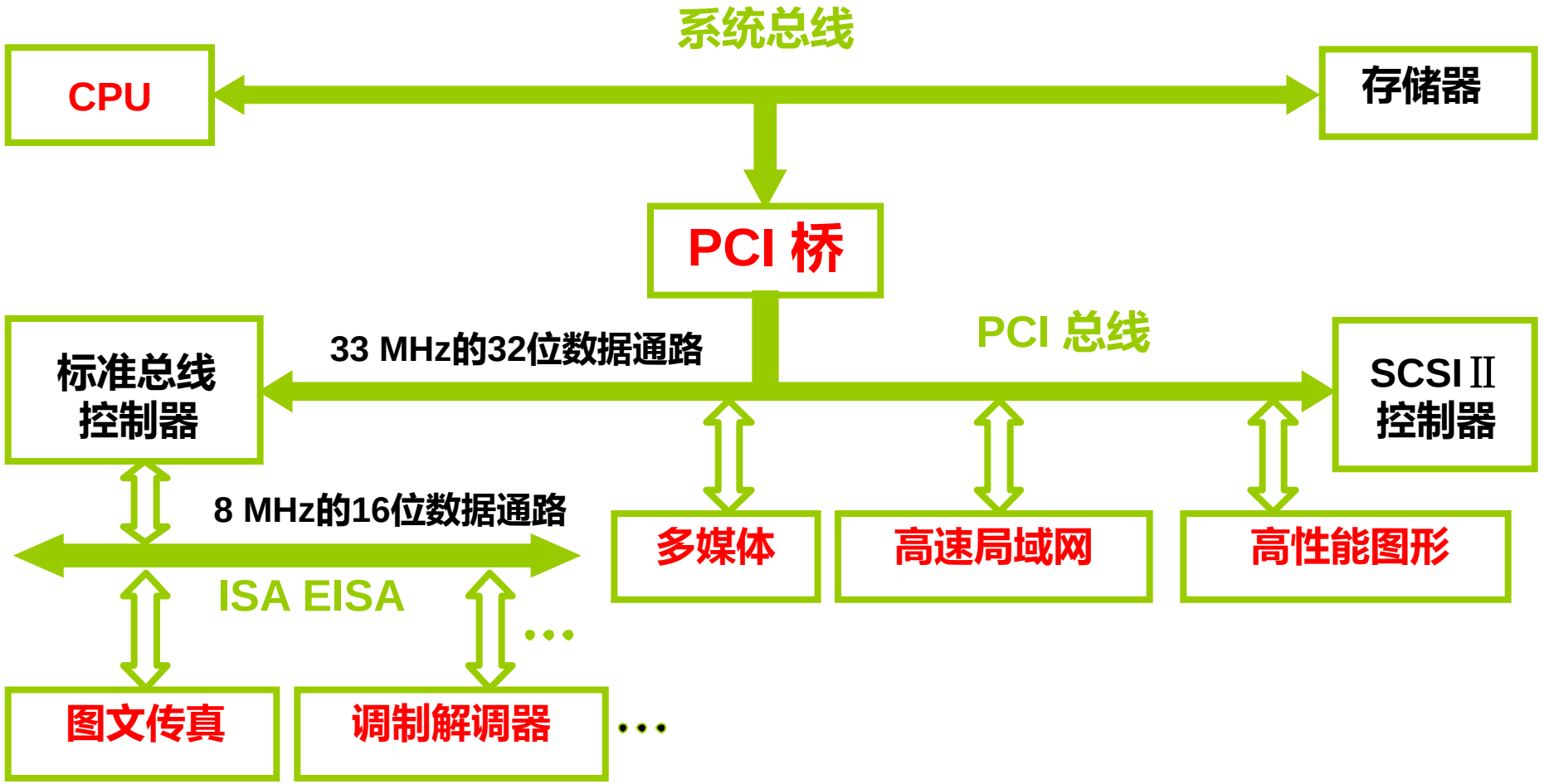


3.4 总线结构

2. VL-BUS局部总线结构

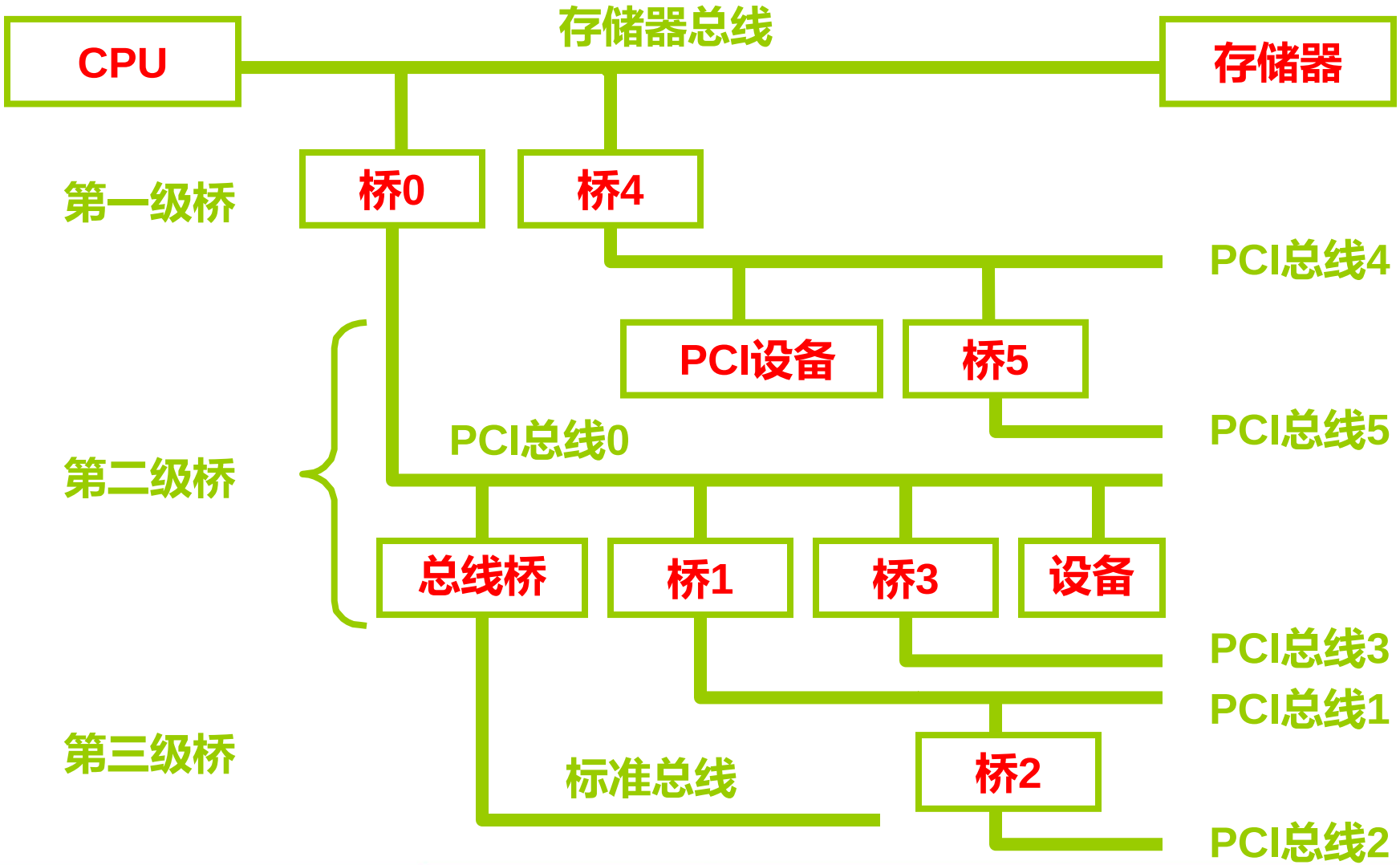


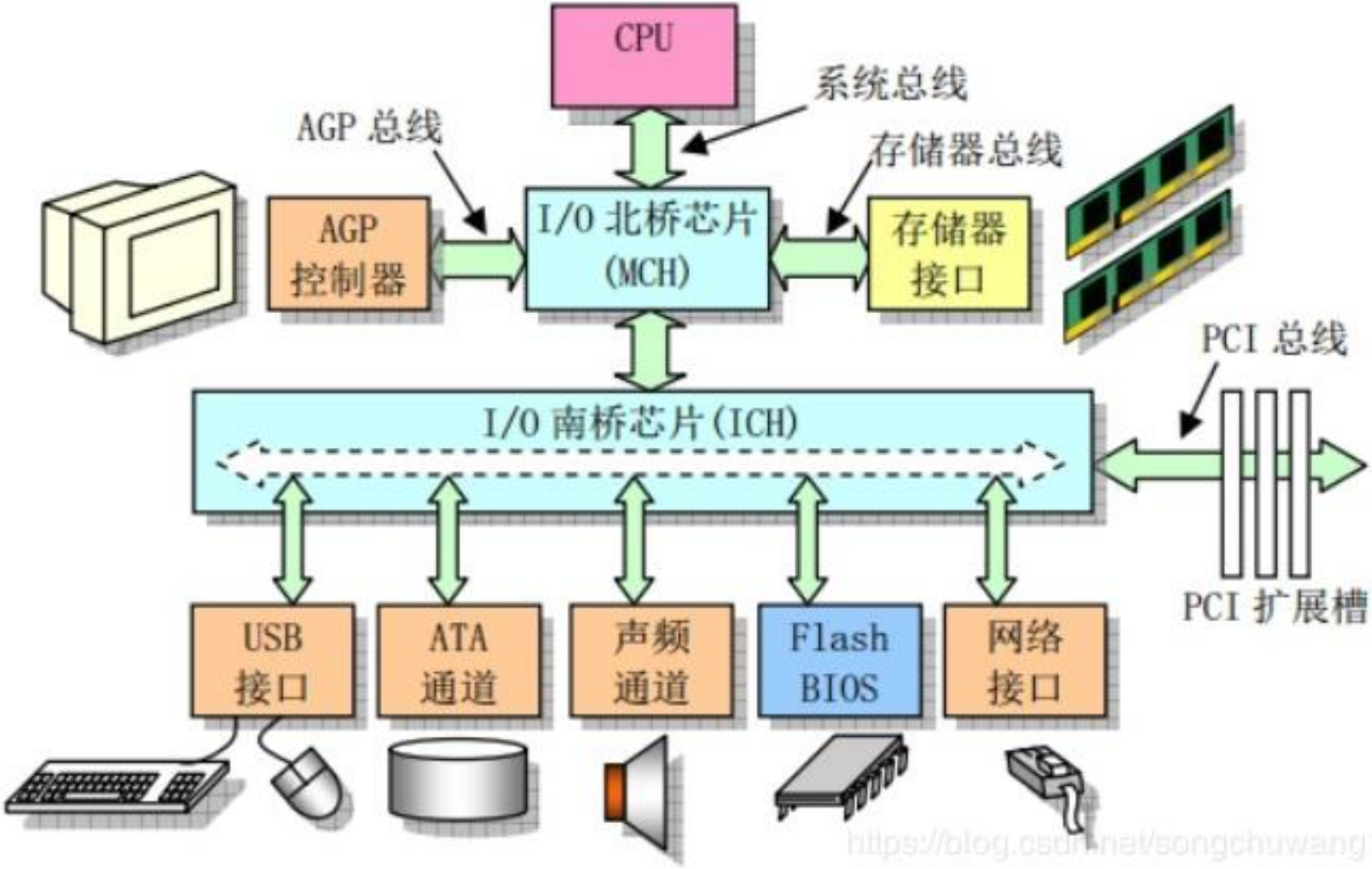
3. PCI 总线结构


系统总线
CPU
存储器
PCI 桥
标准总线
控制器
33 MHz的32位数据通路
PCI 总线
SCSI Ⅱ
控制器
8 MHz的16位数据通路
多媒体
高速局域网
高性能图形
ISA EISA
图文传真
调制解调器

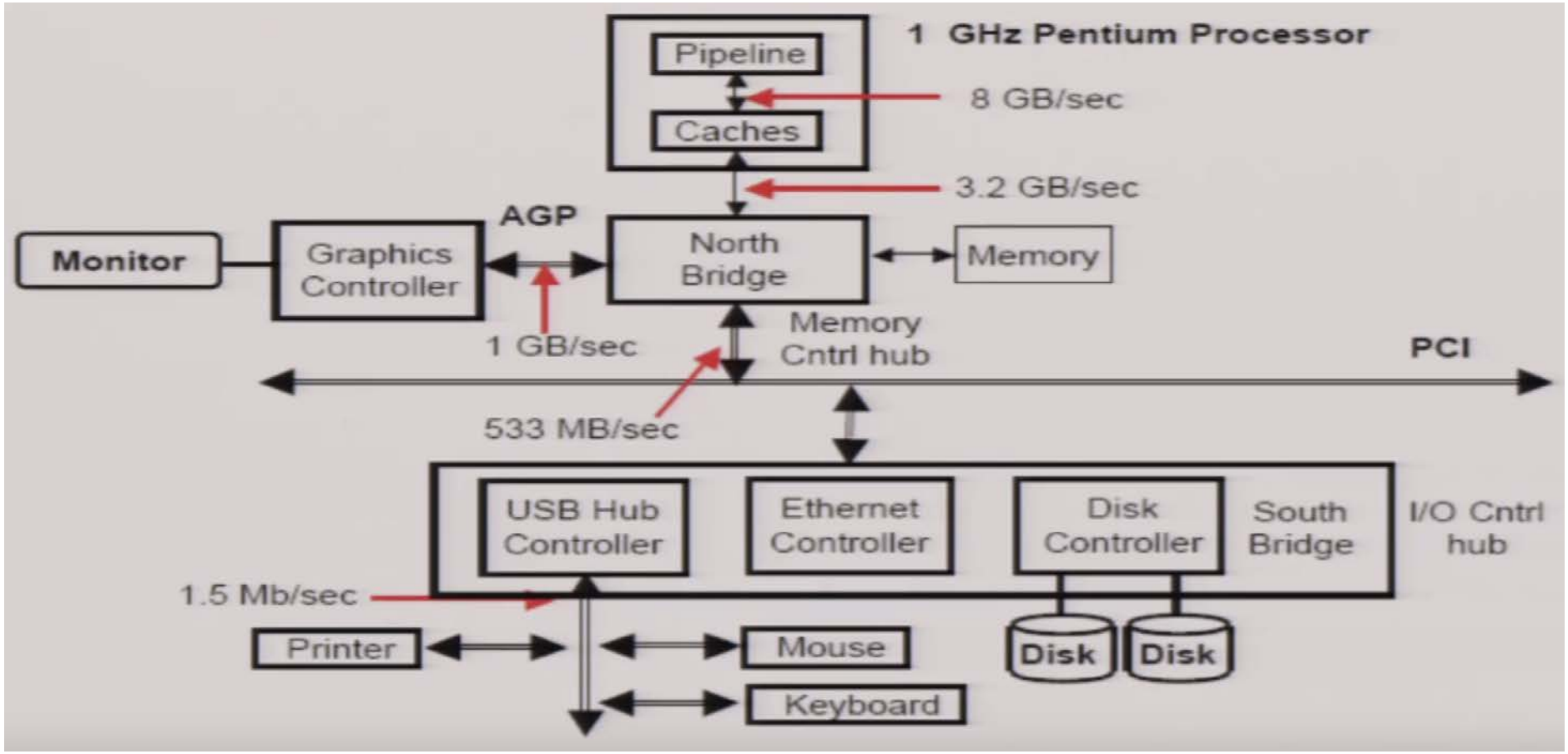
3.4 总线结构

4. 多层 PCI 总线结构

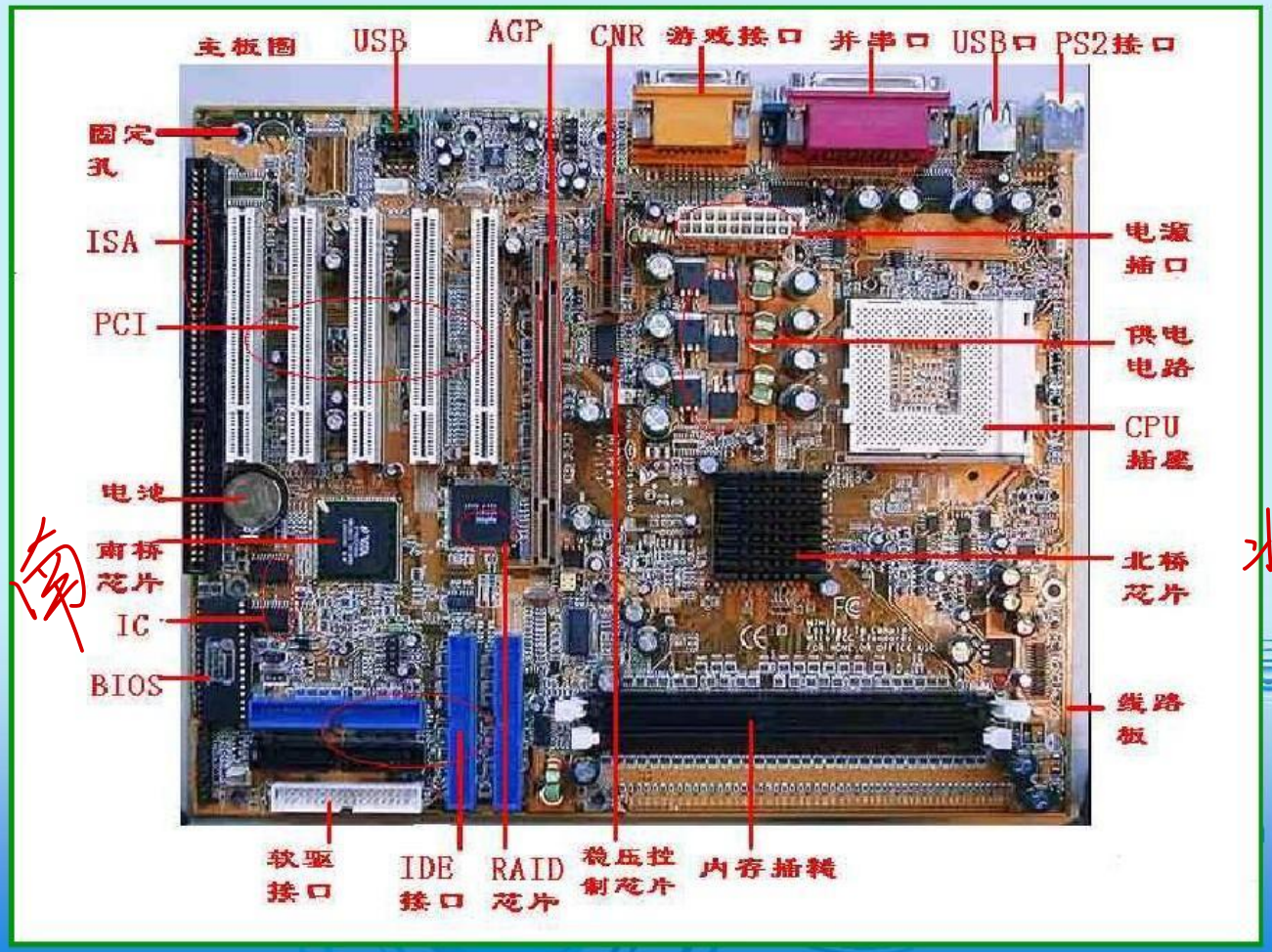




https://blog.csdn.net/songchuwang1868


1 GHz Pentium Processor
Pipeline
Caches
8 GB/sec
3.2 GB/sec
AGP
Monitor
Graphics Controller
North Bridge
Memory
1 GB/sec
Memory Cntrl hub
PCI
533 MB/sec
USB Hub Controller
Ethernet Controller
Disk Controller
South Bridge
I/O Cntrl hub
1.5 Mb/sec
Printer
Mouse
Keyboard
Disk
Disk

微型计算机典型系统主板结构图



3.5 总线控制

一、总线判优控制（仲裁）

判断优先级

1. 基本概念

- 主设备(模块)　　对总线有控制权
- 从设备(模块)　　对总线无控制权，仅响应从主设备发来的总线命令
- 总线判优　　在总线争用时，判定总线给哪对的设备或模块使用的过程

3.5 总线控制

2. 常见判优方式

- 总线判优控制
  - 集中式
    - 链式查询(菊花链, Daisy chain)
    - 计数器定时查询(Query by a counter)
    - 独立请求方式(Centralized, Parallel)
  - 分布式
    - 自举式 (Self-selection)
    - 冲突检测 (Collision detection)

°集中式：将控制逻辑做在一个专门的总线控制器或总线判优器中，通过将所有的总线请求集中起来利用一个特定的判优算法进行判优

°分布式：没有专门的总线控制器，其控制逻辑分散在各个部件或设备中

3.5 总线控制

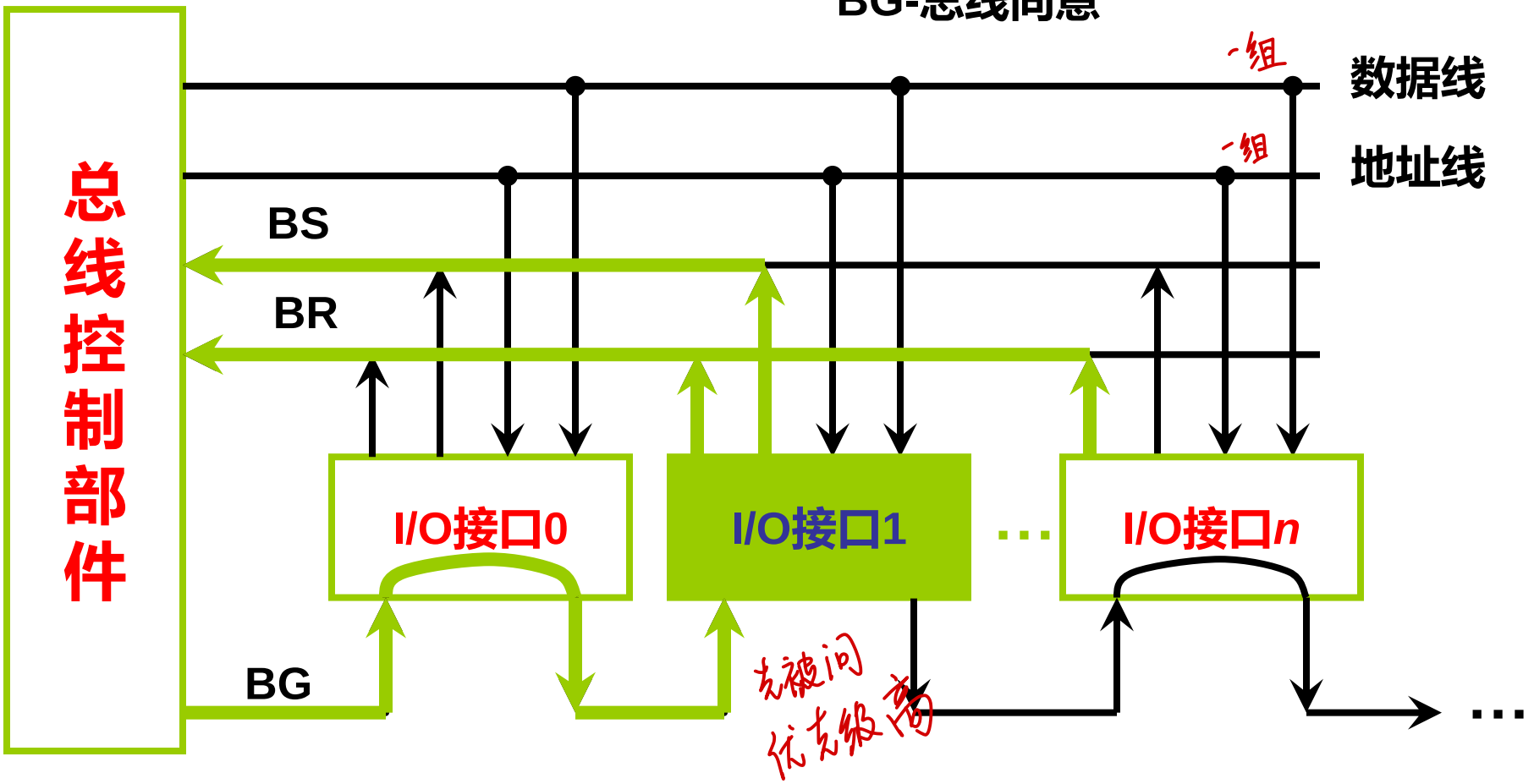
判优方案应在以下两个因素间进行平衡

- 等级性(Priority)—具有高优先级的设备应该先被服务
- 公平性(Fairness)—即使具有最低优先权的设备也不能永远得不到总线使用权

3.5 总线控制

(1) 链式查询方式

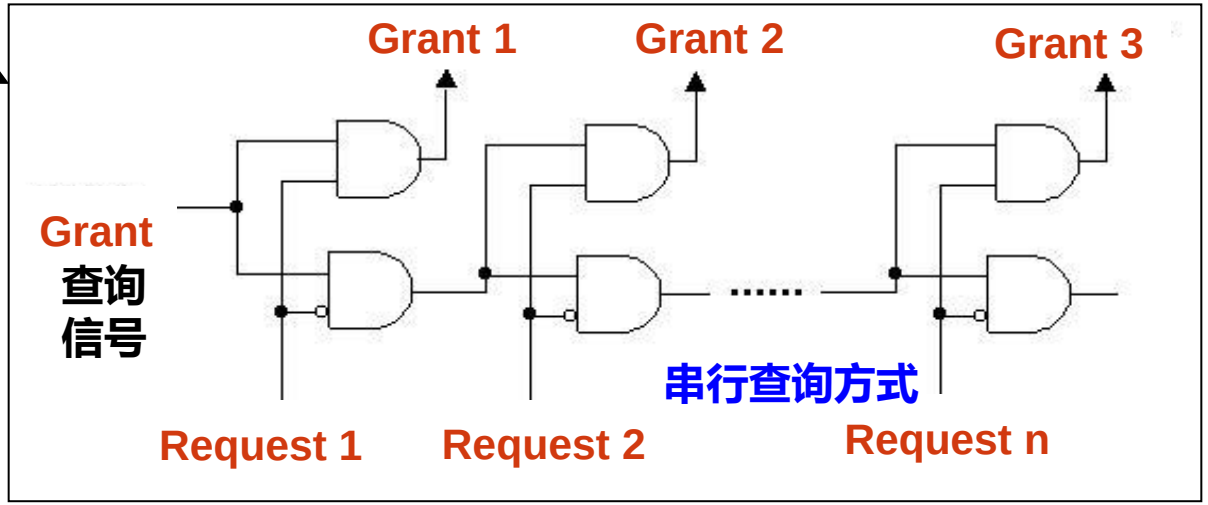
BS -总线忙
BR-总线请求
BG-总线同意



3.5 总线控制

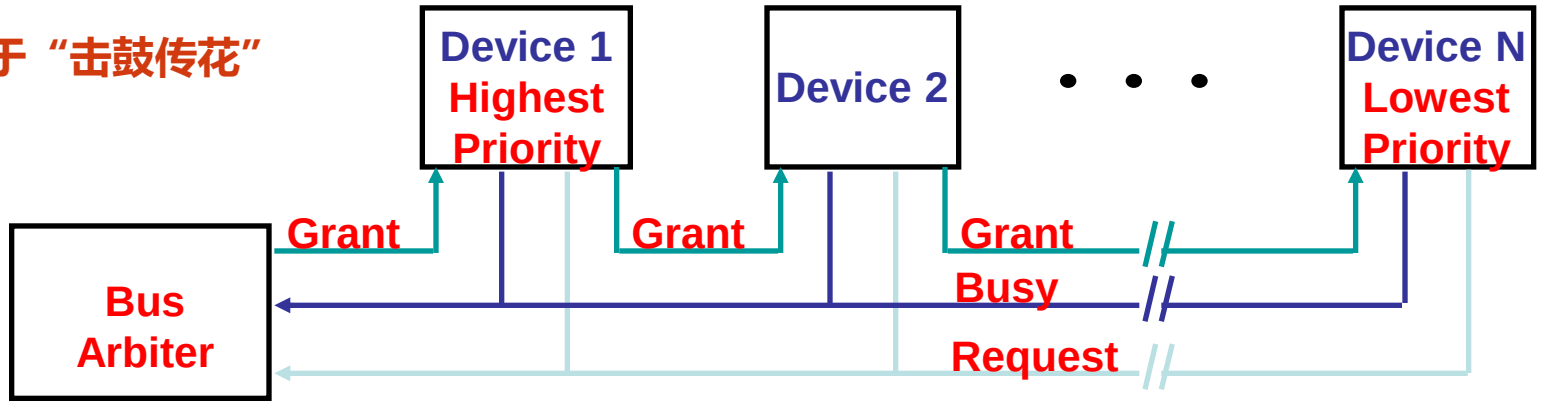
菊花链总线判优

菊花链查询电路



Grant从最高优先权的设备依次向最低优先权的设备串行相连。如果到达的设备有总线请求，则Grant信号就不再往下传，该设备建立总线忙Busy信号，表示它已获得了总线使用权。

相当于“击鼓传花”

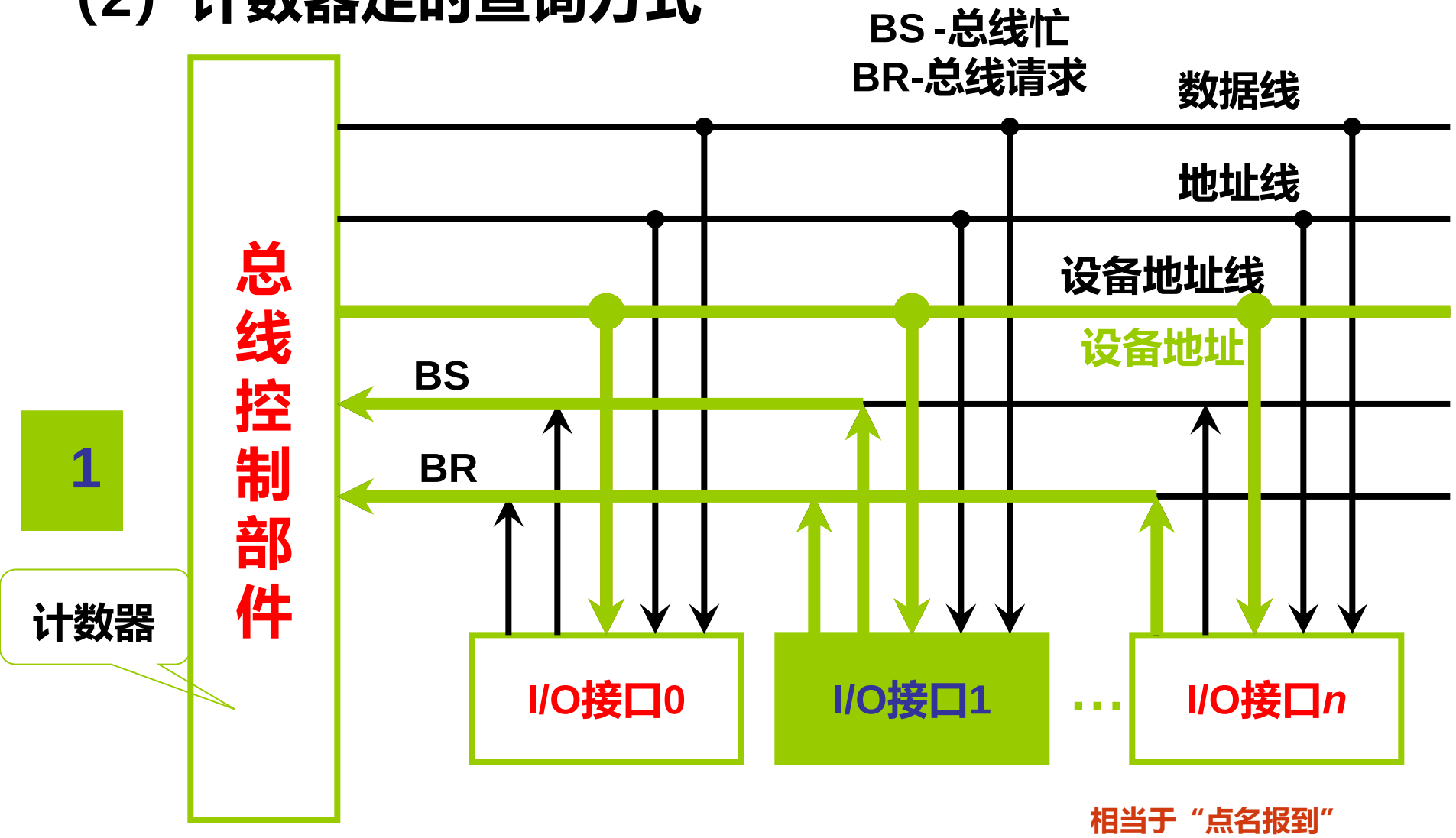


（1）链式查询方式

- 特点：
  - 线数少（2根），设备扩充容易
  - 离总线控制器最近的设备具有最高优先权，优先级低的设备难获得请求。
  - 对电路故障敏感 查询线断
  - 菊花链的使用限制了总线速度

3.5 总线控制

（2）计数器定时查询方式



(2) 计数器定时查询方式

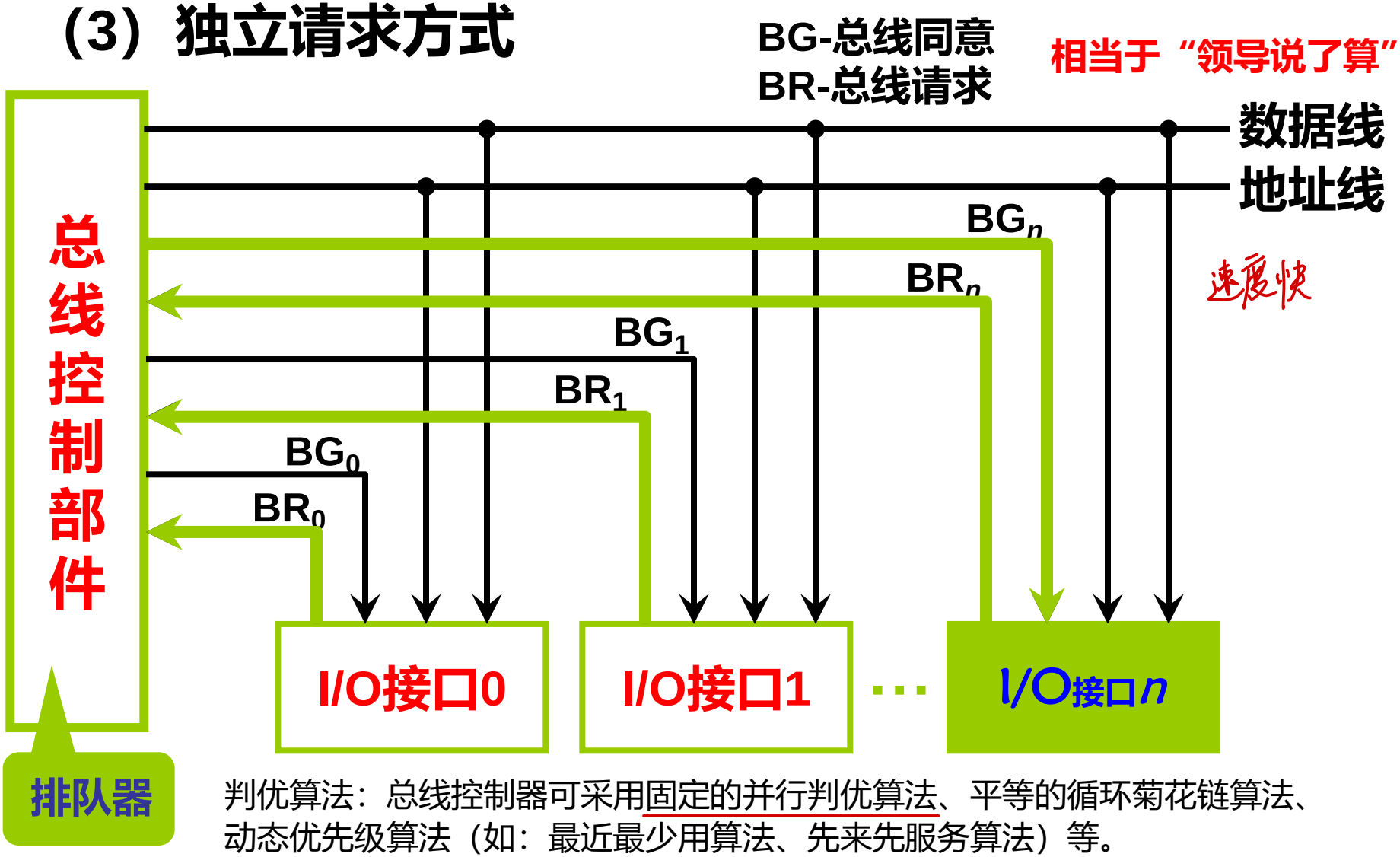
- 特点：

计数器
- 从“0”开始　与链式优先级相同
- 从终止点开始　循环优先级，各个部件优先级相等
- 初始值由程序设定

线数：$\log_2 n$　线多

3.5 总线控制

(3) 独立请求方式



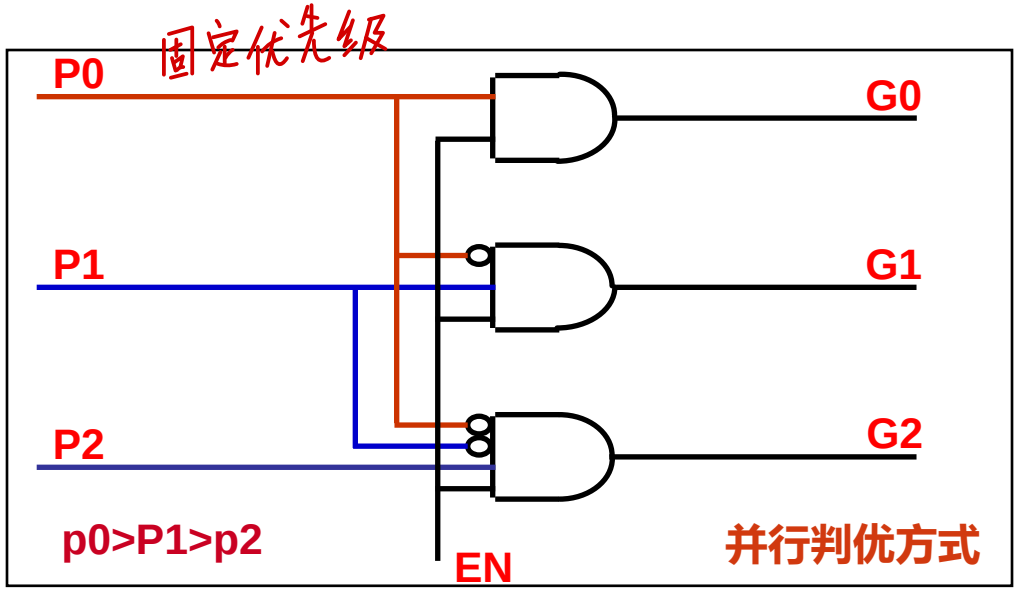
判优算法：总线控制器可采用固定的并行判优算法、平等的循环菊花链算法、动态优先级算法（如：最近最少用算法、先来先服务算法）等。

3.5 总线控制

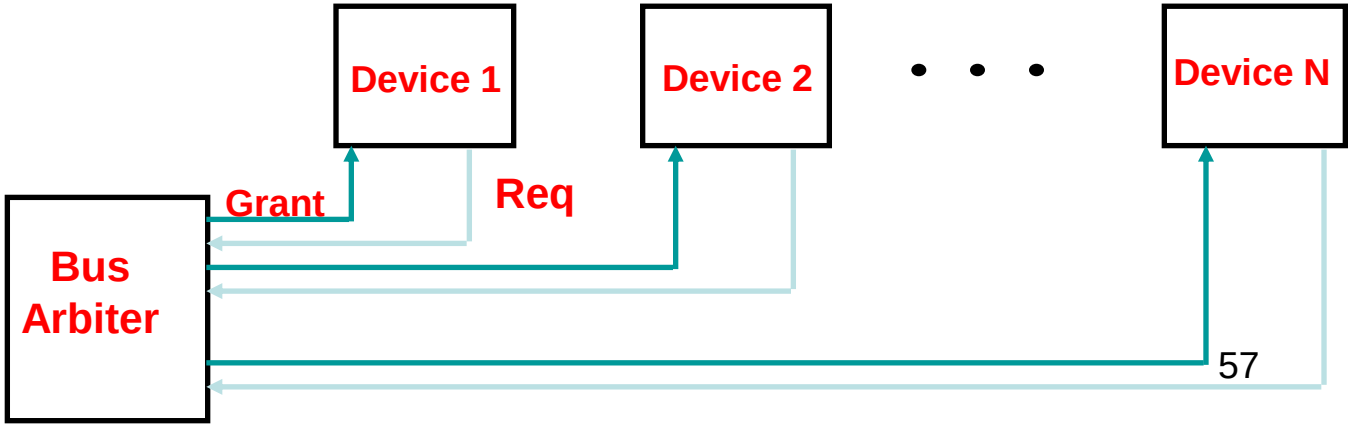
独立请求方式判优

并行判优电路

p0、P1、p2优先级怎样?



- 各设备都有一对总线请求线Req和总线允许线Grant。
- 当某设备要使用总线时，就通过对应的总线请求线将请求信号送到总线控制器。
- 总线控制器中有一个判优电路，可根据各设备的优先级确定选择哪个设备。控制器可给各请求线以固定的优先级，也可编程设置

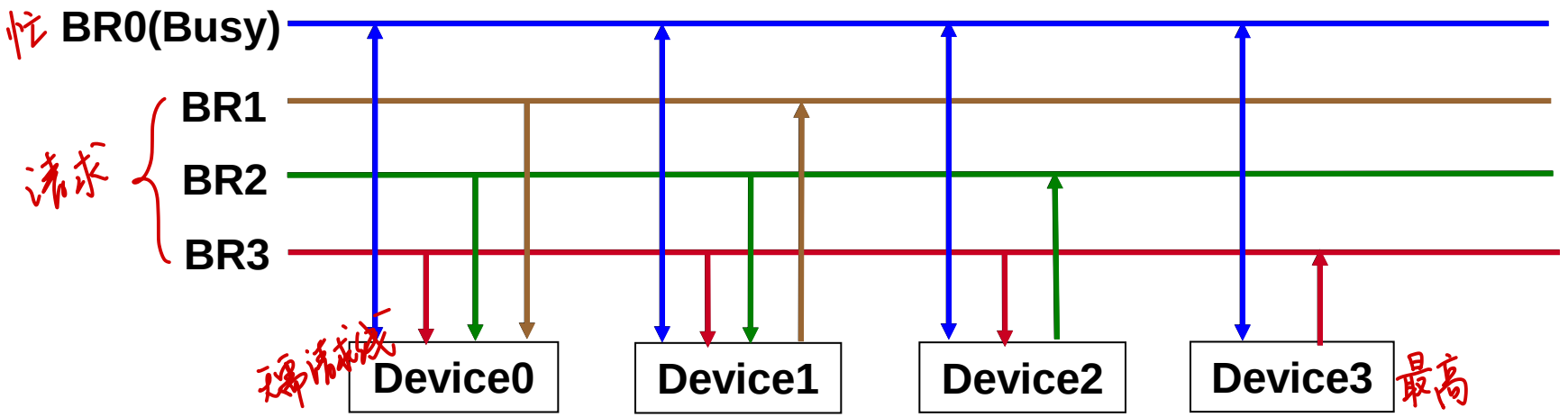


3.5 总线控制

(3) 独立请求方式

- 特点:
  - **响应速度快**
  - **优先控制灵活**
  - **线数多（2n根线），控制复杂**

（4）自举分布式判优



- 优先级固定，各设备独立决定自己是否是最高优先级请求者
- 需请求总线的设备在各自对应的总线请求线上送出请求信号
- 在总线判优期间每个设备将比自己优先级高的请求线上的信号取回分析：
  - 若有总线请求信号，则本设备不能立即使用总线
  - 若没有，则可以立即使用总线，并通过总线忙信号阻止其他设备使用总线
    - 最低优先级设备可以不需要总线请求线，为什么？
    - 需要较多连线用于请求信号，所以，许多总线用数据线DB作为总线请求线
      N个设备要多少请求信号？ N条！
    - NuBus（MacintoshII 中的底板式总线）、SCSI总线等采用该方案

上图中的优先级 (优先级)是什么？ 设备3>设备2>设备1>设备0

3.5 总线控制

冲突检测方式判优

基本思想：

- 当某个设备要使用总线时，它首先检查一下是否有其他设备正在使用总线
- 如果没有，那它就置总线忙，然后使用总线；
- 若两个设备同时检测到总线空闲，则可能会同时使用总线，此时发生冲突；
- 一个设备在传输过程中，它会帧听总线以检测是否发生了冲突；
- 当冲突发生时，两个设备都会停止传输，延迟一个随机时间后再重新使用总线
  - 该方案一般用在网络通信总线上，如：Ethernet总线等。

3.5 总线控制

二、总线通信控制

交换信息数据

1. 目的　　解决通信双方协调配合问题

2. 总线周期

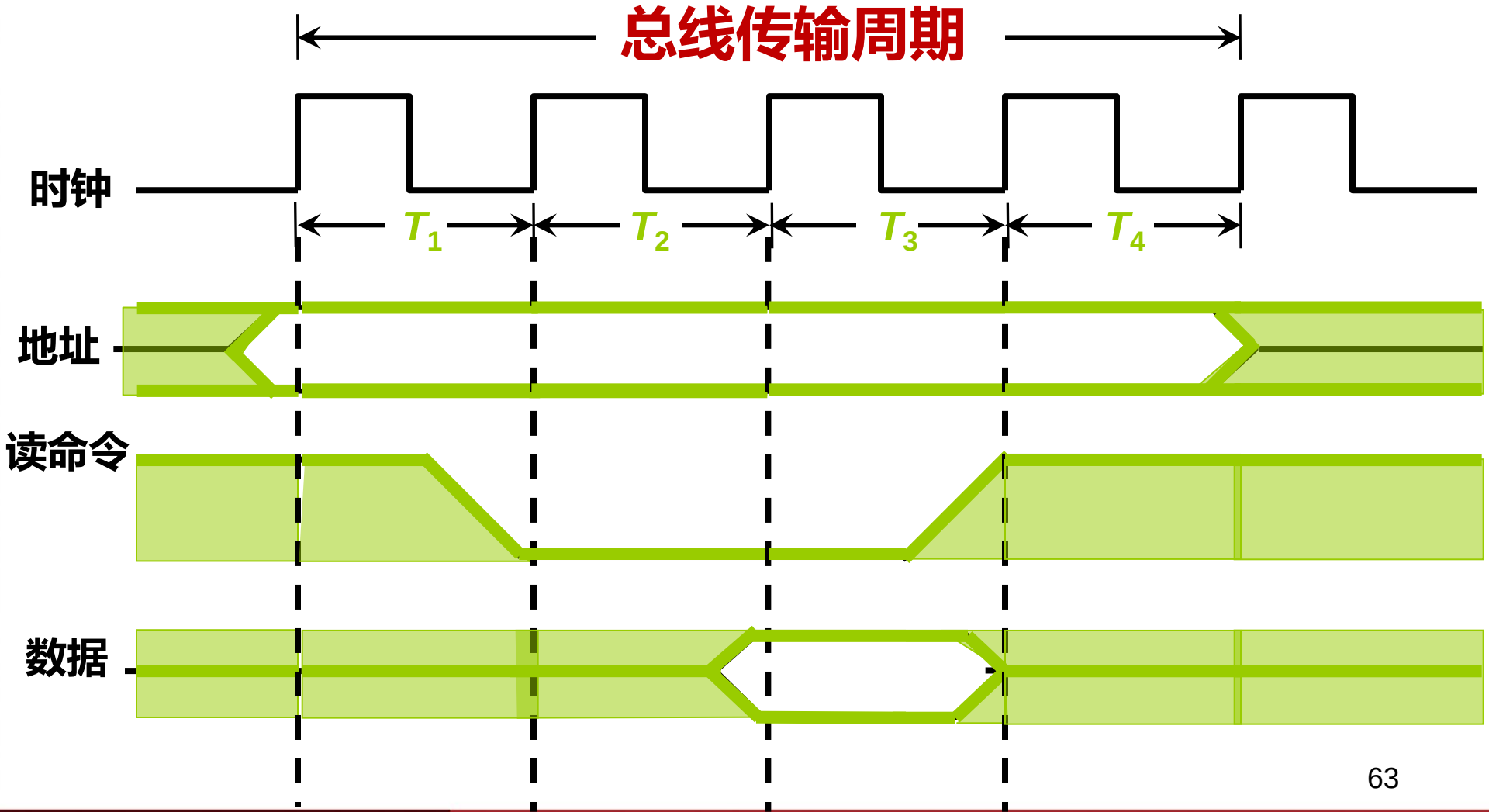
- 申请分配阶段　　主模块申请，总线判优决定
- 寻址阶段　　主模块向从模块给出地址和命令
- 传数阶段　　主模块和从模块交换数据
- 结束阶段　　主模块撤销有关信息

3.5 总线控制

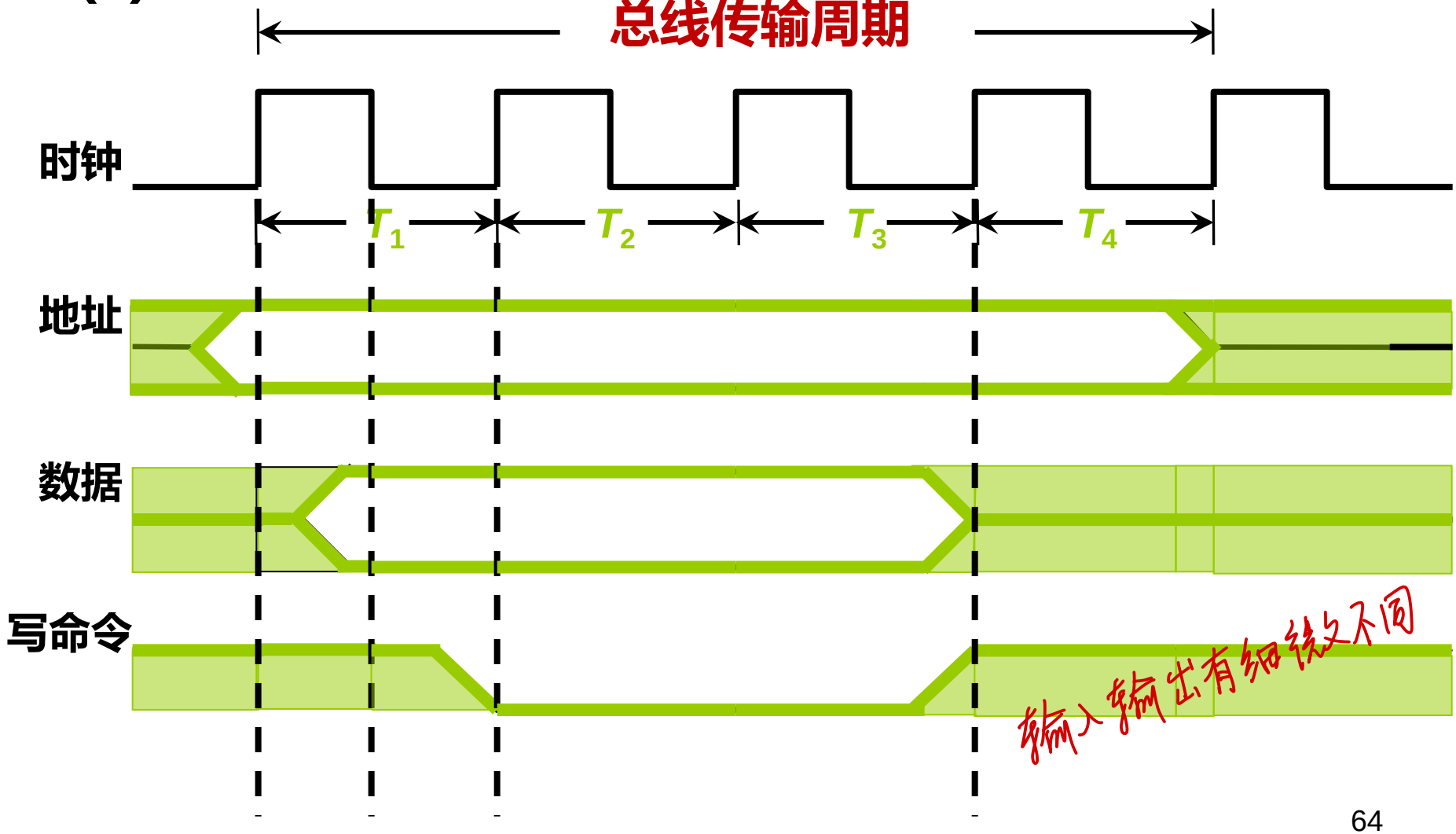
3. 总线通信的四种方式(总线定时方式)

- **同步通信 (synchronous)**
  - 由 **统一时标** 控制数据传送
  - 其控制线上有一个时钟和一个固定的与该时钟相关的通信协议
- **异步通信 (asynchronous)**
  - 采用 **应答方式**，没有公共时钟标准
  - 握手协议（handshaking protocol）
- **半同步通信**
  - **同步、异步结合**
- **分离式通信 (拆分总线事务, Split Bus Transaction)**
  - 充分 **挖掘** 系统 **总线每瞬间** 的 **潜力**

(1) 同步式数据输入



(2) 同步式数据输出



同步通信

- 优点
  - 规定明确、统一，模块间的配合简单一致。
  - 控制逻辑少而速度快
- 缺点
  - 主从模块时间配合属于强制性“同步”，必须在限定时间内完成规定要求。
  - 对所有从模块用同一时限，就必须按照最慢的部件来设计公共时钟，影响了总线的工作效率，也给设计带来了局限性，缺乏灵活性。

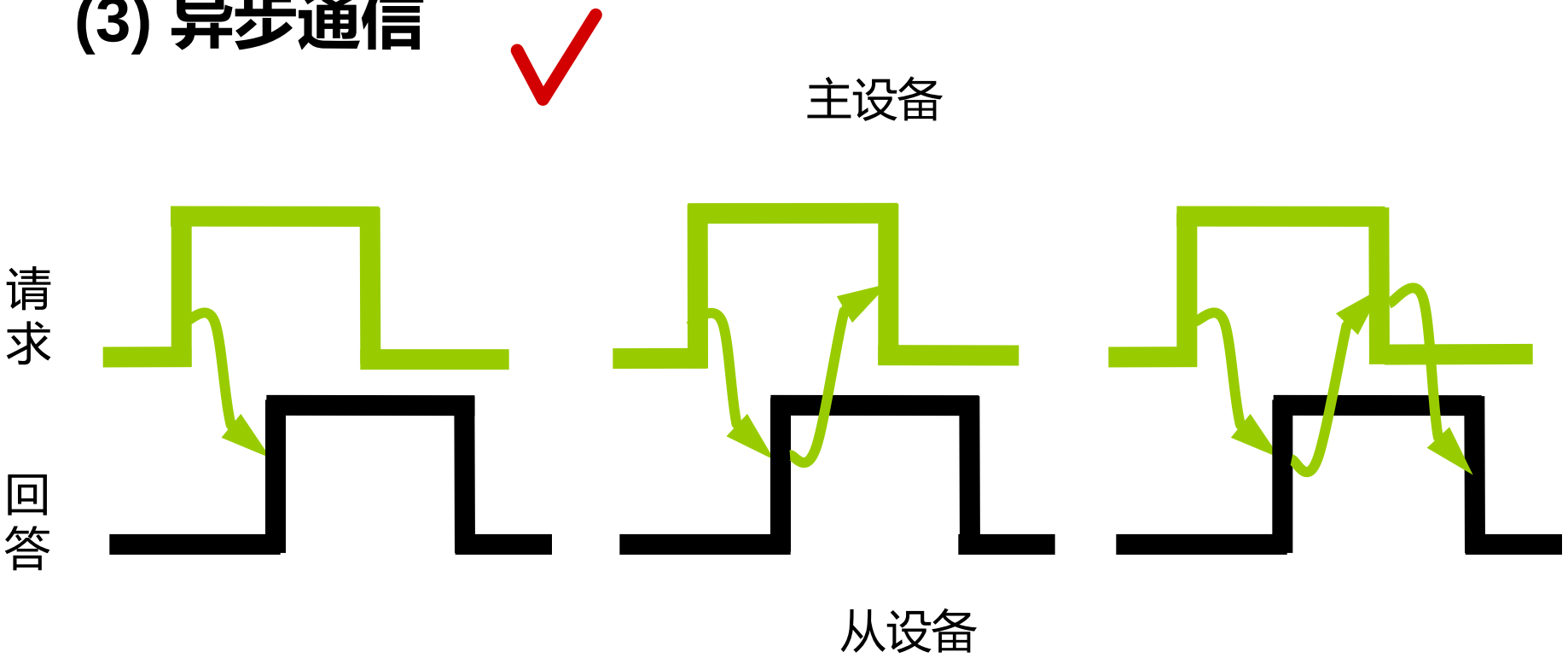
3.5 总线控制

(3) 异步通信

- 非时钟定时，没有一个公共的时钟标准。因此，能够连接带宽范围很大的各种设备。总线能够加长而不用担心时钟偏移（clock skew）问题。
- 采用应答方式，允许各模块速度不一致。
- 主模块发出请求信号（Request）时，一直等待从模块反馈回来的应答信号(Acknowledge)，才开始通信。这需要主从模块之间增加两条应答线（即握手交互信号线Handshaking）。
- 分为不互锁、半互锁和全互锁

3.5 总线控制

(3) 异步通信



3.5 总线控制

(3) 异步通信

- 优点:
  - 灵活，可挂接各种具有不同工作速度的设备
- 缺点:
  - ① 对噪声较敏感（任何时候都可能接收到对方的应答信号）
  - ② 接口逻辑较复杂

(3) 异步通信

- 异步串行通信用波特率来衡量数据传输率

例1.在异步串行传输系统中，假设每秒可传输20个数据帧，一个数据帧包含1个起始位、7个数据位、1个奇校验位、1个结束位。计算其波特率和比特率。

解：波特率=(1+7+1+1)×20=200bps（位/秒）

比特率=20×7=140bps

课后练习

1. 在异步串行传输系统中，若每个数据帧包含1个起始位、8个数据位、1个奇校验位、1个结束位。比特率为160bps，求波特率。

(220bps)

2. 在异步串行传输系统中，若每个数据帧包含1个起始位、8个数据位、1个奇校验位、1个结束位。波特率为1200bps，求比特率。

(872.72bps)

3.5 总线控制

(4) 半同步通信

(同步、异步 结合)

同步　　发送方 用系统 时钟前沿 发信号

接收方 用系统 时钟后沿 判断、识别

异步　允许不同速度的模块和谐工作

增加一条　“等待”响应信号　$\overline{\text{WAIT}}$

3.5 总线控制

以输入数据为例的半同步通信时序

T_1　主模块发地址

T_2　主模块发命令　下降沿时

T_W　当　$\overline{\text{WAIT}}$　为低电平时，等待一个 T

T_W　当　$\overline{\text{WAIT}}$　为低电平时，等待一个 T

⋮

T_3　从模块提供数据

T_4　从模块撤销数据，主模块撤销命令

见课本P65图3.22

3.5 总线控制

上述三种通信的共同点

一个总线传输周期（以输入数据为例）

- 主设备发地址、命令　　占用总线
- 从设备准备数据　　不占用总线　　总线空闲
- 从设备向主设备发数据　　占用总线

3.5 总线控制

(5) 分离式通信

目的：充分挖掘系统总线每瞬间的潜力

一个总线传输周期

子周期1　主设备申请占用总线，使用完后即 放弃总线 的使用权

子周期2　从设备申请 占用总线，将各种信息送至总线上

主设备

3.5 总线控制

原理：将一个事务分成两个子过程：

- 过程1：主控设备A获得总线使用权后，将请求的事务类型、地址及其他信息（如A的标识等）发到总线，从设备B记下这些信息。A发完信息后便立即释放总线，其他设备便可使用总线；
- 过程2：B收到A发来的信息后，按照A的要求准备数据，准备好后，B便请求使用总线，获使用权后，B将A的编号及所需数据送到总线，A便可接收。

3.5 总线控制

CPU 从 磁盘读取数据的过程：

CPU向通道发出定位信息； 约几微秒；占用总线

通道指挥磁盘定位，先定位磁道，再定位扇区； 约几十毫秒；总线空闲

开始传送数据；约几微妙；占用总线

传送结束；

采用分离式通信：

CPU向通道发出定位信息；几微秒；占用总线

通道收到定位信息，指挥指挥磁盘定位；约几十毫秒；释放总线

通道装备好信息，申请占有总线；约几微秒；

传送数据；约几微妙；占用总线

传送结束；

3.5 总线控制

分离式通信特点

1. 各设备有权申请占用总线

2. 采用同步方式通信，不等对方回答

3. 各设备准备数据时，不占用总线

4. 总线被占用时，无空闲

充分发挥了总线的有效占用

积土为山
积水为海

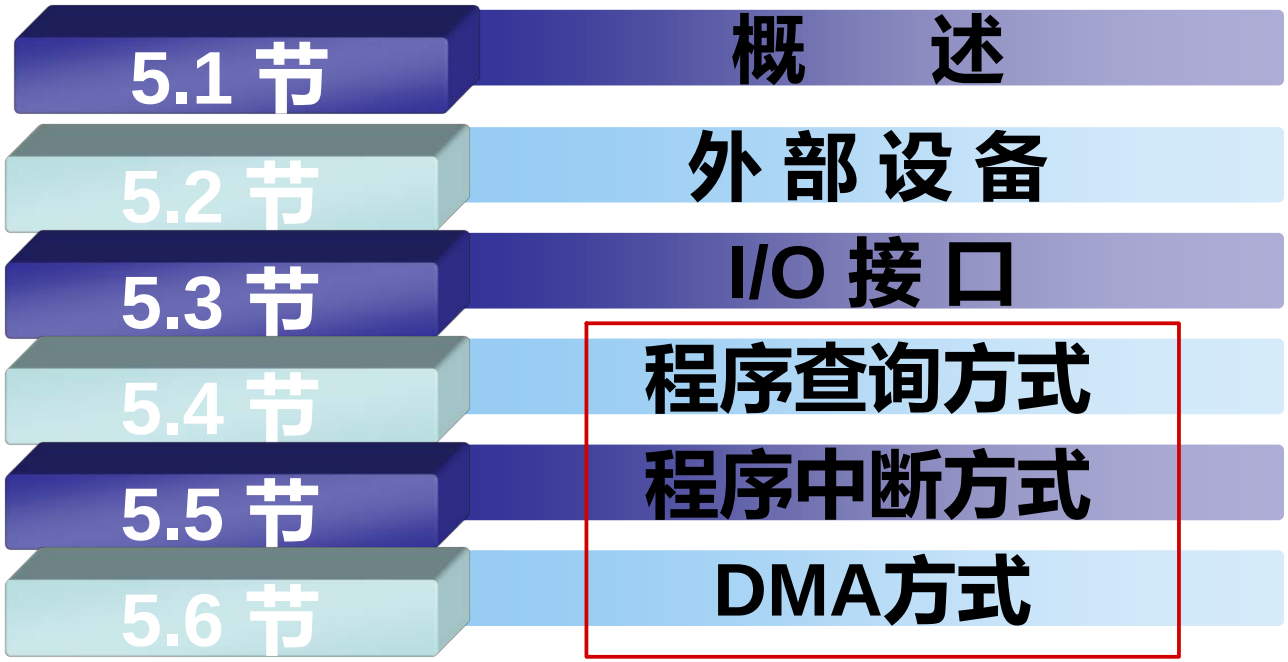
计算机组成原理

第五章 输入输出系统

阙夏
计算机与信息学院

2020/11/4

第五章　输入输出系统



大纲要求

(一) I/O系统基本概念

(二) 外部设备

1. 输入设备：键盘、鼠标等
2. 输出设备：显示器、打印机 等
3. 辅助存储器：硬盘存储器、磁盘阵列、光盘存储器等

(三) I/O接口（I/O控制器）

1. I/O接口的功能和基本结构
2. I/O端口及其编址

(四) I/O方式

1. 程序查询方式
2. 程序中断方式

中断的基本概念；

中断响应过程；

中断处理过程；

多重中断和中断屏蔽的概念。

3. DMA方式

DMA控制器的组成；DMA传送过程。

4. 通道方式

大纲要求

七、输入输出(I/O)系统

(一) I/O 系统基本概念

(二) 外部设备

1. 输入设备:键盘、鼠标
2. 输出设备:显示器、打印机
3. 外存储器:硬盘存储器、磁盘阵列

(三) I/O 接口(I/O 控制器)

1. I/O 接口的功能和基本结构
2. I/O 端口及其编址

(四) I/O 方式

1. 程序查询方式
2. 程序中断方式

中断的基本概念,中断响应过程,中断处理过程,多重中断和中断屏蔽的概念。

3. DMA 方式

DMA 控制器的组成,DMA 传送过程。

输入输出系统

- **定义：**是计算机系统中的**主机**与**外部**进行通信的系统。
- **组成：**它由**外部设备**和**I/O控制系统（接口、总线和管理软件）**两部分组成，是计算机系统的重要组成部分。
- **外部设备：**包括**I/O设备**和**辅助存储设备**（如如磁盘存储器、磁带存储器、光盘存储器等）。
- 从某种意义上也可以把磁盘、磁带和光盘等设备看成一种I/O设备，所以**I/O设备**与**外部设备**这两个名词经常是通用的。
- 在计算机系统中，通常把**CPU**和**主存**之外的部分称为**I/O系统**。
- **I/O系统特点**是：**异步性**、**实时性**和**设备无关性**。

5.1 概述

一、输入输出系统的发展概况（四阶段）

1. 早期阶段　分散连接　（程序查询方式）

CPU 和 I/O **串行** 工作，I/O通过CPU与主机信息交换。

2. 接口模块和DMA（Direct Memory Access） 阶段

总线连接
- 中断方式　需要CPU
- DMA 方式　不通过CPU

CPU 和 I/O **并行** 工作，I/O通过接口与主机信息交换 。

5.1 概述

3. 具有通道(Channel)结构的阶段

通道：一种专门用于输入输出操作的部件，是一种具有特殊功能的处理器。CPU启动通道后，可以继续运行自己的程序，而通道则同时执行通道程序，控制I/O设备进行直接数据传送。

4. 具有 I/O 处理机的阶段

I/O处理机（Peripheral Processor ）方式比通道方式具有更强的独立性和专用性。IOP有自己的**指令系统**，可以通过对I/O处理机的编程，实现独立于CPU的I/O操作。

二、输入输出系统的组成

1. I/O 软件

主要任务：

- 将用户编制的程序（数据）输入到主机内；
- 将运算结果输送给用户；
- 实现输入输出系统与主机工作的协调等。

(1) I/O 指令 CPU 指令的一部分

操作码	命令码	设备码

操作码：I/O指令与其它指令的判别码

命令码：I/O设备具体操作

设备（地址）码：用于从多台I/O设备中选择某台设备

5.1 概述

(2) 通道指令（Channel Control Word）

通道自身的指令；

指出数据组的首地址、传送字数或末地址、操作命令；

如 IBM/370 通道指令为 64 位。

2. I/O 硬件

设备　　I/O 接口　　（总线型）

设备　　设备控制器　　通道　　（通道型）

三、I/O 与主机的联系方式

1. I/O 编址方式

(1)统一编址: 指把I/O地址（I/O端口地址）当作存储器的单元进行分配。

在这种方式下CPU不需设置专门的I/O指令，用统一的访问存储器的取数、存数指令就可访问I/O端口。

端口一寄存器

优点： 1）使CPU访问I/O的操作更灵活、更方便，

2）使端口有较大的编址空间。

缺点：1）使端口占用了存储器地址，使内存容量变小。

2）执行速度较慢。

5.1 概述

(2) 不统一编址（单独编址方式）：

指I/O端口地址与存储器地址无关，另行单独编址。

在这种方式下，CPU需要设置专门的I/O指令访问端口。

优点：程序编制清晰、便于理解。

缺点：1）输入输出指令少；

2）增加控制的复杂性。

5.1 概述

2. 设备寻址

每台设备有一个设备号，I/O指令设备码字段指出设备号，用**设备选择电路**识别设备是否被选中。

3. 传送方式

(1) 串行：在主机与接口之间完全按照并行方式实现数据传输，但在接口与I/O设备之间是按照串行方式实现数据传输。（在串行接口中必须设置具有移位功能的数据缓冲器，实现数据格式的串－并转换。）（适用于低速的串行I/O设备及信息远距离传送设备。）

(2) 并行：在主机与接口、接口与I/O设备之间是按照并行方式完成信息传输。（当I/O设备本身按照并行方式工作，主机与I/O设备间距离较近，选用并口）

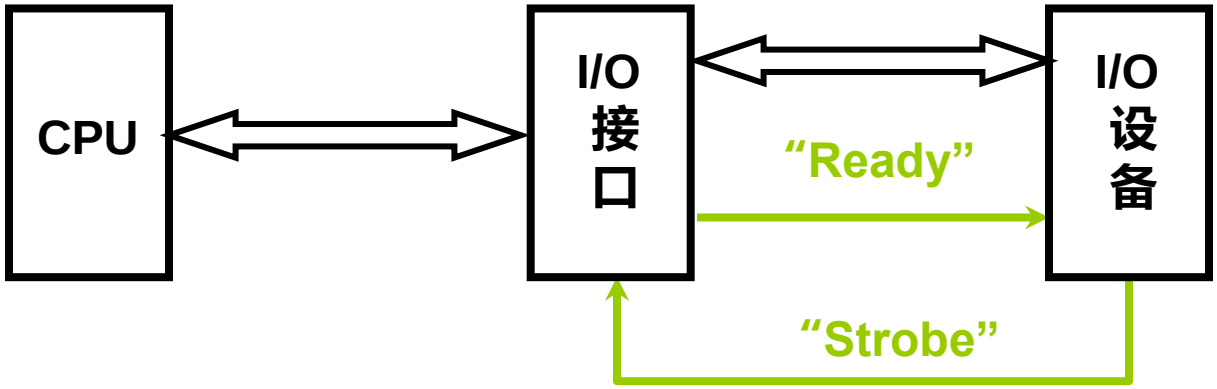
5.1 概述

4. 联络方式

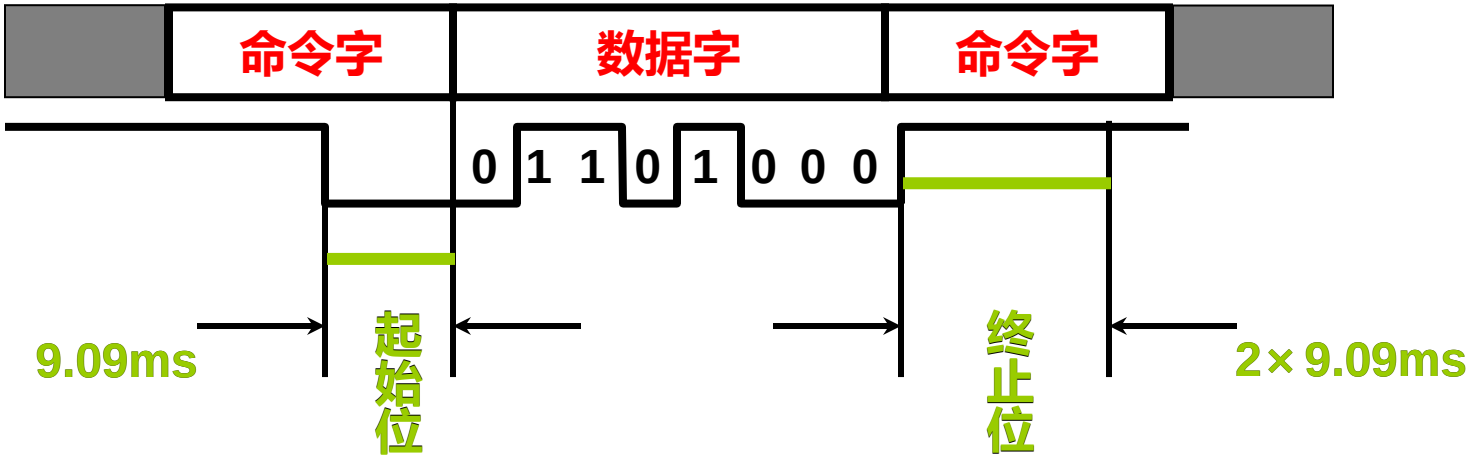
(1) 立即响应 (功能简单的外设)

(2) 异步工作采用应答信号 (I/O设备与主机工作速度不匹配)

并行



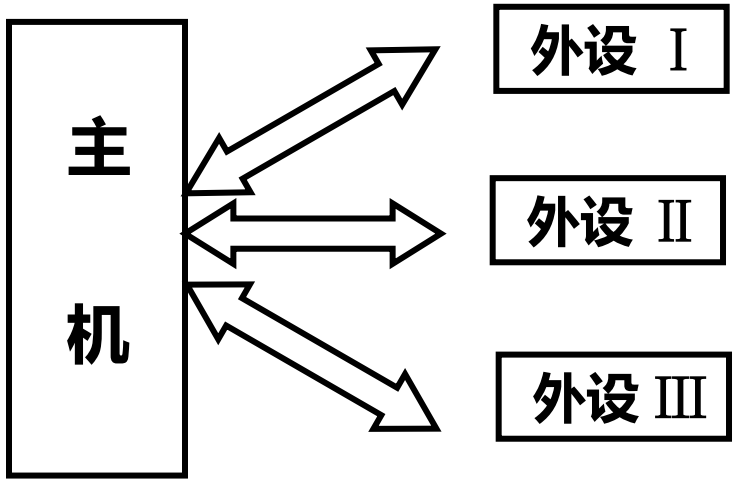
串行



(3) 同步工作采用同步时标

5. I/O 与主机的连接方式

(1) 辐射式连接



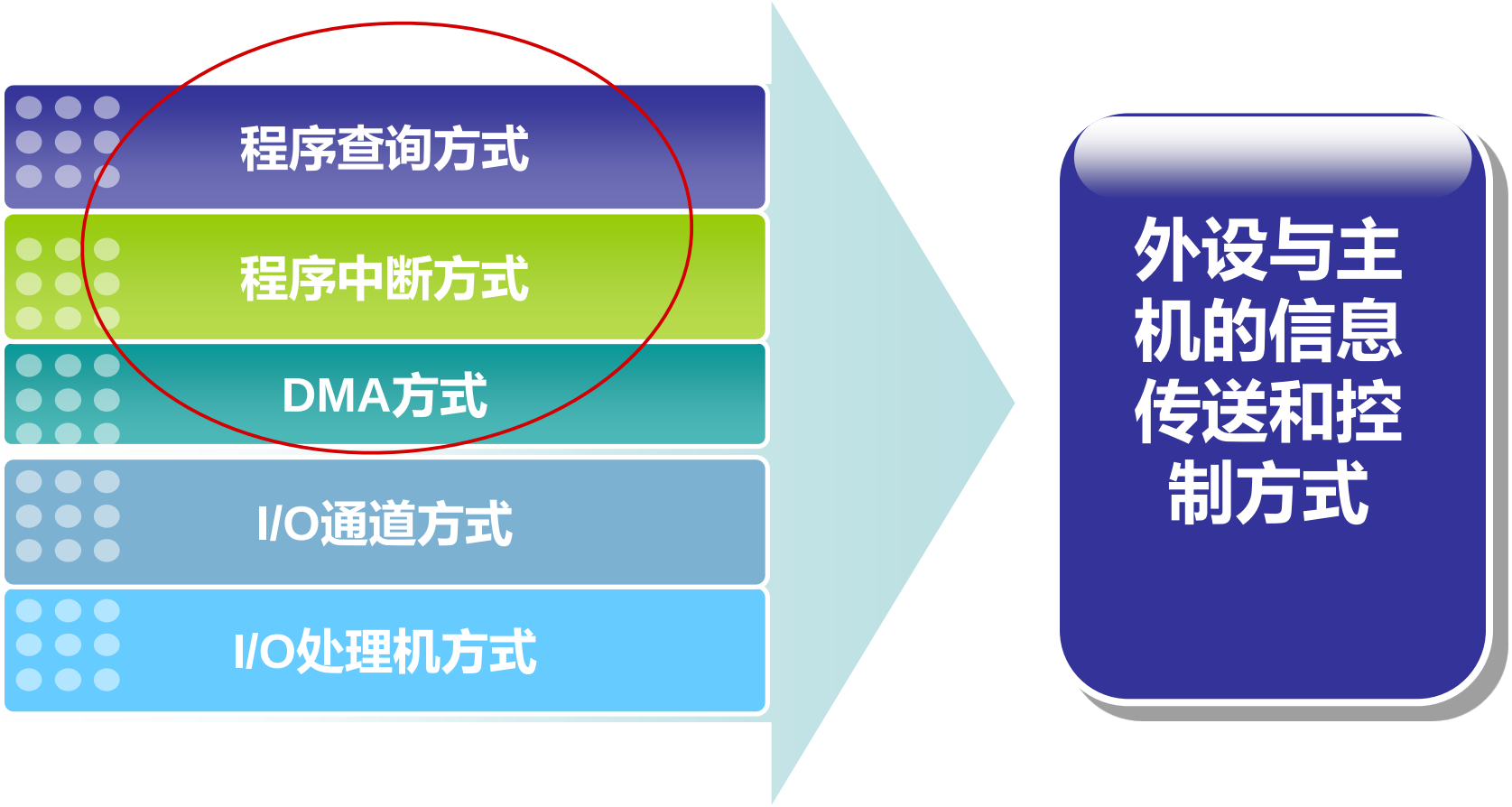
每台设备都配有一套控制线路和一组信号线

不便于增删设备

(2) 总线连接

便于增删设备

四、I/O 与主机信息传送的控制方式

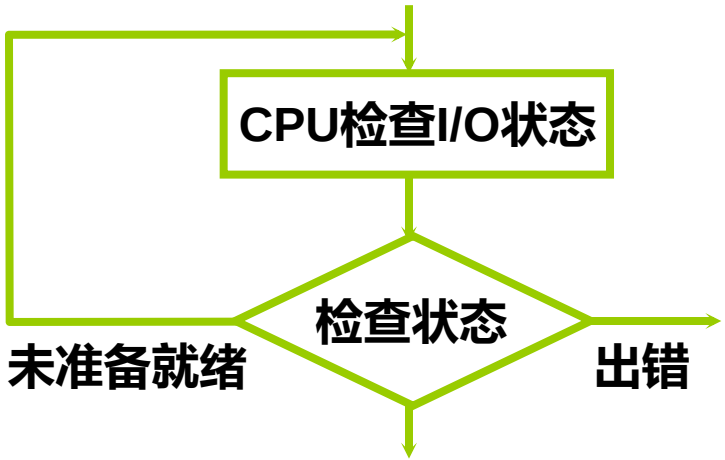


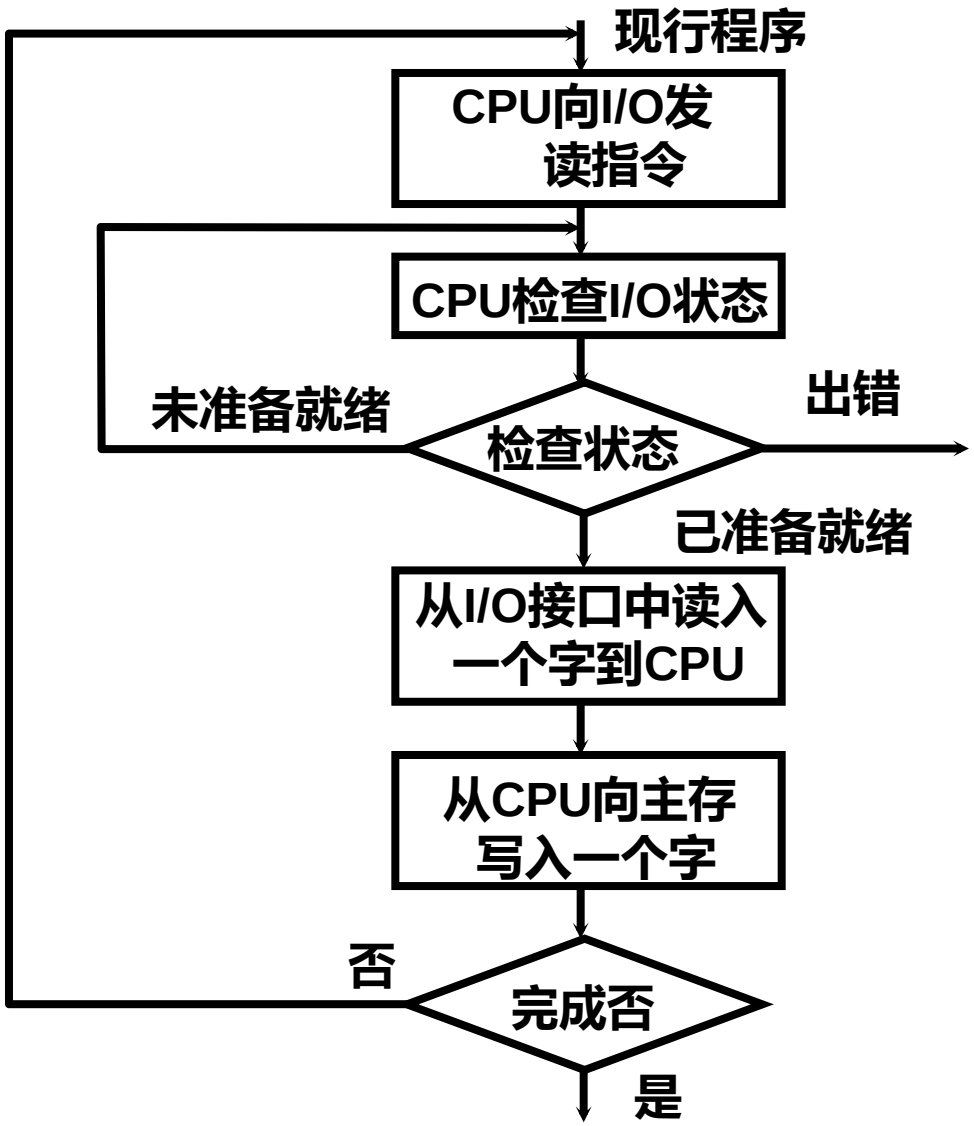
5.1 概述

1. 程序查询方式

CPU 和 I/O 串行工作

踏步等待





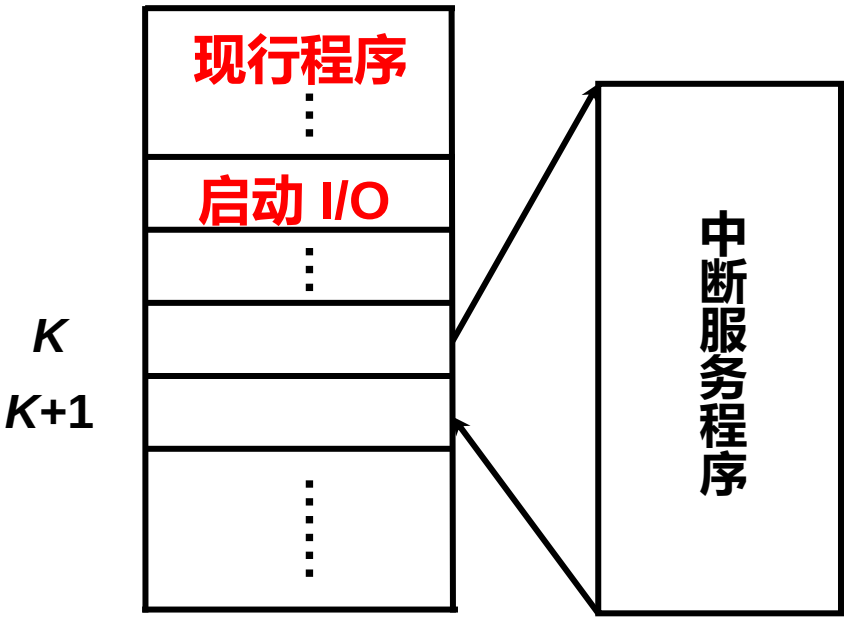
5.1 概述

2. 程序中断方式

适用随机出现的情况

I/O 工作
- 自身准备 —— CPU 不查询
- 与主机交换信息 —— CPU 暂停现行程序

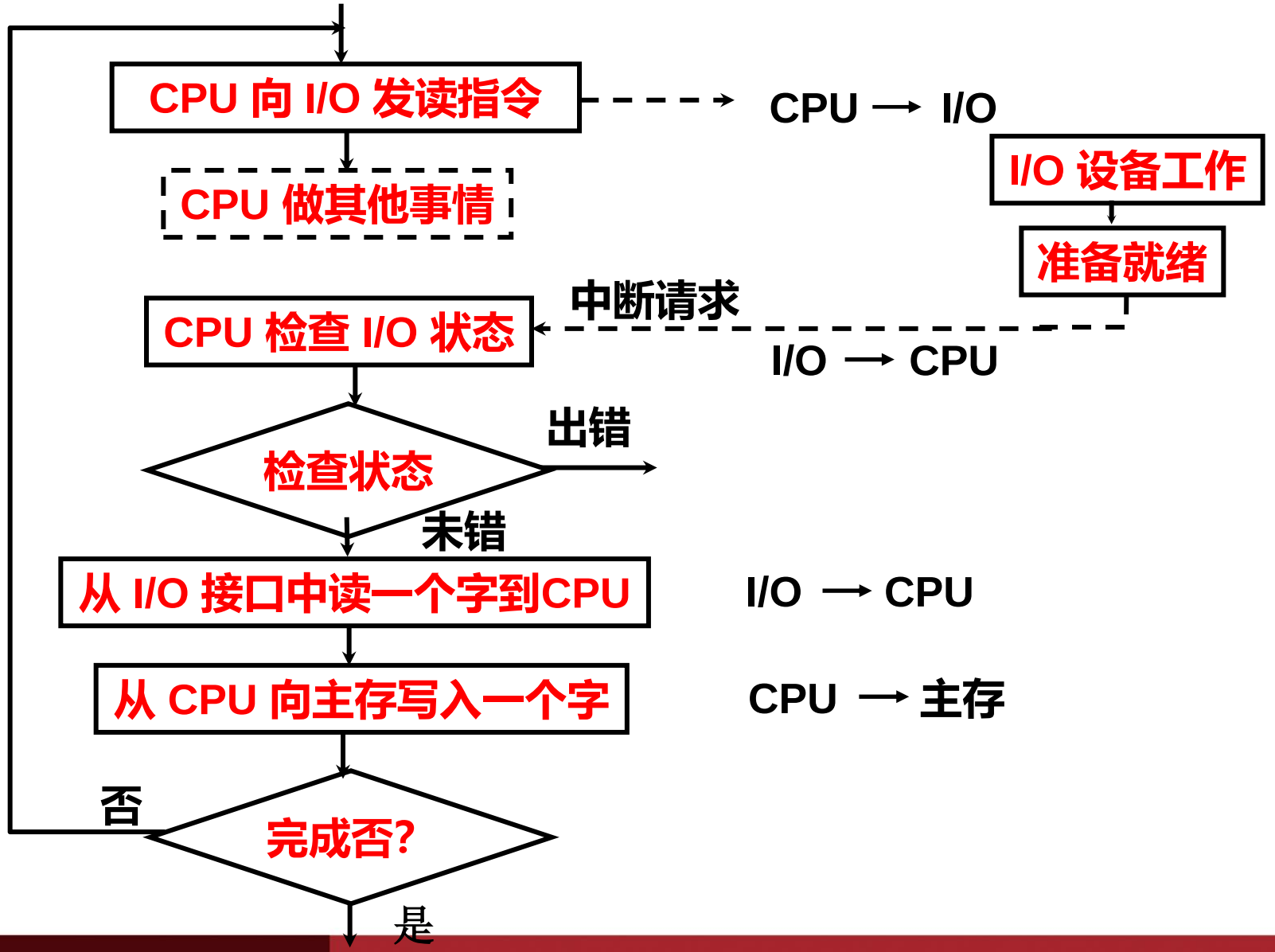
CPU 和 I/O 部分的并行工作



没有踏步等待现象

中断现行程序

程序中断方式流程（读）5.1 概述



3. DMA 方式

- 直接存储器存取(Direct Memory Access, DMA)方式：一种直接依靠**硬件**在主存与设备之间进行数据传送，在数据传送期间**不需要CPU**程序干预的与I/O设备交换数据的工作方式。
- 在这种方式中，**DMA控制器**从CPU手中暂时**接管**对总线的控制，数据交换不必经过CPU而直接通过数据总线在内存和I/O设备之间直接进行。
- **DMA方式**几乎是目前所有计算机系统都要具备的一种重要工作方式。

3. DMA 方式

主存和 I/O 之间有一条直接数据通道

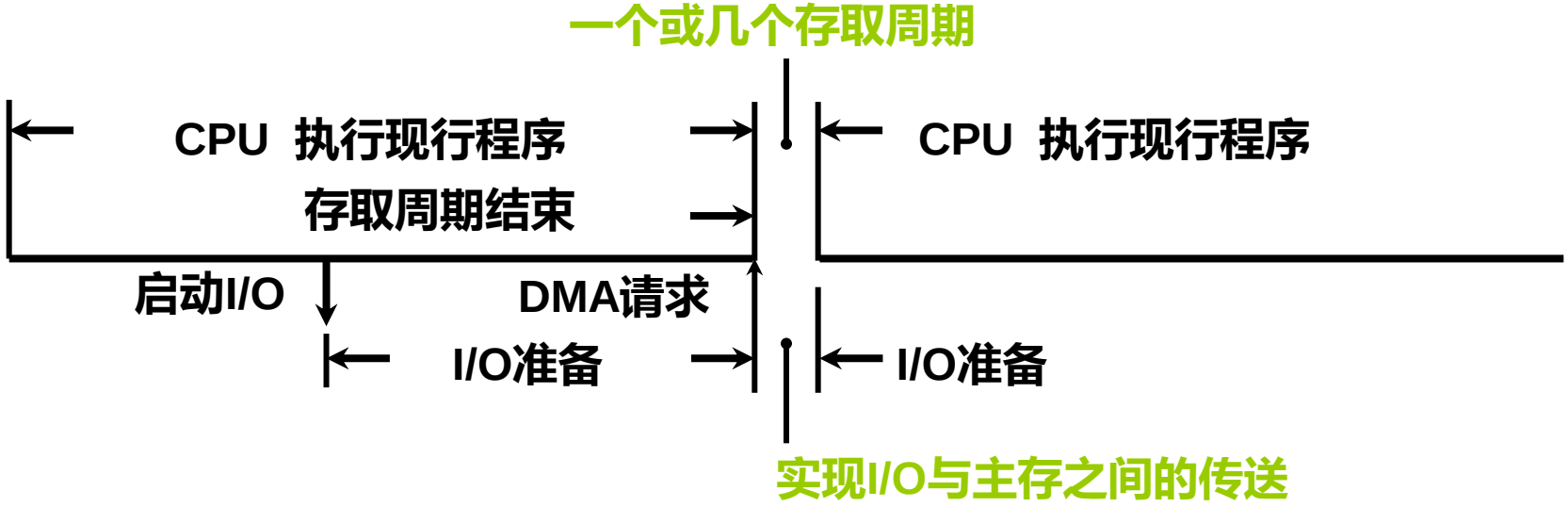
不中断现行程序

周期挪用（周期窃取）

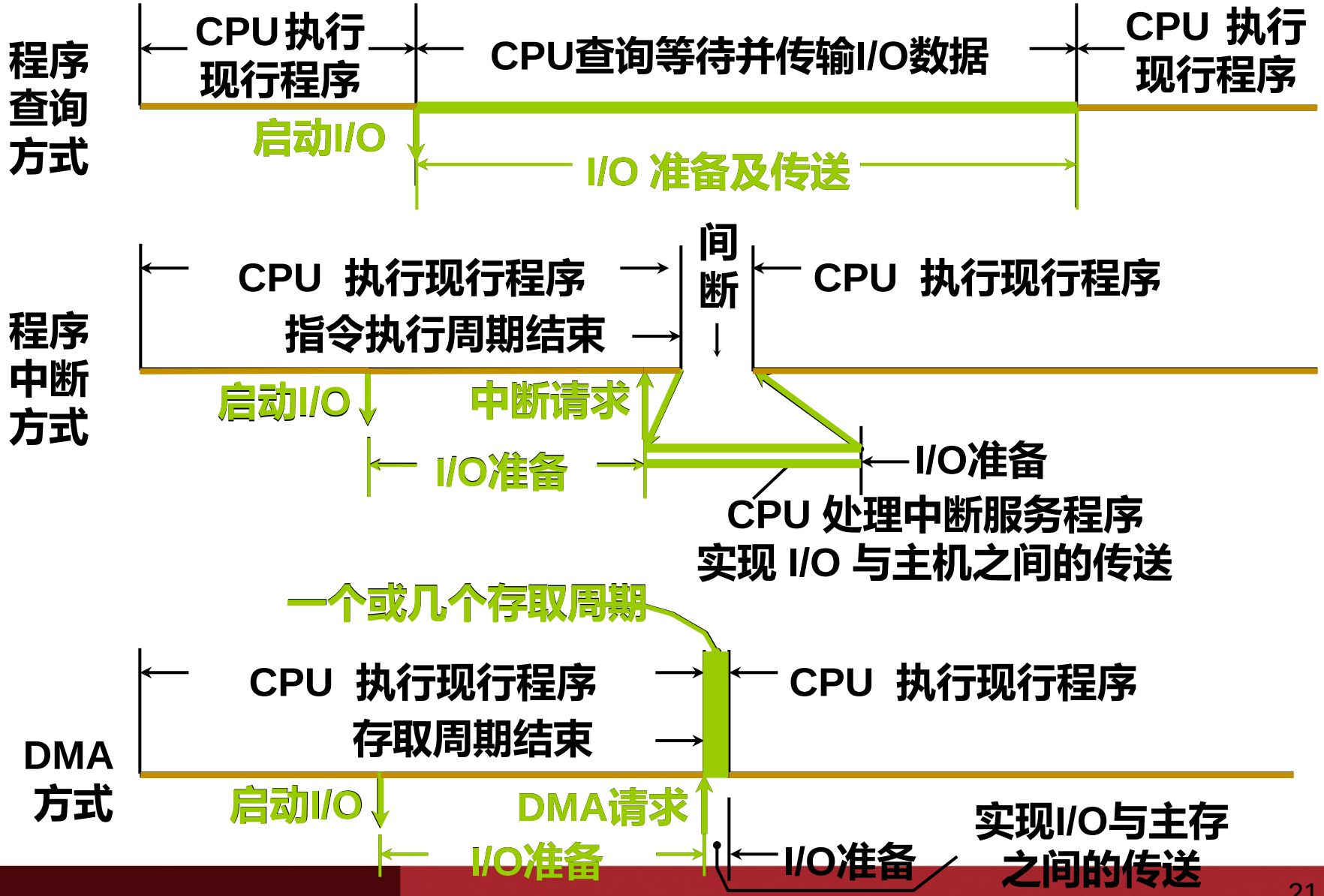
需用总线 → 存取周期

CPU 让出一个(几个)

CPU 和 I/O 并行工作

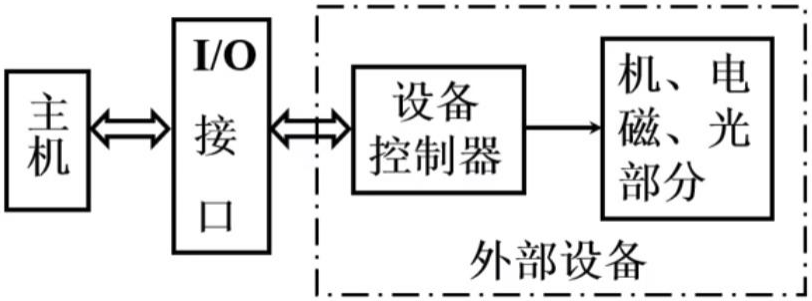


三种方式的 CPU 工作效率比较



5.2 外部设备

一、概述



外部设备大致分三类

1. 人机交互设备　　键盘　鼠标　打印机　显示器
2. 计算机信息驻留设备　　磁盘　光盘　磁带
3. 机 - 机通信设备　　调制解调器等

5.2 外部设备

二、输入设备

1. 键盘

按键

判断哪个键按下

将此键翻译成 ASCII 码 （编码键盘法）

2. 鼠标

机械式、光电式

3. 触摸屏

三、输出设备

1. 显示器

(1) 字符显示　　字符发生器

(2) 图形显示　　主观图像（矢量图）

(3) 图像显示　　客观图像（像素图）

2. 打印机

(1) 击打式　　点阵式（逐字、逐行）

(2) 非击打式　　喷墨（逐字）　　激光（逐页）

5.2 外部设备

四、其他

1. A/D、D/A　　模拟/数字（数字/模拟）转换器

2. 终端　　由键盘和显示器组成

共享一套CPU和存储

完成显示控制与存储、 键盘管理及通信控制

3. 汉字处理　　汉字输入、汉字存储、汉字输出

五、多媒体技术

1. 什么是多媒体（Multimedia）

多种…技术

2. 多媒体计算机的关键技术　　压缩和解压缩、专用芯片（GPU）、大容量存储

5.3 I/O 接口

一、概述

二、接口的功能和组成

三、接口类型

5.3 I/O 接口

一、概述

1、I/O接口(Input/Output Interface)定义：

- I/O 接口是**主机**与**外部设备**进行信息交换的**纽带**。
- 主机通过I/O 接口与外部设备进行**数据交换**。
- 绝大部分I/O 接口电路都是**可编程**的。
- I/O 接口电路包括：**插头 / 插座的形式**、**通讯规程**和**电器特性**等。

5.3 I/O 接口

2、为什么要设置接口?

- 实现设备 ~~(地址)~~ 的选择
- 实现数据缓冲达到速度匹配
- 实现数据串 - 并格式转换
- 实现电平转换
- 传送控制命令
- 反映设备的状态（“忙”、“就绪”、“中断请求”）

接口（Interface）与端口(Port)

3、接口（Interface）和端口（Port）不同：

- **接口**：指一种为在主机和外设之间传送信息而设置的**硬件线路**，也可以指两个软件之间的共同**逻辑边界**。
- **端口**：接口电路中的一些**寄存器**，这些寄存器分别用来存放数据信息、控制信息和状态信息，相应的就是**数据**端口、**控制**端口和**状态**端口。
- 若干个端口加上相应的**控制逻辑**才能组成接口。CPU通过输入指令，从**端口**读入信息，通过输出指令，可将信息**写入**到端口中。

二、接口的功能和组成

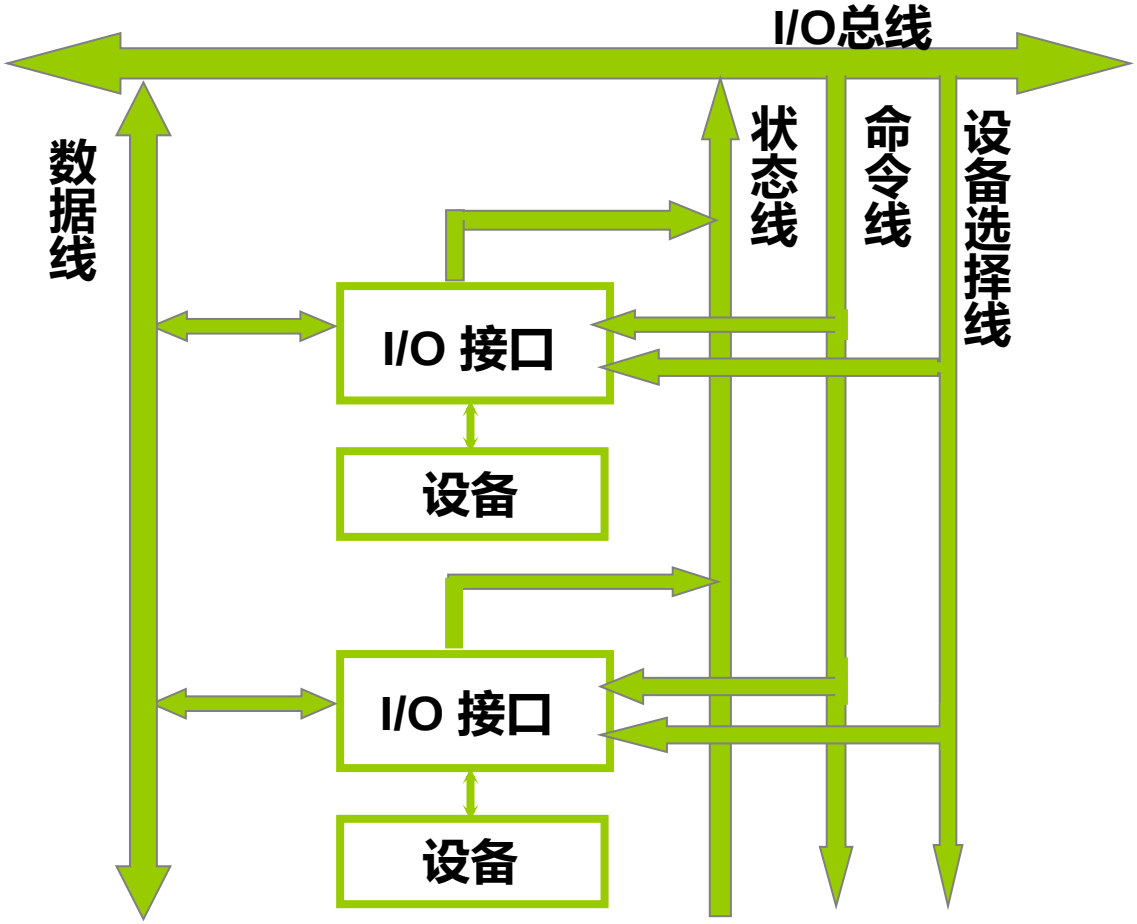
1. 总线连接方式的 I/O 接口电路

(1) 设备选择线

(2) 数据线

(3) 命令线

(4) 状态线



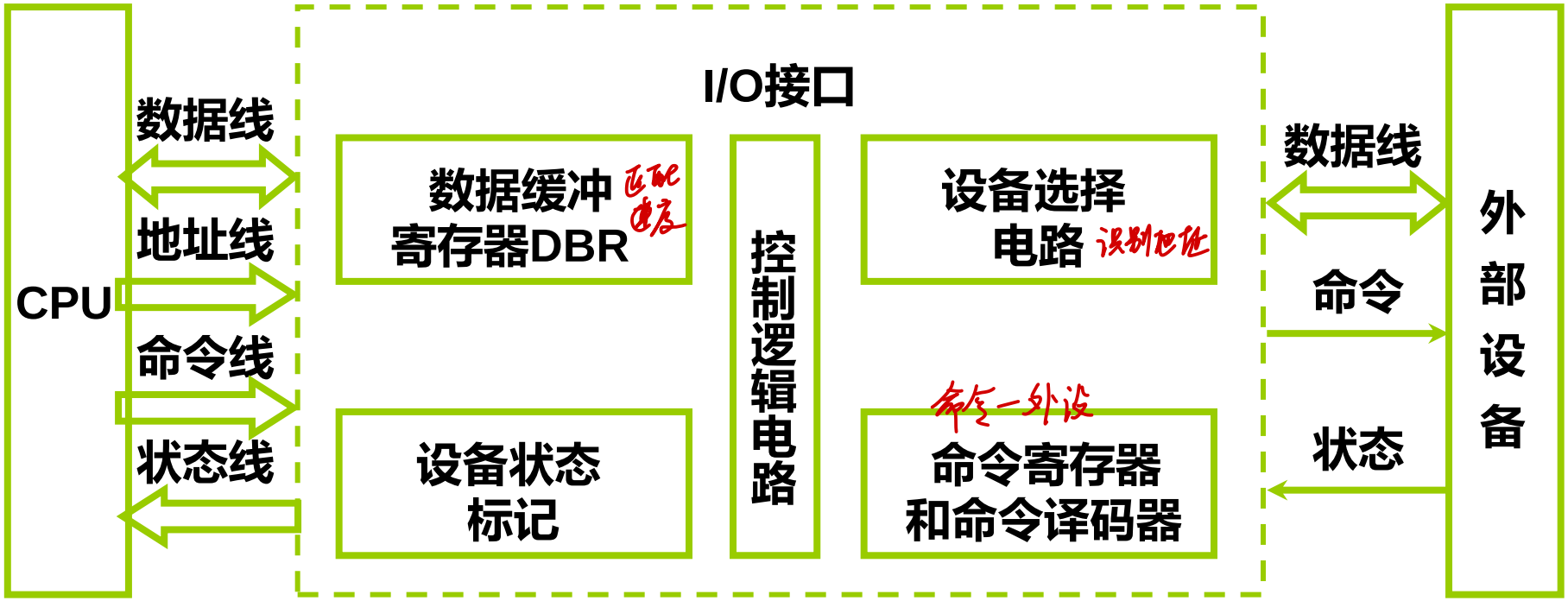
2. 接口的功能和组成

功能	组成
选址功能	设备选择电路
传送命令的功能	命令寄存器　命令译码器
传送数据的功能	数据缓冲寄存器
反映设备状态的功能	设备状态标记

- 完成触发器 D ✓
- 工作触发器 B ✓
- 中断请求触发器 INTR
- 屏蔽触发器 MASK

中断端口特有

3. I/O 接口的基本组成


CPU
数据线
地址线
命令线
状态线
I/O接口
数据缓冲寄存器DBR
设备状态标记
控制逻辑电路
设备选择电路
命令寄存器和命令译码器
数据线
命令
状态
外部设备

三、接口类型

1. 按数据 **传送方式** 分类

并行接口	Intel 8255
串行接口	Intel 8251

2. 按功能 **选择的灵活性** 分类

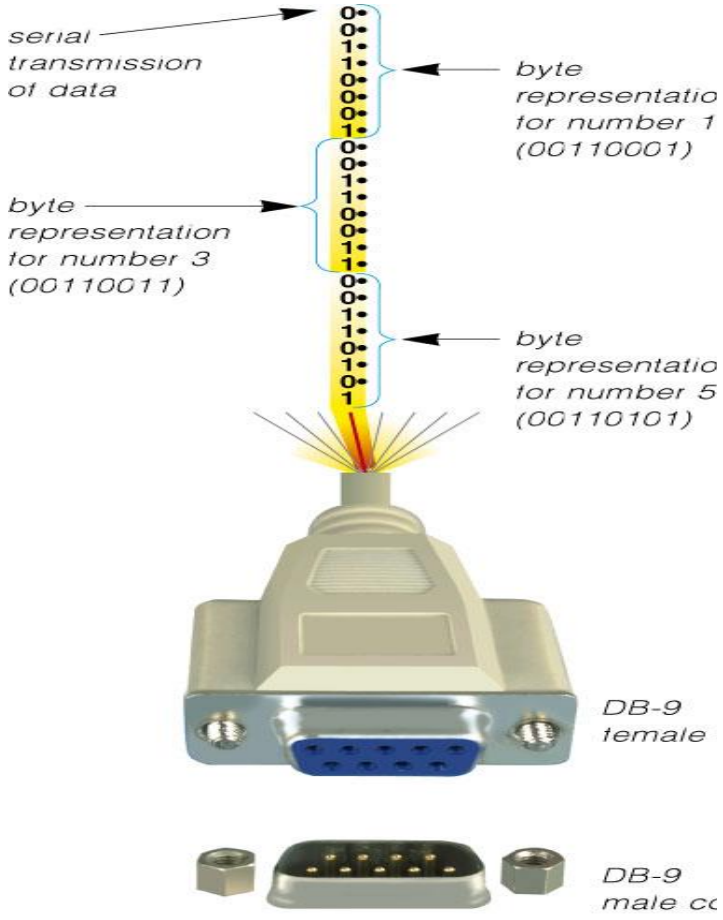
可编程接口	Intel 8255、 Intel 8251
不可编程接口	Intel 8212

3. 按 **通用性** 分类

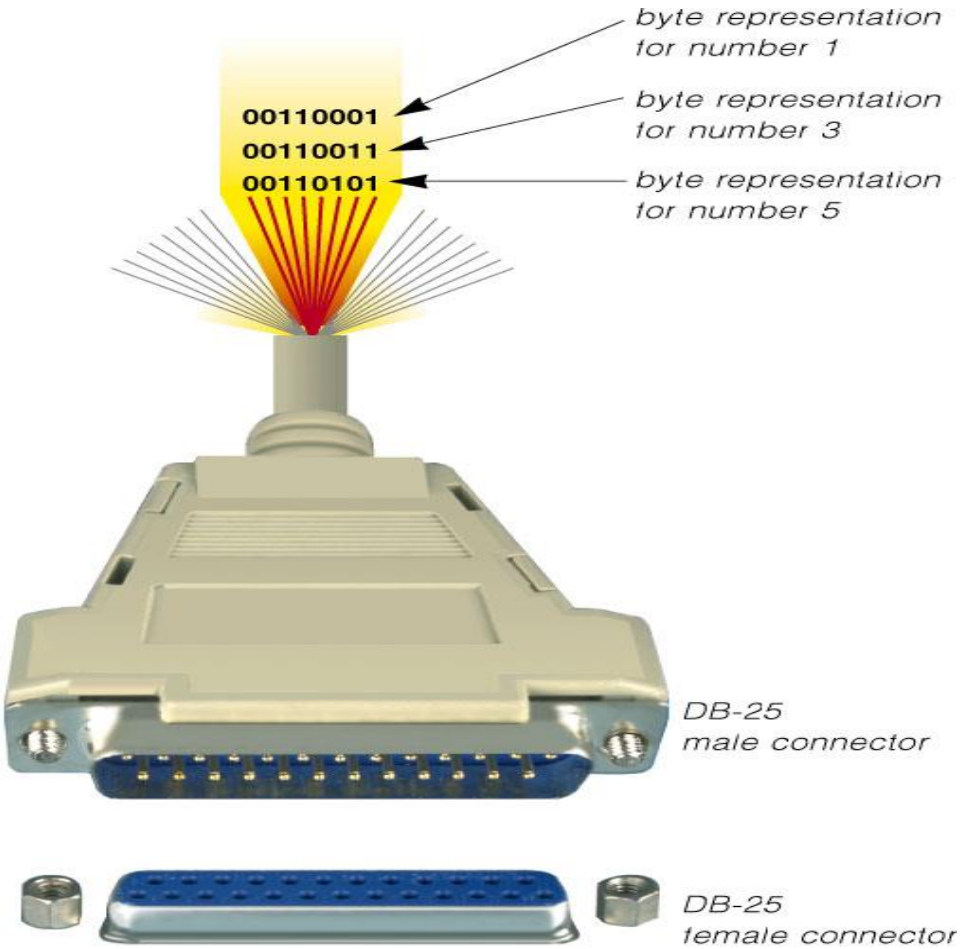
通用接口	Intel 8255、 Intel 8251
专用接口	Intel 8279、 Intel 8275

4. 按数据传送的 **控制方式** 分类

中断接口	Intel 8259
DMA 接口	Intel 8257

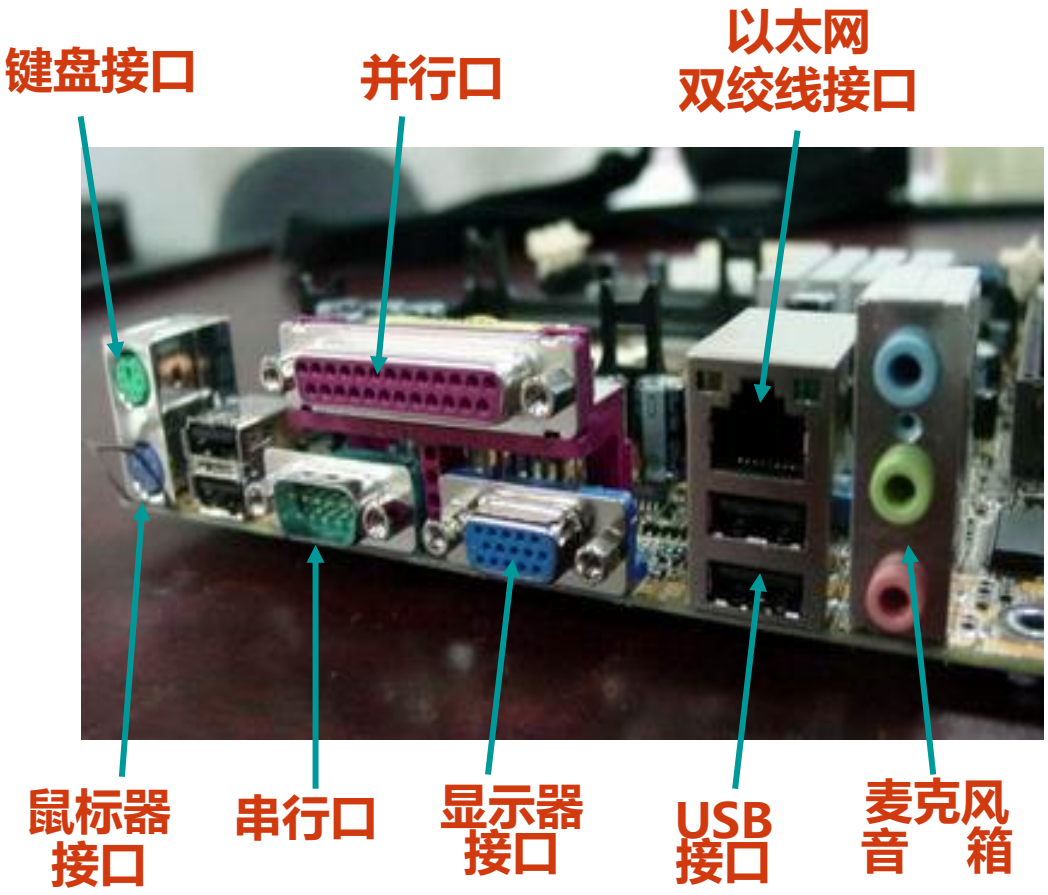

serial
transmission
of data
byte
representatio
for number 1
(00110001)
byte
representation
for number 3
(00110011)
byte
representatio
for number 5
(00110101)
DB-9
female
DB-9
male co

串行口

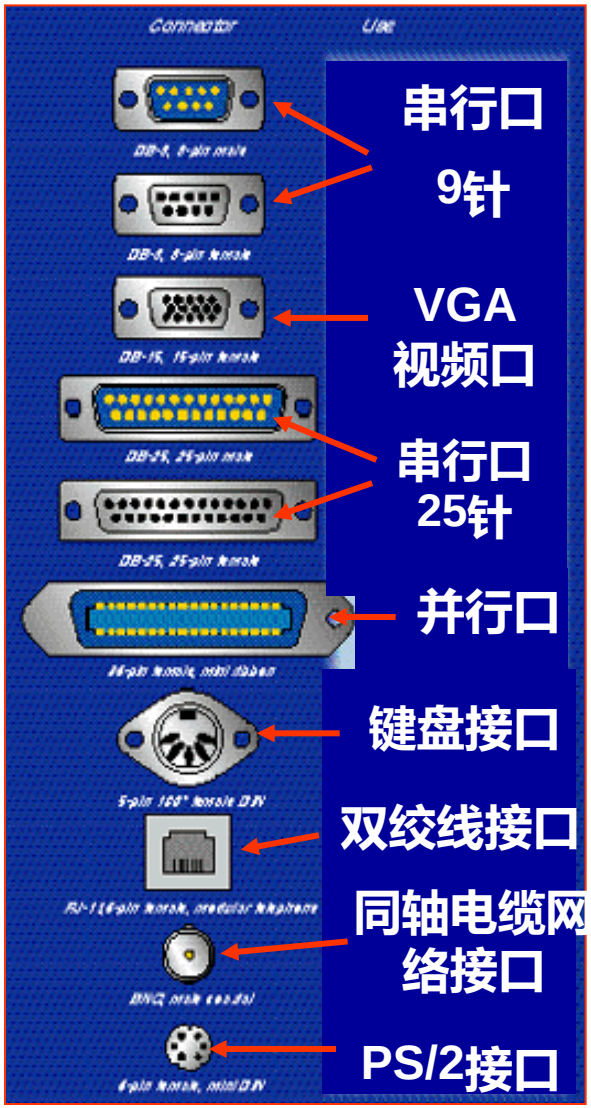

byte representation
for number 1
byte representation
for number 3
byte representation
for number 5
00110001
00110011
00110101
DB-25
male connector
DB-25
female connector

并行口

I/O设备接口



(安装在主板上的I/O设备接口)



5.4 程序查询方式

5.4.1、程序查询方式的流程

5.4.2 程序查询方式的接口电路

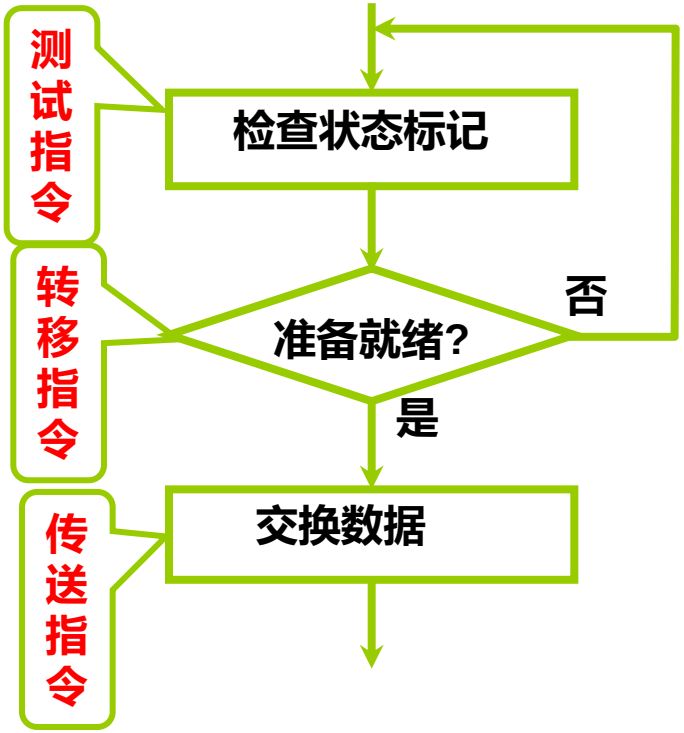
5.4.1 程序查询方式的流程

一、程序查询流程

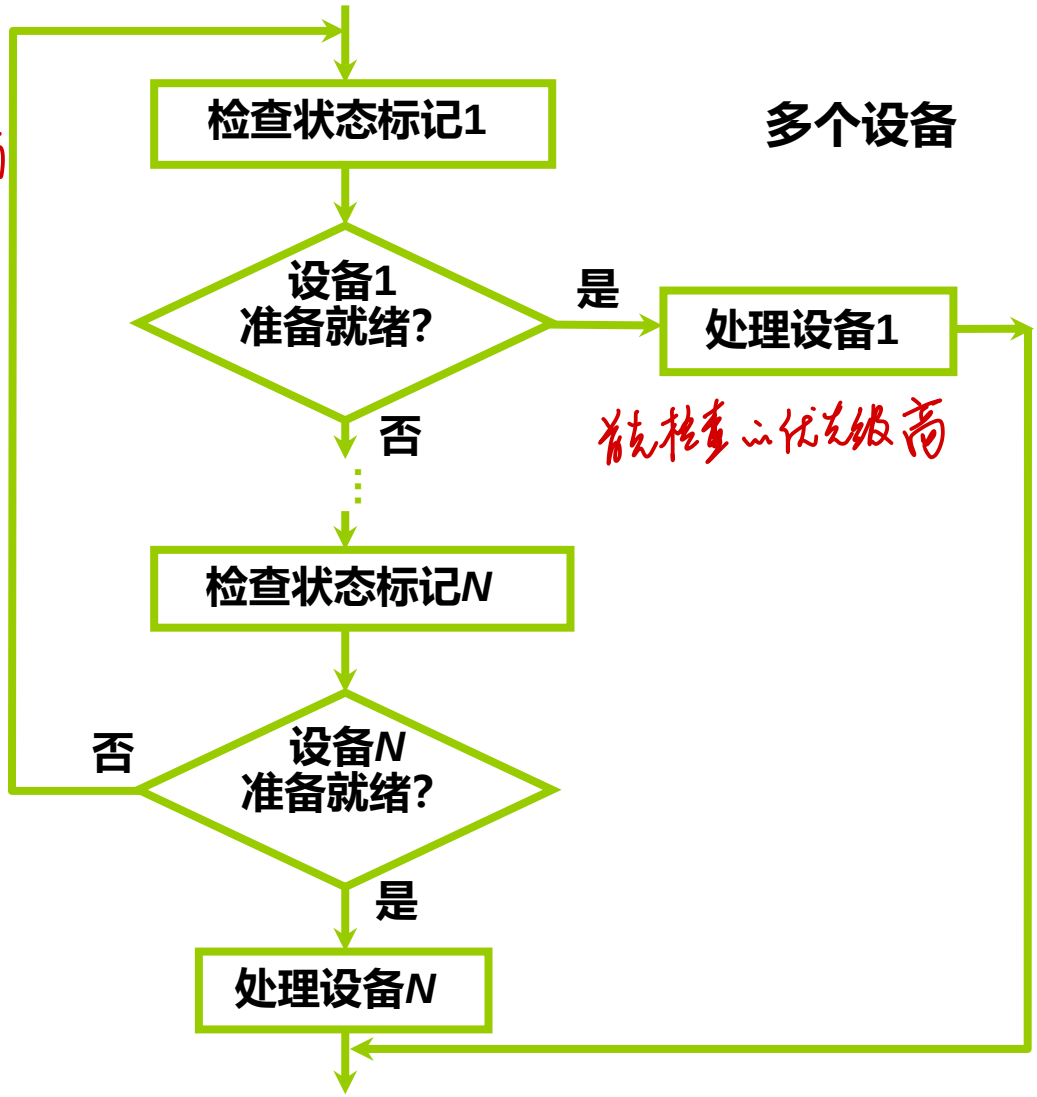
1. 查询流程

CPU完全参与

单个设备



多个设备



轮询查询优先级高

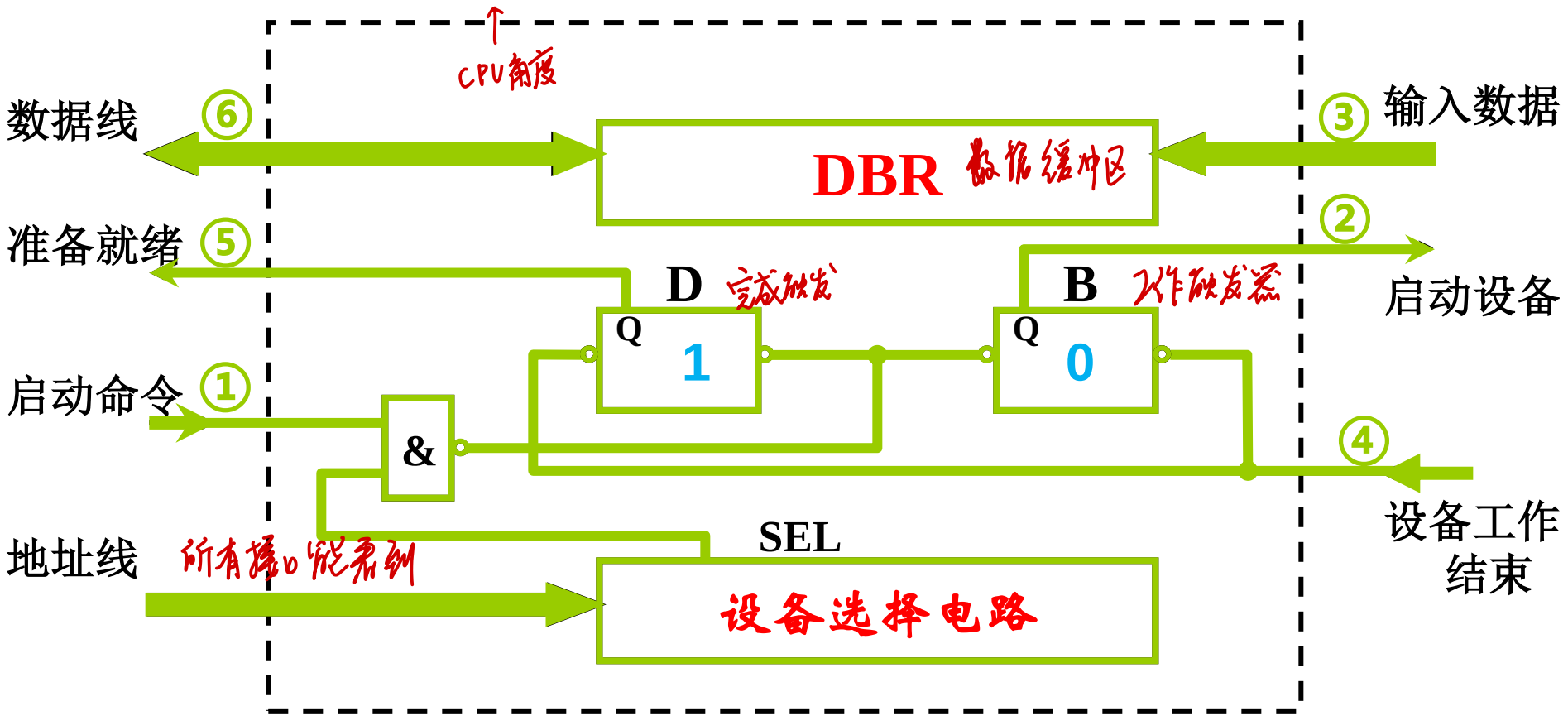
2. 程序流程

保护寄存器内容
设置计数器
设置内存缓冲区首址
启动外设
准备好?
否
是
传送一个数据
修改内存地址
修改计数值
未完
传送完?
完
结束I/O传送
与保存断点不一样
控制

5.4.2 程序查询方式的接口电路

一、程序查询方式的接口电路

以输入为例



5.5 程序中断方式

5.5.1 中断的概念

5.5.2 I/O中断的产生

5.5.3 程序中断的接口电路

5.5.4 I/O中断处理过程

5.5.5 中断服务程序流程

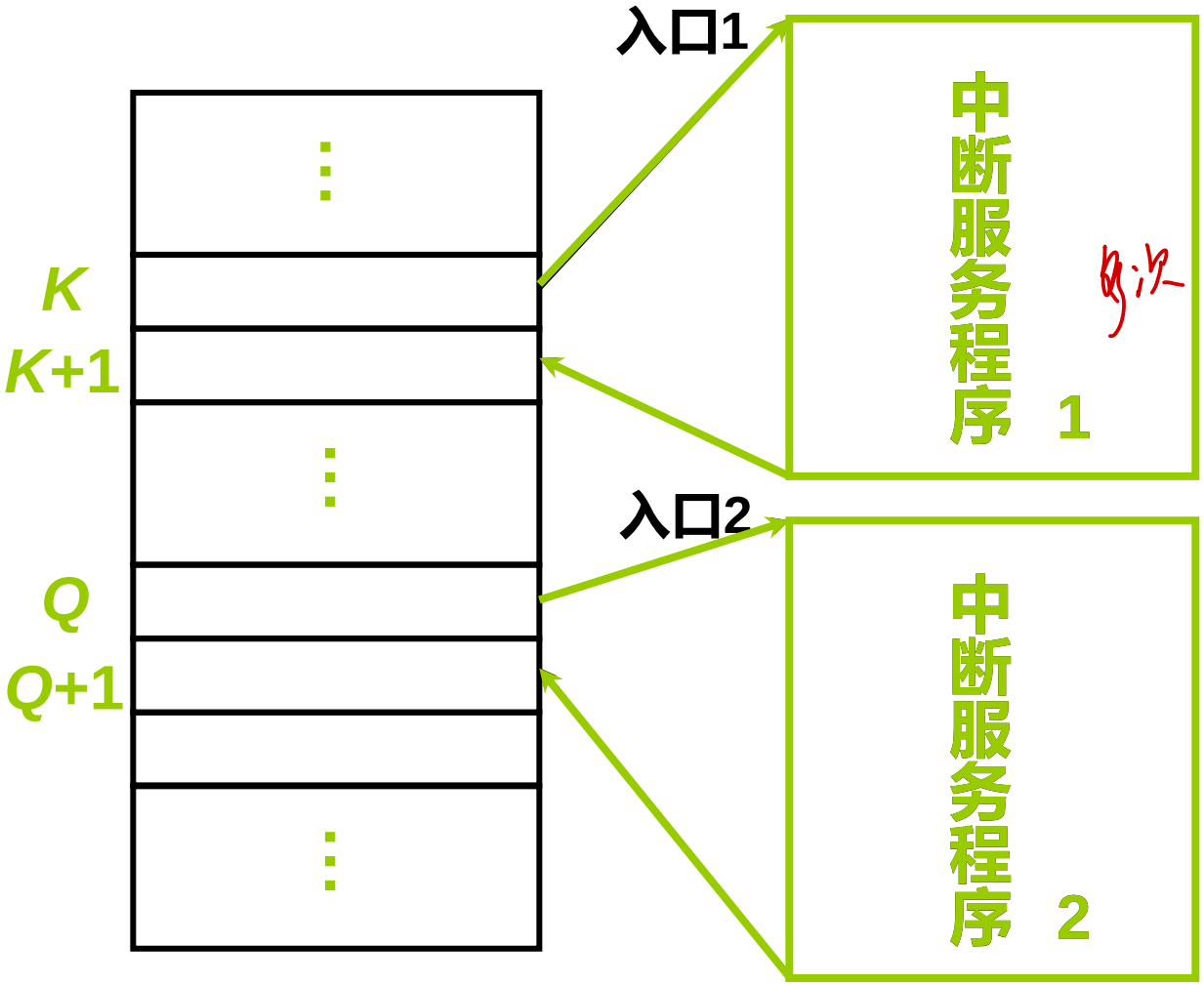
5.5.1 中断的概念

一、中断（Interrupt）的概念

是指在计算机执行期间，系统内发生任何**非寻常**的或**非预期**的**急需处理**事件，使得CPU**暂时中断**当前正在执行的程序而转去执行相应的事件处理程序。待处理完毕后又**返回**原来被中断处继续执行执行的过程。

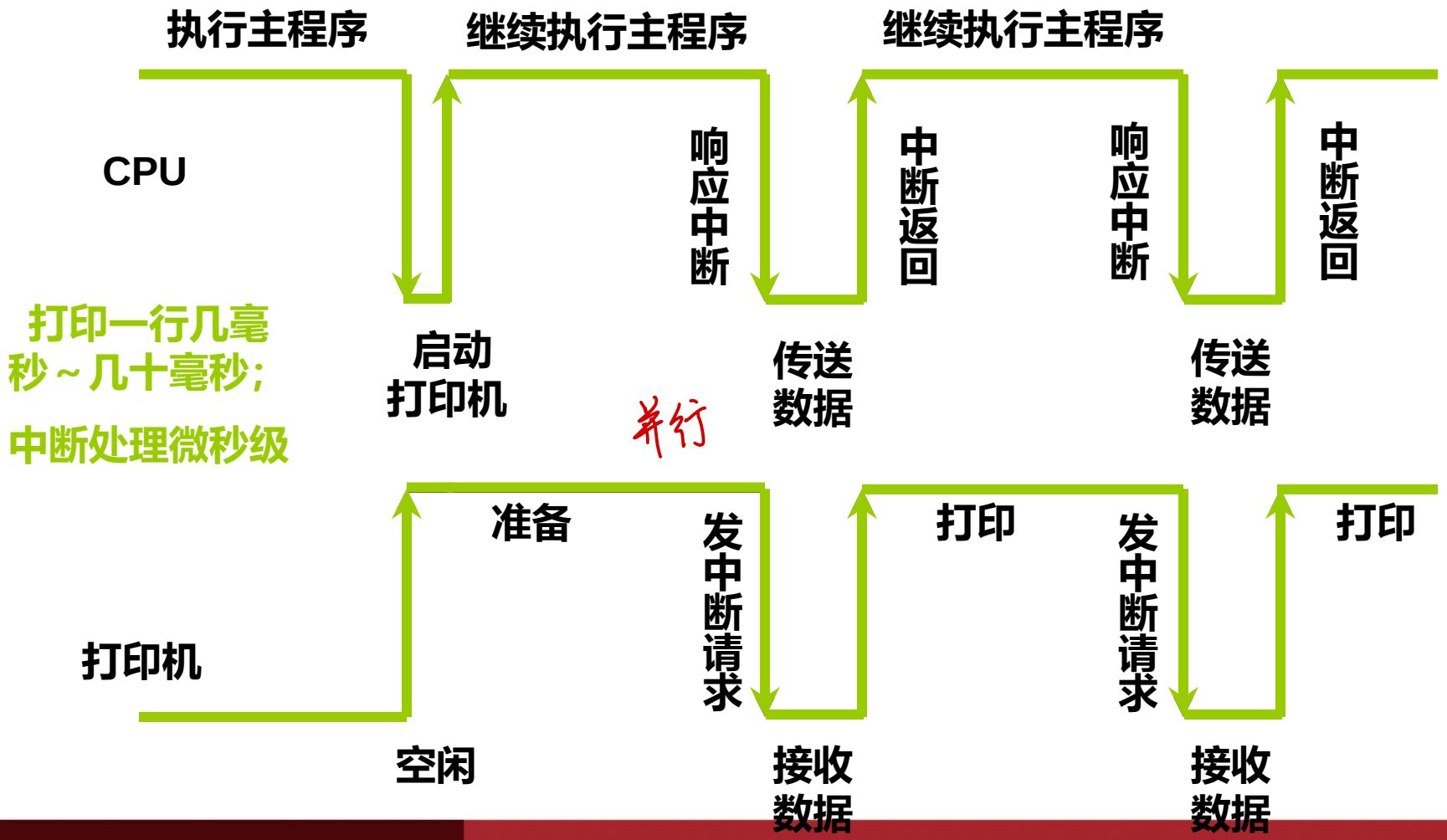
中断的种类和方式很多。

5.5.1 中断的概念



5.5.1 中断的概念

实现CPU与I/O设备并行工作 （以打印机为例）



5.5.2 I/O中断的产生

一、中断的产生

设置中断的必要性：

1．**分时操作**： 采用中断技术能够使CPU与外围设备并行工作，实现分时操作，大大提高了CPU的利用率。

2．**实时处理**： 实时控制是单片机系统一个重要的应用领域，它要求CPU对工作现场的各种参数和状态的随机变化能够快速响应并加以及时处理，而这些功能只有采用中断技术才能得以实现。

3．**故障处理**： 计算机在运行过程中常常会突然发生故障，如电源掉电、内存出错及程序故障等。

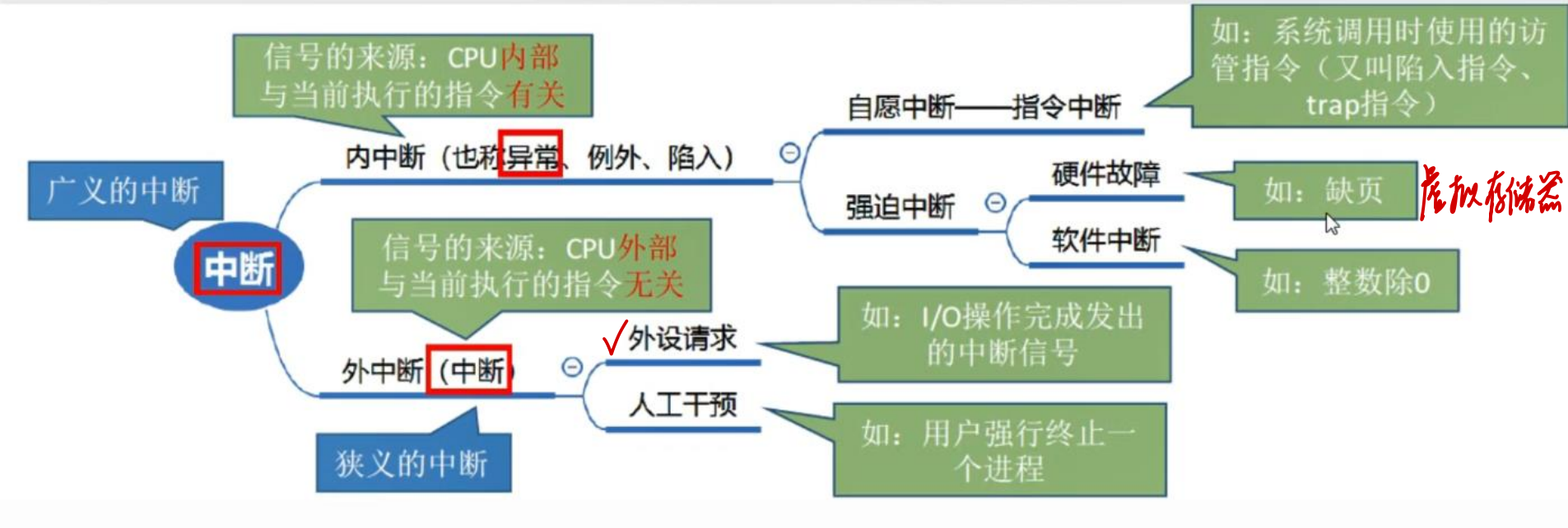
5.5.2 I/O中断的产生

二、中断的分类

1. 按中断来源分：

（1）内中断：发生在**主机内部**的中断称为内中断

（2）外中断：由**主机外部**事件引起的中断称为外中断 外设



5.5.3程序中断方式的接口电路

向量中断：硬件形式

一、程序中断方式的接口电路

1. 中断请求触发器和中断屏蔽触发器

（1）中断请求触发器

- 当中断发送引起中断的事件时，先将它保存在设备控制器的中断请求触发器INTR中，即将“中断请求触发器”置“1”。
- 当中断请求触发器为“1”时，向CPU发出“中断请求”信号。
- 每个中断源有一个**中断请求触发器**。全机的多个中断请求触发器构成中断请求寄存器，其内容称为**中断字**或**中断码（号）**。CPU进行中断处理时，根据中断字确定中断源，转入相应的服务程序。

5.5.3程序中断方式的接口电路

(2) 中断屏蔽:

为了便于控制中断请求信号的产生，也为了利用屏蔽码改变中断处理的有限级别，当产生中断请求后，用程序方式**有选择地**封锁部分中断，而允许其余**部分中断**仍得到响应，称为中断屏蔽。

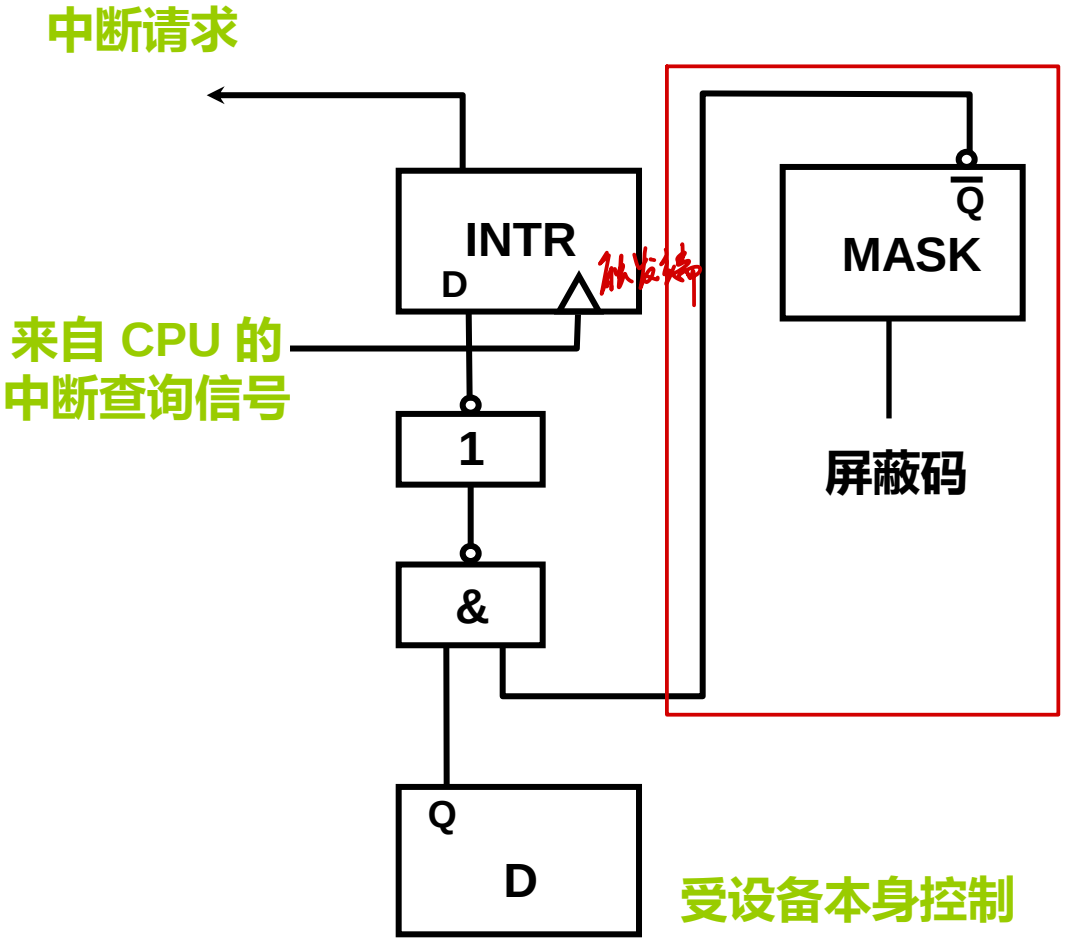
- 实现方法是为每个中断源设置一个中断屏蔽触发器MASK来屏蔽该设备的中断请求。即用程序方式将该触发器**置“1”**，则对应的设备**中断被封锁**，若将其**置“0”**，才允许该设备的中断请求得到**响应**。

5.5.3程序中断方式的接口电路

- 各设备的中断屏蔽触发器组成**中断屏蔽寄存器**。中断屏蔽寄存器通常是一个与系统中的中断请求寄存器同样位数的专用寄存器。CPU可以根据需要给中断屏蔽寄存器写入屏蔽字（二进制代码），代码的每一位可屏蔽一种中断源。如当中断屏蔽寄存器的某位为“1”时，表示该位所对应的中断被屏蔽。
- 有些中断请求是**不可屏蔽**的。也就是说，不管中断系统是否开中断，这些中断源的中断请求一旦提出，CPU必须立即响应。例如，电源掉电就是不可屏蔽中断。所以，中断又分为**可屏蔽中断**和**非屏蔽中断**。非屏蔽中断具有最高优先权。

5.5.3程序中断方式的接口电路

(3) 接口电路中几个信号间的关系



INTR
中断请求触发器

INTR = 1 有请求

MASK
中断屏蔽触发器

MASK = 1 被屏蔽

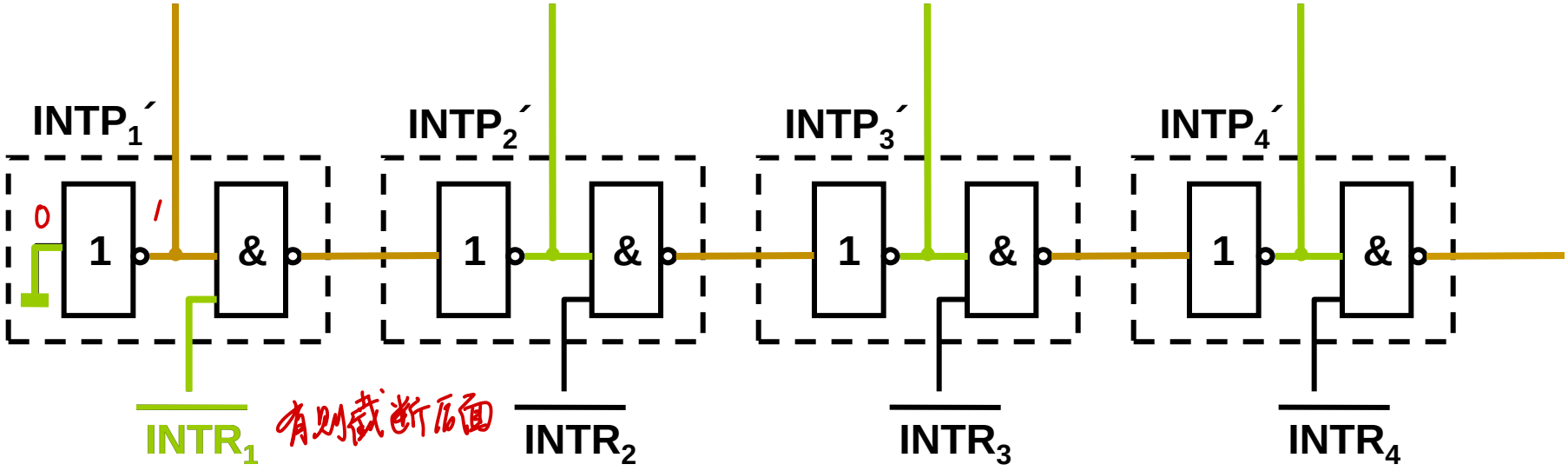
D 完成触发器

5.5.3程序中断方式的接口电路

2. 排队器

类似总线

排队 { 硬件　在 CPU 内、在接口电路中（链式排队器）
　　 { 软件　详见第八章

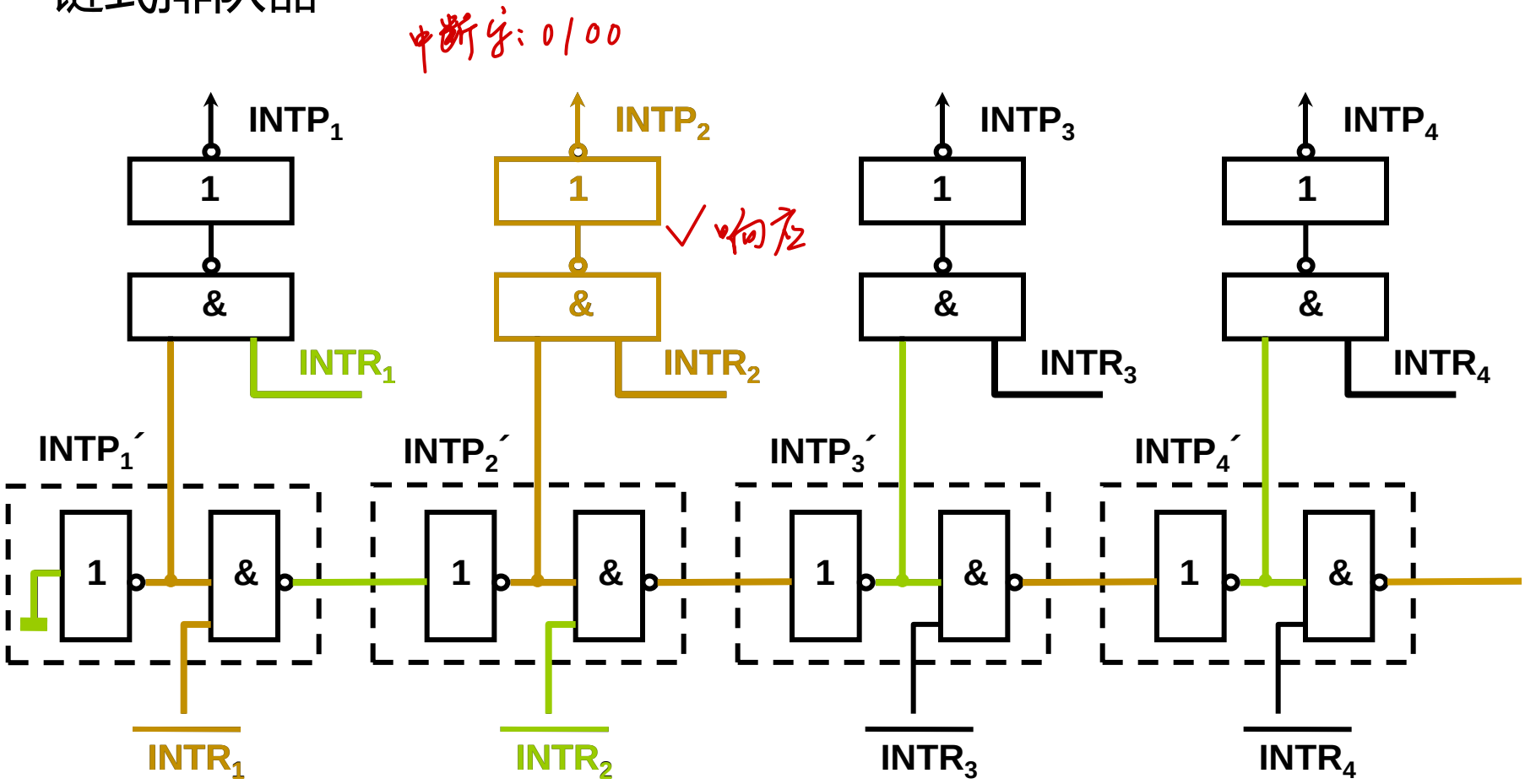


设备 1#、2#、3#、4# 优先级按 降序排列

$INTR_i = 1$ 有请求　　即 $\overline{INTR_i} = 0$

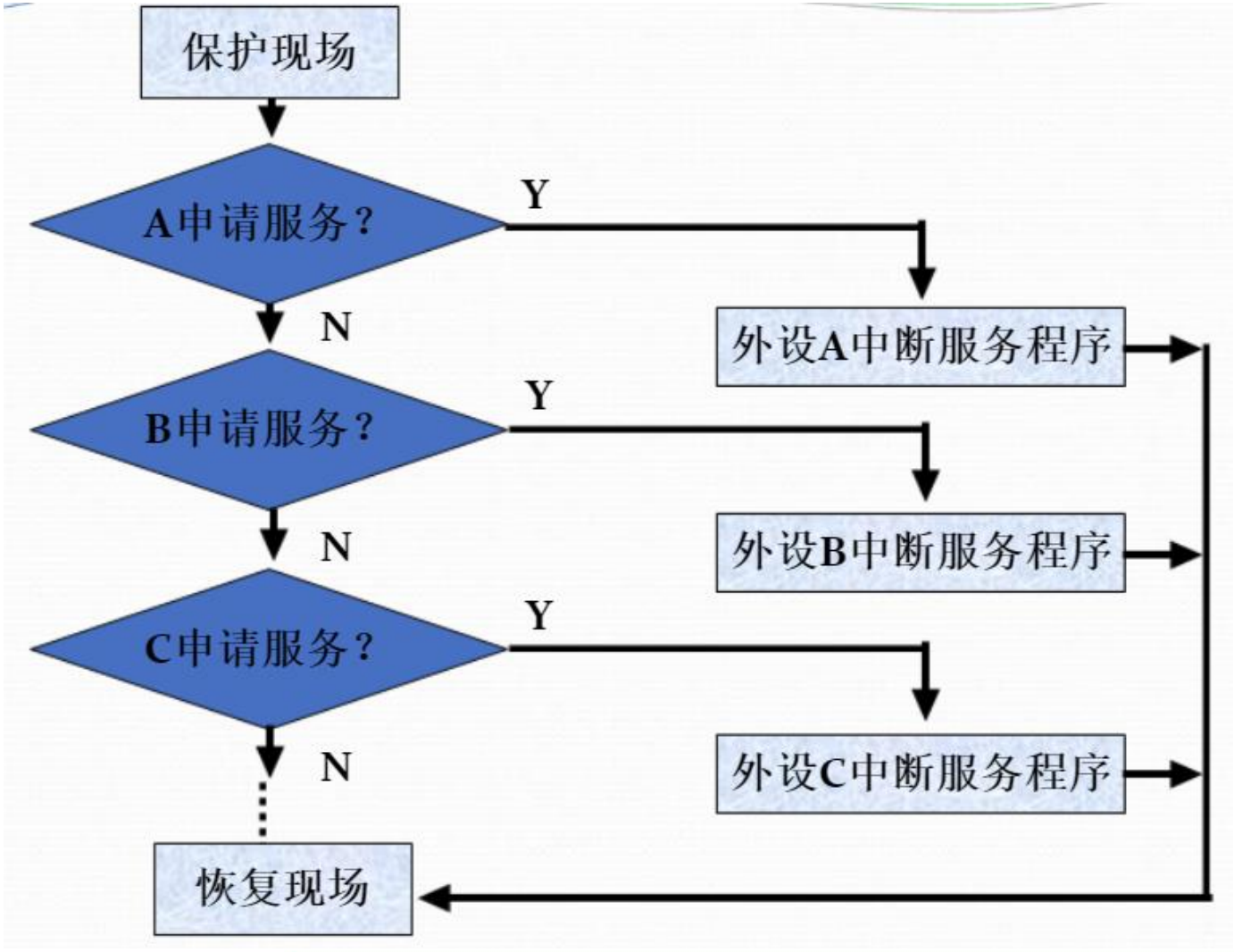
5.5.3程序中断方式的接口电路

链式排队器



5.5.3程序中断方式的接口电路

使用软件查询法解决中断优先级只需少量硬件电路。



5.5.3程序中断方式的接口电路

3. 中断向量地址形成部件

入口地址 { 由软件产生　　详见第八章
　　硬件向量法　　由 硬件 产生 向量地址　存放中断服务程序的首地址
　　再由 向量地址 找到 入口地址

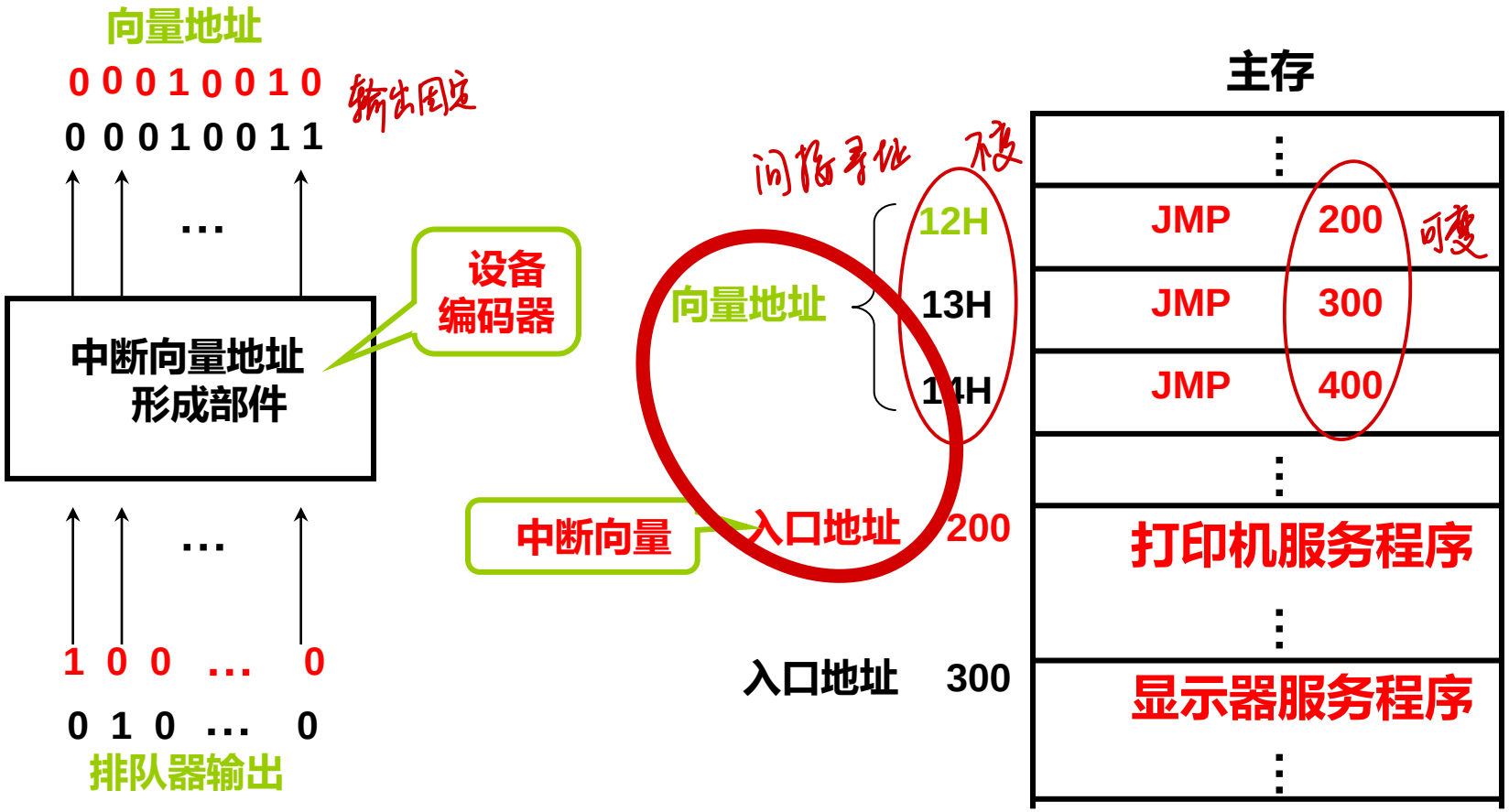
区分几个概念：

中断（号）码：系统分配给每个中断源的代号，以便识别和处理。

中断向量：中断服务程序的**入口地址**。

向量地址：保存中断向量的主存地址。

5.5.3程序中断方式的接口电路



单选题 0.5分

设置

中断向量是（ ）。

A 子程序入口地址

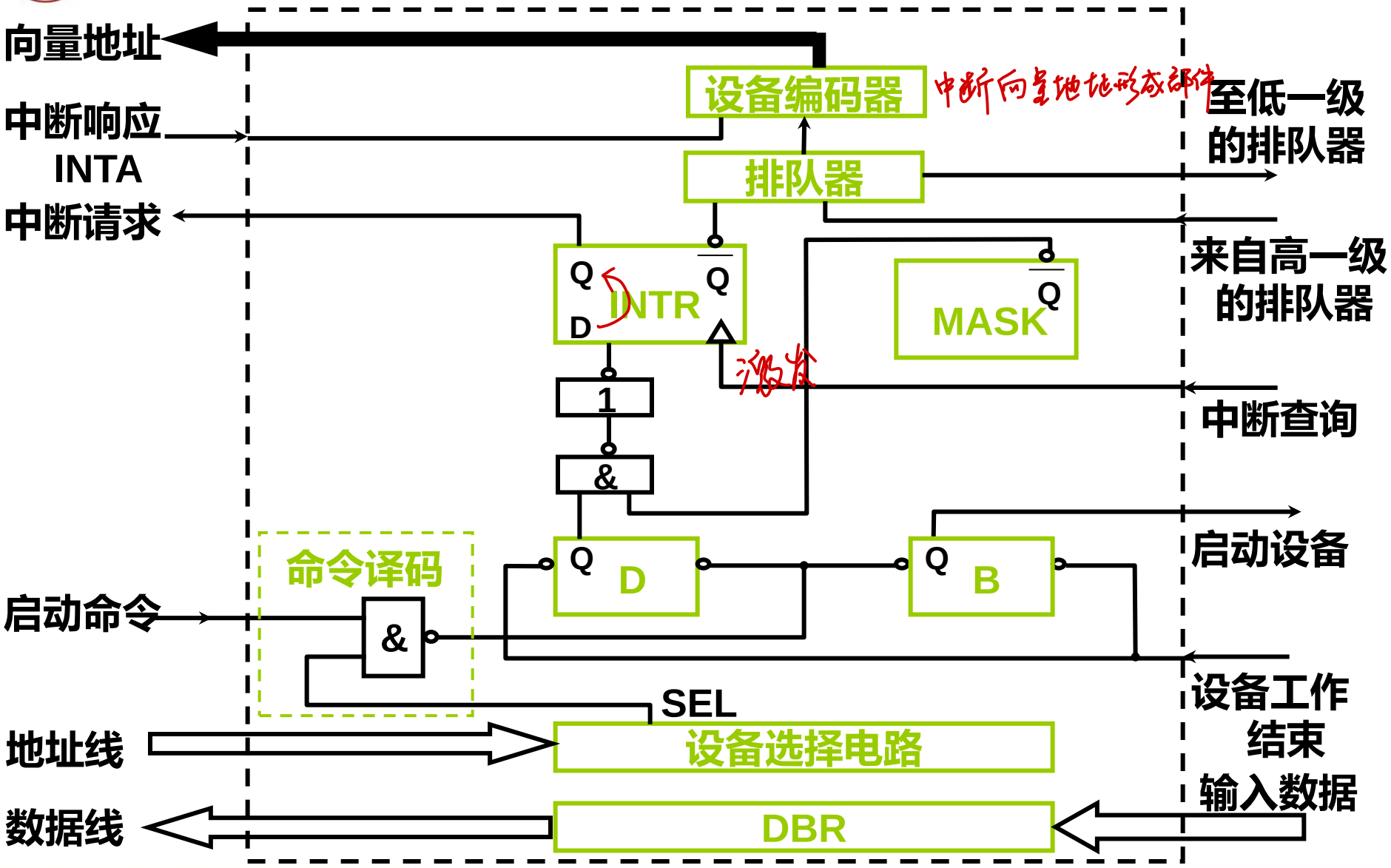
B 中断服务程序入口地址

C 中断服务程序入口地址的地址

D 以上答案均不正确

提交

4. 程序中断方式接口电路的基本组成



四、I/O 中断处理过程

1. CPU 响应中断的条件和时间

(1) 条件

中断允许触发器 **EINT = 1**

用 **开中断** 指令置 **“1”** EINT

用 **关中断** 指令置 **“ 0”** EINT 或硬件 **自动复位**

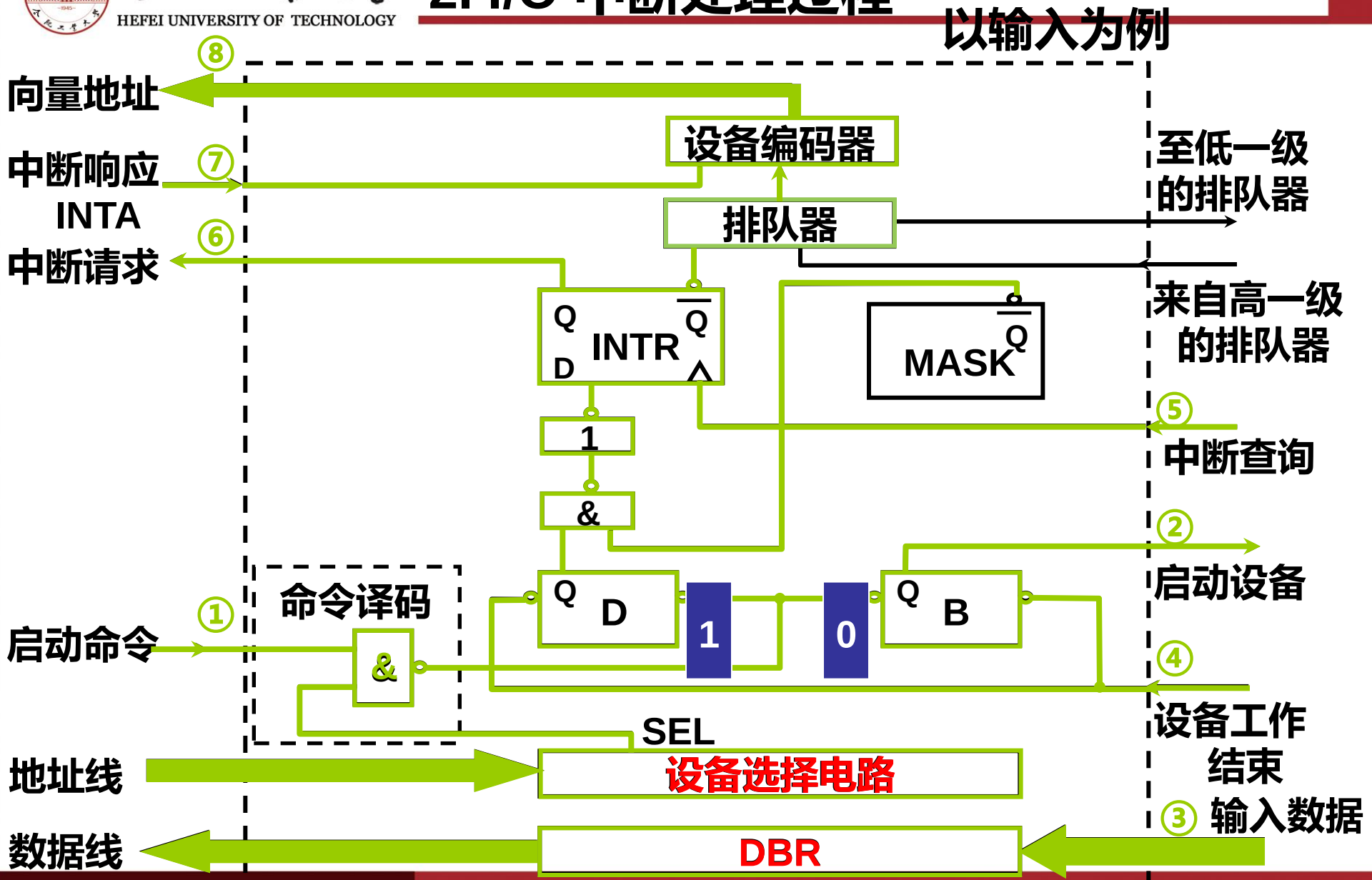
(2) 时间

当 D = **1**（随机）且 MASK = **0** 时

在每条指令执行阶段的结束前

CPU 发 中断查询信号（将 INTR 置“1”）

2. I/O 中断处理过程


以输入为例
⑧
向量地址
设备编码器
至低一级的排队器
中断响应 INTA
⑦
排队器
⑥
中断请求
来自高一级的排队器
Q D INTR Q
MASK Q
1
⑤
中断查询
&
②
启动设备
命令译码
Q D
1
0
Q B
①
启动命令
&
④
设备工作结束
SEL
地址线
设备选择电路
③ 输入数据
数据线
DBR

5.5.5 中断服务程序流程

1. 中断服务程序的流程

机器指令系统中没有的指令，它是CPU在中断周期内由硬件自动完成的一条指令。

(1) 保护现场

无专门指令

- 程序断点的保护 —— **中断隐指令**完成
- 寄存器内容的保护 —— 进栈指令

(2) 中断服务

对不同的 I/O 设备具有不同内容的设备服务

(3) 恢复现场

出栈指令

(4) 中断返回

中断返回指令

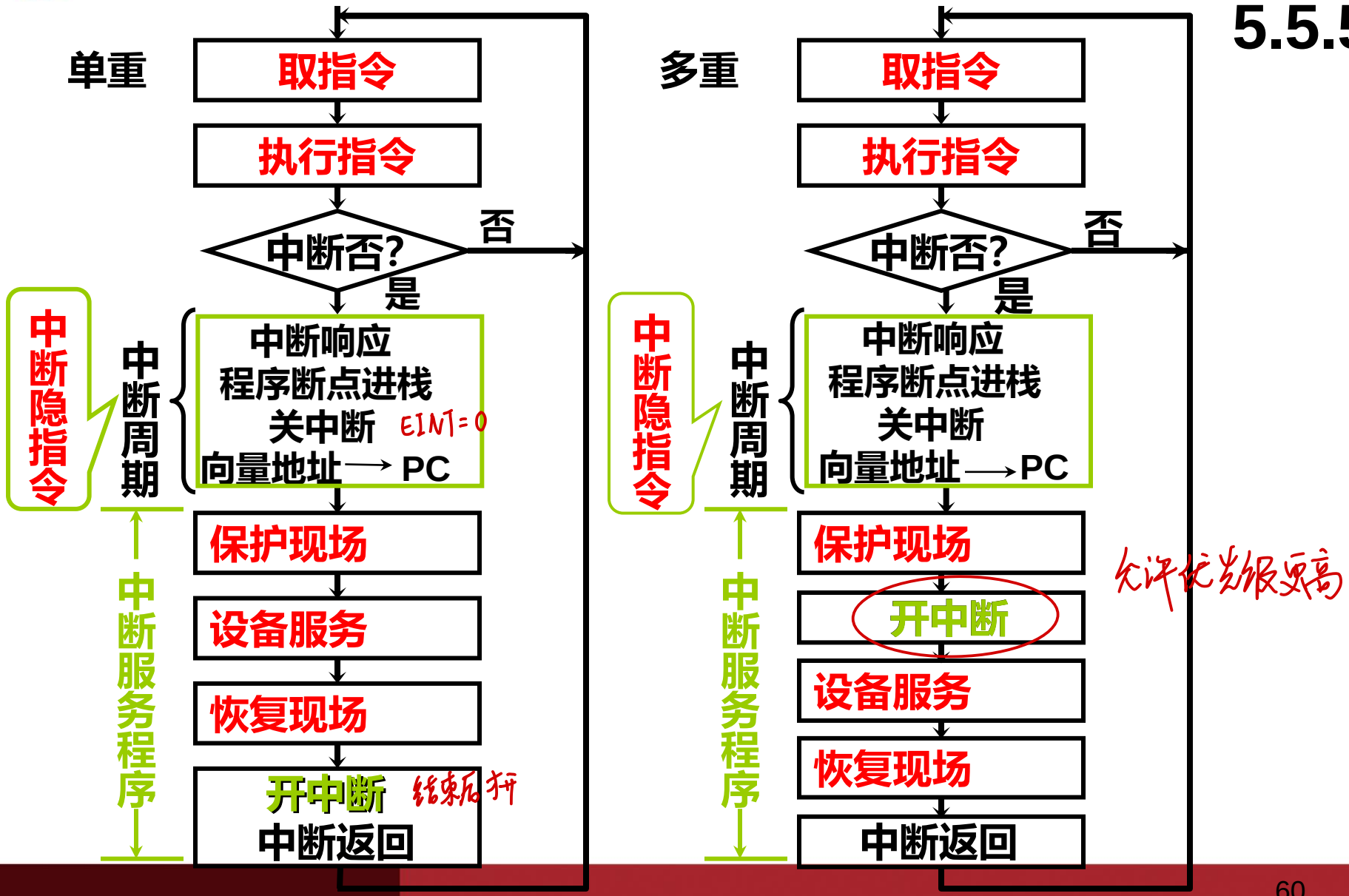
2. 单重中断和多重中断

单重 中断：**不允许中断** 现行的 **中断服务程序**

多重 中断 (中断嵌套)：**允许级别更高** 的中断源 **中断** 现行的 **中断服务程序**

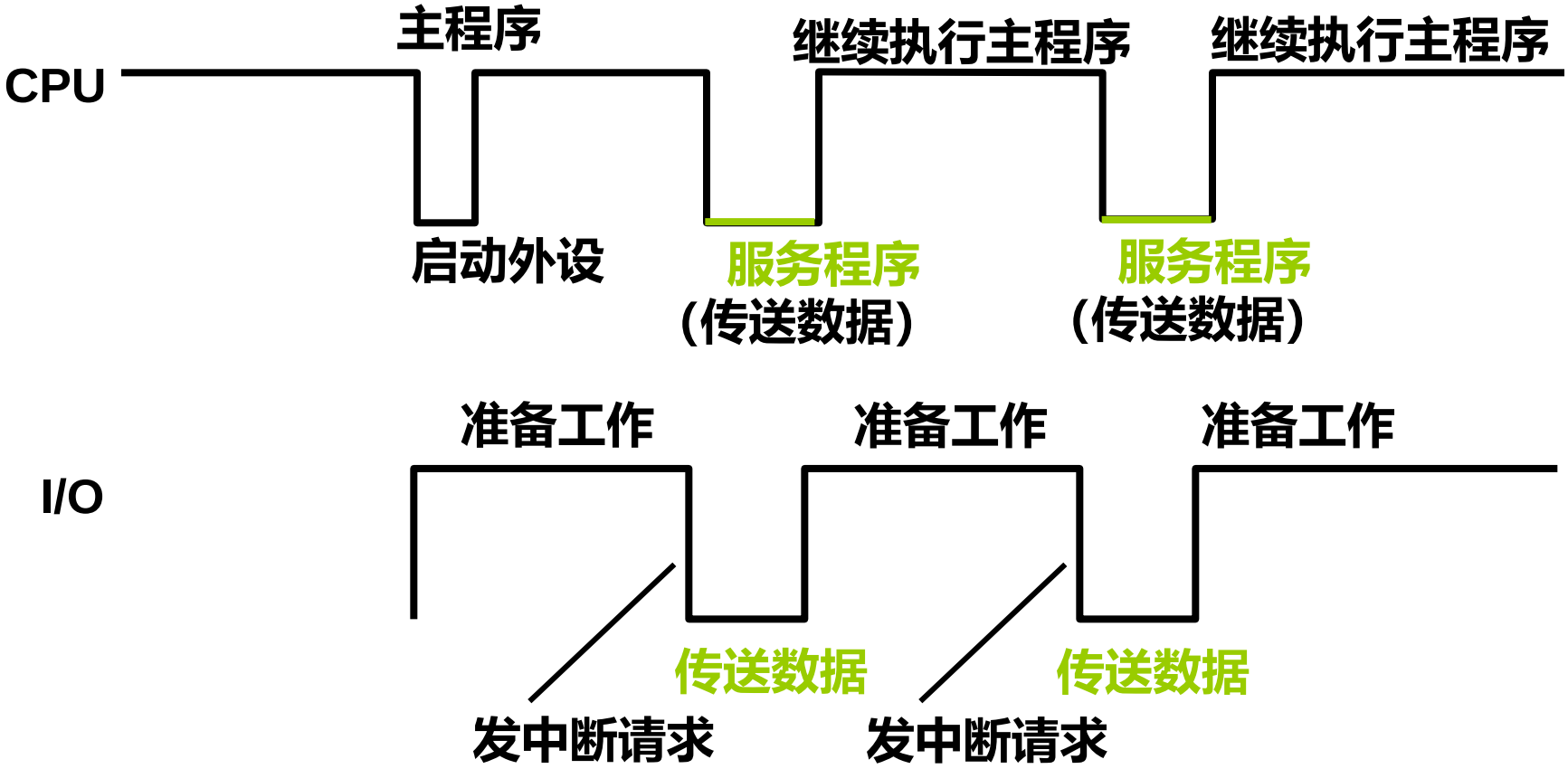
3. 单重中断和多重中断的服务程序流程

5.5.5



5.5.5 中断服务程序流程

主程序和服务程序抢占 CPU 示意



宏观 上 CPU 和 I/O 并行 工作

微观 上 CPU 中断现行程序 为 I/O 服务

5.5 程序中断方式

中断过程包含五个阶段：

- 中断请求；
- 判断优先级（排队）；
- 中断响应（中断周期操作）；
- 中断处理；
- 中断返回。

附加：多核CPU对中断的处理*

多核CPU的中断处理和单核有很大不同。多核的各处理器核心之间需要通过中断方式进行通信，所以CPU芯片内部既有各处理器核心的**本地中断控制器**，又有负责仲裁各核之间中断分配的**全局中断控制器**。

现今的多核处理器在中断处理和中断控制方面主要使用的是**APIC**（Advanced Programmable Interrupt Controllers），即**高级编程中断控制器**。它是基于中断控制器两个基础功能单元——本地单元以及I/O单元的分布式体系结构。在多核系统中，多个本地和I/O APIC单元能够作为一个整体通过APIC总线互相操作。

APIC的功能有：

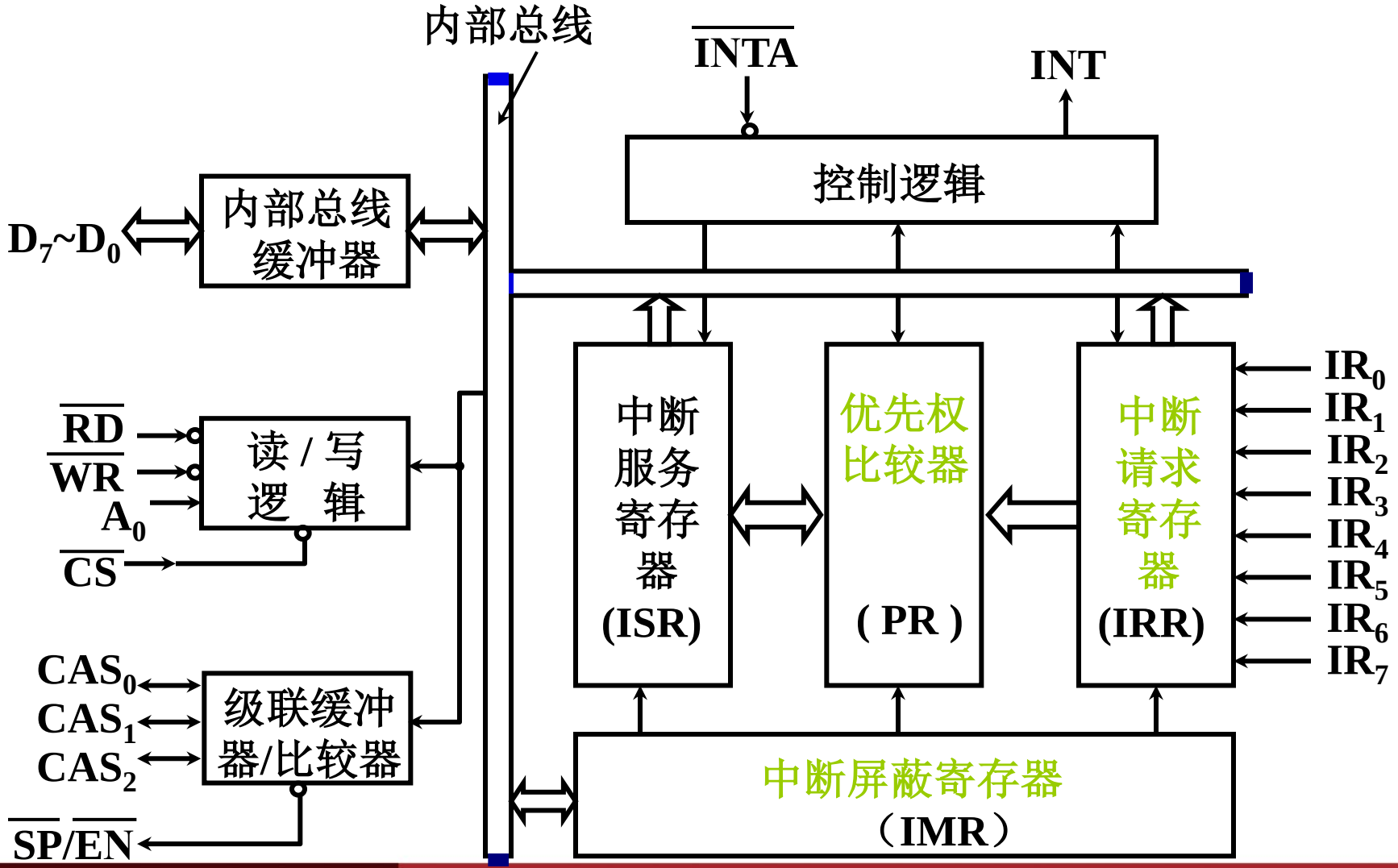
1. 接受来自处理器中断引脚的内部或外部I/O APIC的中断，然后将这些中断发送给处理器核心进行处理；

2. 在多核处理器系统中，接收和发送核内中断消息；

对于外部设备发出的中断请求，由全局中断控制器接收请求并决定交给CPU的哪一个核心处理。也可针对APIC编程，让所有的中断都被一个固定的CPU处理。

5.5 程序中断方式

程序中断接口芯片 8259A 的内部结构



5.5 程序中断方式

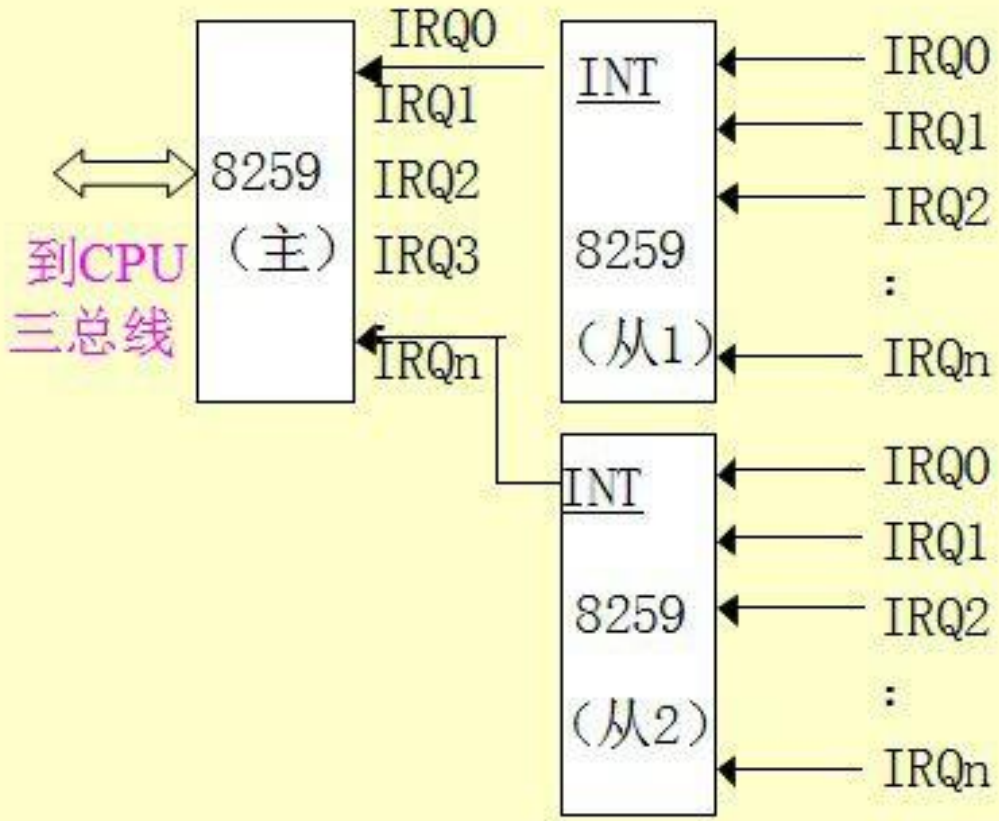


图6-18 8259A非缓冲方式下级连示意图

5.5 DMA方式

5.5.1 DMA方式的特点

5.5.2 DMA接口的功能和组成

5.5.3 DMA的工作过程

5.5.4 I DMA接口的类型

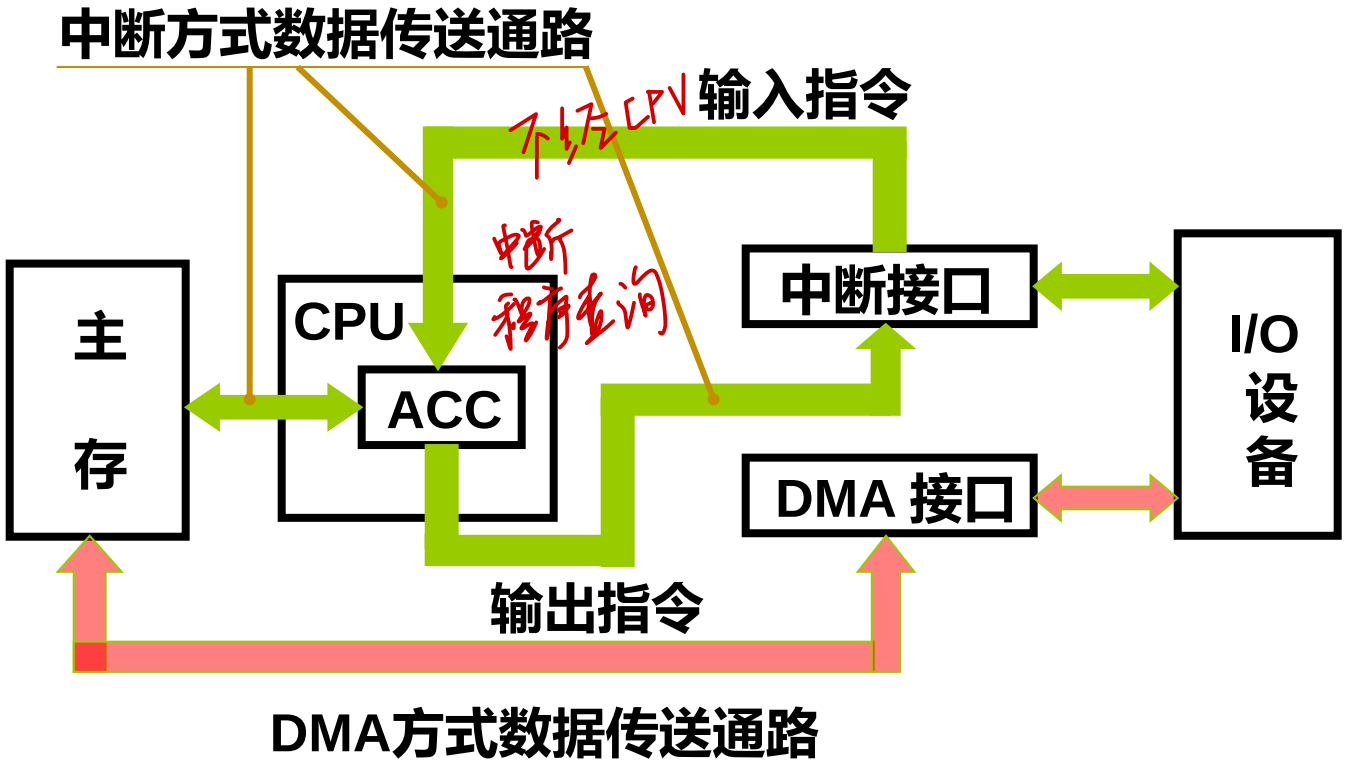
5.6.1 DMA方式的特点

一、DMA 方式的特点

- 特点：主机与I/O并行工作，主存和I/O之间有一条**直接数据通路**，CPU启动I/O后，不必查询I/O是否准备好，当I/O准备就绪后，发出DMA请求，此时CPU**不直接参与**I/O和主存间的信息交换，只是把外部总线的使用权暂时交付DMA，仍然可以完成自身内部的操作，所以不必中断现行程序，只需暂停一个或几个**存取周期**访存，CPU效率更高。

5.6.1 DMA方式的特点

1. DMA 和程序中断两种方式的数据通路



5.6.1 DMA方式的特点

2. DMA与主存交换数据三种工作方式

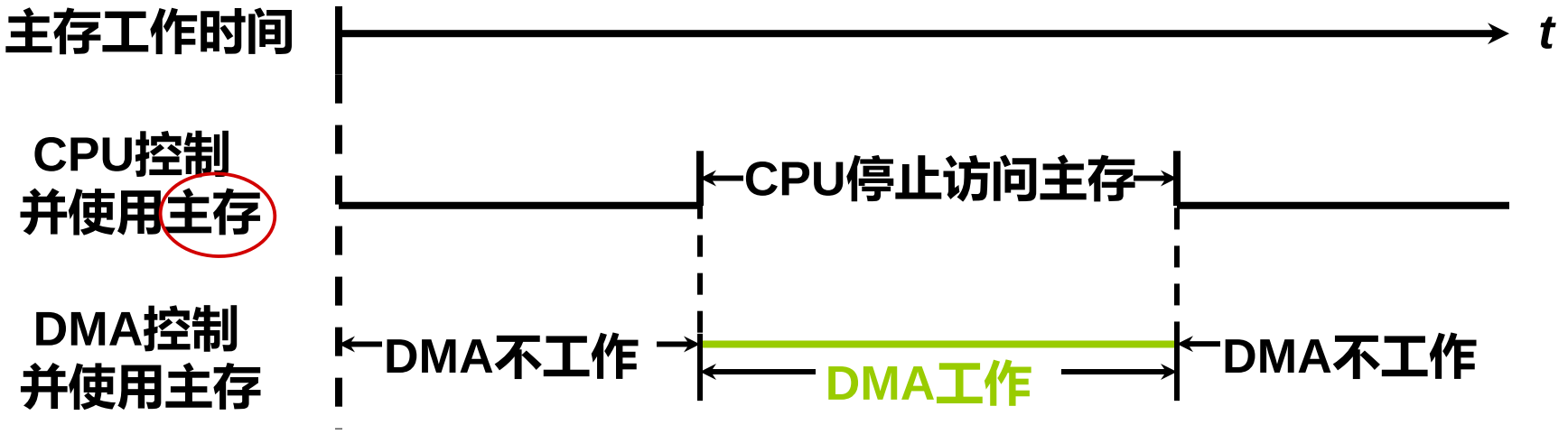
(1) 停止 CPU 访问主存

占用总线

优点： 控制简单

缺点： CPU 处于不工作状态或保持状态

未充分发挥 CPU 对主存的利用率 低



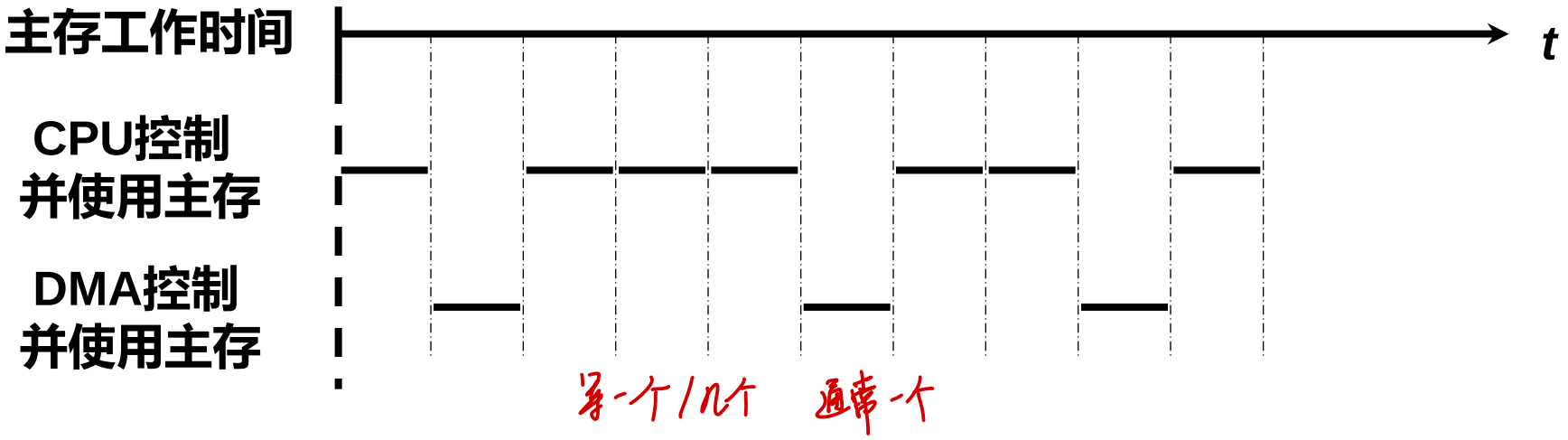
5.6.1 DMA方式的特点

(2) 周期挪用（或周期窃取）

DMA 访问主存有三种可能情况：

- CPU 此时不访存，DMA直接访问主存
- CPU 正在访存，等存取周期结束CPU才能让出总线
- CPU 与 DMA **同时请求访存**

此时 **CPU 将总线控制权让给 DMA**



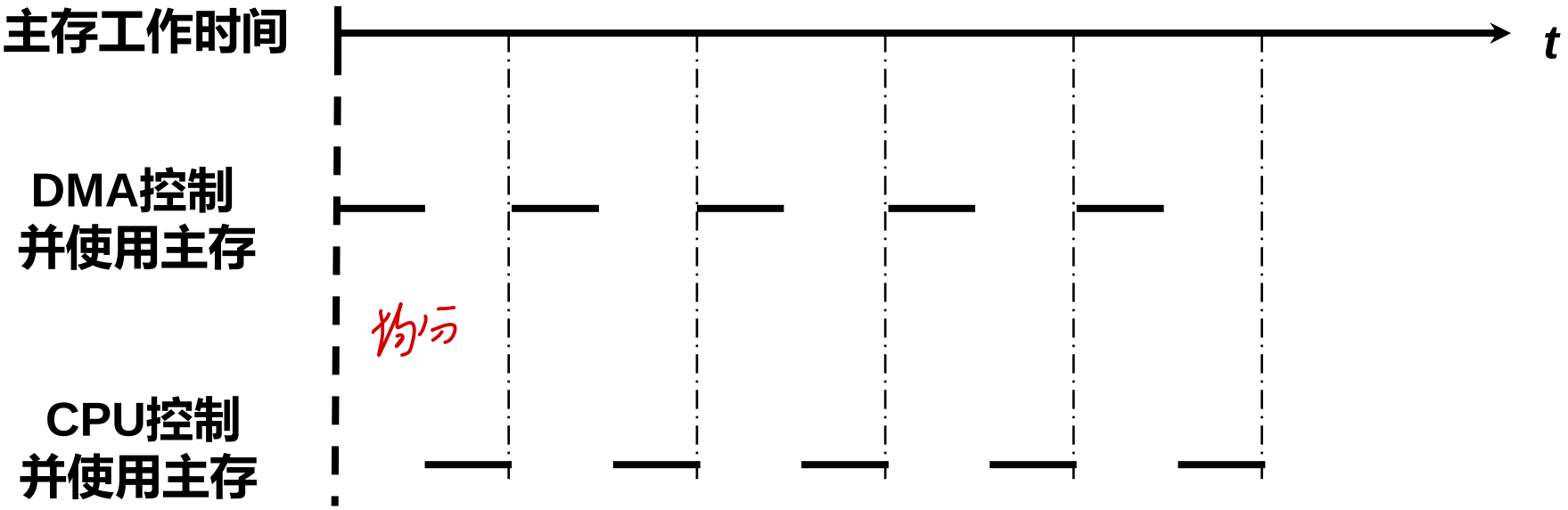
5.6.1 DMA方式的特点

(3) DMA 与 CPU 交替访问

适用于CPU工作周期**大于**主存存取周期的情况。

CPU **工作周期** → **所有指令执行过程中的一个基准时间**

- C_1 专供 DMA 访存
- C_2 专供 CPU 访存



不需要**建立和归还**总线使用权

5.6.2 DMA 接口的功能和组成

一、DMA 接口的功能和组成

1. DMA 接口功能

(1) 向 CPU **申请** DMA 传送

(2) 处理总线 **控制权的转交**

(3) **管理** 系统总线、**控制** 数据传送

(4) **确定** 数据传送的 **首地址和长度**

修正 传送过程中的数据地址和数据长度

(5) DMA 传送结束时，**给出操作完成信号**

DAR 存放传送数据的主存起始地址，每传送一个字后，DAR 内容自动加 1，指向主存下一个存储区

WC 记录待传送的字数，每传送一个字后，WC 内容自动减 1，减为 0 表示传送结束

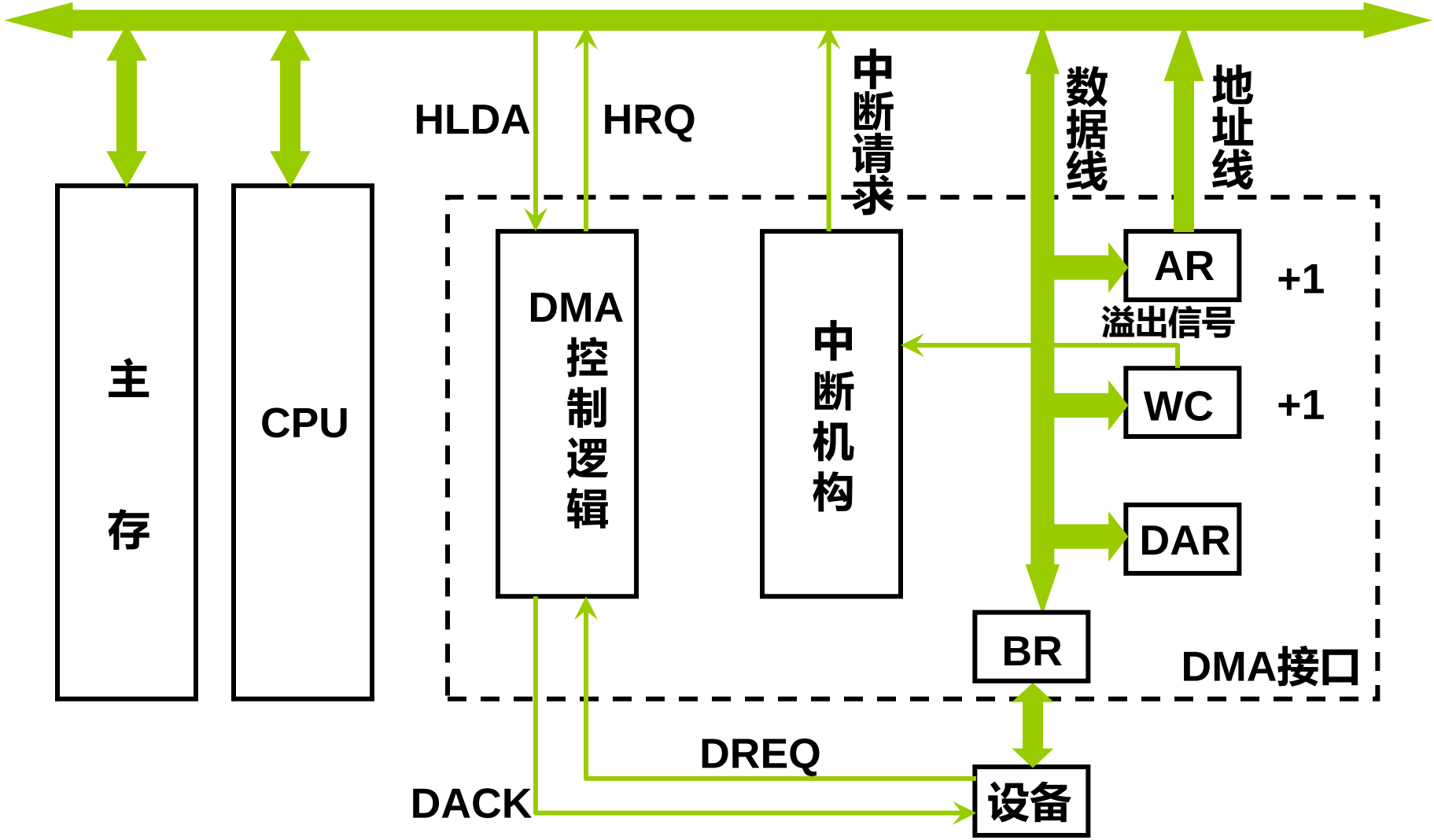
AR 存放传送数据的主存起始地址，每传送一个字后，AR 内容自动加 1，指向主存下一个存储区

的字地址；如果数据块传送方向与地址增加方向相反，则 AR 内容自动减 1

WC 记录待传送的字数，每传送一个字后，WC 内容自动减 1，减为 0 表示传送结束

进位时，表示待传送的数据块已全部传送完毕。

2. DMA 接口组成



5.6.3 DMA 的工作过程

一、DMA 的工作过程

1. DMA 传送过程

预处理、数据传送、后处理

(1) 预处理

通过几条输入输出指令预置如下信息

- 通知 DMA 控制逻辑传送方向（入/出）
- 设备地址——→DMA 的 DAR
- 主存地址——→DMA 的 AR
- 传送字数——→DMA 的 WC

5.6.3 DMA 的工作过程

(2) DMA 传送过程示意图

CPU

预处理:
主存起始地址 → DMA
设备地址 → DMA
传送数据个数 → DMA
启动设备

↓

数据传送:
继续执行主程序
同时完成一批数据传送

↓

后处理:
中断服务程序
做 DMA 结束处理

↓

继续执行主程序

数据传送

DMA请求

否 ← 允许传送? → 是

主存地址送总线
数据送I/O设备（或主存）
主存地址 加 1
传送个数 减 1

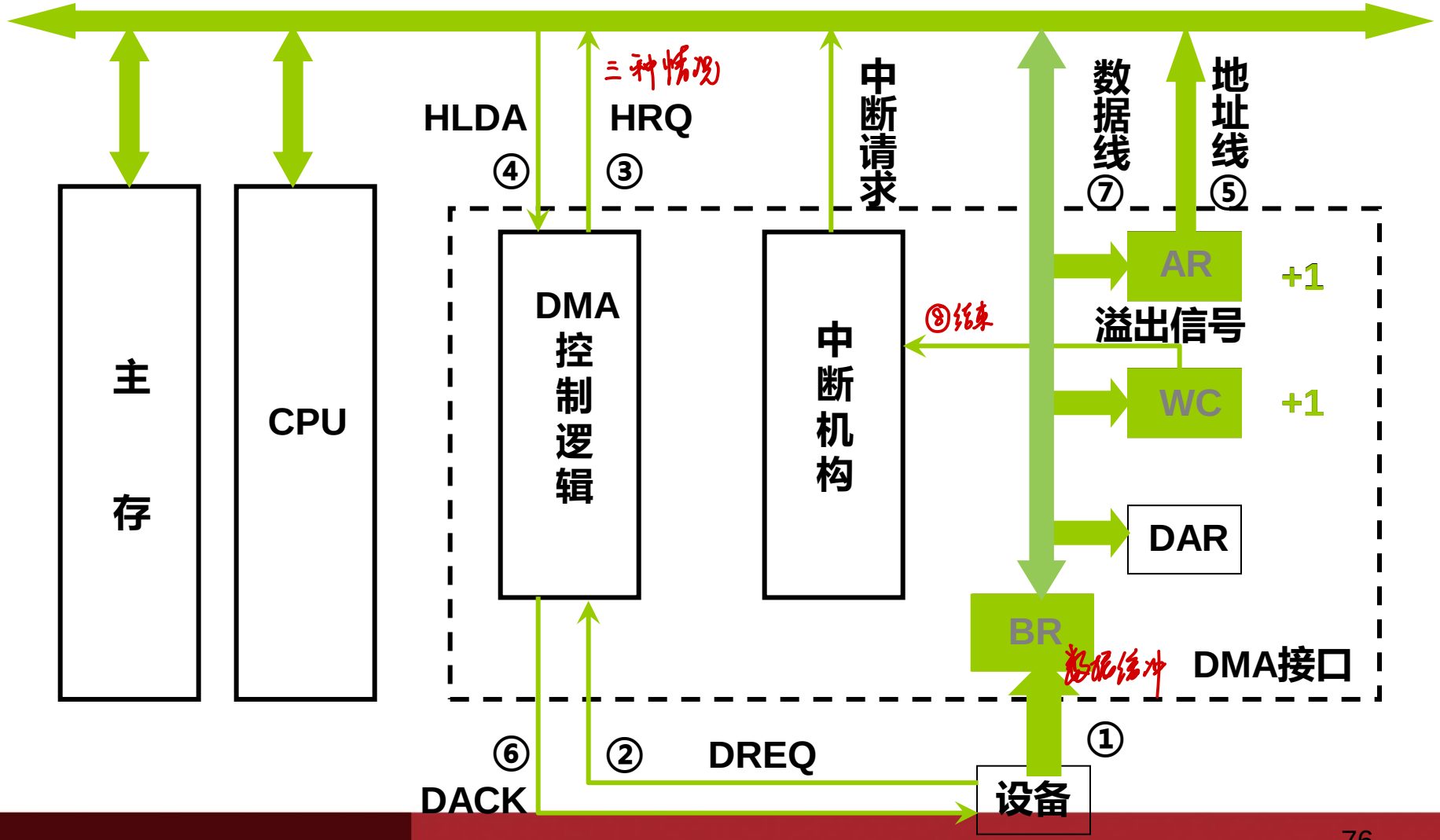
一次
一个
挪用一个存取周期

否 ← 数据块传送结束? → 是

向CPU申请 程序中断

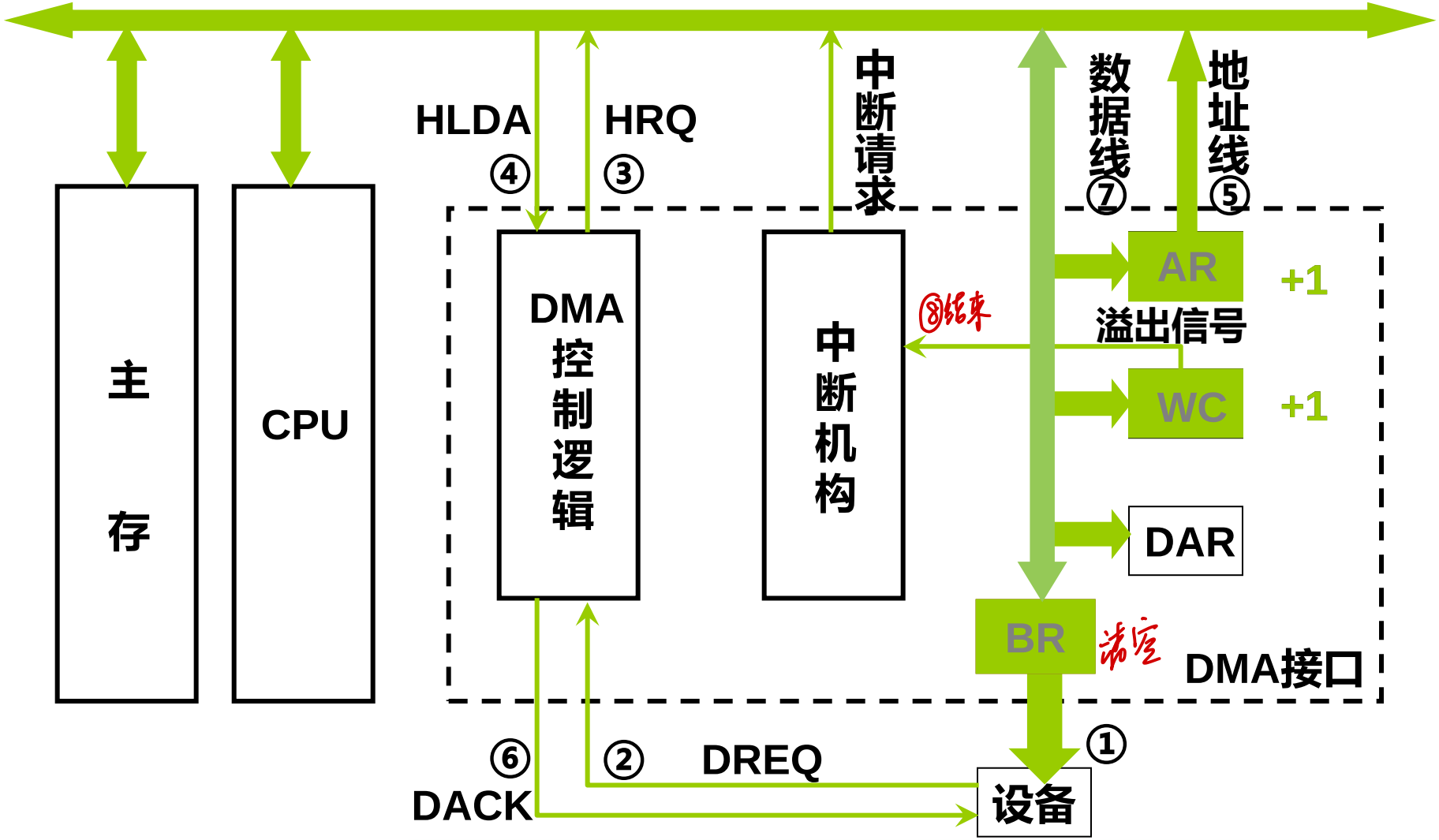
5.6.3 DMA 的工作过程

(3) 数据传送过程（输入）



5.6.3 DMA 的工作过程

(4) 数据传送过程（输出）



5.6.3 DMA 的工作过程

(5) 后处理

CPU

校验送入主存的数是否正确

是否继续用 DMA

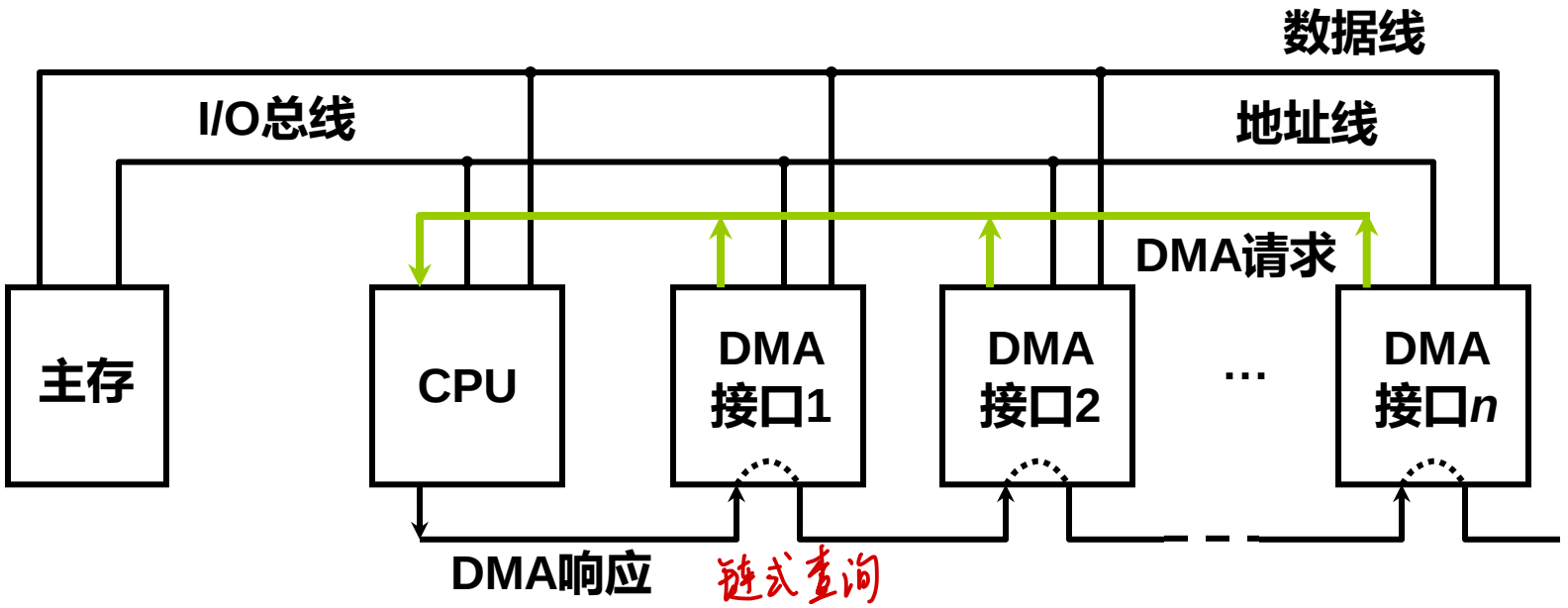
测试传送过程是否正确，错则转诊断程序

由中断服务程序完成

5.6.3 DMA 的工作过程

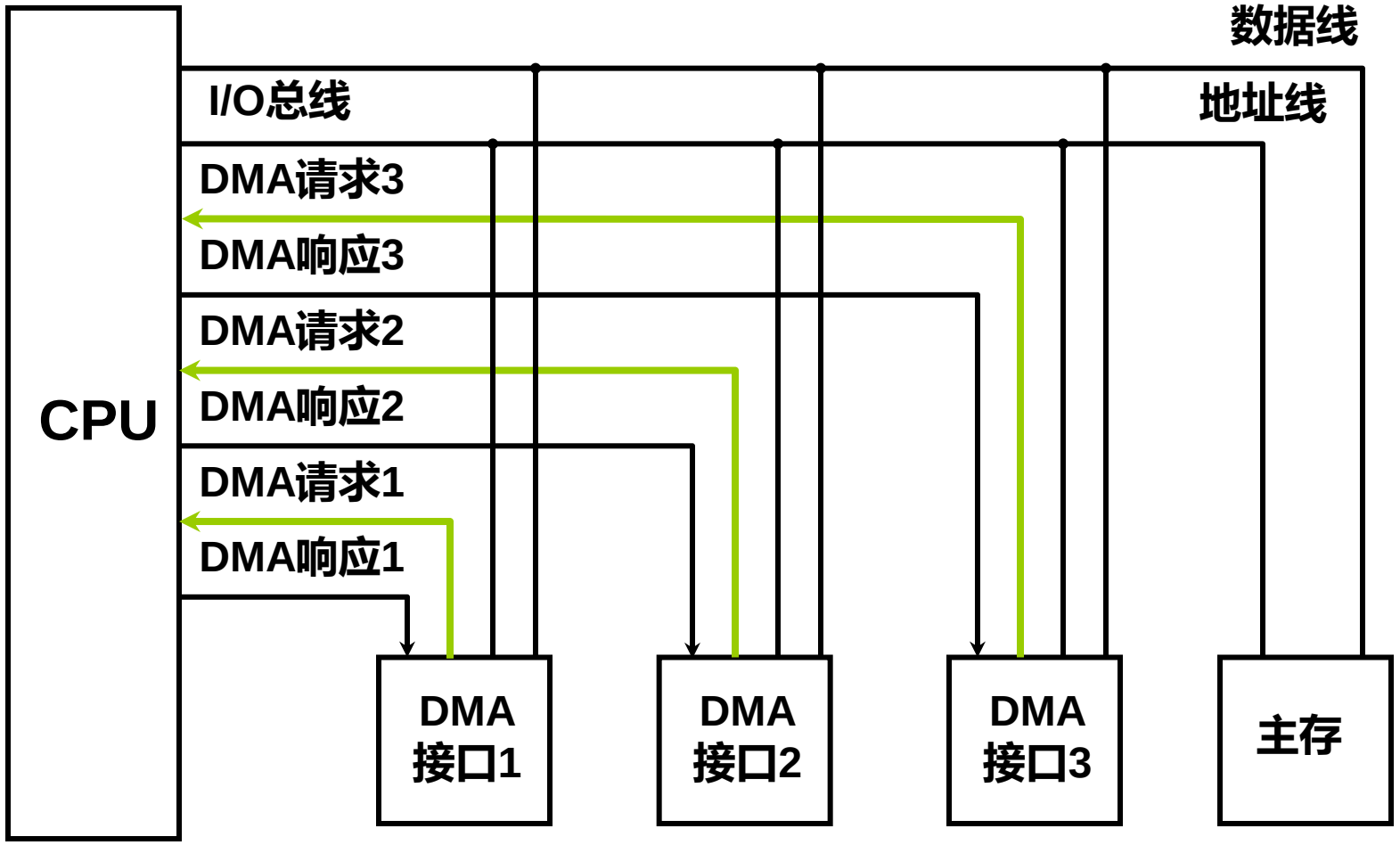
2. DMA 接口与系统的连接方式

(1) 具有公共请求线的 DMA 请求



5.6.3 DMA 的工作过程

(2) 独立的 DMA 请求



5.6.3 DMA 的工作过程

3. DMA 方式与程序中断方式的比较

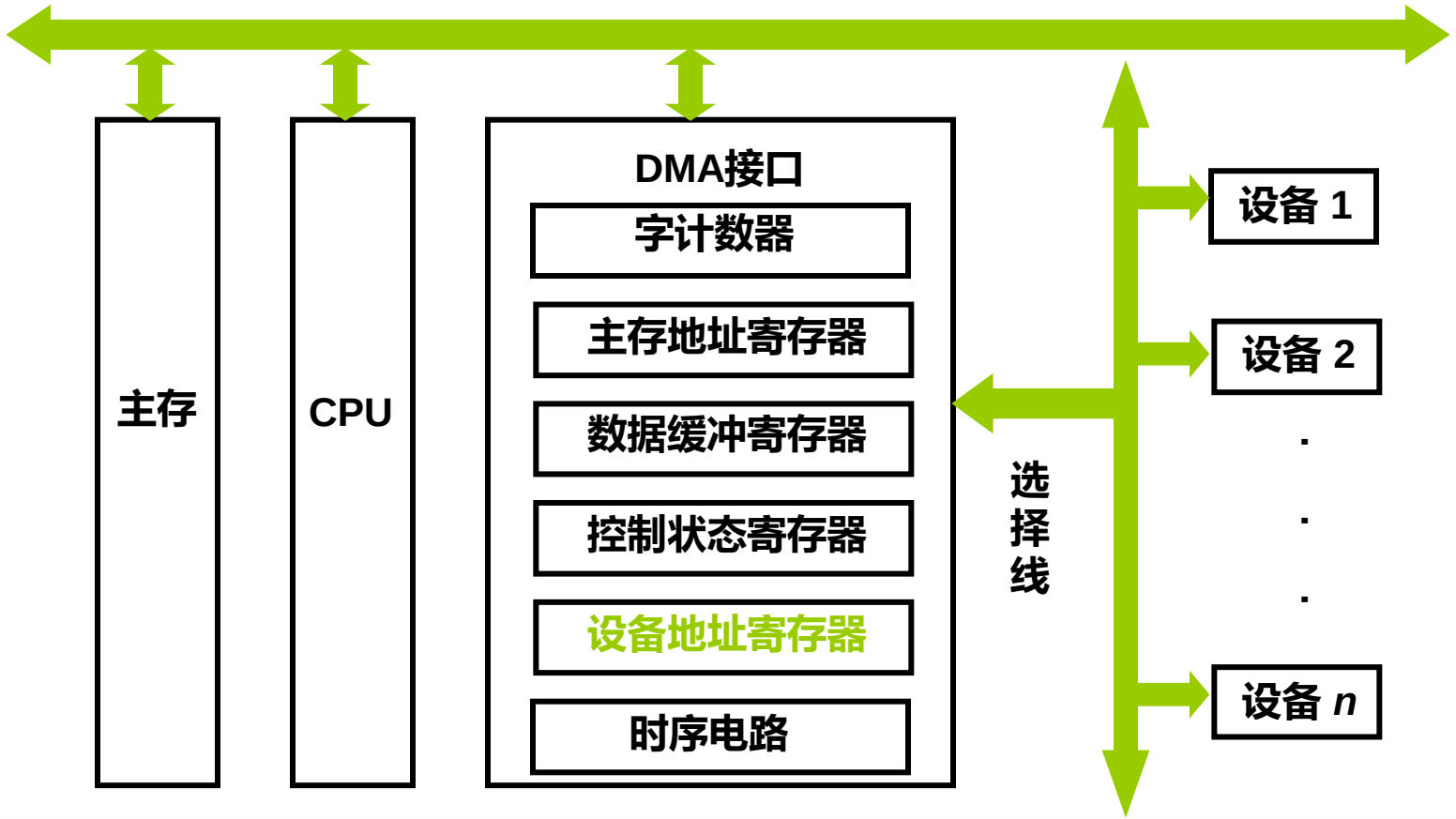
	中断方式	DMA 方式
(1) 数据传送	程序	硬件 DMA控制逻辑
(2) 响应时间	指令执行结束	存取周期结束
(3) 处理异常情况	能 只有中断	不能
(4) 中断请求	传送数据	后处理 传送完后
(5) 优先级	低	高

5.6.4 DMA接口的类型

一、DMA 接口的类型

1. 选择型　在 **物理上** 连接 **多个** 设备
在 **逻辑上** 只允许连接 **一个** 设备

同一时间只有一个设备

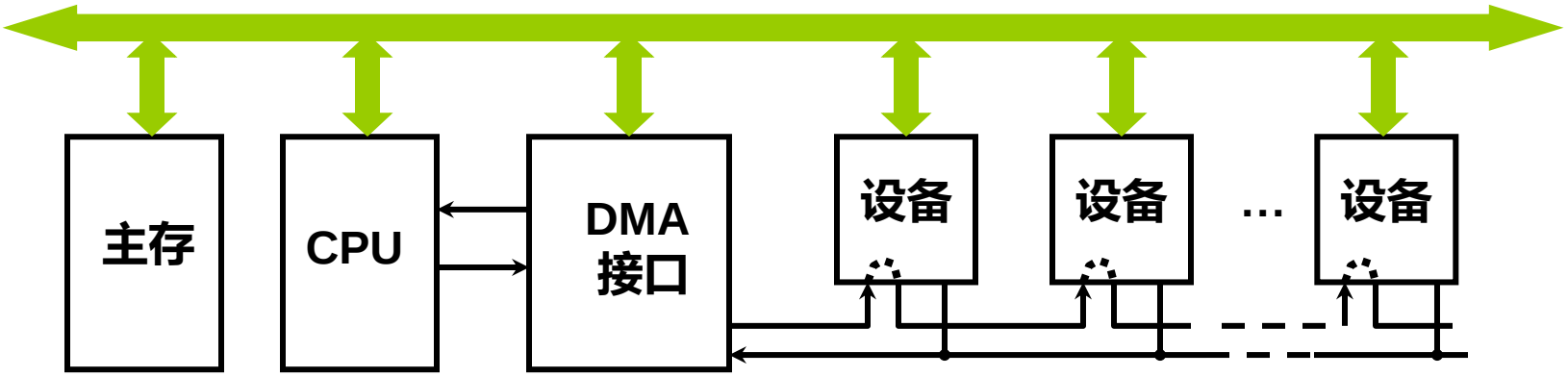


5.6 DMA 方式

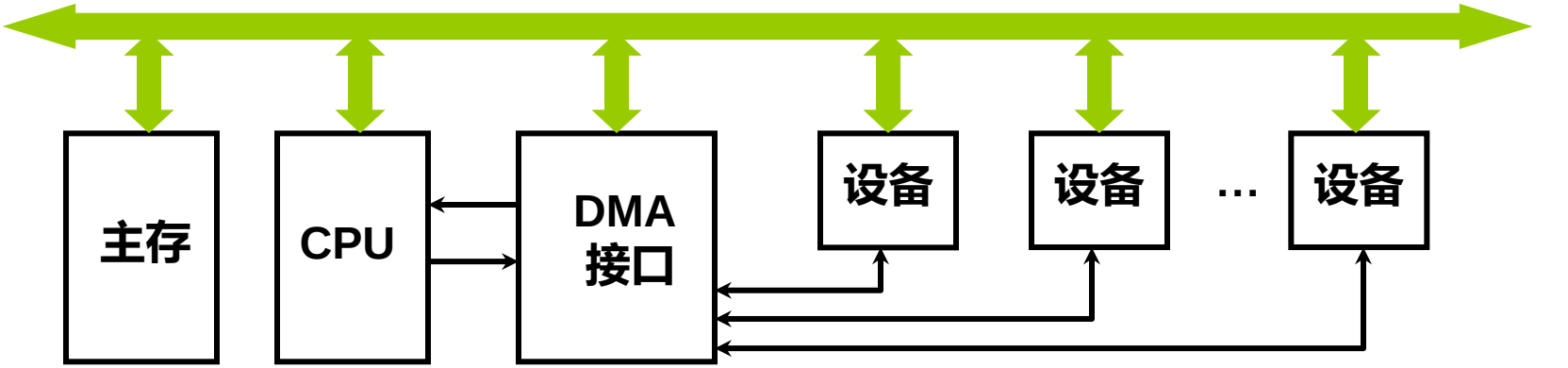
2. 多路型

在 **物理上** 连接 **多个** 设备
在 **逻辑上** 允许连接 **多个** 设备同时工作

链式

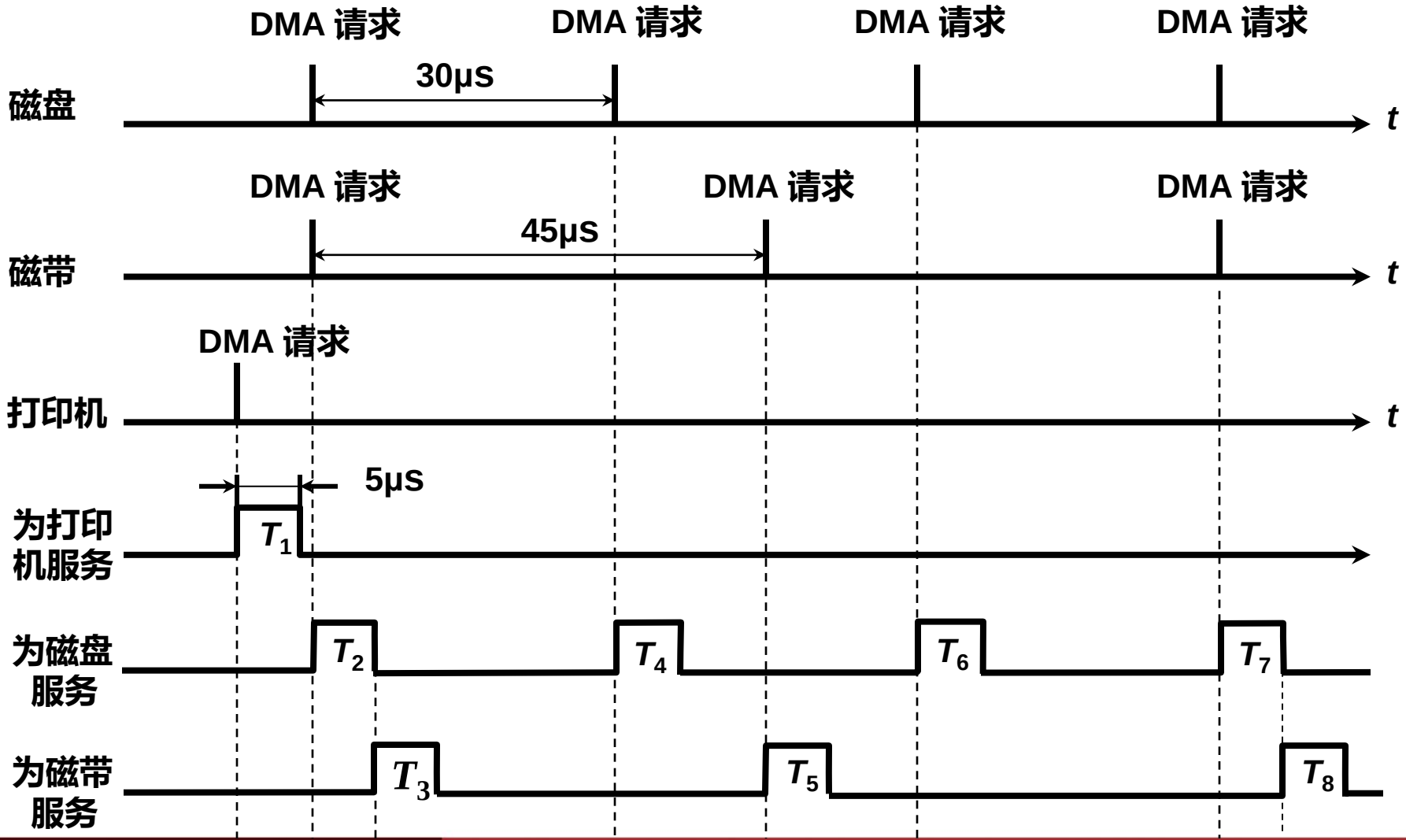


独立请求式



5.6 DMA 方式

3. 多路型 DMA 接口的工作原理



DMA技术应用

- IDE硬盘、SATA硬盘等高速辅助存储器；
- 显示卡、PCIE、一部分网卡都是支持DMA的，USB 2.0/3.0设备也是支持DMA的，这是指在底层驱动支持。
- 采用数据块传送方式的外围设备，如行式打印机，激光打印机，卡片阅读机，部分绘图仪等。

读万卷书，行万里路。

5.6.2 DMA 接口的功能和组成

- DMAC（Direct Memory Access Controller）是指直接内存访问控制器。
- 数据传送方式中，它是实现**地址的修改**与**传送字节数**技术的主要功能部件。

5.6.2 DMA 接口的功能和组成

3. DMA 与主存交换数据的三种传输方式

（1）单字节传输方式；

一次DMAC只传输一个字节，效率很低。

但是在传输过程中**CPU有机会获取对总线的控制器**。

（2）数据块传输方式；

数据以数据块的方式进行传输。

只要DREQ启动就会连续地传送数据块。

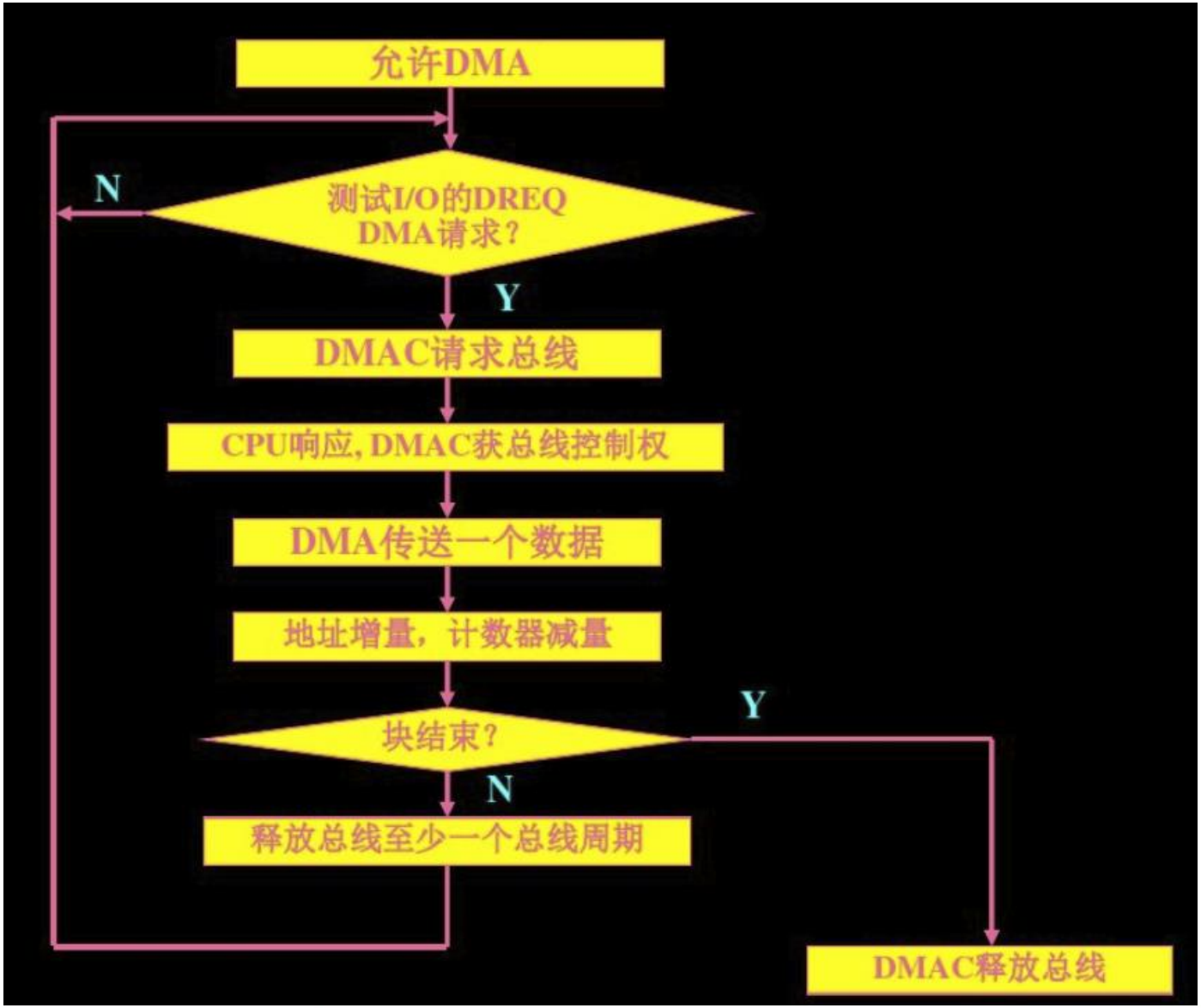
一次请求传送一个数据块，效率高。

在数据的传送期间，CPU长时间无法控制总线。

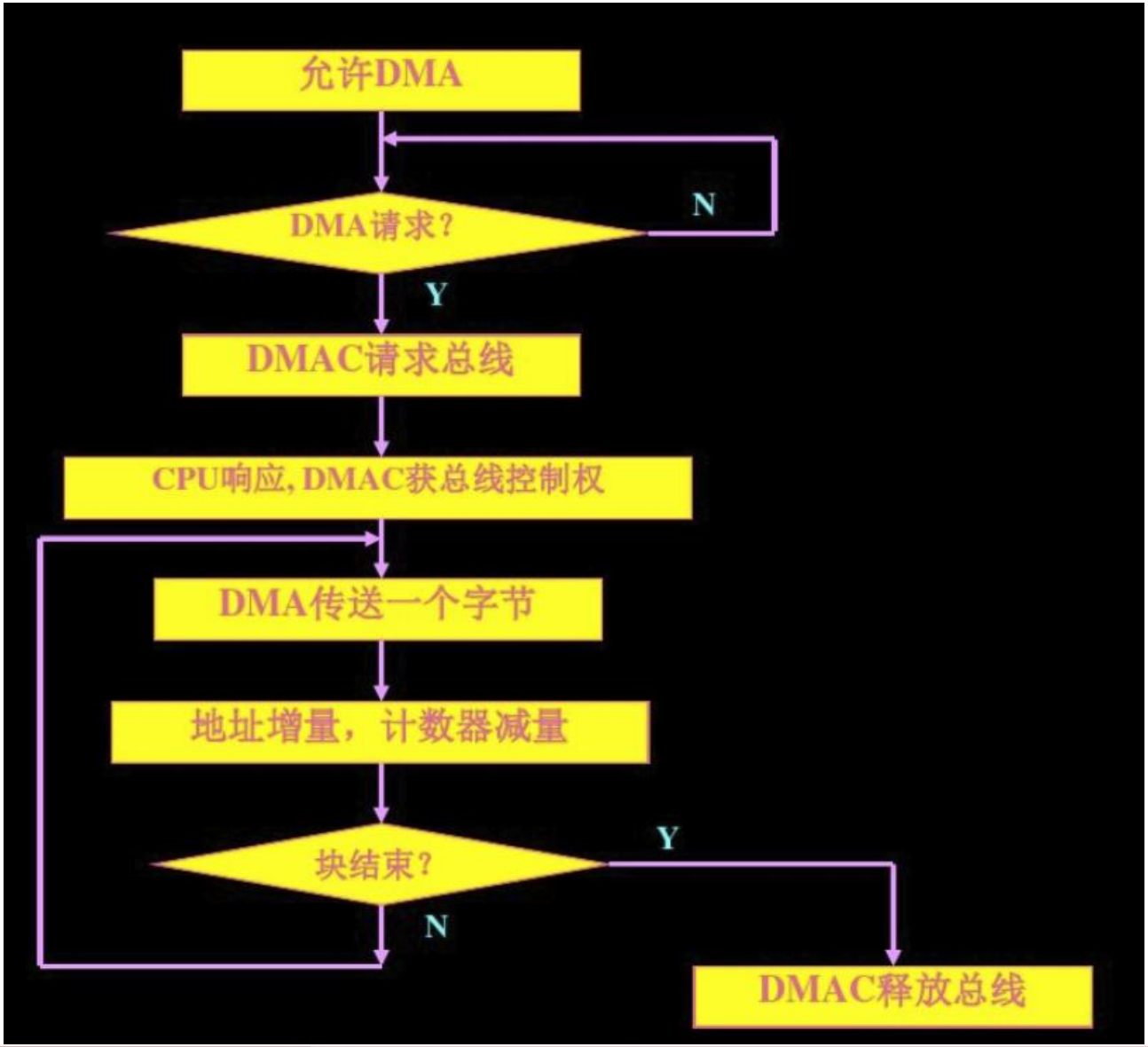
（3）按需传输方式；

DREQ信号有效就连续传输数据，否则不能进行数据的传输。

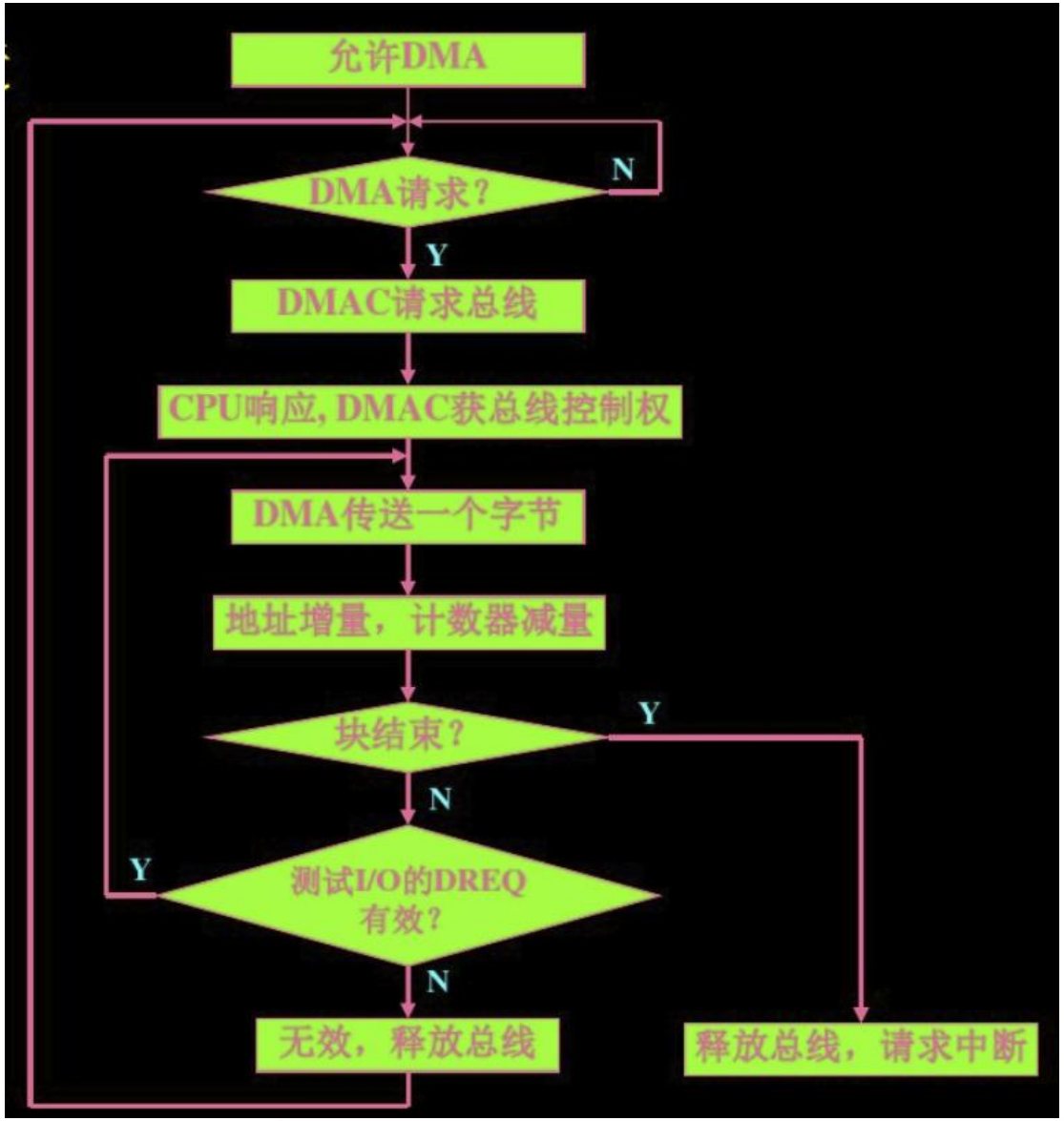
单字节传输方式


允许DMA
测试I/O的DREQ
DMA请求？
N
Y
DMAC请求总线
CPU响应，DMAC获总线控制权
DMA传送一个数据
地址增量，计数器减量
块结束？
Y
N
释放总线至少一个总线周期
DMAC释放总线

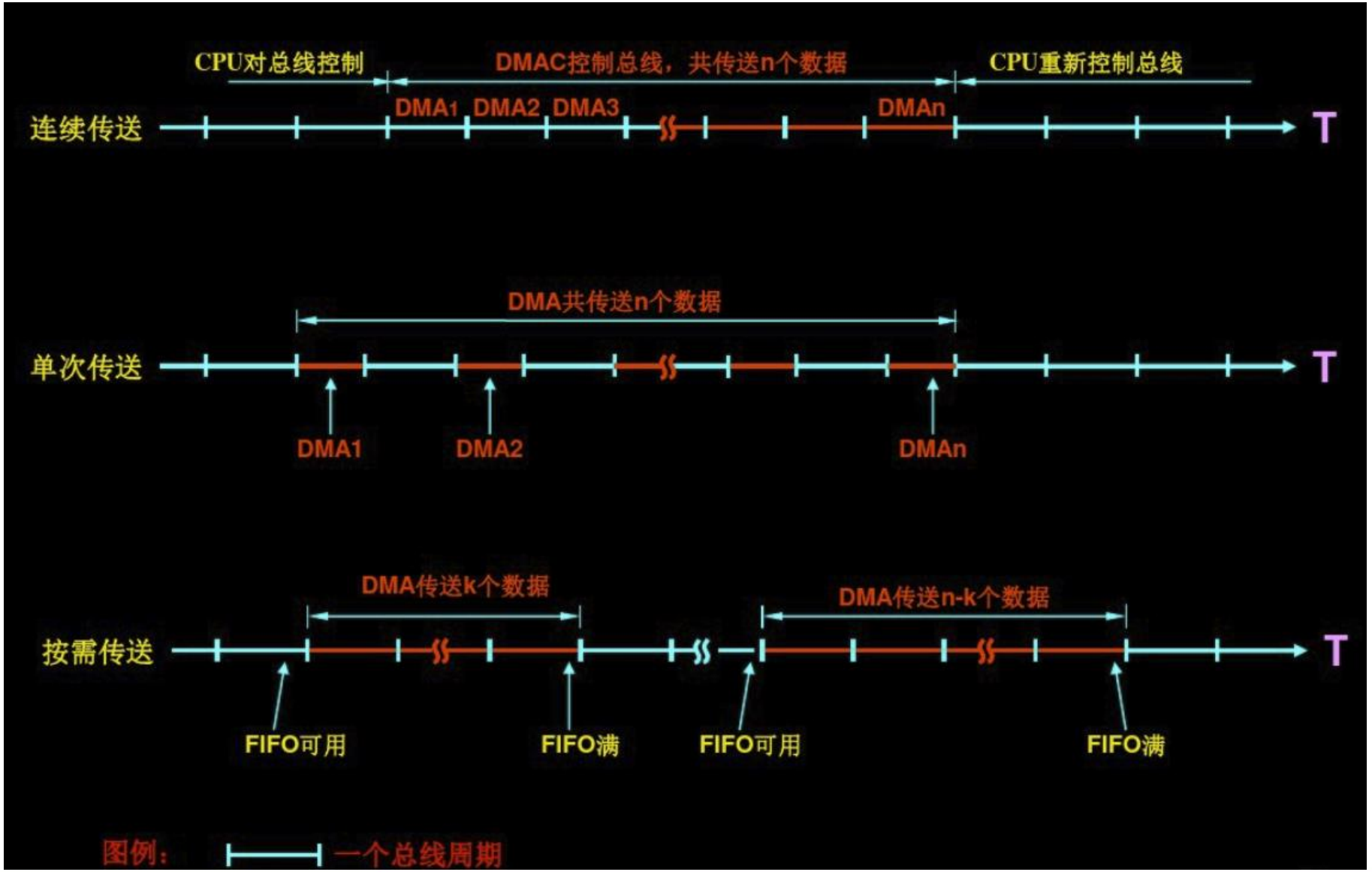
数据块传输方式

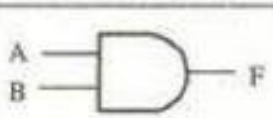
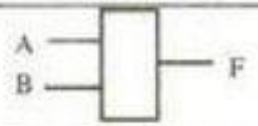
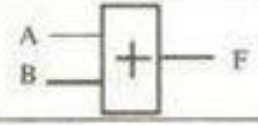
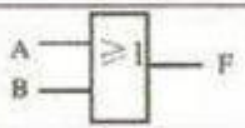
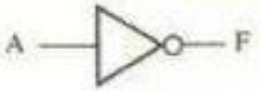
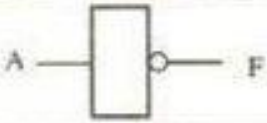
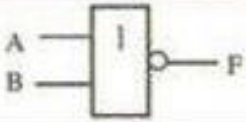
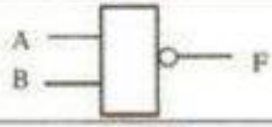
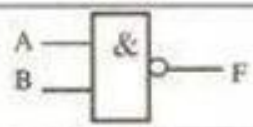
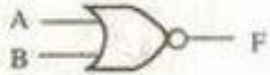
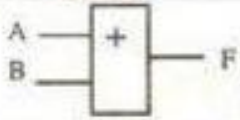
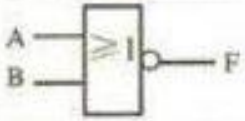
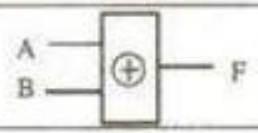
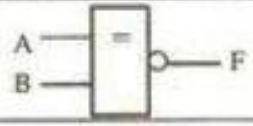
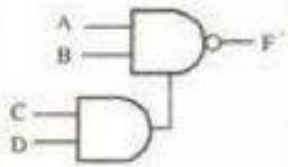
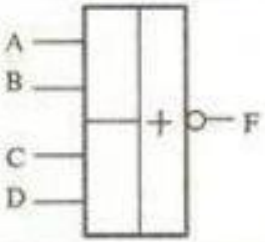
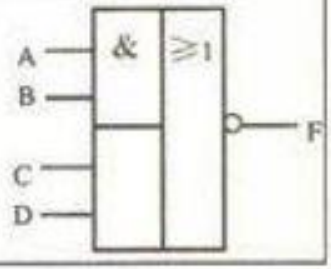

允许DMA
DMA请求?
N
Y
DMAC请求总线
CPU响应, DMAC获总线控制权
DMA传送一个字节
地址增量，计数器减量
块结束?
Y
N
DMAC释放总线

按需传输方式

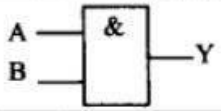
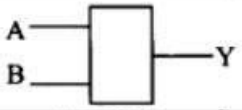
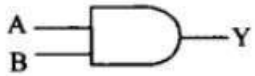
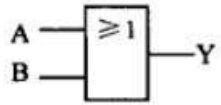
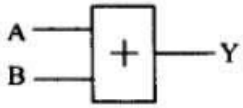
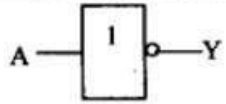
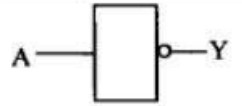
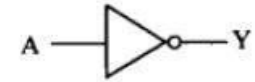
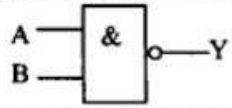
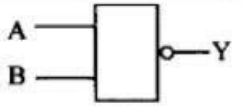
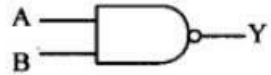
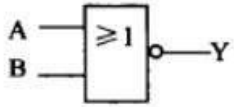
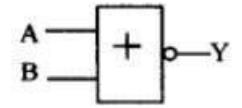
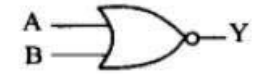
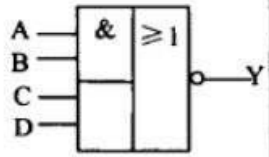
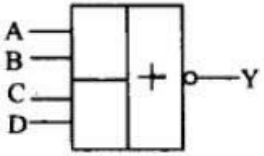
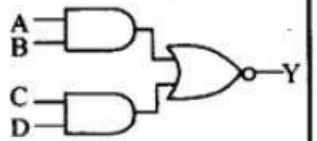
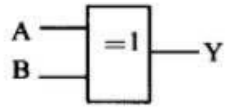
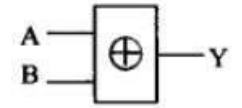
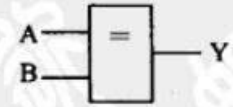
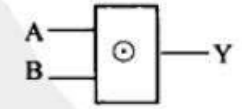

允许DMA
DMA请求?
N
Y
DMAC请求总线
CPU响应, DMAC获总线控制权
DMA传送一个字节
地址增量，计数器减量
块结束?
Y
N
Y
测试I/O的DREQ
有效?
N
无效，释放总线
释放总线，请求中断

三种传输方式示意图


CPU对总线控制
DMAC控制总线，共传送n个数据
CPU重新控制总线
连续传送
DMA1
DMA2
DMA3
DMAn
T
DMA共传送n个数据
单次传送
DMA1
DMA2
DMAn
T
DMA传送k个数据
DMA传送n-k个数据
按需传送
FIFO可用
FIFO满
FIFO可用
FIFO满
T
图例：
一个总线周期

逻 辑 门	IEEE 推荐符号	我国部颁符号	输出表达式	标 准 符 号
"与"门			F=A·B	
"或"门			F=A+B	
"非"门			$F=\overline{A}$	
"与非"门			$F=\overline{A\cdot B}$	
"或非"门			$F=\overline{A+B}$	
"异或"门			$F=A\oplus B$ $=A\overline{B}+\overline{A}B$	
"与或非"门			$F=\overline{AB+CD}$	

中国电子爱好者DZAHZ.COM

名称	国标符号	曾用符号	国外流行符号
与			
或			
非			
与非			
或非			
与或非			
异或			
同或			

计算机组成原理

第7章 指令系统

系统结构研究所

大　纲

四.　指令系统

（一）　指令格式

1、指令的基本格式

2、定长操作码指令格式

3、扩展操作码指令格式

（二）　指令的寻址方式

1、有效地址的概念

2、数据寻址和指令寻址

3、常见寻址方式

（三）　CISC和RISC的基本概念

Contents

1 7.1 机器指令

2 7.2 操作数类型和操作类型

3 7.3 寻址方式

4 7.4 指令格式举例

5 7.5 RISC 技术

7.1 机 器 指 令

指令：就是让计算机执行某种操作的命令。

指令系统：一台计算机中所有机器指令的集合称为这台计算机的指令系统。

系列计算机：指基本指令系统相同且基本体系结构相同的一系列计算机。

向下兼容

系列机能解决软件兼容问题的必要条件是该系列的各种机种有共同的指令集，而且新推出的机种的指令系统一定包含旧机种的所有指令，因此在旧机种上运行的各种软件可以不加任何修改地在新机种上运行。

7.1 机 器 指 令

指令的一般格式：

操作码字段	地址码字段

n位操作码字段的指令系统最多能够表示2^n条指令。

1. 操作码 指令应该执行什么性质的操作和具有何种功能。

(1) 长度固定（译码简单）

用于指令字长较长的情况，RISC

如 IBM 370 操作码 8 位

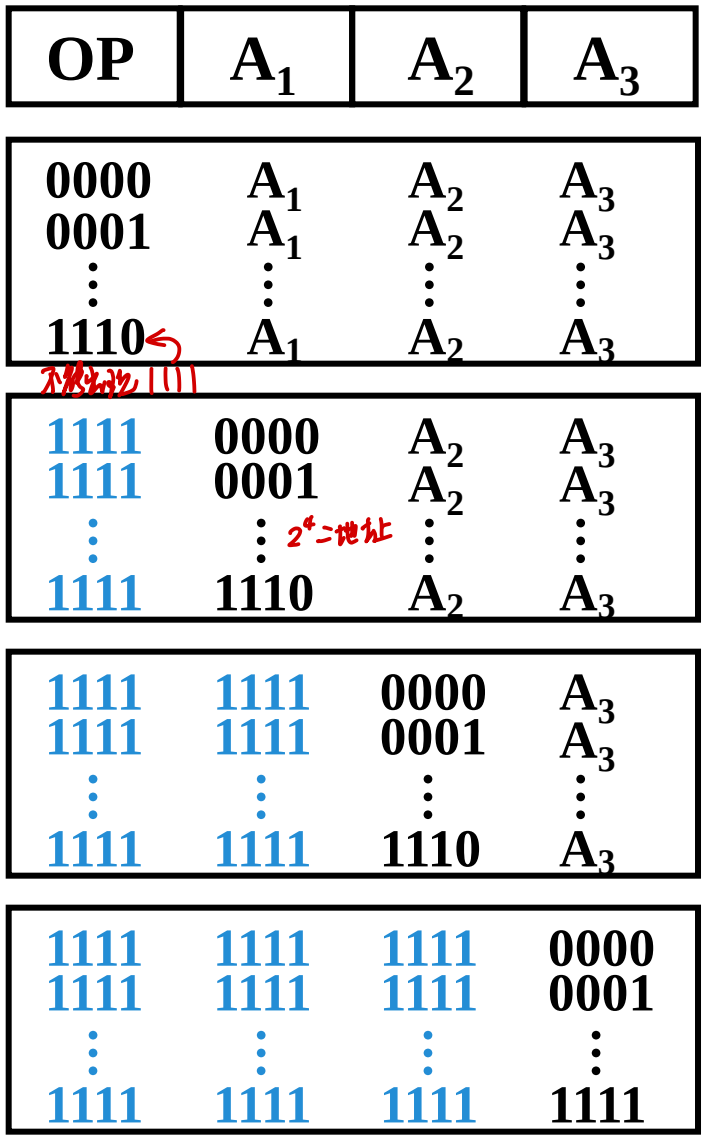
(2) 长度可变

操作码分散在指令字的不同字段中

(3) 扩展操作码技术

操作码的位数随地址数的减少而增加

①指令编码不够重复
②短码不能为长码前缀



(3) 扩展操作码技术

操作码的位数随地址数的减少而增加

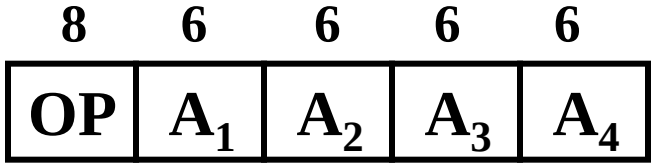
OP	A_1	A_2	A_3
4 位操作码			
0000	A_1	A_2	A_3
0001	A_1	A_2	A_3
⋮	⋮	⋮	⋮
1110	A_1	A_2	A_3
8 位操作码			
1111	0000	A_2	A_3
1111	0001	A_2	A_3
⋮	⋮	⋮	⋮
1111	1110	A_2	A_3
12 位操作码			
1111	1111	0000	A_3
1111	1111	0001	A_3
⋮	⋮	⋮	⋮
1111	1111	1110	A_3
16 位操作码			
1111	1111	1111	0000
1111	1111	1111	0001
⋮	⋮	⋮	⋮
1111	1111	1111	1111

三地址指令操作码
每减少一种可多构成
2^4 种二地址指令

二地址指令操作码
每减少一种可多构成
2^4 种一地址指令

2. 地址码

(1) 四地址



A_1 第一操作数地址

A_2 第二操作数地址

A_3 结果的地址

A_4 下一条指令地址

(A_1) OP $(A_2) \longrightarrow A_3$

设指令字长为 32 位

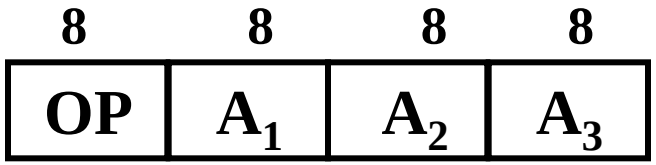
操作码固定为 8 位

4 次访存 ?

寻址范围 $2^6 = 64$

若 PC 代替 A_4

(2) 三地址



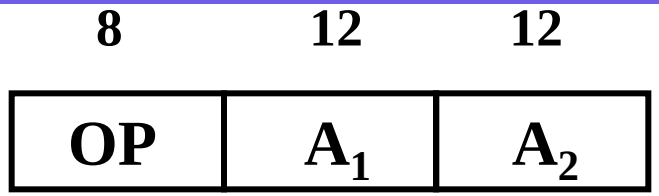
(A_1) OP $(A_2) \longrightarrow A_3$

4 次访存

寻址范围 $2^8 = 256$

若 A_3 用 A_1 或 A_2 代替

(3) 二地址



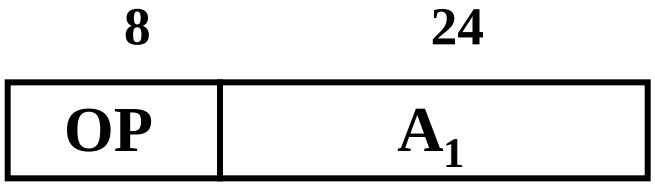
或

(A_1) OP $(A_2) \longrightarrow A_1$　　4 次访存

(A_1) OP $(A_2) \longrightarrow A_2$　　寻址范围 $2^{12} = 4$ K

若结果存于 ACC　3次访存　　若ACC 代替 A_1（或A_2）

(4) 一地址



2 次访存 { 取指 / 取操作数

(ACC) OP $(A_1) \longrightarrow$ ACC　　寻址范围 $2^{24} = 16$ M

(5) 零地址　无地址码

1.无需任何操作数，如空操作指令、停机指令等；2.所需操作数地址是默认的。

二、指令字长

指令字长决定于
- 操作码的长度
- 操作数地址的长度
- 操作数地址的个数

1. 指令字长 固定

指令字长 = 存储字长＝机器字长

2. 指令字长 可变

按字节的倍数变化

小结

- 当用一些硬件资源代替指令字中的地址码字段后
  - 可扩大指令的寻址范围
  - 可缩短指令字长
  - 可减少访存次数
- 当指令的地址字段为寄存器时

  三地址　OP　R_1，R_2，R_3

  二地址　OP　R_1，R_2

  一地址　OP　R_1
  - 可缩短指令字长
  - 指令执行阶段不访存

7.2 操作数类型和操作种类

一、操作数类型

地址	无符号整数
数字	定点数、浮点数、十进制数
字符	ASCII 标准为7位
逻辑数	逻辑运算

二、数据在存储器中的存放方式

字地址 低字节

0	3	2	1	0
4	7	6	5	4

字地址 为 低字节 地址

字地址 低字节

0	0	1	2	3
4	4	5	6	7

字地址 为 高字节 地址

存储器中的数据存放（存储字长为32位）

边界对准

				地址（十进制）
字（地址 0）				0
字（地址 4）				4
字节（地址11）	字节（地址10）	字节（地址 9）	字节（地址 8）	8
字节（地址15）	字节（地址14）	字节（地址13）	字节（地址12）	12
半字（地址18）✓		半字（地址16）✓		16
半字（地址22）✓		半字（地址20）✓		20
双字（地址24）▲				24
双字				28
双字（地址32）▲				32
双字				36

边界未对准

			地址（十进制）
字(地址2)		半字(地址0)	0
字节(地址7)	字节(地址6)	字(地址4)	4
半字(地址10)		半字(地址8)	8

三、操作类型

1. 数据传送

源	寄存器	寄存器	存储器	存储器
目的	寄存器	存储器	寄存器	存储器
例如	MOVE	STORE MOVE PUSH	LOAD MOVE POP	MOVE

置“1”，清“0”

2. 算术逻辑操作

加、减、乘、除、增 1、减 1、求补、浮点运算、十进制运算

与、或、非、异或、位操作、位测试、位清除、位求反

如 8086　ADD SUB MUL DIV INC DEC CMP NEG
AAA AAS AAM AAD
AND OR NOT XOR TEST

3. 移位操作

算术移位　逻辑移位

循环移位（带进位和不带进位）

4. 转移

(1) 无条件转移　JMP

(2) 条件转移

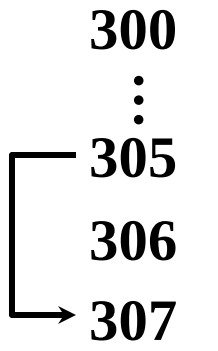
结果为零转　（Z = 1）JZ

结果溢出转　（O = 1）JO

结果有进位转（C = 1）JC

跳过一条指令 SKP（skip）

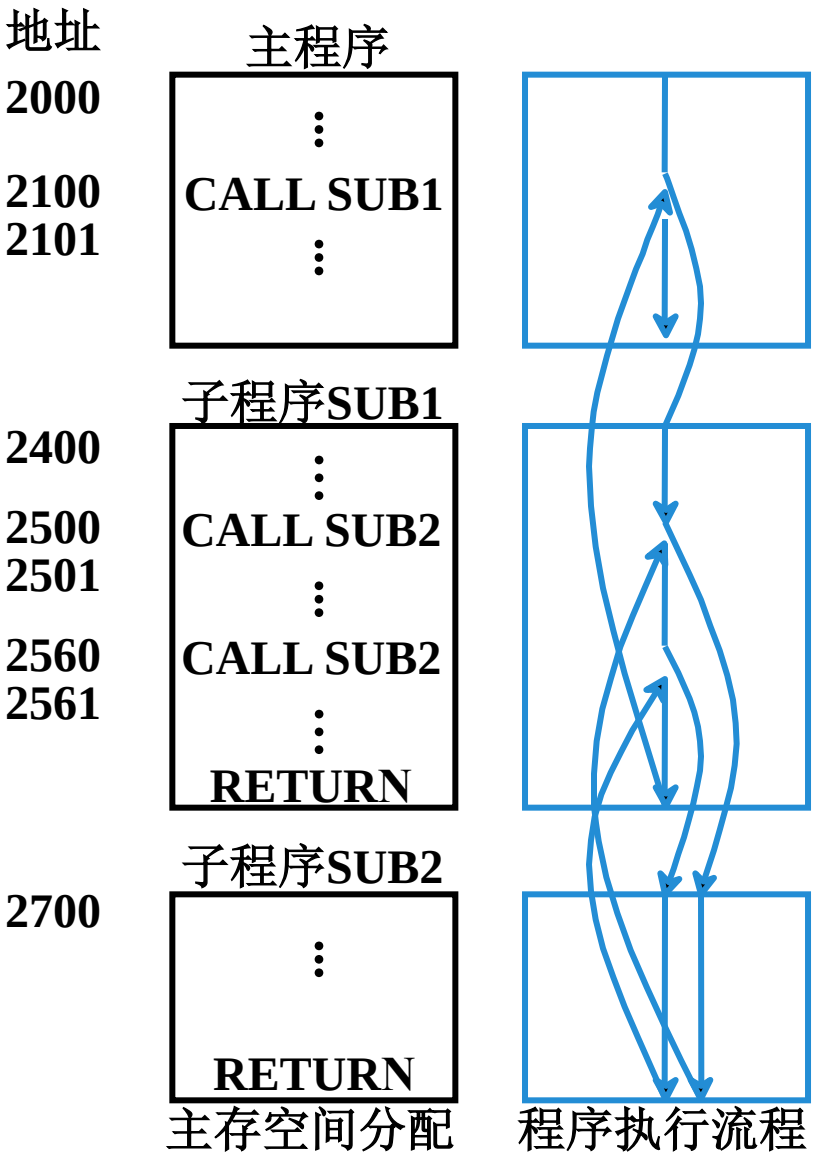
如



完成触发器

SKP DZ D = 0 则跳

(3) 调用和返回


地址
主程序
2000
2100
CALL SUB1
2101
子程序SUB1
2400
2500
CALL SUB2
2501
2560
CALL SUB2
2561
RETURN
子程序SUB2
2700
RETURN
主存空间分配
程序执行流程

(4) 陷阱（Trap）与陷阱指令

意外事故的中断

- 一般不提供给用户直接使用

  在出现事故时，由 CPU 自动产生并执行（隐指令）

- 设置供用户使用的陷阱指令

  如 8086　　INT　TYPE　　软中断

  8位常数，表示中断类型

  提供给用户使用的陷阱指令，完成系统调用

5. 输入输出

入　　端口地址 ⟶ CPU 的寄存器

如　　IN AL, *n*　　IN AL, DX

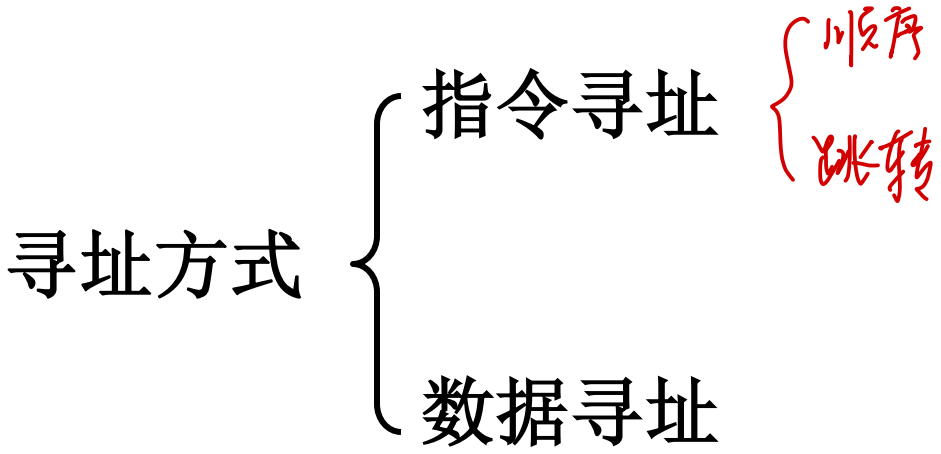
出　　CPU 的寄存器 ⟶ 端口地址

如　　OUT *n*, AL　　OUT DX, AL

7.3 寻址方式

寻址方式　确定 本条指令 的 操作数地址

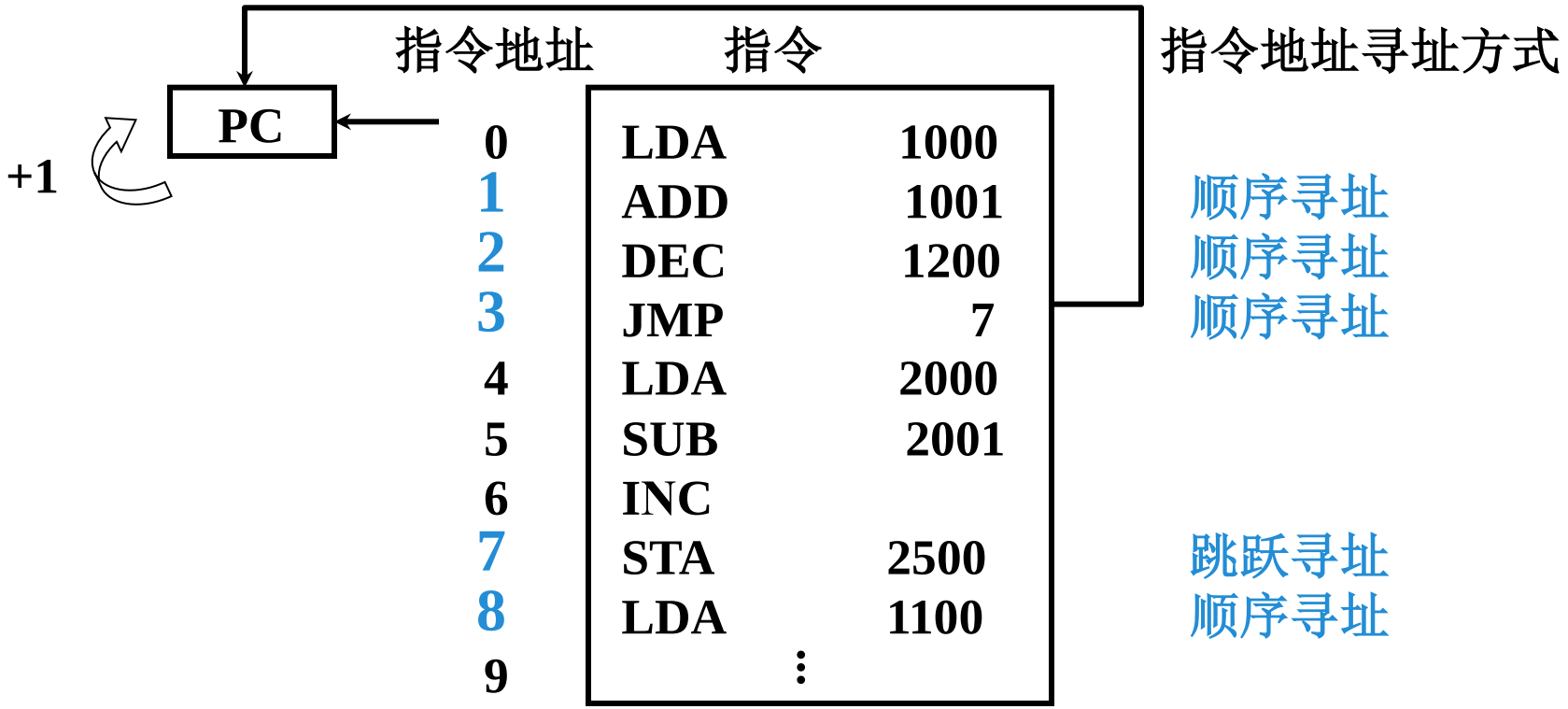
下一条 欲执行 指令 的 指令地址



一、指令寻址

顺序　　(PC) + 1 → PC　　下一条指令

跳跃　　由转移指令指出



二、数据寻址

操作码	寻址特征	形式地址 A

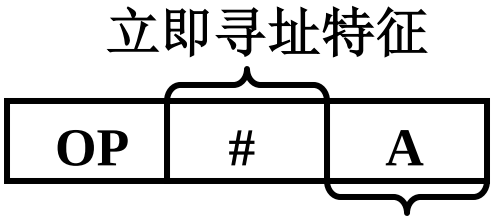
形式地址　　指令字中的地址　A

有效地址　通常不相等　操作数的真实地址　EA

约定　　指令字长 = 存储字长 = 机器字长

1. 立即寻址

形式地址 A 就是操作数



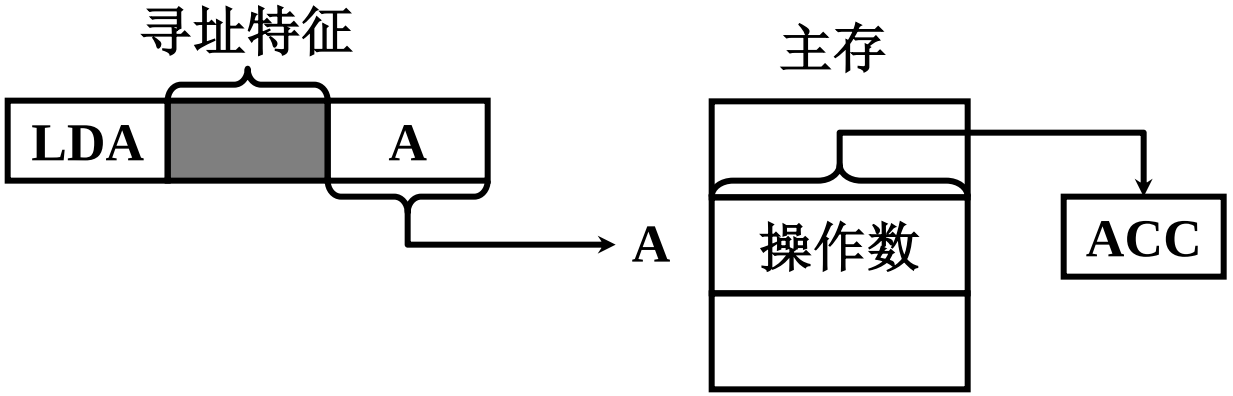
ADD #4

可正可负　补码

- 指令执行阶段不访存
- A 的位数限制了立即数的范围

2. 直接寻址

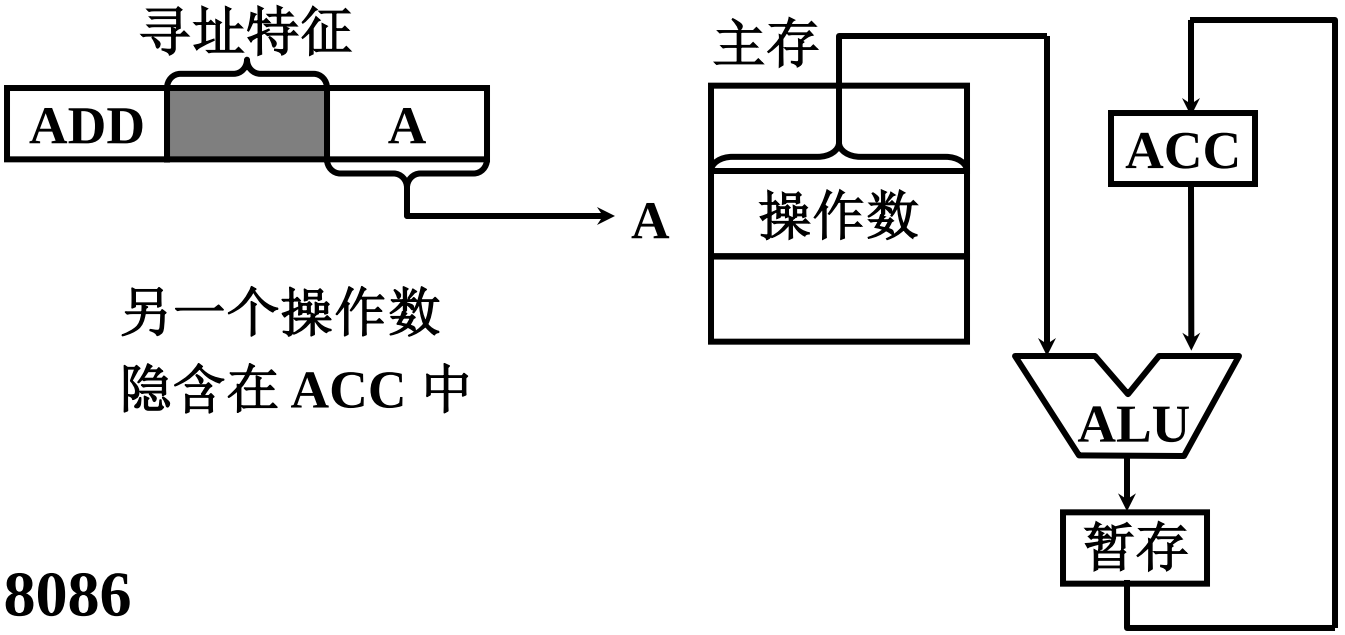
EA = A　　有效地址由形式地址直接给出



- 执行阶段访问一次存储器
- A 的位数决定了该指令操作数的寻址范围
- 操作数的地址不易修改（必须修改A）

3. 隐含寻址

操作数地址隐含在操作码或某个寄存器中



如 8086

MUL 指令　被乘数隐含在 AX（16位）或 AL（8位）中

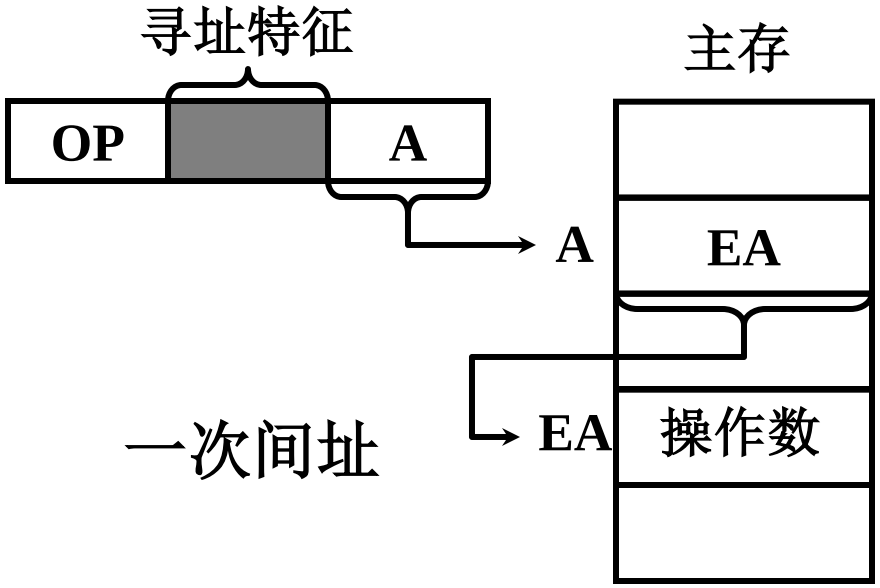
MOVS 指令　源操作数的地址隐含在 SI 中

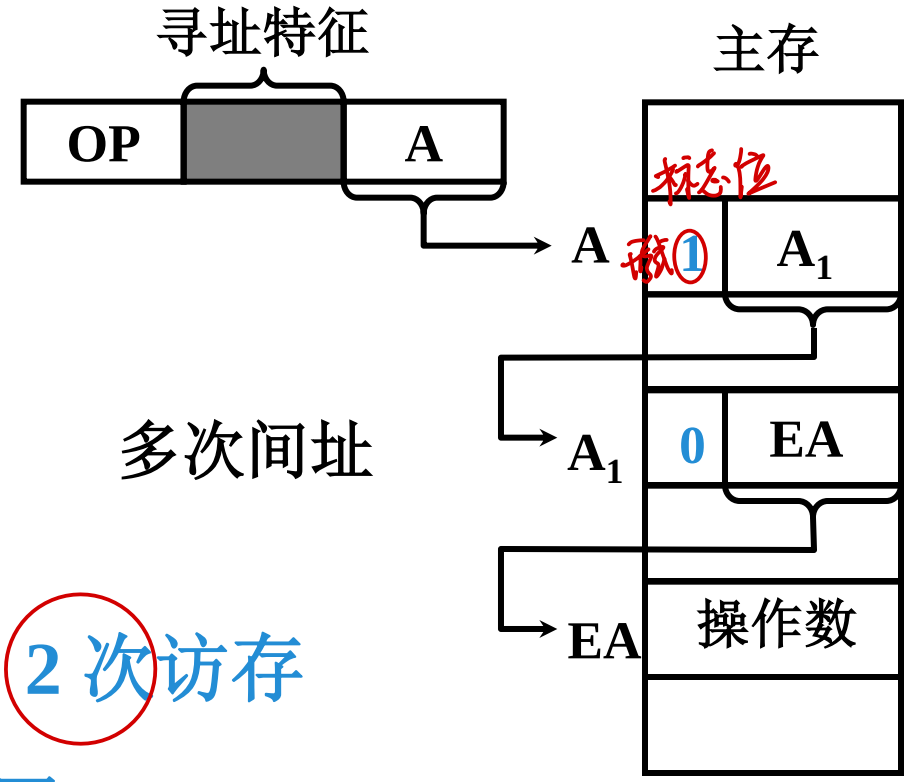
目的操作数的地址隐含在 DI 中

- 指令字中少了一个地址字段，可缩短指令字长 增加寻址范围

4. 间接寻址

EA =（A） 有效地址由形式地址间接提供

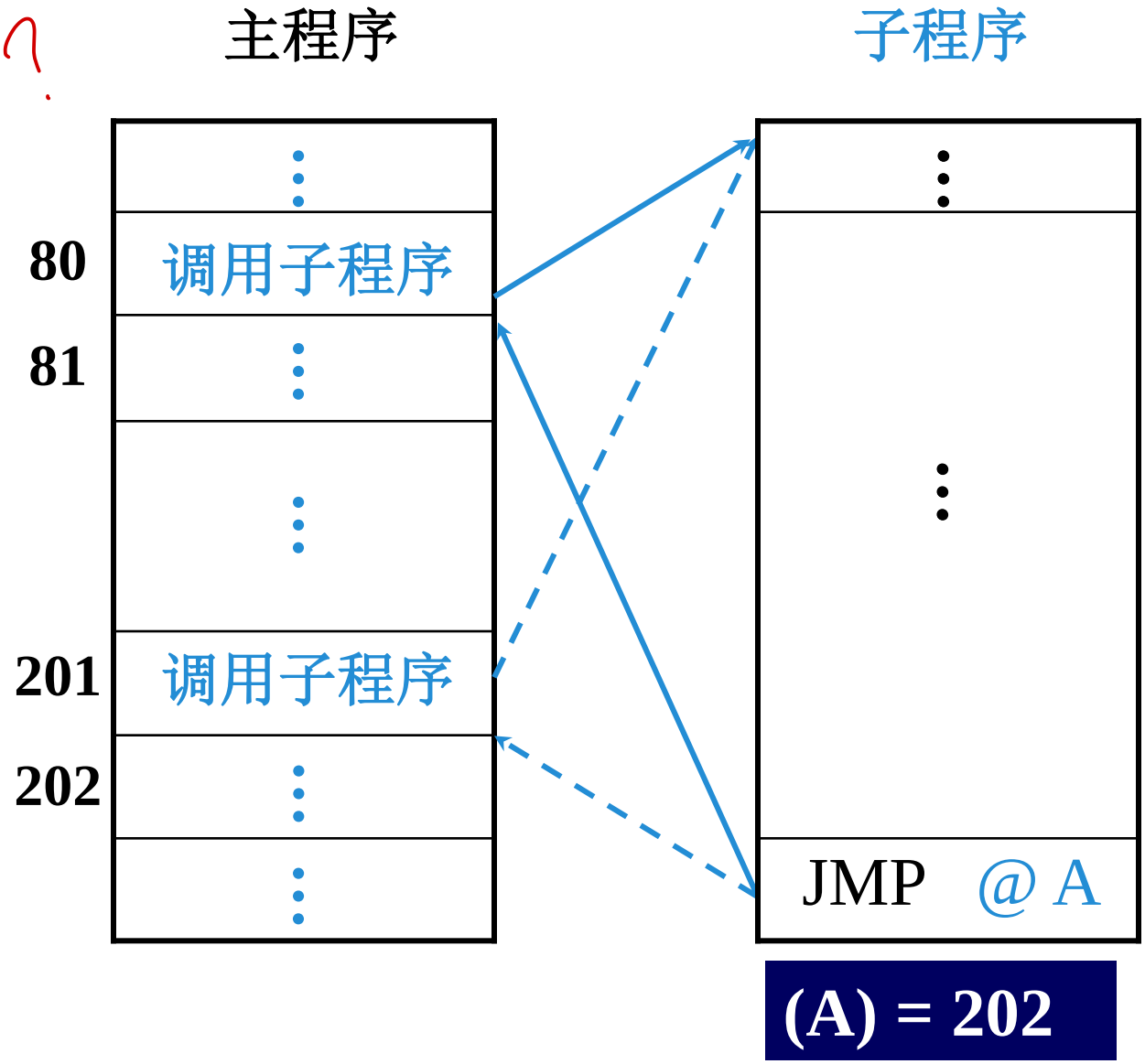




- 执行指令阶段 2 次访存
- 可扩大寻址范围
- 便于编制程序

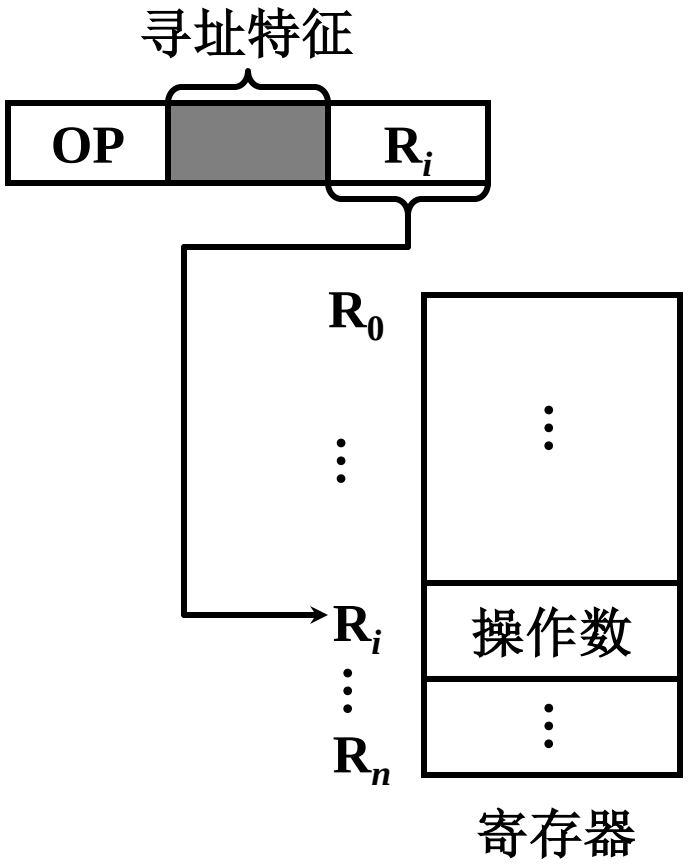
多次访存

间接寻址编程举例



5. 寄存器寻址

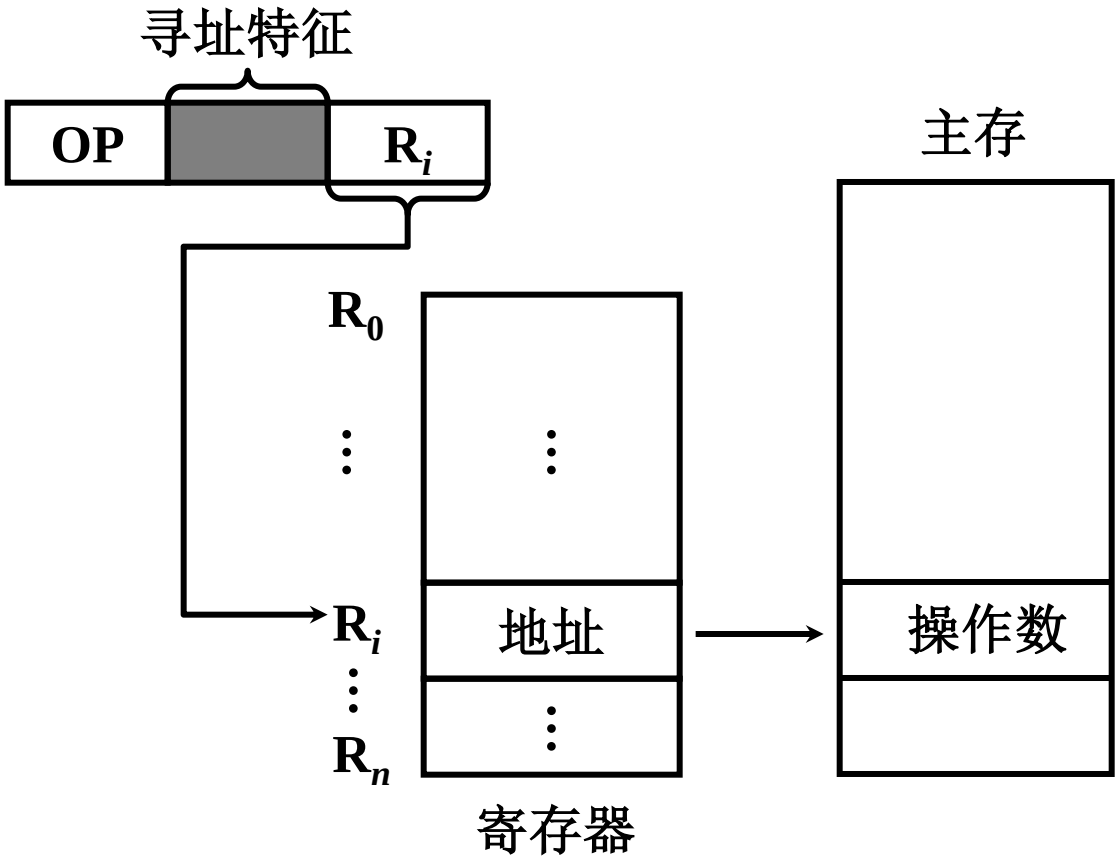
$EA = R_i$　　有效地址即为寄存器编号



- 执行阶段不访存，只访问寄存器，执行速度快
- 寄存器个数有限，可缩短指令字长

6. 寄存器间接寻址

$EA = (R_i)$　　有效地址在寄存器中

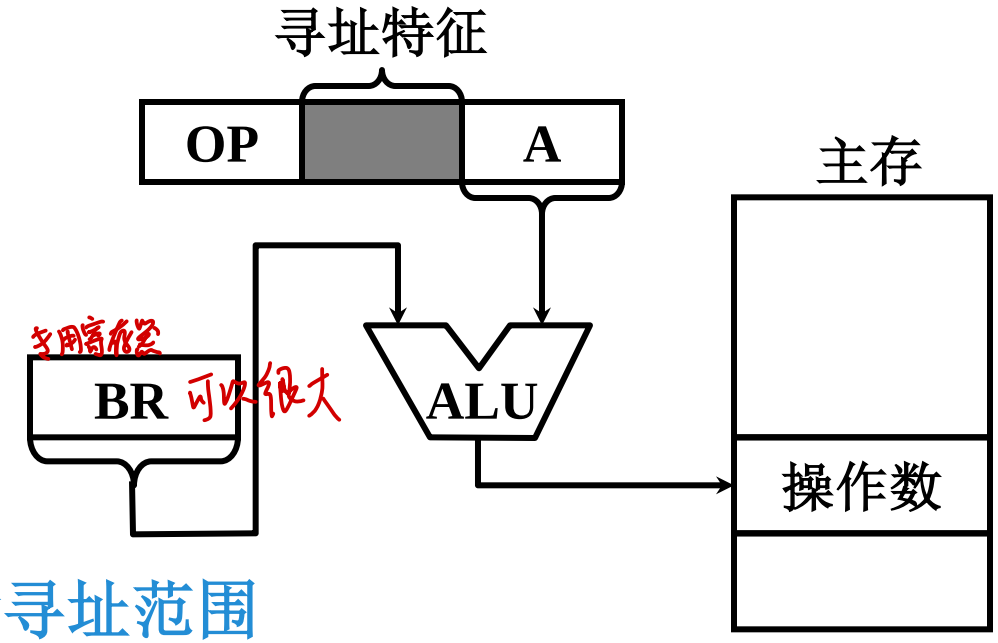


- 有效地址在寄存器中， 操作数在存储器中，执行阶段访存

7. 基址寻址

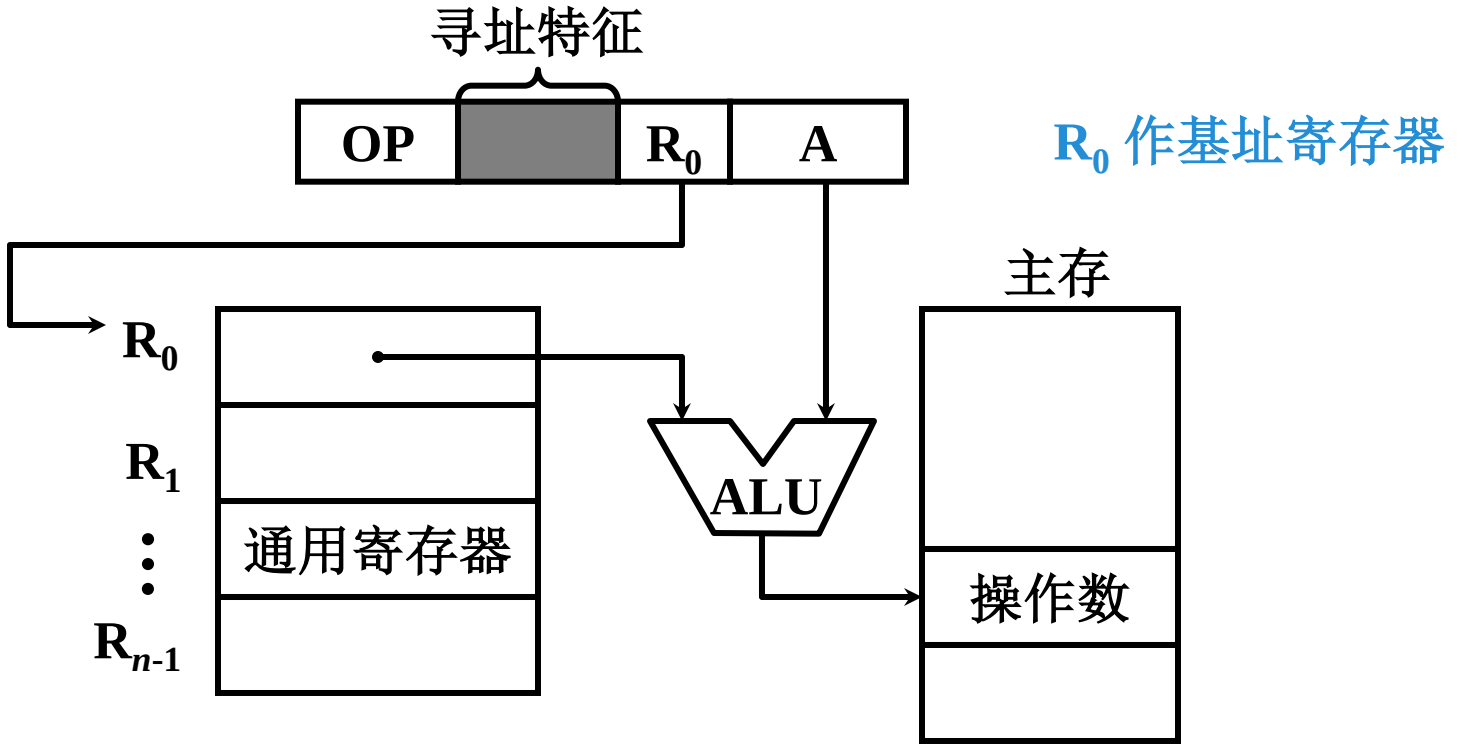
(1) 采用专用寄存器作基址寄存器

EA = (BR) + A　　BR 为基址寄存器



- 可扩大寻址范围
- 有利于多道程序
- BR 内容由操作系统或管理程序确定
- 在程序的执行过程中 BR 内容不变，形式地址 A 可变

(2) 采用通用寄存器作基址寄存器



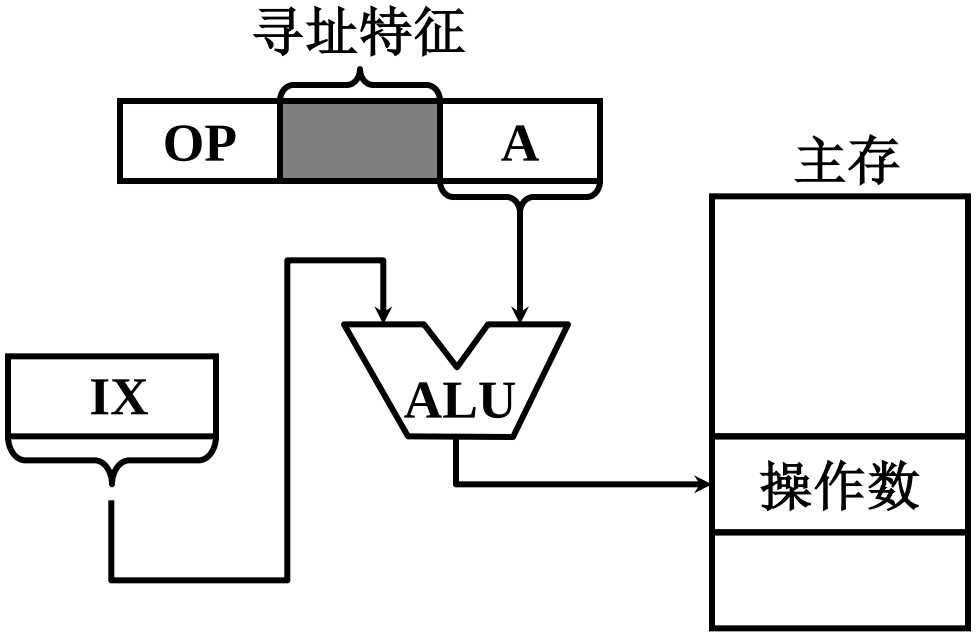
- 由用户指定哪个通用寄存器作为基址寄存器
- 基址寄存器的内容由操作系统确定
- 在程序的执行过程中 R_0 内容不变，形式地址 A 可变

8. 变址寻址

$EA = (IX) + A$

IX 为变址寄存器（专用）

通用寄存器也可以作为变址寄存器



- 可扩大寻址范围
- IX 的内容由用户给定
- 在程序的执行过程中 IX 内容可变，形式地址 A 不变
- 便于处理数组问题

例 设数据块首地址为 D，求 *N* 个数的平均值

直接寻址

```
LDA   D
ADD   D + 1
ADD   D + 2
  ⋮
ADD   D + ( N -1 )
DIV   # N
STA   ANS
```

共 *N* + 2 条指令

变址寻址

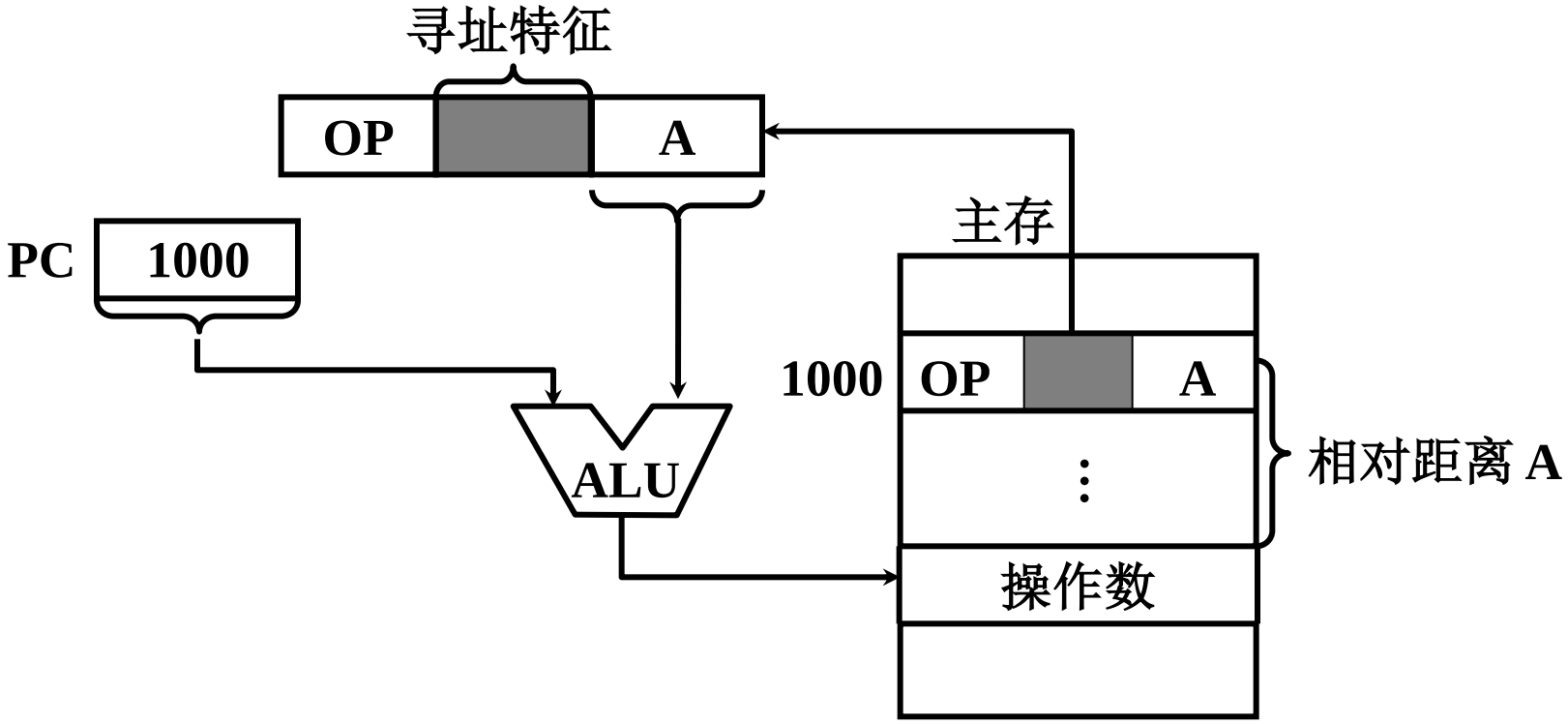
```
      LDA   # 0        0 ⟶ ACC
      LDX   # 0        X 为变址寄存器
M     ADD   X, D       D 为形式地址
      INX              (X) +1⟶ X
      CPX   # N       (X) 和 #N 比较
      BNE   M          结果不为零则转
      DIV   # N
      STA   ANS
```

共 8 条指令

9. 相对寻址

EA = (PC) + A

A 是相对于当前指令的位移量（可正可负，补码）



- A 的位数决定操作数的寻址范围
- 有利于编写浮动程序
- 广泛用于转移指令

(1) 相对寻址举例


```
        LDA    # 0
        LDX    # 0
M       ADD    X, D
M+1     INX
M+2     CPX    # N
M+3     BNE    M        ──→  * – 3
        DIV    # N
        STA    ANS
```

* 相对寻址特征

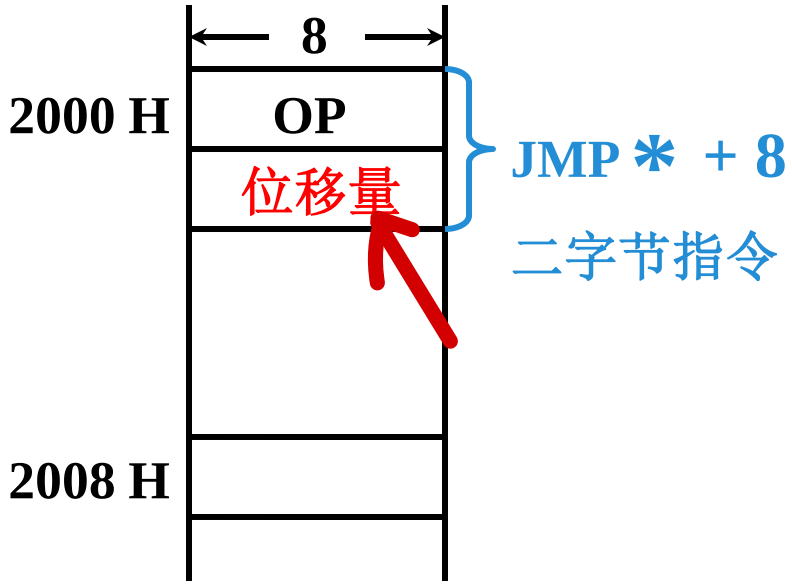
M 随程序所在存储空间的位置不同而不同

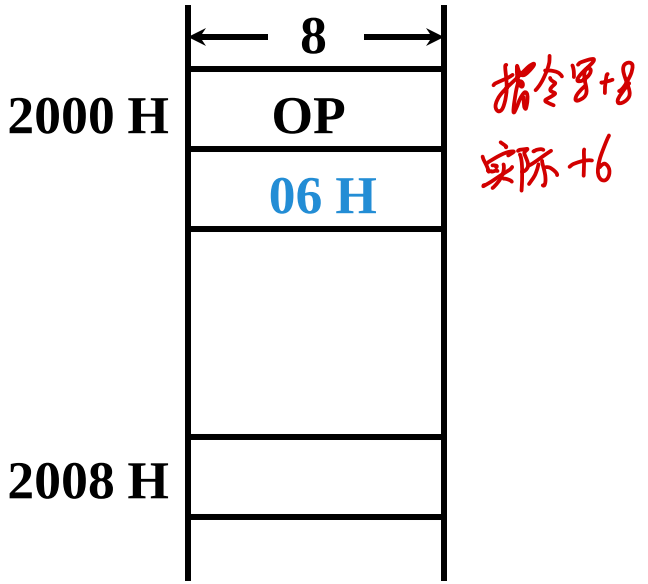
而指令 BNE *– 3 与 指令 ADD X, D 相对位移量不变

指令 BNE *– 3 操作数的有效地址为

EA = (M+3) – 3 = M

(2) 按字节寻址的相对寻址举例





设 当前指令地址 PC = 2000H

转移后的目的地址为 2008H

因为 取出 JMP * + 8 后 PC = 2002H

故 JMP * + 8 指令 的第二字节为 2008H - 2002H = 06H

+6

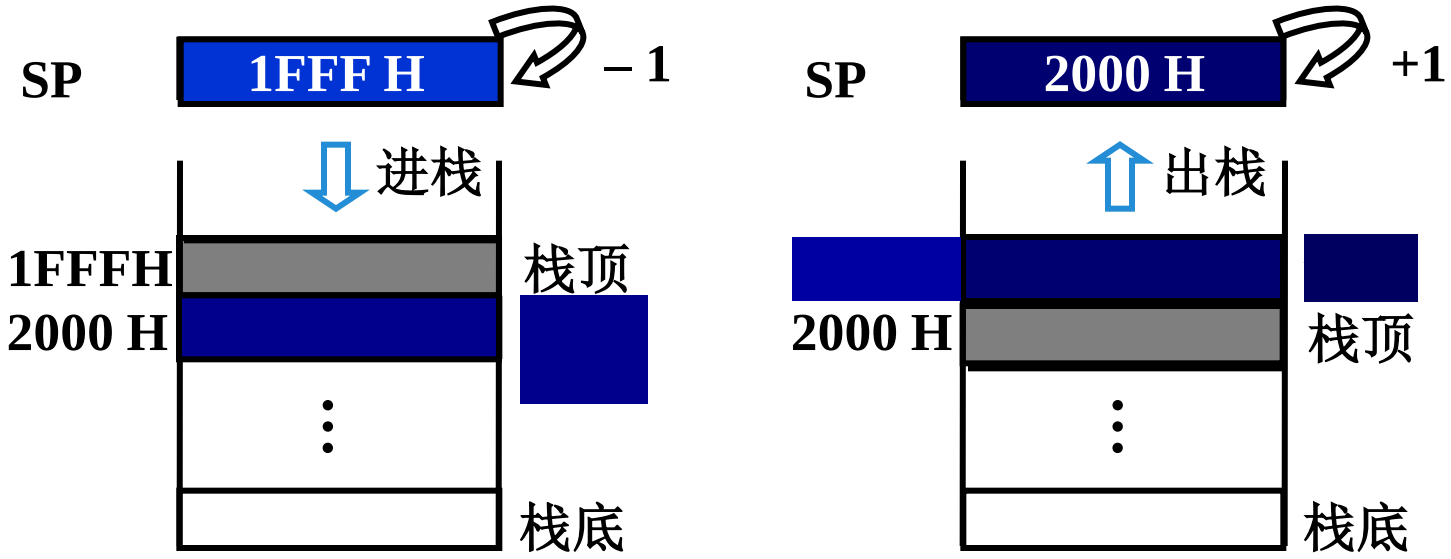
10. 堆栈寻址

(1) 堆栈的特点

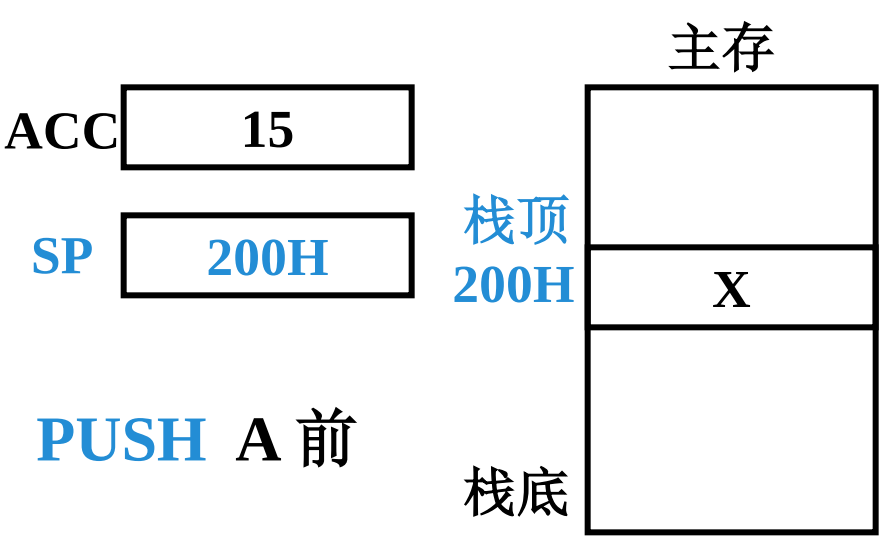
堆栈 { 硬堆栈 / 软堆栈 } —— 多个寄存器 / 指定的存储空间

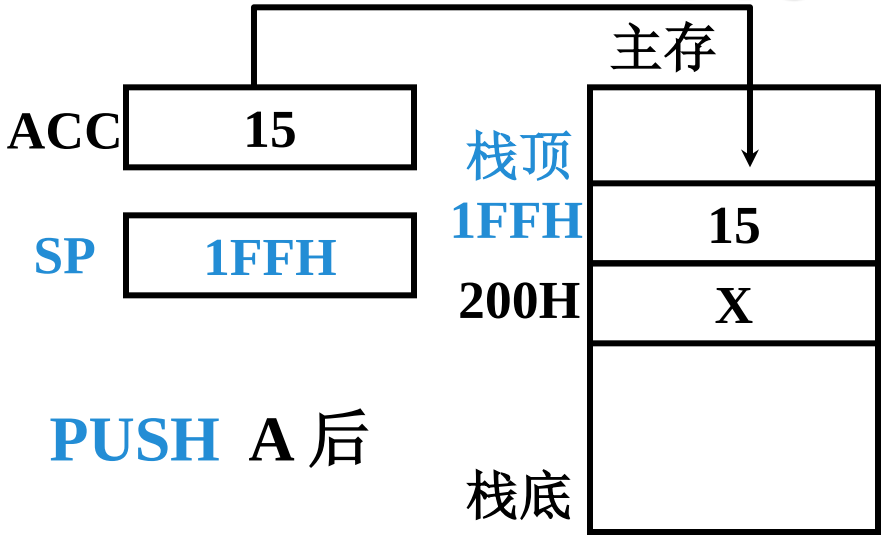
先进后出（一个入出口） 栈顶地址 由 SP 指出

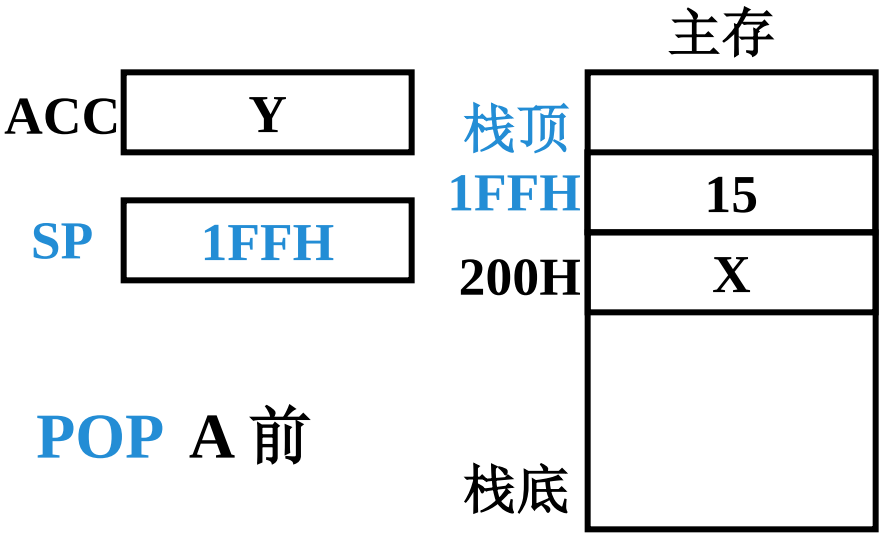
进栈 （SP）– 1 ⟶ SP 出栈 （SP）+ 1 ⟶ SP

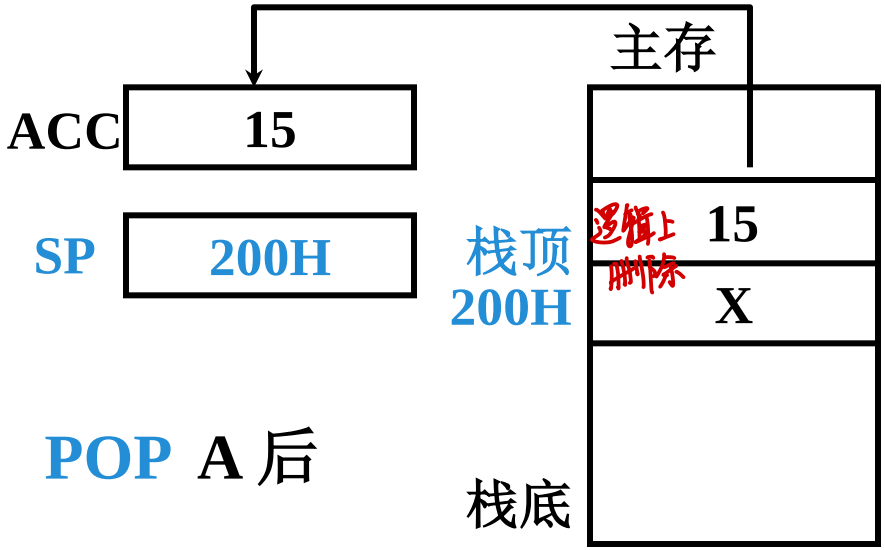


(2) 堆栈寻址举例


主存
ACC
15
SP
200H
栈顶
200H
X
PUSH A 前
栈底


主存
ACC
15
SP
1FFH
栈顶
1FFH
15
200H
X
PUSH A 后
栈底


主存
ACC
Y
SP
1FFH
栈顶
1FFH
15
200H
X
POP A 前
栈底


主存
ACC
15
SP
200H
栈顶
200H
15
X
POP A 后
栈底

(3) SP 的修改与主存编址方法有关

① 按 字 编址

进栈　（SP）– 1 ⟶ SP

出栈　（SP）+ 1 ⟶ SP

② 按 字节 编址

存储字长 16 位　进栈　（SP）– 2 ⟶ SP

出栈　（SP）+ 2 ⟶ SP

存储字长 32 位　进栈　（SP）– 4 ⟶ SP

出栈　（SP）+ 4 ⟶ SP

选择

假设（R）=1000，（1000）=2000，（2000）=3000，（3000）=5000，（PC）=4000,则以下寻址方式下访问到的指令操作数的值是多少

（1） 直接寻址　2000

（2） 间接寻址 1000

（3） 寄存器间接寻址　R

（4） 相对寻址 -1000

7.4 指令格式举例

一、设计指令格式时应考虑的各种因素

1. 指令系统的 兼容性（向上兼容）

旧机型运行新机型代码反之也可

向下兼容

2. 其他因素

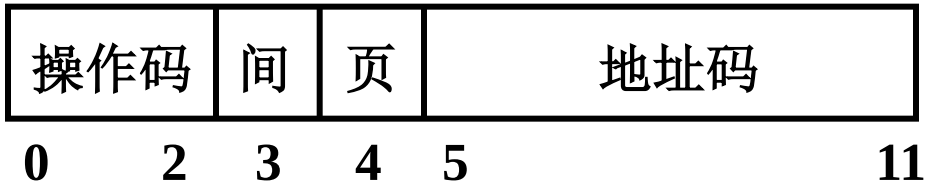
操作类型	包括指令个数及操作的难易程度
数据类型	确定哪些数据类型可参与操作
指令格式	指令字长是否固定
	操作码位数、是否采用扩展操作码技术，
	地址码位数、地址个数、寻址方式类型
寻址方式	指令寻址、操作数寻址
寄存器个数	寄存器的多少直接影响指令的执行时间

二、指令格式举例

1. PDP – 8

指令字长固定 12 位

访存类指令



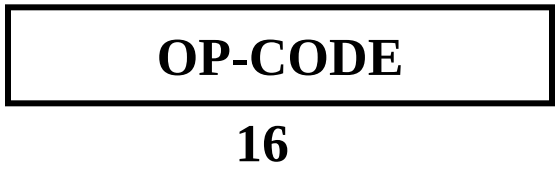
I/O 类指令

寄存器类指令

采用扩展操作码技术

2. PDP – 11

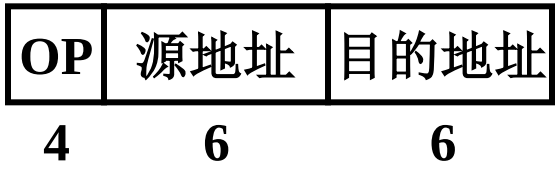
指令字长有 16 位、32 位、48 位三种



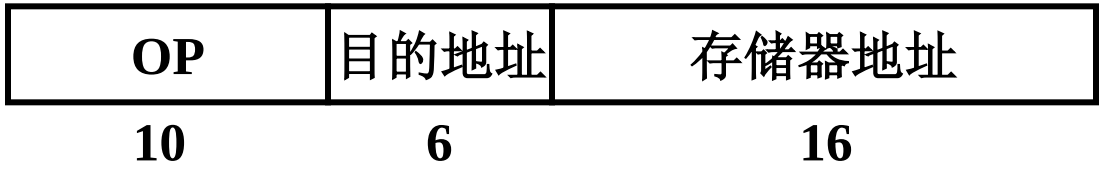
零地址 (16 位)

扩展操作码技术

一地址 (16 位)



二地址 R – R (16 位)

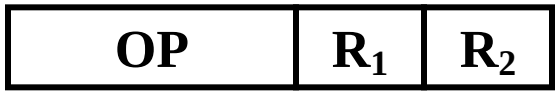


二地址 R – M (32 位)

二地址 M – M (48 位)

3. IBM 360

RR 格式



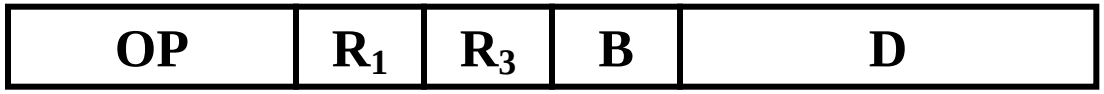
二地址 R – R

RX 格式

二地址 R – M

基址加变址寻址

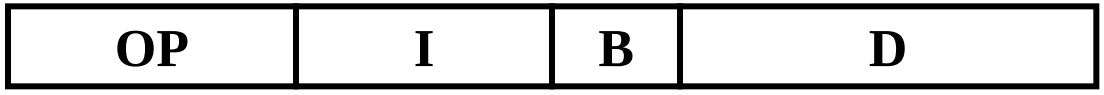
RS 格式



三地址 R – M

基址寻址

SI 格式



立即数 – M

基址寻址

SS 格式

二地址 M – M

基址寻址

4. Intel 8086

(1) 指令字长　1 ~ 6 个字节

INC　AX　1 字节

MOV　WORD PTR[0204], 0138H　6 字节

(2) 地址格式

零地址	NOP		1 字节	
一地址	CALL	段间调用	5 字节	
	CALL	段内调用	3 字节	
二地址	ADD AX，BX		2 字节	寄存器 – 寄存器
	ADD AX，3048H		3 字节	寄存器 – 立即数
	ADD AX，[3048H]		4 字节	寄存器 – 存储器

三、指令格式设计举例

根据需要设计指令格式 ——自学

- 扩展操作码技术

一、RISC 的产生和发展

RISC（Reduced Instruction Set Computer）

CISC（Complex Instruction Set Computer）

80 — 20 规律　　—— RISC技术

- **典型程序中 80% 的语句仅仅使用处理机中 20% 的指令**
- **执行频度高的简单指令，因复杂指令的存在，执行速度无法提高**

? **能否用 20% 的简单指令组合不常用的 80% 的指令功能**

二、RISC 的主要特征

- 选用使用频度较高的一些 简单指令，
  复杂指令的功能由简单指令来组合
- 指令 长度固定、指令格式种类少、寻址方式少
- 只有 LOAD / STORE 指令访存,其余指令的操作都在寄存器之间进行。
- CPU 中有多个 通用 寄存器
- 采用 流水技术，大部分指令在一个时钟周期内完成
- 采用 组合逻辑 实现控制器
- 采用 优化 的 编译 程序

二、RISC 的主要特征

- 1956年：一条指令就可以编出通用程序
- 一条传送指令CMOVE就可以做出一台计算机
- 1982年：8位的单指令计算机（SIC）出现

三、CISC 的主要特征

8086 有专用寄存器

- 系统指令 复杂庞大，各种指令使用频度相差大
- 指令 长度不固定、指令格式种类多、寻址方式多
- 访存 指令 不受限制
- CPU 中设有 专用寄存器
- 大多数指令需要 多个时钟周期 执行完毕
- 采用 微程序 控制器
- 难以 用 优化编译 生成高效的目的代码

四、RISC和CISC 的比较

1. RISC更能 充分利用 VLSI 芯片的面积

2. RISC 更能 提高计算机运算速度

   指令数、指令格式、寻址方式少，
   通用 寄存器多，采用 组合逻辑，
   便于实现 指令流水 提高运算速度

3. RISC 便于设计，可 降低成本，提高 可靠性

4. RISC 有利于编译程序代码优化

5. RISC 不易 实现 指令系统兼容

第7章 指令系统回顾

- **7.1 机器指令**
  - 指令的一般格式：扩展操作码技术
  - 指令字长：影响因素
- **7.2 操作数类型和操作类型**
  - 操作数类型：地址、数字、字符、逻辑数据
  - 数据在存储器中存放方式：边界对齐、大小端
  - 操作类型：数据传送、算数逻辑、移位、转移、I/O
- **7.3 寻址方式**

第7章 指令系统回顾

- **7.3 寻址方式**
  - 指令寻址：顺序和跳跃
  - 数据寻址：有效地址计算、特点
    - 立即数、直接、隐含、间接
    - 寄存器、寄存器间接、基址、变址、相对
- **7.4 指令格式举例**
  - 考虑因素、设计举例
- **7.5 RISC技术**
  - RISC产生背景和特征、CISC特征、对比

Thank You !

计 算 机 组 成 原 理

指令字长为16位，每个地址码为6位，采用扩展操作码的方式，设计14条二地址指令，100条一地址指令，100条零地址指令。

（1）画出扩展图。（2）给出指令译码逻辑。

（3）计算操作码平均长度。

解：（1）操作码扩展如下：

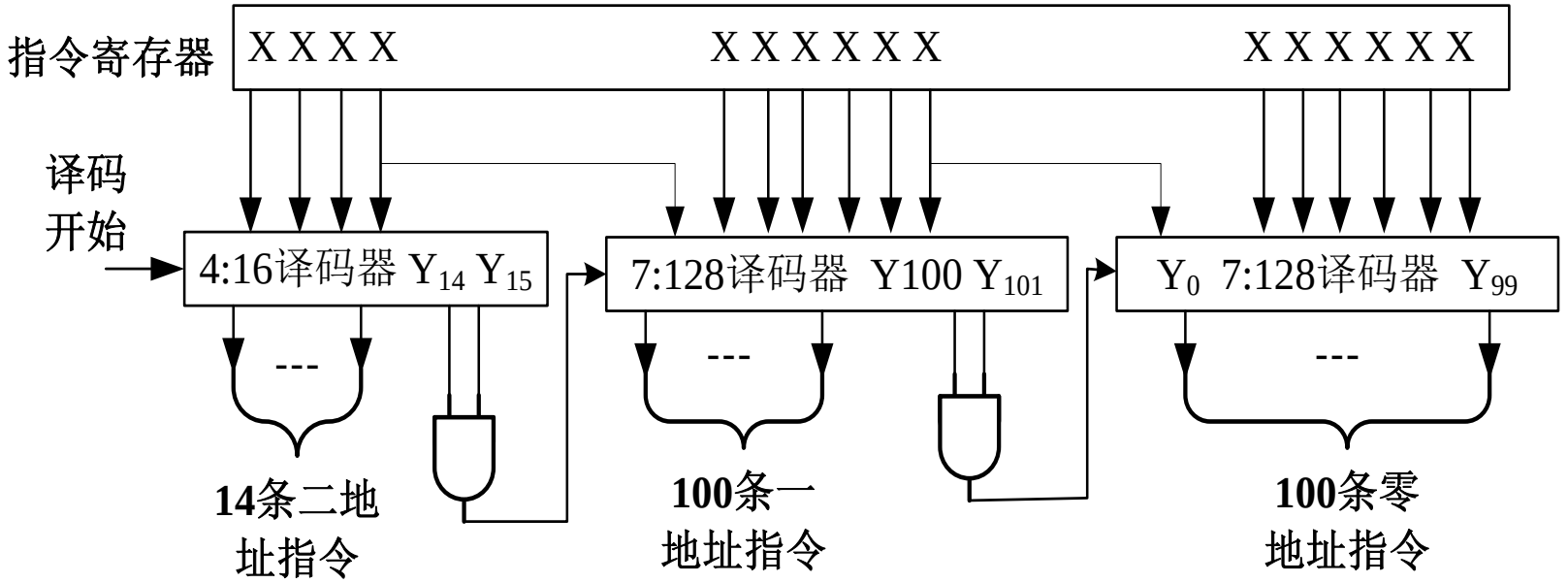
```
0000  xxxxxx  xxxxxx
 ┆     ┆      ┆        14条二地址指令
1101  xxxxxx  xxxxxx
1110  000000  xxxxxx
 ┆     ┆      ┆        100条二地址指令
1111  100011  xxxxxx
1111  100100  000000
 ┆     ┆      ┆        100条二地址指令
1111  100101  100011
```

指令字长为16位，每个地址码为6位，采用扩展操作码的方式，设计14条二地址指令，100条一地址指令，100条零地址指令。

（1）画出扩展图。（2）给出指令译码逻辑。

（3）计算操作码平均长度。

解：（2）指令译码逻辑：



指令字长为16位，每个地址码为6位，采用扩展操作码的方式，设计14条二地址指令，100条一地址指令，100条零地址指令。

（1）画出扩展图。（2）给出指令译码逻辑。

（3）计算操作码平均长度。

解：（3）操作码平均长度

=（4×14+10×100＋16×100）/214≈12.4

例：设相对寻址的转移指令占两个字节，第一字节是操作码，第二字节是相对位移量，用补码表示。每当CPU从存储器取出一个字节时，即自动完成(PC)+1→PC。

（1）设当前PC值为3000H，问转移后的目标地址范围是多少？

JMP 8× 6✓

解：（1）由于相对寻址的转移指令为两个字节，第一个字节为操作码，第二个字节为相对位移量，且用补码表示，故其范围为-128～+127，即80H～7FH。又因PC当前值为3000H,且CPU取出该指令后，PC已修改为3002H，因此最终的转移目标地址范围为3081H～2F82H，即3002H + 7FH =3081H至3002H-80H=2F82H

思考：若PC为16位，位移量可正可负，PC相对寻址范围为多大？

解：相对寻址中，PC提供基准地址，位移量提供修改量，位移量为16位可正可负，则相对寻址范围为：（PC）-2^{15}～（PC）$+2^{15}-1$

例：设相对寻址的转移指令占两个字节，第一字节是操作码，第二字节是相对位移量，用补码表示。每当CPU从存储器取出一个字节时，即自动完成(PC)+1→PC。

（2）若当前PC值为2000H,要求转移到01BH，则转移指令第二字节的内容是什么？

解：（2）若PC当前值为2000H，取出该指令后PC值为2002H，故转移指令第二字节应为

201BH－002H=19H。

例：设相对寻址的转移指令占两个字节，第一字节是操作码，第二字节是相对位移量，用补码表示。每当CPU从存储器取出一个字节时，即自动完成(PC)+1→PC。

（3）若当前PC值为2000H，指令JMP * -9（*为相对寻址特征）的第二字节的内容是什么？

解：根据汇编语言指令JMP*－9，即要求转移后的目标地址为2000H－09H=1FF7H，但因为CPU取出该指令后PC值已修改为2002H，故转移指令的第二字节的内容应为-11（十进制），写成补码为F5H。

Computer Organization

计算机组成原理

第八章 CPU 的结构和功能

系统结构研究所

大　　纲

（一） CPU的功能和基本结构

（二） 指令执行过程

（三） 数据通路的功能和基本结构

（四） 控制器的功能和工作原理

1. 硬布线控制器

2. 微程序控制器

微程序、微指令和微命令；微指令格式；微命令的编码方式；微地址的形成方式

（五） 指令流水线

1. 指令流水线的基本概念

2 .指令流水线的基本实现

3. 超标量和动态流水线的基本概念

Contents

8.1 CPU 的结构

8.2 指令周期

8.3 指令流水

8.4 中断系统

8.1 CPU 的结构

一、 CPU 的功能

1. 控制器的功能

取指令

分析指令

执行指令，发出各种操作命令

控制程序输入及结果的输出

总线管理

处理异常情况和特殊请求

指令控制

操作控制

时间控制

处理中断

数据加工

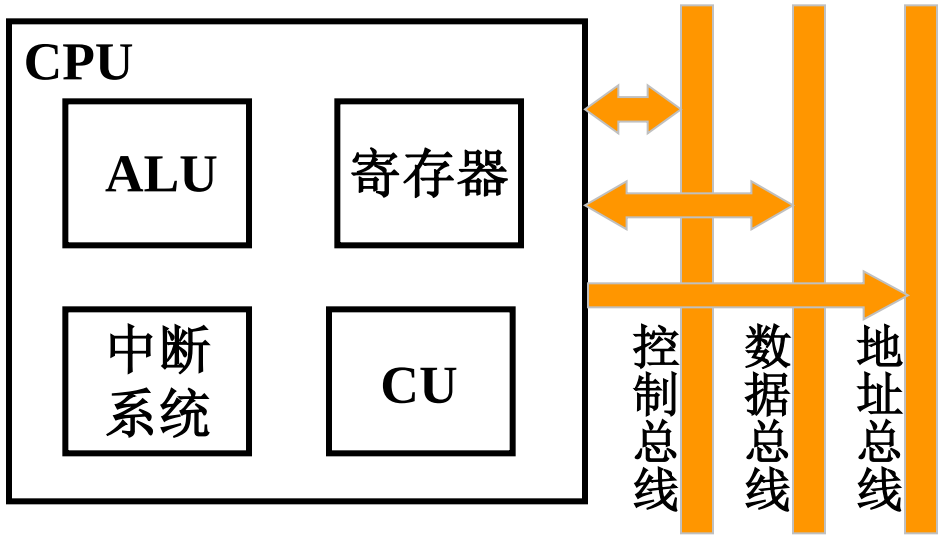
2. 运算器的功能

实现算术运算和逻辑运算

二、CPU 结构框图

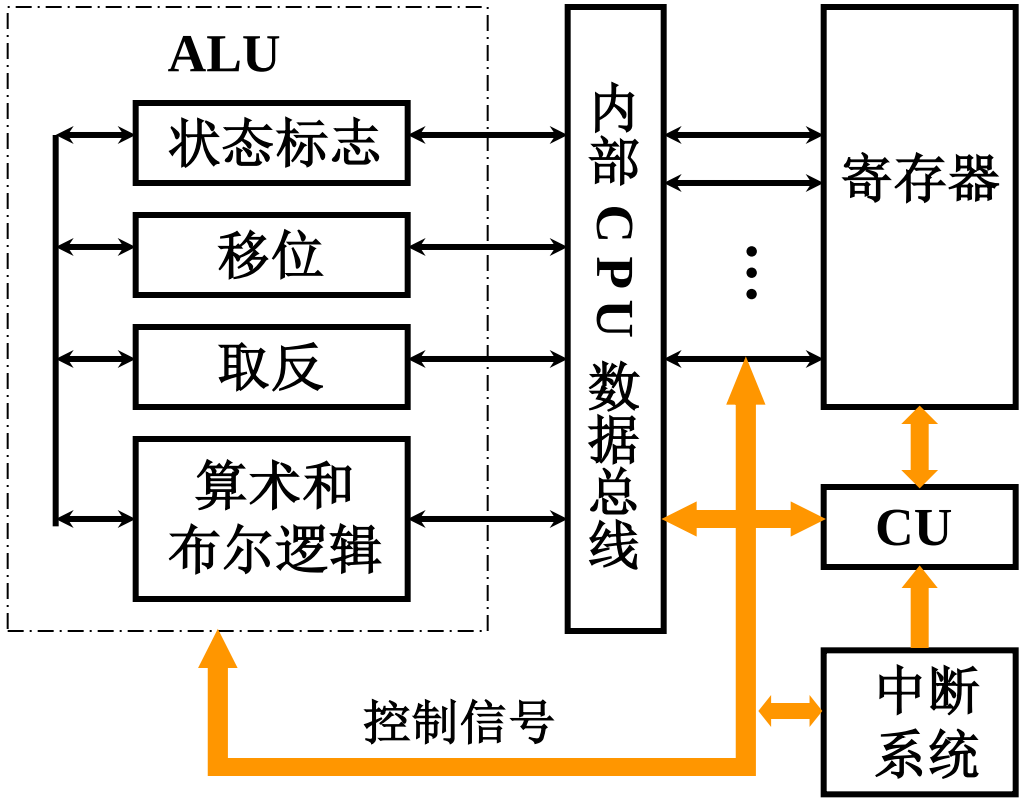
1. CPU 与系统总线

指令控制	PC　IR
操作控制 时间控制	CU　时序电路
数据加工	ALU　寄存器
处理中断	中断系统



2. CPU 的内部结构

8.1



三、 CPU 的寄存器

8.1

1. 用户可见寄存器

(1) 通用寄存器　　存放操作数

可作 某种寻址方式所需的 专用寄存器

(2) 数据寄存器　　存放操作数（满足各种数据类型）

两个寄存器拼接存放双倍字长数据

MQ,X

(3) 地址寄存器　　存放地址，其位数应满足最大的地址范围

用于特殊的寻址方式　段指针　栈指针

(4) 条件码寄存器　存放条件码，可作程序分支的依据

如 正、负、零、溢出、进位等

2. 控制和状态寄存器

(1) 控制寄存器

PC → MAR → M → MDR → IR

控制 CPU 操作

其中 MAR、MDR、IR　　用户不可见

PC　　用户可见

(2) 状态寄存器

状态寄存器　　存放条件码和其它状态信息

PSW 寄存器　　存放程序状态字

3. 举例　　Z8000　　8086　　MC 68000

四、 控制单元 CU 和中断系统 8.1

1. CU 产生全部指令的微操作命令序列

组合逻辑设计 硬件实现 硬连线逻辑

微程序设计 软件方式 存储逻辑

参见 第 4 篇

2. 中断系统

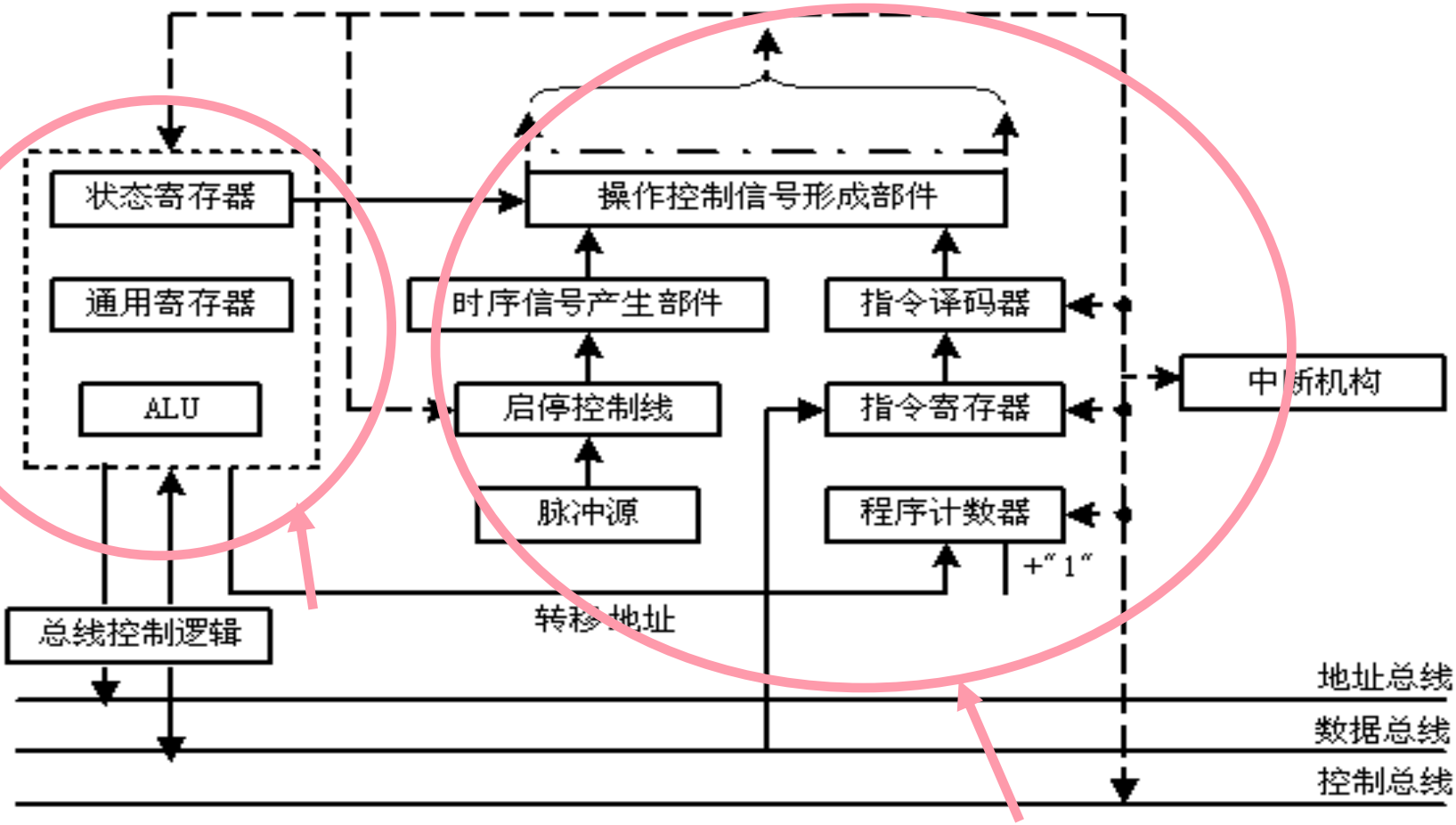
参见 8.4 节

五、ALU

参见 第 6 章

补充： CPU基本组成原理图

指令寄存器----IR
程序计数器---PC



执行部件

CPU 由 执行部件 和 控制部件组成

CPU 包含 数据通路 和 控制器

控制部件

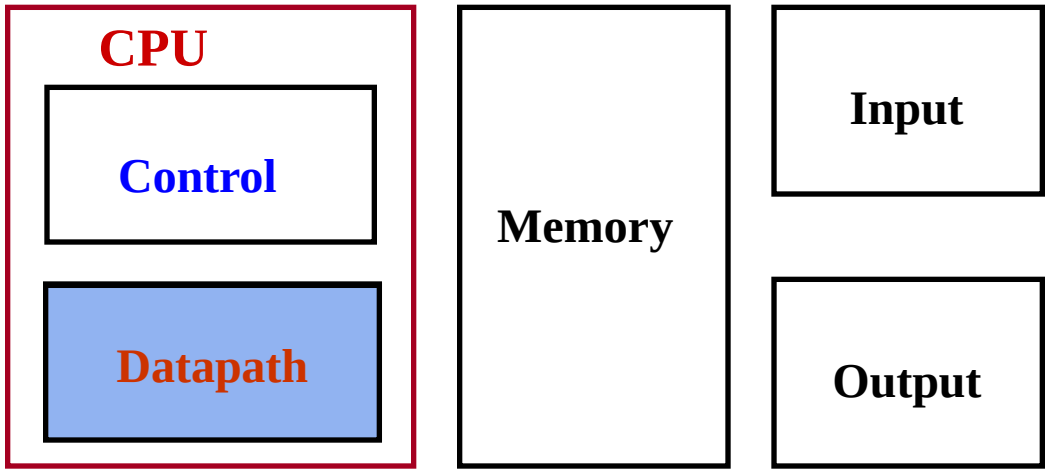
控制器 由 指令译码器 和 控制信号形成部件 组成

- PC、IR
- 指令译码器ID：对操作进行译码，向控制器提供操作的特定信号。
- 时序发生器：产生各种时序信号
  - 脉冲源：通常由石英晶体振荡器构成。产生一定频率的脉冲信号作为整个机器的时钟脉冲，是机器周期和工作脉冲的基准信号。在机器刚加电时，产生总清信号(reset)
  - 启停线路：保证可靠地送出或封锁完整的时钟脉冲，控制时序信号的发生或停止，从而启动机器工作或停机。

- 时序控制信号形成部件：当机器启动后，在CLK时钟作用下，根据当前正在执行的指令的需要，产生相应的时序控制信号，并根据被控功能部件的反馈信号调整时序控制信号。
- 状态寄存器（PSR）：存放PSW,PSW表明了系统基本状态，是控制程序执行的重要依据。通过状态判是否准备好

数据通路

- 计算机的五大组成部分：



- 什么是数据通路（DataPath）？
  - 指令执行过程中，数据所经过的路径，包括路径中的部件。它是指令的执行部件。
- 控制器（Control）的功能是什么？
  - 对指令进行译码，生成指令对应的控制信号，控制数据通路的动作。能对执行部件发出控制信号，是指令的控制部件。

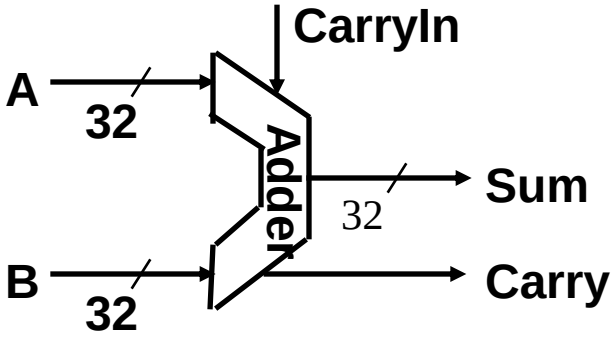
数据通路的基本结构

- 数据通路由两类部件组成
  - 组合逻辑元件（也称操作元件）
  - 存储元件（也称状态元件）
- 元件间的连接方式
  - 总线连接方式
  - 分散连接方式
- 数据通路如何构成？
  - 由“操作元件”和“存储元件”通过总线方式或分散方式连接而成
- 数据通路的功能是什么？
  - 进行数据存储、处理、传送

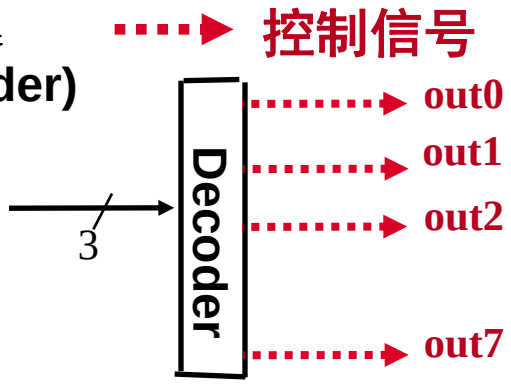
因此，数据通路是由操作元件和存储元件通过总线方式或分散方式连接而成的进行数据存储、处理、传送的路径。

操作元件：组合逻辑电路

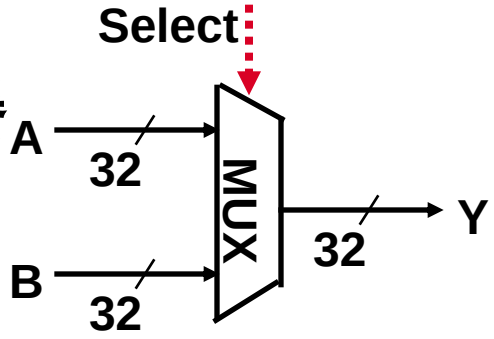
❖ 加法器 (Adder)



◦ 译码器 (Decoder)

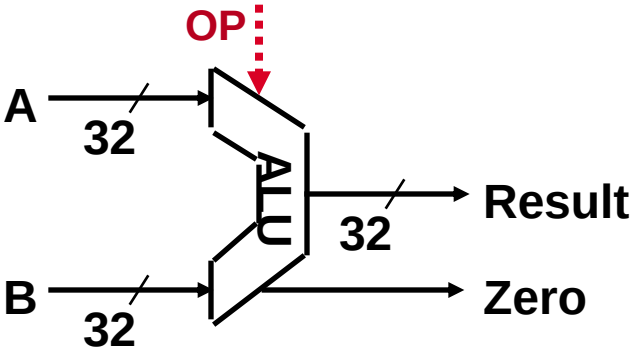


多路选择器 (MUX)



何时要用到adder, ALU, MUX or Decoder?

算逻部件 (ALU)



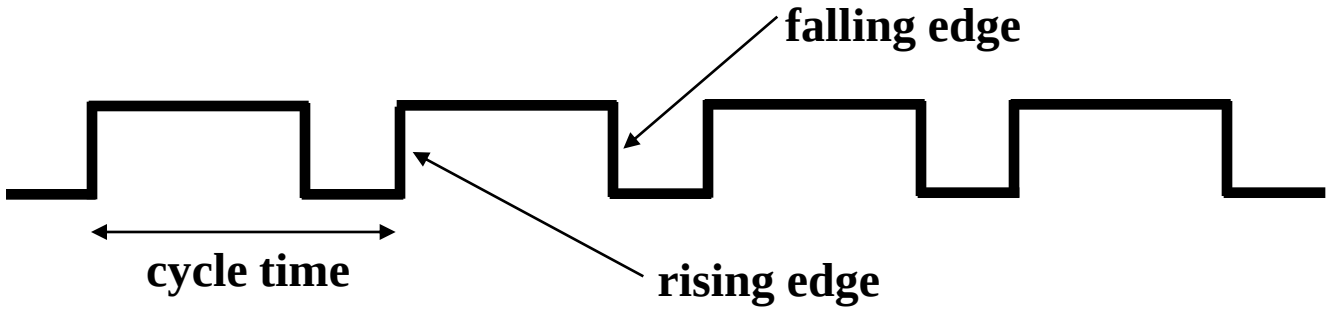
组合逻辑元件的特点：

其输出只取决于当前的输入。即：输入一样，其输出也一样

定时：所有输入到达后，经过一定的逻辑门延时，输出端改变，并保持到下次改变，不需要时钟信号来定时

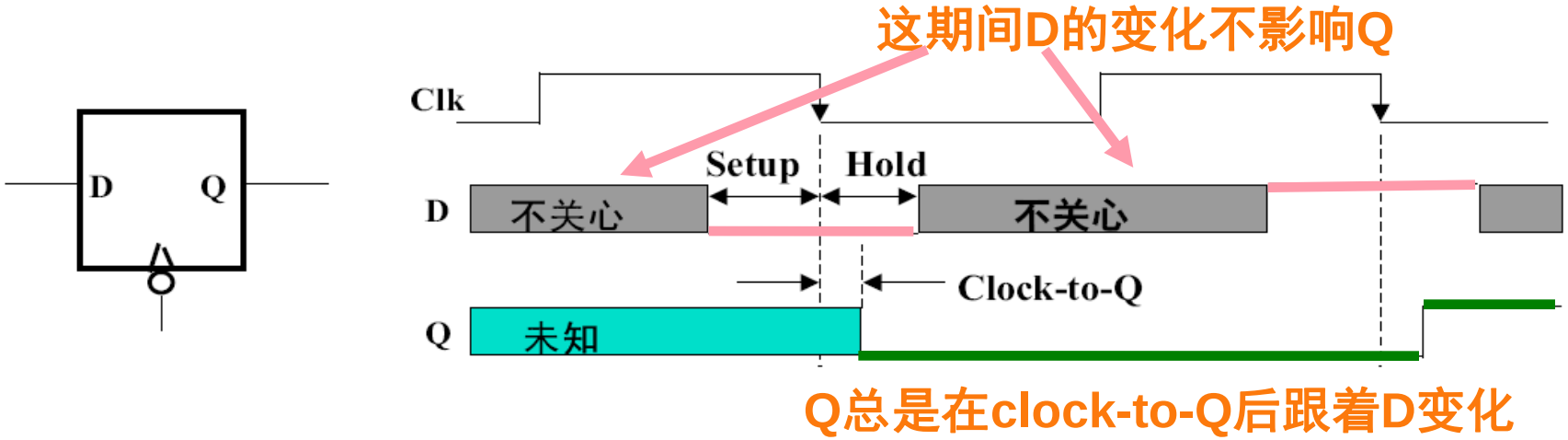
状态元件：时序逻辑电路

- 状态（存储）元件的特点：
  - 具有存储功能，在时钟控制下输入被写到电路中
  - 输入端状态由时钟决定何时被写入，输出端状态随时可以读出
- 定时方式：规定信号何时写入状态元件或何时从状态元件读出
  - 边沿触发（edge-triggered）方式：
    - 状态单元中的值只在时钟边沿改变。每个时钟周期改变一次。
      - 上升沿（rising edge） 触发：在时钟正跳变时进行读/写。
      - 下降沿（falling edge）触发：在时钟负跳变时进行读/写。



- 最简单的状态单元（回顾：数字逻辑电路课程内容）：
  - D触发器：一个时钟输入、一个状态输入、一个状态输出

存储元件中何时状态被改变？



- 建立时间（**Setup Time**）：在触发时钟边沿 之前 输入必须稳定
- 保持时间（**Hold Time**）：在触发时钟边沿 之后 输入必须保持
- **Clock-to-Q time: (Latch Prop - 锁存延迟)**
  - 在触发时钟边沿，输出并不能立即变化

切记：状态单元的输入信息总是在一个时钟边沿到达后的“Clk-to-Q”时才被写入到单元中，此时的输出才反映新的状态值

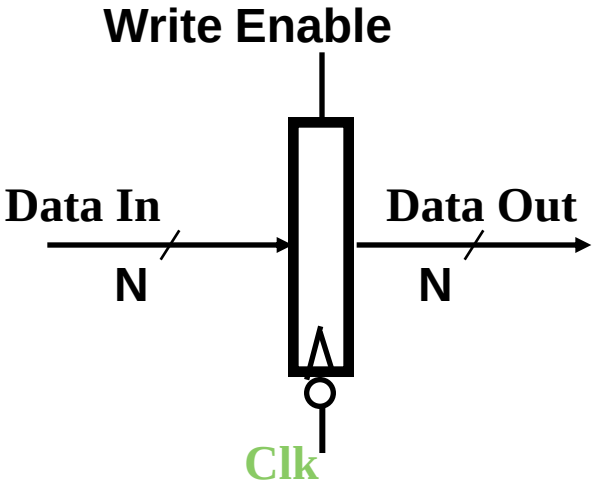
数据通路中的状态元件有两种：寄存器(组) + 存储器

存储元件: 寄存器和寄存器组

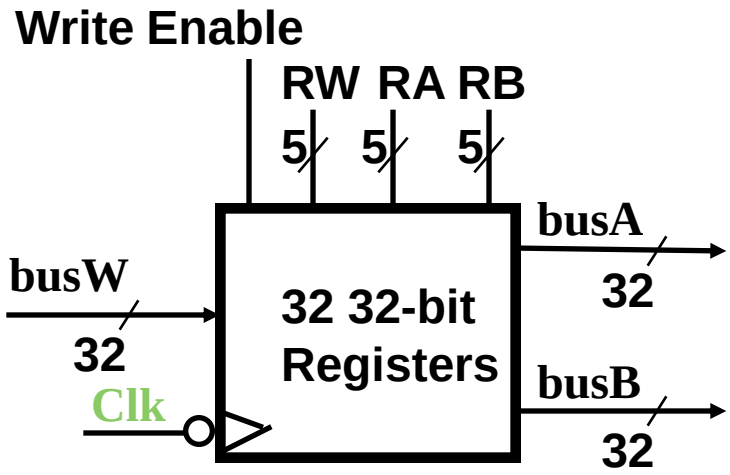
- **寄存器（Register）**
  - 有一个写使能（Write Enable-WE）信号

    0: 时钟边沿到来时，输出不变

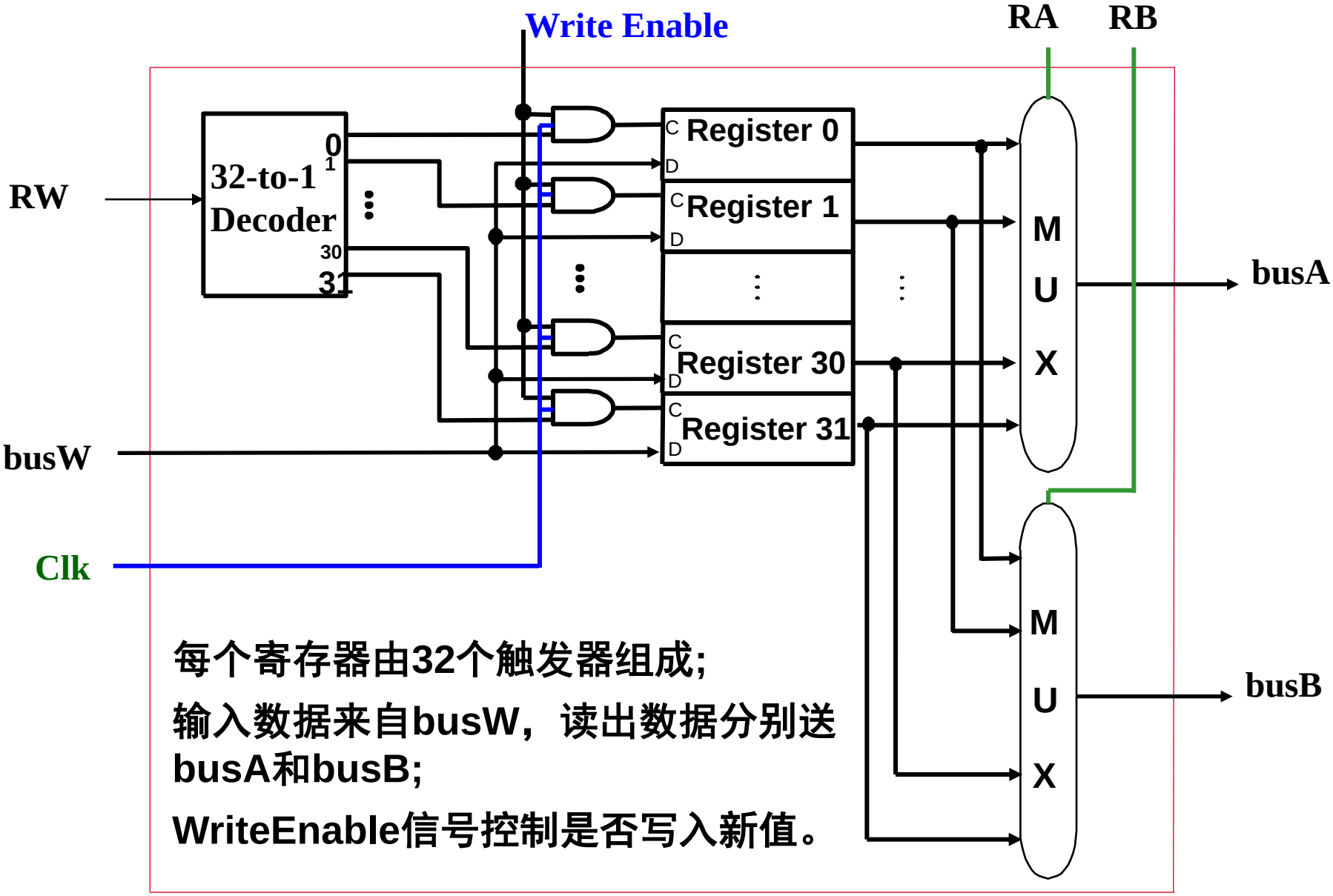
    1: 时钟边沿到来时，输出开始变为输入
  - 若每个时钟边沿都写入，则不需WE信号



- **寄存器组（Register File）**
  - 两个读口（组合逻辑操作）：busA和busB分别由RA和RB给出地址。地址RA或RB有效后，经一个“取数时间(AccessTime)”，busA和busB有效。
  - 一个写口（时序逻辑操作）：写使能为1的情况下，时钟边沿到来时，busW传来的值开始被写入RW指定的寄存器中。



寄存器组的内部结构

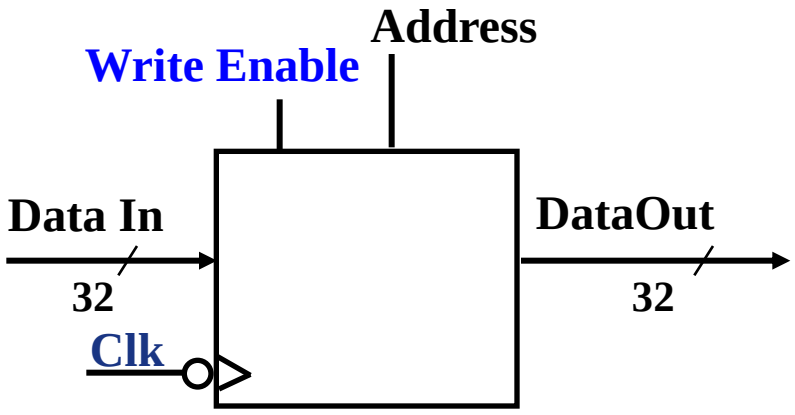


每个寄存器由32个触发器组成;

输入数据来自busW，读出数据分别送busA和busB;

WriteEnable信号控制是否写入新值。

存储元件: 理想存储器



- 理想存储器（ idealized memory ）
  - Data Out：32位读出数据
  - Data In： 32位写入数据
  - Address：读写公用一个32位地址
  - 读操作（组合逻辑操作）：地址Address有效后，经一个“取数时间AccessTime”，Data Out上数据有效。
  - 写操作（时序逻辑操作）：写使能为1的情况下，时钟Clk边沿到来时，Data In传来的值开始被写入Address指定的存储单元中。

为简化数据通路操作说明，把存储器简化为带时钟信号Clk的理想模型。

数据通路与时序控制

- 同步系统(Synchronous system)
  - 所有动作有专门时序信号来定时
  - 由时序信号规定何时发出什么动作

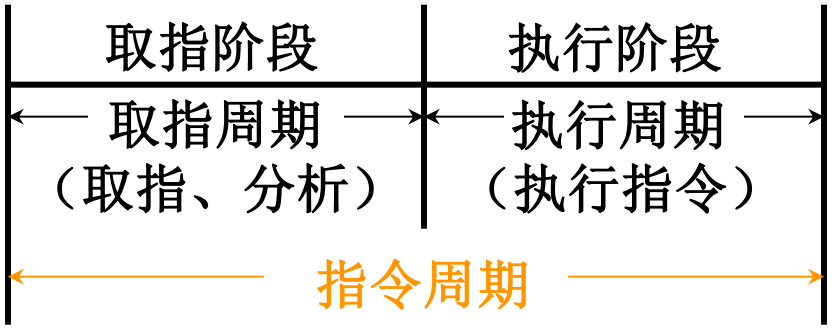
  例如，指令执行过程每一步都有控制信号控制，由定时信号确定控制信号何时发出、作用时间多长
- 什么是时序信号？
  - 同步系统用于同步控制的定时信号，如时钟信号
- 什么叫指令周期？
  - 取并执行一条指令的时间
- 早期计算机的三级时序系统
  - 机器周期 - 节拍 - 脉冲
  - 指令周期可分为取指令、读操作数、执行并写结果等多个基本工作周期，称为机器周期。
  - 机器周期有取指令、存储器读、存储器写、中断响应等不同类型

一、 指令周期的基本概念

1．指令周期

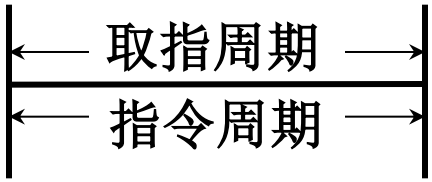
取出并执行一条指令所需的全部时间

完成一条指令 { 取指、分析 —— 取指周期
执行 —— 执行周期

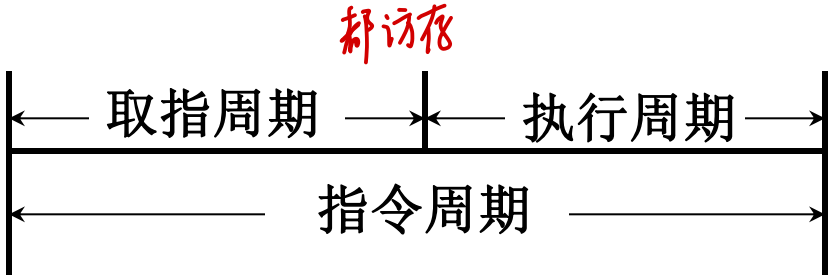


各个计算机划分不同

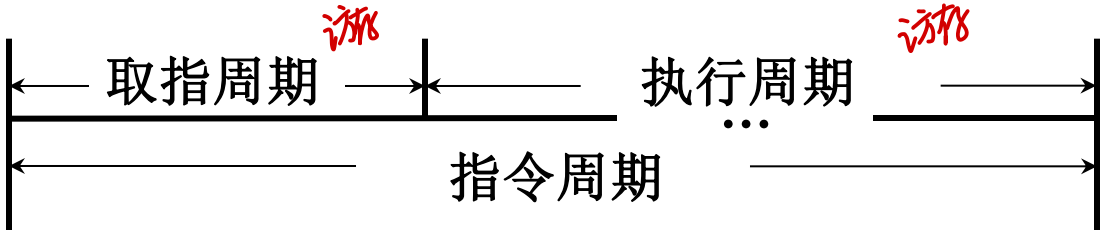
2. 每条指令的指令周期不同



NOP



ADD mem

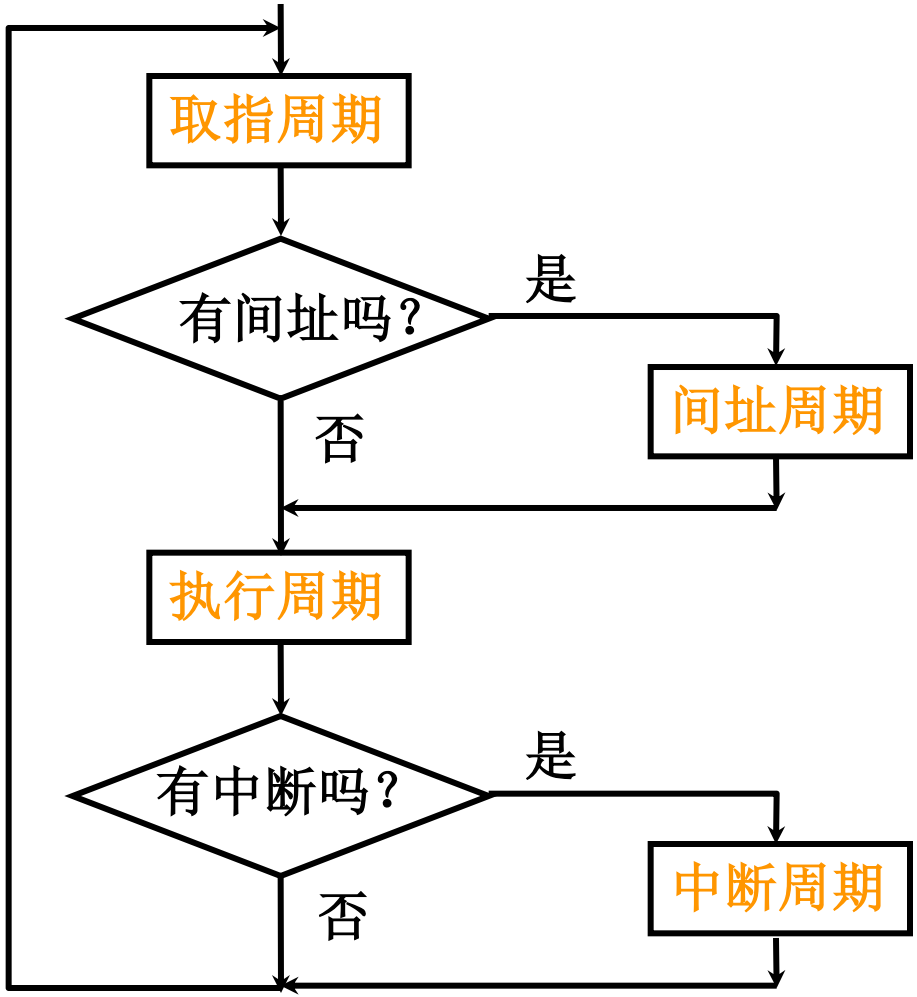


MUL mem

3. 具有间接寻址的指令周期

4. 带有中断周期的指令周期

5. 指令周期流程



6. CPU 工作周期的标志

8.2

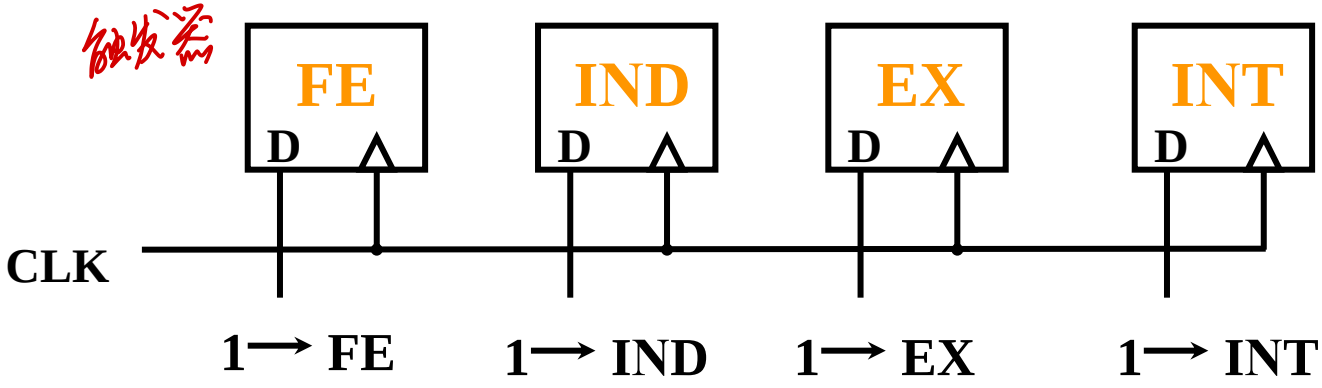
CPU 访存有四种性质

看四个触发器输出

看处于哪个周期 → 访存类型

取	指令	取指周期
取	地址	间址周期
存/取	操作数或结果	执行周期
存	程序断点	中断周期

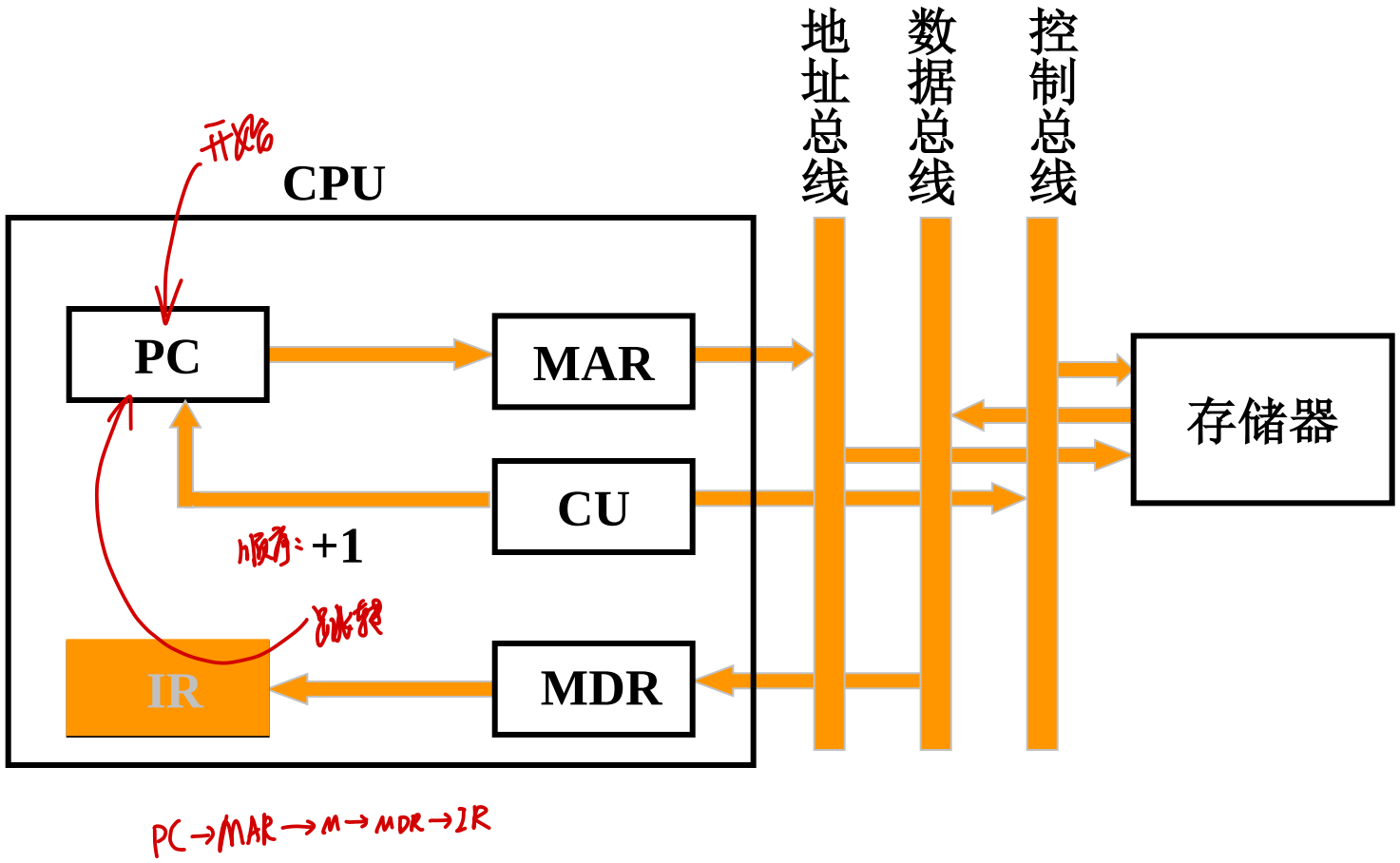
CPU 的
4个工作周期



二、 指令周期的数据流

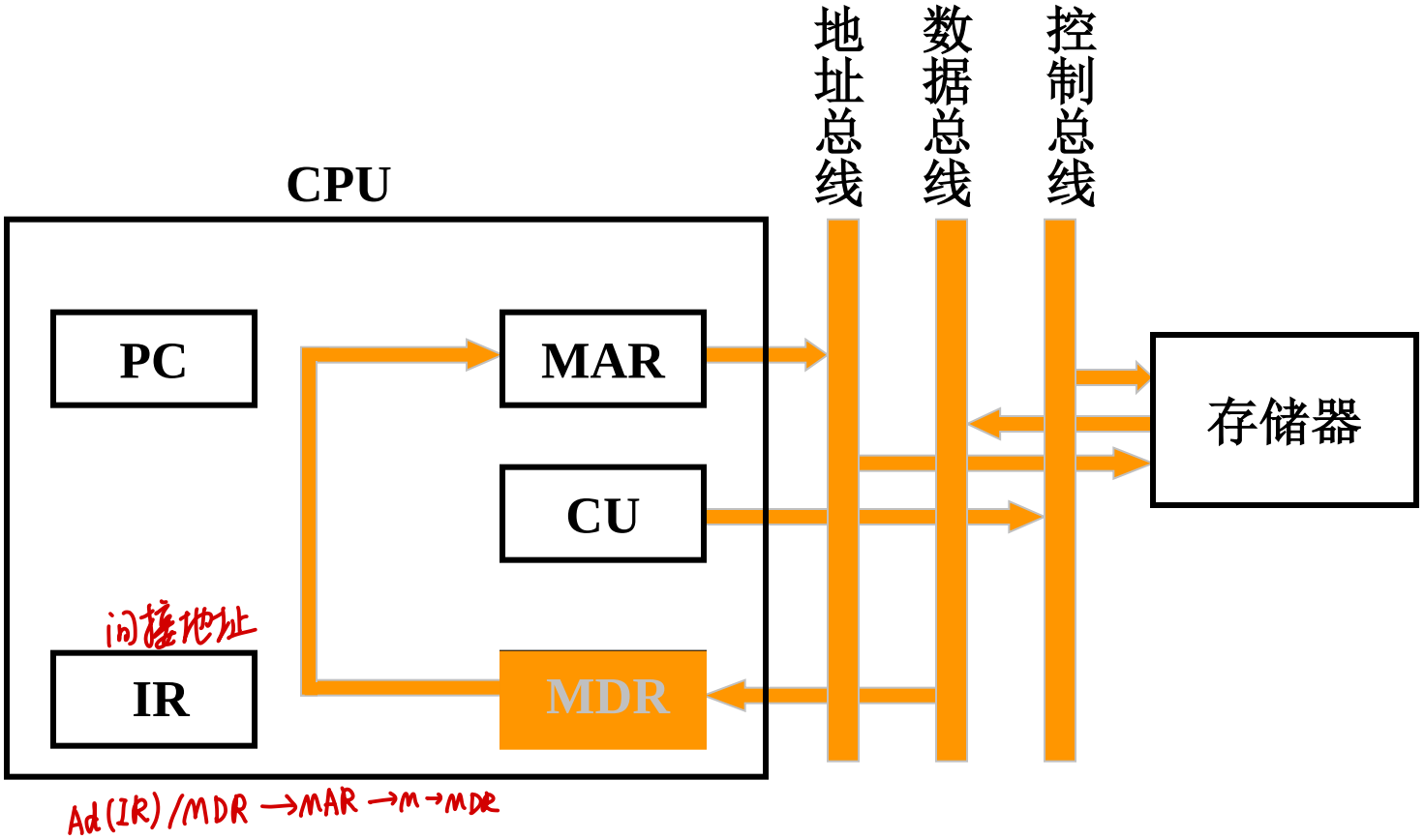
8.2

1. 取指周期数据流



2. 间址周期数据流

8.2



3. 执行周期数据流

不同指令的执行周期数据流不同

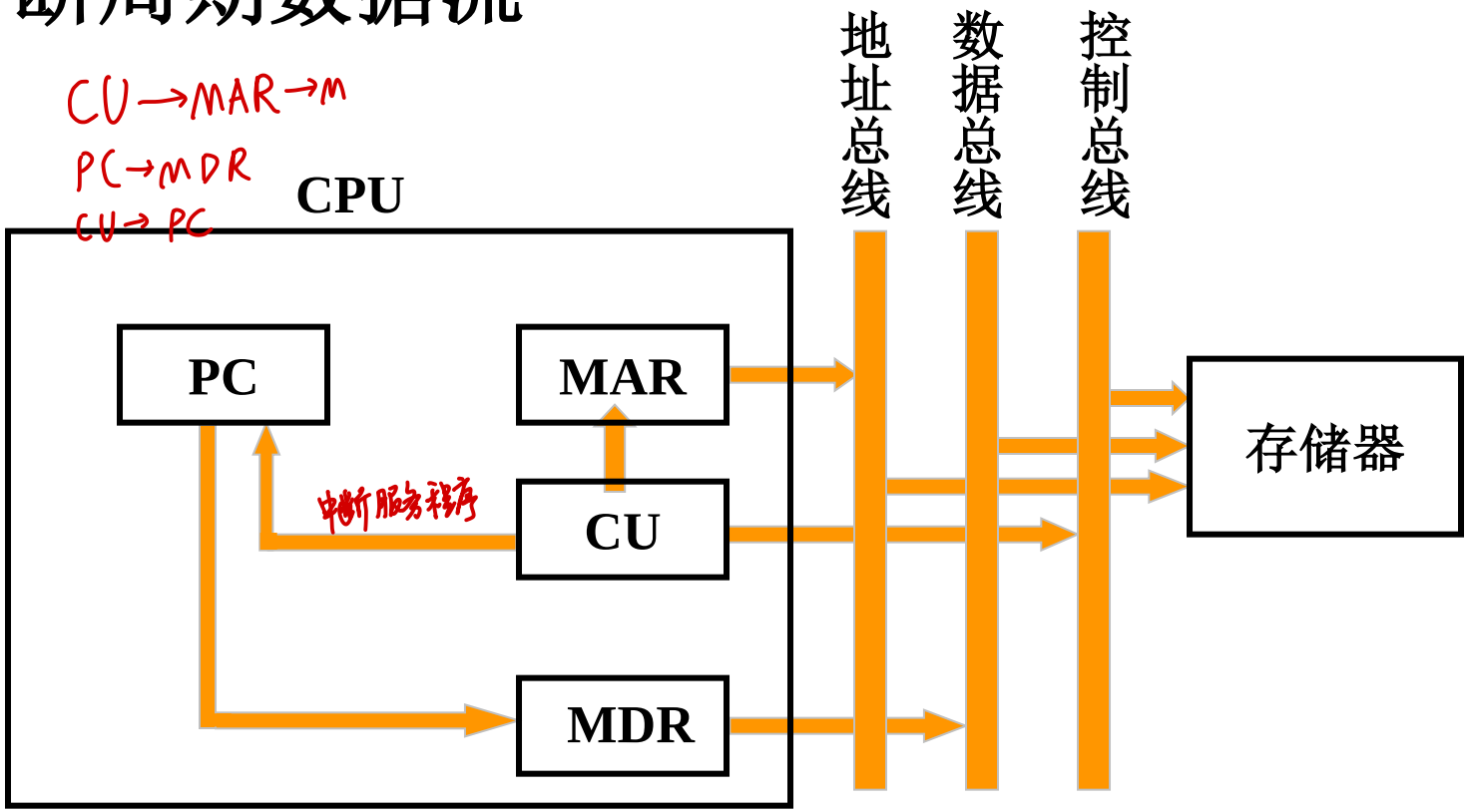
前两者相对固定

保存程序断点
转中断程序入口地址
关中断 EINT=0

中断隐指令

4. 中断周期数据流

CU→MAR→M
PC→MDR
CU→PC



8.3 指令流水

一、如何提高机器速度

1. 提高访存速度

   高速芯片　　Cache　　多体并行

2. 提高 I/O 和主机之间的传送速度

   中断　　DMA　　通道　　I/O 处理机　　多总线

3. 提高运算速度

   高速芯片　　改进算法　　快速进位链

- 进一步提高整机处理能力

  高速器件　　改进系统结构，开发系统的并行性

二、系统的并行性

8.3

1. 并行的概念

并行 {
- 并发：两个或两个以上事件在同一时间段发生
- 同时：两个或两个以上事件在同一时刻发生

时间上互相重叠

2. 并行性的等级

过程级（程序、进程）	粗粒度	软件实现
指令级（指令之间） （指令内部）	细粒度	硬件实现 ← 流水

三、指令流水原理

1. 指令的串行执行

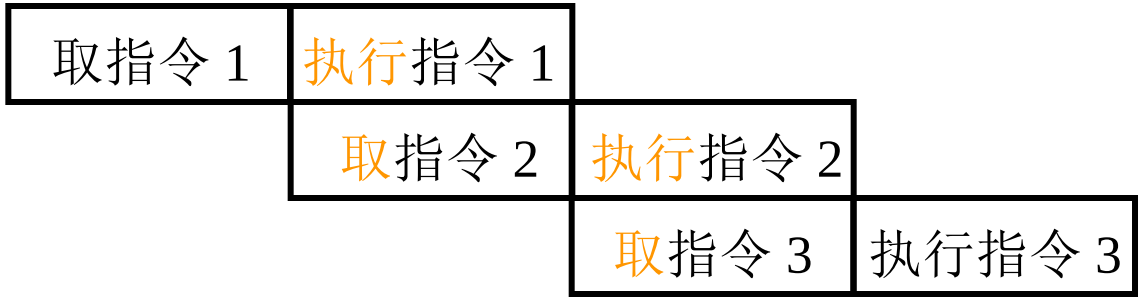
取指令 1	执行指令 1	取指令 2	执行指令 2	取指令 3	执行指令 3	…

6T

取指令　　取指令部件　　完成　　总有一个部件 空闲

执行指令　　执行指令部件　　完成

2. 指令的二级流水



4T

指令预取

若 取指 和 执行 阶段时间上 完全重叠

速度提高 1 倍

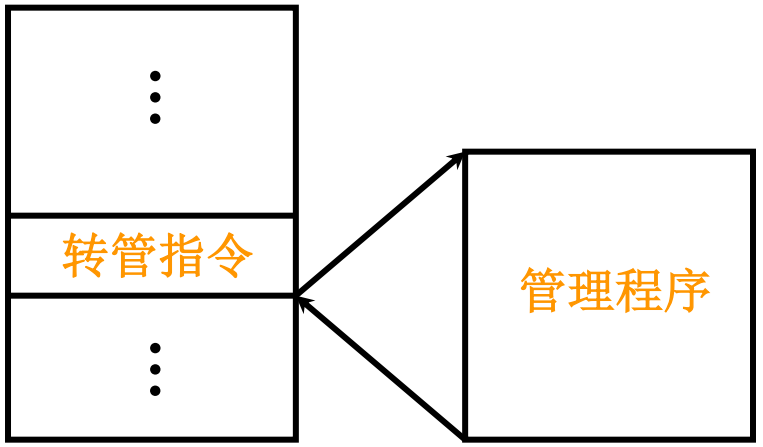
8.4 中断系统

一、概述

1. 引起中断的各种因素

(1) 人为设置的中断

如　转管指令



(2) 程序性事故　溢出、操作码不能识别、除法非法

(3)　硬件故障

(4)　I/O 设备

(5)　外部事件　　用 键盘中断 现行程序

中断的分类

1. 按中断来源分：

（1）内中断：发生在主机内部的中断称为内中断

（2）外中断：由主机外部事件引起的中断称为外中断

2. 按中断服务程序入口地址的获取方式分

（1）向量中断：外部设备在提出中断请求的同时，通过硬件自动形成中断服务程序入口地址。CPU响应中断后，将根据向量地址直接转入相应中断服务程序。这种具有产生向量地址的中断称为向量中断。

（2）非向量中断：非向量中断在产生中断的同时不能直接提供中断服务程序入口地址，而只产生一个中断查询程序的入口地址。需要通过中断查询程序确定中断源和中断服务程序的入口地址。用软件方法获取

3. 按中断源位置分：
   - 硬件中断
   - 软件中断
4. 按是否可屏蔽分：CPU根据需要可以禁止响应某些中断源的中断请求。
   - 可屏蔽中断：CPU可以禁止响应的中断
   - 不可屏蔽中断：CPU必须响应的中断。

2. 中断系统需解决的问题

8.4

(1) 各中断源 如何 向 CPU 提出请求？

(2) 各中断源 同时 提出 请求 怎么办？

(3) CPU 什么 条件、什么 时间、以什么 方式 响应中断？

(4) 如何 保护现场？

(5) 如何 寻找入口地址？

(6) 如何 恢复现场，如何 返回？

(7) 处理中断的过程中又 出现新的中断 怎么办？

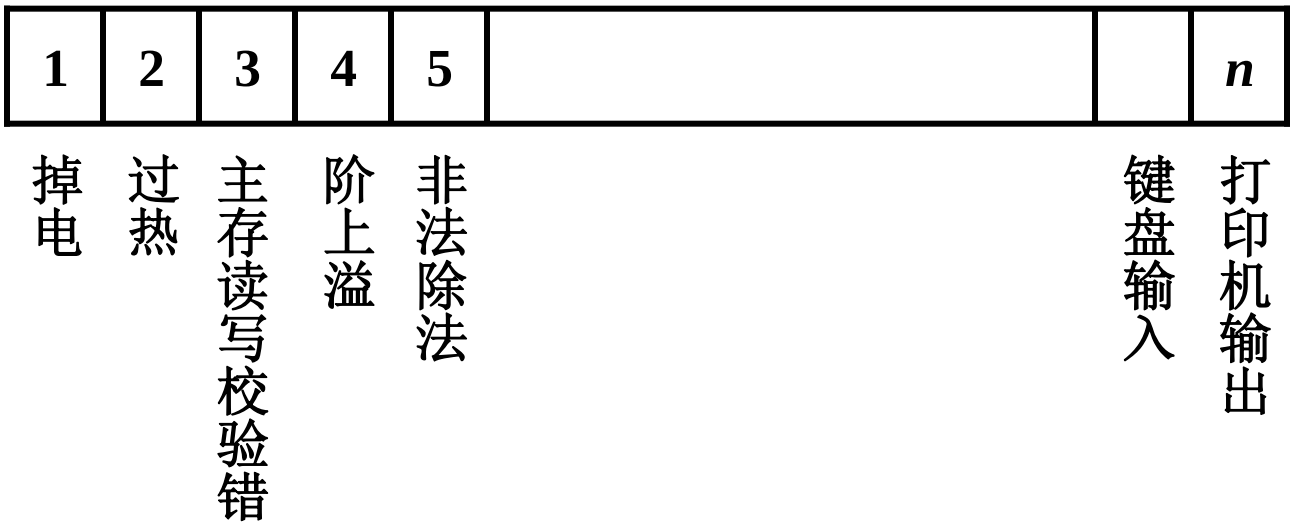
硬件 + 软件

二、中断请求标记和中断判优逻辑 8.4

1. 中断请求标记 INTR

一个请求源 一个 INTR 中断请求触发器

多个INTR 组成 中断请求标记寄存器



INTR 分散 在各个中断源的 接口电路中 CHP5

INTR 集中 在 CPU 的中断系统 内

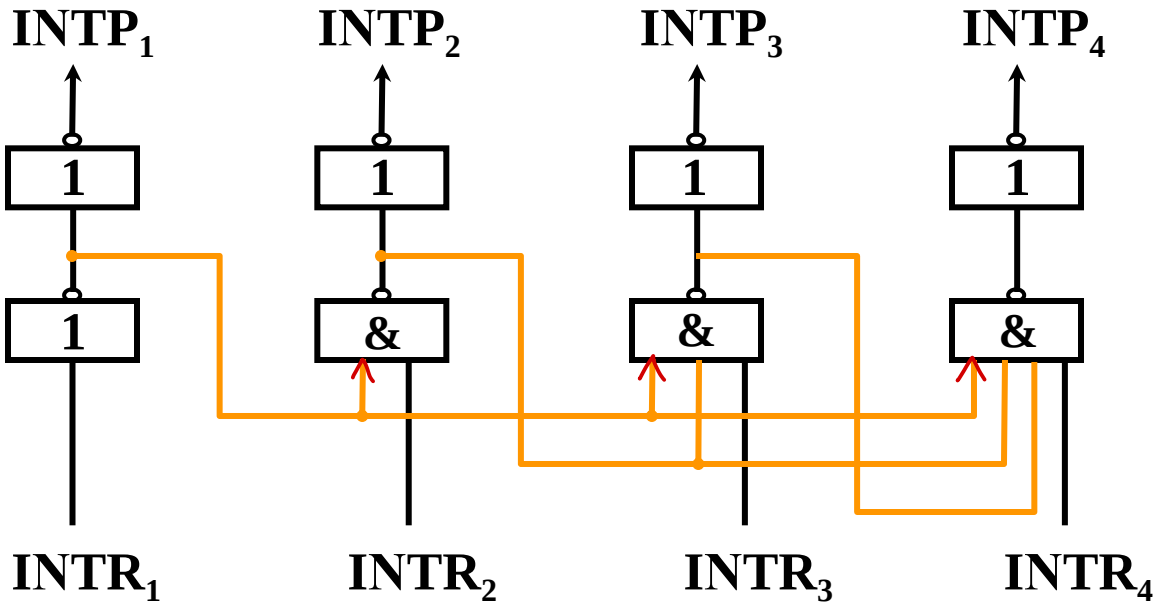
2. 中断判优逻辑

(1) 硬件实现（排队器）

① 分散 在各个中断源的 接口电路中　链式排队器

参见 第五章

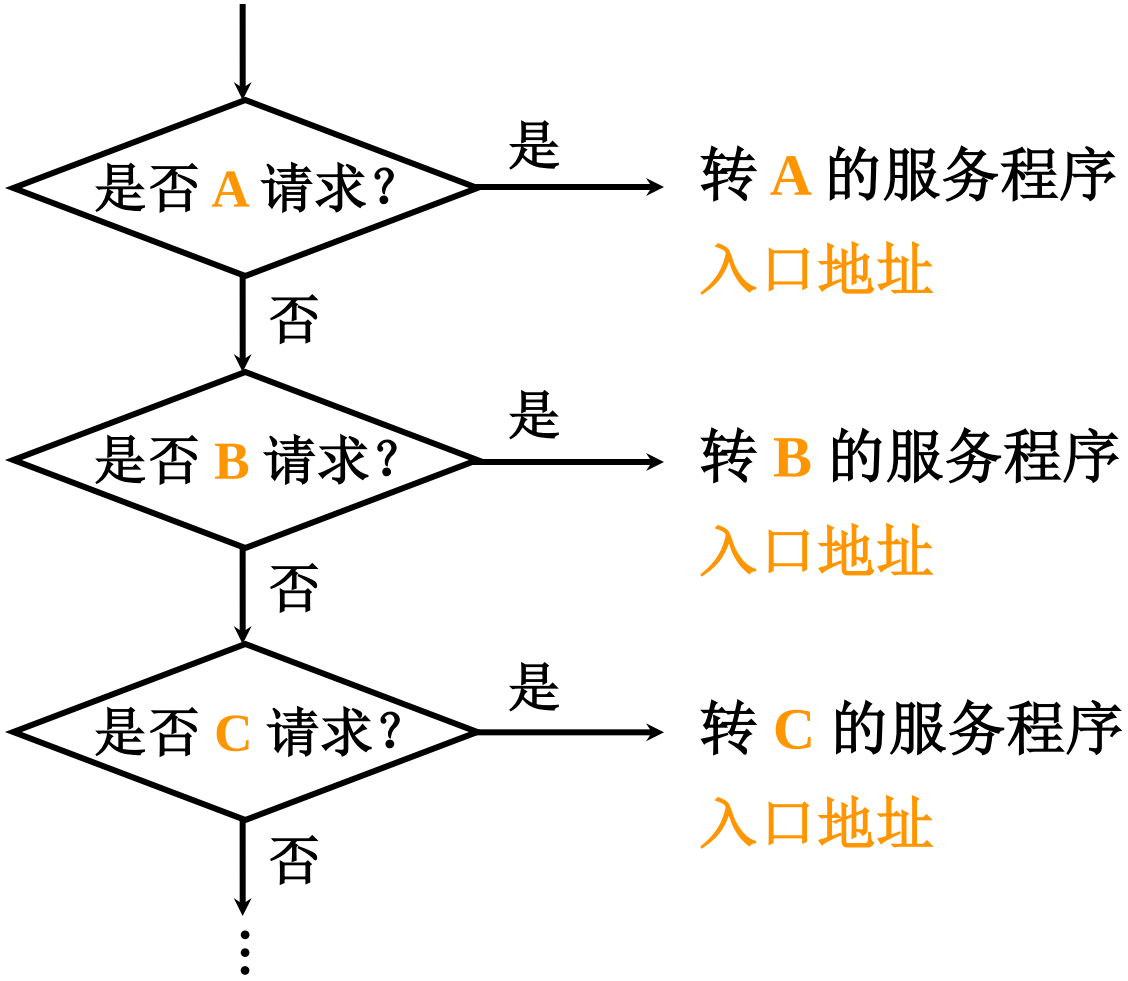
② 集中 在 CPU 内



$INTR_1$、$INTR_2$、$INTR_3$、$INTR_4$ 优先级 按 降序 排列

(2) 软件实现（程序查询）

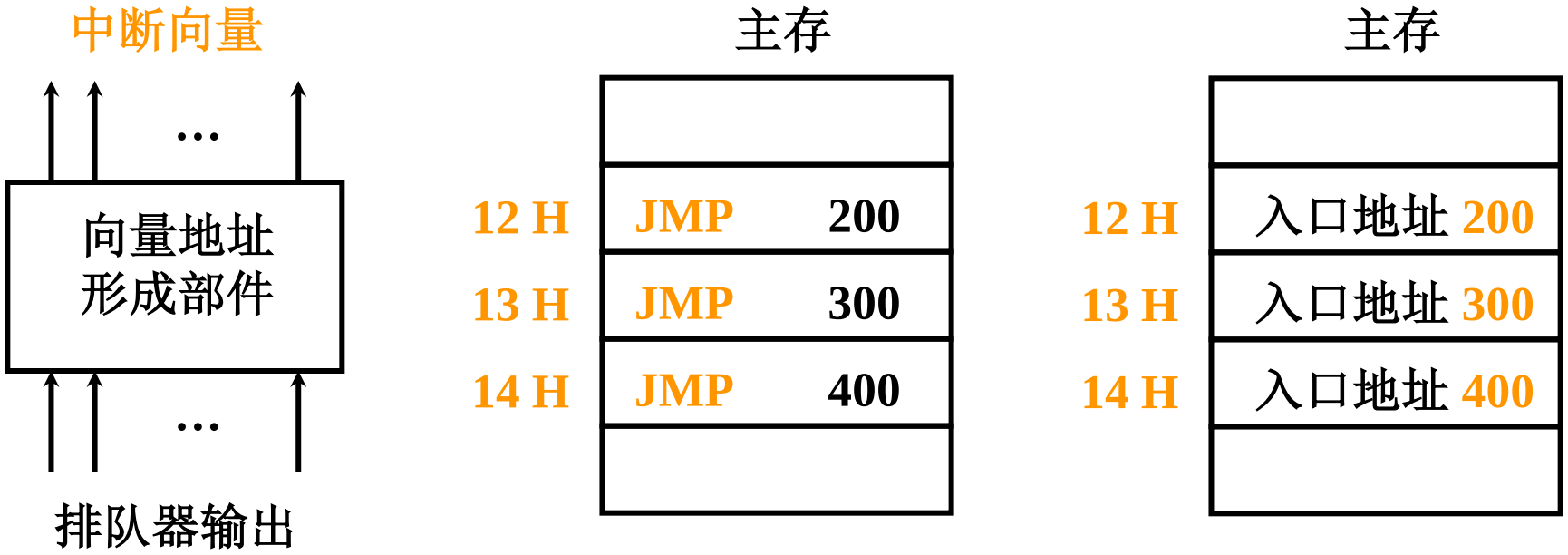
A、B、C 优先级按 降序 排列



三、中断服务程序入口地址的寻找 8.4

1. 硬件向量法

中断向量表固定



向量地址 12H、13H、14H

入口地址 200、 300、 400

2. 软件查询法

8.4

八个中断源 1，2，…　8 按 降序 排列

中断识别程序（入口地址 M）

地 址	指 令	说 明
M	SKP DZ $1^{\#}$	$1^{\#}$ D = 0 跳（D为完成触发器）
	JMP $1^{\#}$ SR	$1^{\#}$ D = 1 转$1^{\#}$ 服务程序
	SKP DZ $2^{\#}$	$2^{\#}$ D = 0 跳
	JMP $2^{\#}$ SR	$2^{\#}$ D = 1 转$2^{\#}$ 服务程序
	⋮	
	SKP DZ $8^{\#}$	$8^{\#}$ D = 0 跳
	JMP $8^{\#}$ SR	$8^{\#}$ D = 1 转$8^{\#}$ 服务程序

四、中断响应

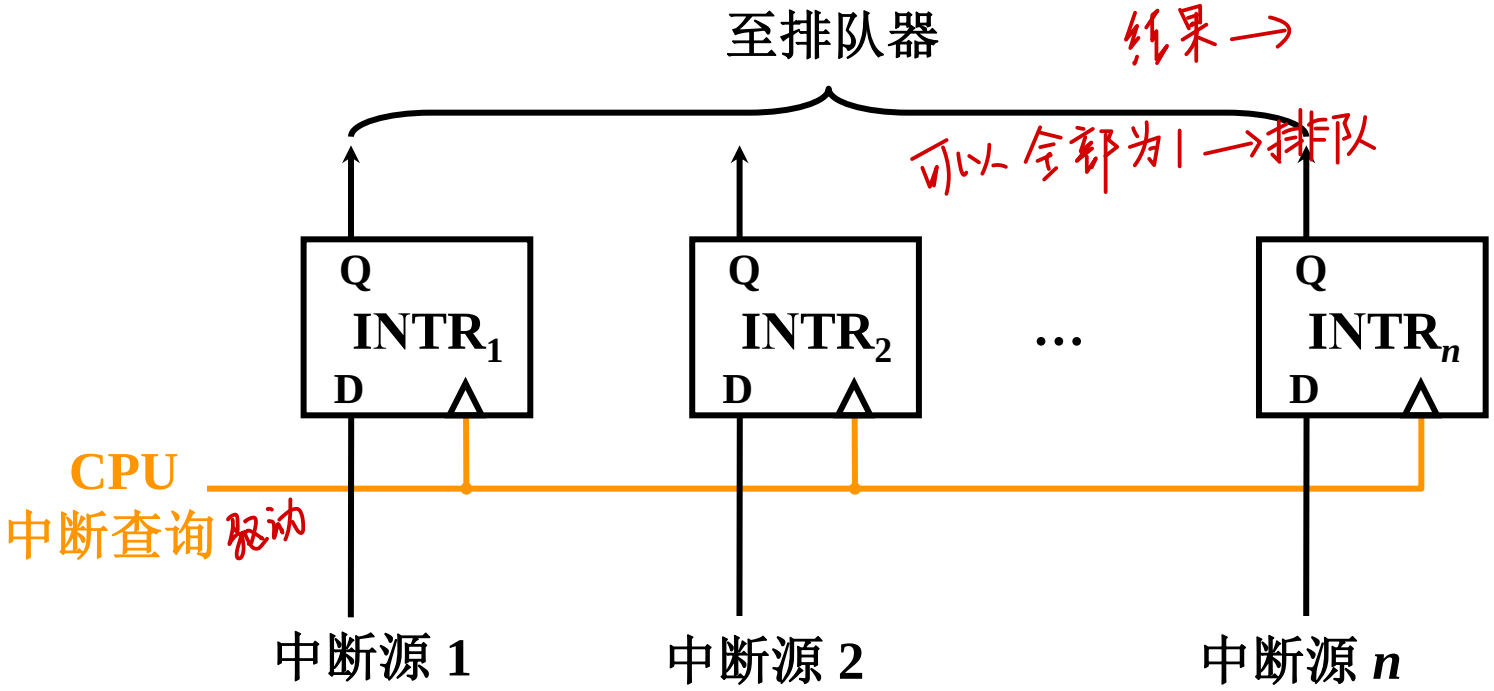
8.4

1. 响应中断的 条件

允许中断触发器 EINT = 1（可被开中断指令置1也可被关中断指令置0）

2. 响应中断的 时间

指令执行周期结束时刻由CPU 发查询信号



3. 中断隐指令

8.4

(1) 保护程序断点

① 断点存于 特定地址（0 号地址） 内　② 断点 进栈

(2) 寻找服务程序入口地址

向量地址 ⟶ PC（硬件向量法）

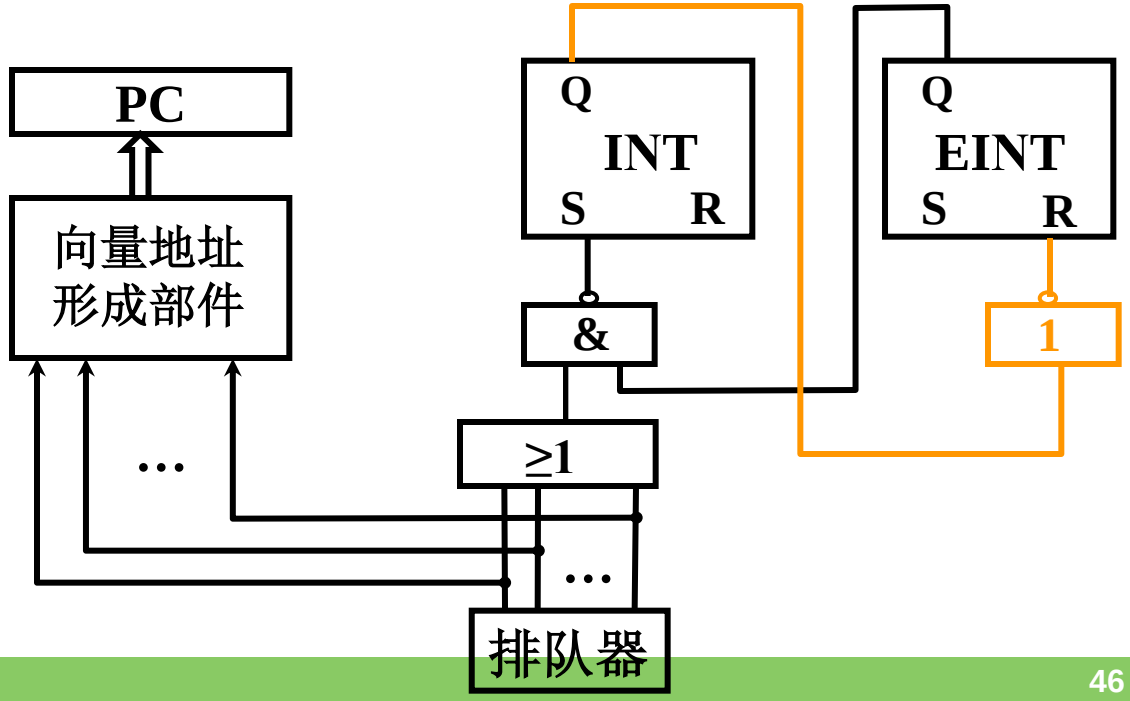
中断识别程序 入口地址 M ⟶ PC （软件查询法）

(3) 硬件 关中断

INT 中断标记

EINT 允许中断

R – S 触发器



五、保护现场和恢复现场 8.4

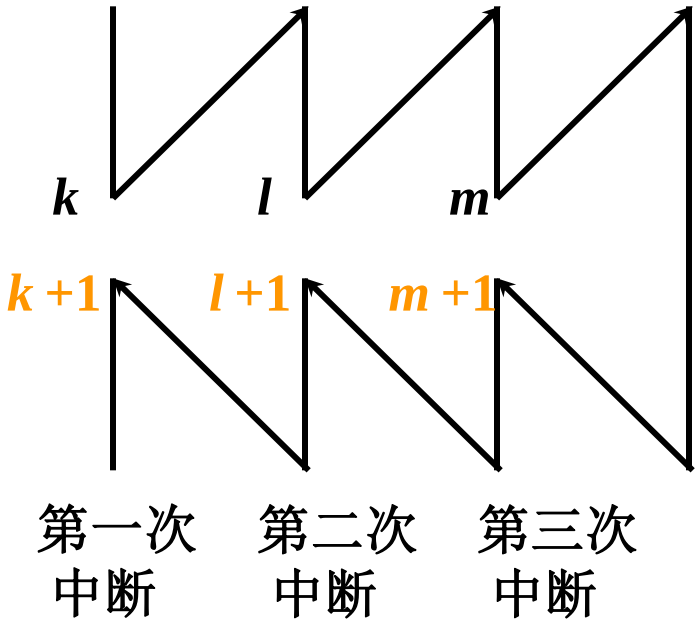
1. 保护现场
   - 断点 —— 中断隐指令 完成
   - 寄存器 内容 —— 中断服务程序 完成
2. 恢复现场 —— 中断服务程序 完成

中断服务程序

保护现场	PUSH
其它服务程序	视不同请求源而定
恢复现场	POP
中断返回	IRET

六、中断屏蔽技术

1. 多重中断的概念

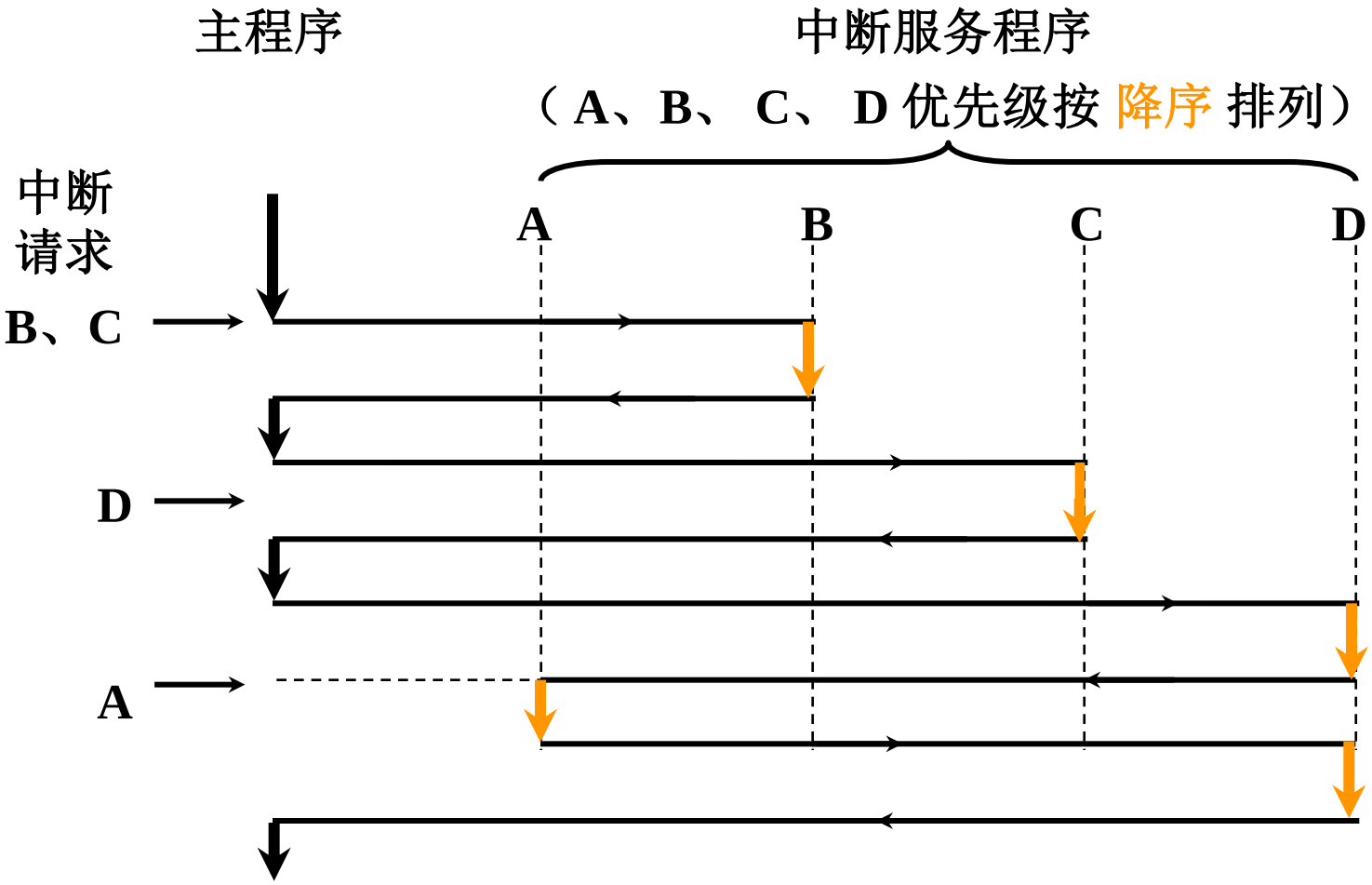


程序断点 $k+1$, $l+1$, $m+1$

2. 实现多重中断的条件

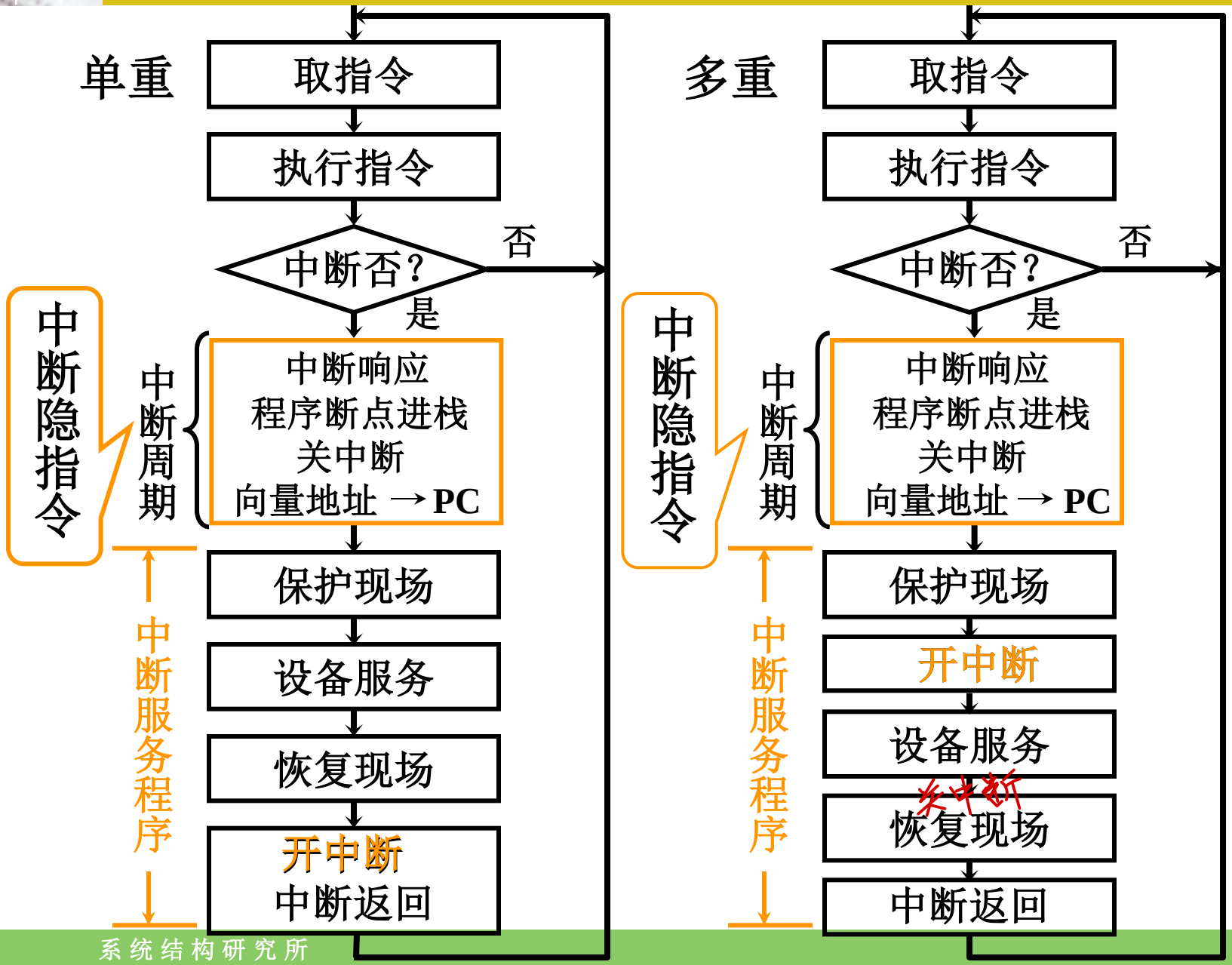
(1) 提前 设置 开中断 指令

(2) 优先级别高 的中断源 有权中断优先级别低 的中断源



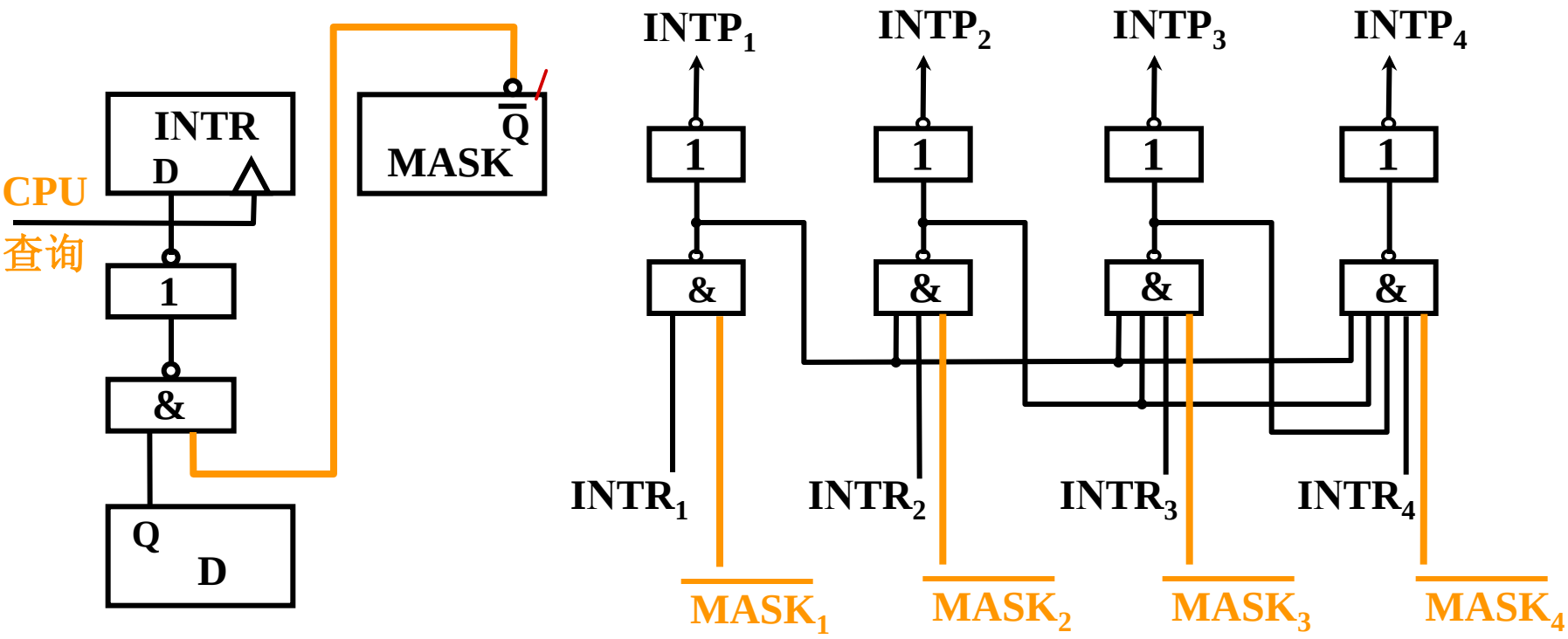
单重中断和多重中断的服务程序流程

5.5



3. 屏蔽技术

(1) 屏蔽触发器的作用



MASK = 0（未屏蔽）　　**$MASK_i$ = 1**（屏蔽）

INTR 能被置 “1”　　**$INTP_i$ = 0**（不能被排队选中）

(2) 屏蔽寄存器和屏蔽字

16个中断源 1，2，3，… 16 按 降序 排列

优先级	屏 蔽 字	
1	1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1	不被任何打断
2	0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1	
3	0 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1	
4	0 0 0 1 1 1 1 1 1 1 1 1 1 1 1 1	
5	0 0 0 0 1 1 1 1 1 1 1 1 1 1 1 1	
6	0 0 0 0 0 1 1 1 1 1 1 1 1 1 1 1	
⋮	⋮	
15	0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1	
16	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1	任何都可打断

屏蔽字与中断源的优先级一一对应

(3) 屏蔽技术可改变处理优先等级

响应优先级　　不可改变

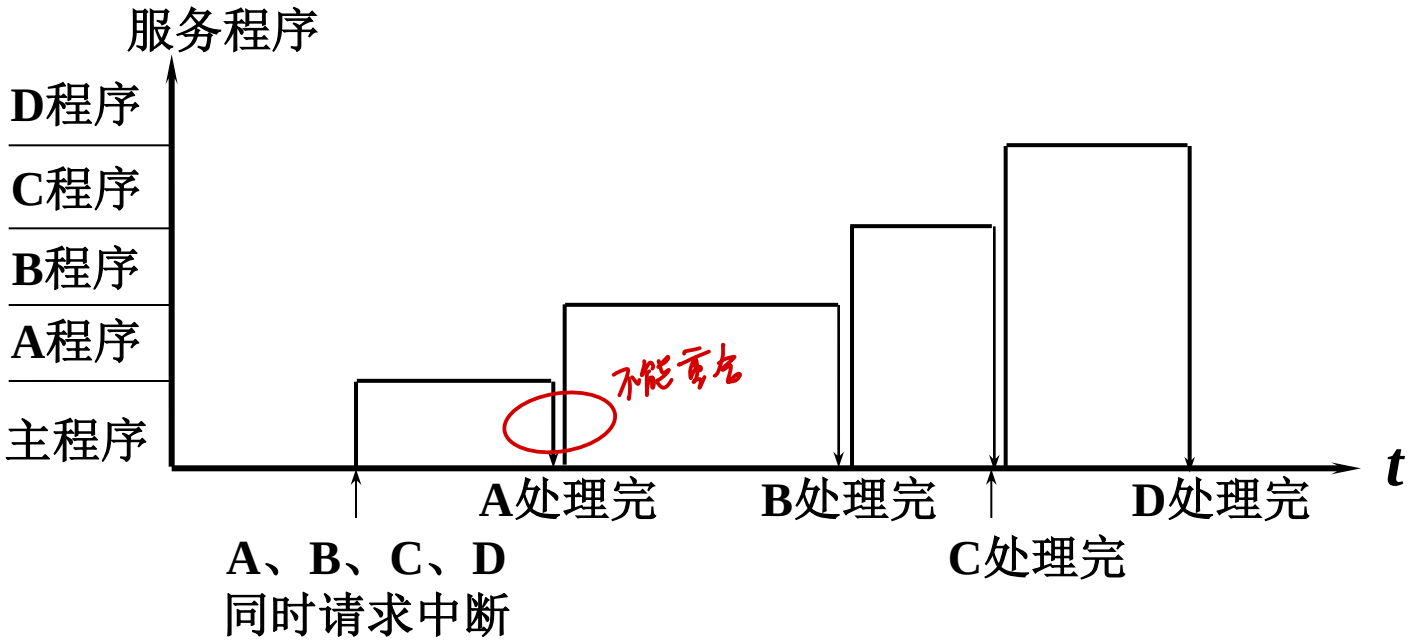
处理优先级　　可改变（通过重新设置屏蔽字）

中断源	原屏蔽字	新屏蔽字
A	1 1 1 1	1 1 1 1
B	0 1 1 1	0 1 0 0
C	0 0 1 1	0 1 1 0
D	0 0 0 1	0 1 1 1

响应优先级　A→B→C→D 降序排列

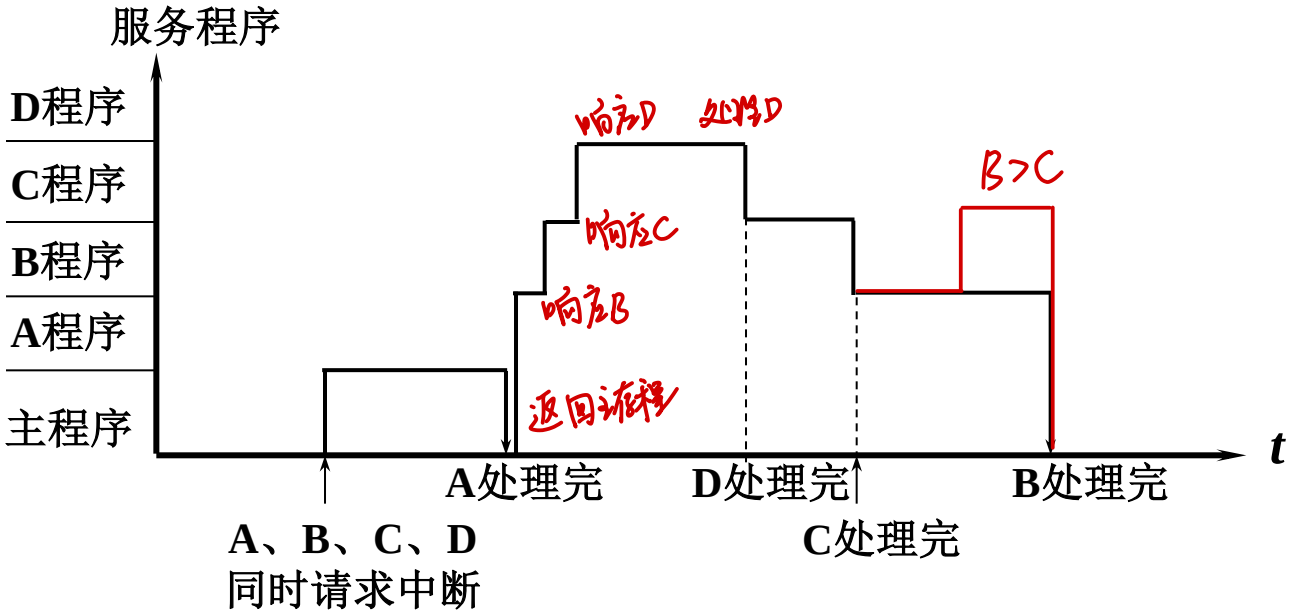
处理优先级　A→D→C→B 降序排列

(3) 屏蔽技术可改变处理优先等级



CPU 执行程序轨迹（原屏蔽字）

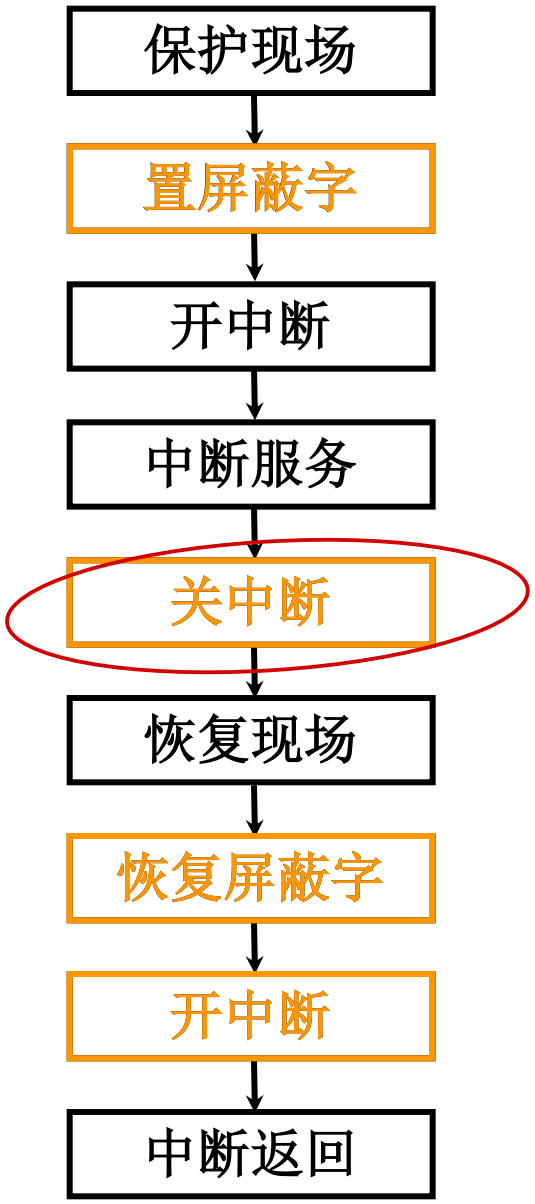
(3) 屏蔽技术可改变处理优先等级



CPU 执行程序轨迹（新屏蔽字）

- 响应优先级 A→B→C→D 降序排列
- 处理优先级 A→D→C→B 降序排列

(4) 新屏蔽字的设置



(5) 屏蔽技术的其他作用

可以 人为地屏蔽 某个中断源的请求

便于程序控制：e.g., 屏蔽阶上溢

4. 多重中断的断点保护

8.4

(1) 断点进栈　　中断隐指令 完成

(2) 断点存入“ 0 ” 地址　　中断隐指令 完成

中断周期　　0 ⟶ MAR

命令存储器写

PC ⟶ MDR　　断点 ⟶ MDR

(MDR) ⟶ 存入存储器

三次中断，三个断点都存入 “ 0 ” 地址

？ 如何保证断点不丢失？

例 题

1. 某机有五个中断源，按中断响应的优先顺序由高到低为**L0,L1,L2,L3,L4**，现要求优先顺序改为**L4,L2,L3,L0,L1**，写出各中断源的屏蔽字。

中断源	屏蔽字 0 1 2 3 4
L0 L1 L2 L3 L4	

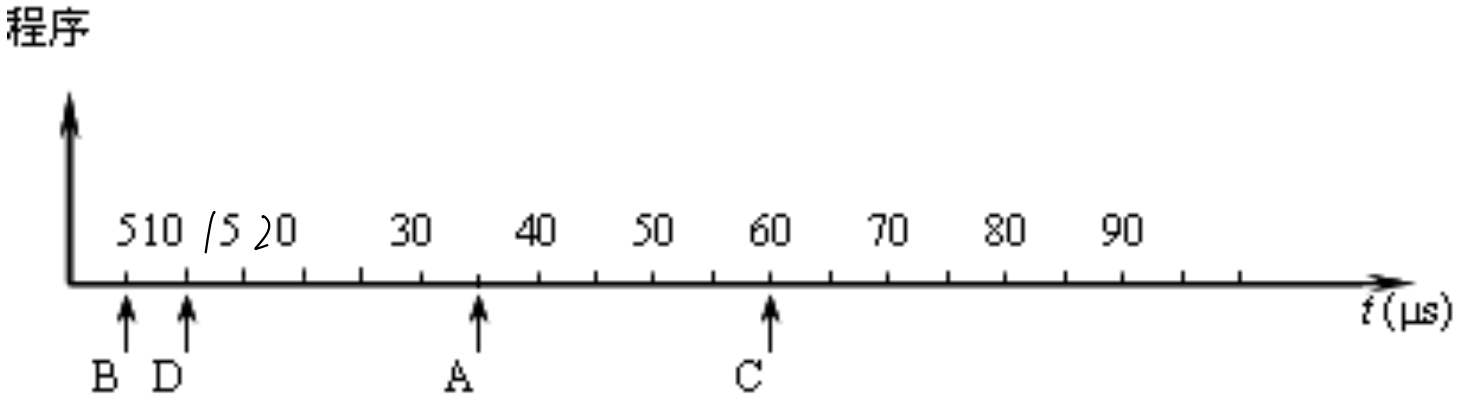
中断源	屏蔽字 0 1 2 3 4
L0	1 1 0 0 0
L1	0 1 0 0 0
L2	1 1 1 1 0
L3	1 1 0 1 0
L4	1 1 1 1 1

例 题

2. 设某机有四个中断源A、B、C、D，其硬件排队优先次序为A > B > C > D，现要求将中断处理次序改为D > A > C > B。

（1）写出每个中断源对应的屏蔽字。

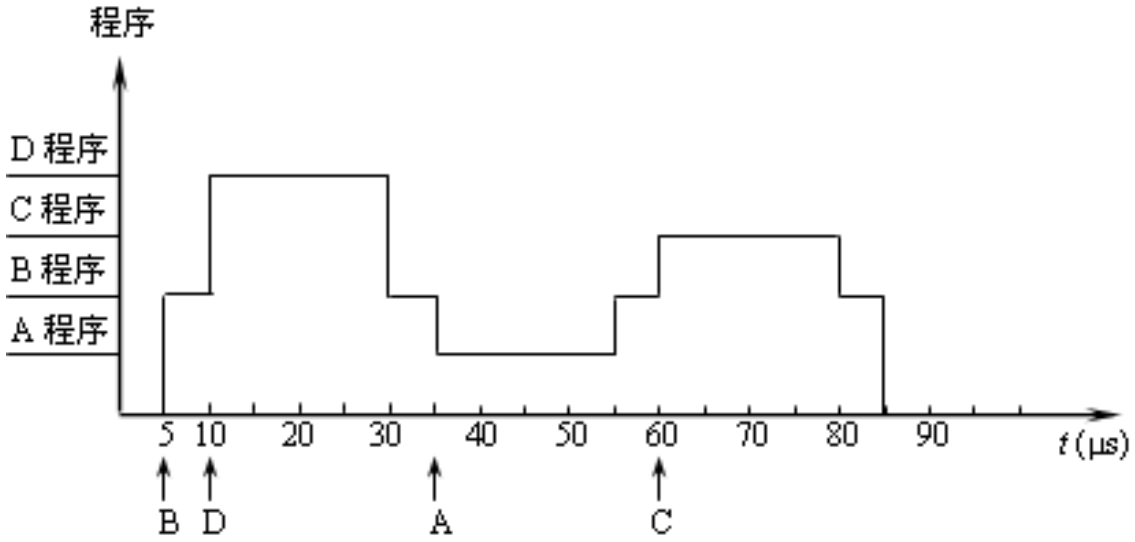
（2）按下图时间轴给出的四个中断源的请求时刻，画出CPU执行程序的轨迹。设每个中断源的中断服务程序时间均为20μs。



答：（1）在中断处理次序改为D > A > C > B后，每个中断源新的屏蔽字如表所示。

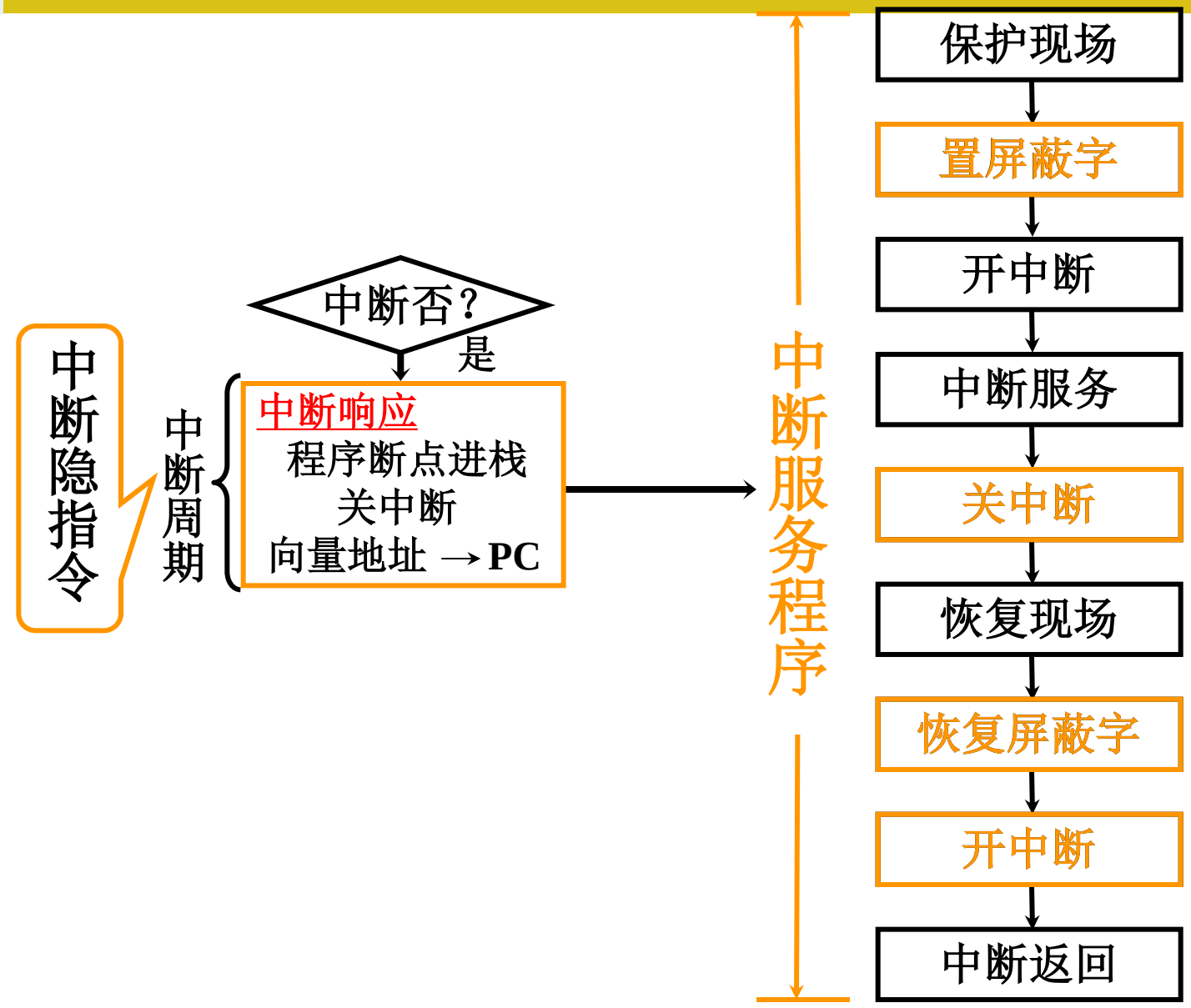
中断源	屏蔽字			
	A	B	C	D
A	1	1	1	0
B	0	1	0	0
C	0	1	1	0
D	1	1	1	1

（2）根据新的处理次序，CPU执行程序的轨迹如图所示



第八章 CPU 的结构和功能

- 8.1 CPU的结构
  - 功能：指令、操作、时间控制、数据、中断处理
  - CPU的结构：寄存器、CU、ALU、中断系统
- 8.2 指令周期
  - 概念、四个工作周期及其数据流
- 8.3 指令流水（...）
- 8.4 中断系统
  - 引起中断因素、要解决的七个问题
  - 中断请求标记、判优逻辑、中断响应（条件、时间、中断隐指令）、入口地址寻找
  - 中断屏蔽技术：多重中断、屏蔽字（改变处理优先级）


中断隐指令
中断周期
中断否？
是
中断响应
程序断点进栈
关中断
向量地址 →PC
中断服务程序
保护现场
置屏蔽字
开中断
中断服务
关中断
恢复现场
恢复屏蔽字
开中断
中断返回

Thank You !

系统结构研究所

第 9 章　控制单元的功能

系统结构研究所 · 计算机组成原理

第 9 章　控制单元的功能

9.1　操作命令的分析

9.2　控制单元的功能

9.1 操作命令的分析

完成一条指令分 4 个工作周期

包含

- 取指周期
- 间址周期
- 执行周期
- 中断周期

一、取指周期

PC ⟶ MAR ⟶ 地址线

1 ⟶ R　(启动主存读操作)

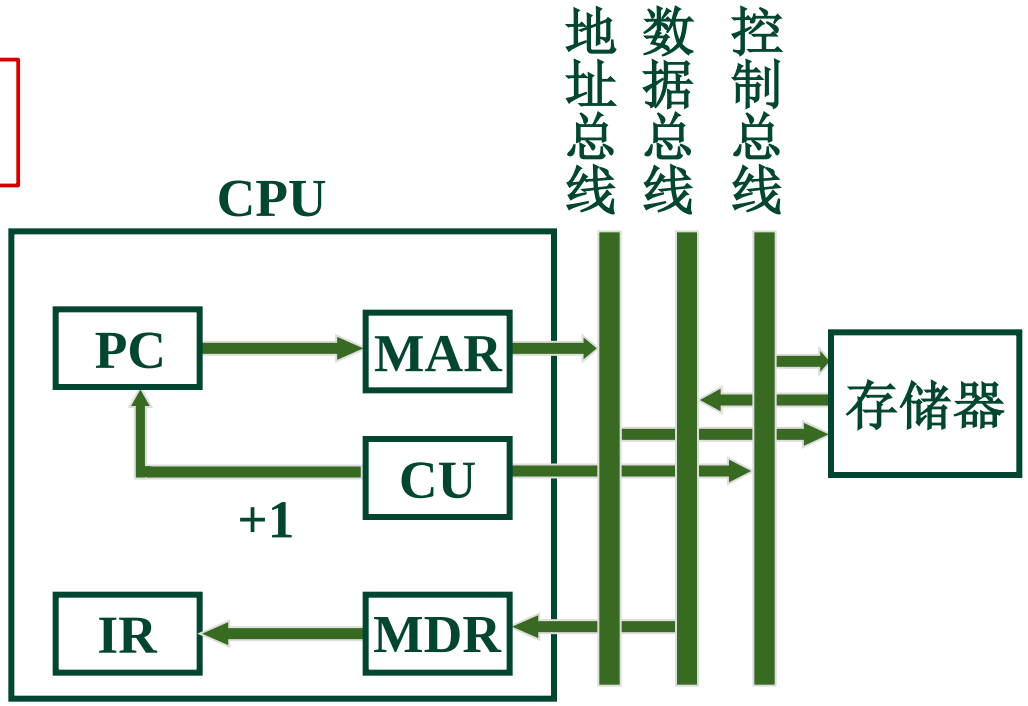
M (MAR) ⟶ MDR

（将MAR所指的主存单元的内容读至MDR）

MDR ⟶ IR

OP（IR）⟶ CU

(PC) + 1 ⟶ PC



二、间址周期

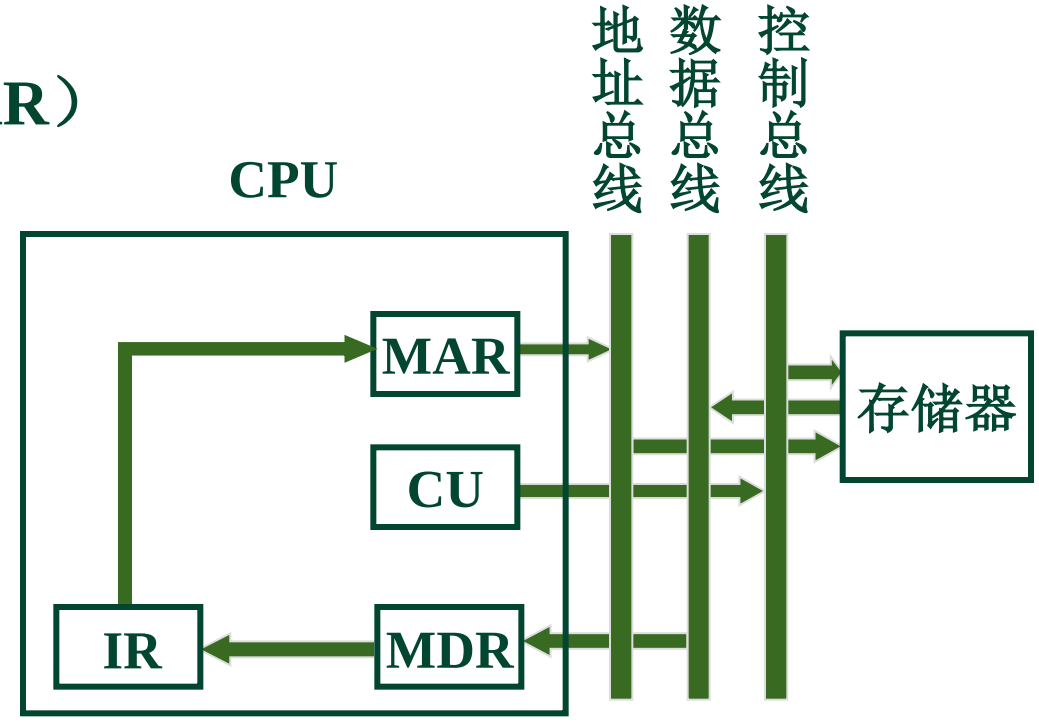
Ad (IR) ⟶ MAR

（指令形式地址 ⟶ MAR）

1⟶R

M (MAR) ⟶ MDR

MDR⟶ Ad (IR)



三、执行周期

1. 非访存指令

(1) CLA　清A　　$0 \longrightarrow ACC$

(2) COM　取反　　$\overline{ACC} \longrightarrow ACC$

(3) SHR　算术右移　　$L(ACC) \longrightarrow R(ACC)$, $(ACC_0 \longrightarrow ACC_0)$

(4) CSL　循环左移　　$R(ACC) \longrightarrow L(ACC)$, $(ACC_0 \longrightarrow ACC_n)$

(5) STP　停机指令　　$0 \longrightarrow G$　（运行标志触发器）

三、执行周期

2. 访存指令

(1) 加法指令　ADD　X

Ad(IR) ⟶ MAR

1 ⟶ R

M(MAR) ⟶ MDR

(ACC) + (MDR) ⟶ ACC

(2) 存数指令　STA　X

Ad(IR) ⟶ MAR

1 ⟶ W

ACC ⟶ MDR

MDR ⟶ M(MAR)

(3) 取数指令　LDA　X

Ad (IR) → MAR

1 → R　读

M (MAR) → MDR

MDR → ACC

3. 转移指令

(1) 无条件转　JMP　X

Ad (IR) → PC

(2) 条件转移　BAN　X　（负则转）

$A_0 \cdot Ad(IR) + \bar{A}_0(PC) \rightarrow PC$

(结果为负即A_0=1)

4. 三类指令的指令周期

非访存 指令周期 | 取指周期 | 执行周期 |

直接访存 指令周期 | 取指周期 | 执行周期 |

间接访存 指令周期 | 取指周期 | 间址周期 | 执行周期 |

转移 指令周期 | 取指周期 | 执行周期 |

间接转移 指令周期 | 取指周期 | 间址周期 | 执行周期 |

四、中断周期

9.1

硬件实现

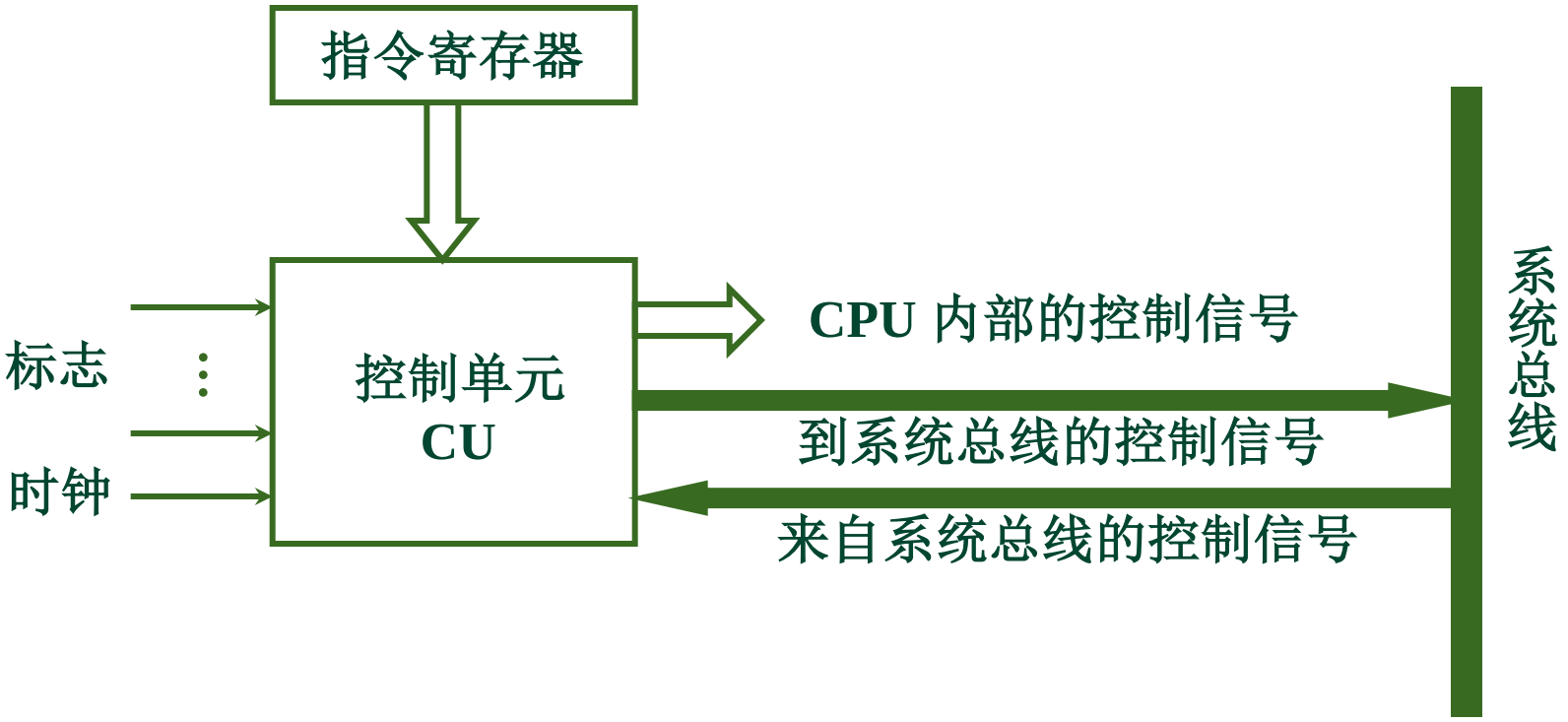
程序断点存入 “ 0 ” 地址	程序断点 进栈
0 ⟶ MAR	(SP) – 1 ⟶ MAR
1 ⟶ W	1 ⟶ W
PC ⟶ MDR	PC ⟶ MDR
MDR ⟶ M (MAR)	MDR ⟶ M (MAR)

中断识别程序入口地址 M ⟶ PC

软件实现

0 ⟶ EINT（置“0”）	0 ⟶ EINT（置“0”）

一、控制单元的外特性


指令寄存器
标志
时钟
控制单元
CU
CPU 内部的控制信号
到系统总线的控制信号
来自系统总线的控制信号
系统总线

1. 输入信号

(1) 时钟

CU 受时钟控制 三级时序系统

一个时钟脉冲

发一个操作命令或一组需同时执行的操作命令

(2) 指令寄存器 OP (IR) → CU

控制信号 与操作码有关

(3) 标志

CU 受标志控制

(4) 外来信号

如 INTR 中断请求

HRQ 总线请求

2. 输出信号

(1) CPU 内的各种控制信号

$R_i \longrightarrow R_j$

(PC) + 1 ⟶ PC

ALU　＋、－、与、或 ……

(2) 送至控制总线的信号

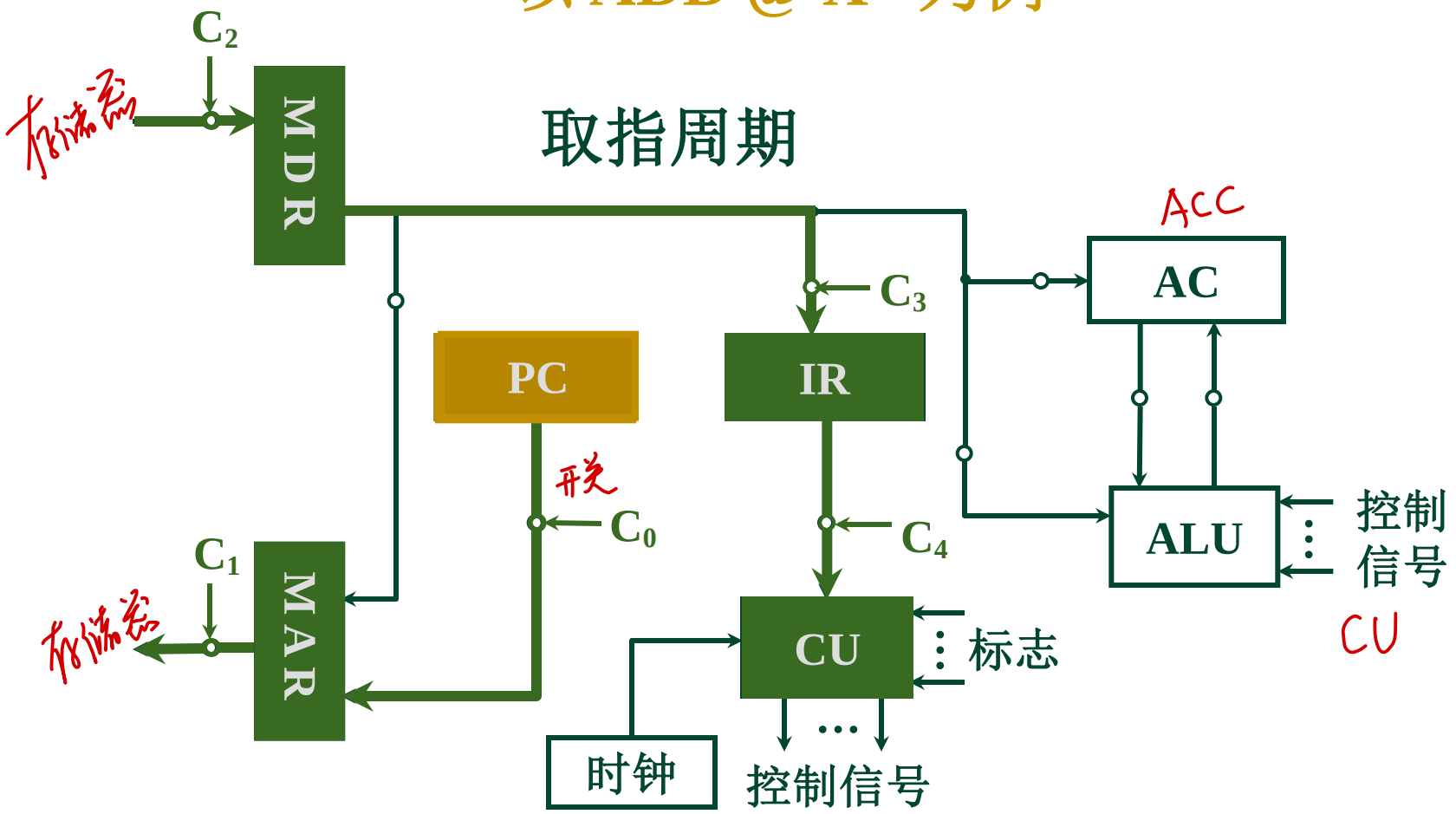
$\overline{\text{MREQ}}$	访存控制信号
$\overline{\text{IO}}/\text{M}$	访 IO/ 存储器的控制信号
$\overline{\text{RD}}$	读命令
$\overline{\text{WR}}$	写命令
INTA	中断响应信号
HLDA	总线响应信号

DMA 接口

二、控制信号举例

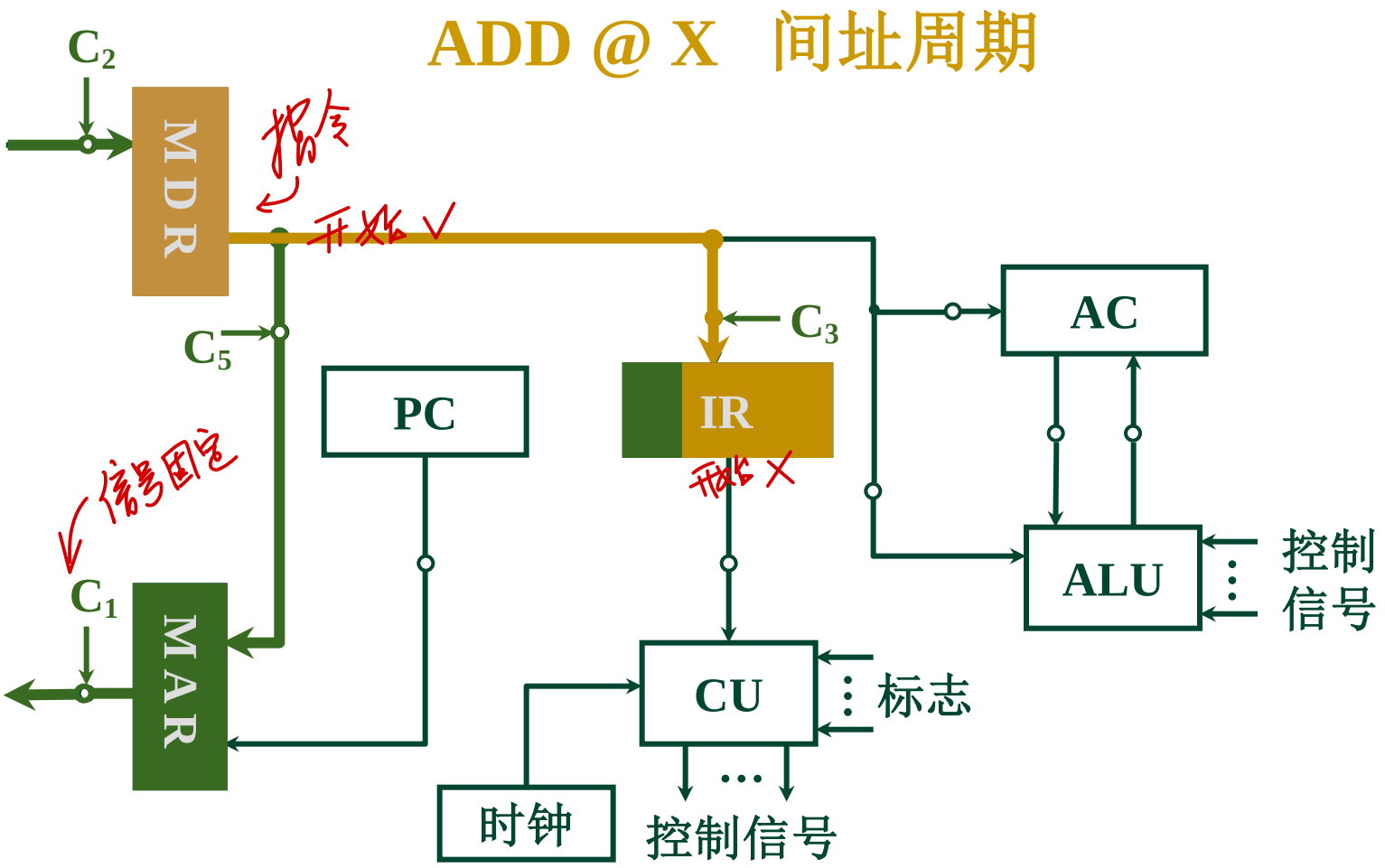
1. 不采用 CPU 内部总线的方式

以 ADD @ X 为例



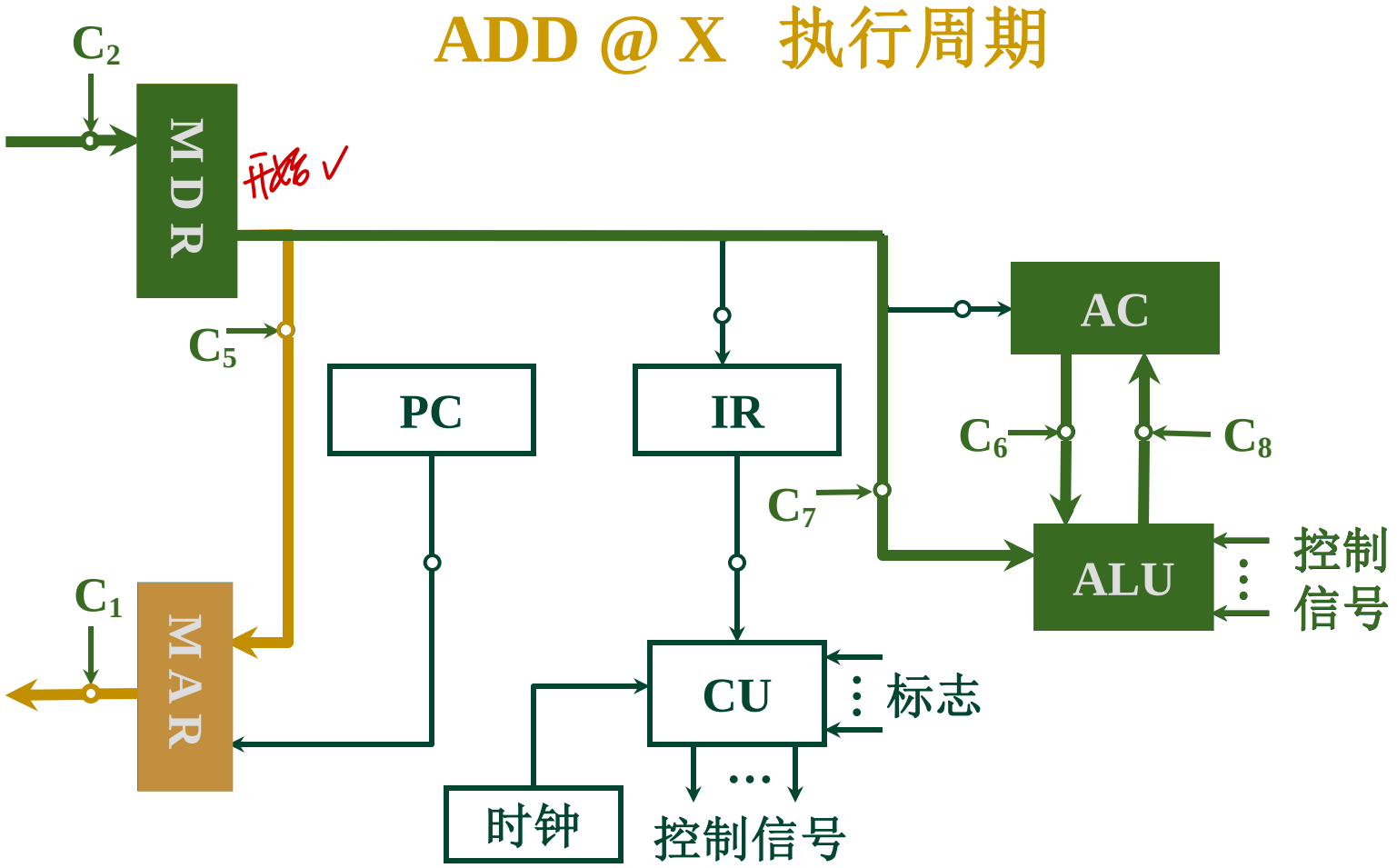
二、控制信号举例

1. 不采用 CPU 内部总线的方式



二、控制信号举例

1. 不采用 CPU 内部总线的方式



2. 采用 CPU 内部总线方式

(1) ADD @ X 取指周期

- PC ⟶ MAR ⟶ 地址线

  PC_O　　MAR_i

- CU 发读命令 1 ⟶ R

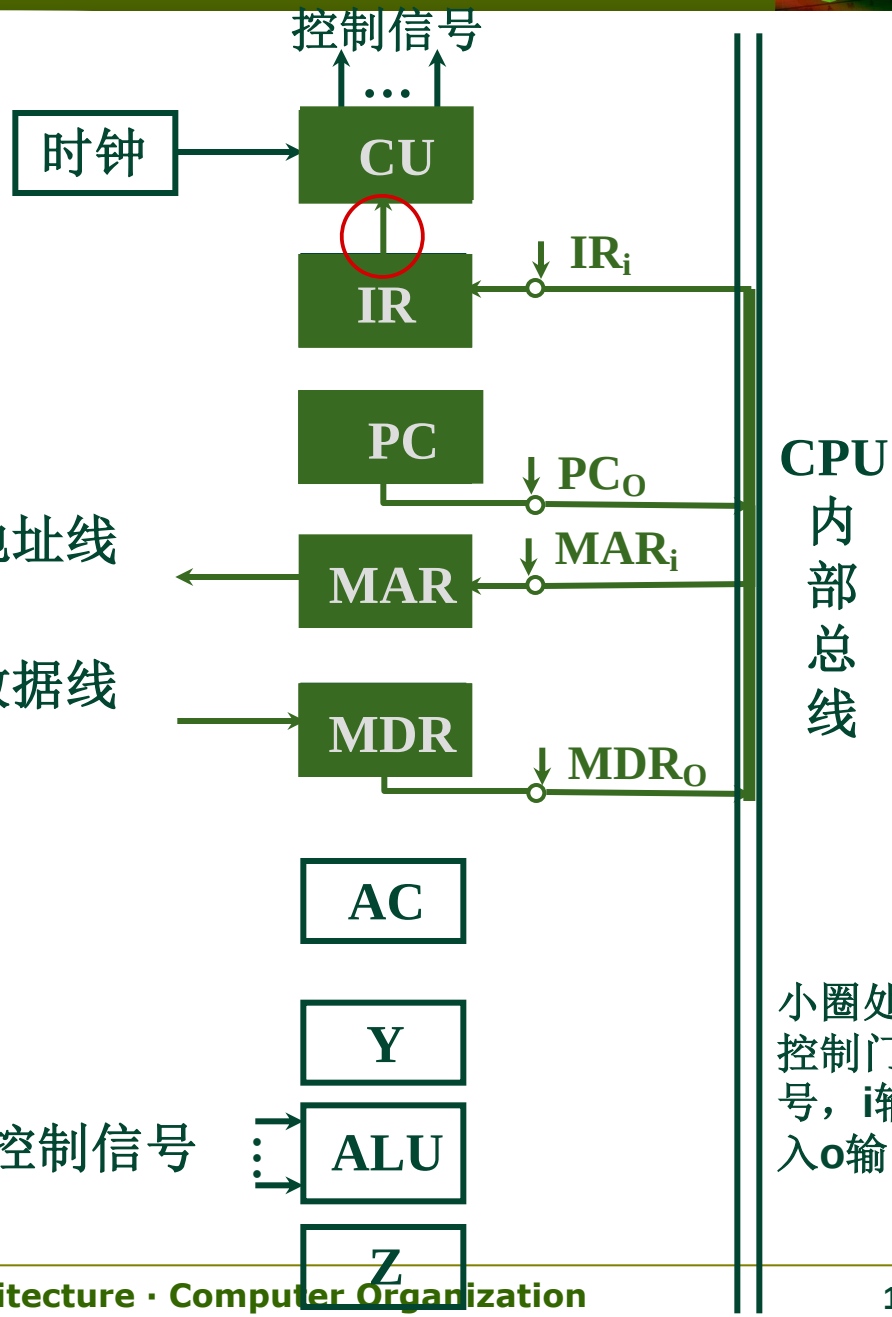
- 数据线 ⟶ MDR

- MDR ⟶ IR

  MDR_O　　IR_i

- OP（IR）⟶ CU　（通过总线）

- （PC）+ 1 ⟶ PC

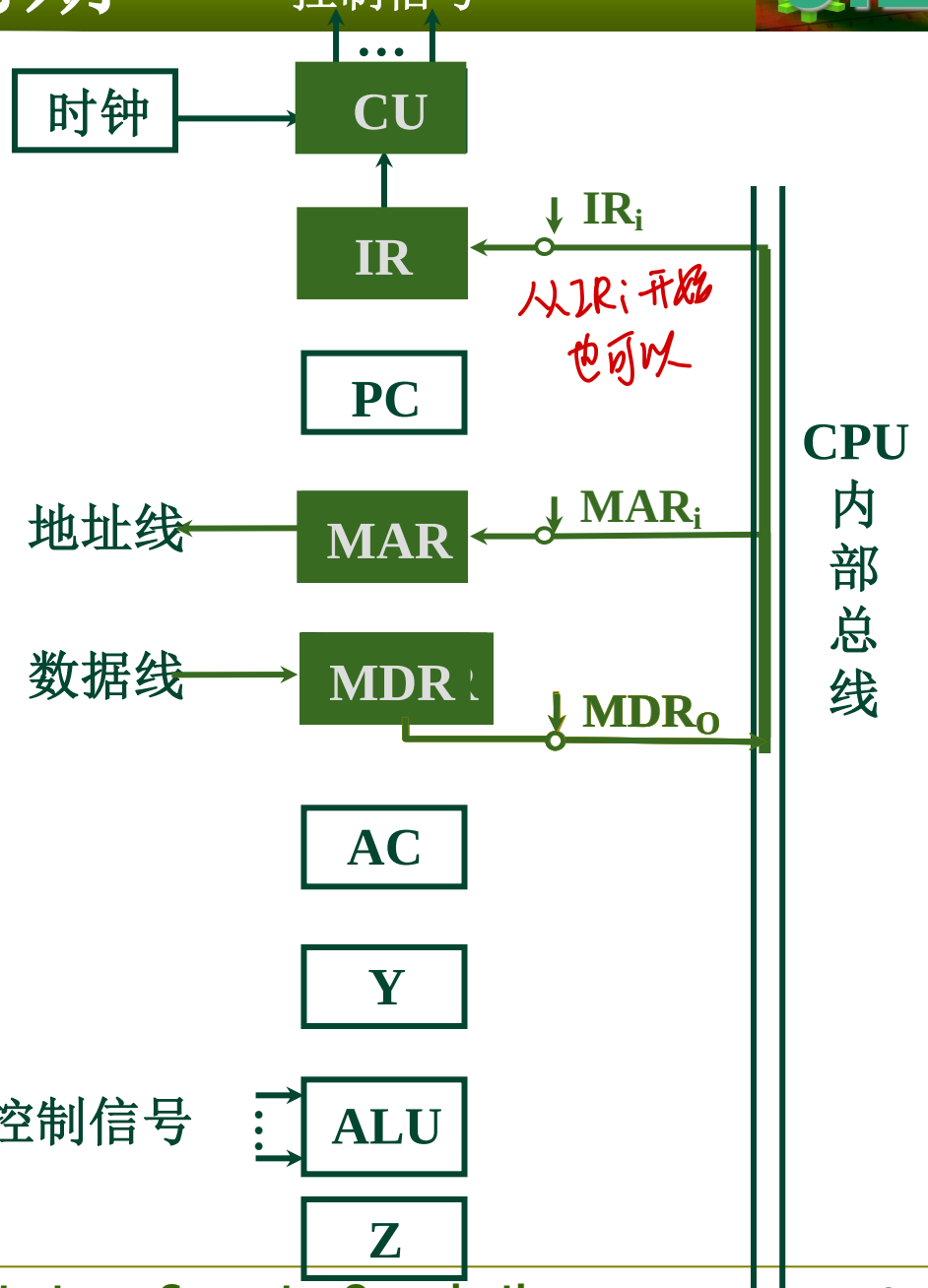


(2) ADD @ X 间址周期

形式地址 ⟶ MAR

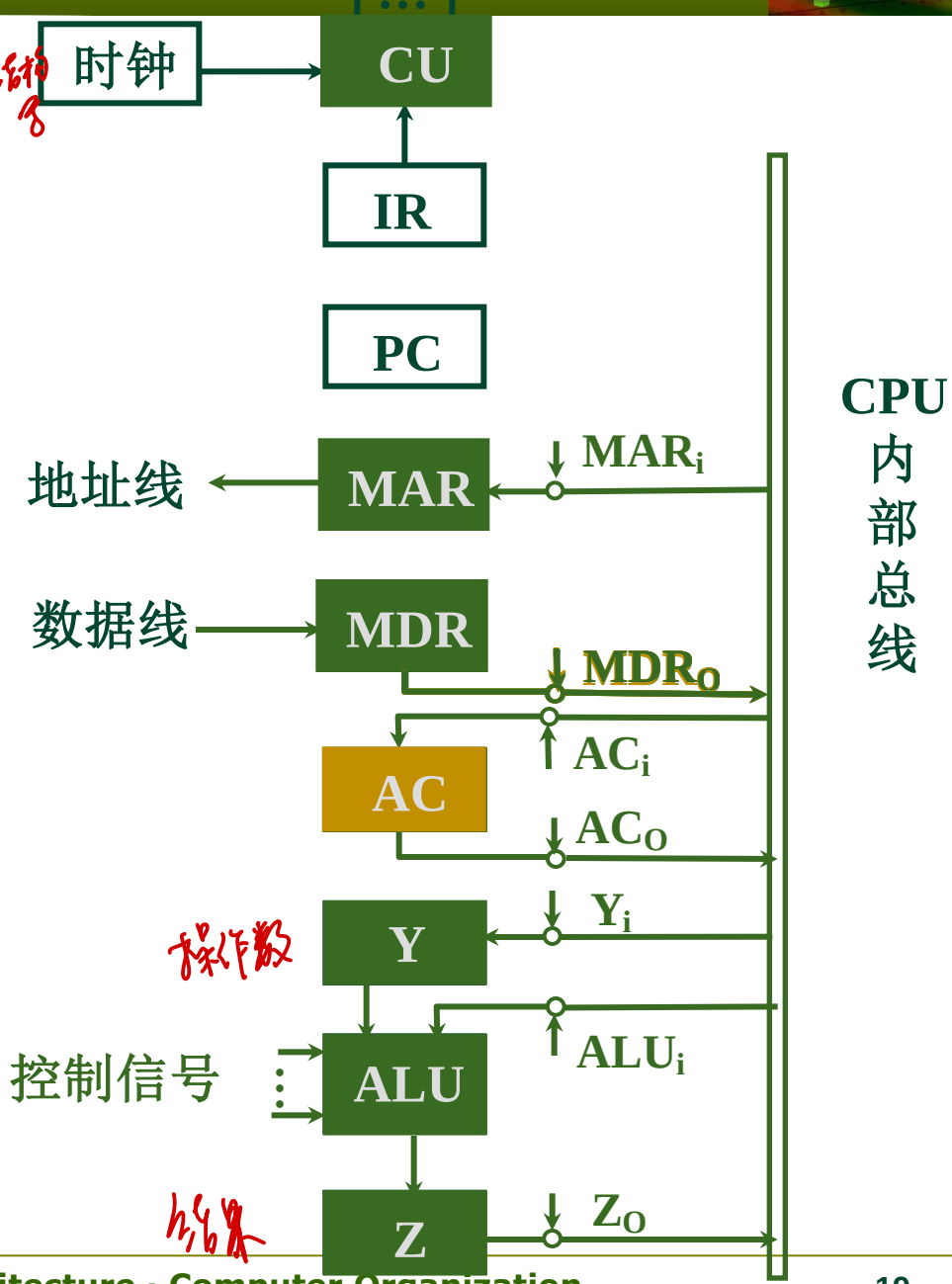
- MDR ⟶ MAR ⟶ 地址线
  MDR_O MAR_i
- 1 ⟶ R
- 数据线 ⟶ MDR
- MDR ⟶ IR
  MDR_O IR_i

有效地址 ⟶ Ad（IR）



(3) ADD @ X 执行周期

- MDR ⟶ MAR ⟶ 地址线
  MDR_O MAR_i
- 1 ⟶ R
- 数据线 ⟶ MDR
- MDR ⟶ Y ⟶ ALU
  MDR_O Y_i
- AC ⟶ ALU
  AC_O ALU_i
- （AC）+（Y）⟶ Z
- Z ⟶ AC
  Z_O AC_i



(1) A: MDR

B: IR

C: MAR

D: PC

(2) PC → MAR → MM → MDR → IR

(3) ~~MDR~~ X → MAR → MM → MDR → ALU → AC

(4) ~~MDR~~ Y → MAR → MM → MDR → ALU → ~~ADD~~ → AC

(5) ~~MDR~~ Z → MAR → ~~MM~~, AC → MDR → MM

三、多级时序系统

1. 机器周期 (CPU周期)

(1) 机器周期的概念

所有指令执行过程中的一个基准时间

(2) 确定机器周期需考虑的因素

每条指令的执行 步骤

每一步骤 所需的 时间

(3) 基准时间的确定

- 以完成 最复杂 指令功能的时间 为准 (访存)
- 以 访问一次存储器 的时间 为基准

若指令字长 = 存储字长　　取指周期 = 机器周期

2. 时钟周期（节拍、状态）

- 一个机器周期内可完成若干个微操作
- 每个微操作需一定的时间，以时钟信号来控制产生每一个微操作命令
- 时钟信号控制节拍发生器，产生节拍，每个节拍宽度对应一个时钟周期
- 将一个机器周期分成若干个时间相等的时间段（节拍、状态、时钟周期）
- 时钟周期是控制计算机操作的最小单位时间
- 用时钟周期控制产生一个或几个微操作命令

多级时序系统

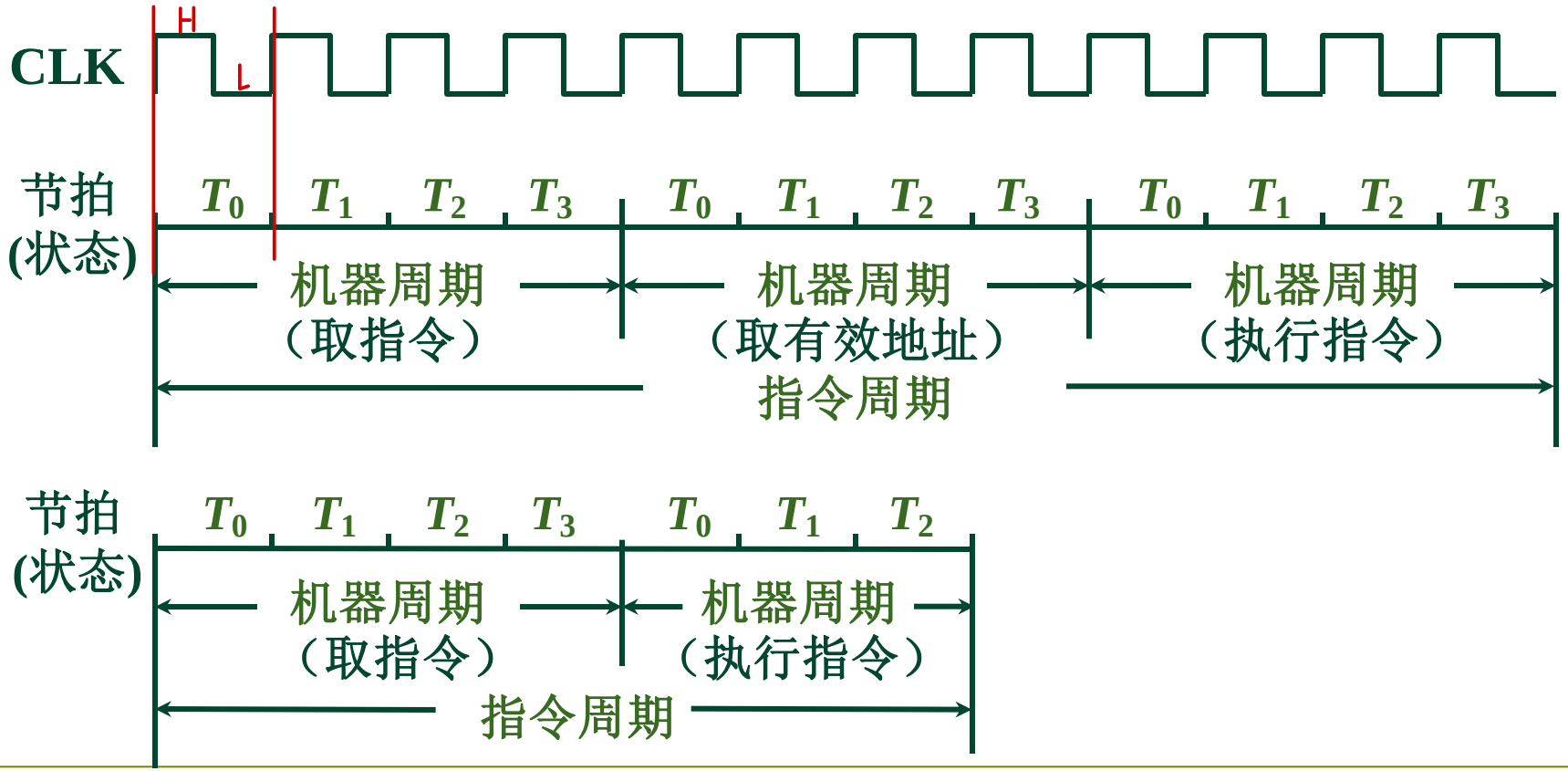
- 指令周期是从取指令、分析指令到执行完该指令所需的时间。
- 不同的指令，其指令周期长短可以不同。
- 在时序系统中通常不为指令周期设置时间标志信号，因而也不将其作为时序的一级

多级时序系统

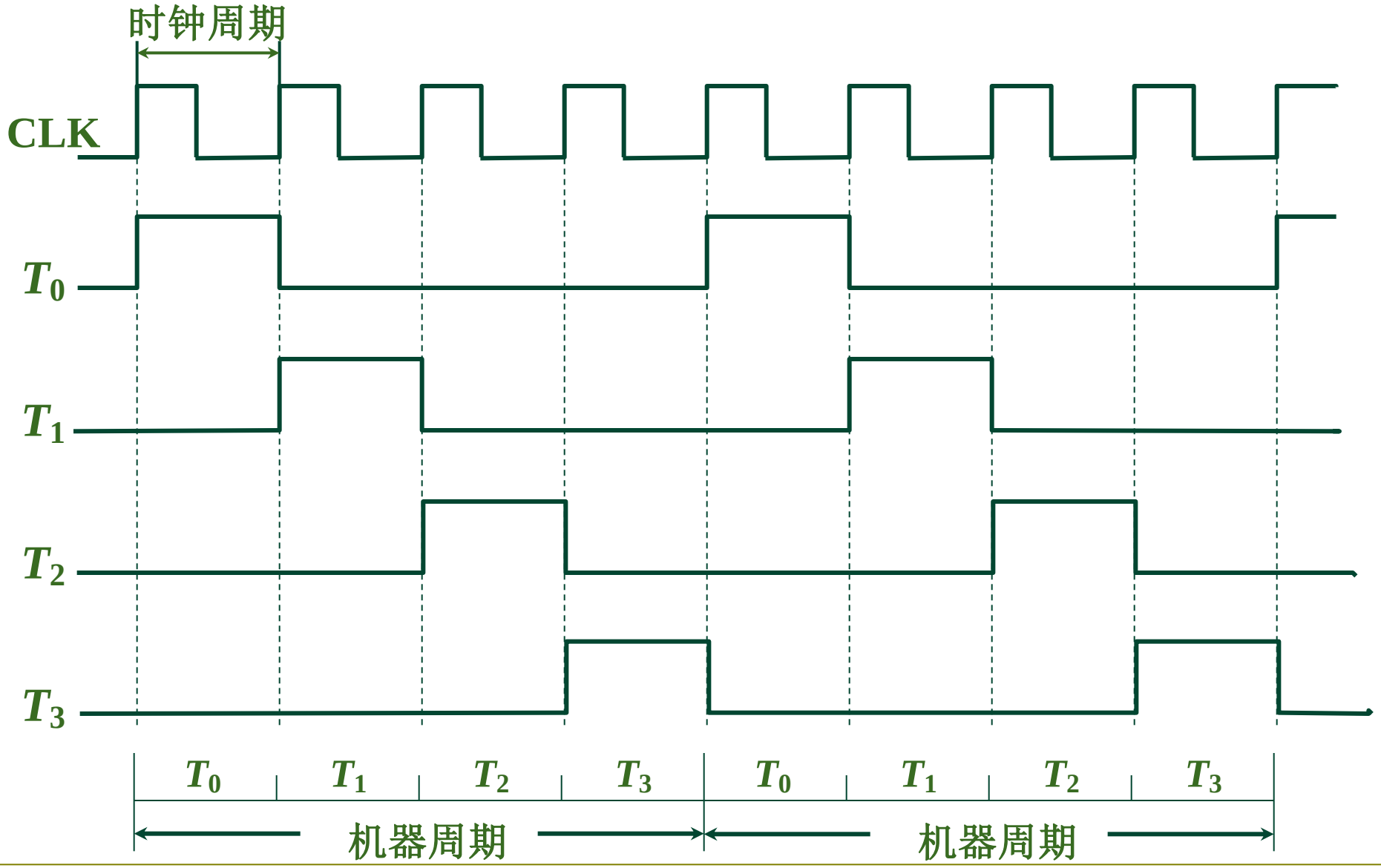
机器周期、节拍（状态）组成多级时序系统

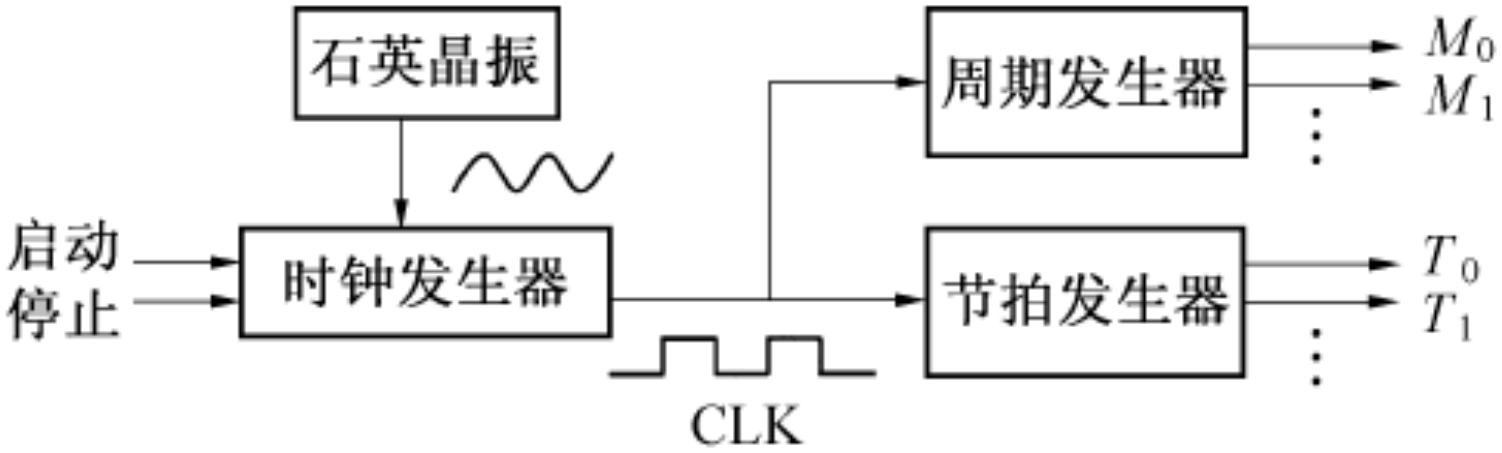
一个指令周期包含若干个机器周期

一个机器周期包含若干个时钟周期

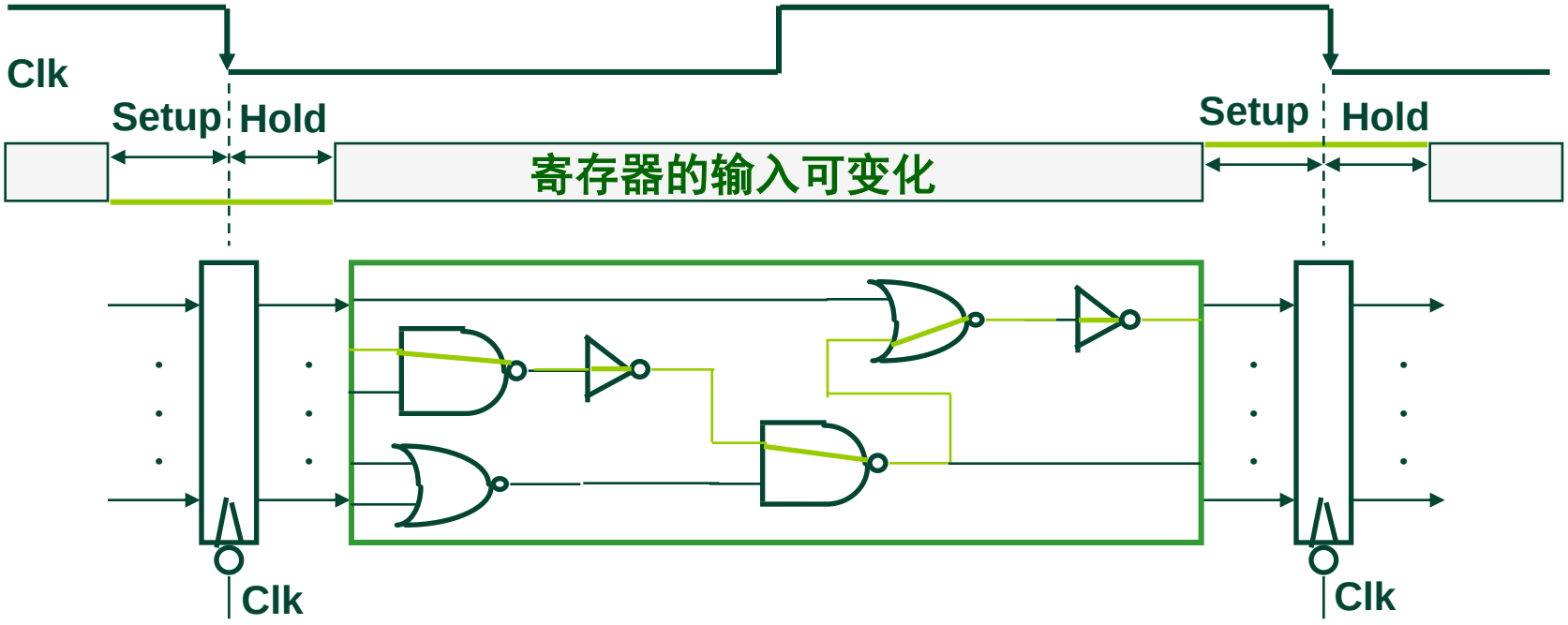


时钟周期（节拍、状态）





- 主振电路中的石英晶体振荡器产生一系列正弦信号，经时钟发生器整形分频得到时钟脉冲信号，成为机器的主频脉冲。主频脉冲再经过周期发生器和节拍发生器产生周期电位和节拍电位。



数据通路由 “… + 状态元件 + 操作元件(组合电路) + 状态元件 + …” 组成

只有状态元件能存储信息，所有操作元件都须从状态单元接收输入，并将输出写入状态单元中。其输入为前一时钟生成的数据，输出为当前时钟所用的数据

- 假定采用下降沿触发（负跳变）方式（也可以是上升沿方式）
  - 所有状态单元在下降沿写入信息，经过Latch Prop (clk-to-Q) 后输出有效
  - Cycle Time = Latch Prop + Longest Delay Path + Setup + Clock Skew(最大偏移)

机器速度与机器主频的关系

- 机器的 主频 f 越快 机器的 速度也越快
- 在机器周期所含时钟周期数 相同 的前提下，
  两机 平均指令执行速度之比 等于 两机主频之比

$$\frac{\text{MIPS}_1}{\text{MIPS}_2} = \frac{f_1}{f_2}$$

见例题

- 机器速度 不仅与 主频有关，还与机器周期中所含时钟周期（主频的倒数）数 以及指令周期中所含的 机器周期数有关

- 机器运行速度还与字长和计算机体系结构有关。

字长越长，单位时间内完成的数据运算就越多，运算速度越快

体系结构：存储器采用分级结构，处理器流水结构、多机结构等

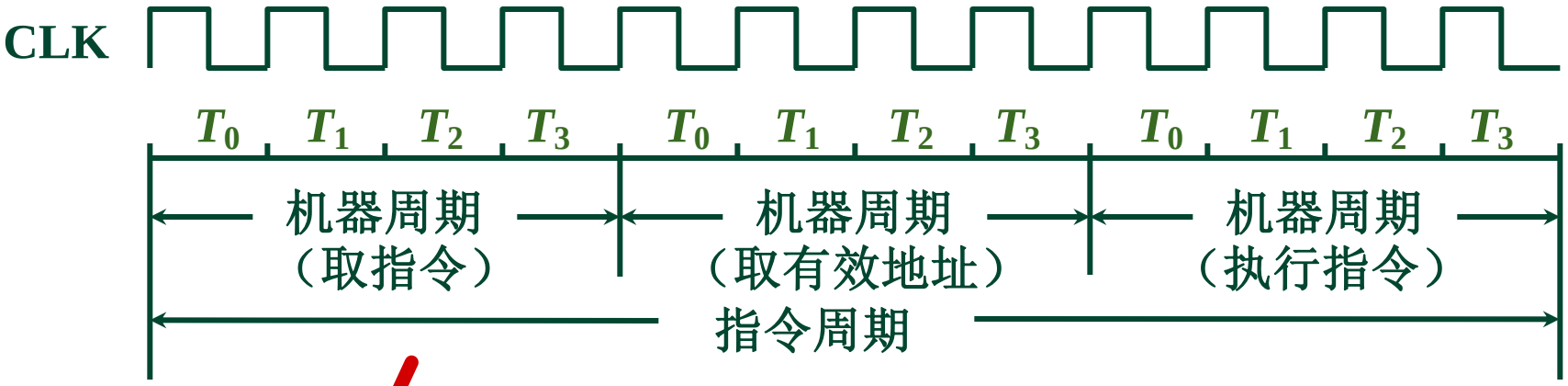
❖ 其他因素：主存速度、Cache、总线传输速度、硬盘速度

四、CU的控制方式

产生不同微操作命令序列所用的时序控制方式

1. 同步控制方式

任一微操作均由 统一基准时标 的时序信号控制



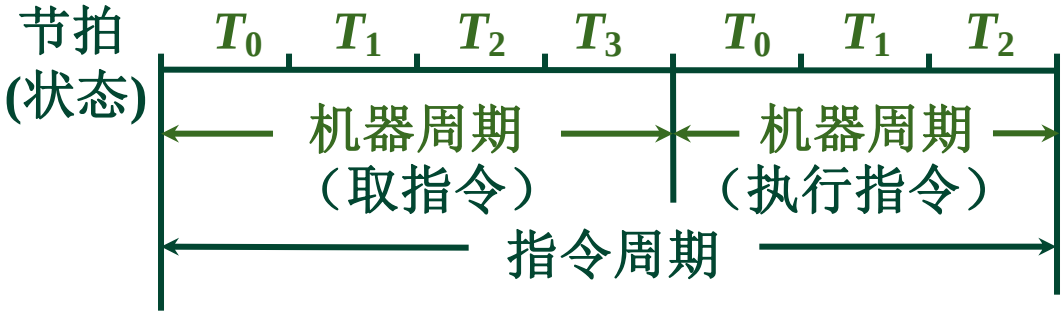
(1) 采用 定长 的机器周期

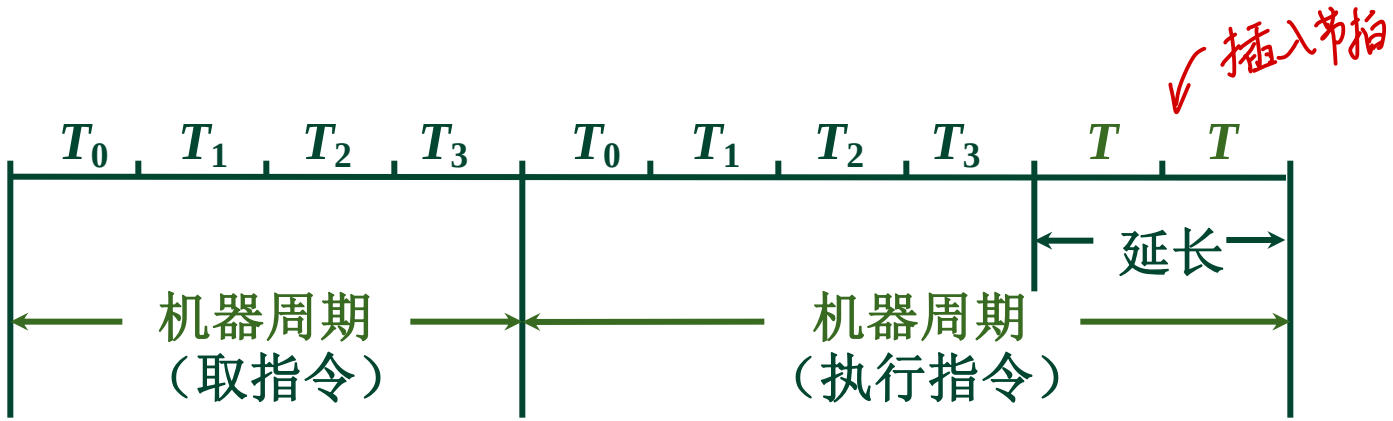
以 最长 的 微操作序列 和 最繁 的微操作作为 标准

机器周期内 节拍数相同

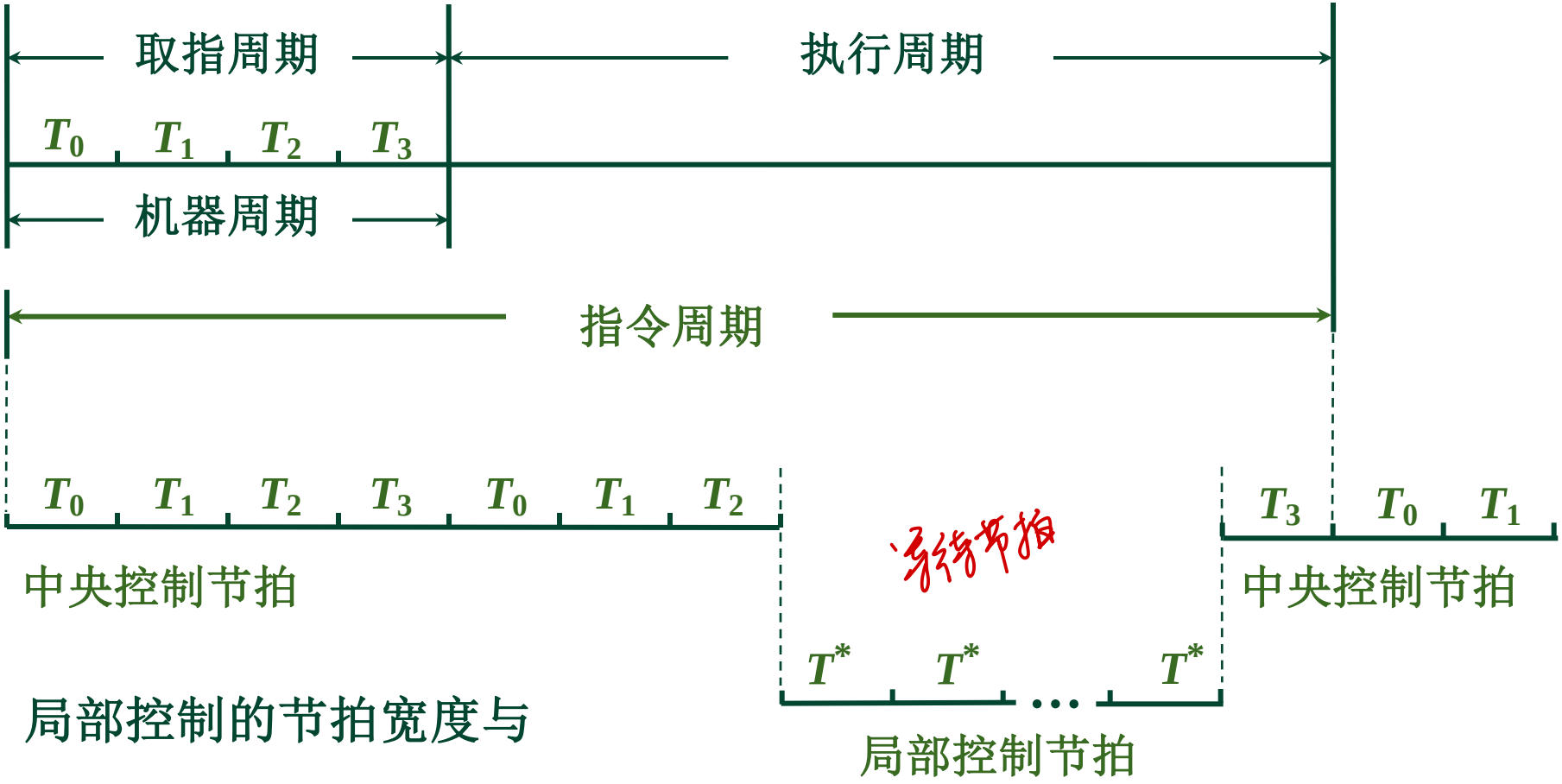
(2) 采用不定长的机器周期

机器周期内 节拍数不等





(3) 采用中央控制和局部控制相结合的方法



局部控制的节拍宽度与

中央控制的节拍宽度一致

四、 CU的控制方式

2. 异步控制方式

无基准时标信号

无固定的周期节拍和严格的时钟同步

采用 应答方式

3. 联合控制方式

同步与异步相结合

大部分统一、小部分区别对待

如：取指同步、I/O异步

4. 人工控制方式

(1) Reset

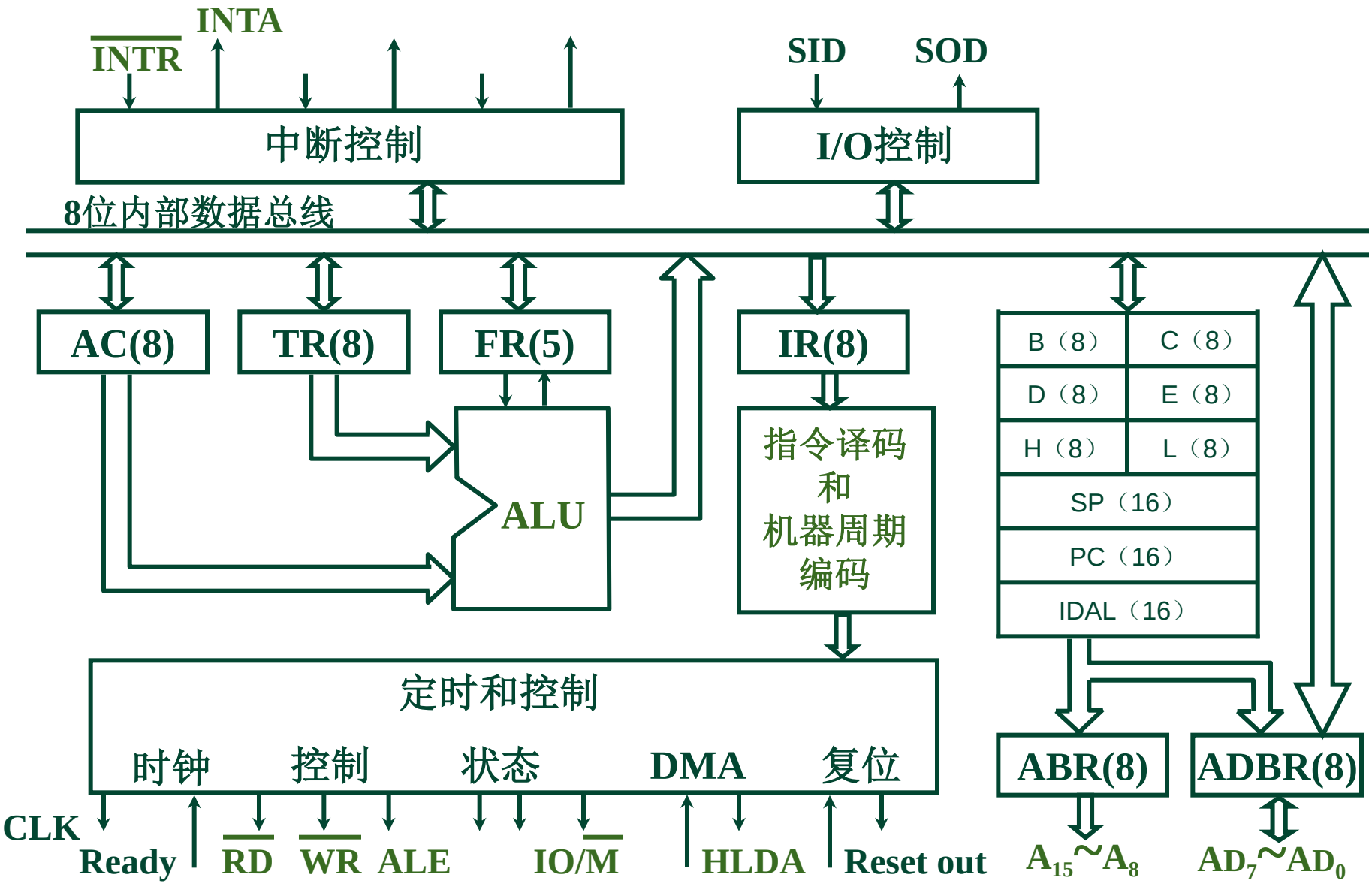
(2) 连续 和 单条 指令执行转换开关

(3) 符合停机开关

五、多级时序系统实例分析

9.2

1. 8085 的组成



2. 8085 的外部引脚

9.2

(1) 地址和数据信号

A_{15}~A_8　AD_7~AD_0

SID　　SOD

(2) 定时和控制信号

入　X_1　X_2

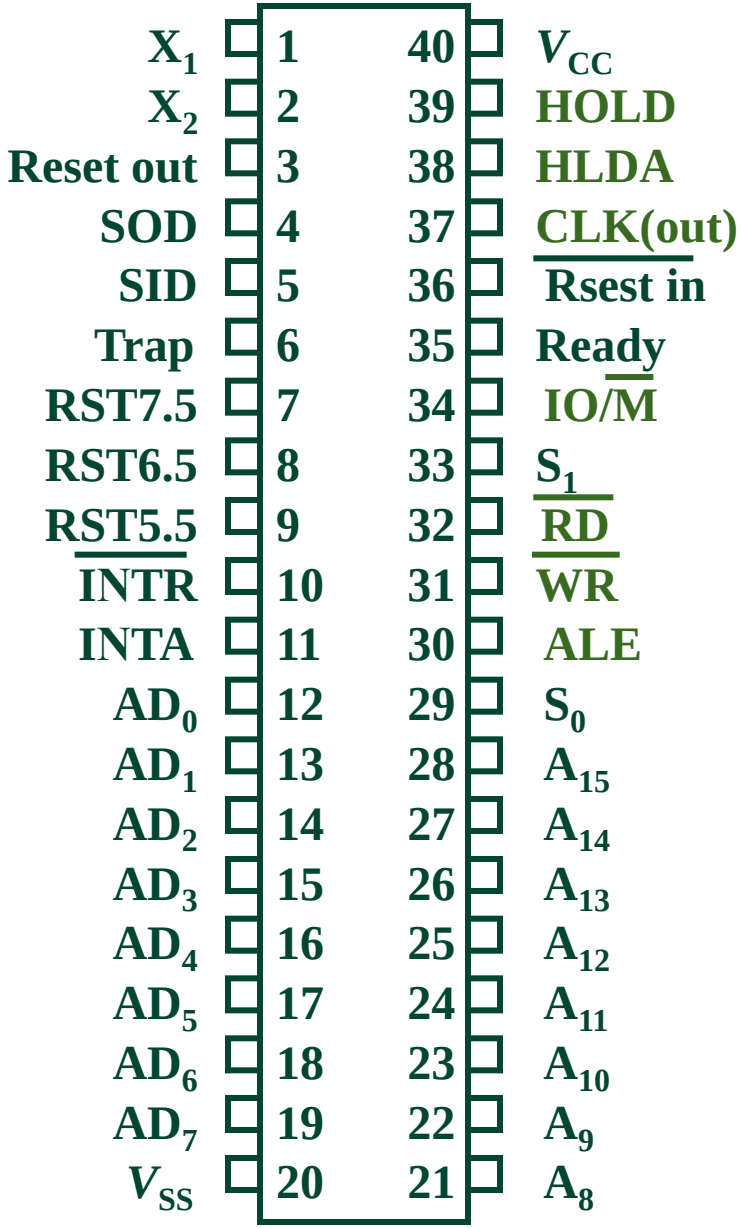
出　CLK　ALE　S_0　S_1

　　$\overline{IO/M}$　$\overline{RD}$　$\overline{WR}$

(3) 存储器和 I/O 初始化

入　HOLD　Ready

出　HLDA



(4) 与中断有关的信号

入　$\overline{\text{INTR}}$

出　INTA

Trap 重新启动中断

(5) CPU 初始化

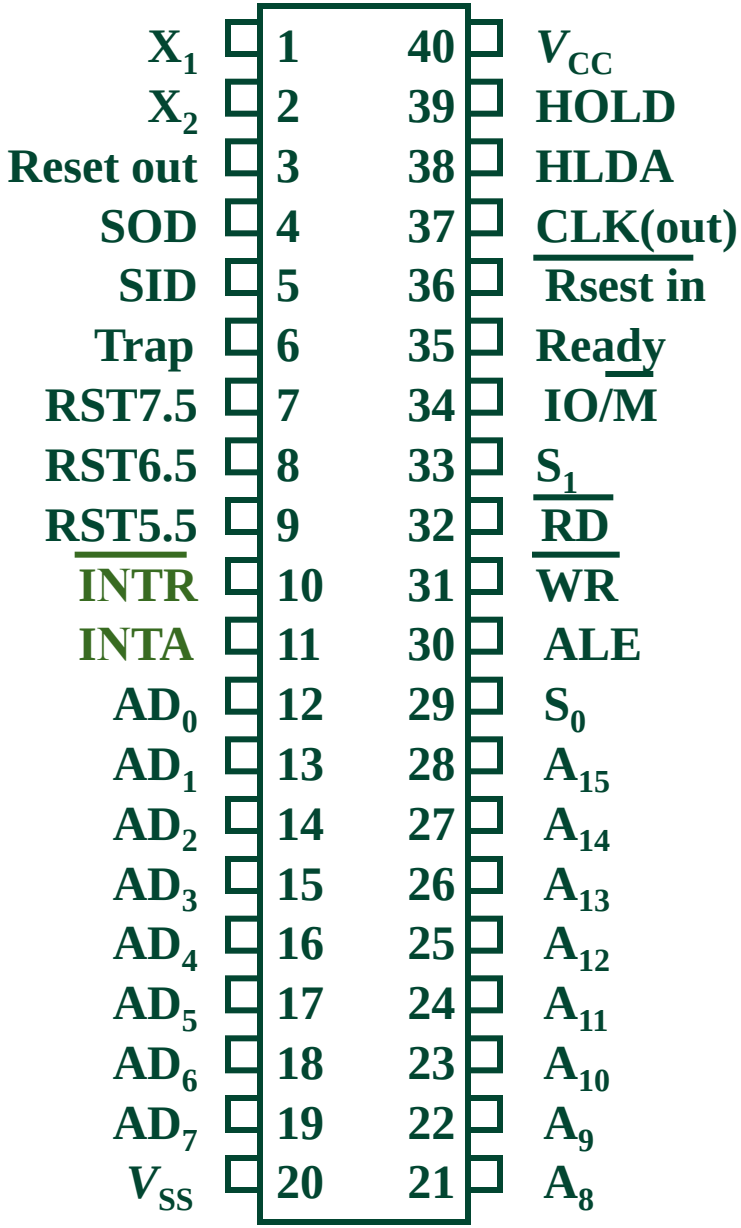
入　$\overline{\text{Reset in}}$

出　Reset out

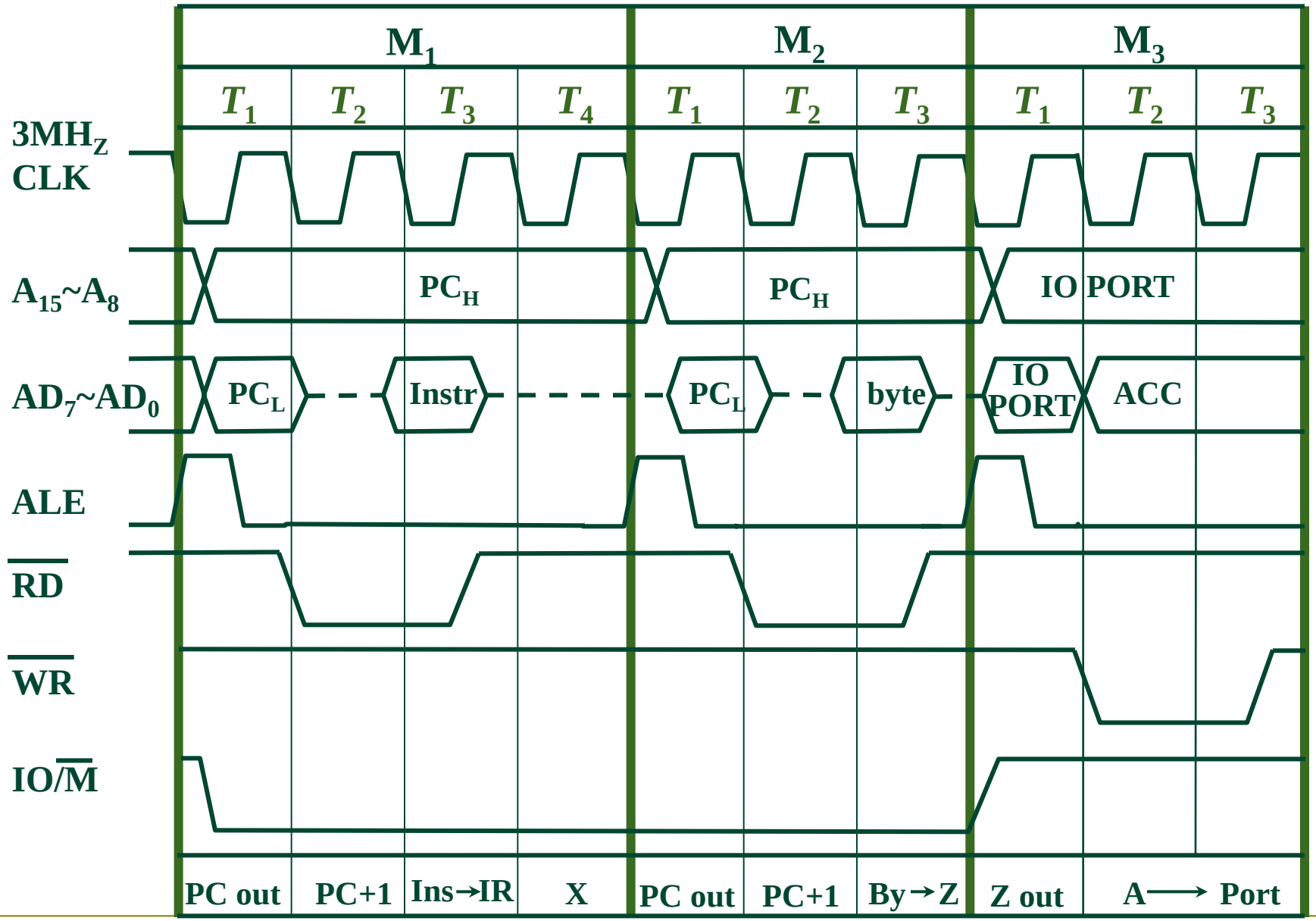
(6) 电源和地

V_{CC}　+5 V

V_{SS}　地



3. 机器周期和节拍（状态）与控制信号的关系



小结

以一条输出指令（I/O 写）为例：

（指令字长16位、数据线8位，分两次取）

机器周期 M_1　取指令操作码

机器周期 M_2　取设备地址

机器周期 M_3　执行 ACC 的内容写入设备

每个 控制 信号在 指定机器周期 的

指定节拍 *T* 时刻 发出

第九章 控制单元的功能（回顾）

- **9.1 微操作命令分析：**四个工作周期的微操作
- **9.2 CU的功能**
  - **外特性：**输入、输出
  - **控制信号举例 《=》 发出微操作命令**
  - **多级时序系统：**机器周期、时钟周期
  - **控制方式：**
    - 同步：定长与不定长时钟周期、中央与局部控制结
    - 异步、联合、人工控制方式

例.设某机平均执行一条指令需要两次访问内存，平均需要3个CPU周期，每个CPU周期平均包含4个节拍周期。若机器主频为240MHz，问：

（1）若主存为“0等待”（即不需要插入等待周期），问执行一条指令的平均时间为多少？

（2）若每次访问内存需要插入2个等待周期，问执行一条指令的平均时间又是多少？

解：因为主频为240MHz，所以节拍周期=（1/240）μs每个

因为每个CPU周期平均包含4个节拍周期，所以：

CPU周期=节拍周期×4＝4/240MHz=(1/60)μs

若访存不需要插入等待周期，则执行一条指令平均需要3个CPU周期，所以：

指令周期＝3×CPU周期＝3×（1/60）μs=(1/20)μs=0.05μs

机器平均速度＝1/0.05μs=20 MIPS

（2）平均执行一条指令需要两次访问内存，每次访问内存需要插入2个等待周期，所以：

指令周期＝0.05μs＋2×(1/240)μs＝(1/20)μs +(1/120)μs

=(7/120)μs

机器平均速度＝120/7≈17MIPS

$$\frac{MIPS_1}{MIPS_2} = \frac{f_1}{f_2}$$

返回

若某机主频为200MHZ，每个指令周期平均为2.5CPU周期，每个CPU周期平均包括2个主频周期，问：

(1)该机平均指令执行速度为多少MIPS？

(2)若主频不变，但每条指令平均包括5个CPU周期，每个CPU周期又包含4个主频周期，平均指令执行速度又为多少MIPS?

由此可得出什么结论？

解：（1）主频为为200MHz，所以主频周期＝1/200MHz=0.005μs

每个指令周期平均为2.5CPU周期，每个CPU周期平均包括2个主频周期，所以一条指令的执行时间＝2.5×2×0.005μs＝0.025μs

该机平均指令执行速度＝1/0.025＝40MIPS。

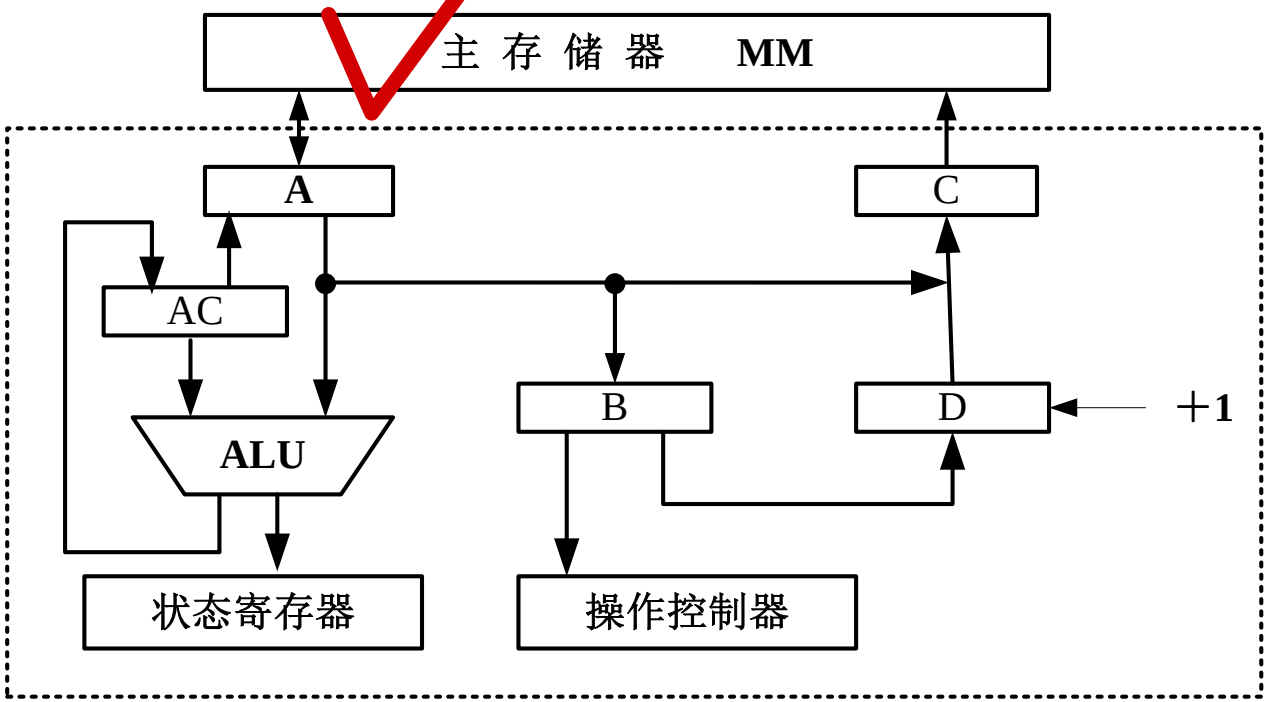
（2）每条指令平均包括5个CPU周期，每个CPU周期又包含4个主频周期，所以一条指令的执行时间＝4×5×0.005μs＝0.1μs

该机平均指令执行速度＝1/0.1＝10MIPS

(3)说明指令的复杂程度会影响指令的平均执行速度。

例. CPU结构如图所示，其中包括一个累加寄存器AC、一个状态寄存器和其他四个寄存器，各部分之间的连线表示数据通路，箭头表示信息传送方向

（1）标明图中四个寄存器的名称



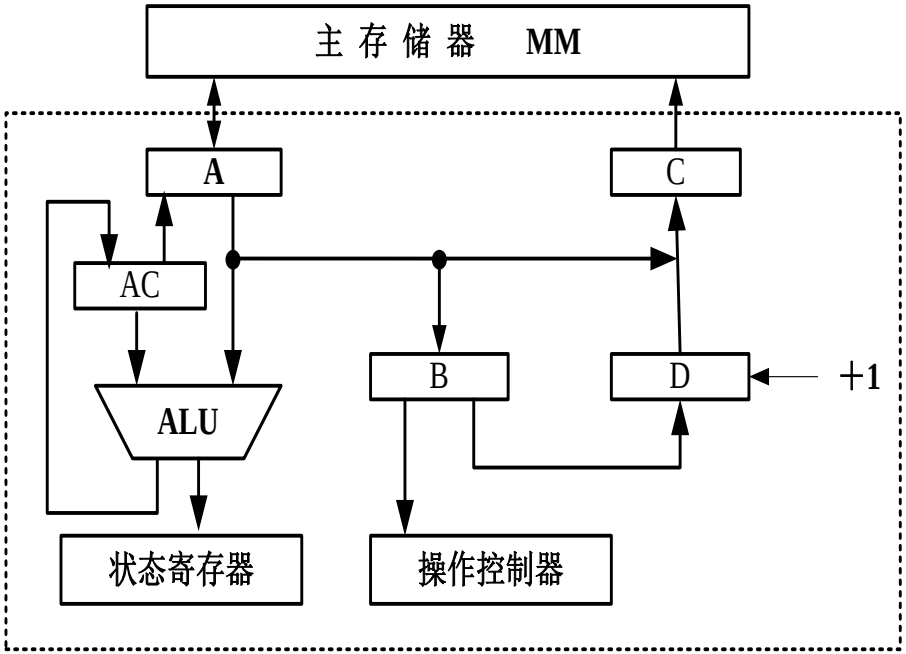
解：A为MDR，B为IR，C为MAR，D为PC

（2）简述取指令的数据通路

（3）简述完成指令LDA X的数据通路（X为内存地址，LDA的功能为（X）→(AC)）

（4）简述完成指令ADDY的数据通路（Y为内存地址，ADD功能为（AC）＋（Y）→（AC））

（5）简述完成指令STA Z的数据通路（Z为内存地址，STA功能为（AC）→(Z)）



解：(2)取指：PC →MAR →MM →MDR →IR

(3) LDA X: X →MAR →MM →MDR →ALU →AC

(4) ADD Y: Y →MAR →MM →MDR →ALU →ADD →AC

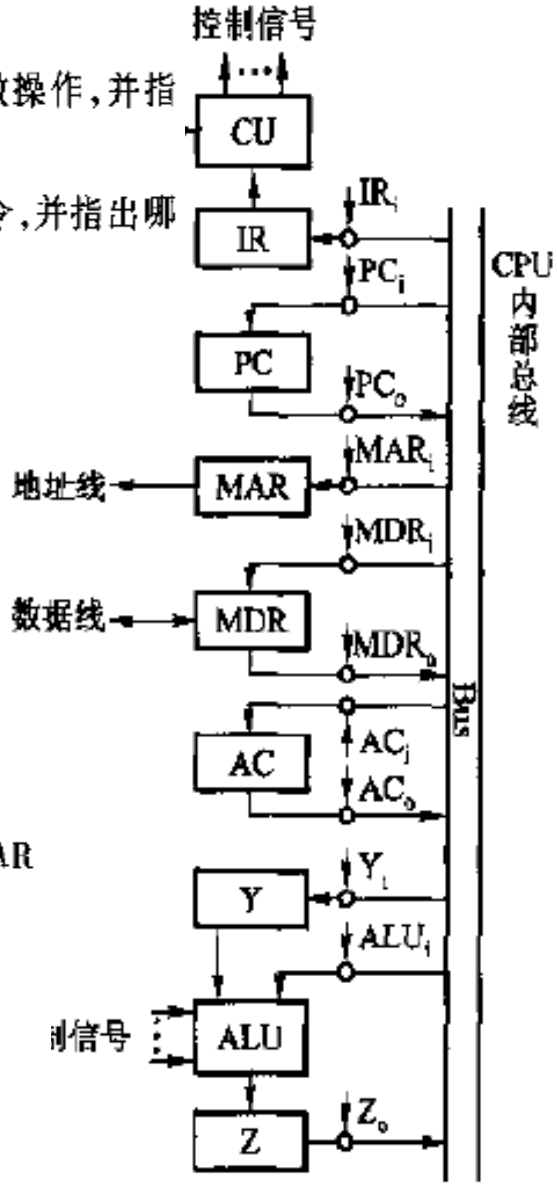
(5) STA Z: Z →MAR , AC →MDR → MM

微操作命令分析

设 CPU 内部采用总线连接方式，如图

（1）写出完成 LDA X，STA X，ADD X（X 均为主存地址）三条指令所需的全部微操作，并指出哪些控制信号有效。

（2）当上述三条指令均为间接寻址时，写出完成这些指令所需的全部微操作命令，并指出哪些控制信号有效。



（1）上述三条指令的取指操作均相同，即

PC→Bus→MAR	PC_o 和 MAR_i 有效，现行指令地址→MAR
1→R	CU 发读命令
数据线→MDR	现行指令从存储器→数据线→MDR
MDR→Bus→IR	MDR_o 和 IR_i 有效，现行指令→IR
(PC) +1→PC	形成下条指令的地址

① LDA X 指令

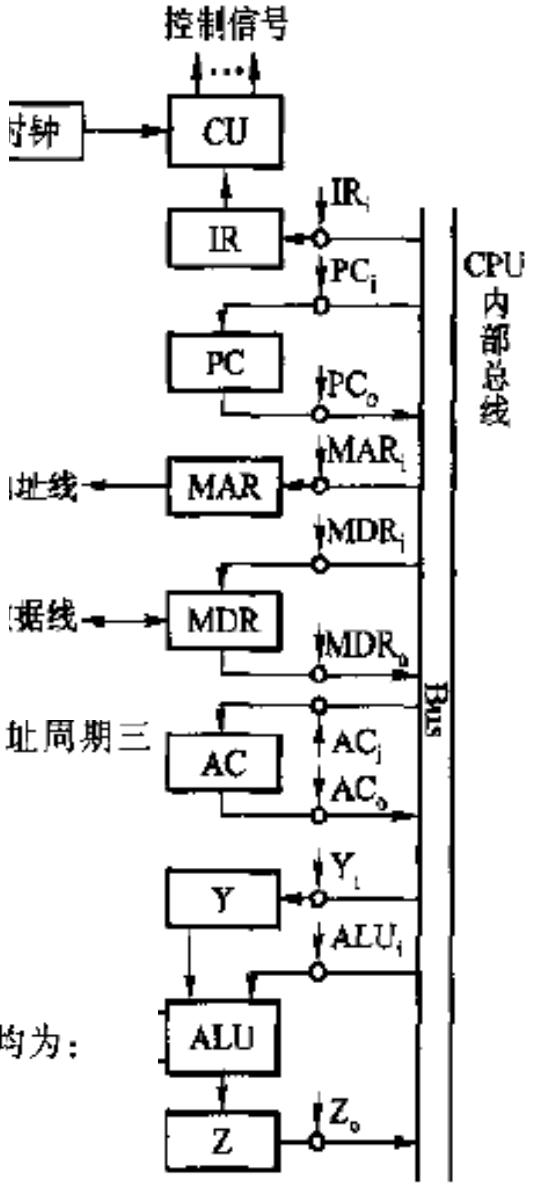
Ad(MDR)→Bus→MAR	MDR_o 和 MAR_i 有效，指令的地址码字段→MAR
1→R	CU 发读命令
数据线→MDR	操作数从存储器→数据线→MDR
MDR→Bus→AC	MDR_o 和 AC_i 有效，操作数→AC

② STA　X 指令

Ad(MDR)→Bus→MAR	MDR_o 和 MAR_i 有效，指令的地址码字段→MAR
1→W	CU 发写命令
AC→Bus→MDR	AC_o 和 MDR_i 有效，欲写入的数据→MDR
MDR→数据线	数据经数据线写入存储器

③ ADD　X 指令

Ad(MDR)→Bus→MAR	MDR_o 和 MAR_i 有效，指令的地址码字段→MAR
1→R	CU 发读命令
数据线→MDR	操作数从存储器→数据线→MDR
MDR→Bus→Y	MDR_o 和 Y_i 有效，操作数→Y
(AC) + (Y) →Z	AC_o 有效，CU 向 ALU 发加命令，结果→Z
Z→AC	Z_o 和 AC_i 有效，结果→AC



(2) 对于间接寻址的取数、存数和加法指令，其取指周期的操作是不变的，进入间址周期三条指令的间址操作均相同，具体的微操作是：

Ad(MDR) →Bus→MAR	MDR_o 和 MAR_i 有效，形式地址→MAR
1→R	CU 发读命令
数据线→MDR	有效地址从存储器→数据线→MDR

间址周期结束时有效地址在 MDR 中，进入执行周期后，三条指令的第一个微操作均为：

MDR→Bus→MAR	MDR_o 和 MAR_i 有效，有效地址→MAR

其余的微操作不变。

指令、机器和时钟周期

设某机主频为 8 MHz，每个机器周期平均含 2 个时钟周期，每条指令平均有 2.5 个机器周期，试问该机的平均指令执行速度为多少 MIPS？若机器主频不变，但每个机器周期平均含 4 个时钟周期，每条指令平均有 5 个机器周期，则该机的平均指令执行速度又是多少 MIPS？由此可得出什么结论？

【解】 根据主频为 8 MHz，得时钟周期为 1/8 = 0.125 μs，机器周期为 0.125 × 2 = 0.25 μs，指令周期为 0.25 × 2.5 = 0.625 μs。

（1）平均指令执行速度为 1/0.625 = 1.6 MIPS。

（2）若机器主频不变，机器周期含 4 个时钟周期，每条指令平均含 5 个机器周期，则指令周期为 0.125 × 4 × 5 = 2.5 μs，故平均指令执行速度为 1/2.5 = 0.4 MIPS。

（3）可见机器的速度并不完全取决于主频。

Computer Organization

Thank You !

Computer Architecture Research Institute · Computer Organization

第10章　控制单元的设计

系统结构研究所· 计算机组成原理

第10章　控制单元的设计

10.1　组合逻辑设计

10.2　微程序设计

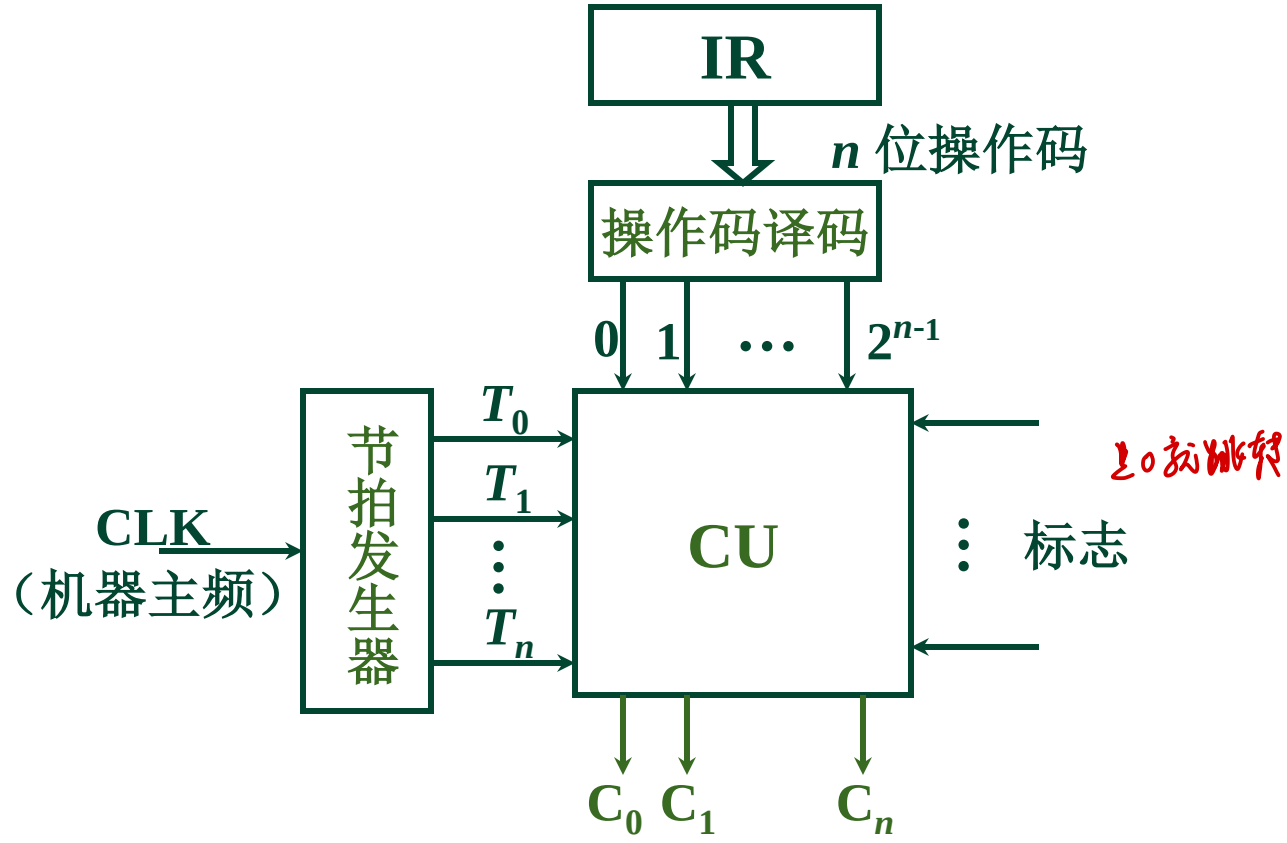
控制器的类型

- 组合逻辑型：核心是微操作产生部件，用组合逻辑设计思想，以布尔代数为工具设计。输入信号来自指令译码器的输出、时序发生器的时序信号和程序运行结果特征及状态，输出为带有时间标志的微操作控制信号。

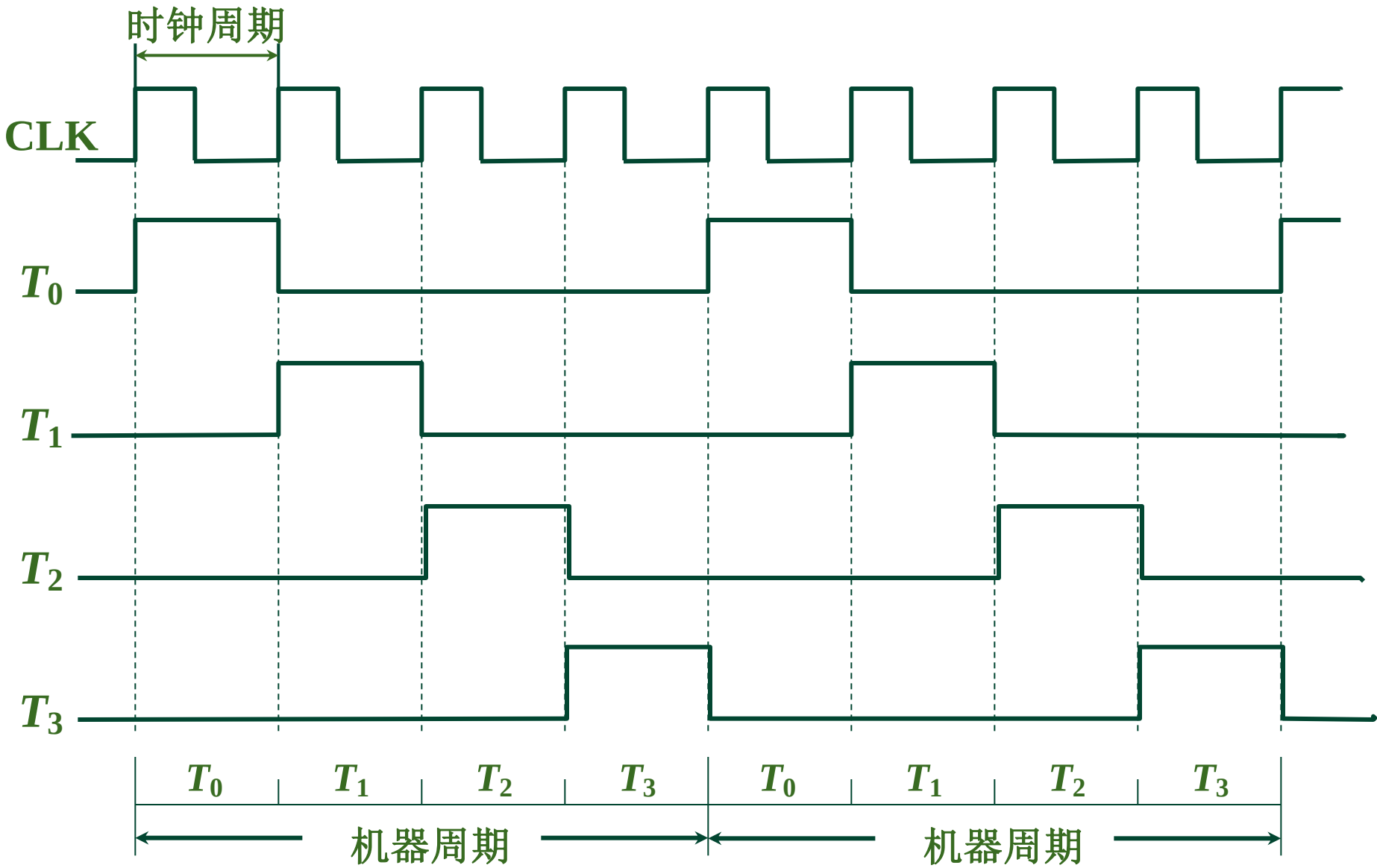
- 微程序控制型：将机器指令分解为基本微命令序列，用二进制码表示微命令，并编成微指令，多条微指令形成微程序。每种机器指令对应一段微程序，在制造CPU时固化在CPU中的一个控制存储器中（CS或CM）中。执行一条机器指令时，CPU依次从CS中取微指令，从而产生微命令。

一、组合逻辑控制单元框图

1. CU 外特性



2. 节拍信号



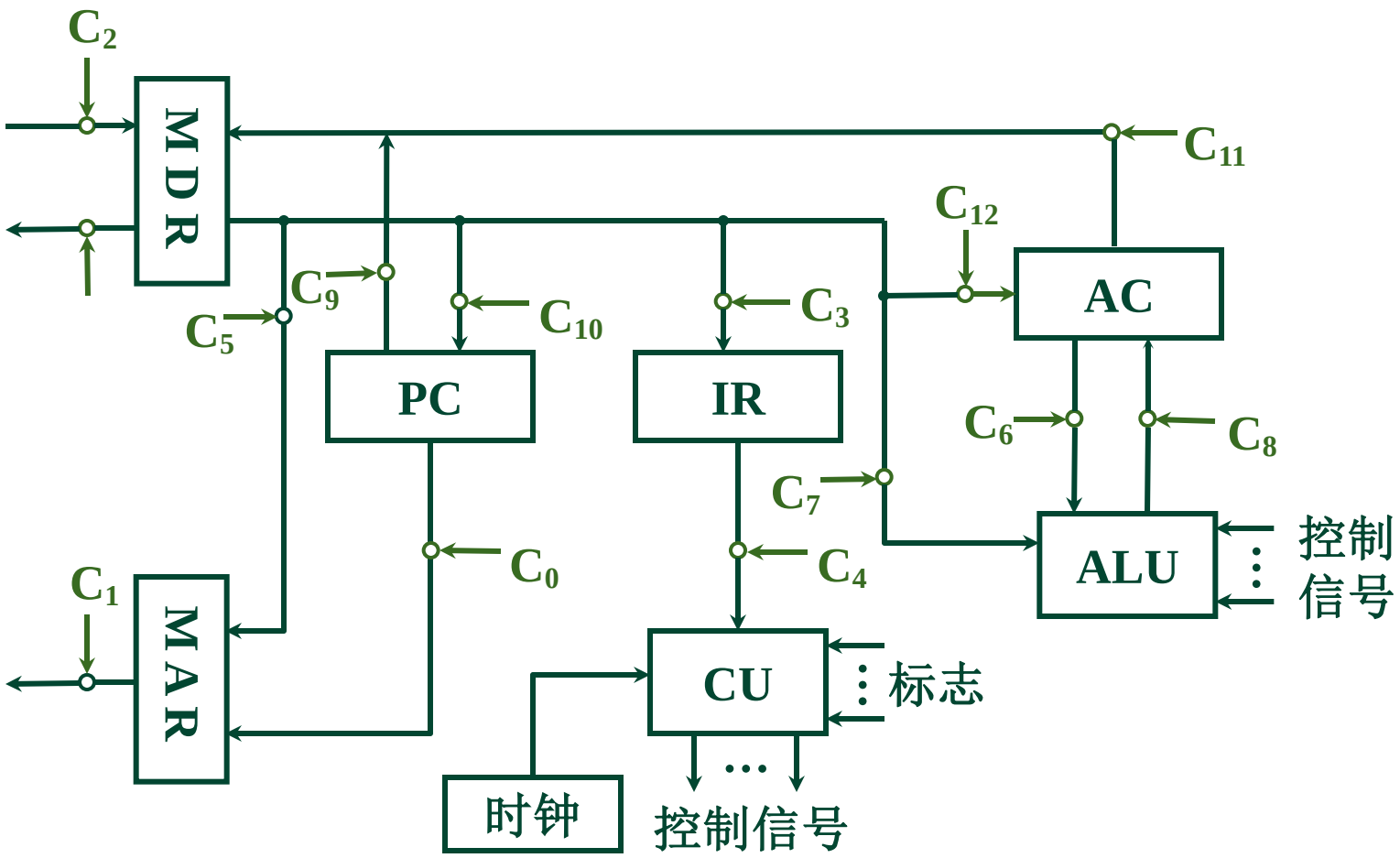
二、微操作的节拍安排

10.1

假设：采用 同步控制方式

一个 机器周期 内有 3个节拍（时钟周期）

CPU 内部结构采用非总线方式



1. 安排微操作时序的原则

原则一　微操作的 先后顺序不得 随意 更改

原则二　被控对象不同 的微操作　　并行

尽量安排在 一个节拍 内完成

原则三　占用 时间较短 的微操作

尽量 安排在 一个节拍 内完成

并允许有先后顺序

2. 取指周期 微操作的 节拍安排

T_0 PC ⟶ MAR

1 ⟶ R

T_1 M (MAR) ⟶ MDR

(PC) + 1 ⟶ PC

T_2 MDR ⟶ IR

OP (IR) ⟶ ID

原则二

原则二

有先后次序

从源及寄存器操作速度快

原则三

3. 间址周期 微操作的 节拍安排

T_0 Ad (IR) ⟶ MAR

1 ⟶ R

T_1 M (MAR) ⟶ MDR

T_2 MDR ⟶ Ad (IR)

4. 执行周期 微操作的 节拍安排

① CLA

T_0

T_1 } 啥都不干

T_2 $0 \longrightarrow AC$

② COM

T_0

T_1 在哪个节拍无所谓

T_2 $\overline{AC} \longrightarrow AC$

③ SHR

T_0

T_1

T_2 $L(AC) \longrightarrow R(AC)$

$AC_0 \longrightarrow AC_0$

④ CSL	T_0		
	T_1		
	T_2	R (AC) ⟶ L (AC)	$AC_0 \longrightarrow AC_n$
⑤ STP	T_0		
	T_1		
	T_2	0 ⟶ G	
⑥ ADD X	T_0	Ad (IR) ⟶ MAR	1 ⟶ R
	T_1	M (MAR) ⟶ MDR	
	T_2	(AC) + (MDR) ⟶ AC	
⑦ STA X	T_0	Ad (IR) ⟶ MAR	1 ⟶ W
	T_1	AC ⟶ MDR	
	T_2	MDR ⟶ M (MAR)	

398 - 400
10.1 - 10.3

⑧ LDA X

T_0 Ad (IR) ⟶ MAR 1 ⟶ R

T_1 M (MAR) ⟶ MDR

T_2 MDR ⟶ AC

⑨ JMP X

T_0

T_1

T_2 Ad (IR) ⟶ PC

⑩ BAN X

T_0

T_1

T_2 $A_0 \cdot Ad(IR) + \overline{A}_0 \cdot PC \longrightarrow PC$

5. 中断周期 微操作的 节拍安排

T_0　0 ⟶ MAR　　1 ⟶ W　　硬件关中断

T_1　PC ⟶ MDR

T_2　MDR ⟶ M (MAR)　　向量地址 ⟶ PC

中断隐指令完成

三、组合逻辑设计步骤

- 确定指令功能和指令格式。
- 确定机器周期、节拍和时钟周期，确定机器周期是固定的还是可变长的。
- 根据指令功能和CPU的结构图，进行微操作命令分析
- 对指令微操作进行节拍安排，确定每条指令所需的机器周期及在各机器周期需完成的操作，排出操作时间表。
- 根据操作时间表写出微操作的逻辑表达式，即

  微操作＝周期·节拍·指令码·其他条件
- 根据微操作的表达式，画出组合逻辑电路

三、组合逻辑设计步骤

1. 列出操作时间表

工作周期标记	节拍	状态条件	微操作命令信号	CLA	COM	ADD	STA	LDA	JMP
FE 取指	T_0		PC⟶ MAR						
			1⟶ R						
	T_1		M(MAR) ⟶MDR						
			(PC) +1 ⟶PC						
	T_2		MDR⟶ IR						
			OP(IR) ⟶ID						
		I	1⟶ IND						
		$\bar{I}$	1⟶ EX						

间址特征

三、组合逻辑设计步骤

1. 列出操作时间表

工作周期标记	节拍	状态条件	微操作命令信号	CLA	COM	ADD	STA	LDA	JMP
IND 间址	T_0		Ad (IR) ⟶ MAR						
			1⟶ R						
	T_1		M(MAR) ⟶MDR						
	T_2		MDR⟶ Ad (IR)						
		$\overline{\mathbf{IND}}$	1⟶ EX						

间址周期标志

三、组合逻辑设计步骤

10.1

1. 列出操作时间表

工作周期标记	节拍	状态条件	微操作命令信号	CLA	COM	ADD	STA	LDA	JMP
EX 执行	T_0		Ad (IR) ⟶ MAR						
			1 ⟶ R						
			1 ⟶ W						
	T_1		M(MAR) ⟶ MDR						
			AC ⟶ MDR						
	T_2		(AC)+(MDR) →AC						
			MDR ⟶ M(MAR)						
			MDR ⟶ AC						
			0 ⟶ AC						

三、组合逻辑设计步骤

1. 列出操作时间表

工作周期标记	节拍	状态条件	微操作命令信号	CLA	COM	ADD	STA	LDA	JMP
FE 取指	T_0		PC⟶ MAR	1	1	1	1	1	1
			1⟶ R	1	1	1	1	1	1
	T_1		M(MAR) ⟶MDR	1	1	1	1	1	1
			(PC) +1⟶ PC	1	1	1	1	1	1
	T_2		MDR⟶ IR	1	1	1	1	1	1
			OP(IR)⟶ ID	1	1	1	1	1	1
		I	1⟶ IND			1	1	1	1
		$\bar{I}$	1⟶ EX	1	1	1	1	1	1

三、组合逻辑设计步骤

1. 列出操作时间表

工作周期标记	节拍	状态条件	微操作命令信号	CLA	COM	ADD	STA	LDA	JMP
IND 间址	T_0		Ad (IR) ⟶ MAR			1	1	1	1
			1⟶ R			1	1	1	1
	T_1		M(MAR) ⟶ MDR			1	1	1	1
	T_2		MDR⟶ Ad (IR)			1	1	1	1
		$\overline{\mathbf{IND}}$	1⟶ EX			1	1	1	1

三、组合逻辑设计步骤

1. 列出操作时间表

工作周期标记	节拍	状态条件	微操作命令信号	CLA	COM	ADD	STA	LDA	JMP
EX 执行	T_0		Ad (IR) ⟶ MAR			1	1	1	
			1⟶ R			1		1	
			1 ⟶ W				1		
	T_1		M(MAR) ⟶ MDR			1		1	
			AC⟶ MDR				1		
	T_2		(AC)+(MDR) ⟶AC			1			
			MDR⟶ M(MAR)				1		
			MDR ⟶ AC					1	
			0⟶ AC	1					

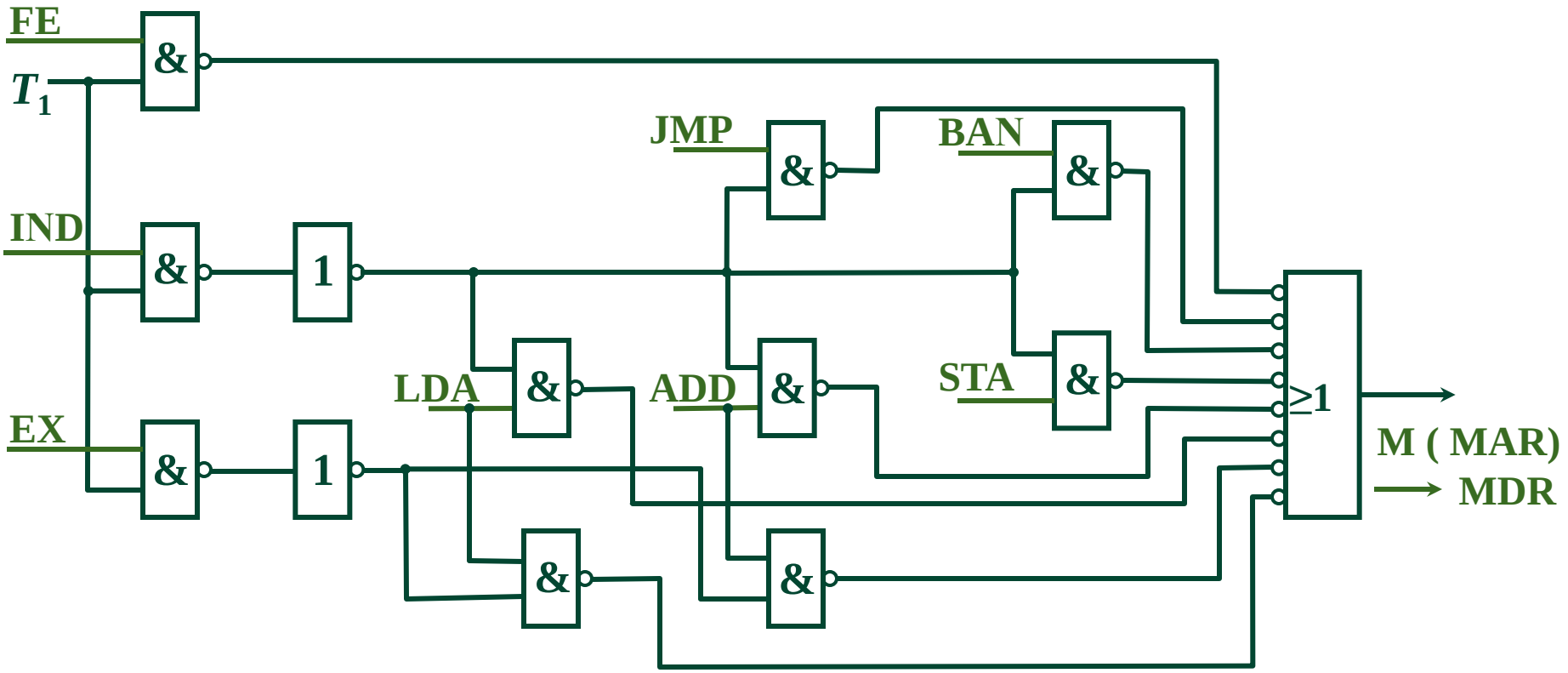
2. 写出微操作命令的最简表达式

$$M(MAR) \longrightarrow MDR$$

$$= FE \cdot T_1 + IND \cdot T_1 (ADD + STA + LDA + JMP + BAN) + EX \cdot T_1 (ADD + LDA)$$

$$= T_1\{FE + IND (ADD + STA + LDA + JMP + BAN) + EX (ADD + LDA)\}$$

3. 画出逻辑图



特点

- 思路清晰，简单明了
- 庞杂，调试困难，修改困难
- 速度快 （RISC）

10.2 微程序设计——基本概念

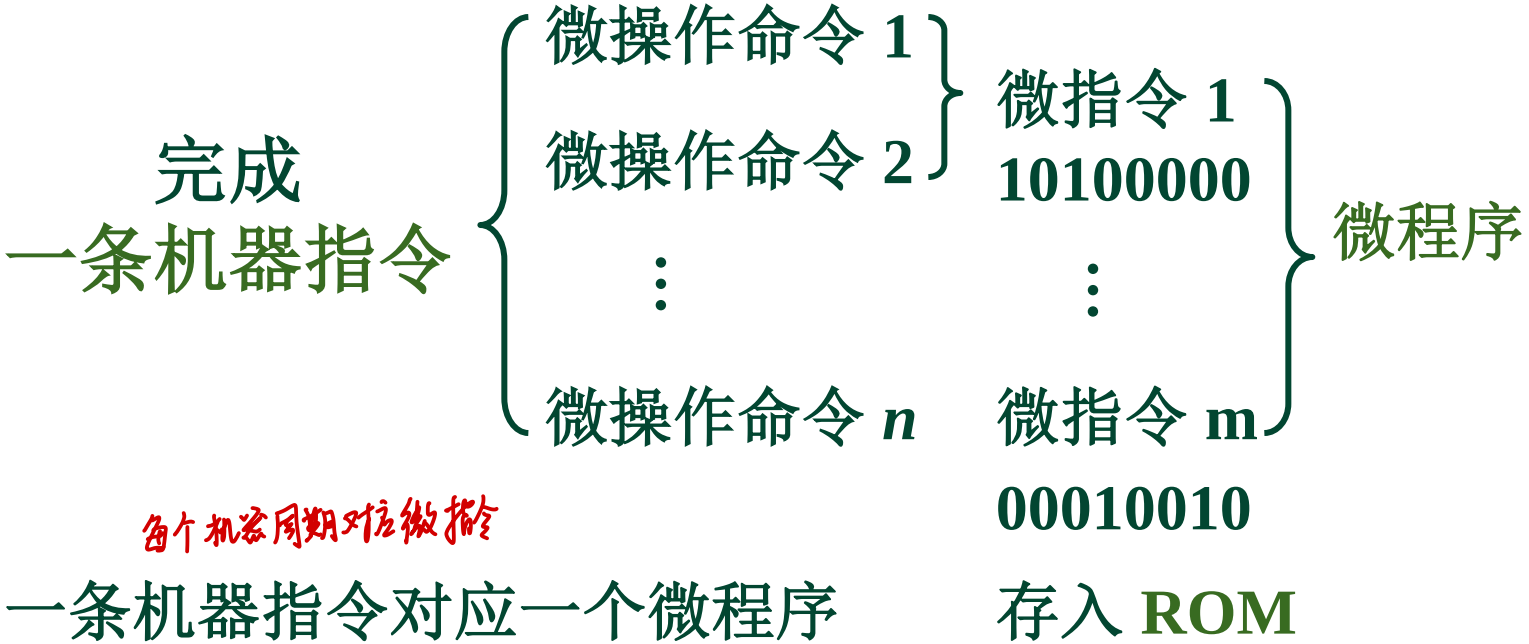
- 微命令：微程序控制计算机中的微操作控制信号。
- 微操作：执行部件接受微指令后所进行的操作，是指令序列中最基本、不可分割的动作。
  - 例如，一条加法指令要分成四步完成：取指令，计算地址，取数，加法运算，每一步实现需若干个微操作
- 微指令：在微程序控制的计算机中，同时发出的控制信号所执行的一组微操作。
  - 是在机器的一个节拍中，一组实现一定操作功能的微命令
  - 将一条指令分解成若干条微指令，按次序执行微指令，即可实现指令的功能
- 微程序：由微指令组成的序列称为微程序。一个微程序的功能对应一条机器指令的功能。

10.2 微程序设计—— 基本概念

- 控制存储器：微程序存放于存储器中，由于该存储器主要存放控制命令（信号）和下一条执行的微指令地址（简称下址），因此被称为控制存储器。
  - 执行一条指令就是执行一段存放在控制存储器中的微程序。
  - 用ROM实现，因为一台计算机指令系统是固定的，所以微程序是固定的，所以控制存储器可以用ROM实现
- 微周期：执行一条微指令和取出下一条微指令所需时间
  - 通常一个微周期与一个CPU周期时间相等
- 相斥性微命令：不能在一个微周期出现的微命令
  - 如读命令和写命令
- 相容性微命令：能在一个微周期出现的微命令。

一、微程序设计思想的产生

1951 英国剑桥大学教授 Wilkes



存储逻辑

微指令的格式

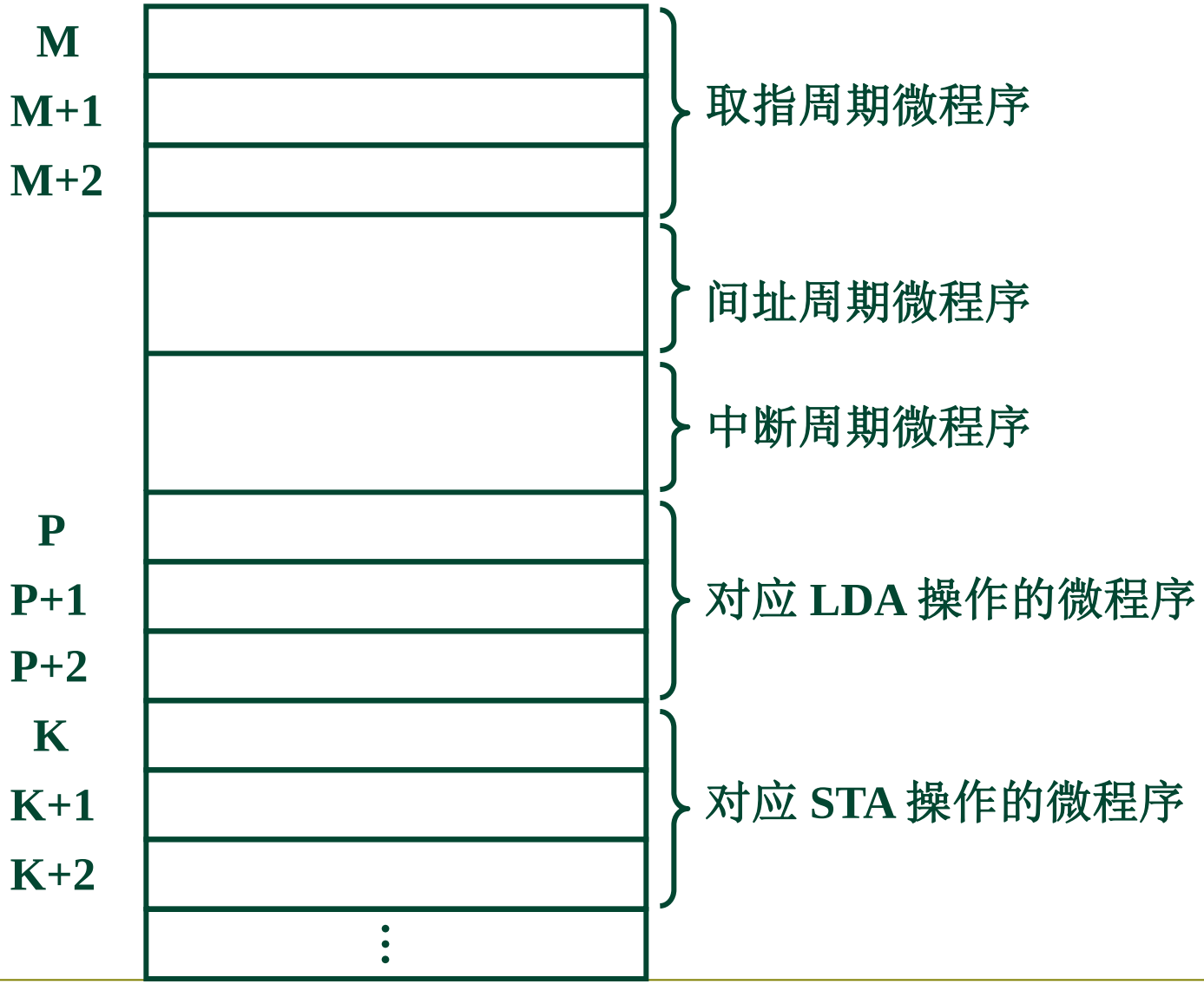
微指令：

控制字段	下址字段

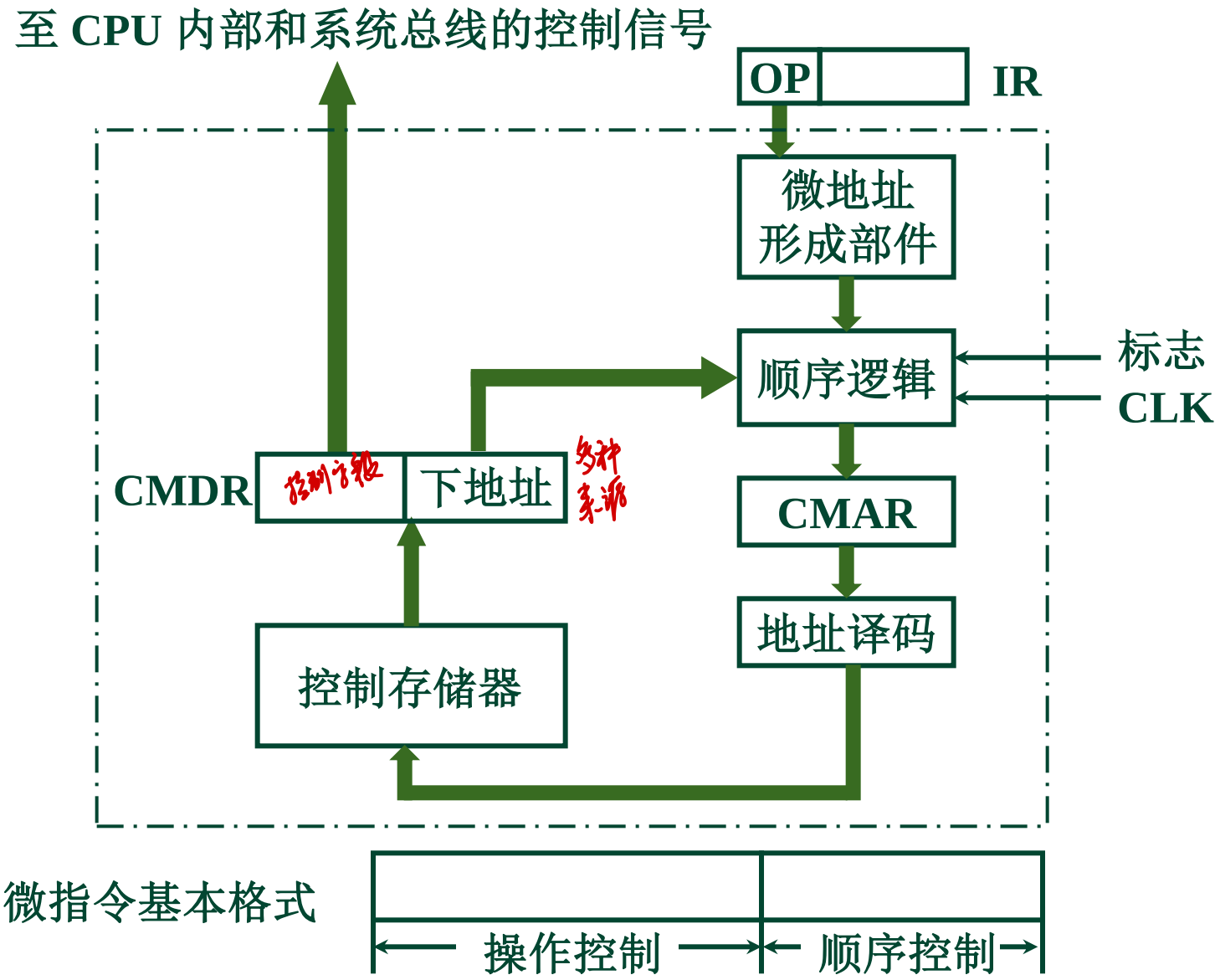
- 控制字段：操作控制，发出各种控制信号
- 下址字段：顺序控制，指出下条微指令地址，以控制微指令序列的执行顺序

二、微程序控制单元框图及工作原理

1. 机器指令对应的微程序

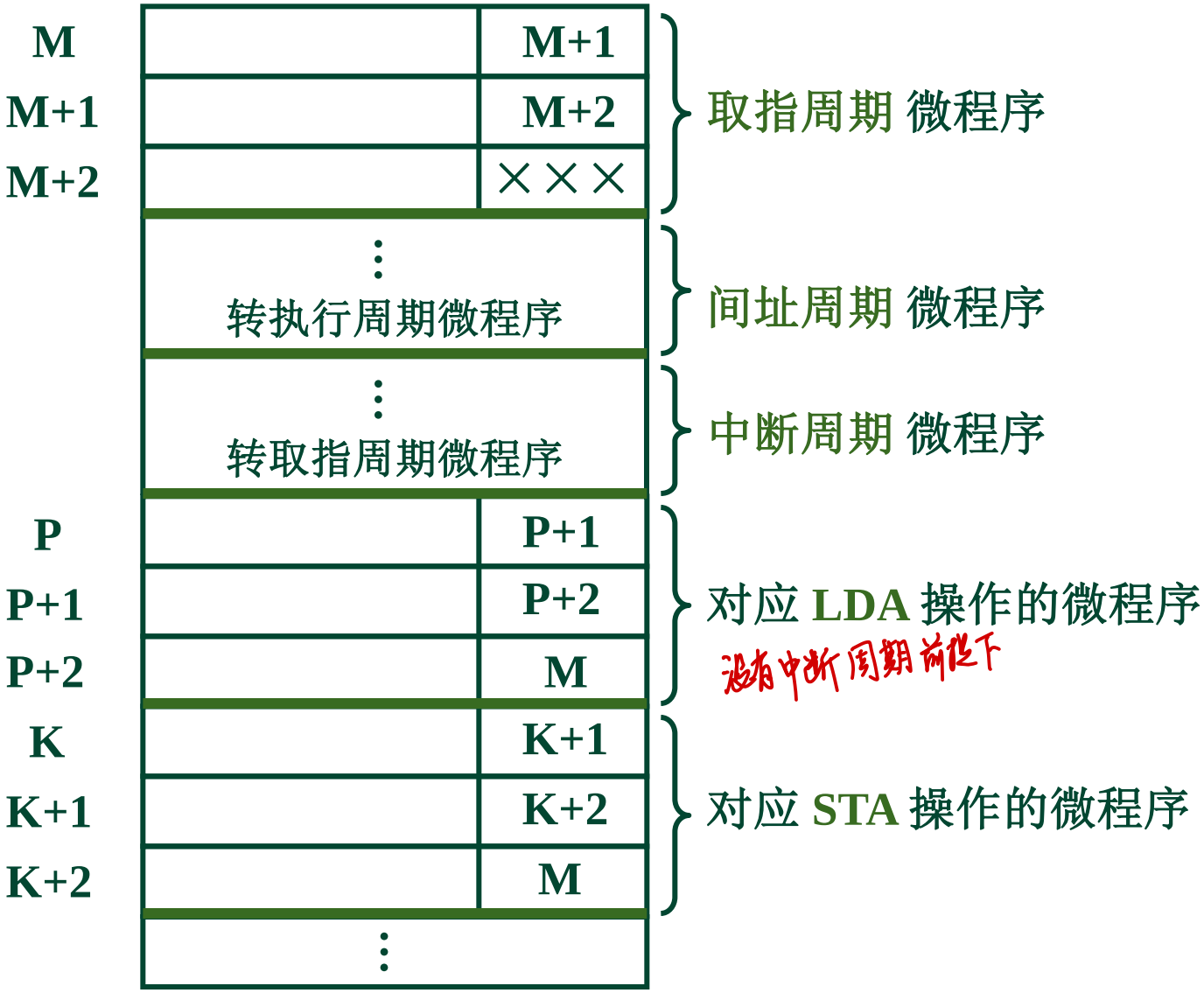


2. 微程序控制单元的基本框图

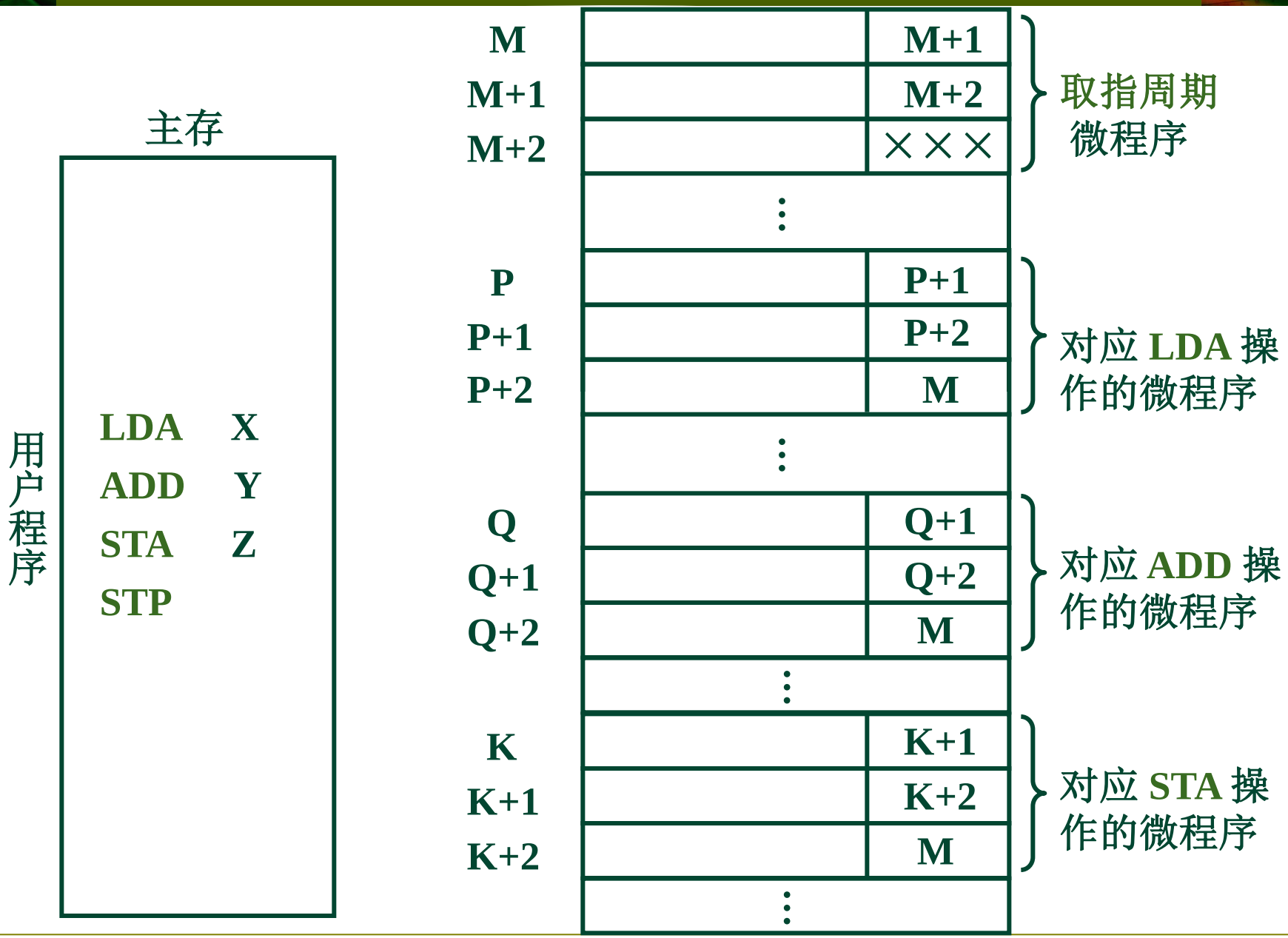


二、微程序控制单元框图及工作原理

10.2



3. 工作原理


控存
主存
用户程序
LDA X
ADD Y
STA Z
STP
M
M+1
M+2
P
P+1
P+2
Q
Q+1
Q+2
K
K+1
K+2
M+1
M+2
×××
P+1
P+2
M
Q+1
Q+2
M
K+1
K+2
M
取指周期微程序
对应 LDA 操作的微程序
对应 ADD 操作的微程序
对应 STA 操作的微程序

3. 工作原理

(1) 取指阶段　　执行取指微程序

M ⟶ CMAR　　一个节拍，一条微指令

CM (CMAR) ⟶ CMDR

由 CMDR 发命令

形成下条微指令地址　M + 1

Ad (CMDR) ⟶ CMAR

CM (CMAR) ⟶ CMDR　　取指令

由 CMDR 发命令

形成下条微指令地址　M + 2

Ad (CMDR) ⟶ CMAR

CM (CMAR) ⟶ CMDR

由 CMDR 发命令

(2) 执行阶段　执行 LDA 微程序

OP (IR) ⟶ 微地址形成部件 ⟶ CMAR　　(P ⟶ CMAR)

CM (CMAR) ⟶ CMDR

由 CMDR 发命令

形成下条微指令地址　P + 1

Ad (IR) ⟶ MAR　　1 ⟶ R

Ad (CMDR) ⟶ CMAR

CM (CMAR) ⟶ CMDR

由 CMDR 发命令

形成下条微指令地址　CMAR

M (MAR) ⟶ MDR

Ad (CMDR) ⟶ CMAR

CM (CMAR) ⟶ CMDR

由 CMDR 发命令

形成下条微指令地址　M

MDR ⟶ AC

Ad (CMDR) ⟶ CMAR　　(M ⟶ CMAR)

(3) 取指阶段　执行取指微程序

M ⟶ CMAR

CM (CMAR) ⟶ CMDR

由 CMDR 发命令

⋮

全部微指令存在 CM 中，程序执行过程中 只需读出

关键

- 微指令的 操作控制字段如何形成微操作命令
- 微指令的 后续地址如何形成

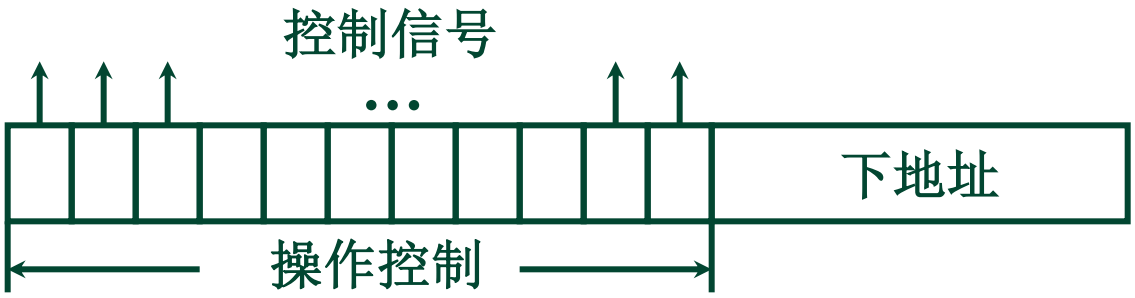
三、微指令的编码方式（控制方式）

1. 直接编码（直接控制）方式

长位数

在微指令的操作控制字段中，

每一位代表一个微操作命令



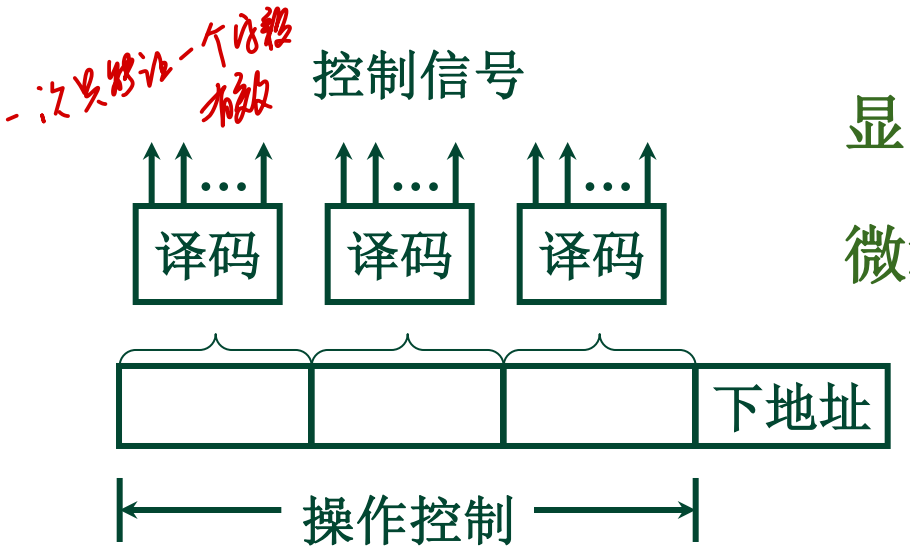
速度最快

某位为“1”表示该控制信号有效

2. 字段直接编码方式

将微指令的控制字段分成若干 “段”，

每段经译码后发出控制信号



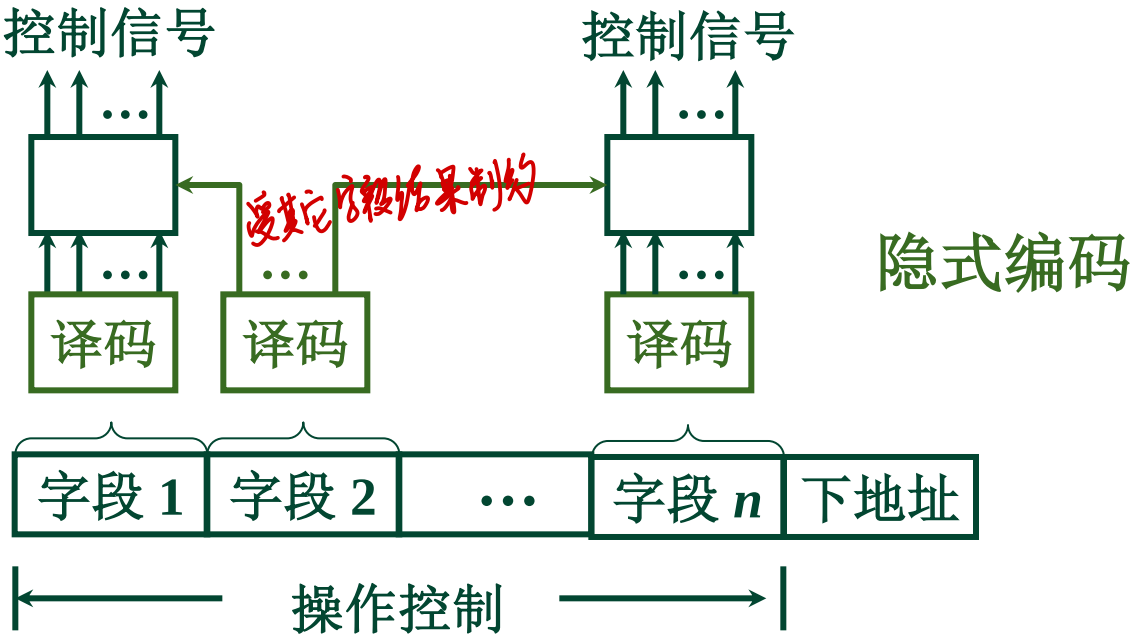
显式编码

微程序执行速度较慢

每个字段中的命令是 互斥 的

缩短 了微指令 字长，增加 了译码 时间

3. 字段间接编码方式



4. 混合编码

直接编码和字段编码（直接和间接）混合使用

5. 其他

四、微指令序列地址的形成

10.2

1. 微指令的 下地址字段 指出(断定方式)

2. 根据机器指令的 操作码 形成：根据机器指令的操作码，由微地址形成部件形成对应该机器指令微程序的首地址。

3. 增量计数器（顺序地址）

$$(CMAR) + 1 \longrightarrow CMAR$$

4. 分支转移（转移指令）

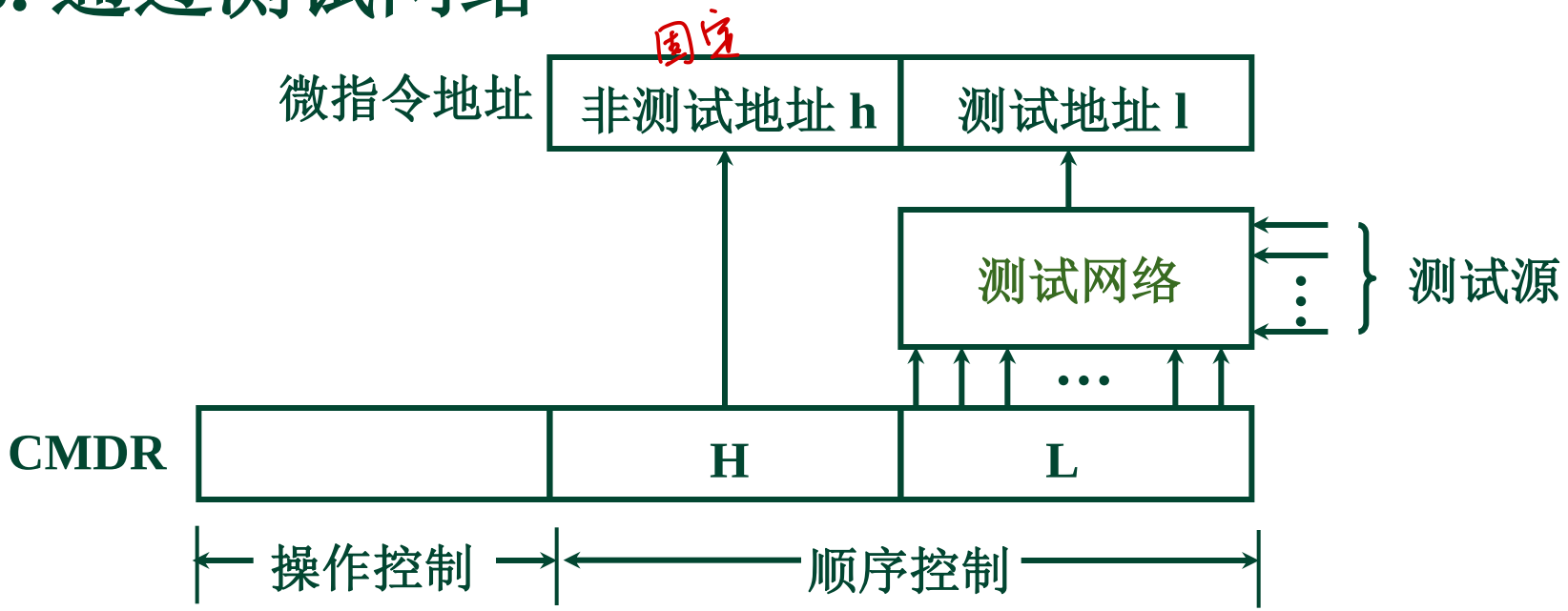
操作控制字段	转移方式	转移地址

转移方式　　指明判别条件

转移地址　　指明转移成功后的去向

四、微指令序列地址的形成

5. 通过测试网络



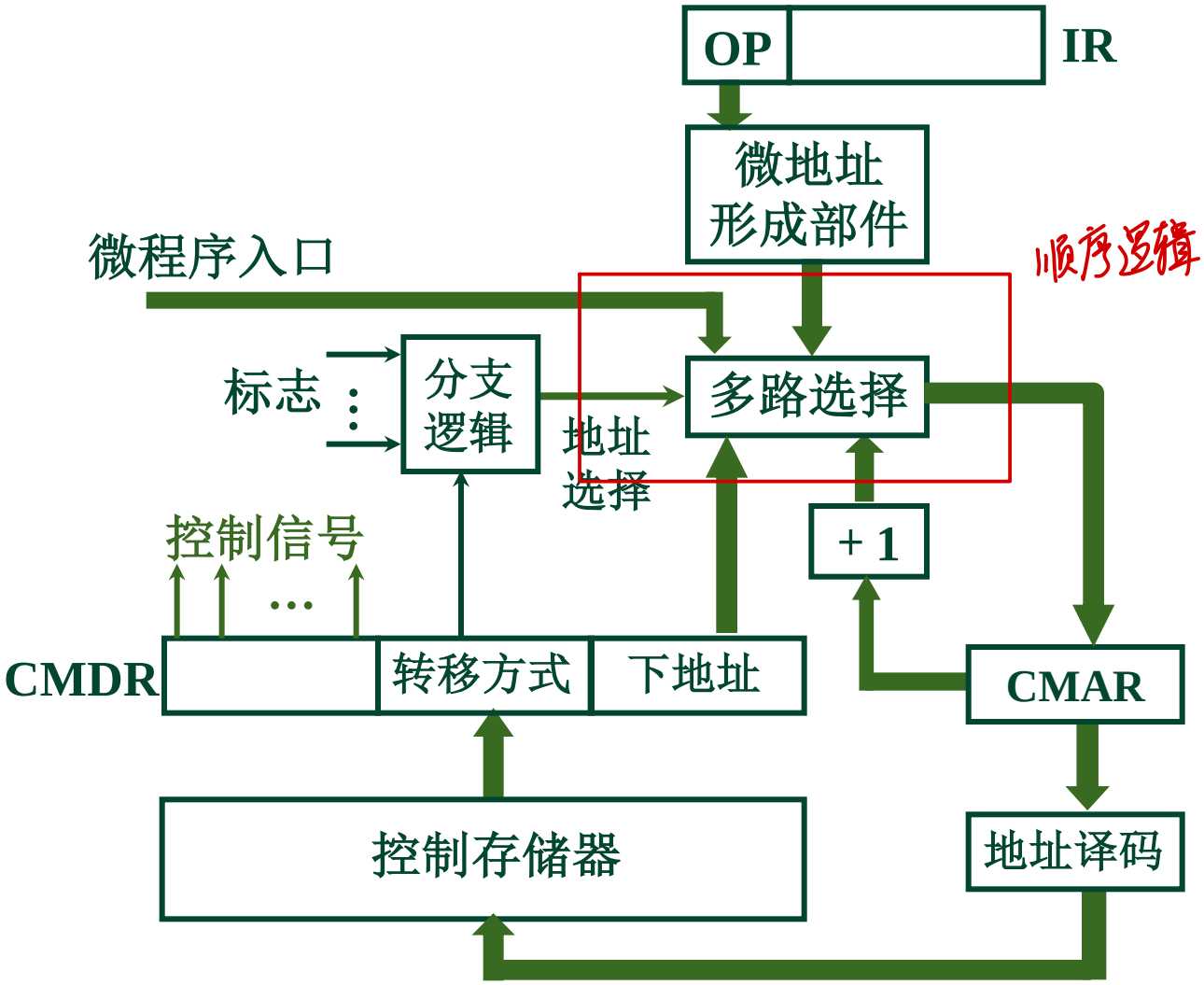
6. 由硬件产生微程序入口地址

加电后，第一条微指令地址　　由专门 硬件 产生

中断周期　　由 硬件 产生 中断周期微程序首地址

四、微指令序列地址的形成

7. 后续微指令地址形成方式原理图



五、微指令格式

1. 水平型微指令

一次能定义并执行多个并行操作

如　直接编码、字段直接编码、字段间接编码、直接和字段混合编码

2. 垂直型微指令

类似机器指令操作码 的方式

由微操作码字段规定微指令的功能

3. 两种微指令格式的比较

(1) 水平型微指令比垂直型微指令 并行操作能力强，灵活性强

(2) 水平型微指令执行一条机器指令所要的微指令 数目少，速度快

(3) 水平型微指令 用较短的微程序结构换取较长的微指令结构

(4) 水平型微指令与机器指令 差别大

六、静态微程序设计和动态微程序设计

静态　微程序无须改变，采用 **ROM**

动态　通过 改变微指令 和 微程序 改变机器指令，
有利于仿真，采用 **EPROM**

七、毫微程序设计

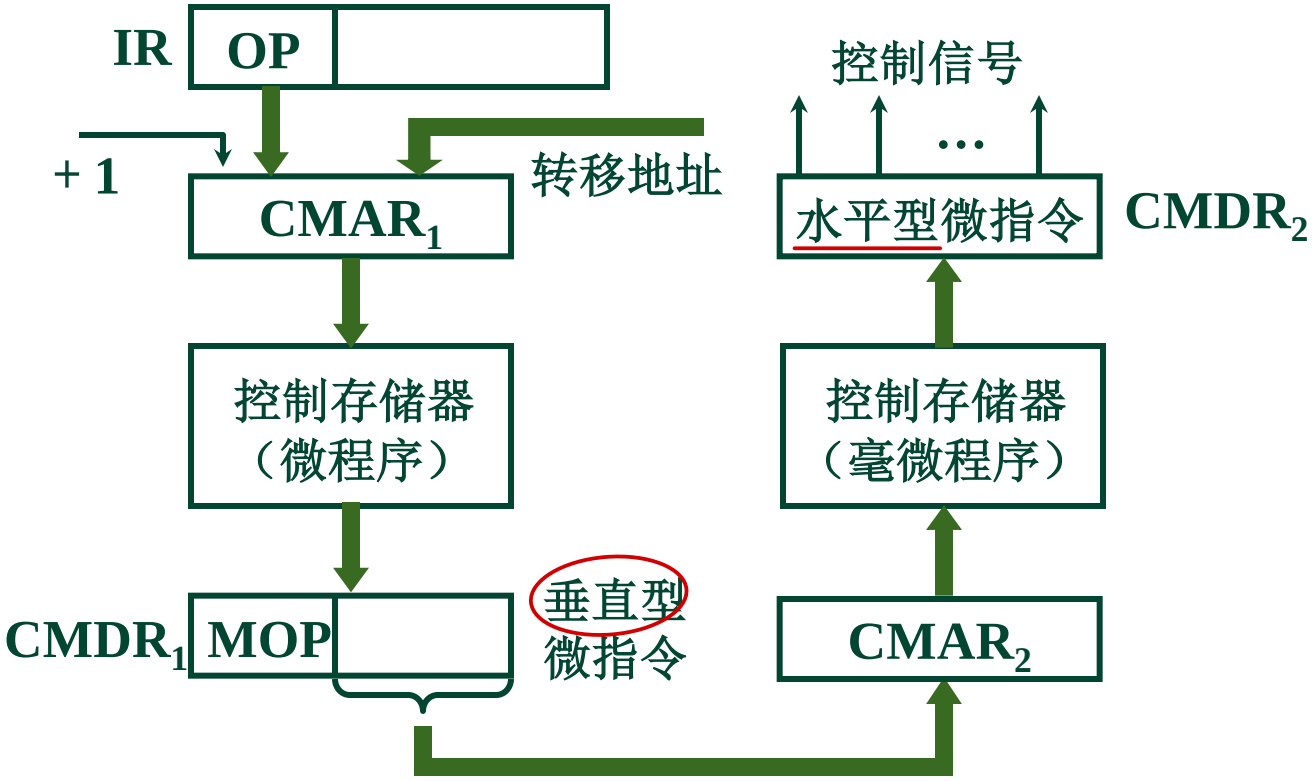
1. 毫微程序设计的基本概念

微程序设计　　用 微程序解释机器指令

毫微程序设计　用 毫微程序解释微程序

毫微指令与微指令 的关系好比 微指令与机器指令 的关系

2. 毫微程序控制存储器的基本组成



八、串行微程序控制和并行微程序控制

串行 微程序控制

取第 *i* 条微指令	执行第 *i* 条微指令	取第 *i*+1 条微指令	执行第 *i*+1 条微指令

并行 微程序控制

取第 *i* 条微指令	执行第 *i* 条微指令		
	取第 *i*+1 条微指令	执行第 *i*+1 条微指令	
		取第 *i*+2 条微指令	执行第 *i*+2 条微指令

九、微程序设计举例

10.2

1. 写出对应机器指令的微操作及节拍安排

假设 CPU 结构与组合逻辑相同

(1) 取指阶段微操作分析　　3 条微指令

T_0　PC ⟶ MAR　　1 ⟶ R

T_1　M (MAR) ⟶ MDR　　(PC) + 1 ⟶ PC

T_2　MDR ⟶ IR　　OP (IR) ⟶ 微地址形成部件

若需考虑 T_2 如何安排这条微指令？

则取指操作需 3 条微指令

① Ad (CMDR) ⟶ CMAR　下地址段

② OP (IR) ⟶ 微地址形成部件 ⟶ CMAR　操作码

(2) 取指阶段的微操作及节拍安排

3→6

考虑到需要 形成后续微指令的地址

T_0 PC ⟶ MAR　　1 ⟶ R

T_1 Ad (CMDR) ⟶ CMAR

T_2 M (MAR) ⟶ MDR　(PC)+1 ⟶ PC

T_3 Ad (CMDR) ⟶ CMAR

T_4 MDR ⟶ IR　　OP (IR) ⟶ 微地址形成部件

T_5 OP (IR) ⟶ 微地址形成部件 ⟶ CMAR

(3) 执行阶段的微操作及节拍安排

考虑到需形成后续微指令的地址

取指微程序的入口地址 M
由微指令下地址字段指出

- 非访存指令

① CLA 指令

T_0　0 ⟶ AC

T_1　Ad (CMDR) ⟶ CMAR

② COM 指令

T_0　$\overline{\text{AC}}$ ⟶ AC

T_1　Ad (CMDR) ⟶ CMAR

③ SHR 指令

T_0　　L (AC) ⟶ R (AC)　　$AC_0 \longrightarrow AC_0$

T_1　　Ad (CMDR) ⟶ CMAR

④ CSL 指令

T_0　　R (AC) ⟶ L (AC)　　$AC_0 \longrightarrow AC_n$

T_1　　Ad (CMDR) ⟶ CMAR

⑤ STP 指令

T_0　　0 ⟶ G

T_1　　Ad (CMDR) ⟶ CMAR

• 访存指令

⑥ ADD 指令

T_0 Ad (IR) ⟶ MAR　　1 ⟶ R

T_1 Ad (CMDR) ⟶ CMAR

T_2 M (MAR) ⟶ MDR

T_3 Ad (CMDR) ⟶ CMAR

T_4 (AC) + (MDR) ⟶ AC

T_5 Ad (CMDR) ⟶ CMAR

⑦ STA 指令

T_0 Ad (IR) ⟶ MAR　　1 ⟶ W

T_1 Ad (CMDR) ⟶ CMAR

T_2 AC ⟶ MDR

T_3 Ad (CMDR) ⟶ CMAR

T_4 MDR ⟶ M (MAR)

T_5 Ad (CMDR) ⟶ CMAR

⑧ LDA 指令

T_0 Ad (IR) ⟶ MAR　　1 ⟶ R

T_1 Ad (CMDR) ⟶ CMAR

T_2 M (MAR) ⟶ MDR

T_3 Ad (CMDR) ⟶ CMAR

T_4 MDR ⟶ AC

T_5 Ad (CMDR) ⟶ CMAR

• 转移类指令

⑨ JMP 指令

T_0　　Ad (IR) ⟶ PC

T_1　　Ad (CMDR) ⟶ CMAR

⑩ BAN 指令

T_0　　$A_0 \cdot Ad(IR) + \overline{A_0} \cdot (PC) \longrightarrow PC$

T_1　　Ad (CMDR) ⟶ CMAR

全部微操作　20个　有些重复出现

微指令　38条

2. 确定微指令格式

(1) 微指令的编码方式

采用直接控制

(2) 后续微指令的地址形成方式

由机器指令的操作码通过微地址形成部件形成

由微指令的下地址字段直接给出

(3) 微指令字长

由 20 个微操作

确定 操作控制字段　　最少 20 位

由 38 条微指令

确定微指令的 下地址字段 为 6 位

微指令字长 可取 20 ＋ 6 ＝ 26 位

(4) 微指令字长的确定

38 条微指令中有 19 条

是关于后续微指令地址 ⟶ CMAR

其中 { 1 条　OP (IR) ⟶ 微地址形成部件 ⟶ CMAR
　　 { 18 条　Ad (CMDR) ⟶ CMAR

若用 Ad (CMDR) 直接送控存地址线

则 省去了输至 CMAR 的时间，省去了 CMAR

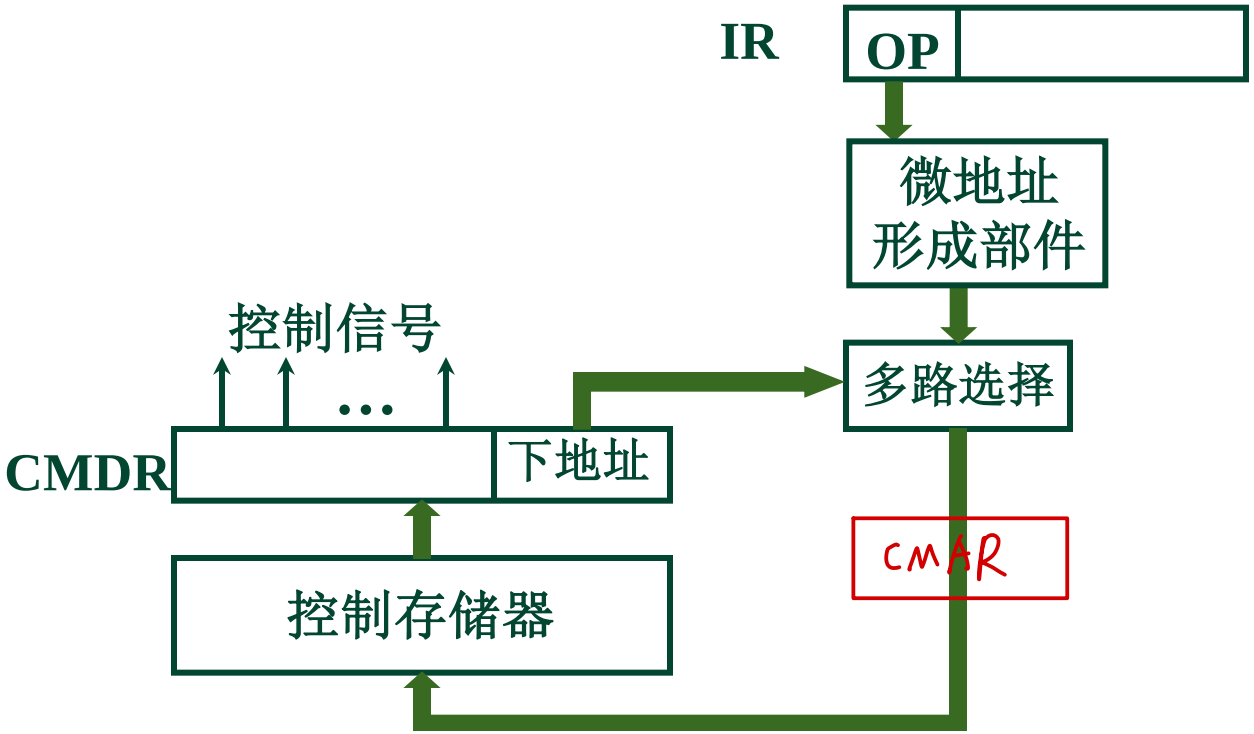
同理　OP (IR) ⟶ 微地址形成部件 ⟶ 控存地址线

可省去 19 条微指令，2 个微操作

38 － 19 ＝ 19　　20 － 2 ＝ 18

下地址字段最少取 5 位　　操作控制字段最少取 18 位

(5) 省去了 CMAR 的控制存储器



考虑留有一定的余量

取操作控制字段　18 位→24 位
下地址字段　5 位→ 6 位
} 共 30 位

(6) 定义微指令操作控制字段每一位的微操作

3. 编写微指令码点

微程序名称	微指令地址(八进制)	微指令（二进制代码）														
		操作控制字段									下地址字段					
		0	1	2	3	4	…	10	…	23	24	25	26	27	28	29
取指	00	1	1								0	0	0	0	0	1
	01			1	1						0	0	0		1	0
	02					1					×	×	×	×	×	×
CLA	03										0	0	0	0	0	0
COM	04										0	0	0	0	0	0
ADD	10		1								0	0	1	0	0	1
	11			1							0	0	1	0	1	0
	12										0	0	0	0	0	0
LDA	16		1					1			0	0	1	1	1	1
	17			1							0	1	0	0	0	0
	20										0	0	0	0	0	0

PC→MAR

PC+1→PC

MDR→IR

0→ACC

Ad(IR)→MAR

1→R

M(MAR)→MDR

1→R

Ad(IR)→MAR

M(MAR)→MDR

第十章 控制单元的设计 回顾

- **10.1 组合逻辑设计**
  - **CU框图、微操作的节拍安排（三个原则）**
  - **设计步骤：**列操作时间表、写逻辑表达式、画逻辑图
- **10.2 微程序设计**
  - **设计思想：**微程序、微指令、控存
  - **CU框图和工作原理**
  - **关键：微指令编码方式**（直接、字段直接和间接编码）和**后续地址形成**（下地址、操作码经微地址形成部件、增量计数、分支转移、测试网络、硬件产生）
  - **微指令格式：**水平型、垂直型；静态、动态微程序设计
  - **设计步骤：**微操作分析和节拍安排、确定微指令格式、编写微指令码点

Computer Organization

Thank You !

Computer Architecture Research Institute · Computer Organization

校验码：加校验位

控制单元

ENIAC

几代

摩尔定律